

40 Final
40 Vize
20 proje

Yeni Başlayanlar İçin

C# ile Nesne Tabanlı Programlama

Fahrettin ERDİNÇ

abaküs

abaküs 002

C# İLE NESNE TABANLI PROGRAMLAMA

Fahrettin ERDİNÇ

1. Baskı: Şubat 2015

7. Baskı: Temmuz 2021

ISBN: 978-605-9129-00-8

Kapak ve sayfa düzeni: Cem Demirezen

Satış: satis@abakuskitap.com

Baskı: Ezgi Matbaacılık San. Tic. Ltd. Şti.
Sanayi Cad. Altay Sok. No:14
Çobançeşme-Yenibosna/İSTANBUL
Tel: 0212 452 23 02
Matbaa Sertifika No: 45029

Bu kitabın bütün yayın hakları Abaküs Kitap Yayın Eğitim Ltd. Şti.'ne aittir. Yayınevimizin yazılı izni olmaksızın kısmen veya tamamen alıntı yapılamaz, kopya edilemez, çoğaltılamaz ve yayınlanamaz.

Abaküs Kitap Yayın Eğitim Ltd. Şti.

Yayıncılık sertifika no: 47204

Hobyar Mah. Cemal Nadir Sok. No:24/326 K:3 Cağaloğlu-Fatih/İST.

Tel.: (0212) 514 68 61

www.abakuskitap.com - editor@abakuskitap.com



Fahrettin Erđinç

1972 yılında Yunanistan'ın İskeçe ilinde doğdu. 1985 yılında Türkiye'ye göç etti. 1991 yılında Dokuz Eylül Üniversitesi Elektrik-Elektronik Mühendisliği'nde okumaya başladı ve aynı yıl programlama ile tanıştı. Okulda C, Mikroişlemci ve PIC mikro-kontrolör programlama öğrendi. Mezun olduktan sonra sistem destek ve bilgisayar ağları konusunda çalıştı. 1996 yılında kendi firması olan Bilimsel Akademi'yi kurdu ve bilgisayar eğitimleri vermeye başladı. 1996 yılından beri Yazılım Uzmanlığı, Sistem ve Ağ Uzmanlığı, Web Programlama, PIC Programlama, Arduino ve PLC Programlama konusunda dersler vermektedir. Ders dışında bilişim sektörünün gelişmesine katkıda bulunmak için makale ve kitaplar yazan Fahrettin Erđinç boş zamanlarında hobi olarak nesnelerin interneti, robotik ve motorsikletlerle ilgilenmektedir.

Bilimsel Akademi

1996 yılında kurulan Bilimsel Akademi, o günden bu güne güçlü bilgi birikimi ve tecrübesiyle “Bilişim” ve “Kariyer Eğitimleri” konusunda hizmetlerine kendi binasında devam etmektedir.

Kendi alanında uzman eğitmenlere sahip olan Bilimsel Akademi İzmir’de ve Türkiye’de birçok ilke de imza atmıştır; İzmir’deki ilk “PC Donanımı & Tamiri”, “3D AutoCAD”, “3D Studio MAX”, “Dreamweaver, Fireworks & Flash ile Web Tasarımı”, “Corel DRAW”, “Visual Studio.NET”, “Linux”, “Ethical Hacking”, “PLC Programlama” “PIC Programlama” “Arduino Programlama” ve “CNC Programlama” eğitimleri bunlardan sadece birkaçıdır.

2002 – 2009 döneminde Dokuz Eylül Üniversitesi Sürekli Eğitim Merkezi (DESEM) ile ortak eğitimler düzenleyen Bilimsel Akademi, 2010-2011 öğretim yılında gücünü Ege Üniversitesi Sürekli Eğitim Merkezi (EGESEM) ile birleştirmiş, Ege Üniversitesi’nin akademik desteğini de yanına alarak eğitim kalitesi ve müşteri memnuniyetini daha da artırmıştır.

Bilimsel Akademi örgün eğitimleri ile İzmir’lilere uzaktan eğitimle de tüm Türkiye’ye hizmet vermeye devam etmektedir.

Teşekkürler...

Bu kitabın hazırlanması sırasında desteklerini esirgemeyen eşim Zeynep Erdiñ'e, sayfa sayfa tüm kitabı gözden geçirerek bana önerilerde bulunan Abaküs Kitap çalışanlarına, bu kitabı yazmam için bana destek ve cesaret veren Bilimsel Akademi personeli ve öğrencilerine, emeği geçen ve adını saymayı unuttuğum herkese çok teşekkür ederim.

Bilimsel Akademi öğrencilerine...

Bu Kitap Kimler İin?

Bu kitap hi C# bilmeyip renmek isteyen veya var olan C# bilgisini geliřtirmek isteyenler iin yazılmıřtır. Kitap zellikle okullarda, kurslarda, meslek liselerinde, yksekokullarda ve niversitelerde C# ile programcılıęa yeni bařlayanlar dřnlerek hazırlanmıřtır. Buna paralel, kitap sadece programcılıęa yeni baslayan đrencileri deęil, aynı zamanda programcılık eęitimi veren đretmenleri de gz nnde bulundurarak hazırlanmıřtır. Bu nedenle, đretmenlerimiz derslerde kitabı rahatlıkla kullanabilirler.

zellikle programcılıęa yeni bařlayanların zorlandığı bazı konularda bolca rnek kod yazılarak konuların daha basit ve anlařılır olması saęlanmıřtır. Blm sonlarındaki uygulamalar ile konuların iyice pekiřtirilmesi amalanmıřtır. En son blmdeki Sıka Kullanılan Algoritmalar konusu ile de okul hayatımızda ya da sınavlarda sıka karřımıza ıkan sorulara cevaplar sunulmuřtur.

Konular tamamen sıfırdan bařlar ve temel/orta dzeyde programcılık becerilerini kazandırmayı amalar. Kitaptaki konular mmkn olduęunca kk ve mantıklı paralara ayrılmıř ve herkesin kolayca anlayabileceęi rneklerle desteklenmiřtir.

Konuların daha iyi anlařılabilmesi iin nce teorik bilgi verilmiř, gerekliyse algoritması hazırlanmıř, gereken yerlerde grseller kullanılmıř ve son olarak da rnek kodlar yazılmıřtır. Bir resim bin kelimeye bedeldir dřnlmesiyle mmkn olduęunca grsellerle aıklamalar yapılmıřtır.

Fazla teori ve laf kalabalığı ile konuların daha karmařık hale getirilmesinden kaınılmıřtır. Az laf ok iř (kod) prensibi ile her konu uygulamalı ve herkesin anlayacağı řekilde hazırlanmıřtır.

Fahrettin Erdiņ
Bilimsel Akademi

İÇİNDEKİLER

1. Temel Kavramlar	1
Bilgisayar Nedir?	1
Bilgisayarın Yapısı	2
Donanım Nedir?	2
Yazılım Nedir?	3
Algoritma Nedir?	3
Programlama Dilleri	4
Komut Nedir?	4
Program Nedir?	4
Programlama Dili Nedir?	5
Yaygın Programlama Dilleri	5
Assembly Nedir?	7
Makine Dili Nedir?	8
Kaynak Kod Nedir?	8
Editör Nedir?	9
IDE Nedir?	10
Derleyici (Assembler) Nedir?	10
Yorumlayıcı (Interpreter) Nedir?	12
Amaç (Executable) Program Nedir?	12
Alt ve Üst Seviye Diller	13
Problem Çözme Aşamaları	14
Problemin Tanımlanması	14
Algoritmanın Oluşturulması	14
Deneme ve Düzeltme	14
Yazılım Geliştirme Süreci	14
Analiz	14
Tasarım	15
Kodlama	15
Test	15
Kurulum	15
Destek ve Bakım	16
Güncelleme	16
Algoritmanın Oluşturulması	16
Pseudo Kod	16
Akış Diyagramları	17
Kod Yazma & Editörler	18
Neden Visual Studio?	18
Visual Studio Express veya Community Edition	20
Nereden Bulurum?	20

Visual Studio İçinde Neler Var?	21
Visual Studio (VS) Arayüzü ve İlk Proje	21
İlk Programımız	24
Komut Satırı Derleyicisi (csc.exe)	25
Derleyici Parametreleri	29
2. Bilgisayar Mimarisi	31
Bilgisayar Nedir?	31
Bilgisayar Nasıl Çalışır?	32
İşlemci Nasıl Çalışır?	34
Fetch (Komut Al)	35
Decode (Komutu Çöz)	36
Execute (Çalıştır)	36
Store (Kaydet)	36
CPU ve RAM İlişkisi	37
Onlu ve İkili (Dijital) Sayı Sistemi	38
Onlu Sayı Sistemi	38
İkili Sayı Sistemi	39
RAM Bellek Yapısı	40
3. Temel Algoritma Bilgisi	43
Algoritma Nedir?	43
Algoritma Nasıl Olmalıdır?	44
Algoritmanın Faydaları	44
Bazı Terim ve Kavramlar	44
Değişken	45
Atama	46
Sorgu	47
Dallanma	47
Sayaç	48
Döngü	49
Yer Değiştirme Algoritması	50
Metot (Fonksiyon)	51
Açıklamalı Algoritma Örnekleri	52
Genel Yöntem Algoritması	52
Günlük Program Algoritması	54
Hava Durumu Algoritması	55
Saat Alarmı Algoritması	56
Girilen Sayının Karesini Bulan Algoritma	57
Girilen İki Sayının Toplamını Bulan Algoritma	58
Üç Sayının Ortalamasını Bulan Algoritma	59
Dikdörtgenin Alanını ve Çevresini Bulan Algoritma	60

Dairenin Alanını ve Çevresini Bulan Algoritma	61
Girilen Yaşa Göre İşlem Yapan Algoritma	62
Girilen İki Sayıdan Büyük Olanı Bulan Algoritma	63
Sadece Pozitif Sayıların Karesini Bulan Algoritma	64
Geçti-Kaldı Algoritması	65
1'den 5'e Kadar Olan Sayıların Toplamı	66
1'den N'e Kadar Olan Sayıların Toplamı	67
1'den N'e Kadar Olan Çift Sayıların Toplamı	69
1'den N'e kadar Olan Sayıların Çarpımı (Faktoriyel)	70
4. .NET Framework & C#	73
Microsoft .NET Framework Nedir?	73
.NET Framework Sürümleri	75
Çekirdek Sürümü	75
SDK Sürümü	75
.NET Çalışma Mantiği	76
Nesne Tabanlı Programlama (Object Oriented Programming)	79
Class (Sınıf) Nedir?	79
.Net Framework Assembly Nedir?	80
Object (Nesne) Nedir?	81
Namespace (Aduzayı) Nedir?	83
Özellik (Property) & Nesne Özellikleri	86
Metot (Method)	88
Olay (Event)	89
C# Nedir?	89
Visual Studio	90
Visual Studio Alternatifleri	90
Visual Studio 2015 Sürümleri	92
Visual Studio Community 2015 Gereksinimleri	93
Visual Studio Kurulumu	94
Visual Studio İçinde Neler Var?	97
Visual Studio ile Proje Oluşturma	98
Projelerin Açılması	101
Projelerin Kaydedilmesi ve Kapatılması	103
Console Application	103
MSDN	105
Visual Studio Arayüzü	106
5. Değişkenler & Veri Giriş-Çıkışı	115
Hatırlatma	115
Açıklama Satırları	117
C# Yazım ve Noktalama Kuralları	119

Noktalı Virgöl (;)	119
Güzel (Süslü) Parantezler {...} ve Blok Kavramı	120
Boşluk Karakteri	122
Büyük-Küçük Harf Kullanımı	123
Veri Giriş-Çıkış Komutları	124
Veri Çıkışı	124
Özel Karakterler	126
Sözel Toplama	128
Veri Girişi	129
Değişkenler	131
Değişkenlerin Fiziksel Yapısı	132
Veri Tipleri	133
Değişken Oluşturma ve Değer Atama	139
Toplama & Atama	146
Çarpma ve Atama	147
Sabitler	150
Değişken Ömrü ve Kapsamı	151
Veri Tipi Dönüştürme	153
Convert Sınıfı	156
İstemsiz ve İstemli Veri Tipi Dönüşümü	157
Boxing & Unboxing	159
1. Uygulamalar	161
İlk Program	161
Ad Yazdırma	164
Girilen Sayının Karesi	165
Girilen İki Sayının Toplamı	166
Üç Sayının Ortalaması	167
Dikdörtgenin Alanı ve Çevresi	168
Dairenin Alanı ve Çevresi	169
Inch-Cm Dönüşümü	170
Fahrenheit-Santigrat Dönüşümü	171
Tek Bilinmeyenli Denklem Çözümü	172
6. Operatörler	173
Operatör Nedir?	173
Aritmetik Operatörler	173
Arttırma ve Azaltma Operatörleri	175
Eşitlik ve İlişki Operatörleri	177
Mantıksal Operatörler	177
Atama Operatörleri	179
Operatörlerde Öncelik Sırası	180

7. Karar Yapıları	181
Karar Yapısı Nedir?	181
If Komutu	182
If-Else Komutu	185
If-Else If-Else Komutu	187
Switch Komutu	190
?: Operatörü	191
Dallanma Komutu (goto) & Etiket Kullanımı	192
2. Karar/Sorgu Uygulamaları	193
Yaş Kontrolü	193
Sadece Pozitif Sayıların Karesini Alan Program	194
Dairenin Alanı ve Çevresi	194
Dikdörtgenin Alanı ve Çevresi	195
Girilen İki Sayıdan Büyük Olanı Bulan Program	196
Girilen İki Sayının Durumu	197
Geçti-Kaldı Programı	198
Sayaç Kullanımı	199
1'den N'e Kadar Olan Sayıların Yazdırılması	200
1'den N'e Kadar Olan Çift Sayıların Yazdırılması	201
Ekrandan Belli Bir Adet Sayı Alınması	202
1'den N'e Kadar Olan Sayıların Toplamı	203
Faktöriyel Hesabı	203
Haftanın Gününü Yazmak	204
Alistirmalar	205
8. Döngüler	207
Döngü Nedir?	207
Döngü Tipleri	209
For Döngüsü	209
Foreach Döngüsü	212
While Döngüsü	214
Do-While Döngüsü	215
Break ve Continue Komutları	217
Break Komutu	217
Continue Komutu	218
Sonsuz & Ölü Döngü	218
2. Döngü Uygulamaları	219
Beş Defa Merhaba Yaz	220
1'den N'e Kadar Olan Sayıların Yazdırılması	221
1'den N'e Kadar Sayıların Toplamı	222
N Adet Sayının Ortalaması	223
Faktöriyel Hesabı	224

Girilen İki Değer Arasındaki Sayıları Ekrana Yaz	225
Girilen İki Değer Arasındaki Sayıların Toplamı	226
Tek Bilinmeyenli Denklem	226
Tek Bilinmeyenli Denklem Sürüm 2	227
Alıştırmalar	228

9. Metotlar 229

Metot Nedir?	229
Metot Nasıl Oluşturulur?	231
Erişim Belirteci	232
Dönüş Tipi	234
Değer Döndürmeyen Metotlar	234
Tek Parametrelili Metotlar	235
Çok Parametrelili Metotlar	235
Main Metodu	236
Metot Nasıl Çağırılır?	237
Lambda (İfadeleri) Fonksiyonları	237
Metot Uygulamaları	239
Merhaba Metodu	240
N Defa Merhaba Yaz	241
Parametrelili Merhaba Metodu	241
Kare Metodu	242
Kare Metodu Sürüm 2	243
Konsolu Oku Metodu	244
Topla Metodu	245
1'den N'e kadar Olan Çift Sayıların Yazdırılması	246
X ile Y Arasındaki Sayıların Yazdırılması	246
1'den N'e Kadar Sayıların Toplamı	247
N Faktöriyel	248
1'den N'e Kadar Olan Sayıların Faktöriyeli	249
Alıştırmalar	250

10. Sayısal ve Sözel Metotlar 251

.NET Framework Metotları	251
Math Sınıfı ve Matematiksel Metotlar	254
Math Sınıfı Uygulamaları	254
Konsoldan Okunan Sayının Karekökü	254
X ile Y Arası Sayıların Karekökleri	256
Bir Sayının N. Kuvveti	257
1'den N'e Kadar Olan Sayıların Kareleri Toplamı	258
Dairenin Alanı ve Çevresi	258
Çeşitli Metotlar	259

11. Diziler**263**

Dizi Nedir?

263

Dizi Tanımlama

264

İki Boyutlu Diziler

268

Array Sınıfı, Dizi Özellikleri ve Metotları

271

Metinler ve Diziler

272

Dizi Uygulamaları

272

Dizi Elemanlarının Konsola Yazdırılması

273

Konsoldan Okunan Adların Dizide Saklanması

273

N Adet Sayının Ortalaması

275

Kelime Uzunluğu

276

Sayıların Sıralanması

277

Dizide Arama Yapmak

278

Dizide Arama Yapmak (Metotsuz Yöntem)

279

Bir String'in Karakterleri ile Çalışmak

280

İki Boyutlu Dizinin elemanları

281

Alistirmalar

282

12. Sıkça Kullanılan Algoritmalar**283**

Tek/Çift Sayı

283

Tam Bölünebilme

284

Çarpım Tablosu

286

En Büyük Sayı

286

Asal Sayı

288

Rastgele Sayı & Sayısal Loto

290

Üçgen Alanı

291

Üçgen Kenar Uzunlukları

293

Daire İşlemleri

294

Kombinasyon

295

İkinci Dereceden Denklem Kökleri

296

Sayı Tahmin Programı

297

Fibonacci Sayıları

299

Sayının Basamakları

299

Çarpmanın Toplama ile Yapılması

300

Bölmenin Çıkarma İle Yapılması

301

Onluk Sayının İkili Sayıya Çevrilmesi

303

Rastgele Sayı Dizisi

304

Dizi Elemanlarının Toplamı ve Ortalaması

304

Dizilerde Matematiksel İşlemler

306

Dizinin En Büyük veya En Küçük Elemanı

306

Girilen Kelimenin Tersten Yazılması	309
Dizideki Pozitif ve Negatif Elemanlar	310
Dizide Doğrusal Arama Yapmak	311
Sayı Sıralama	313
İki Boyutlu Dizilerde Matematiksel İşlemler	315
13. Nesne Tabanlı Programlama	317
Bellek Alanları	317
Stack Bellek Alanı	318
Heap Bellek Alanı	319
Static Bellek Alanı	320
Nesne Tabanlı Programlama Nedir?	320
Bilgi Gizleme (Encapsulation) & Soyutlama	323
Çok Biçimlilik (Polymorphism)	323
Kalıtım (Inheritance)	324
Sınıflar ve Nesneler	324
Sınıf ve Nesne Oluşturma	327
Sınıfın Alanları	328
Sınıfın Metotları	328
Alan ve Metotların (Üyelerin) Kullanımı	333
Visual Studio Sınıf İşlemleri	334
Nesne Kopyalama	337
Erişim Belirteçleri	338
Yapıcı Metotlar (Constructors)	339
Parametrelili Yapıcı Metotlar	343
Varsayılan Yapıcı Metotlar (Default Constructor)	346
Yıkıcı Metot (Destructor)	348
Soyutlama (Kapsülleme / Encapsulation)	350
Özellikler (Properties), set ve get Erişimcileri	353
Static Öğeler	357
Static Alanlar	358
Static Metotlar	361
Sabit Değişkenler (const)	365
Statik Yapıcı Metotlar	365
Static Sınıflar	367
This Göstericisi	370
Sınıf Uygulamaları	371
Dikdörtgen Sınıfı ve Nesneleri	371
Statik Dikdörtgen Sınıfı	372
Yapıcı Metot	373
Statik Yapıcı Metot Uygulaması	374
Statik Matematik Metotları	375

14. Kalıtım & Polimorfizm	377
Kalıtım (Inheritance) Nedir?	377
Sınıf Türetme	379
Türetilmiş Sınıflarda Erişim Denetimi	383
Sealed (Kalıtımsız) Sınıflar	386
Yapıcı/Yıkıcı Metotlar & Kalıtım	387
Çok Biçimlilik (Polymorphism) ve Sanal Metotlar	388
Özet (Abstract) Sınıflar ve Metotlar	391
Arayüzler (Interfaces)	393
IEnumerable Arayüzü	396
Foreach Döngüsü (Hatırlatma)	396
Yield Operatörü	398
Uygulamalar	399
Satranç Sınıfı	399
Dövüşçü Karakterler	400
Polimorfizm ile Alan Hesabı	401
Özet Sınıf ile Şekillerin Alan Hesabı	402
Arayüz ile Şekillerin Alan Hesabı	403
15. Yapılar & Numaralandırma	405
Yapı (Struct) Nedir?	405
Yapılarda Yapıcı Metot Kullanımı	408
Numaralandırma (Enum)	409
Uygulamalar	414
Öğrenci Bilgilerini Tutan Yapı	414
Yapılarda Nesne Kopyalama	415
Yapılarda Parametrelili Yapıcı Metot Kullanımı	416
Şehir Kodlarının Enum ile Belirlenmesi	417
16. İstisna Yönetimi (Exception Handling)	419
İstisna (Exception Handling) Yönetimi Nedir?	419
İstisna Yönetimi	422
Çoklu Hata Yakalama Blokları	427
throw (fırlat) Komutu	430
17. Dosya İşlemleri	435
Dosya İşlemleri	435
Klasör İşlemleri	436
Dosya İşlemleri	439
FileStream Sınıfı	442
Dosyaya Yazma İşlemi & StreamWriter	443
Dosyadan Okuma İşlemi & StreamReader	445
	xvii

Uygulamalar	447
Klasör Oluşturma & Try-Catch Kullanımı	447
Dosyaya Yazma & Try-Catch Kullanımı	448
Dosyaya Veri Yazma & Try-Catch Kullanımı	449
Dosya Okuma & Try-Catch Kullanımı	450

18. Şablonlar (Generics) ve Koleksiyonlar	453
Şablon (Generic) Nedir?	453
Şablon Metotlar	454
Şablon Sınıflar	458
Şablon Tipli Koleksiyonlar	459
Stack (Yığıt)	460
Queue (Kuyruk)	461
List (Liste)	463
Dictionary (Sözlük)	465

Önsöz

Programcılığa 1991 yılında C dili ile başladım. O gün bu gündür programcılıkla uğraşmakta ve 1996 yılından beri de Bilimsel Akademi’de programcılık eğitimleri vermekteyim. Çooook uzun yılların verdiği tecrübeye dayanarak, özellikle programcılığa yeni başlayanlara güzel bir kitap yazabileceğimi düşündüm. Yılların da verdiği tecrübeyle, daha kolay bir şekilde programcılık öğretebileceğimi düşünerek bu kitabı yazmaya karar verdim.

İnsanların genelde programcılığa başlarken yaşadığı sıkıntıları ve hangi konularda zorlandıklarını biliyorum. Bu bilginin ışığında, fazla teoriye kaçmadan, konuya odaklanarak, kolay anlaşılacak örneklerle temel programlama mantığını anlatmaya çalıştım. Uzun lafın kısıası, yıllardır kullandığım ve başarılı olduğuna inandığım bazı yöntemlerle herkese programcılık öğretebileceğimi düşünüyorum. Eğer okuyucu/öğrenci de biraz gayret ederse ortaya güzel şeylerin çıkacağını biliyorum.

Çok karmaşık matematiksel bazı konulardan bilerek kaçındım. Dolayısıyla dört işlem bilen herkes kitaptaki uygulamaları yapabilir. Eğer temel mantığı kavırsanız daha karmaşık matematiksel işlemleri zaten yapabilirsiniz.

Kitap içinde kullanılan İngilizce terimlerin Türkçe karşılıklarını yazarak, daha sonra hem Türkçe hem de İngilizce olarak kullandım. Böylece bu terimlerle karşılaştığınızda daha iyi anlamınızı sağlamaya çalıştım.

Kitaptaki grafik ve diyagramların çoğunu hatta neredeyse tamamını kendim yaptım. Böylece arada başka bir elemana gerek kalmadan doğrudan düşüncelerimi size aktarmaya çalıştım. Grafikleri mümkün olduğunca sade ve anlaşılır tutmaya çalıştım. Kullandığım verilerle ilgili internet linklerini de yazdım. Böylece daha güncel bilgiye linkleri kullanarak ulaşabileceksiniz.

Unutmayın! Bu kitap size programcılığın temel kavramlarını ve temel mantığını öğretmek için yazıldı. Temeli iyi atarsanız iyi bir programcı olmaya hazırsınız demektir. Eğer kitabı güzelce okur ve tüm uygulamaları dikkatle yaparsanız temel algoritmaları öğrenmiş olursunuz. Temel yapıları öğrendikten sonra ileri düzey konuları çok daha kolay öğrenebilirsiniz.

Kitapla ilgili görüş ve önerileriniz veya gözünüze çarpan hatalar varsa lütfen e-mail adresime gönderin.

Başarılar...
Fahrettin Erdiç
Bilimsel Akademi
fahrettin@bilimsel.com.tr

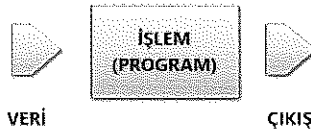
1

Temel Kavramlar

- Bilgisayarın Donanım ve Yazılımı
- Algoritma
- Programlama Dilleri
- Problem Çözme Aşamaları
- Yazılım Geliştirme Süreci
- Editörler
- Visual Studio
- İlk Program
- C# Komut Satırı Derleyicisi

Bilgisayar Nedir?

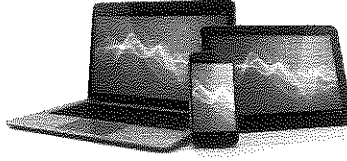
En basit tanımıyla, girilen verileri alan ve yazılan programa göre çıktı elde eden makinelere **bilgisayar** denir.



Şekil 1.1 Temel Bilgisayar Grafiği

Yukarıdaki şekilden de görülebileceği gibi, bir bilgisayarın iş yapabilmesi için bir programa ihtiyacı vardır. Programcı olarak bizim amacımız, bilgisayara istenilen işi yapmasını sağlayan programı yazmaktır.

Günümüz bilgisayarları çok değişik şekillerde ve boyutlarda karşımıza çıkabilir; akıllı telefon, masaüstü, dizüstü, tablet, panel veya PLC. Şekli veya görüntüsü nasıl olursa olsun tüm bu cihazların iş yapabilmesi için bir programa ihtiyacı vardır. Program-sız bilgisayar benzinsiz arabaya benzer; iş yapamaz.



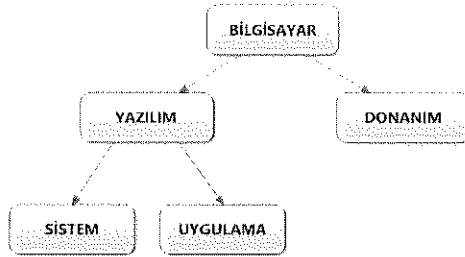
Şekil 1.2 Bilgisayarların iş yapabilmesi için programa ihtiyacı vardır

Bilgisayarın Yapısı

İster büyük olsun ister küçük, ister tablet olsun ister masaüstü, bir bilgisayar iki temel öğeden oluşur:

- Donanım
- Yazılım

Aşağıdaki konularda bu iki öğeyle ilgili biraz daha ayrıntılı bilgi vermeye çalışacağım.



Şekil 1.3 Yazılım ve donanım

Donanım Nedir?

Bilgisayarı oluşturan elektronik ve mekanik ağıtlara donanım denir. Donanım ele-manları genelde dört gruba ayrılır. Bunlar:

- **Giriş Elemanları:** Fare, Klavye, Tarayıcı, Mikrofon, CD/DVD vb.
- **İşleme Elemanları:** İşlemci, RAM, Chipset, Veriyolları vb.
- **Çıkış Elemanları:** Ekran, Yazıcı, CD/DVD, Hoparlör vb.
- **Yardımcı Elemanlar:** Güç Kaynağı, Kasa, Kesintisiz Güç Kaynağı vb.

Yazılım Nedir?

Bilgisayarın çalışması ve belli bir işi yapabilmesi için kullanılan programlara **yazılım** denir. Genelde bir bilgisayarda kullanılan yazılımlar aşağıdaki gibi sınıflandırılır.

- **İşletim Sistemi:** Adından da belli olduğu gibi, sistemin işlemesi, yani bilgisayarın çalışması ve temel görevleri icra edebilmesi için kullanılan yazılımdır. İşletim sistemleri programların çalıştırılması, kapatılması, dosya yönetimi ve bellek yönetimi gibi en temel görevleri yürütürler. Windows, Linux ve UNIX gibi yazılımlar işletim sistemlerine örnek olarak verilebilir.
- **Uygulama Programı:** Belli bir işi yapmak üzere üretilen yazılımlardır. Örneğin Word programı yazı işleri için kullanılırken Autocad programı çizim işleri için kullanılır.
- **Programlama Dilleri:** Program yazmak için üretilen yazılımlardır. BASIC, Pascal, Java, C, C++, C# bu tür yazılımlara örnek verilebilir.
- **Oyunlar**

Bizim bir programcı olarak amacımız belli bir işletim sistemi üzerinde çalışabilecek uygulama programlar veya oyunlar geliştirmektir.

Algoritma Nedir?

Algoritma sözcüğü, Özbekistan'ın **Harezmi**, bugünkü adıyla **Hiva** kentinde doğmuş olan İbn Musa El Harezmi adındaki **İran**'lı âlimden kaynaklanır. Bu âlim 9. Yüzyılda cebir alanındaki algoritmik çalışmalarını kitaba dökerek matematiğe çok büyük bir katkı sağlamıştır. Avrupalılar adını doğru düzgün telaffuz edemedikleri için bize algoritma olarak gelmiştir.

Algoritma, bir sorunu (problemi) çözmek için kullanılan yöntemdir. Eğer önünüzde bir problem varsa, bu problemi çözmek için değişik yöntemler, yani algoritmalar kullanabilirsiniz.

Eğer 5 adet sayıyı büyükten küçüğe doğru sıralayan bir program yazmak istiyorsanız, matematik bilginize göre 10-15 farklı yöntem kullanabilirsiniz. Tam sayıyı söylemek gerekirse 17 farklı sıralama algoritması vardır. (Bu algoritmaların hepsini incelemek için http://tr.wikipedia.org/wiki/Sıralama_algoritması veya https://en.wikipedia.org/wiki/Sorting_algorithm linkine tıklayabilirsiniz. Buradaki sayfalarda sıralama algoritmalarının hepsi animasyonla anlatılıyor ☺)

Algoritmadaki asıl mesele sizin konu hakkındaki bilginizdir. Eğer konuya hâkimseniz sorunu kolay ve hızlı bir yöntemle çözebilirsiniz. Eğer konuya hâkim değilseniz, ya da uğraştığınız sorun hakkında uzman değilseniz berbat bir algoritma ortaya çıkara-

bilirsiniz. Örneğin, doğru düzgün matematik bilginiz yoksa yukarıda bahsedilen sıralama algoritmanız ya kötü olacak ya da hiç çalışmayacaktır.

Uzun lafın kisası: Eğer bir konu hakkında uzman değilseniz doğru düzgün algoritma da yapamazsınız. O nedenle önce sorun hakkında ayrıntılı bilgi edinmeniz gerekmektedir.

Programlama Dilleri

Program yazarken kullanılan bir sürü teknik terim vardır. Bu kitapta ve diğer programlama kitaplarında bu teknik terimlerin çoğu ile karşılaşacaksınız. Bazı terimlerin anlamını bilebilirsiniz ama bazı terimleri ya bilmezsiniz ya da yanlış bilirsiniz. Yanlış veya eksik terimler konunun yeterli ya da tam anlaşılmasına engel olabilir. Bu nedenle bu kitapta programlama konusu ile ilgili terimlere fazla ayrıntıya girmeden ve kafa karıştırmadan yer vermeye çalışacağız.

Komut Nedir?

Bilgisayara belli bir iş yaptırmaya yarayan emir sözcüğüdür. Bu emir sözcüğü **break** gibi basit bir komut da olabilir **for()** döngüsü gibi daha karmaşık bir yapı da olabilir. Sonuçta hepsi bilgisayara belli bir işi yaptırmaya yarayan emir sözcükleridir.

Program Nedir?

Belli bir kurala göre bir araya gelen komutlar topluluğudur; oluşturduğunuz algoritma göre komutlar belirli bir mantıkla ve problemi çözebilecek şekilde bir araya getirilir ve program oluşur. Aşağıda görüldüğü gibi, belli bir kurala ve mantığa göre bir araya getirilen komutlar bir program oluşturur.

Komut1

Komut2

Komut3

...

Aşağıda C# dili ile yazılmış basit bir program görülmektedir. Bu program dışarıdan bir isim alıp **"merhaba isim"** şeklinde bir çıktı vermektedir.

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace adYaz
8 {
9     class Program
10     {
11         static void Main(string[] args)
12         {
13             //değişkenleri tanımlayalım
14             string ad;
15
16             //konsola "Adınızı yazınız" yazdıralım
17             Console.Write("Adınızı yazınız:");
18
19             //konsola yazılan veriyi okuyalım ve ad değişkenine atayalım
20             ad=Console.ReadLine();
21
22             //"merhaba ad" yazdıralım
23             Console.Write("merhaba " + ad);
24
25             //pencerenin kapanmasını engelleyelim
26             Console.ReadLine();
27         }
28     }
29 }

```

Şekil 1.4 Belli bir kurala göre yazılan komutlar bir program oluşturur

Programlama Dili Nedir?

Belirli kurallara göre program yazmamızı sağlayan özel ifadeler ve semboller topluluğudur. Her dilin kendine göre kuralları vardır; bu kurallara uyarak program yazılır ve belirli işlerin yapılması sağlanır.

Her programlama dilinin kendine göre avantajları ve dezavantajları bulunmaktadır. Bazı programlama dilleri kolay öğrenilebilmek için, bazıları matematiksel işlemler yapmak için, bazıları ise genel amaçlı olarak her iş için üretilmiştir.

Her dilin desteklediği veya desteklemediği teknolojiler vardır; örneğin, bazıları ile sadece metin tabanlı program yazabilirken, bazıları ile görsel tabanlı programlar da yazabilirsiniz. Bu nedenle bir programlama dili öğrenirken yetenekleri ve desteklediği teknolojiler iyice araştırılmalı ve ona göre seçim yapılmalıdır.

Yaygın Programlama Dilleri

Genel olarak günümüzde C tabanlı programlama dilleri tercih edilmektedir. C programlama dili 1970'li yılların başında AT&T Bell Laboratuvarları'ndan Dennis Ritchie tarafından UNIX işletim sistemleriyle birlikte kullanılmak üzere tasarlanmış genel amaçlı bir programlama dilidir.

C dilinin atası 1970 yılında Ken Thomson tarafından geliştirilen B dilidir. B dilinin de atası 1967'de Martin Richards tarafından geliştirilen BCPL dilidir.

C# dili bir anda ortaya çıkmamıştır; daha önceki programlama dillerinin iyi yanları alınmış, kötü yanları kaldırılmış veya düzeltilmiştir. C diline yapılan eklemelerle C++ dili, gene C++ diline yapılan iyileştirme ve eklemelerle Java ve C# dilleri ortaya çıkmıştır.

Piyasada yaygın olarak kullanılan aşağıdaki programlama dilleri ile karşılaşabilirsiniz:

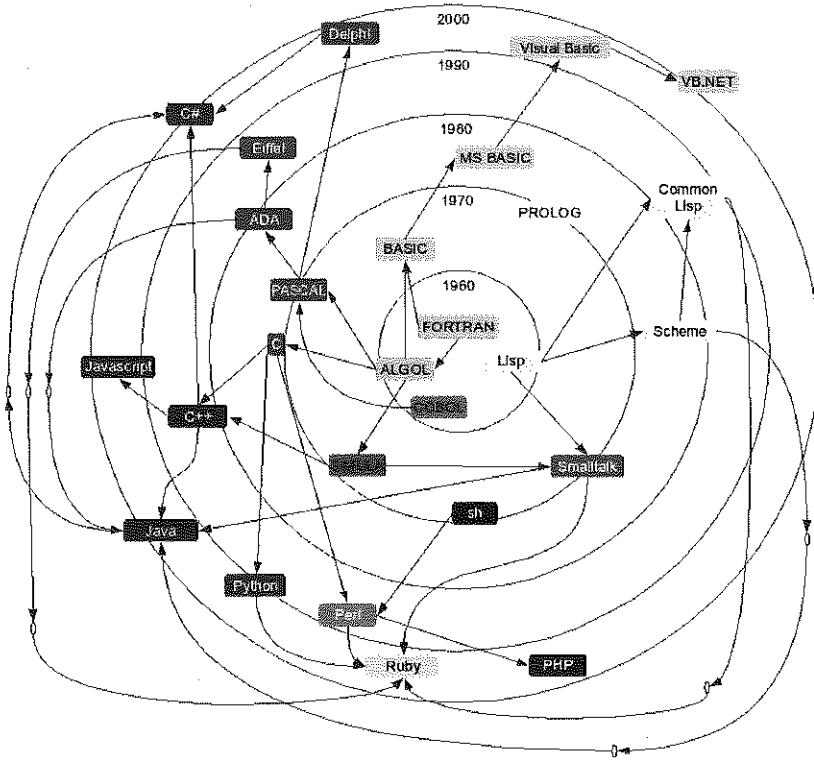
C, C++, C#, Java, Javascript, PHP, Visual Basic, Python

Windows ve Linux gibi işletim sistemleri C dili veya türevleri kullanılarak yazılmışlardır. Gene piyasada yaygın olarak kullanılan elektronik aygıt ve bilgisayar kartlarının sürücülerini çoğunlukla C veya türevleri ile geliştirilmişlerdir.

Peki! Biz hangi dili öğreneceğiz? 1991 yılında bir hocama ben de aynı soruyu sormuştum. Hocamın cevabı hâlâ geçerli: "Hangi dili öğrenirsen öğren ama çok iyi düzeyde öğren, hepsi işini görür." Eğer bir dili çok iyi öğrenirseniz diğer dilleri de kolayca öğrenebilirsiniz. Modern, güçlü, çok amaçlı, nesne tabanlı ve Microsoft tarafından sürekli desteklendiği ve geliştirildiği için biz C# dilini öğreneceğiz. C# dili ile sadece masaüstü yazılımları değil, web tabanlı veya mobil uygulamalar da üretebilirsiniz.

Tüm Türkiye'de ve dünyada C# şu anda en çok aranan ve en çok kullanılan programlama dillerinden biridir. Sadece okullarda değil aynı zamanda piyasada da çok rağbet görmektedir. Konumuz dışında olduğu için daha fazla ayrıntıya girilmeyecektir fakat C# hakkında daha ayrıntılı bilgi için [http://en.wikipedia.org/wiki/C_Sharp_\(programming_language\)](http://en.wikipedia.org/wiki/C_Sharp_(programming_language)) adresini ziyaret edebilirsiniz.

Aşağıdaki şekilde yaygın olarak kullanılan bilgisayar dillerini ve hangi dilden türediklerini görebilirsiniz. İyice incerseniz, C dilinin atasının ALGOL, C, C++, C#, Java, Javascript ve Python dillerinin atasının da C olduğunu görebilirsiniz.



Şekil 1.5 Programlama dillerinin tarihi

Assembly Nedir?

Mikroişlemcileri ve mikro denetleyicileri programlamak için kullanılan özel programlama dillerine **assembly** denir. Her işlemcinin kendi assembly dili vardır ve sadece kendi dilinde yazılan programları çalıştırabilirler. Bu nedenle, örneğin Intel x86 için yazdığınız kodlar Motorola işlemcide çalışmaz.

Assembly dilleri daha çok endüstriyel uygulamalarda ve otomasyonda, özellikle PIC, ATMEL ve 8051 gibi cihazları programlamak için kullanılır. Ayrıca, işletim sisteminin çekirdeğinin yazılmasında da genellikle assembly tercih edilir çünkü bu dilde yazılan programlar donanım seviyesinde bazı komutlara sahiptir ve daha iyi performans elde edilmesini sağlar. Örneğin, C# ile yazdığınız programda bir verinin hangi adrese kaydedileceğini belirleyemezsiniz ama assembly ile yazdığınız programda bir verinin ya da değişkenin RAM bellekte hangi adrese kaydedileceğini belirleyebilirsiniz.

Aşağıda Intel 8086 Assembly ile yazılan küçük bir program-görülmektedir. Dikkat ederseniz, komutlar birkaç harften oluşmaktadır ve anlaşılması son derece zordur.

Karekök alma gibi basit matematiksel işlemleri bile yapabilen komutlar yoktur. Bu nedenle ve basit programları yazmak bile oldukça karmaşık ve zaman alıcı olabilir.

```

model small
stack 100h
data
msg      db      'Hello world!$'
code
start:
        mov     ah, 09h    ; Display the message
        lea     dx, msg
        int     21h
        mov     ax, 4C00h  ; Terminate the executable
        int     21h
end start

```

Assembly'de program yazmak zor ve zaman alıcıdır fakat donanım seviyesinde işlemler yapılmasına izin verir ve fazla donanım ihtiyacı yoktur. Bu nedenle özellikle robotik uygulamalarda, bazı elektronik cihazlarda, düşük güç gerektiren elektronik elemanlar tarafından tercih edilir.

Makine Dili Nedir?

Sadece 0 ve 1'lerden oluşan komutlarla yazılan programlama dillerine makine dili denir. Aynı assembly dillerde olduğu gibi, her işlemcinin kendine özel makine dili vardır ve sadece o işlemcide çalışır. Makine kodları ile yazılmış bir program aşağıdaki gibi görünebilir:

```

01001001
11001010
01011010
10100101

```

Makine dili ile program yazmak çok zor ve çok zaman aldığı için pek tercih edilmez. Onun yerine daha üst seviye diller olan assembly veya C gibi diller kullanılır.

Kaynak Kod Nedir?

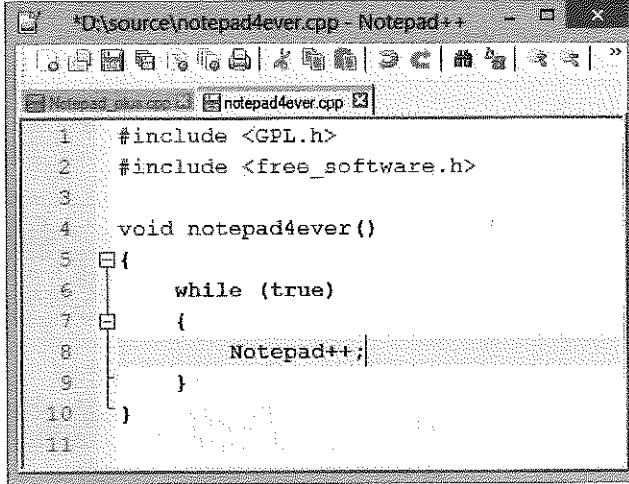
Herhangi bir programlama dili kullanılarak yazılmış metinlere **kaynak kod** denir. Farklı programlama dillerinde yazdığınız kaynak kodlar (dosyalar) farklı uzantılara sahip olur. Örneğin, C# dili ile yazdığınız kaynak kodun dosya uzantısı **.cs** olurken, Visual Basic ile yazılan dosyaların uzantısı **.vb** olmaktadır.

Herhangi bir programın kaynak kodlarını **Not Defteri** veya **Notepad++** gibi basit editörlerle yazabileceğiniz gibi **Visual Studio** gibi daha gelişmiş editörlerle de yazabilirsiniz.

Editör Nedir?

Kaynak kodları oluşturmak yani kod yazmak veya düzenlemek için kullanılan yazılımlardır. Editör **Not Defteri** gibi basit bir yazılım da olabilir, **Visual Studio** gibi çok daha gelişmiş bir editör de olabilir.

Aşağıdaki şekilde çok yaygın kullanılan editörlerden biri olan **Notepad++** editörü görülmektedir. Notepad++ yaklaşık 5 MB yer kaplayan bir editördür ve HTML, CSS, Javascript, PHP, C, C++, C# gibi birçok dilde kod yazmanızı ve kodları düzenlemenizi sağlar. Programı indirmek için <http://notepad-plus-plus.org/> adresini ziyaret edebilirsiniz.



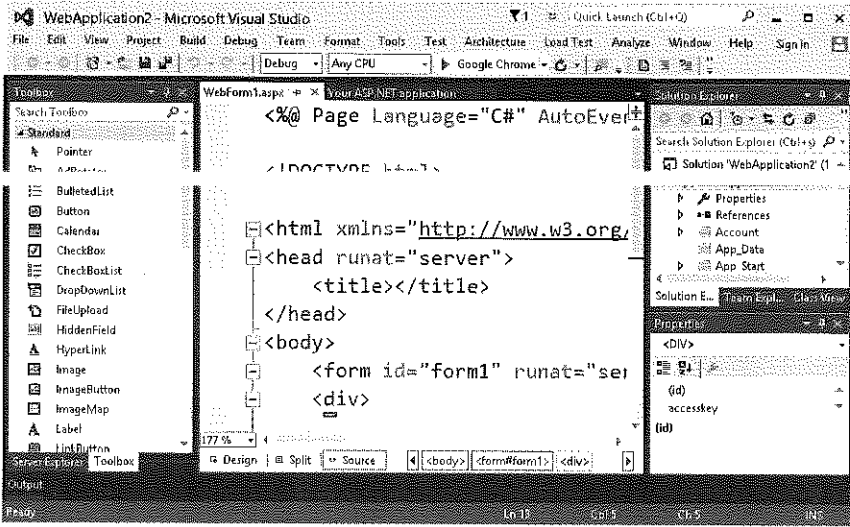
Şekil 1.6 Notepad++ editörü

İnternette herhangi bir arama motorunu kullanarak ücretli veya ücretsiz birçok editör bulabilirsiniz. Örneğin, arama motoruna “**best free editör for c#**” yazdığınızda <http://www.software-pointers.com/en-coding-csharp-tools.html> gibi adreslere ulaşarak buradan farklı editörleri bulup indirebilirsiniz. Ben bu sayfayı ziyaret ettiğimde C# dili için kullanabileceğim dokuz adet editöre ulaştım. Bunlar arasında **jEdit**, **Notepad++** ve **SharpDevelop** oldukça fazla sayıda kullanıcı kitlesine sahiptir. Eğer tüm C# profesyonellerinin kullandığı editör hangisi diye sorarsanız, tartışmasız **Visual Studio** derim.

IDE Nedir?

IDE, Integrated Development Enviroment, yani Tümlleşik Geliştirme Ortamı kelimelerinin kısaltılmışıdır. IDE'ler yazılım geliştirmek için kullanılan ve editörlere göre nispeten daha gelişmiş yazılımlardır. Genelde bu tür yazılımların kodları renklendirme, kod tamamlama, hata bulma ve derleme gibi ekstra özellikleri vardır. **Visual Studio** veya **NetBeans** gibi yazılımlar IDE'ye örnek olarak verilebilir.

Aşağıdaki şekilde Microsoft tarafından üretilen ve en çok kullanılan IDE'lerden biri olan **Visual Studio** görülmektedir. Dikkat edilirse Notepad'den daha karmaşık bir arayüze ve daha gelişmiş bazı özelliklere sahiptir. Eğer herhangi bir Visual Studio sürümünü bilgisayarınıza yüklerseniz başka bir araca gerek kalmadan kod yazma, hata ayıklama ve derleme gibi işlemleri rahatlıkla yapabilirsiniz.



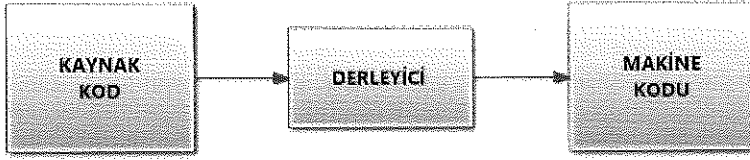
Şekil 1.7 Visual Studio

Derleyici (Assembler) Nedir?

Belli bir programlama diliyle yazılan kaynak kodların işlemcinin anladığı makine koduna çevrilmesini sağlayan yazılımlara **derleyici** denir. Kaynak kod, derleyici ve makine kodu arasındaki ilişkiyi aşağıdaki şekilde görebilirsiniz; yazılan kaynak kodlar uygun bir derleyici ile bilgisayarın anladığı dil olan makine diline çevrilmektedir.

YÜKSEK SEVİYELİ DİL

DÜŞÜK SEVİYELİ DİL



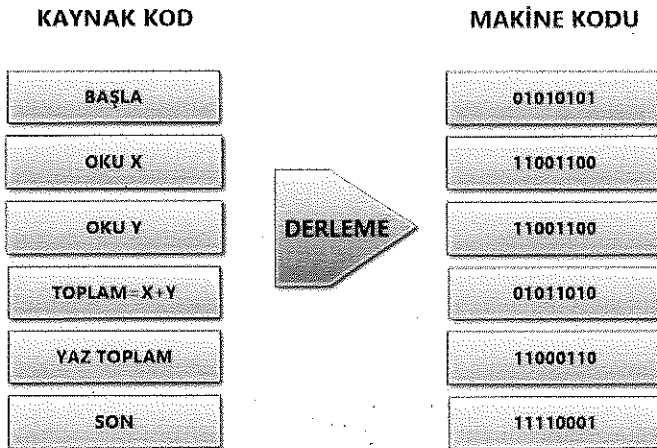
Şekil 1.8 Kaynak kod ve derleyici ilişkisi

Bir programı BASIC, C# veya Assembly gibi bir dille yazmış olabilirsiniz. Hiçbir işlemci bu kodları doğrudan çalıştıramaz. Bu kodların çalışması için önce uygun bir **derleyici** ile makine diline çevrilmesi ve ondan sonra çalıştırılması gerekmektedir.

Her derleyici her dill makine diline çevirmez; farklı programlama dilleri için farklı derleyiciler bulunmaktadır. Bu nedenle yazdığımız kodlar uygun bir derleyici ile makine diline çevrilmelidir. Örneğin C ile kod yazıyorsanız **Borland C** derleyicisini tercih edebilirsiniz. Bu derleyici hem bir editör olarak kod yazmanızı sağlar hem de yazdığınız kodların derlenmesini, yani makine diline çevrilmesini sağlar.

C# dili ile kod yazanlar genelde Visual Studio'yu tercih etmektedir. Visual Studio hem gelişmiş bir editördür hem de C# ile yazılan kodları derleyerek makine diline çevirir.

Uzun lafın kısıası, aşağıda da görüldüğü gibi, derleyiciler belli bir programlama dili ile yazılan kodların makine diline çevrilmesini ve işlemci tarafından çalıştırılmasını sağlar.



Şekil 1.9 Derleme işlemi.

Yorumlayıcı (Interpreter) Nedir?

Bir önceki konuda anlatıldığı gibi, derlenen kodlar makine diline çevrilir ve bilgisayar tarafından çalıştırılabilir hale gelir. Bir kod derlendiği zaman artık çalışır hale gelmiştir ve bir daha üzerinde değişiklik yapılmadığı sürece derlenmesi gerekmeden çalışır.

Yorumlanan kodlar ise satır satır makine diline çevrilir ve öyle çalıştırılır. Bu kodlar her çalıştırıldığında tekrar derlenir. Bu nedenle yorumlanarak çalışan kodlar derlenerek çalışan kodlara göre biraz yavaştır.

Çoğu BASIC editörü yazılan kodları derlemez, yorumlayarak çalıştırır. Bir başka örnek de PHP dilidir; PHP kodları derlenmez, her çalıştığında yorumlanır yani satır satır derlenir. Bu nedenle PHP dili ASP.NET'e göre yavaş çalışıyor şeklinde iddialarla sıklıkla karşılaşılır.

Amaç (Executable) Program Nedir?

Bilgisayar işletim sistemi tarafında çalıştırılabilir ve bilgisayar tarafından çalıştırılabilen programlara amaç program denir.



Şekil 1.10 Amaç Program

Borland C, Notepad veya Visual Studio gibi bir editör kullanarak kodları yazarsanız bu kodlar sizin **kaynak** kodlarınız olur. Eğer bu **kaynak** kodları derleyip **.exe** uzantılı bir program elde ederseniz bu **amaç** program ya da **çalıştırılabilir** program olur. Bu programın özelliği, kaynak kodlara gerek kalmadan uygun işletim sistemine sahip her bilgisayarda çalışıyor olmasıdır.

Eğer kullandığınız ya da kurduğunuz bir program varsa bu programın **.exe** uzantılı çalıştırılabilir dosyasını "Program Files" klasöründeki ilgili alt klasörde bulabilirsiniz. Eğer **.exe** dosyaya çift tıklarsanız programınız çalışmaya başlar. Program kurulduğunda başlat menüsünde oluşturulan kısayollar aslında **.exe** uzantılı amaç dosyalarını çalıştıran kısayollardır.

Alt ve Üst Seviye Diller

Bir programlama dili makine diline ne kadar yakınsa o kadar alt seviye bir dil olur. En alt seviye dil doğrudan donanım ile konuşabilen yani işlemciyi doğrudan çalıştıran makine dilidir.

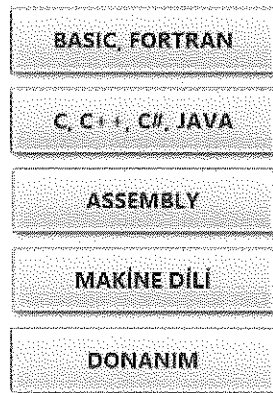
Alt seviye diller genel olarak daha az donanıma ihtiyaç duyar, daha hızlı çalışır ve donanım bazında komutlar çalıştırmamıza olanak sağlar. Bu nedenle, özellikle hızlı çalışması istenen, daha yüksek performans beklenen yazılımlarda alt seviye diller tercih edilebilir. Örneğin işletim sistemleri, kart sürücüler veya otomasyon projelerinde bu tür diller tercih edilir.

Alt seviyeli dillerle yazılan kodlarda daha iyi performans elde edilmesine rağmen bu tür dillerle program yazmak oldukça zor ve zaman alıcıdır. Ayrıntılı ve oldukça profesyonel programlama bilgisi gerekmektedir ve genellikle mecbur kalınmadığı sürece pek tercih edilmez.

Üst seviye diller ise insanların daha kolay öğrenebileceği şekilde tasarlanmıştır. Program yazmak daha kolaydır ve çok hızlı uygulama geliştirmemizi sağlar. Buna karşılık donanım ihtiyaçları daha fazladır ve daha yavaş çalışabilirler.

Yüksek seviyeli diller genelde performansın çok önemli olmadığı ve yüksek donanım gücünün olduğu projelerde tercih edilir. Ayrıca, öğrenilmesi nispeten kolay olduğu için programcılığa yeni başlayanlar tarafından tercih edilebilir.

Aşağıda alt ve üst seviye dillerle ilgili bir tablo görülmektedir. Herhangi bir donanımın doğrudan çalıştırdığı dil makine dilidir. Diğer diller makine diline derlenip o şekilde çalıştırılır.



Şekil 1.11 Alt ve Üst seviye diller

Problem Çözme Aşamaları

İster günlük hayatta, ister matematikte, ister bilgisayarda olsun her problemin çözüm aşaması vardır. Eğer herhangi bir “Problem Çözme” eğitimine katılırsanız bunları öğrenirsiniz. Eğer iyi algoritmalar hazırlamak ve kaliteli kod yazmak istiyorsanız problem çözmeyi de öğrenmeniz lazım. Bir problemi çözmek için aşağıdaki aşamaları izlemelisiniz:

Problemin Tanımlanması

Bu aşamada, problemin eksiksiz bir şekilde açığa çıkarılması ve tanımlanması gereklidir. Problem ne kadar iyi tanımlanırsa, çözüm de o kadar iyi ve doğru olur.

Eğer problem konusunda uzman değilseniz, bir uzmana danışın ve konuyu enine boyuna iyice öğrenin. Mümkünse bir çözüm grubu oluşturun ve beyin fırtınası yapın.

Algoritmanın Oluşturulması

Problemi ve istekler iyice tanımlandıktan sonra, çözüm aşamaları mantıklı bir sıraya koyulmalıdır. Problem için birden fazla çözüm bulunabilir; bu çözümlerden en uygunu, en basiti ve en etkili olanı seçilmelidir. Algoritma gerek akış diyagramı ile gerekse pseudo (sözel) kodlarla kâğıda dökülmelidir.

Deneme ve Düzeltme

Oluşturulan algoritma veya çözüm denenir. Eğer iyi sonuç verdiyse kullanılır, iyi sonuç vermediyse kullanılan algoritma ya da kodlar düzeltilerek tekrar denenir. Kodlar yavaş çalışıyorsa kötü bir algoritma tercih edilmiş olabilir. Başka algoritmalar araştırılıp öğrenilerek daha hızlı çalışan programlar elde edilebilir.

Yazılım Geliştirme Süreci

Ciddi bir yazılım geliştirmenin değişik aşamaları vardır. Bu aşamaların neredeyse her biri ayrı uzmanlık alanıdır. Buradaki amacım size bu aşamaları ayrıntılı olarak öğretmek değil sadece farkındalık yaratmaktır.

Bir yazılım projesinde temel olarak aşağıdaki aşamalar bulunmaktadır:

Analiz

En önemli aşamalardan biridir; bu aşamada müşteri istekleri belirlenir ve belirlenen

isteklere göre programın işlevleri ortaya çıkartılır. Yazılımın neler yapacağı, hangi sorunların çözüleceği bu aşamada belirlenir.

Basit bir depo programında bile analiz aşaması vardır; müşterinin deposu ziyaret edilir, işlem ve evraklar gözlenir, en ince ayrıntısına kadar kurumun işleyişi ile ilgili notlar tutulur. Uzun lafın kısıası, bu aşamada algoritma hazırlanır.

Tasarım

Yapılan analiz sonucunda hangi teknolojilerin kullanılacağı belirlenir. Yazılacak olan program web tabanlı mı? Yoksa masaüstü mü olmalı? Hangi programlama dili kullanılmalı? Hangi veritabanı seçilmeli? Hepsi bu aşamada belirlenir.

Kodlama

Analiz sonucunda algoritma, kullanılacak dil ve teknolojiler belirlendikten sonra kodlar yazılmaya başlanır. Programcılar ellerindeki analiz raporlarına ve algoritmaya bakarak veritabanını oluştururlar, program arayüzünü geliştirirler ve kodları yazarlar.

Test

Kodlama sonucunda iki türlü hata ortaya çıkabilir; biri programlama diline ait komutsal (syntax) hatalardır. Bu hataları düzeltmeden program zaten çalışmaz. Diğeri ise mantık hatalarıdır. Yani, tüm kodlar doğru olmasına rağmen bilgi eksikliğinden veya yanlış düşünceden ve yöntemden meydana gelen hatalardır. Bu durumda program çalışır fakat hatalı sonuç verir. Böyle bir durumda algoritma tekrar gözden geçirilmesi ve kodlar yeniden yazılmalıdır.

Bu aşamada özel olarak eğitilmiş kullanıcı veya programcılar yazılan tüm kodları test ederler ve hata bulmaya çalışırlar. Arayüz, kod, veritabanı ve değişik ayarlardan dolayı hatalar olabilir.

Yazılım müşteriye teslim edilmeden önce iyice test edilir, bulunan hatalar düzeltilir ve mümkün olduğunca hatasız bir program üretilmeye çalışılır.

Kurulum

İyice test edilip güzel çalıştığından emin olunan program müşteriye kurulur ve gerekli eğitimler verildikten sonra devreye alınır. Bu aşamada müşteri programı kullanmaya başlar.

Destek ve Bakım

Her üründe olduğu gibi yazılımlarda da sorunlar çıkar ve müşteriler desteğe ihtiyaç duyar. Yazılım müşteriye kurulduktan ve devreye alındıktan sonra gerek acemi kullanıcılar gerekse sorun yaşayan tecrübeli kullanıcılara destek verilerek yazılımın sağlıklı çalışması sağlanmalıdır.

Ayrıca belirli dönemlerde program veya veritabanının yedeklerinin alınması, özellikle veritabanının en yüksek performansta çalışması için bakımları yapılmalıdır.

Güncelleme

Bu aşamada müşterilerden gelen öneri ve şikâyetler doğrultusunda program üzerinde değişiklikler yapılır; hatalar varsa düzeltilir, eksiklikler varsa gerekli eklemeler yapılır.

Algoritmanın Oluşturulması

Algoritma oluşturmanın değişik yöntemleri vardır. Bu yöntemlerden biri yapılacak işi kâğıda satır satır yazarak tarif eden ve adına pseudo (yalancı) kod denen yöntemdir. Bir diğer yöntem ise algoritmanın belirli şekillerle akış diyagramının çıkarılması ve görsel hale getirilmesidir.

Pseudo Kod

Pseudo kodlar, problem çözme aşamalarını kendi cümlelerimizle kâğıda döktüğümüz kodlardır. Örneğin, 2 sayıdan büyük olanı bulmak istiyorsanız aşağıdaki gibi bir pseudo kod yazabilirsiniz:

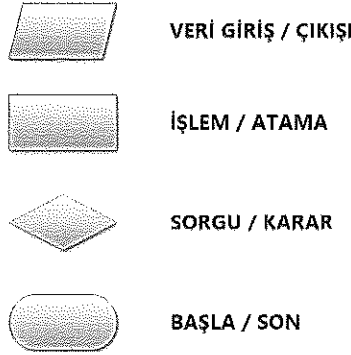
```
İlk sayıyı al (s1)
İkinci sayıyı al (s2)
Eğer s1>s2 ise
  Ekrana "s1 büyük" yaz
Aksi takdirde "s2 büyük" yaz
```

Pseudo kodlarda belli bir standart yoktur. Herkes kendi kafasına göre pseudo kod oluşturabilir. Bu da nispeten pseudo kodların zor anlaşılmasını sağlar. Özellikle çok büyük projelerde alt alta gelişigüzel yazılan pseudo kodlar yanlış anlamalara ve hatalara neden olabilir.

Akış Diyagramları

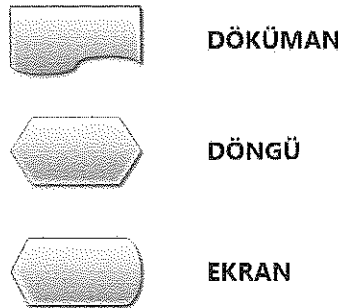
Akış diyagramı, özel bazı şekillerle algoritmanın ifade edilmesini sağlar. Akış diyagramları sadece programcılıkta kullanılmaz; değişik işleri gerçekleştirmek, kişilerin yapacağı iş ve işlemlerin doğru sırayla yapılmasını sağlamak, makinelerde işlemlerin belli bir sıraya göre yapılmasını sağlamak, belirli şartlar gerçekleştiğinde belirli işlemlerin yapılmasını sağlamak vb. amaçlarla birçok yerde kullanılır.

Çok kullanılan akış diyagramı şekillerini ve anlamlarını aşağıdaki şekilde görebilirsiniz.



Şekil 1.12 Çok kullanılan akış diyagramı şekilleri ve anlamları

Yukarıda belirtilen ve sık kullanılan şekillerin dışında da kullanılan bazı akış diyagramı şekilleri vardır. Kafamızın karışmaması için biz bu kitapta yukarıda görülen en temel şekilleri kullanacağız. Aşağıda karşınıza çıkabilecek fakat bizim pek kullanmayacağımız şekilleri de görebilirsiniz:

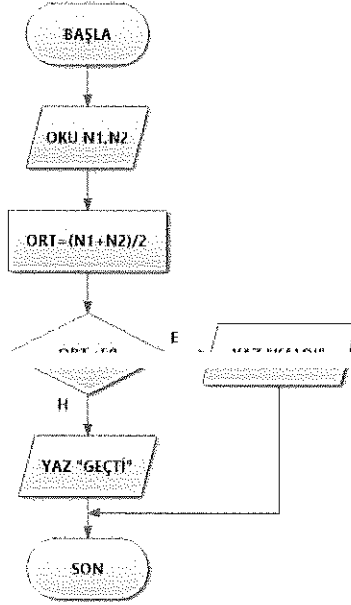


Şekil 1.13 Akış diyagramı şekilleri ve anlamları

Akış diyagramları artık neredeyse standart hale gelmiştir. Bu nedenle yapılacak işlemleri başka insanlara anlatmak ve kodların başkaları tarafından daha kolay anlaşılmasını sağlamak akış diyagramları sayesinde daha kolay hale gelmiştir.

Pseudo kodlara göre bir kıyaslama yapılırsa, akış diyagramlarını okumak ve anlamak daha kolay görünmektedir. Ayrıca, görsel olduğu için akılda daha kalıcı olabilir. Buna karşın özellikle programcılığa yeni başlayanlar için akış diyagramlarında kullanılan simgeler kafaların karışmasına neden olabilir.

Girilen iki ders notunun ortalamasını bulan algoritmanın akış diyagramı aşağıdaki gibidir:



Şekil 1.14 İki ders notunun ortalamasını bulan akış diyagramı

Kod Yazma & Editörler

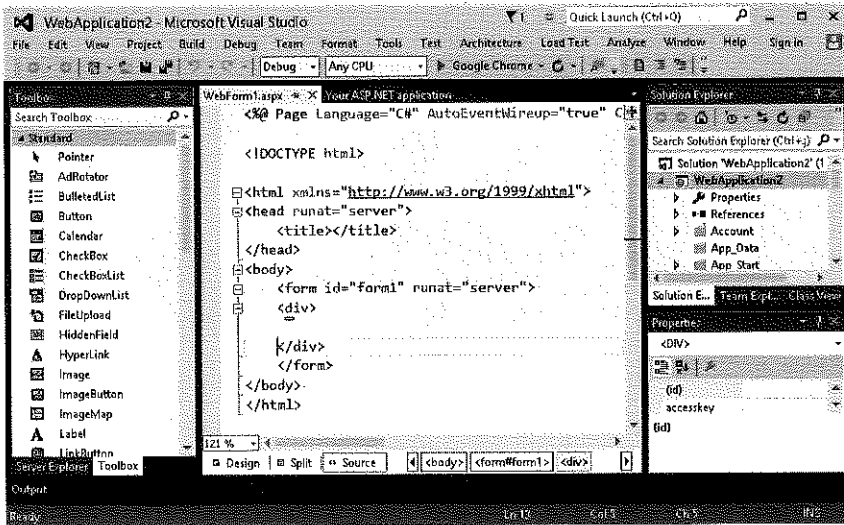
Algoritmayı oluşturduktan sonra sıra kod yazımına gelir. Kod yazmak için **Not Defteri** gibi basit bir editör de kullanabilirsiniz, **Visual Studio** gibi gelişmiş bir editör de. Biz kodlarımızı yazarken sağladığı bazı avantajlardan dolayı **Visual Studio**'yu tercih edeceğiz.

Neden Visual Studio?

- En yaygın kullanıma sahip C# editörü Visual Studio'dur. Herhangi bir yazılım firmasında çalışmaya başlarsanız muhtemelen Visual Studio kullanacaksınız. Bu nedenle Visual Studio'yu iyi bilmek zorundasınız.

- Visual Studio'nun eğitim veya amatörler için ücretsiz sürümleri vardır. Bu sürümleri <https://www.visualstudio.com/tr/> adresinden inceleyip indirebilirsiniz. Tamamen ücretsiz olan **Visual Studio Community** sürümünü indirip kurarak bu kitaptaki uygulamaları gerçekleştirebilirsiniz.
- Çok farklı platformlar için kod yazmamızı sağlar. Örneğin, Windows, iOS veya Android için uygulama yazabilirsiniz. Bu özellikten faydalanmak için Visual Studio 2013 ve daha sonrası sürümleri seçmelisiniz.
- Herkese ve her amaca yönelik ücretli-ücretsiz sürümleri bulunmaktadır. Acemi veya öğrenciler ücretsiz sürümleri kullanarak kod yazabilirken, kurumsal yazılımcılar daha gelişmiş işlevler sağlayan sürümleri kullanabilir.
- Microsoft tarafından üretilir ve sürekli geliştirilir; Yeni teknolojileri desteklemesinin yanında sürekli güncel tutulur.
- Farklı komutları farklı renkte gösterir ve kod yazmayı kolaylaştırır.
- Otomatik kod tamamlama özelliği sayesinde daha hızlı uygulama geliştirmemizi sağlar.
- C#, C++, VB.NET, HTML, CSS, Javascript ve diğer birçok dili destekler.
- Debugger özelliği sayesinde, yazılan hatalı kodları gösterir, hatalarla ilgili bilgi verir ve düzeltilmesini sağlar.
- C#, VB.NET ve C++ gibi yaygın kullanılan dillerin derleyicileri hazır olarak geldiği için ekstra bir program kullanmanıza gerek yok.
- Sadece masaüstü uygulamaları değil, konsol, mobil ve web uygulamaları da geliştirilmesini sağlar.

Uzun lafın kısısı, Visual Studio kod yazmaya odaklanmanızı sağlar. Aşağıdaki şekilde Visual Studio'nun arayüzünü görebilirsiniz.



Şekil 1.15 Visual Studio 2015

Visual Studio Express veya Community Edition

Nereden Bulurum?

Visual Studio Express veya **Community** Edition, Microsoft tarafından ücretsiz sunulan sürümlerdir. Bu sürümleri kullanarak eğitim veya amatör amaçlı masaüstü, web veya mobil uygulamalar geliştirebilirsiniz. Microsoft tarafından ücretsiz olarak sunulan bu editörlerin son sürümlerini <https://www.visualstudio.com/tr/vs/> adresinden indirip bilgisayarınıza kurabilirsiniz.

Bu kitap yazılırken **Visual Studio Community 2015** sürümü kullanılmıştır. Bu kitapta uygulamaları yapmak için elinizde olan veya indirebileceğiniz herhangi bir sürümü kullanabilirsiniz. Eğer elinizde herhangi bir sürüm yoksa ücretsiz olan **Visual Studio Community** sürümünü kurmanızı tavsiye ederim. Her ne kadar Visual Studio iyi ve yaygın kullanılan bir editör olsa da kitaptaki uygulamaları yapmak için başka bir editör de kullanabilirsiniz.

C# dilinin doğal editörü Visual Studio'dur. Her ne C# her ne de Visual Studio Microsoft tarafından geliştirilmektedir. O nedenle en az hata ve sorunla karşılaşmak için Notepad++ gibi diğer editörleri pek tavsiye etmem. Ayrıca, Visual Studio Community veya Express sürümleri bedava olduğundan dolayı diğer editörleri tercih etmek için pek sebep de yok. Aşağıdaki şekilde görülen <https://www.visualstudio.com/vs/> adresini ziyaret edip **Download Visual Studio/Visual Studio İndir** bağlantısına tıkladığınızda karşılaşılabileceğiniz sayfayı görüyorsunuz.



Şekil 1.16 Visual Studio indirme bağlantısı

Visual Studio İçinde Neler Var?

İndirdiğiniz sürüme göre değişmekle birlikte Visual Studio içinde aşağıdaki bileşenler bulunabilir:

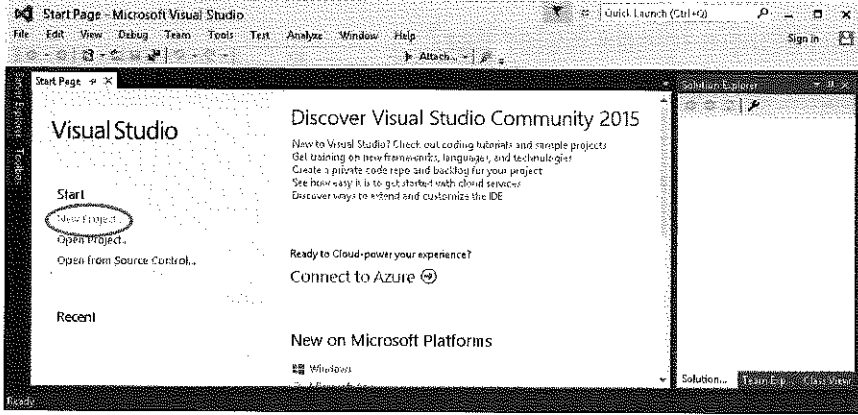
- .NET uygulamaları geliştirmek için editör
- .NET Framework
- MVC ve Web Formları
- Masaüstü ve web uygulamaları geliştirmek için kontroller
- C# ve VB.NET Programlama dilleri
- C# ve VB derleyicileri
- Web Sunucusu (IIS Express)
- Veritabanı Sunucusu (SQL Server Compact)
- Değişik teknolojileri destekleyen eklentiler (Extension)

Visual Studio (VS) Arayüzü ve İlk Proje

Bu başlık altında programcılığa küçük bir giriş yapacağız. Amacım çok fazla ayrıntıya girmek değil, sadece Visual Studio kullanarak basit bir kod yazmak ve sizi havaya sokmak. Ayrıntılara daha sonraki bölümlerde gireceğiz.

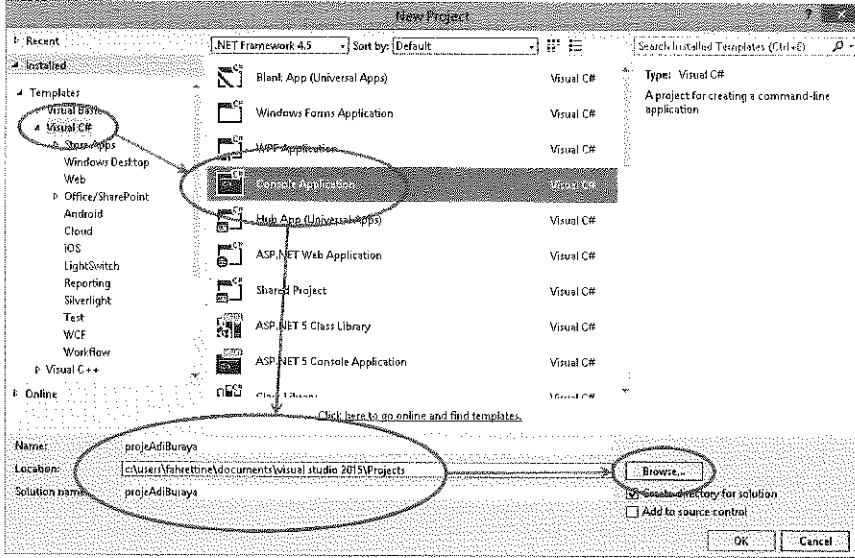
Visual Studio ilk açıldığında karşınıza aşağıdaki başlangıç ekranı gelir. Genelde yeni başlayanlar Visual Studio'yu biraz karışık bulur ama zaman geçtikçe alışırılar ve bir daha vazgeçemezler.

Ekranda ne var ne yok hepsini anlamaya çalışmayın; zamanla işimize yarayacak öğeleri teker teker öğreneceğiz. Siz şimdilik sadece yapmanız gereken işe odaklanın. Yapmamız gereken iş Visual Studio'yu açmak ve basit bir program yazmak için bir proje açmak. Eğer VS programını hâlâ kurmadıysanız sonraki bölümlerde nasıl indirildiğini ve nasıl kurulduğunu bulabilirsiniz.



Şekil 1.17 Visual Studio Açılış Ekranı (Start Page)

Visual Studio açıldığında gördüğünüz sayfa **Start Page** adını taşır. Eğer internete bağlıysanız bu sayfada Microsoft dünyasından bazı yenilikler hakkında haber metinleri bulabilirsiniz. Bu sayfadayken **New Project...** linkine tıklarsanız aşağıdaki **New Project** ekranına ulaşırsınız:



Şekil 1.18 New Project penceresi

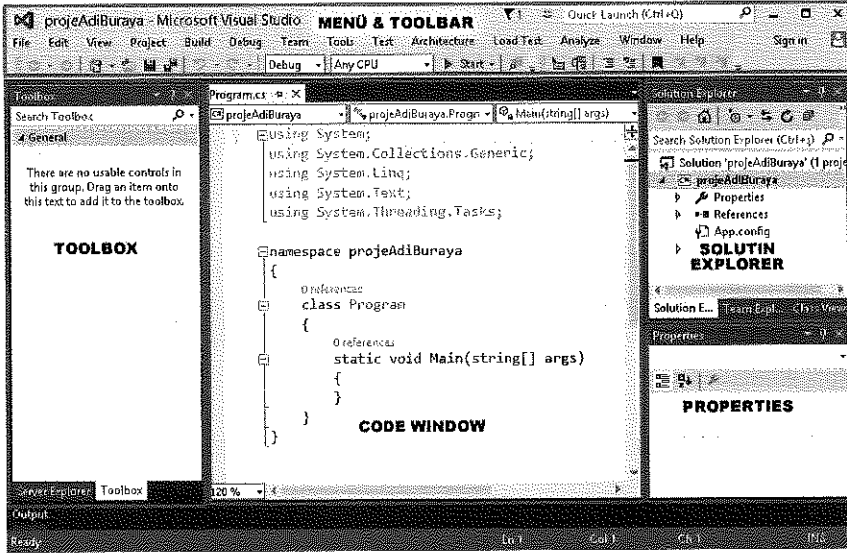
New Project penceresinde **Templates** → **Visual C#** → **Console Application** komutunu çalıştırın.

Pencerenin altında **Name** kutusuna proje adını, **Location** kutusuna da proje kayıt yerini yazın. Tüm projeler varsayılan olarak **c:\users\kullanıcı\documents\visu-**

al studio 201x\ Projects klasörüne kaydedilmektedir. Eğer Türkçe işletim sistemi kullanıyorsanız bu adres c:\Kullanıcılar\kullanıcı\Belgelerim\visual studio201x\ Projects olarak görünmektedir. İsterseniz **Browse...** düğmesine tıklayarak proje kayıt yerini kendiniz belirleyebilirsiniz.

Create directory for solution kutusunu işaretleyin. Böylece projeniz için otomatik bir klasör oluşturulur ve diğer projelerle karışması önlenir.

Proje açıldığında aşağıdaki ekran gelir:

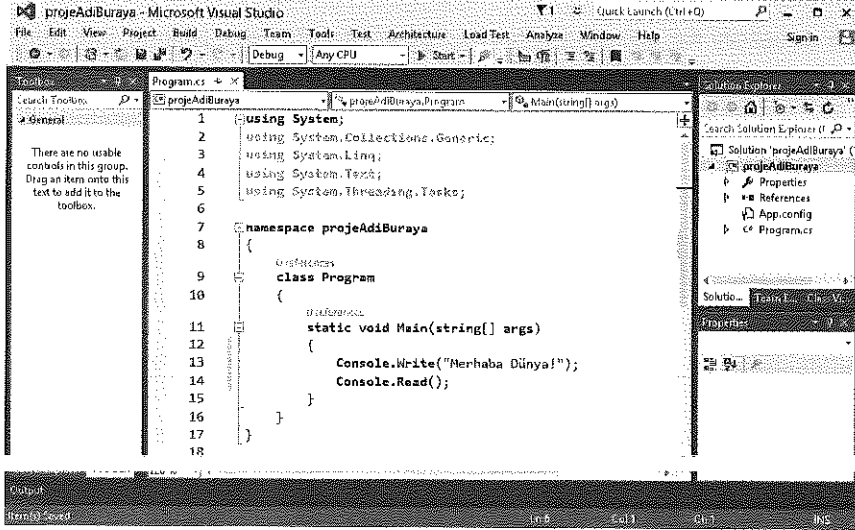


Şekil 1.19 Visual Studio Arayüz elemanları

Kodlarımızı **Code Window** içine yazacağız; zaten dikkat ederseniz bazı kodlar hazır olarak gelir. Yukarıda gösterilen arayüz elemanlarını açıp kapatmak için **View** menüsünü kullanabilirsiniz.

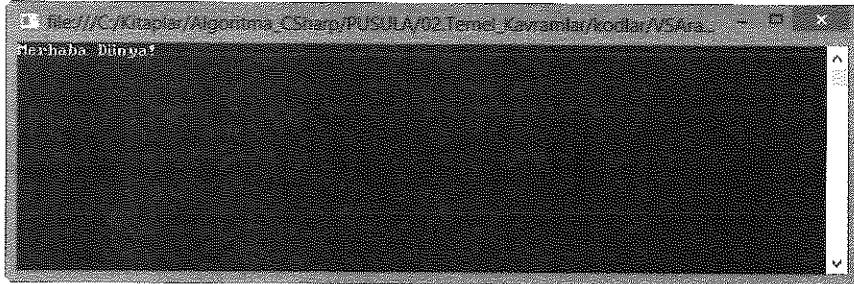
İlk Programımız

Code penceresinde **static Void Main** ile başlayan kodların içine aşağıdaki kodları aynen yazınız:



Şekil 1.20 İlk programımız

Programı çalıştırmak için **F5** fonksiyon tuşuna basın veya yukarıda **Toolbar→Start** butonuna tıklayın. Eğer hata yoksa aşağıdaki ekranı görürsünüz.

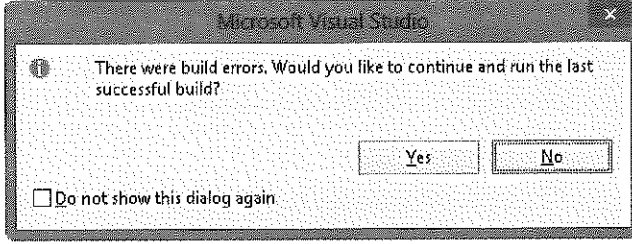


Şekil 1.21 Program çıktısı

Program penceresini **"ENTER"** ile veya pencerenin sağ üst köşesindeki **(x)** işaretinden kapatabilirsiniz.

Yukarıda yazmanız istenilen kodları yanlış yazabilirsiniz; Bir komutu yanlış yazabilir veya bir karakteri gözden kaçırabilirsiniz. Örneğin herhangi bir noktayı yazabilirsiniz. Bizim için çok önemsiz görünen bir şey C# dili için çok önemli olabilir. Komutların sonuna noktalı virgül (;) koymazsam sorun çıkmaz diye düşünebilirsiniz. Ya

da **Console** ile **Write()** arasına nokta (.) koymayı unutmuş olabilirsiniz. Bütün bunlar C# dili için çok önemlidir ve kodları mutlaka kuralına göre yazmalısınız. Eğer kodları hatalı yazdıysanız aşağıdaki gibi bir pencere çıkar:



Şekil 1.22 Hatalı kod penceresi

Hata penceresi ile karşılaştıysanız **No** diyerek pencereyi kapatın ve hatanızı düzeltip programı **F5** ile yeniden çalıştırın.

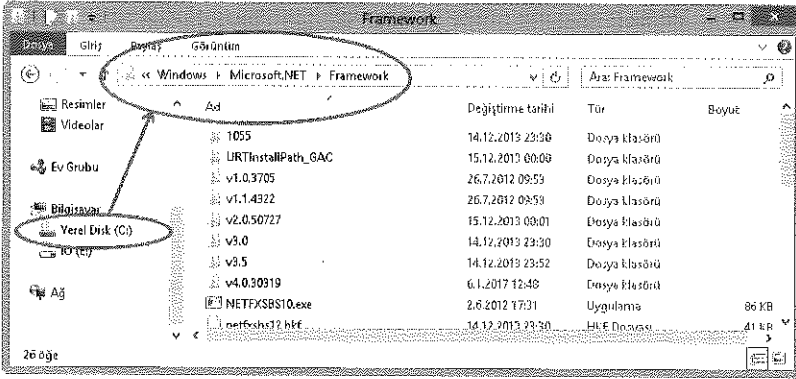
Şimdilik sadece basit bir program yazarak programcılığa biraz ısınmaya çalıştık. Önümüzdeki konularda daha karmaşık algoritma ve daha karmaşık kodlarla uğraşmaya başlayacağız.

Komut Satırı Derleyicisi (csc.exe)

Visual Studio kurulduğunda, .NET Framework ve derleyiciler de otomatik olarak kurulmuş olur. Yukarıda yaptığımız örnek uygulamadaki gibi, eğer bilgisayarınızda Visual Studio kurulu ise **F5** fonksiyon tuşuna basarak veya **Toolbar→Start** butonuna tıklayarak kodlarımızı hem derleyip hem de çalıştırabiliriz. Gerçekten de büyük kolaylık...

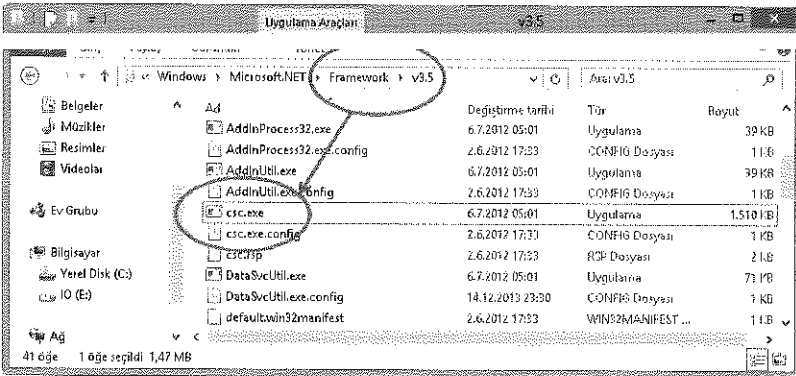
Peki! Editor olarak Visual Studio değil de, örneğin Notepad++ kullanıyorsanız ne yapacaksınız? Bilgisayara Notepad++ veya benzeri bir editör kurulduğunda .NET Framework otomatik olarak kurulmaz. Dolayısıyla bilgisayarınızda C# derleyicisi de olmaz. Bu durumda .NET Framework yazılımını indirip kurmanız gerekmektedir. .NET Framework içinde C# derleyicisi de gelir.

Belki de bilgisayarınızda .NET Framework daha önceden kurulmuş olabilir. .NET Framework'ün kurulu olup olmadığını kontrol etmek için aşağıdaki şekilde görüldüğü gibi diskteki **Windows\Microsoft\Framework** klasörünü kontrol edin. Eğer .NET Framework kurulu ise v2.0, v3.0, v3.5, v4.0 gibi klasörler görünecektir.



Şekil 1.23 Framework klasörü

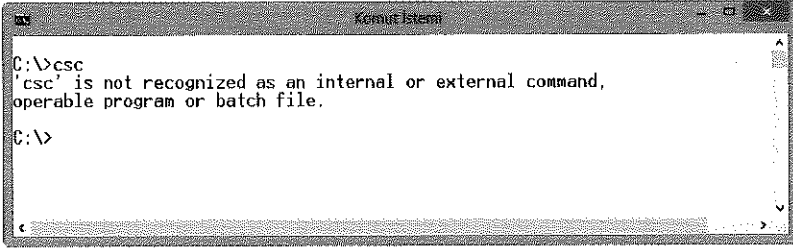
Framework klasörü içinde bulunan v3.x/v4.x klasörlerinden birine tıklayarak içinde **csc.exe** dosyası olup olmadığını kontrol edebilirsiniz. Aşağıdaki şekilde **v3.5** klasörü içinde bulunan C# derleyici dosyası olan **csc.exe**'yi görebilirsiniz.



Şekil 1.24 C# derleyicisi (csc.exe)

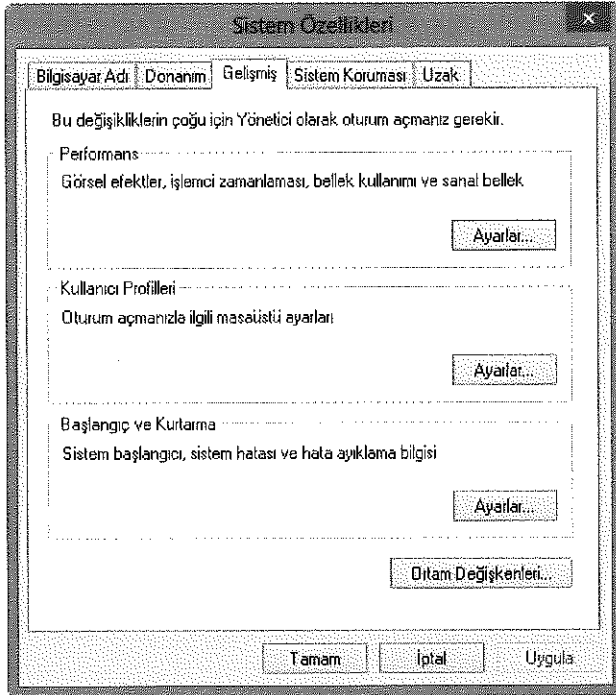
Eğer bilgisayarınızda .NET Framework kurulu değil ve dolayısıyla derleyici de yoksa .NET Framework indirip kurmanız gerekmektedir. Arama motoruna ".NET Framework indir" yazarak arama yapın ve .NET Framework yazılımını indirip kurun. Kurduktan sonra da **Windows\Microsoft.NET\Framework** klasörüne giderek **csc.exe** dosyasını bulun.

.NET Framework kurulumundan sonra derleyiciyi kullanabilmek için küçük bir ayar gerekmektedir. Derleyicinin düzgün çalışıp çalışmadığını kontrol etmek için komut satırını açın ve **csc** yazıp ENTER'layın. Eğer aşağıdaki gibi bir sonuç alırsanız derleyici ayarı yapılmamış demektir.



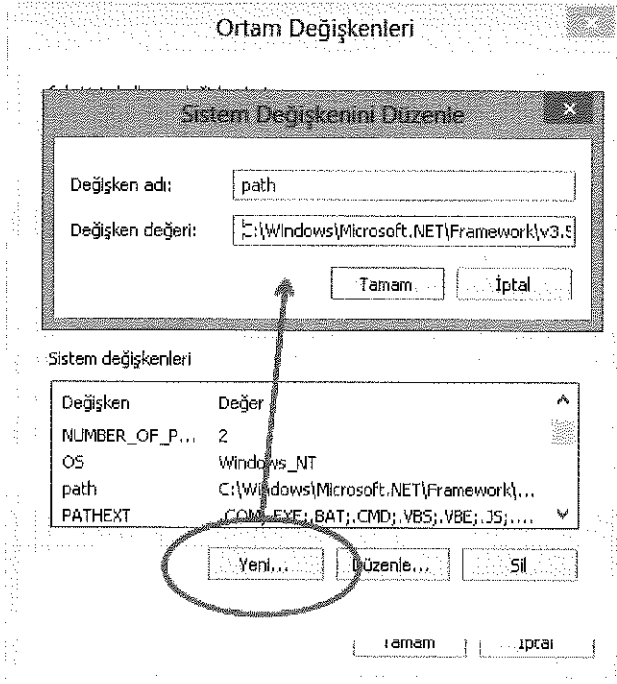
Şekil 1.25 Derleyici ayarı yapılmamış

Derleyici ayarlarını yapmak için **Denetim Masası → Sistem ve Güvenlik → Sistem → Gelişmiş Sistem Ayarları → Ortam Değişkenleri** düğmesine tıklayın. Açılan pencerede **Sistem değişkenleri** bölümünde görülen **Yeni** düğmesine tıklayın. Değişken adı olarak **path**, Değişken değeri olarak **csc.exe** dosyasının adresini yazın ve **Tamam** düğmesine tıklayarak pencereyi kapatın.



Şekil 1.26 Derleyici yolunun belirtilmesi

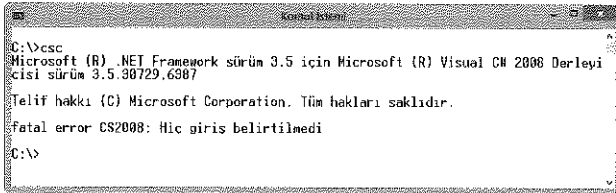
Yukarıdaki ayarı yaptıktan sonra, eğer komut satırı açıksa kapatıp yeniden açın. Komut satırı açıldıktan sonra **csc** yazıp ENTER'layarak sonucu görün. Eğer aşağıdaki gibi bir ekran görürseniz derleyici normal çalışıyor demektir. Eğer hata alıyorsanız büyük bir olasılıkla derleyici adresini (C:\Windows\Microsoft.NET\Framework\v3.x) yanlış yazdınız demektir.



Şekil 1.27 Derleyici normal çalışıyor

Not: Komut satırında işlem yapmak için temel düzeyde DOS komutu biliyor olmanız gerekmektedir. Eğer DOS bilmiyorsanız komut satırında sorun yaşayabilirsiniz. Bu nedenle, illa derleme işlemlerini komut satırından yapacağım diyorsanız DOS'taki dosya, klasör ve listeleme komutlarını öğrenmeniz gerekmektedir. DOS konusu bu kitabın kapsamı dışında olduğu için bu kitapta anlatılmayacaktır.

Komut satırı derleyicisini ayarladıktan sonra **Not Defteri** ile aşağıdaki kodları yazıp **ilkprogram.cs** adıyla **C:\vs** adresine kaydedelim. Dosya uzantısının "**cs**" olduğuna özellikle dikkat ediniz. Eğer dosyayı "**txt**" uzantısıyla kaydederseniz derleyici kaynak kodların bulunduğu dosyayı tanıyamaz ve hata alırsınız.

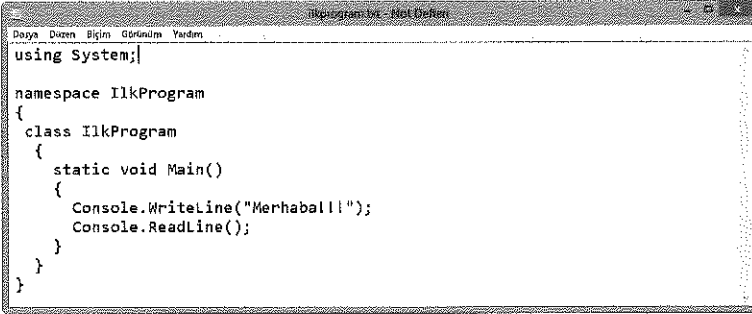


Şekil 1.28 ilkprogram.cs dosyası ve C# kaynak kodları

Dosyanızı kaydettikten sonra komut satırına gidin ve **C:\vs>** klasörünün içindeyken dosyanızı derlemek için aşağıdaki komutu çalıştırın:

csc ilkprogram.cs

Yukarıdaki komutu çalıştırdıktan sonra **c:\vs** klasörüne gidip dosyaları gözlemleyin. Bu dosyalar arasında **ilkprogram.exe** dosyasının da olduğunu göreceksiniz. Bu dosya, **ilkprogram.cs** dosyasının derlenmesiyle elde edilen amaç (executable/ çalıştırılabilir) dosyadır. Bu dosyayı çalıştırmak için Windows ortamında çift tıklayın veya komut isteminde **C:\vs>ilkprogram** komutunu yazıp ENTER'layın.



```

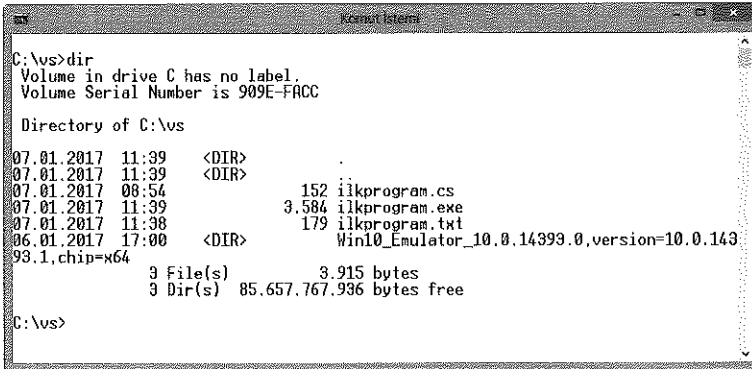
using System;

namespace IlkProgram
{
    class IlkProgram
    {
        static void Main()
        {
            Console.WriteLine("Merhaba!!!");
            Console.ReadLine();
        }
    }
}

```

Şekil 1.29 kaynak ve amaç dosyalar

Derlenen dosyanın çalıştırılması sonucu aşağıdaki ekran görünecektir. Çıktı olarak "Merhaba!!!" metni görüntülenir.



```

C:\vs>dir
Volume in drive C has no label.
Volume Serial Number is 909E-FACC

Directory of C:\vs

07.01.2017  11:39    <DIR>          .
07.01.2017  11:39    <DIR>          ..
07.01.2017  08:54                152 ilkprogram.cs
07.01.2017  11:39            3,584 ilkprogram.exe
07.01.2017  11:38                179 ilkprogram.txt
06.01.2017  17:00    <DIR>          Win10_Emulator_10.0.14393.0,version=10.0.143
93.1,chip=x64
               3 File(s)              3,915 bytes
               3 Dir(s)  85,657,767,936 bytes free

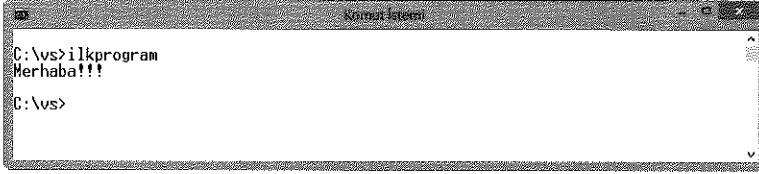
C:\vs>

```

Şekil 1.30 Derlenen dosyanın çalıştırılması

Derleyici Parametreleri

Şu ana kadar kullandığımız C# derleyicisinin birçok parametresi bulunmaktadır. Bu parametreleri öğrenmenin veya hatırlamanın en kolay yolu **csc** komutu hakkında bilgi almaktır. Komut satırında **csc** derleyicisi hakkında bilgi almak için **csc /?** veya **csc /help** komutlarını kullanabilirsiniz.



Şekil 1.31 csc yardım bilgisi

Sıklıkla kullanılan derleyici komut parametreleri aşağıdaki gibidir:

Komut	Açıklama
csc kaynak.cs	Kaynak kodlarımızın derlenmesini sağlar. Sonuçta *.EXE uzantılı bir dosya elde ederiz.
csc /out: yenidosya.exe kaynak.cs	Normalde kaynak dosyayı derlediğiniz zaman aynı ada sahip amaç dosya oluşur. Eğer amaç (EXE) dosya adının farklı olmasını istiyorsanız yandaki sözdizimini kullanırsınız.
csc /t: library kaynak.cs	C# derleyicisini kullanarak dll dosyaları da derleyebilirsiniz. Yandaki sözdizimi ile kaynak.dll adlı bir dll dosyası elde edersiniz.
csc /t: module kaynak.cs	C# derleyicisini kullanarak modülleri de derleyebilirsiniz. Yandaki sözdizimi ile kaynak.netmodule adlı bir dll dosyası elde edersiniz.
csc /t: winexe kaynak.cs	C# derleyicisini kullanarak Windows form uygulamalarını da derleyebilirsiniz. Form uygulamasını derlemek için yandaki sözdizimini kullanınız.

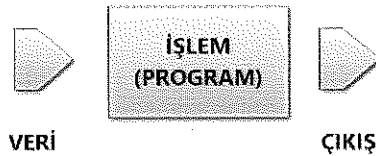
2

Bilgisayar Mimarisi

- Bilgisayar Nasıl Çalışır?
- İşlemci Nasıl Çalışır?
- CPU-RAM İlişkisi
- Onlu ve İkili Sayı Sistemleri

Bilgisayar Nedir?

Dışarıdan veri alan ve bu veriler üzerinde matematiksel ve mantıksal işlem yapıp dışarıya veren makinelere bilgisayar denir.



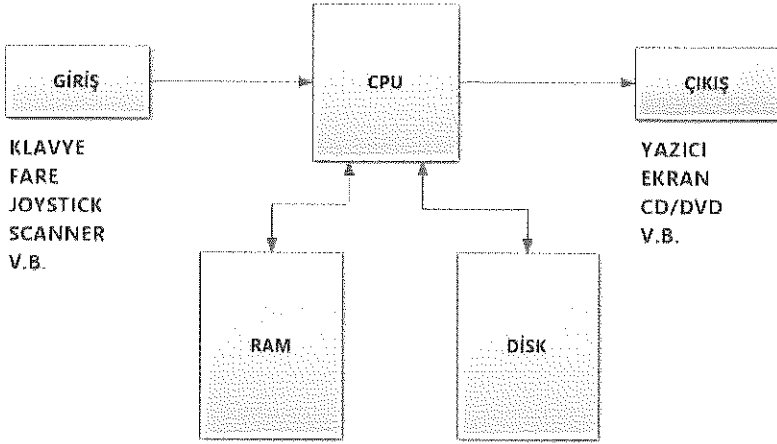
Şekil 2.1 Bilgisayar

Bilgisayarlar çok değişik şekillerde ve işlevlerde karşımıza çıkabilir. Artık herkesin evinde ve işinde kolayca görebileceğiniz tablet, cep telefonu, masaüstü, dizüstü gibi cihazlar hayatımızın vazgeçilmez birer parçası haline gelmiştir. İşin en güzel tarafı ise tüm bu cihazların iş yapabilmesi için bir programa ihtiyaç duyması.

İyi bir bilgisayar programcısı olmak için bilgisayarın nasıl çalıştığını iyi bilmek zorundayız. Özellikle işlemci, RAM, sabit disk ve giriş/çıkış aygıtları arasındaki iş akışının iyi anlaşılmasında fayda var.

Bilgisayar Nasıl Çalışır?

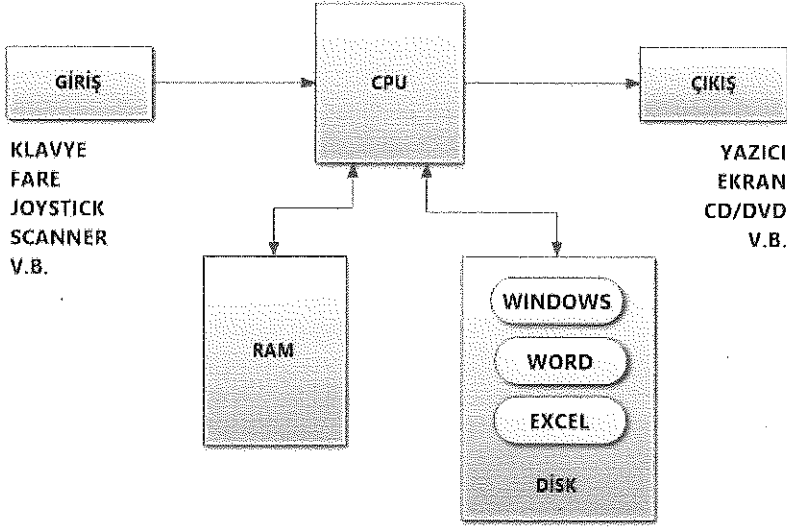
Çok fazla ayrıntıya girmeden size temel bilgisayar donanımını ve bir bilgisayarın nasıl çalıştığını anlatmaya çalışacağım. Temel bir bilgisayar sistemi aşağıdaki gibidir:



Şekil 2.2 Temel Bilgisayar Donanımı

- **CPU (Central Processor Unit):** Türkçe'ye Merkezi İşlem Birimi olarak tercüme edilen CPU, bilgisayarın beynidir; giriş aygıtlarından gelen komutları ve verileri alır, matematiksel ve mantıksal işlemleri yapar ve işlem sonuçlarını çıkış aygıtlarına gönderir. Ayrıca önemli bulduğumuz verilerin diske kaydedilmesini sağlar.
- **Giriş Aygıtları:** CPU'ya komut göndermeye veya veri girmeye yarayan klavye ve fare gibi donanımlardır.
- **Çıkış Aygıtları:** CPU tarafından gerçekleştirilen işlem sonuçlarını görüntülemeye yarayan ekran ve yazıcı benzeri donanımlardır.
- **Sabit Disk:** Kaydedilen dosyaların veya yüklenen programların bulunduğu kalıcı bellektir.
- **RAM:** Çalışan programların ve üzerinde işlem yapılan verilerin bulunduğu geçici bellektir. Normalde sabit diskte bulunan bir program, çalıştırıldığı zaman RAM belleğe taşınır. RAM bellek sabit diskten çok daha hızlı çalışır; bu nedenle normalde sabit diskte bulunan bir program çalıştırıldığı zaman RAM belleğe taşınır ve CPU program ile burada etkileşim kurar.

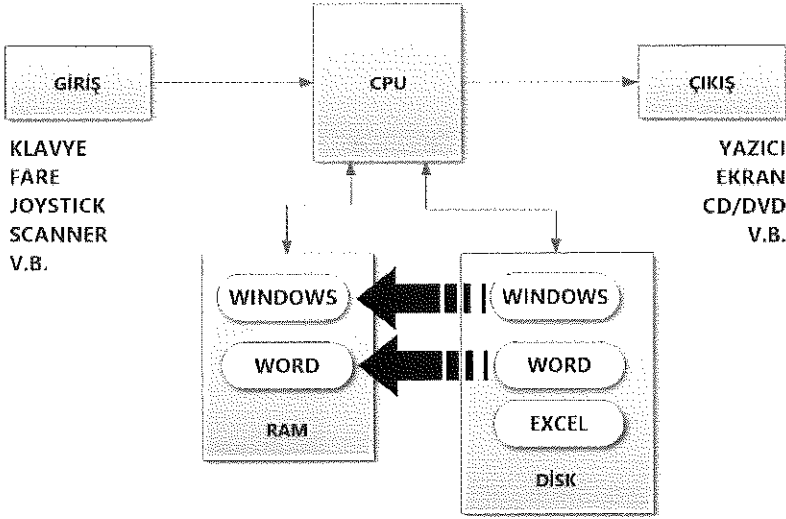
Bilgisayarınıza daha önceden Windows, Word ve Excel programlarını yüklediğinizi varsayalım. Eğer bilgisayarınız kapalı ise, aynı aşağıdaki şekilde görüldüğü gibi bütün programlar sabit diskte. RAM bellek ise tamamen boş. Ayrıca şunu da belirtmekte fayda var; elektrik yoksa veya bir anlığına kesilirse RAM bellek tamamen silinir. Yani RAM geçici bir bellektir. Eğer verilerinizin kalıcı olmasını, elektrik kesilse bile silinmemesini istiyorsanız, verileri sabit diske kaydetmek zorundasınız.



Şekil 2.3 Bilgisayar kapalı iken bütün programlar sabit diskte bulunur.

Eğer bilgisayarınızı çalıştırırsanız, sabit diskte bulunan Windows (veya kullandığınız herhangi bir işletim sistemi) otomatik olarak RAM belleğe yüklenir ve bütün işlemler RAM bellekte yapılır. Windows çalıştıktan sonra örneğin Word programını çalıştırırsanız, normalde sabit diskte bulunan Word programı RAM belleğe yüklenir ve bütün kelime işlemleri RAM bellekte yapılır.

Aşağıdaki şekilde görüldüğü gibi, çalıştırılan programlar RAM belleğe yüklenir ve ilgili işlemler orada gerçekleştirilir. Eğer Word programında bir iş yapar ve bu işi kalıcı olarak kaydetmek isterseniz belli bir isimle sabit diske kaydetmelisiniz. Bu işe, yani kaydedilen verilere "**dosya**" denir. Dosyalar RAM bellekte yapılan işlerin kalıcı olarak sabit diske kaydedilmesini sağlar. Daha sonra bu dosyalar açılıp sabit diskten RAM belleğe taşınarak üzerinde değişiklikler yapılabilir.



Şekil 2.4 Çalıştırılan programlar sabit diskten RAM belleğe yüklenir

Eğer çalıştırdığınız bir programı kapatırsanız RAM bellekten silinir fakat sabit diskte durmaya devam eder. İsterseniz aynı programı daha sonra tekrar açıp üzerinde çalışabilirsiniz; çalıştırılan program gene RAM belleğe taşınır.

Eğer aynı anda bir sürü program çalıştırırsanız ve yeterli büyüklükte RAM belleğiniz yoksa bilgisayar yavaş çalışmaya başlar veya kilitlenebilir. RAM bellekte yeterli alan olmadığı zaman ilgili programın sadece bir kısmı RAM'e taşınır, geri kalan kısmı sabit diskten çalıştırılır. Bu durumda CPU işlemlerin bir kısmını sabit disk üzerinde yapacağından ve sabit disk de RAM belleğe göre daha yavaş çalışacağından dolayı program da yavaş çalışır. Yani eğer programların hızlı çalışmasını istiyorsanız veya aynı anda birden çok programın başarılı bir şekilde çalışmasını istiyorsanız RAM belleğin yeterli veya yüksek olması gerekmektedir.

İşlemci Nasıl Çalışır?

Önceki bölümde öğrendiğimiz gibi, işlemci kendisine gelen veri ve komutları alır, gerekli işlemleri yapar ve işlem sonuçlarını yazıcı veya ekran gibi çıktı aygıtlarında bizlere gösterir.

Bir işlemcinin bir işi yapması veya bir komutu çalıştırması dört aşama ile açıklanır. Bunlar:

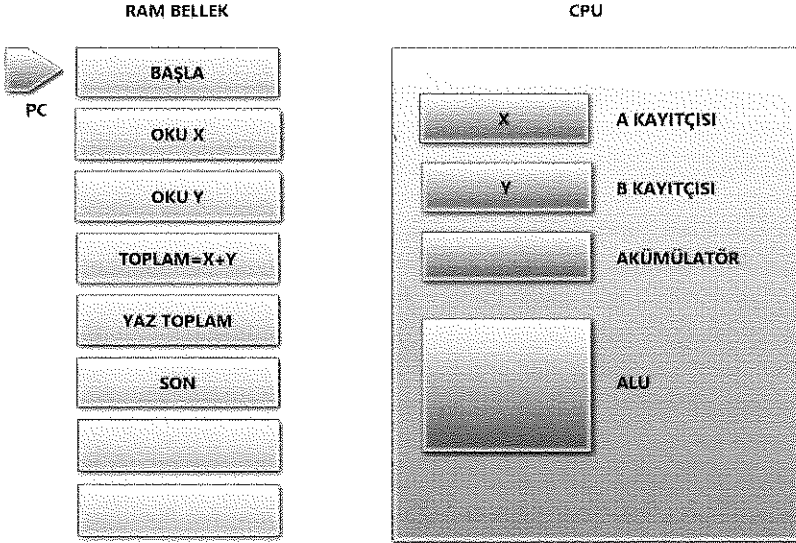
- Fetch (Komut Al)

- Decode (Komutu Çöz)
- Execute (Komutu Çalıştır)
- Store (Varsa Sonucu Kaydet)

Bu aşamalar bir programcı için oldukça önemlidir; çünkü her şey bu aşamalarda meydana gelir. Eğer bu aşamaları iyi bilerseniz daha iyi program yazarsınız. Bu nedenle sizlere bu aşamaları çokook basit bir şekilde ve mümkün olduğunca kafa karıştırmadan anlatmaya çalışacağım.

Fetch (Komut Al)

Yazılan programlar bilgisayara yüklenir ve daha sonra da belli bir görevi gerçekleştirmek için çalıştırılır. Çalıştırılan programların RAM belleğe taşındığını öğrenmiştik. İki sayının toplamını bulan bir programı yazdığımızı ve çalıştırdığımızı düşünelim. Bu program RAM bellekte aşağıda görüldüğü gibi alt alta komutlar şeklinde görünecektir. Normalde RAM bellekte kodlar makine dili olarak bulunur ama ben anlaşılması kolay olsun diye pseudo kod olarak göstereceğim. Bahsi geçen işlemci de benim kafadan uydurduğum hayali bir işlemcidir. Ama diğer işlemciler de aynı mantıkla çalışır.



Şekil 2.5 RAM bellekteki komutlar ve CPU

PC (Program Counter) çalıştırılması gereken komutu gösterir. **CPU**, çalıştırılması gereken komutu öğrenmek için **PC**'a bakar. Daha sonra **PC**'in gösterdiği yere (RAM bellek adresine) gidip oradaki komutu alır, yani **fetch** eder.

Bir program ilk çalışmaya başladığında **PC** ilk komutu gösterir. İlk komut **fetch** edildikten sonra **PC** bir sonraki komutu göstermeye başlar; yani bir alt komuta kayar. Böylece sonraki adımda çalıştırılacak komut belirlenmiş oldu.

Decode (Komutu Çöz)

Komut **fetch** edildikten sonra sıra bu komutla ne yapılacağına karar vermeye gelir. Şekil 2.5'te **PC** (Program Counter) tarafından gösterilen komut alınır ve ne anlama geldiği çözülür. Buradaki komutumuz programın başlatılmasını sağlayan **BAŞLA** komutudur. **BAŞLA** komutunu alan ve bu komutu çözerek hangi işin yapılacağına karar veren **CPU** bir sonraki aşamaya geçer.

Execute (Çalıştır)

Fetch edilerek **RAM** bellekten alınan komut, **decode** edilmiş ve ne iş yapılacağına karar verilmiştir. Bu aşamada artık gerekli işlem yapılacak yani komut çalıştırılacaktır. Buradaki komut **BAŞLA** komutudur ve programın çalışmasını sağlar.

Store (Kaydet)

Bu aşamada, eğer **execute** işlemi sonunda bir değer ortaya çıkmışsa **RAM** belleğe kaydedilir. **BAŞLA** komutu sadece programın başlamasını sağlar ve herhangi bir değer veya sonuç ortaya çıkarmaz. Bu nedenle burada **store** işlemi yapılmaz. Ama daha sonraki aşamalarda göreceğiniz gibi, eğer **execute** aşamasında bir değer (örneğin iki sayının toplam sonucu) ortaya çıkarsa **RAM** belleğe kaydedilir.

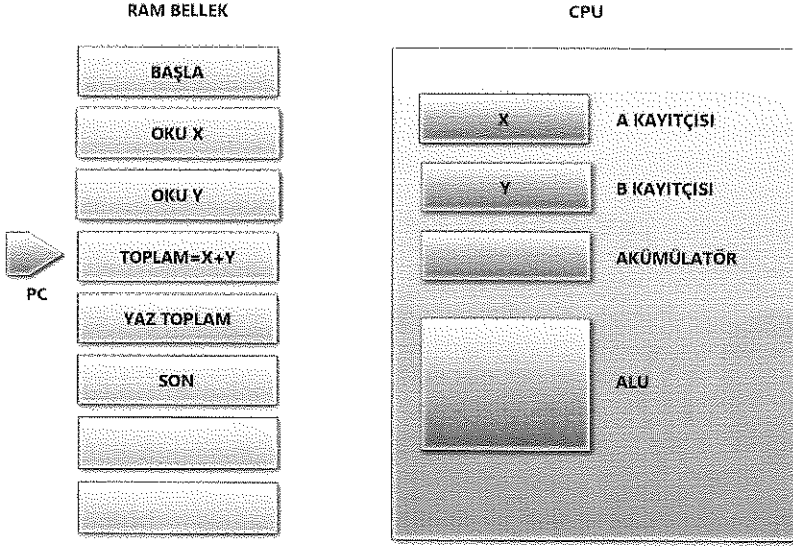
Store işleminden sonra **CPU** bir sonraki komut için **PC**'a bakar. Burada **fetch** edilmesi gereken komutu görür ve alır. **Fetch** işleminden sonra **PC** bir sonraki komutu gösterir, yani **PC** bir alta kayar. Daha sonra alınan komut **decode** edilir ve ne iş yapılması gerektiğine karar verilir. **OKU X** komutu dışarıdan bir sayı alınmasını ve **X** değişkeni içinde saklanmasını emreder. (Değişkenler dışarıdan alınan verileri saklamaya yarayan belleklerdir.) Komut **execute** edilir, dışarıdan alınan değer **A** kayıtçısına yerleştirilir. (Kayıtçılar, **CPU** içinde bulunan ve dışarıdan alınan verileri geçici olarak saklamaya yarayan belleklerdir.) **Store** işlemi gerekmediğinden bir sonraki komuta geçilir ve **OKU Y** komutu için de aynı şeyler yapılır. Farklı olarak alınan veri **B** kayıtçısına yerleştirilir.

Yukarıdaki işlemler sonucunda **RAM** bellekten alınan **X** ve **Y** sayıları **CPU** içindeki kayıtçılara taşınmış, **RAM** bellek ve işlemci aşağıda Şekil 2.6'daki gibi olmuştur.

Sıradaki komut Şekil 2.6'dan da görüldüğü gibi **TOPLAM=X+Y** komutudur. Bu komut ile **X** ve **Y** toplanır (Bu işlem **ALU** ile yapılır.) ve işlem sonucu **Akümülatör**'de

saklanır. (Akümülatör işlemlerin yapıldığı ve sonuçların saklandığı kayıttır.) Bu işlem sonucunda bir değer elde edildiğinden, **execute** işleminden sonra **store** işlemi de yapılır ve bulunan sonuç RAM belleğe kaydedilir.

TOPLAM=X+Y komutu çalıştıktan sonra onun altında buluna **YAZ TOPLAM** komutu çalıştırılır ve bulunan işlem sonucu ekrana veya başka bir cihaza yazdırılır. Daha sonraki adımda da program bitirilir.



Şekil 2.6 İşlemci, PC ve RAM bellek durumu.

CPU ve RAM İlişkisi

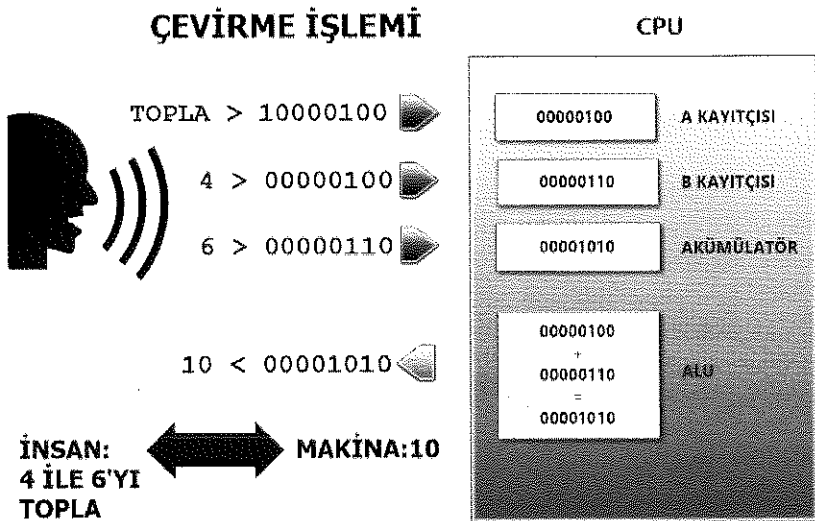
Yukarıdaki örneklerden ve şekillerden de anlaşılacağı gibi CPU ve RAM sürekli bir iletişim halindedir. Yani RAM, CPU'nun çalışma belleğidir; CPU programları RAM belleğe yükler, komutları oradan alır, işlem sonuçlarını da oraya yazar. Bu nedenle RAM bellek ve RAM belleğin yapısı çok iyi bilinmelidir çünkü RAM belleğin yapısı bizim değişken ve veri tiplerinin yapısını ortaya çıkaracaktır.

Program yazarken dışarıdan alınan verileri veya işlem sonuçlarını saklamak için değişkenler kullanırız. Bu değişkenlerin bazıları **BIT** tipindedir, yani sadece 0 (sıfır) veya 1 (bir) gibi değerleri saklarlar. Bazıları ise **BYTE** tipindedir ve 0-255 arası değerleri saklarlar. **BIT**, **BYTE**, **INTEGER** gibi veri tipleri RAM belleğin fiziksel yapısından kaynaklanır. Bu nedenle RAM belleğin yapısı iyi bilinmelidir.

Onlu ve İkili (Dijital) Sayı Sistemi

İnsanlar günlük hayatlarında 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 rakamlarından oluşan Onlu Sayı Sistemi'ni kullanırlar. Bilgisayarlar ve diğer bütün elektronik cihazlar ise 0 ve 1 rakamlarından oluşan İkili Sayı Sistemi'ni kullanırlar ve bütün işlemleri bu sayı sistemi ile yaparlar. Bu nedenle eğer bilgisayara bir sayı girecekseniz bunun ikili sayı sistemine çevrilmesi gerekmektedir. Gene eğer bilgisayardan bir değer okuyacaksanız bunun insanların anlayacağı onlu sayı sistemine çevrilmesi gerekmektedir.

Aşağıdaki şekilde de görüldüğü gibi, insanın makineye verdiği komutlar veya veriler ikili sayı sistemine çevrilmektedir. Makine içinde tüm işlemler ikili sayı sistemi ile yapılmaktadır. En son, elde edilen sonuçlar insanın anlayacağı sistem olan onlu sayı sistemine çevrilmekte ve dışarı öyle gönderilmektedir.



Şekil 2.7 İnsan ve makine etkileşimi

İyi bir programcı olabilmek için ikili, onlu hatta onaltılı sayı sistemini ve bunların birbirine çevrilmesini iyi bilmek gerekmektedir. İkili (dijital) sayı sistemini iyi bilmeyen programcılar maalesef bazı algoritmalarda sorun yaşamaktadır. İyi bir programlama dili öğrenmek için önce bilgisayarın dilini öğrenmek gerekmektedir. Bunun için sizlere, çok fazla teoriye girmeden ama temel düzeyde onlu ve ikili sayı sistemlerinden ve birbirlerine çevrilmesinden bahsedeceğim.

Onlu Sayı Sistemi

Günlük hayatta kullandığımız sayı sistemi onlu sayı sistemidir. 0 – 9 arası rakamlardan oluşur. Basamak sayısı arttıkça sola doğru olan rakamların basamak değeri 10

sayısının üsleri olarak artar. Burada herhangi bir sayının sıfırıncı kuvvetinin 1 olduğunu hatırlatmakta fayda var. Yani:

$$10^0=1$$

$$10^1=10 \text{ (sayının kendisi)}$$

$$10^2=100 \text{ (sayının karesi)}$$

$$10^3=1000 \text{ (sayının küpü)}$$

$$10^4=10000 \text{ (sayının 4. kuvveti)}$$

Eğer 1453 sayısını yukarıdaki mantıkla gösterecek olursak aşağıdaki açılımı elde ederiz:

$$1453=1 \times 10^3 + 4 \times 10^2 + 5 \times 10^1 + 3 \times 10^0$$

Onlu sayıları ikili sayıya dönüştürmek için sayı sürekli 2'ye bölünür. Son bölüneni ve kalan kısımları tersten yazdığınızda onlu sayının ikili karşılığını buluruz.

Örnek: 19 sayısının ikili karşılığını bulalım:

$$19/2=9 \text{ Kalan}=(1)$$

$$9/2=4 \text{ Kalan}=(1)$$

$$4/2=2 \text{ Kalan}=(0)$$

$$2/2=(1) \text{ Kalan}=(0)$$

$$(19)_{10}=(10011)_2$$

Örnek: 61 sayısının ikili karşılığını bulalım:

$$61/2=30 \text{ Kalan}=(1)$$

$$30/2=15 \text{ Kalan}=(0)$$

$$15/2=7 \text{ Kalan}=(1)$$

$$7/2=3 \text{ Kalan}=(1)$$

$$3/2=(1) \text{ Kalan}=(1)$$

$$(61)_{10}=(111101)_2$$

İkili Sayı Sistemi

Elektronik cihazların, bilgisayarların, cep telefonlarının vb. cihazların kullandığı sayı sistemidir. 0 ve 1 rakamlarından oluşur. Bu rakamlara BIT (Binary Digit) denir. 0 ve 1 değerleri çok değişik şekillerde karşımıza çıkabilir:

$$0 = \text{FALSE} = \text{OFF} = \text{PASİF} = \text{LOW}$$

$$1 = \text{TRUE} = \text{ON} = \text{AKTİF} = \text{HIGH}$$

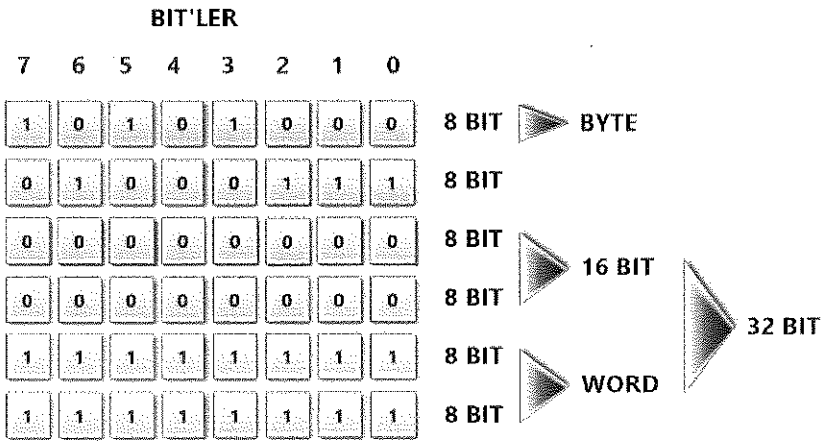
Onluk sayı sisteminde olduğu gibi her basamak değeri 2 sayısının üsleri olarak artar. İkili sayılar aşağıdaki gibi onluk sayı sistemine çevrilir:

$$(0101)_2 = (0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0)_{10} = 0 + 4 + 0 + 1 = 5$$

$$(10101)_2 = (1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0)_{10} = 21$$

RAM Bellek Yapısı

Basitçe söylemek gerekirse RAM bellek yan yana sekiz gözden oluşan ve üst üste koyulan raflar gibidir. Bu raftaki her bir göze 0 (sıfır) veya 1 (bir) gibi ikili sayılar yerleştirilir.



Şekil 2.8 RAM bellek yapısı

1 Byte'taki tüm değerler 0 (00000000) olursa onluk değeri 0, 1 Byte'taki tüm değerler 1 (11111111) olursa onluk değeri 255 olur.

1 Word'teki tüm değerler 0 (00000000 00000000) olursa onluk değeri 0, 1 Word'teki tüm değerler 1 (11111111 11111111) olursa onluk değeri 65.535 olur.

Eğer 65.535'ten (16 bit) daha büyük değerlerle çalışılacaksa bu değerler için RAM bellekte 4 byte (32 bit) yer ayırmak gerekir. Bu da 0-4.294.967.295 arasındaki değerleri kapsar.

RAM bellek yapısı özellikle değişkenler ve veri tipleri için önemlidir. Belleğe kaydedilen farklı türdeki veriler farklı miktarda alan kaplar. Aşağıda değişik örnekler bulabilirsiniz:

- Bay/bayan bilgisi 0/1 olarak kaydedilecekse her kayıt RAM bellekte 1 bit yer kaplar.
- Evli/bekar bilgisi 0/1 olarak kaydedilecekse her kayıt RAM bellekte 1 bit yer kaplar.
- 0-255 arasındaki değerler RAM bellekte 1 byte yani 8 bit yer kaplar; Örneğin, öğrencilerin 0-100 arasında notu girilecekse her not RAM bellekte 1 byte yer kaplar.
- 0-65.535 arası değerler RAM bellekte 2 byte yani 16 bit yer kaplar. Örneğin, 0-5.000 kg arasında değişen yük konteynır ağırlığını kaydetmek için RAM bellekte 2 byte yer ayırmak gerekir.
- 0-4 Milyar arası değerler RAM bellekte 4 byte yani 32 bit yer kaplar. Örneğin cirosu 1 ile 4 milyar arasında değişen bir şirketin bilgisini kaydetmek için RAM bellekte 4 byte (32 bit) yer ayırmak gerekir.

3

Temel Algoritma Bilgisi

- Algoritma Nedir?
- Algoritma Nasıl Olmalıdır?
- Algoritmanın Faydaları
- Bazı Terim ve Kavramlar
- Açıklamalı Algoritma Örnekleri

Bu bölümde size temel bazı algoritmaları fazla ayrıntıya girmeden akış diyagramları veya pseudo kod ile anlatmaya çalışacağım.

Bu bölümdeki amacımız hiç kod yazmadan akış diyagramı ve/veya pseudo kod ile algoritma oluşturmak ve programlama mantığını anlamaktır. Fakat asıl konuya başlamadan önce aşağıdaki bazı konuları açıklamak istiyorum.

Algoritma Nedir?

Biliyorum, algoritmanın ne olduğuna daha önce de değinmiştik. Ama yeri gelmişken algoritmayı bir daha tanımlamakta fayda var: Algoritma, bir sorunu (problemi) çözmek için kullanılan yöntemdir. Algoritmayı düz yazı ile (pseudo kod) oluşturabileceğiniz gibi akış diyagramlarını da kullanabiliriz.

Algoritma Nasıl Olmalıdır?

- Büyük ve karmaşık işlemler küçük ve daha basit, kolay anlaşılabilir parçalara bölünmelidir. Böl ve yönet mantığı...
- Her adım son derece belirleyici olmalıdır; algoritmanın adımlarını oluştururken belirsizlikler olmamalı, her şey açık açık yazılmalıdır.
- Tüm işlemler yapılma sırasına göre baştan sona doğru gösterilmelidir.
- Karşılaşılabilecek tüm ihtimaller düşünülerek hazırlanmalıdır.
- Algoritma sonlu olmalıdır. Yani, belirli bir adım sonunda algoritma sona ermelidir.

Algoritmanın Faydaları

Programı yazmadan önce algoritmasını oluşturmak ve algoritmayı test etmek çok önemli bir alışkanlıktır ve bazı avantajlar sağlar. Algoritmanın faydaları aşağıdaki gibidir:

- Programın kodlanmasını kolaylaştırır; algoritmaya bakarak daha kolay kod yazabilirsiniz.
- Algoritması belli olan bir programı sadece siz değil, başkaları da yazabilir. Böylece takım çalışması sağlanmış olur ve kodlama daha hızlı ilerler.
- Özellikle mantıksal hata yapma ihtimali azalır; eğer algoritmayı daha önce yapmış ve doğruluğunu test etmişseniz programın hatalı sonuçlar vermesi zorlaşır.
- Algoritmayı hazırlarken konu hakkındaki uzmanlığınızı da test etmiş olursunuz. Eğer konu hakkında yeterli bilgiye sahip değilseniz bilgi eksikliğinizi gidermeli ya da dışarıdan destek almalısınız.

Bazı Terim ve Kavramlar

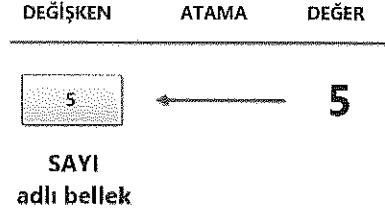
Algoritmanın ve bundan sonra yapacağımız çalışmaların daha iyi anlaşılabilmesi için bazı temel bilgilerin bilinmesinde fayda var. Aksi takdirde bazı konular havada kalacak ve algoritma tam olarak anlaşılamayacaktır. Bu nedenle bu bölümde programcılıkta ve algoritmada kullanılan bazı terim ve kavramları anlatmaya çalışacağız. Değişken, atama, sorgu, dallanma gibi bazı konuları temel düzeyde öğrenip algoritmaya ondan sonra başlayacağız. Bu kavramlar daha sonra yeri geldiğinde daha ayrıntılı anlatılacaktır.

Değişken

Değişkenler, dışarıdan alınan veya bir işlem sonucunda elde edilen verilerin saklandığı belleklerdir. Bu belleklere belli bir isim verilir ve daha sonra bu isimle bellek içinde bulunan değer tekrar tekrar kullanılır.

Örneğin, bir sayının karesini hesaplayan bir program yapacaksınız. Önce bu sayıyı alıp **SAYI** veya belli bir isme sahip bir değişkene aktarmak/atamak zorundasınız.

Eğer 5 değerini dışarıdan alır ve **SAYI** isimli bir belleğe (**değişkene**) atarsak aşağıdaki gibi bir sonuç elde ederiz; burada dışarıdan alınan değer artık **SAYI** isimli belleğin içindedir.



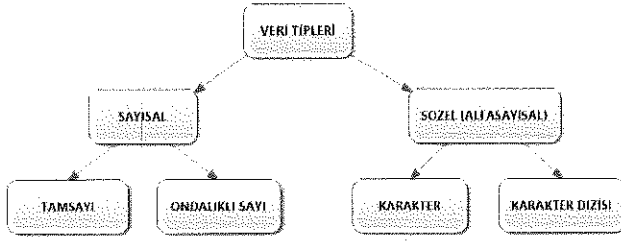
Şekil 3.1 Değişken ve atama

Bundan sonraki işlemlerde **SAYI** değişkenini kullanarak alınan değer üzerinde aşağıdaki gibi işlemler yapabiliriz:

SAYI+5	(SAYI değerine 5 ekle)
2+SAYI	(2 ile SAYI değişkeninin içindeki değeri topla)
5*SAYI	(SAYI değişkeninin içindeki değeri 5 ile çarp)
2*SAYI+1	(SAYI değişkenini 2 ile çarp ve 1 ekle)

Değişkenler farklı veri tiplerinde olabilir. Örneğin bazı değişkenler 3, 20, 1453 gibi **tamsayı** (integer) tipinde veriler barındırırken bazı değişkenler 2.14, 1.4545 gibi **ondalıklı** (float, double) değerler barındırabilir. Bazı değişkenler ise sayı veya rakam saklamayıp 'a', 'k', '@' gibi **karakterler** (char) saklayabilir. Eğer karakterlerin bir araya gelmesiyle oluşan "ali" veya "bilimsel" gibi değerler varsa, bu tür değerleri saklayan değişkenlere **sözel** (string) değişken denir.

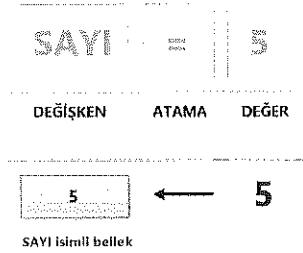
Değişkenler tanımlanırken veri tipi belirtilir ve sadece ilgili tipte değerleri saklayabilir. Örneğin tamsayı olarak tanımlanan bir değişkende ondalıklı sayı veya metin tipinde veriler saklayamazsınız. Genel olarak programcılıkta kullanılan veri tipleri aşağıdaki gibidir:



Şekil 3.2 Değişken veri tipleri

Atama

İyi anlaşılması gereken konulardan bir tanesi de atama işlemidir. Atama işleminde eşitliğin sağındaki ifade soldaki değişkene aktarılır. Aşağıdaki şekle dikkat ederseniz sağdaki **5** değeri atama (=) ifadesi ile soldaki değişkene aktarılmaktadır. Bu işlem sonucunda artık **SAYI** değişkeninin içinde **5** değeri vardır.



Şekil 3.3 Atama işlemi

Programcılıkta (=) eşittir ifadesi atama işlemi için kullanılır ve matematikteki eşittir (=) ile karıştırılmamalıdır. Atama işleminde eşittir ifadesinin sağ tarafındaki işlemler yapılır ve bulunan sonuç soldaki değişkene aktarılır.

değişken=değişken1+değişken2+...

← işlemleri yap ve soldaki değişkene aktar

Aşağıda atama ile ilgili değişik örnekler görebilirsiniz:

- **SAYI = 2** (SAYI değişkeninin içine 2 değerini koy)
- **SAYI = 5** (SAYI değişkeninin içine 5 değeri atandı ve önceki değer olan 2 silindi)
- **SAYI = SAYI+1** (SAYI değişkenini 1 arttır ve sonucu gene sayı değişkeninde sakla. Bu işlem ile 5 olan sayı değişkenine 1 eklendi ve elde edilen 6 değeri soldaki SAYI değişkenine atandı; sonuçta SAYI değişkeninin değeri 1 arttırılmış oldu)

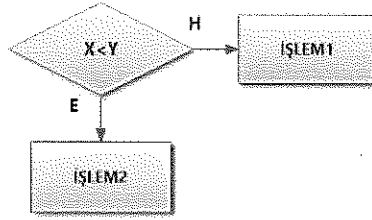
- **TOPLAM=SAYI+5** (SAYI değişkenine 5 ekle ve sonucu TOPLAM değişkeninde sakla. Sonuçta TOPLAM değişkeninin içindeki değer $6+5=11$ olur)
- **TOPLAM=TOPLAM-2** (TOPLAM değişkeninden 2 çıkar ve sonucu gene TOPLAM değişkeninde sakla. Bu durumda TOPLAM değişkeni $11-2=9$ olur)
- **ALAN=PI*R*R** (PI değerini R değişkenin karesi ile çarp ve işlem sonucunu ALAN değişkeninde sakla)
- **CEVRE=2*PI*R** (PI değerini R değişkeni ve 2 ile çarp, sonucu CEVRE değişkenine ata)

Sorgu

Program içine dışarıdan alınan değerlerin veya program içinde üretilen bazı değer ve değişkenlerin durumunun kontrol edilmesi işlemine **sorgu** denir. Yapılan sorgu sonucunda duruma göre farklı işlemler yapılabilir. Örneğin, sorgu sonucu olumlu ise toplama, olumsuz ise çıkarma işlemi yapılabilir.

Sorgu komutları akış diyagramlarında baklava içine alınır. Baklava şeklinin yanında genelde sorgu sonucunu gösteren **Evet/Hayır** veya **Doğru/Yanlış** gibi bazı ifadeler vardır. Aşağıdaki şekilde görülen **E** harfi sorgudaki şartın sağlandığını gösteren **Evet** anlamına, **H** harfi ise sorgudaki şartın sağlanmadığını gösteren **Hayır** anlamına gelmektedir.

Aşağıdaki akış diyagramında **X** ile **Y** arasındaki durum sorgulanmaktadır; eğer **X<Y** şartı sağlanırsa (Evet) **İŞLEM2**, sağlanmazsa (Hayır) **İŞLEM1** yapılacaktır.



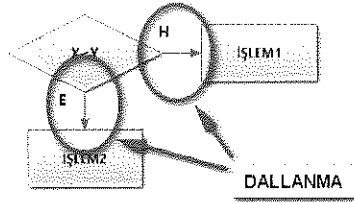
Şekil 3.4 Sorgu işlemi

Dallanma

Sorgu işlemi sonucunda algoritmanın iki veya daha fazla dala ya da yöne ayrılmasına **dallanma** denir.

Aşağıdaki akış diyagramında **X** değişkeninin değeri kontrol edilir, eğer **X** değişkeninin değeri **Y**'den küçükse başka bir işlem, büyükse başka bir işlem yapılır. Yani, baklava içindeki şart sağlanırsa ayrı bir dal, sağlanmazsa ayrı bir dal takip edilir.

Baklava yanındaki **E** evet anlamına gelir ve şartın sağlandığını gösterir. Bu durumda program aşağı dallanır. **H** harfi ise hayır anlamına gelir ve şartın sağlanmadığını belirtir. Bu durumda program sağa doğru dallanır. Doğal olarak programımız farklı dallarda farklı işler yapar. Örneğin şart sağlanırsa $X+Y$ işlemi yapılabilirken, sağlanmazsa $X-Y$ gibi bir işlem yapılabilir.

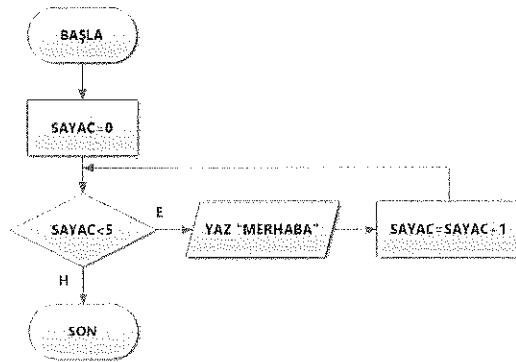


Şekil 3.5 Dallanma

Sayaç

Sayaçlar bir işlemin kaç defa yapıldığını sayan değişkenlerdir. Eğer belirli sayıda işlem yapılacaksa veya bir işlemin kaç defa yapıldığını öğrenmek istiyorsanız **sayaç** kullanmalısınız.

Bir sayacın değeri kontrol edilerek işlem sayısı öğrenilebilir. Örneğin ekrana 5 defa "**MERHABA**" yazdırılacaksa bir sayaç değişkeni tanımlanır ve sayaç 5 oluncaya kadar ekrana "**MERHABA**" yazdırılır.



Şekil 3.6 Sayaç değişkeni ve çalışma mantığı

Yukarıdaki akış diyagramında **SAYAC** isimli değişkenin çalışma mantığı görülmektedir. **SAYAC** değişkeni hayatına 0 (**SAYAC=0**) ile başlar. Sonraki adımda **SAYAC** değişkeninin değeri sorgu ile kontrol edilir. Eğer **SAYAC** değişkeninin değeri 5'ten küçükse program sağa dallanır ve ekrana "**MERHABA**" yazdırılır. Bir sonra-

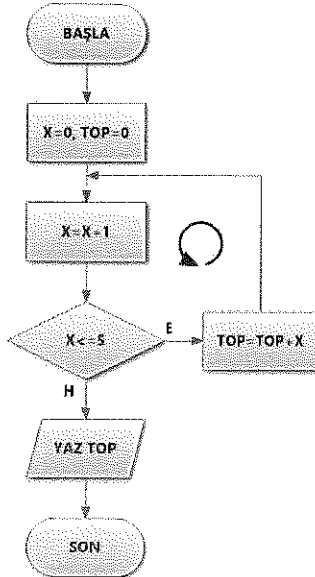
ki aşamada **SAYAC** değişkeninin değeri **1** artırılır. **SAYAC=SAYAC+1** ifadesi, **SAYAC** değişkenini **1** artır ve yeni değeri gene **SAYAC** değişkeninde sakla demektir. **SAYAC** değişkeni ilk aşamada **0** olduğu için ekrana "**MERHABA**" yazdıktan sonra **SAYAC=SAYAC+1** ifadesi ile **1** olur. Daha sonra program gene sorgu ifadesine girer ve **SAYAC** değişkeninin durumuna göre program işleyişine devam eder.

Sayaç değişkenleri **SAYAC=SAYAC+1** ifadesi ile bir birim artabileceği gibi **SAYAC=SAYAC+2** ifadesi ile iki birim artabilir veya **SAYAC=SAYAC-1** ifadesi ile de bir birim azalabilir. Yapacağımız programın ihtiyacına göre değişik formüllere sahip sayaçlar kullanabiliriz.

Döngü

Belli bir şart sağlandığı sürece veya belirli bir şart sağlanana kadar belirli işlemlerin defalarca yapılmasını sağlayan kodlara **döngü** denir. Örneğin **1**'den **5**'e kadar olan sayıların toplamı bulunacaksa burada **5** defa toplama işleminin tekrar edilmesi gerekmektedir. Bunun için de döngüler kullanılmaktadır.

Aşağıdaki akış diyagramında ikinci adımında **X** değişkenine sıfır değeri atanır. Daha sonra **X** değişkeni **1** artırılır. Bir sonraki adımda sorgu komutu ile **X** değişkeninin durumu kontrol edilir. Eğer **X** değişkeni **5** veya daha küçükse algoritma sağa dallanır ve döngüye girer. Döngü sorgudaki şart sağlandığı sürece çalışır. **X** değişkeni **5** olana kadar döngü devam eder ve **X=X+1** ile **TOP=TOP+X** işlemleri yapılır.



Şekil 3.7 Döngü

Döngülerin daha iyi takip edilmesini sağlamak için değişken tablolarını kullanabilirsiniz. Değişken tablolarına döngüdeki değişkenler yazılır ve her döngüde ya da adımda değişken değerleri hesaplanır. Böylece döngünün doğru çalışıp çalışmadığı da kontrol edilmiş olur.

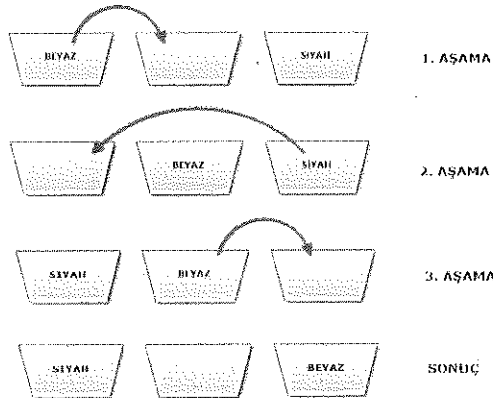
Yukarıdaki döngü için aşağıdaki değişken tablosunu oluşturabiliriz:

X	$X \leq 5$	$TOP = TOP + X$
1	EVET	1
2	EVET	3
3	EVET	6
4	EVET	10
5	EVET	15
6	HAYIR	İŞLEM YOK

Yer Değiştirme Algoritması

Özellikle sıralama algoritmaları başta olmak üzere değişik algoritmalarda yer değiştirme işlemi karşımıza çıkmaktadır. Adından da anlaşılacağı gibi burada amaç iki değişkenin değerlerini karşılıklı olarak yer değiştirmektir. Eğer $A=5$ ve $B=10$ gibi iki değişkeniniz varsa ve bu değişkenlerin değerlerini karşılıklı olarak birbirine aktarmak istiyorsanız **yer değiştirme** yapmak zorundasınız.

Yer değiştirme üç aşamada gerçekleşir; Mantığı grafikte anlatmaya çalışayım. Eğer elinizde iki kova varsa ve kovaların birinde siyah boya diğerinde de beyaz boya varsa ve boyaları karşılıklı olarak diğer kovaya koymak istiyorsanız boş bir kova alıp aşağıdaki işlemleri yapmak zorundasınız.



Şekil 3.8 Yer değiştirme mantığı

1. Birinci aşamada boş bir kova alınır ve beyaz boya boş kovaya dökülür.
2. İkinci aşamada siyah boya artık boş olan eski beyaz boyanın yerine dökülür.
3. Üçüncü ve son aşamada ise önce boş kovaya dökülen beyaz boya eski siyah boyanın yerine koyulur.

Sonuçta boyalar tek bir kova yardımıyla yer değiştirmiş olur. Değişkenlerde de aynı mantık geçerlidir. Yer değiştirme işlemine başlamadan önce yardımcı bir değişken tanımlayacağız ve yer değiştirme mantığını uygulayacağız. Yer değiştirme mantığının pseudo kodları aşağıdaki gibidir.

yardımcı=beyaz (beyaz boyayı yardımcı kovaya dök)

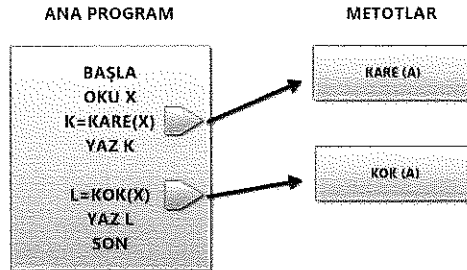
beyaz=siyah (siyah boyayı beyaz boyanın kovalarına dök)

siyah=yardımcı (yardımcıdaki beyaz boyayı siyah boyanın kovalarına dök)

Metot (Fonksiyon)

Belli bir işi yapmaya yarayan (hazır veya kullanıcı tarafından oluşturulan) kodlara **metot** veya **fonksiyon** denir. İster **metot** denilsin ister **fonksiyon**, aynı şey kastedilir.

Metotlar program içinde kullanıcı tarafından oluşturulabilirler. Bu tür metotlara kullanıcı tanımlı metot denir. Kullanıcı bazı kodlar yazar ve bu kodları belli bir isimle kaydeder. Daha sonra program içinde eğer ihtiyacı varsa bu kodları çağırır ve çalıştırır.



Şekil 3.9 Ana program ve fonksiyonlar

Örneğin yukarıdaki şekilde **KARE(A)** metodu görülmektedir. Bu metot kendisine gönderilen değer karesini hesaplar ve ana programa gönderir. Ana programda her kare hesabı yapıldığında bu metot kullanılır.

K=KARE(X) komutuyla, **KARE(A)** metoduna ana programdan **X** değeri gönderilir. Gerekli olan işlemler metot tarafından yapılır ve işlem sonucu ana programdaki **K** değişkenine atanır.

Kullanıcı tanımlı metotların dışında kullandığınız dilin içinde gelen hazır bazı metotlar da bulunmaktadır. Bu metotlar önceden yazılmış ve bazı dosyaların içine kaydedilmiştir. Eğer bu metotları kullanacaksanız ilgili dosyayı program içine alıp sanki sizin tarafınızdan yazılmış gibi kullanabilirsiniz.

Metotlar programcıya zaman ve hız kazandırdığı için çok tercih edilmektedir. Ayrıca sürekli tekrar eden aynı kodları yazmak zorunda olmadığımız için program daha sade ve kısa görünmektedir.

Açıklamalı Algoritma Örnekleri

Bu bölümde çok basitten başlayarak bazı algoritmaları akış diyagramı ve/veya pseudo kod ile oluşturacağız. Öncelikle basit bir algoritma nasıl olmalıdır örneğiyle başlayıp izlenmesi gereken genel yöntemi öğreneceğiz. Daha sonra günlük hayattan bazı algoritmalar yapacağız. En sonunda da işin içine matematiğin de girdiği bazı algoritmaları oluşturacağız.

Lütfen bundan sonraki sayfalarda anlatılan örnekleri en baştan başlayarak inceleyiniz.

Genel Yöntem Algoritması

Bu algorithma sizlere bir algoritma hazırlarken genel olarak kullanmanız gereken yöntem ve aşamaları anlatacağım.

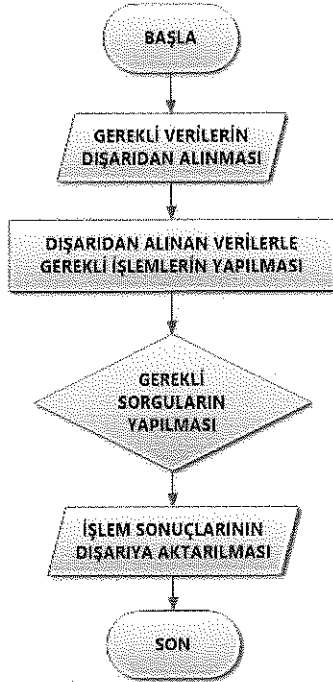
Genelde basit bir algoritma 3 temel aşamadan oluşur:

- **Veri girişi:** Dışarıdan gerekli verilerin alınması.
- **İşlem ve sorgu komutları ile alınan verilerin işlenmesi.**
- **Veri çıkışı:** Elde edilen işlem sonuçlarının ekrana veya kâğıda dökülmesi.

Yukarıdaki işlemleri algorithmaya dökerek olursak algorithmamız aşağıdaki gibi olacaktır:

Adımların Açıklanması:

1. Her algoritma köşeleri yuvarlatılmış bir dikdörtgen içine alınan BAŞLA komutu ile başlar.
2. İkinci adımda dışarıdan alınması gereken veriler alınır; dışarıdan alınan veriler klavye veya fare gibi bir cihazla elde edileceği gibi barkod okuyan bir taramayıcıyla da elde edilebilir.
3. Üçüncü ve dördüncü adımda elde edilen veriler belli formüller kullanarak bir işlemden veya sorgudan geçirilir. Örneğin dışarıdan alınan verilerin toplanması veya bu verilerin belli değerler arasında olup olmadığının sorgulanması yapılabilir.
4. Beşinci adımda elde edilen işlem sonuçları dışarıya aktarılır ve kullanıcıya gösterilir. Dışarıya aktarılan verilerin kullanıcıya gösterilmesini sağlayan cihazlar genelde ekran veya yazıcıdır.
5. Son adımda ise algoritma SON komutu ile bitirilir; unutmayın, önceki konularda bahsedildiği gibi her algoritmanın bir başı ve sonu olmalıdır.

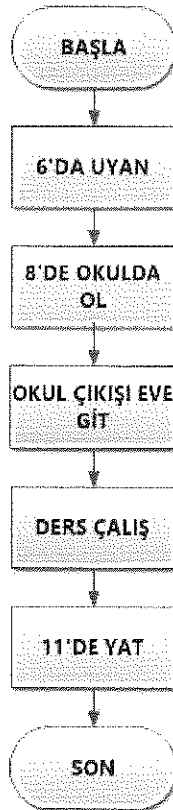
**Şekil 3.10 Genel Yöntem Algoritması**

Günlük Program Algoritması

Kolay ve anlaşılır bir algoritma ile devam edelim; bu algorithmada günlük programımızı kâğıda dökceğiz.

Adımların Açıklanması:

1. Her algoritma **BAŞLA** komutu ile başlar. **BAŞLA** komutu köşeleri yuvarlak dikdörtgen içine yazılır.
2. **6'DA UYAN** komutu bir iş veya eylem olduğu için dikdörtgen içine yazılmalıdır. Her türlü işlem ve atama komutu dikdörtgen içine yazılmalıdır.
3. Bu algorithmadaki diğer adımlar da bir iş veya eylem komutu olduğu için dikdörtgen içine yazılmıştır.
4. Her algoritmanın bir sonu olmalıdır; buradaki **SON** komutu aynı **BAŞLA** komutunda olduğu gibi yuvarlak köşeli dikdörtgen içine alınmış ve algoritma tamamlanmıştır.



Şekil 3.11 Günlük Program Algoritması

Buradaki algoritma akış diyagramını yerine pseudo kod ile yazılsaydı aşağıdaki gibi olacaktı:

```

Başla
6'da uyan
8'de okulda ol
Okul çıkışı eve git
Ders çalış
11'de yat
Son

```

Hava Durumu Algoritması

Evden çıkmadan önce hava durumuna göz atıp ona göre giyinmekte fayda var. Anlaşılması kolay olsun diye günlük hayatımızdan bir algoritma ile devam edeceğiz.



Şekil 3.12 Hava Durumu Algoritması

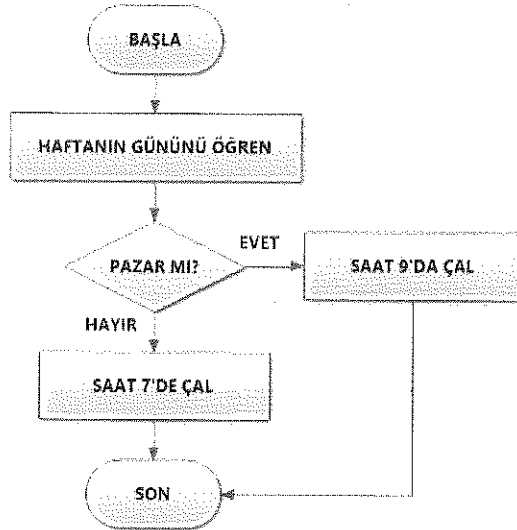
Adımların Açıklanması:

1. Her algoritma **BAŞLA** komutu ile başlar ve **SON** komutu ile biter.
2. Algoritma başladıktan sonra yapılan ilk iş dışarıya bakmaktır. Hatırlayacak olursanız dikdörtgen şekli bir iş/işlem veya atama yapılacağı zaman kullanılıyordu. Bu adımdaki "dışarıya bakma" bir iş veya eylem olduğu için dikdörtgen içine yazılmıştır.

3. Dışarının durumu görüldükten sonra havanın sıcaklığı ile ilgili bir sorgulama yapılmaktadır. Eğer hava sıcaksa t-shirt, soğuksa kazak giyilecek. Dikkat edilirse "sıcak mı?" şeklindeki soru/sorgu ifadesi baklava içine yazılmıştır. Tüm soru veya sorgu ifadeleri baklava içine yazılmalıdır.
4. Hava sıcaklığı sorgulandıktan sonra, duruma göre t-shirt veya kazak giyilmektedir.
5. Son adımda ise algoritma bitirilmektedir.

Saat Alarmı Algoritması

Buradaki örnekte bizi hafta içi farklı hafta sonu farklı saatlerde uyandıran bir saat alarmı algoritması yapacağız. Eğer günlerden Pazar ise alarm saat 9'da, diğer günlerde ise saat 7'de çalsın.



Şekil 3.13 Saat alarmı algoritması

Adımların Açıklaması:

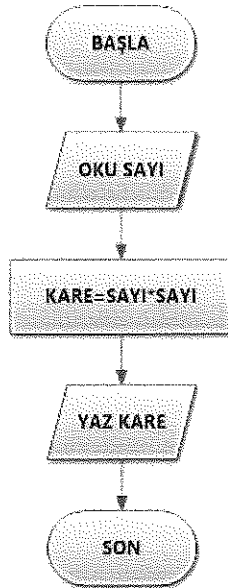
1. İlk adım **BAŞLA** komutudur ve programın başlamasını sağlar.
2. İkinci adımda haftanın günü öğrenilmektedir. Bu bir işlem olduğu için dikdörtgen içine alınmıştır.
3. Üçüncü adımda ise günlerden Pazar olup olmadığı kontrol edilir. Eğer Pazar ise algoritma sağa doğru dallanır ve alarmın saat 9'da çalması sağlanır. Bu bir sorgu veya karar verme komutu olduğu için baklava içine alınmıştır.
4. Dördüncü adımda ise günlerden Pazar olmaması halinde alarmın saat 7'de çalması sağlanır.
5. Son adımda ise **SON** komutu ile algoritma biter.

Girilen Sayının Karesini Bulan Algoritma

Bu örneğimizde biraz daha gerçekçi bir algoritma yapacağız; girilen bir sayının karesini bulup yazdıran bir algoritma oluşturacağız.

Adımların Açıklanması:

1. İlk adımda **BAŞLA** komutu ile programın başlatılması sağlanır.
2. İkinci adımda dışarıdan bir değer alınıp **SAYI** değişkenine aktarılır; bu adımda dışarıdan veri girişi olduğu için bu komut paralelkenar şekli içine alınmıştır.
3. Üçüncü adımda bir işlem/hesaplama yapıldığı için dikdörtgen içine alınmıştır. Burada bir atama işlemi yapılmaktadır; eşitliğin sağındaki **SAYI** kendisiyle çarpılıp karesi bulunmakta ve işlem sonucu soldaki **KARE** isimli değişkene aktarılmaktadır. Yani bulunan sonuç **KARE** değişkeninde saklanmaktadır.
4. Dördüncü adımda **KARE** değişkeni dışarı yazılmaktadır. Hatırlayacağınız gibi giriş/çıkış işlemlerinde komut paralelkenar içine alınmaktadır.
5. **SON** komutu ile algoritma bitirilmektedir.



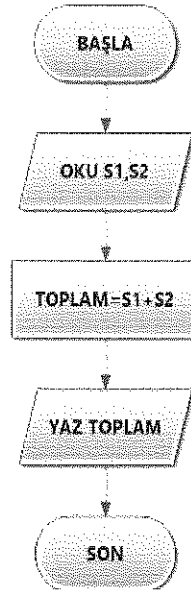
Şekil 3.14 Sayının karesi

Girilen İki Sayının Toplamını Bulan Algoritma

Bu örneğimizde girilen iki sayının toplamını bulan algoritmayı oluşturacağız. Sayıların pozitif veya negatif olmasını kontrol etmeyeceğiz; yani girilen değerler pozitif veya negatif olabilir.

Adımların Açıklaması:

1. İlk adımda **BAŞLA** komutu ile program başlatılır.
2. İkinci adımda dışarıdan iki değer alınır ve **S1** ile **S2** değişkenlerine atanır.
3. Üçüncü adımda **S1** ile **S2** değişkenleri toplanır ve işlem sonucu **TOPLAM** değişkenine atanır.
4. Dördüncü adımda **TOPLAM** değişkeni dışarı aktarılır (Ekran'da veya yazıcıda gösterilir)
5. Beşinci adımda ise algoritma sonlandırılır.



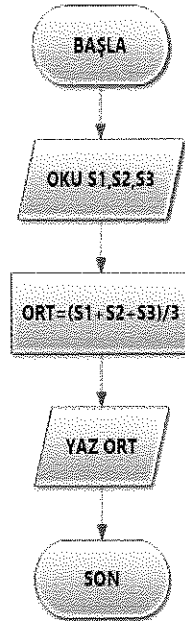
Şekil 3.15 İki sayının toplamı

Üç Sayının Ortalamasını Bulan Algoritma

Bu algorithmada dışarıdan alınan üç sayının ortalamasını bulacağız.

Adımların Açıklanması:

1. İlk adımda **BAŞLA** komutu ile algoritma çalışmaya başlar.
2. İkinci adımda dışarıdan veriler alınır ve **S1, S2, S3** değişkenlerine aktarılır.
3. Üçüncü adımda eşitliğin sağındaki **S1, S2** ve **S3** değerleri toplanarak üçe bölünür ve soldaki **ORT** değişkenine aktarılır.
4. Dördüncü adımda **ORT** değişkeni dışarı aktarılır.
5. Beşinci adımda algoritma sona erer.



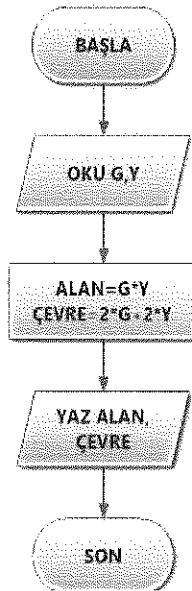
Şekil 3.16 Üç sayının toplamı

Dikdörtgenin Alanını ve Çevresini Bulan Algoritma

Buradaki algorithmada bir dikdörtgenin alanını ve çevresini bulacağız. Dikdörtgenin yüksekliğinin Y , genişliğinin de G olduğunu düşünürsek dikdörtgenin alanı $A=G*Y$, çevresi $Ç=2*(G+Y)$ formülleri ile bulunur. Yukarıdaki formüle dayanarak aşağıdaki algorithmayı çıkarabiliriz:

Adımların açıklanması:

1. İlk adımda algoritma başlatılır.
2. İkinci adımda dışarıdan dikdörtgenin genişliği ve yüksekliği alınıp sırayla G ve Y değişkenlerine aktarılır.
3. Üçüncü adımda alan ve çevre hesaplanır ve sırayla **ALAN** ve **ÇEVRE** değişkenlerine aktarılır.
4. Dördüncü adımda alan ve çevre değişkenleri dışarıya aktarılır.
5. Son adımda algoritma bitirilir.



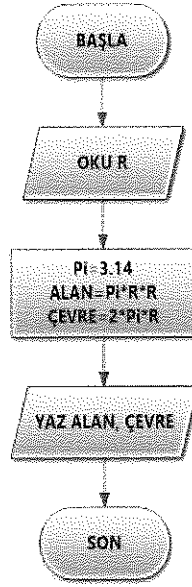
Şekil 3.17 Dikdörtgenin alanı ve çevresi

Dairenin Alanını ve Çevresini Bulan Algoritma

Bu algoritmada dairenin alanını ve çevresini bulacağız. Yarıçapın R olduğunu düşünürsek dairenin alanını $ALAN=PI*R*R$, çevresini de $ÇEVRE=2*PI*R$ formülleri ile bulabiliriz.

Adımların Açıklanması:

1. İlk adımda **BAŞLA** komutu ile algoritma çalıştırılır.
2. İkinci adımda dışarıdan dairenin yarıçapı alınır ve R değişkenine aktarılır.
3. Üçüncü adımda dairenin alanı ve çevresi hesaplanır.
4. Dördüncü adımda dairenin alanı ve çevresi dışarı aktarılır.
5. Son adımda ise algoritma sonlandırılır.



Şekil 3.18 Dairenin alanı ve çevresi

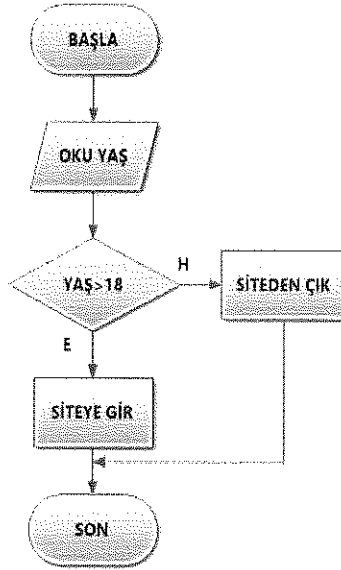
Not: Dikkat ederseniz, belli bir aşamadan sonra artık temel algoritmalar neredeyse birbirine benzer. Sadece parametreler ve formüller değişir. Bu nedenle biraz daha karmaşık ve içinde sorguların da bulunduğu algoritmalara başlayacağız.

Girilen Yaşa Göre İşlem Yapan Algoritma

Bu algorithmada kullanıcının yaşını alacağız. Eğer yaşı 18 veya daha üstü ise işlemlere devam edeceğiz. Aksi takdirde işlem yaptırmayacağız.

Adımların Açıklaması:

1. İlk adımda algoritma başlatılır.
2. İkinci adımda kullanıcının yaşı alınır ve **YAŞ** değişkenine aktarılır
3. Üçüncü adımda **YAŞ** değişkeni sorgulanır; Eğer **YAŞ 18** veya daha büyükse algoritma aşağıya dallanır. **YAŞ** değişkeni 18'den küçükse algoritma sağa dallanır.
4. Dördüncü adımda eğer **YAŞ** değişkeni 18 veya daha büyükse kullanıcı siteye giriş yapar aksi takdirde siteden çıkartılır.
5. Beşinci adımda algoritma sona erer.



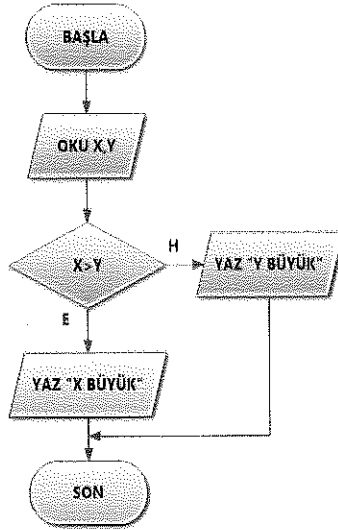
Şekil 3.19 Yaş kontrolü

Girilen İki Sayıdan Büyük Olanı Bulan Algoritma

Bu örneğimizde girilen iki sayıdan büyük olanı bulan algoritmayı oluşturacağız. Girilen iki sayının birbirinden farklı olduğunu varsayıyoruz.

Adımların Açıklaması:

1. İlk adımda program çalışmaya başlar.
2. İkinci adımda dışarıdan iki değer okunur ve bu değerler **X** ile **Y** değişkenlerine atanır.
3. Üçüncü adımda **X>Y** sorgusu yapılır. Eğer **X** büyük ise algoritma aşağı, değilse sağa dallanır.
4. Dördüncü adımda, "**X BÜYÜK**" yazdırılır. Eğer **X>Y** değilse algoritma sağa dallanır ve "**Y BÜYÜK**" yazdırılır.
5. Beşinci adımda algoritma sona erer.



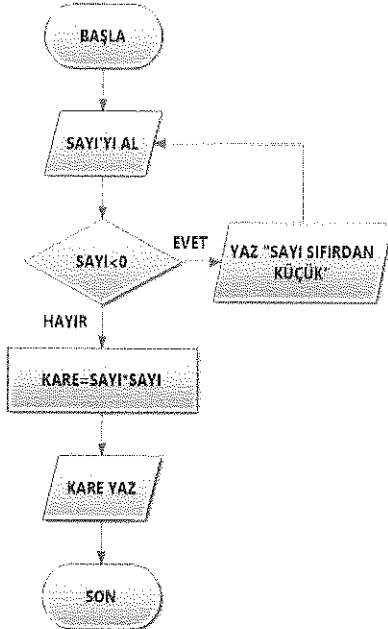
Şekil 3.20 İki sayıdan büyük olanı

Sadece Pozitif Sayıların Karesini Bulan Algoritma

Bu örnek daha önce yapılan kare hesaplama algoritmasına benzemektedir fakat farklı olarak sadece pozitif sayıların karesini almaktadır. Bu örnekte alınan her sayıyı kontrol edeceğiz, eğer sıfırdan büyükse işlem yapacağız, eğer sıfır veya sıfırdan küçükse "**SAYI SIFIRDAN BÜYÜK OLMALI**" diye bir uyarı yazıp yeni sayı girilmesini isteyeceğiz.

Adımları Açıklaması:

1. İlk adımda **BAŞLA** komutu ile program çalışmaya başlar
2. İkinci adımda dışarıdan bir değer alınır ve **SAYI** değişkenine atanır.
3. Üçüncü adımda **SAYI** değişkeni kontrol edilir; eğer içindeki değer sıfırdan büyükse algoritma aşağı doğru dallanır ve **KARE** değişkeni hesaplanır. Eğer **SAYI** değişkeni sıfırdan küçükse algoritma sağa dallanır ve "**SAYI SIFIRDAN KÜÇÜK**" diye uyarı verilir. Uyarıdan sonra yeni bir değer girilmesi istenir.
4. Dördüncü adımda **SAYI** değişkeni kendisiyle çarpılarak karesi bulunur ve işlem sonucu **KARE** değişkenine atanır.
5. Beşinci adımda **KARE** değişkeni dışarı aktarılır
6. Altınca adımda ise **SON** komutu ile algoritma sona erer.



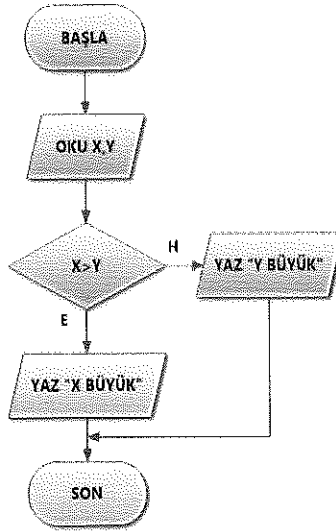
Şekil 3.21 Sadece pozitif sayıların karesi

Girilen İki Sayıdan Büyük Olanı Bulan Algoritma

Bu örneğimizde girilen iki sayıdan büyük olanı bulan algoritmayı oluşturacağız. Girilen iki sayının birbirinden farklı olduğunu varsayıyoruz.

Adımların Açıklaması:

1. İlk adımda program çalışmaya başlar.
2. İkinci adımda dışarıdan iki değer okunur ve bu değerler **X** ile **Y** değişkenlerine atanır.
3. Üçüncü adımda **X>Y** sorgusu yapılır. Eğer **X** büyük ise algoritma aşağı, değilse sağa dallanır.
4. Dördüncü adımda, "**X BÜYÜK**" yazdırılır. Eğer **X>Y** değilse algoritma sağa dallanır ve "**Y BÜYÜK**" yazdırılır.
5. Beşinci adımda algoritma sona erer.



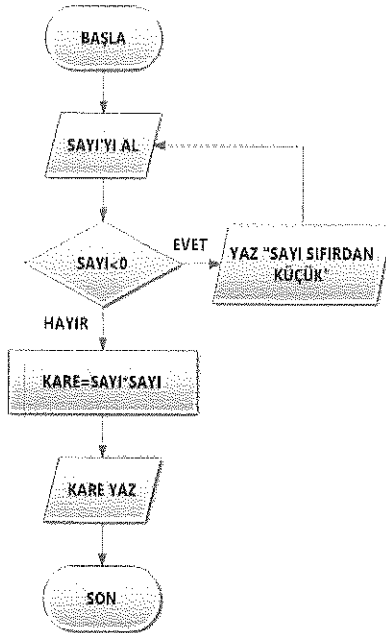
Şekil 3.20 İki sayıdan büyük olanı

Sadece Pozitif Sayıların Karesini Bulan Algoritma

Bu örnek daha önce yapılan kare hesaplama algoritmasına benzemektedir fakat farklı olarak sadece pozitif sayıların karesini almaktadır. Bu örnekte alınan her sayıyı kontrol edeceğiz, eğer sıfırdan büyükse işlem yapacağız, eğer sıfır veya sıfırdan küçükse “**SAYI SIFIRDAN BÜYÜK OLMALI**” diye bir uyarı yazıp yeni sayı girilmesini isteyeceğiz.

Adımları Açıklaması:

1. İlk adımda **BAŞLA** komutu ile program çalışmaya başlar
2. İkinci adımda dışarıdan bir değer alınır ve **SAYI** değişkenine atanır.
3. Üçüncü adımda **SAYI** değişkeni kontrol edilir; eğer içindeki değer sıfırdan büyükse algoritma aşağı doğru dallanır ve **KARE** değişkeni hesaplanır. Eğer **SAYI** değişkeni sıfırdan küçükse algoritma sağa dallanır ve “**SAYI SIFIRDAN KÜÇÜK**” diye uyarı verilir. Uyarıdan sonra yeni bir değer girilmesi istenir.
4. Dördüncü adımda **SAYI** değişkeni kendisiyle çarpılarak karesi bulunur ve işlem sonucu **KARE** değişkenine atanır.
5. Beşinci adımda **KARE** değişkeni dışarı aktarılır
6. Altınca adımda ise **SON** komutu ile algoritma sona erer.



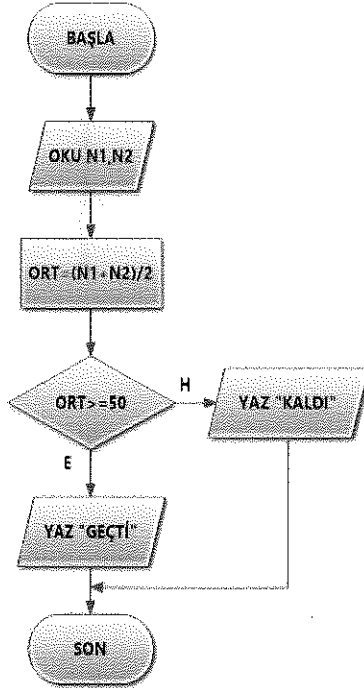
Şekil 3.21 Sadece pozitif sayıların karesi

Geçti-Kaldı Algoritması

Bu algorithmada bir öğrencinin aldığı notlara ve ortalamasına bakarak sınıfta kaldığı veya geçtiği ile ilgili bir örnek yapacağız. Geçme notunun 50 veya daha yüksek olduğunu farz ediyoruz.

Adımların Açıklaması:

1. İlk adımda algoritma başlatılır.
2. İkinci adımda öğrencinin notları alınır ve **N1** ile **N2** değişkenlerine aktarılır.
3. Üçüncü adımda notların ortalaması bulunur ve **ORT** değişkenine atanır.
4. Dördüncü adımda **ORT** değişkeninin durumu kontrol edilir. Eğer **ORT** değişkeninin değeri **50** veya daha yüksek ise algoritma aşağı değilse sağa dallanır.
5. Beşinci adımda **ORT** değişkeninin durumuna göre **GEÇTİ** veya **KALDI** yazdırılır.
6. Son adımda ise algoritma sonlandırılır.



Şekil 3.22 Geçti – kaldı

Not: Algoritmaların ikinci aşamasında da yaptığımız sorgulu algoritmalar birbirine benzemeye başladı; temelde, bir veri alıyoruz, veri üzerinde bir işlem yapıyoruz ve işlem sonucuna uyguladığımız sorguya göre (dallanma) bir başka işlem yapıyoruz. Bundan sonraki örneklerimizde biraz daha zor ve karmaşık algoritmalar yapacağız.

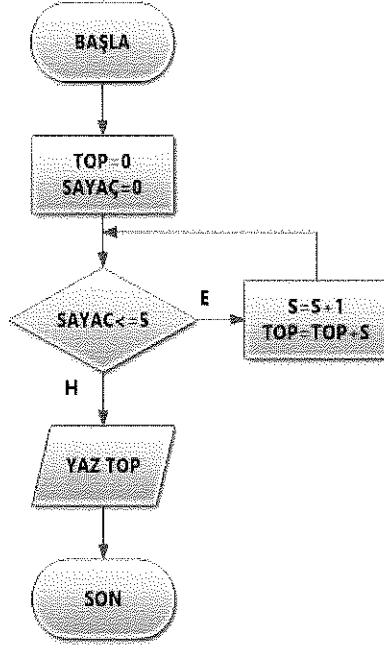
1'den 5'e Kadar Olan Sayıların Toplamı

Buradaki algoritmada 1'den 5'e kadar sayıların toplamını $TOP=1+2+3+4+5$ şeklinde bir formülle bulabiliriz fakat ben kolaya ☺ kaçmayıp bu işi döngü ile yapacağım. Amacım size döngülerin nasıl oluştuğunu anlatmak.

Yapacağım programda **1, 2, 3, 4** ve **5** sayılarını bir sayaç ile elde edeceğim ve elde ettiğim her sayıyı **TOP** değişkeninde toplayacağım. Yani önce **1** değerini **TOP** değişkenine ekleyeceğim, daha sonra **2** değerini, sonra **3, 4** ve **5...** Sonuçta elde edilen tüm sayılar **TOP** değişkeninde toplanmış olacak.

Adımların Açıklanması:

1. Algoritma **BAŞLA** komutu ile başlar.
2. İkinci adımda **TOP** değişkeni ve **SAYAÇ** değişkeni sıfırlanır.
3. Üçüncü adımda **SAYAÇ** değişkeninin değeri kontrol edilir. Eğer **SAYAÇ** değişkeninin değeri **5** veya **5'ten** küçükse **SAYAÇ** değişkeninin değeri **1** artırılır ve önceki **TOP** değeri ile beraber **TOP** değişkeninde toplanır. Yani önceki toplam değerine yeni **SAYAÇ** değeri ilave edilir ve işlem sonucu gene **TOP** değişkeninde saklanır. Bu işlemlerden sonra gene **SAYAÇ** değişkeni kontrol edilir ve **SAYAÇ** değişkeni **5'ten** büyük değilse aynı işlem devam eder.
4. **SAYAÇ** değişkeni **5'ten** büyükse program aşağı dallanır ve **TOP** değişkeni yazdırılır.
5. Son adımda ise algoritma **SON** komutu ile biter.



Şekil 3.23 1'den 5'e sayı toplamı

Bu tür algoritmelerde aşağıdaki gibi bir değişken tablosu yapıp algoritmanın her adımını ya da döngüsünü takip etmekte fayda vardır.

S	ÖNCEKİ TOP	TOP	S<=5?
0	0	0	EVET
1	0	1	EVET
2	1	3	EVET
3	3	6	EVET
4	6	10	EVET
5	10	15	EVET
6	10	İŞLEM YOK	HAYIR

1'den N'e Kadar Olan Sayıların Toplamı

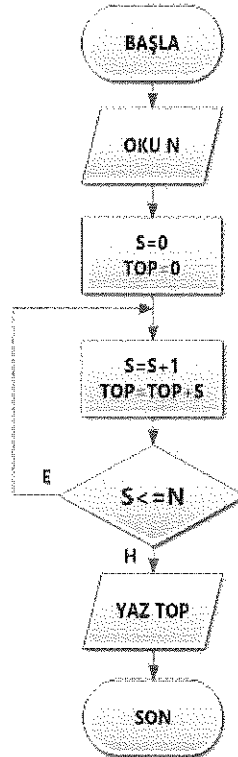
Bu algoritma ve burada kullanılacak mantık programcılıkta karşımıza çok çıkacak. O nedenle bu algoritmanın çok iyi analiz edilmesi ve anlaşılması gerekmektedir.

Bu algoritmada **S** adlı bir sayaç değişkeni kullanacağız. Sayaç **1**'den başlayacak ve girilen **N** değerine kadar artacak. Böylece 1, 2, 3, ..., N değerine kadar olan sayıları elde etmiş olacağız. Sayaç **N** değerine geldiğinde son defa işlem yapılacaktır; sayacın her yeni değerinde **S** değeri **TOP** değişkenine eklenecek.

Adımların Açıklaması:

1. Birinci adımda algoritma başlar.
2. İkinci adımda dışarıdan bir değer alınır ve **N** değişkenine atanır. **1**'den **N** değerine kadar olan sayılar toplanacaktır.
3. Üçüncü adımda **S** adlı sayaç değişkeni ve **TOP** adlı toplam değişkeni sıfırlanır.
4. Dördüncü adımda **S** sayacının değeri **1** artırılır. **S** ile **TOP** değişkeni toplanır ve işlem sonucu soldaki **TOP** değişkenine atanır. Yani her yeni **S** değeri **TOP** değişkenine ilave edilir.
5. Beşinci adımda eğer **S** değişkeni **N** değişkeninden küçükse veya eşitse sola dallanma olur ve **S** değeri **TOP** değişkenine eklenmeye devam eder. Eğer **S** değişkeni girilen **N** değerinden büyükse aşağı dallanma olur.
6. Altıncı adımda **TOP** değişkeni dışarı aktarılır.
7. Son adımda algoritma biter.

Bu tür algoritmalarda aşağıdaki gibi bir değişken tablosu yapıp algoritmanın her adımını takip etmekte fayda vardır. Girilen **N** değerinin 5 olduğunu varsayalım:



Şekil 3.24 1-N arası sayı toplamı

N	S	TOP	S<=N?
5	1	1	EVET
5	2	3	EVET
5	3	6	EVET
5	4	10	EVET
5	5	15	EVET
5	6	İŞLEM YOK	HAYIR

1'den N'e Kadar Olan Çift Sayıların Toplamı

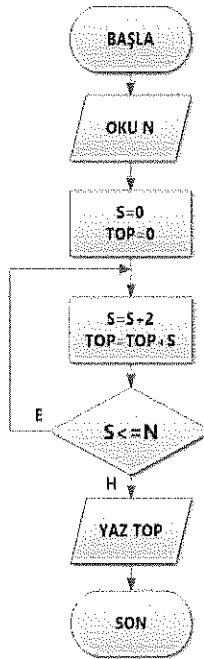
Bu algoritma bir önceki algoritmaya çok benzemektedir. Tek fark çift sayıların toplanmasını sağlamaktır.

Bu algoritmada gene S adlı bir sayaç değişkeni kullanacağız. Sayaç 0'dan başlayacak ve girilen N değerine kadar iki iki artacaktır. Böylece 0, 2, 4, ..., N değerine kadar olan çift sayıları elde etmiş olacağız. Sayaç N değerine geldiğinde son defa işlem yapılacak; sayacın her yeni değerinde S değeri TOP değişkenine eklenecek.

Adımların Açıklaması:

Bu algoritma bir önceki algoritma ile aynıdır. Tek fark sayacın yani **S** değişkeninin **S=S+2** formülünden dolayı 0, 2, 4, 6, ... şeklinde artmasıdır.

Eğer **S=S+2** mantığını ve sayacı anlamakta güçlük çekiyorsanız lütfen sayaç konusunu gözden geçiriniz.



Şekil 3.25 1-N arası çift sayı toplamı

N	S	TOP	S<=N?
9	0	0	EVET
9	2	2	EVET
9	4	6	EVET
9	6	12	EVET
9	8	20	EVET
9	10	İŞLEM YOK	HAYIR

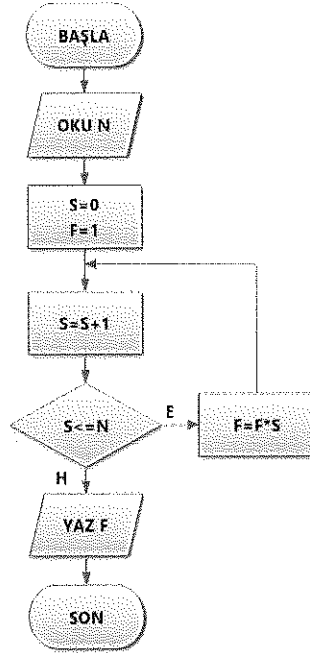
1'den N'e kadar Olan Sayıların Çarpımı (Faktöriyel)

Karşımıza çok çıkan işlemlerden bir tanesi de faktöriyel hesabıdır. Başlıktan da anlayabileceğiniz gibi faktöriyel 1'den N'e kadar olan sayıların çarpımı ile elde edilir.

Faktöriyel hesabında dikkat edilmesi gereken en önemli konu kullanılacak olan ve faktöriyel sonucunu gösteren **F** değişkenidir. Hatırlarsanız daha önceki toplama işlemlerinde **TOP=0** şeklinde bir atama yaparak toplamın başlangıç değerinin sıfır olmasını sağlamıştık. Faktöriyel hesabında ise **F** değişkeninin başlangıç değeri **1** olmalıdır. Eğer sıfır yaparsanız sonuç hep sıfır olur.

Adımların Açıklaması:

1. İlk adımda **BAŞLA** komutu ile algoritma başlatılır.
2. İkinci adımda dışarıdan bir değer alınır ve **N** değişkenine atanır. Alınan bu değer sıfırdan büyük bir değer olduğunu varsayıyoruz.
3. Üçüncü adımda sayaç olarak kullanacağımız **S** değişkenine sıfır, faktöriyel sonucunu atayacağımız **F** değişkenine ise bir değeri atanır.
4. Dördüncü adımda sayaç bir artırılır. Sayaç ile 1'den başlayıp **N**'e kadar olan değerler **1, 2, 3,..., N** şeklinde elde edilecektir.
5. Beşinci adımda sayacın durumu sorgulanır. Eğer sayaç **N** değerinden küçük veya eşitse algoritma sağa aksi takdirde aşağıya dallanır.
6. Altıncı adımda **S ≤ N** sorgusu sağlanmazsa **F** değeri dışarı verilir. Sağlanırsa **F** hesabı devam eder.
7. Son adımda algoritma bitirilir.

**Şekil 3.26 Faktöriyel**

N	S	$S \leq N$	$F = F * S$
5	1	EVET	1
5	2	EVET	2
5	3	EVET	6
5	4	EVET	24
5	5	EVET	120
5	6	HAYIR	İŞLEM YOK

Şu ana kadar yapılan uygulamalarda temel programlama mantığını ve temel programlama kavramlarını öğrenmiş olduk. Çok sık kullanacağımız döngü, dallanma, sorgu gibi yapıları öğrendik ve bu yapıları basit bazı algoritmalarda kullandık.

Eğer tüm algoritmaları ve açıklamaları en baştan okuduysanız akış diyagramlarına da hâkim oldunuz demektir.

Önümüzdeki konularda daha da karmaşık algoritmalara değineceğiz ve bazı yapıları daha da ayrıntılı olarak ele alacağız. Şimdilik temel algoritmaları ve akış diyagramlarını öğrenmiş olduk. Bundan sonraki konularda algoritmaları koda da çevireceğiz.

4

.NET Framework & C#

- .NET Framework Nedir?
- .NET Framework Sürümleri
- .NET Çalışma Mantığı
- Aduzayları
- Nesne Tabanlı Programlama
- C# Nedir
- Visual Studio

Microsoft .NET Framework Nedir?

.NET Framework ve ilgili kavramlar medyada, basında ve kitaplarda oldukça fazla kullanılmaktadır. Ne var ki, maalesef, **.NET Framework** doğru düzgün anlatılmamaktadır. Eğer bu kavramları en azından temel düzeyde bilmezseniz bundan sonraki konularda veya programlama ile ilgili diğer kitaplarda sorun yaşayacağınızı düşünerek **.NET Framework** hakkında temel bazı bilgiler vermeye çalışacağım.

Microsoft **.NET Framework** masaüstü, web, bulut ve mobil uygulamalar geliştirmek için kullanılan bir altyapı yazılımıdır. Kabaca, binlerce hazır fonksiyondan oluşmuş bir yazılımdır diyebiliriz. **.NET Framework** içinde yazdığımız kodları makine diline çevirmeye yarayan derleyiciler, program yazarken bazı işlerin otomatik yapılmasını sağlayan metotlar ve yardım dokümanları bulunmaktadır.

Eğer C#, VB.NET, C++ veya ASP.NET ile uygulama geliştirmek istiyorsanız bilgisayarınızda **.NET Framework** yüklü olmalıdır. Ayrıca, **.NET platformunda** yazılmış bir programı bilgisayarınıza kurup çalıştırmak için de .NET Framework'e ihtiyacınız vardır. Eğer bilgisayarınıza .NET Framework yüklemeyen .NET programları kurarsanız bu programlar çalışmaz hatta belki kurulmasına izin bile verilmez. Neden mi? Çünkü .NET Framework ortamında geliştirilmiş tüm yazılımlar .NET Framework içinde gelen bazı bileşenleri kullanır. Doğal olarak bazı bileşenler eksik olduğu zaman program çalışmaz.

İlk olarak 2000 yılında Beta sürümü ile ortaya atılan Microsoft .NET Framework birçok versiyon çıkararak ve yeni teknolojileri destekleyerek günümüze kadar ulaşmıştır. Aşağıdaki tabloda .NET Framework sürümlerini ve geliştirme aracını görebilirsiniz.

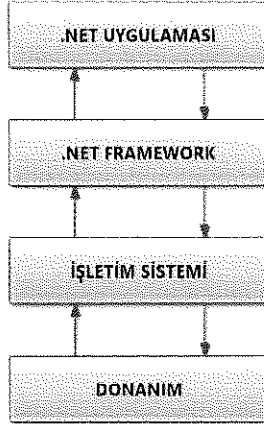
Sürüm	CLR	Yıl	Geliştirme Aracı
1.0	1.0	2002	Visual Studio.NET
1.1	1.1	2003	Visual Studio.NET 2003
2.0	2.0	2005	Visual Studio 2005
3.0	2.0	2006	Expression Blend
3.5	2.0	2007	Visual Studio 2008
4.0	4	2010	Visual Studio 2010
4.5	4	2012	Visual Studio 2012
4.5.1	4	2013	Visual Studio 2013
4.5.2	4	2014	Visual Studio 2015
4.6	4	2015	Visual Studio 2015
4.6.1	4	2015	Visual Studio 2015 Update1
4.6.2	4	2016	Visual Studio 2015 Update2

.NET Framework sürümleri hakkında daha güncel bilgi almak için [http://msdn.microsoft.com/en-us/library/bb822049\(v=vs.110\).aspx](http://msdn.microsoft.com/en-us/library/bb822049(v=vs.110).aspx) adresini ziyaret edebilirsiniz.

.NET Framework, Microsoft tarafından Java'ya rakip olarak üretilmiştir. Zaten çalışma mantığı olarak da Java'ya benzer. Java uygulamalarının çalışması için **JRE / Java Runtime Enviroment** yüklü olmalıdır. Aynı şekilde, .NET uygulamalarının da çalışabilmesi için **.NET Framework** yüklü olmalıdır.

Eğer bilgisayarınıza Visual Studio'nun herhangi bir sürümünü yüklerseniz ilgili .NET Framework de otomatik olarak yüklenir. Böylece bilgisayarınız hem .NET dilleri ile program yazmaya hem de .NET ile üretilmiş programları çalıştırmaya hazır hale gelir.

Kabaca söylemek gerekirse, aşağıdaki şekilden de görebileceğiniz gibi, .NET Framework, .NET programları ile işletim sisteminin iletişim kurmasını sağlar. Eğer bilgisayarınızda .NET Framework yüklü değilse .NET programları çalışmaz, kurulum sırasında .NET Framework'ün yüklenmesini ister.



Şekil 4.1 .NET Framework

.NET Framework Sürümleri

.NET Framework iki farklı sürüme sahiptir; bunlardan birisi az yer kaplayan ve temel bileşenleri kapsayan çekirdek sürümü diğeri ise nispeten daha fazla yer kaplayan SDK sürümü.

Çekirdek Sürümü

Çekirdek sürümü programcılardan ziyade .NET yazılımlarını kurmak ve kullanmak isteyen kullanıcılar için üretilmiştir. Yani yazılımcılar için değil de daha çok dağıtım sonrası uygulamanın çalışacağı işletim sistemleri için uygundur.

.NET uygulamaları için gerekli çalışma zamanı ortamını, web uygulamalarının çalışabilmesi için gerekli olan bir web sunucusunu (IIS) ve desteklenen dillerinin derleyicileri gibi temel bileşenleri sunmaktadır.

SDK Sürümü

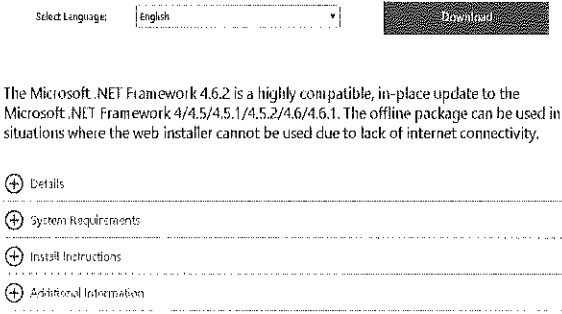
SDK, yani Software Development Kit, adından da belli olduğu gibi daha çok yazılımcılar için üretilmiş bir sürümdür. Çekirdek kurulumuna ek olarak içinde hata ayıklayıcılar, SQL Server Compact sürümü, kılavuzlar ve örnek uygulamaları kapsayan devasa bir dokümantasyona sahiptir.

Eğer programcı iseniz ve C#, C++ ve VB.NET gibi dilleri kullanarak NET Uygulamaları geliştirmek istiyorsanız kesinlikle bu sürümü kuralırsınız.

Nereden Bulurum?

.NET Framework'ü indirmek ve daha ayrıntılı bilgi almak için arama motoruna “.net framework” yazın. Muhtemelen Microsoft'un web sitesine gidecek ve aşağıdaki gibi bir ekranla karşılaşacaksınız. Buradan uygun dili seçip yazılımı indirdikten sonra bilgisayarınıza kurabilirsiniz.

Microsoft .NET Framework 4.6.2 (Offline Installer) for Windows 7 SP1, Windows 8.1, Windows Server 2008 R2 SP1, Windows Server 2012 and Windows Server 2012 R2



Şekil 4.2 :NET Framework sayfası

.NET Framework indirme sayfasını ziyaret ettiğinizde hem .NET Framework'ün son sürümünü indirebilir hem de .NET Framework hakkında çok daha ayrıntılı bilgi bulabilirsiniz. Örneğin “Details” bağlantısına tıklayarak sürüm bilgilerini, “System Requirements” bağlantısına tıklayarak sistem gereksinimlerini öğrenebilirsiniz.

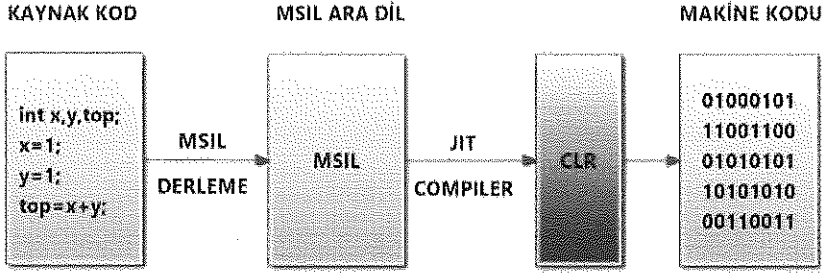
.NET Çalışma Mantığı

Daha önceden de öğrendiğimiz gibi, yazılan kodların bilgisayarda çalışabilmesi için makine koduna çevrilmesi yani **derlenmesi** gerekiyor. Derlenen kodlar hangi işletim sistemi için derlendiyse sadece o işletim sisteminde çalışır. Örneğin yazdığınız kodları Windows işletim sisteminde derlediyseniz sadece o işletim sisteminde çalışır; Mac veya Linux işletim sisteminde çalışmaz. Sonuçta platform bağımlı, sadece belli sistemlerde çalışan bir program ortaya çıkar.

.NET Framework ile geliştirilen programların çalışma mantığı yukarıda anlatılandan biraz farklıdır. **C#** veya **VB.NET** gibi dillerle yazılmış programlar aşağıda Şekil 4.3'te

görüldüğü gibi uygun bir derleyici ile **MSIL** denilen ortak (ara) bir dile çevrilir. Yani kodları ister **C#** ile isterseniz **VB.NET** ile yazın hepsi **MSIL** denilen ortak bir dile çevrilir. Bu nedenle aynı ekipte çalışan bazı programcılar **VB.NET** ile program yazarken diğerleri **C#** veya **C++** ile kod yazabilir.

.NET Framework'te kodların hangi dilde yazıldığıнын pek bir önemi yoktur. Hangi dilde yazarsanız yazın aşağı-yukarı aynı performansı elde edersiniz. Önemli olan algoritma ve kod kalitesidir. Ufak tefek farklar olmakla birlikte bütün programlama dilleri aynı .NET Framework bileşenlerini (class) kullanırlar. Dolayısıyla yeni bir dil öğrenmeye gerek yoktur; hem bildiğiniz ve uzman olduğunuz eski programlama dilini kullanabilir hem de son teknoloji yazılımlar geliştirebilirsiniz. Eski derken VB veya Delphi gibi 90'lı yıllarda (yani .NET Framework'ten önce) çok kullanılan dilleri kast ediyorum.

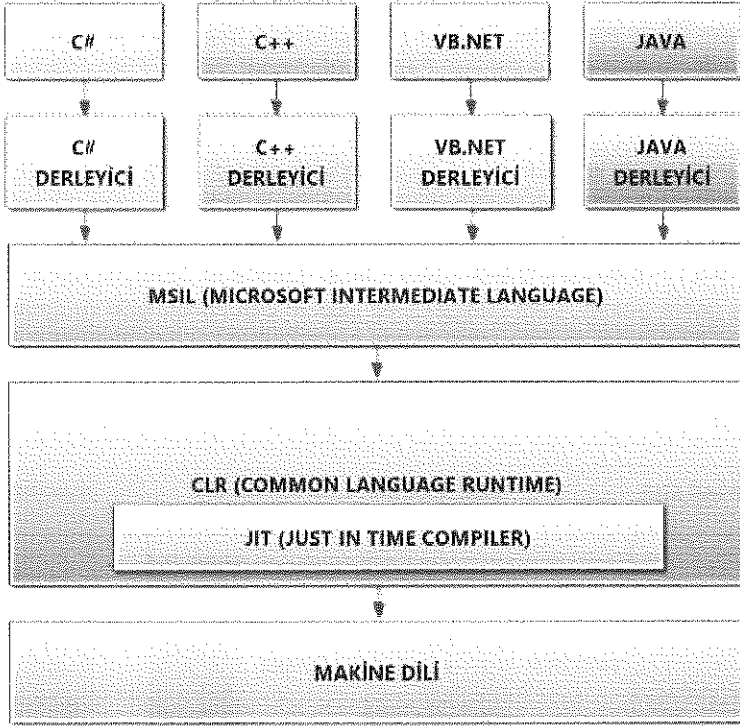


Şekil 4.3 .NET çalışma mantığı

MSIL diline dönüşen kodlar çalıştırıldığı anda **JIT** (Just In Time) compiler kullanılarak **CLR** (Common Language Runtime) diline, yani, bildiğimiz makine koduna çevrilir. Bu işlem sadece bir defa yapılır; kodlar bir defa **CLR** diline çevrildikten sonra başka bir derleme yapılmaz. Program derlendikten sonra işletim sisteminden bağımsız olarak **.NET Framework** olan her sistemde çalışabilir.

Derleme işlemi kodların çalıştığı işlemciye göre yapılır. Örneğin eğer bilgisayarınızda bir Intel i3 işlemci kullanıyorsanız derleme bu işlemciye göre yapılır. Eğer AMD gibi başka bir işlemci kullanıyorsanız derleme işlemi gene bu işlemciye göre yapılır. Dolayısıyla performans da çalışan işlemciye göre değişebilir.

Farklı programlama dilleri **.NET Framework** içinde bulunan ve sadece o dile özel derleyicilerle **MSIL** diline çevrilir. Yani, **.NET Framework** içinde her dile için derleyici yoktur; bu nedenle, eğer .NET uygulamaları geliştirmek istiyorsanız **.NET Framework** tarafından desteklenen bir dil seçmelisiniz.



Şekil 4.4 .NET Framework

Yukarıdaki şekli iyice incelerseniz ne demek istediğimi çok daha iyi anlarsınız. Eğer kodları **C#** ile yazdıysanız kodlar **C# derleyicisi** ile **MSIL** diline çevrilir. Kodları **C++** veya **VB.NET** ile yazdıysanız gene ilgili derleyiciler kullanılarak **MSIL** diline dönüştürülmektedir.

Başta C#, C++, VB.NET olmak üzere Ada, Cobra, Eiffel, Python, Ruby, Javascript gibi diller .NET Framework tarafından desteklenmektedir. .NET Framework tarafından desteklenen tüm dilleri görmek için http://en.wikipedia.org/wiki/List_of_CLI_languages adresini ziyaret edebilirsiniz.

Bir programcının **.NET Framework** hakkında bilmesi gereken bir sürü şey var fakat sanırım şimdilik bu kadar bilgi yeterli. Bizim şu anda bilmemiz gereken şu; eğer **C#** ile program yazıp derlemek istiyorsak bilgisayarımızda **.NET Framework** yüklü olmalı. Eğer daha ayrıntılı bilgiye ihtiyacımız olursa inanın ileriki bölümlerde daha ayrıntılı bilgi gelecektir.

Eğer bilgisayarınıza **Visual Studio** ve **Visual Studio Express** yüklerseniz otomatik olarak **.NET Framework** de yüklenmiş olur.

Nesne Tabanlı Programlama

(Object Oriented Programming)

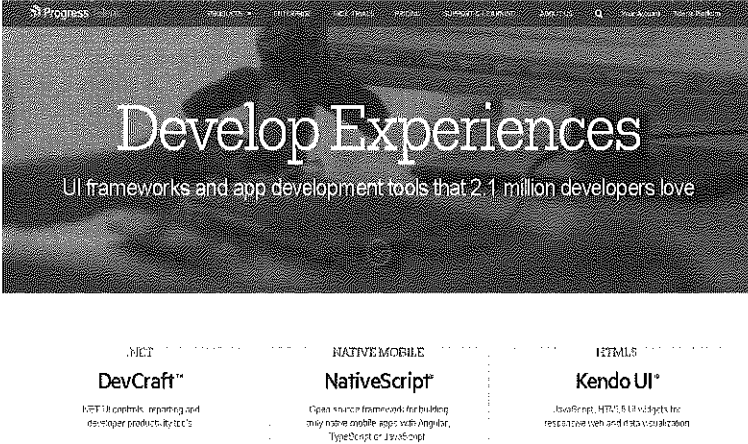
Günümüzde kullanılan C#, VB.NET, C++ ve Java gibi programlama dilleri nesne tabanlıdır. Peki, nedir nesne tabanlı programlama? Nesne tabanlı programlama 1960'lı yıllarda gelişen bir programlama yöntemi veya yaklaşımıdır. Bu programlama yaklaşımında her şey (değişkenler, sayfalar, butonlar) gerçek hayattaki gibi bir nesne kabul edilir. Her nesnenin fiziksel görünümünü belirleyen **özellikleri** ve yaptığı işleri belirleyen **metotları** vardır. Örneğin, kendinizi bir nesne olarak düşünürseniz fiziksel görünümünüzü belirleyen ağırlık, saç rengi, göz rengi, boy gibi özellikleriniz vardır. Ayrıca yürümek, koşmak, konuşmak, ders çalışmak gibi işlevlerimiz yani metodlarımız vardır. Sahip olduğumuz özellik ve metodları kullanarak işlerimizi yaparız. Nesne tabanlı programlamada da nesnelerin sahip olduğu özellikleri ve metodları kullanarak yazılım geliştirebiliriz. Kitabın ilerleyen bölümlerinde nesne tabanlı programlama konusuna daha ayrıntılı değineceğiz.

Class (Sınıf) Nedir?

.NET Framework içinde hazır olarak gelen, daha hızlı ve daha kolay uygulama geliştirmemizi sağlayan kodlara **class/sınıf** denir. Sınıflar tekerleği yeniden icat etmek yerine bazı işlemlerin otomatik yapılmasını ve zaman kazanmamızı sağlarlar. Örneğin, .NET Framework içinde **buton**, **form**, **metin kutusu** gibi birçok **class/sınıf** hazır olarak gelir; biz bu **sınıf** denilen kodlardan formlardaki **buton** veya **metin kutusu** gibi elemanları oluştururuz. Dolayısıyla, form elemanları oluşturmak için kod yazmak zorunda kalmayız. Böylece **sınıf** denilen bu hazır kodlar zaman kazanmamızı ve çok daha hızlı uygulama geliştirmemizi sağlarlar.

Class/sınıf denilen kodlar .NET Framework içinde hazır olarak gelebileceği gibi programcılar tarafından da oluşturulabilir. Hatta .NET Framework'teki form elemanlarına alternatif çok daha fazla işleve sahip olan class'lar özel bazı şirketler tarafından geliştirilmiş ve kimi ücretli kimi ücretsiz olarak satışa sunulmuştur. Bu firmalardan biri oldukça fazla popüler ürüne sahip **telerik** şirketidir. Bu şirketin başta Amerika olmak üzere Hindistan, Almanya, İngiltere ve Avustralya gibi ülkelerde şubeleri ve yazılım departmanları vardır ve birinci amacı .NET Framework sınıfları üretmektir.

Profesyonel hayatta muhtemelen **telerik** veya benzeri firmaların ürünleriyle karşılaşacaksınız. Bu nedenle <http://www.telerik.com/> adresine bir göz atmanızı ve bu firmanın ürünlerini incelemenizi öneririm.



Şekil 4.5 Telerik web sitesi

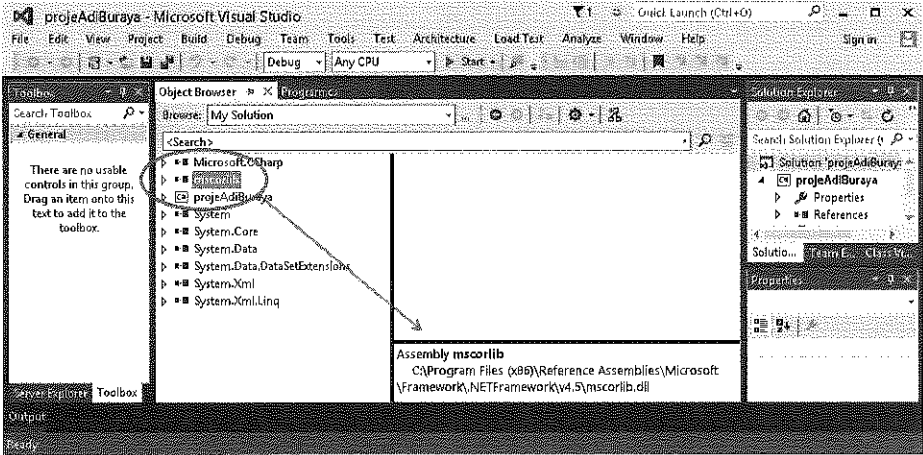
.Net Framework Assembly Nedir?

Bir ya da daha fazla sınıfın içinde kayıtlı olduğu dosyalara **assembly** denir. Assembly dosyaları genelde **.dll** uzantısına sahiptir fakat **.exe** uzantısına da sahip olabilir.

Assembly dosyaları .NET Framework ile beraber yüklenir ve her biri farklı sınıf ya da sınıfları barındırır. Örneğin **microsoft.dll** assembly dosyasının içinde adına “core” denilen ve sıklıkla kullanılan **System.Console** gibi sınıflar bulunur.

Eğer assembly dosyalarını ve içeriğini görmek istiyorsanız Visual Studio’da bir proje oluşturup **VIEWObject Browser** komutunu çalıştırabilirsiniz. Aşağıdaki şekilde görülen **Object Browser** açıldığında sol tarafta sınıfları barındıran assembly dosyaları ve sağ tarafta ise assembly dosyasının bulunduğu adres görülmektedir.

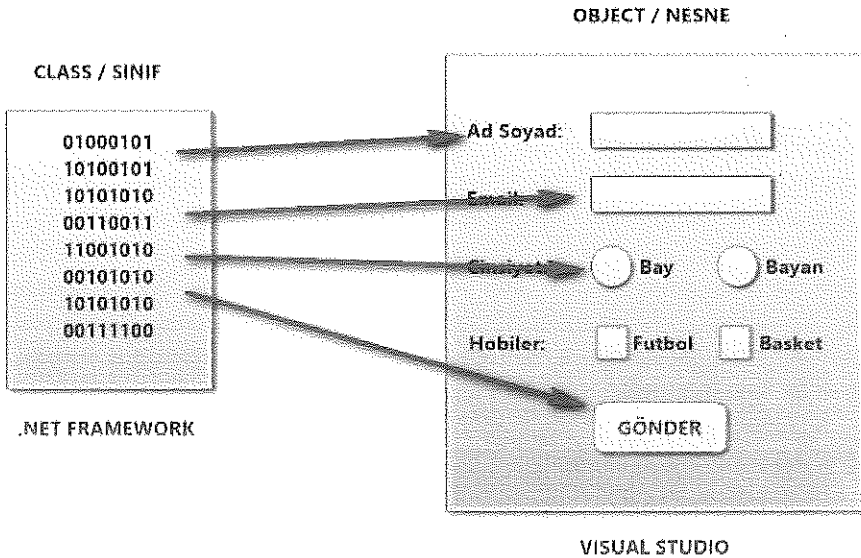
Eğer assembly dosyalarının solundaki küçük üçgene tıklarsanız assembly dosyası içindeki namespace’leri ve her namespace içindeki class’ları görebilirsiniz. Yapı oldukça karmaşık olduğu için şimdilik çok fazla kuralamanızı tavsiye etmem ama .NET ve C# konusunda tecrübe kazandıkça buralar daha fazla ilginizi çekebilir.



Şekil 4.6 Assembly dosyaları ve diskteki konumu

Object (Nesne) Nedir?

Eğer .NET Framework içinde bulunan bir **sınıfı** alıp kullanmaya başlarsanız bir **object** yani **nesne** elde etmiş olursunuz. Örneğin, **metin kutusu** .NET Framework içinde bir **sınıf** olarak, yani bir kod topluluğu olarak bulunur. Eğer bu metin kutusunun bir kopyasını alıp programınızda kullanmaya başlarsanız ondan bir **nesne** oluşturmuş olursunuz.



Şekil 4.7 .Sınıf ve nesne ilişkisi

Sınıf ile **nesne** arasındaki ilişkiyi yukarıdaki şekilden çok daha iyi görebilirsiniz. Bir form oluşturduğunuz ve müşteri ile ilgili bazı verilerin alınacağını düşünün. Müşterinin adını almak için bir metin kutusuna ihtiyacınız var. .NET Framework içinde metin kutusu ile ilgili hazır bir **sınıf** yani kod topluluğu zaten var. Siz .NET Framework içindeki metin kutusu sınıfını alıp onu form üzerine koyarsanız/kopyalarsanız ondan bir nesne oluşturmuş olursunuz. .NET Framework içinde buton, form, onay kutusu gibi binlerce sınıf vardır. Yukarıdaki şeklin sağ tarafında görülen form, metin kutusu, seçenek kutusu, onay kutusu gibi nesneler yukarıda anlatıldığı gibi .NET Framework içinde bulunan sınıflardan elde edilmiştir.

Form elemanları ve diğer birçok sınıf Visual Studio Toolbox'ta kullanıma hazır bir şekilde sizleri bekliyor. Kullanmak istediğiniz herhangi bir sınıfı fareyle Toolbox'tan tutup sürükleyerek formunuza yerleştirirsiniz. Forma yerleştirilen sınıf artık bir nesne olmuştur. Bu nesnenin özellik ve metotlarını kullanarak çok değişik uygulamalar geliştirebilirsiniz.

Örneğin, birkaç metin kutusuna ihtiyacımız varsa fareyle Visual Studio Toolbox'tan **Textbox** adlı sınıfı alır ve formumuza sürükleriz. Böylece **Textbox** sınıfından birkaç **Textbox** nesnesi oluşturmuş oluruz. Aynı şekilde form üzerindeki metin (label), seçenek kutusu (radio), onay kutusu (checkbox), buton (button) ve diğer birçok nesneyi oluşturabiliriz.

Toolbox'ta sadece arayüz oluşturmak için sınıflar olduğunu sanmayın. Veritabanı bağlantısı, veritabanı işlemleri, AJAX, yazdırma ve raporlama gibi çok değişik konular hazır halde onlarca sınıf var. Bu konular bu kitabın kapsamı dışında olduğu için çok ayrıntılı değinmeyeceğiz ama yeri geldiğinde biz de bazı sınıfları kullanacağız.

Bütün sınıflar Toolbox'ta görünmez. Yani her sınıfı sürükleyip formunuza bırakarak kullanamazsınız. Genelde Toolbox'ta en çok kullanılan sınıflar bulunur. Bazı sınıfların ya da sınıflardan türetilen nesnelerin kod yazılarak oluşturulması gerekiyor.

Örneğin **Random** sınıfı rastgele sayı üretmek için kullanılan ve .NET Framework içinde hazır olarak gelen bir kod topluluğudur. Aşağıdaki örneğimizde **Random** sınıfı kullanılarak bir **Random** nesnesi elde edilir. Daha sonra da **Random** nesnesi ile rastgele bir sayı elde edilir.

```
Random rasgele=new Random();
int sayi=rasgele.Next();
Console.Write("rasgele sayı="+sayi);
```

Yukarıdaki örnekten de anlaşılacağı gibi bir sınıftan yeni bir nesne oluşturmak için aşağıdaki kalıbın kullanılması gerekiyor:

```
Sınıf_Adı nesne_adı=new Sınıf_Adı;
```

Şimdilik sadece konuya bir giriş yapmakla yetinelim. Önümüzdeki konularda yeri geldiğinde sınıflardan nesne oluşturup kod içinde kullanmak zorunda kalacağız.

Namespace (Aduzayı) Nedir?

.NET Framework içinde kullanılmaya hazır binlerce sınıf yani hazır kod bulunur. **Namespace**'ler **class** denilen hazır kodları sınıflandırmaya yarayan klasör benzeri yapılardır. Tüm **class**'lar yaptıkları işe göre kategorize edilmiş ve belli **namespace**'lerin içine koyulmuştur.

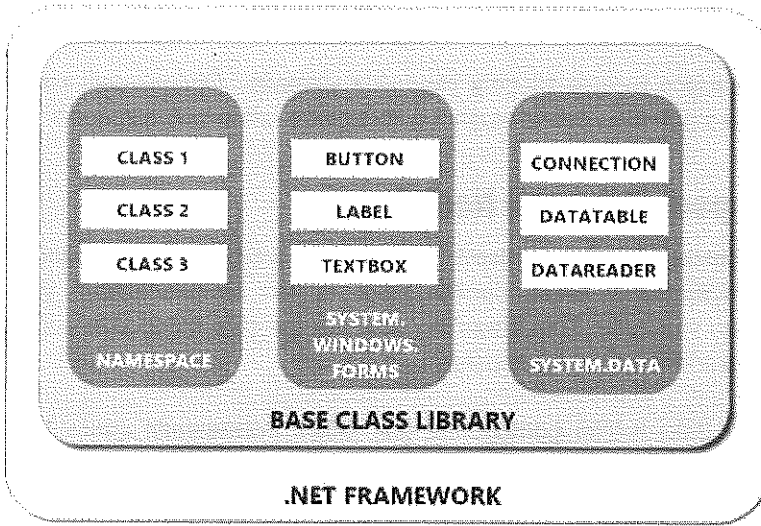
Aşağıdaki Şekil 4.8'de gördüğümüz gibi, **.NET Framework** içinde bulunan tüm kodlar (Base Class Library) **Namespace** denilen yapılara/klasörlere bölünmüştür. **Namespace**'lerin içinde de ilgili **Class**'lar bulunur. Örneğin, **System.Windows.Forms namespace**'inde buton, radyo, onay kutusu gibi Windows form elemanları bulunur.

Bir **namespace** içinde bulunan **class**'ları program içinde kullanmak istiyorsanız **using** komutu ile ilgili **namespace**'i programa eklemelisiniz. Örneğin yukarıda bahsi geçen form elemanlarını projede kullanmak istiyorsanız aşağıdaki kodu yazmalısınız:

```
using System.Windows.Forms;
```

Ekrana veri yazmaya yarayan **Console.Write()** komutunu veya ekrandan veri okumaya yarayan **Console.ReadLine()** komutunu kullanabilmek için aşağıdaki kodu yazmalısınız:

```
using System;
```

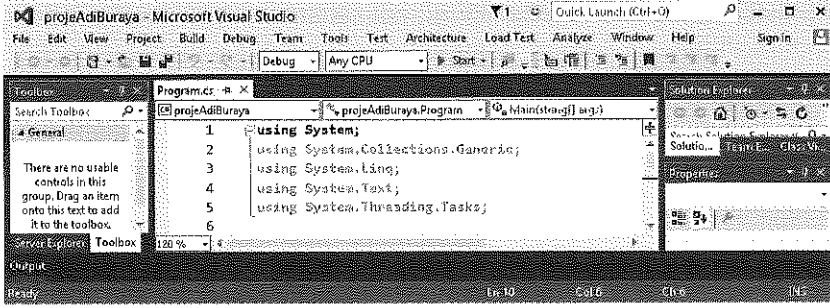


Şekil 4.8 Namespace ve Class yapısı

Sık kullanılan **namespace**'ler aşağıda listelenmiştir ve bunların en çok kullanılanları Visual Studio tarafından otomatik olarak programa eklenir.

```
System
System.Text
System.Collections
System.Web
System.Web.UI
System.Web.UI.WebControls
System.Forms
System.Data
System.Linq
```

Eğer Visual Studio'da bir proje açarsanız otomatik olarak eklenen aduzaylarını aşağıda görebilirsiniz. Çoğunlukla, eklenen aduzayları istediğiniz kodları yazmak için yeterlidir. Eğer kullanacağınız sınıfları içeren aduzayı otomatik olarak eklenmediyse **using** komutunu kullanarak kodların en üstüne ekleyebilirsiniz.



Şekil 4.9 Projeye otomatik eklenen aduzayları

Peki! Aduzaylarını eklemesek sınıfları kullanamayacak mıyız? Gene kullanabiliriz fakat daha uzun kod yazmak zorunda kalırız. Örneğin **Console** sınıfının **Write()** metodu **System** sınıfının içindedir ve ekrana veri çıkışı için kullanılır. Eğer **using System;** komutunu kullandıysanız ve **System** aduzayını projenize dâhil ettiyseniz ekrana metin veya veri çıkışı için aşağıdaki komutu yazmanız yeterlidir:

```
Console.Write("metin");
```

Eğer **System** aduzayını projenize eklemeydiyseniz yukarıdaki kodu çalıştırmak için aşağıdaki gibi yazmamız gerekiyor:

```
System.Console.Write("metin");
```

Visual Studio bizi küçük bir sıkıntıdan kurtarıp sıkça kullanılan aduzaylarını projeye ekler. Böylece çoğu aduzayını hatırlamaya ve yazmaya gerek kalmadan sınıfları doğrudan kendi adlarıyla kullanabiliriz.

Proje geliştirirken sıklıkla kullanacağımız aduzayları ve açıklamalarını aşağıdaki tabloda bulabilirsiniz.

Aduzayı	Açıklama
System	Temel sınıfları barındırır. Ayrıca, diğer tüm aduzayları da bu aduzayı içine gömülmüştür. Yani, bu aduzayı olmadan herhangi bir proje geliştiremezsiniz. Komut satırı uygulaması geliştirirken kullanılan Console sınıfı, veritiplerini dönüştürmek için kullanılan Convert sınıfı, matematiksel işlem yapmak için kullanılan Math sınıfı gibi birçok sınıf bu aduzayı içinde bulunur.
System.IO	Dosya oluşturmak, dosyalara veri yazmak veya dosyalardan veri okumak gibi işlemleri yapmak, yani Input/Output (Giriş/Çıkış) yapmak için kullanılan aduzayıdır.
System.Data	Veritabanı işlemleri gerçekleştirmek için kullanılan sınıfları barındıran aduzayıdır. Bu aduzayı kullanılarak değişik veritabanı yazılımlarına bağlantı yapılabilir. Kullanıcılardan alınan verilerin tablolara kaydedilmesi veya sorgular yapıp tablolardan alınan verilerin listelenmesi vb. işlemleri sağlayan sınıflar bu aduzayı içinde bulunur. Bu aduzayının altında System.Data.Common , System.Data.OleDb , System.Data.SqlClient ve System.Data.SqlTypes gibi alt aduzayları da bulunur.
System.Text	ASCII, Unicode, UTF-7 ve UTF-8 gibi karakterlerin çözülmesini (encoding) sağlayan tüm sınıfları içerir.
System.Collections	Çeşitli nesne koleksiyonlarını tanımlayan arayüz ve sınıfları içerir. Örnek olarak; list, arraylist ve binnary verilebilir.
System.Web	İstemci/sunucu iletişimini sağlayan sınıf ve arayüzleri içerir. Bu aduzayı, http isteğiyle bilgi içeren http request sınıfını, istemciye giden çıktıyı yöneten http response sınıfını ve sunucu tarafı zamanlar için erişim sağlayan HttpServerUtility nesnesini içerir. Bu aduzayı aynı zamanda çerez işleme, dosya transferi, exception information (istisna bilgisi) ve çıktı ön bellekleme denetimlerini de içerir.
System.Web.UI	Bir web sayfası üzerinde kullanıcı arayüzünü oluşturan kontrollerin oluşturulması için gereken sınıfları barındırır. Bu aduzayı ortak bir işlevsellikle HTML ya da web kullanıcı kontrolleri olmak üzere tüm kontrolleri içeren control sınıfına sahiptir. Ayrıca web uygulamasında bir sayfaya istem olduğu anda oluşturulan sayfa kontrolünü de içerir.
System.Windows.Forms	Windows tabanlı, Microsoft Windows işletim sisteminin sahip olduğu, zengin kullanıcı arayüzü olanaklarını kullanan uygulamaları geliştirmek için gereken tüm sınıfları içerir.
System.Xml	Xml (extensible markup language) dili için standartlara dayalı destek sağlar.
System.Linq	LINQ teknolojisine ait sınıfları barındırır.
System.Net	Ağ/dağıtık programlama için gerekli olan sınıfları barındırır.

Özellik (Property) & Nesne Özellikleri

Nesnelerin renk, boyut, konum gibi niteliklerini değiştirmeye veya ayarlamaya yarayan parametrelere özellik denir. Örneğin web sayfanıza ya da formunuza bir buton

eklediğinizi düşünün. Butonun yüksekliğini ve genişliğini değiştirmek için sırasıyla **Height** ve **Width** özelliklerini kullanmalısınız.

```
Button1.Width=100;
Button1.Height=50;
```

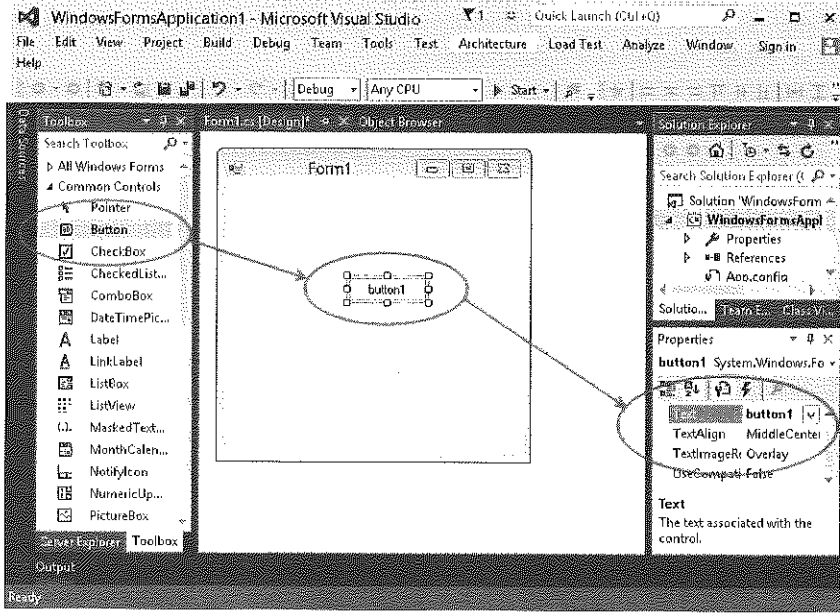
Yukarıda yazılan komutlarla **button1** nesnesinin **Width** özelliği **100**, **Height** özelliği **50** olarak değiştirilmiştir. Yukarıdakine benzer komutlar kullanarak **button1** nesnesinin birçok özelliğini değiştirebiliriz.

Eğer Visual Studio gibi bir editör kullanırsanız bir nesnenin özelliklerini sağdaki **Properties** penceresi ile değiştirebilirsiniz. Visual Studio'da soldaki **Toolbox** denilen panelden kullanılmak istenen sınıf (kontrol) fare ile form üzerine sürüklenir ve nesne oluşturulur. Daha sonra da bu nesnenin özellikleri sağdaki **Properties** paleti ile değiştirilir.

Aşağıdaki şekilde Visual Studio programını görüyorsunuz. En soldaki Toolbox üzerinden **Button** sınıfı alınıp fare ile form üzerine sürüklenerek **button1** nesnesi oluşturulmuştur. (Nesneye ilk ismi Visual Studio otomatik olarak vermektedir. Nesne adı istenirse **Properties** panelinden değiştirilebilir.) Buton oluşturulduktan sonra (seçiliyken) sağdaki **Properties** panelinden butonun genişliği ve yüksekliği değiştirilmiştir. Eğer değişiklik yapmak istediğiniz nesne seçili değilse üzerine tıklayarak seçebilirsiniz. **Properties** penceresi kullanılarak ilgili nesnenin yazıtipi, rengi, ardaan rengi gibi birçok özelliği değiştirilebilir.

Form üzerinde değişik nesneler oluşturup özelliklerini inceleyerek konu üzerinde pratik yapabilirsiniz. Anlamadığınız bazı özellikler olabilir ama korkmanıza gerek yok. Zamanla işinize yarayacak birçok nesneyi ve özelliklerini öğreneceksiniz.

Windows Form elemanlarına **Toolbox** üzerinden erişebilmek için öncelikle bir Windows masaüstü uygulaması başlatmalısınız. Bunun için **New Project→Templates→Visual C#→Windows Form Application** menüsünden proje oluşturmanız gerekmektedir. Projeyi oluşturduktan sonra ekranın solunda bulunan **Toolbox** panelini kullanarak form üzerine istediğiniz elemanları oluşturabilirsiniz.



Şekil 4.10 Nesne oluşturma ve özelliklerinin değiştirilmesi

Metot (Method)

Nesnelerin yapabildiği işlere metot denir. Metotlar bazı dillerde fonksiyon ya da prosedür adıyla da anılır. Program içinde metin kutusu gibi bir nesne kullandığınızda metotlar sayesinde hiç kod yazmanıza gerek kalmadan bazı işleri yapabilirsiniz. Örneğin metin kutusunu temizlemek için **Clear()** metodunu kullanabilirsiniz.

```
TextBox.Clear();
```

Yukarıda yazılan komutla **TextBox** isimli nesnenin **Clear()** metodunu çalıştırdınız. Bu metot **.NET Framework** içinde hazır gelir ve metin kutularının içindeki yazıların kolayca temizlenmesini sağlar.

Dikkat ederseniz özelliklerle metotların kullanımı birbirinden farklıdır. Bir nesnenin özelliğini değiştirmek için atama (=) operatörü kullanılırken, metodunu çalıştırmak için parantez () kullanılır. Yani durum aşağıdaki gibidir:

```
Nesne.Özellik=Değer; (Nesnelerin özelliklerine yeni bir değer atanması)
Nesne.Metot(); (Nesnenin metodunun çalıştırılması)
```

Olay (Event)

Nesnelerin dışarıdan gelen etkilere gösterdiği tepkiye olay denir. Örneğin bir düğmeye tıkladığınızda bir olay gerçekleşir ve bazı işleri gerçekleştiren komutlar çalıştırılır.

Örneğin **Buton** isimli nesneye tıkladığımızda (**click** olayı) bazı kodların çalışmasını istiyorsak aşağıdaki kodları yazmalıyız:

```
Buton_Click()
{
    //bazı kodlar...
}
```

Yukarıdaki kodlar **Buton** nesnesine her tıklandığında, yani her **Click()** olayı gerçekleştiğinde güzel parantez ({...}) içindeki kodların çalışmasını sağlar. Yani siz bir nesneyi bir olayla etkilersiniz ya da tetiklersiniz o da bazı kodları çalıştırarak tepki gösterir. Bu programlama mantığına **Olay Güdümlü Programlama** (Event Driven Programming) denir.

Önümüzdeki derslerde yukarıda bahsi geçen **aduzayı**, **sınıf**, **nesne**, **özellik**, **metot** ve **olay** gibi kavramları uygulamalarla gerçekleştireceğiz ve daha iyi anlamaya çalışacağız. Şu anda sizin yapmanız gereken sadece bunların mantığını anlamaktır.

C# Nedir?

C# (Si Şarp diye telaffuz edilir) özellikle .NET uygulamaları üretmek için Microsoft tarafından geliştirilmiş ve halen geliştirilmeye devam eden bir dildir. .NET Framework 1.0 sürümü ile 2000 yılında piyasaya sürülen **C#** Java'ya rakip olarak ortaya çıkmıştır. Doğrusunu söylemek gerekirse, .NET Framework'ün doğal programlama dili **C#**'tir.

C ve C++ dilleri **C#** dilinin atasıdır. Genel olarak **C#** dilinin özelliklerini tanımlamak gerekirse C++ dilinin gücünü ve Visual Basic dilinin kolaylıklarını barındırmaktadır. Daha spesifik konuşmak gerekirse **C#** nesne tabanlı, olay güdümlü oldukça güçlü bir dildir.

C# dilini kullanarak hem masaüstü, hem web, hem de mobil uygulamalar geliştirebilirsiniz. Özellikle Türkiye'de **C#** gerek Windows uygulamaları gerekse ASP.NET uygulamaları için en çok tercih edilen dildir. O nedenle, eğer bir dil konusunda uzmanlaşmak istiyor ve hâlâ hangi dilde uzmanlaşacağınıza karar veremiyorsanız **C#** en iyi seçeneklerden biridir.

C# hakkında daha ayrıntılı bilgi için [http://en.wikipedia.org/wiki/C_Sharp_\(programming_language\)](http://en.wikipedia.org/wiki/C_Sharp_(programming_language)) veya <http://msdn.microsoft.com/en-us/vstudio/hh341490.aspx> adreslerini ziyaret edebilirsiniz. Kitabın ilerleyen bölümlerinde C# dili hakkında çok daha ayrıntılı bilgiler öğreneceğiz.

Visual Studio

Eğer .NET ve C#’tan bahsettiysek **Visual Studio**’dan da bahsetmek gerekir. Biliyorum, ilk bölümde **Visual Studio** hakkında temel bilgiler vermiştim ama burada daha ayrıntılı bahsedeceğim.

Visual Studio Microsoft firmasının .NET uygulamaları geliştirmek, çalıştırmak ve hata ayıklamak için piyasaya sürdüğü editördür. Aslında editör demek biraz hafif kalıyor çünkü IDE (Integrated Development Environment) demek gerekiyor. IDE, sadece basit editörler gibi kod yazmakla kalmayıp aynı zamanda kod tamamlama, renklendirme, hata bulma, derleme ve çalıştırma gibi ek özellikler de sağlıyor.

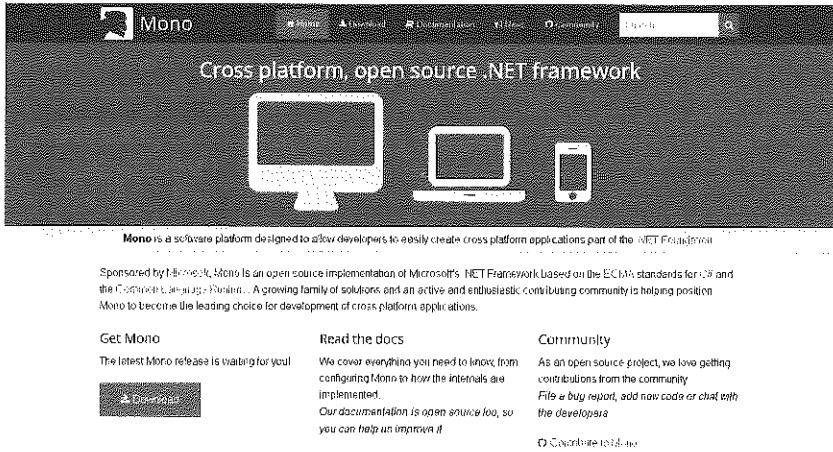
Değişik alternatifler olmasına rağmen benim şu ana kadar gördüğüm tüm yazılım firmaları .NET uygulamalarını Visual Studio ile geliştiriyorlar.

Visual Studio Alternatifleri

Visual Studio’ya alternatif yazılımlar hangileri diye sorarsanız MonoDevelop ya da SharpDevelop programlarını sayabilirim. Her iki yazılım da geniş kitleler tarafından fazla bilinmese de özellikle Linux veya Mac kullanıyorsanız (Visual Studio sadece Windows işletim sisteminde çalıştığı için) MonoDevelop’tan başka şansınız yok gibi görünüyor.

MonoDevelop hem Windows hem de Linux ve Mac sürümlerine sahip olmasıyla dikkat çekiyor. Yani isterseniz Linux üzerinde .NET uygulamaları geliştirebilirsiniz. Yaklaşık 30 MB gibi bir boyuta sahip olması, hem masaüstü hem de web uygulamaları geliştirilmesine izin veriyor olması MonoDevelop’u Linux kullanıcılarının gözdesi haline getirmiştir. Programı ücretsiz olarak <http://monodevelop.com/> adresinden indirip inceleyebilirsiniz.

Aşağıda MonoDevelop editörünün arayüzünü görüyorsunuz. Visual Studio kadar güçlü ve zengin içerikli bir editör olmasa da iyi bir geliştirme ortamı sağlar ve Linux kullanıcılarının editör ihtiyaçlarını giderir.



Şekil 4.12 Mono Project

Visual Studio 2015 Sürümleri

Visual Studio farklı amaçlara hitap eden birçok farklı sürüme sahiptir. Örneğin VS 2017 sürümünün **Enterprise**, **Professional**, **Community** ve **Express** gibi birçok sürümü vardır. Sürümleri karşılaştırmak ve işinizi görecekt en iyi seçimi yapabilmek için <https://www.visualstudio.com/vs/compare/> adresini ziyaret edebilirsiniz.

	Visual Studio Community	Visual Studio Professional	Visual Studio Enterprise	Visual Studio Test Professional	MSDN Platforms
Supported Usage Scenarios	***	****	*****	*****	*****
Debugging and Diagnostics	**	***	****	*****	*****
Testing Tools	*	**	***	****	*****
Integrated Development Environment	***	****	*****	*****	*****
Development Platform Support	***	****	*****	*****	*****
Xamarin Mobile Development	**	***	****	*****	*****
Architecture and Modeling	*	**	***	****	*****
Lab Management	*	**	***	****	*****
Team Foundation Server features	*	**	***	****	*****
Collaboration Tools	***	****	*****	*****	*****

Şekil 4.13 VS Özellik Karşılaştırması

Express sürümleri hariç, hangi sürümü kullanırsanız kullanın hepsiyle web, masaüstü, bulut, sunucu ve telefon uygulamaları geliştirebilirsiniz. Ben yeni başlayanlara **Community** sürümünü tavsiye ediyorum. **Community** sürümü aslında **Visual Studio Professional** sürümünün birkaç özelliği kırılmış ve ücretsiz halidir.

Visual Studio Express sürümleri daha çok öğrenciler veya hobi amaçlı programlamayla uğraşanlar için geliştirilmiş ücretsiz sürümlerdir. Ücretli sürümlerden farklı ola-

rak sadece belli bir amaç için üretilmişlerdir; ücretli sürümlerle hem web hem masaüstü hem de mobil uygulamalar geliştirmek mümkün iken **Express** sürümler ile sadece web, sadece masaüstü veya sadece mobil uygulamalar geliştirebilirsiniz. Yani ne tür uygulama geliştirecekseniz ilgili VS Express sürümünü kurmalısınız. Örneğin masaüstü uygulamalar geliştirecekseniz **Windows Masaüstü için Visual Studio Express** sürümünü kurmalısınız.

Eğer **Express** sürümlerden herhangi birini veya hepsini indirip kurmak istiyorsanız <https://www.visualstudio.com/vs/visual-studio-express/> adresini ziyaret edebilirsiniz. Sayfa açıldıktan sonra biraz aşağıya doğru gidin ve aşağıdaki şekilde görülen **Express** sürümleri inceleyin. Bu sayfada şiddetle ücretsiz olan **Visual Studio Community** sürümü tavsiye ediliyor ama illa **Express** sürümü kullanacağım dererseniz indirme imkânı sunuluyor. Eğer ASP.NET uygulamaları geliştirecekseniz Web, Windows mobil uygulamaları geliştirecekseniz Windows 10, masaüstü uygulamaları geliştirecekseniz Windows for Desktop sürümünü indirip kurmanız gerekiyor.

Still want Visual Studio Express?

Express for Desktop

Supports the creation of desktop applications for Windows.

Express for Web

Create standards-based, responsive websites, web APIs, or real-time online experiences using ASP.NET.

Express for Windows 10

Provides the core tools for building compelling, innovative apps for Universal Windows Platform. Windows is required.

Team Foundation Server 2015 Express

Free source-code-control, project management, and team-collaboration platform.

Şekil 4.14 Visual Studio Express Sürümleri

Kitaptaki uygulamalar **Visual Studio Community 2015** sürümü ile yapılmıştır. Fakat siz Visual Studio 2008, 2012, 2013 veya Visual Studio 2017 gibi farklı sürümleri de kullanabilirsiniz. Kodlama açısından fark etmez.

Visual Studio Community 2015 Gereksinimleri

Günümüz bilgisayarlarının çoğunun donanımı Visual Studio yazılımının gereksinimlerini karşılamaktadır. Bununla birlikte, maalesef Visual Studio'yu her sisteme kuramayabilirsiniz. Her yazılımda olduğu gibi Visual Studio'nun da minimum yazılım ve donanım ihtiyaçları vardır. Eğer belirtilen işletim sistemine veya donanıma sahip değilseniz bilgisayarınızı güncellenmeniz gerekecektir. Kurulumda ve kullanımda sorun yaşamamak için lütfen aşağıdaki şartların sağlanmasına dikkat edin. Eğer aşağıda belirtilen şartlar sağlanmazsa Visual Studio 2015 kurulumunda sorun yaşayabilirsiniz. Sorun yaşanması veya sisteminizin uygun olmaması durumunda VS2012 veya

VS2013 gibi daha eski sürümleri de kurabilirsiniz. Kitaptaki kodlar daha eski sürümlerde de çalışır.

Desteklenen İşletim Sistemleri:

- Windows 10
- Windows 8.1
- Windows 8
- Windows 7 SP 1
- Windows Server 2012 R2
- Windows Server 2012
- Windows Server 2008 R2 SP1

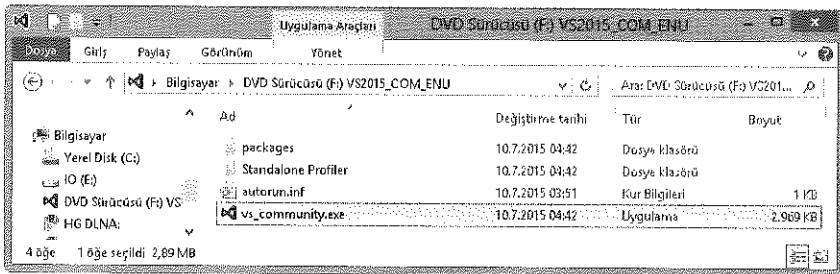
Donanım Gereksinimleri:

- 1,6 GHz veya daha hızlı işlemci
- 1 GB RAM
- Diskte 4 GB boş alan
- 5400 RPM sabit disk sürücüsü
- DirectX destekli ekran kartı (1024x768 veya daha yüksek çözünürlük)

Visual Studio Kurulumu

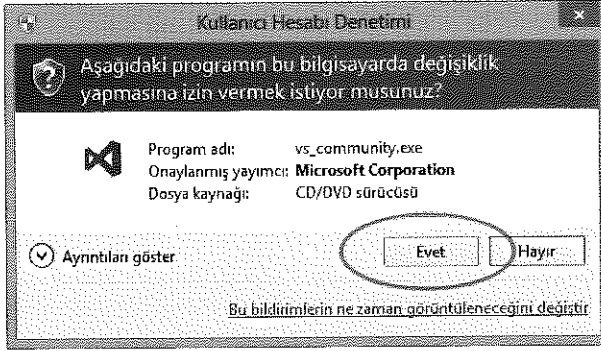
Eğer sisteminizi test ettiyseniz ve bilgisayarınızın minimum donanım ve yazılım ihtiyaçlarını karşıladığını düşünüyorsanız kurulumu başlayabilirsiniz. Bu başlık altında size **Visual Studio Community Edition 2013** sürümünün kurulumunu anlatacağım. Diğer sürümlerin (VS2012/VS2015) kurulumu da hemen hemen aynıdır.

Kurulum CD'sini taktığınızda kurulum otomatik başlar. Eğer başlamazsa Windows Gezgin'i'ni açın ve CD sürücüye tıklayın. Sağ tarafta görünen **vs_community.exe** dosyasına sağ tıklayın ve yönetici olarak çalıştırın.



Şekil 4.15 VS 2015 kurulumu

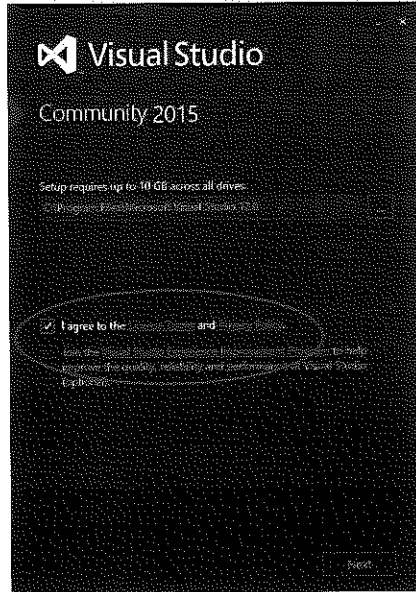
Kurulum dosyası çalıştığı zaman aşağıdaki gibi bir pencere ile karşılaşacaksınız. Bu pencere sizden kurulum için izin istemektedir. **Evet/Yes** olarak yanıtlayın.



Şekil 4.16 Kurulum İzni

Visual Studio kurulumu başladıktan sonra karşınıza aşağıdaki ekran gelir. Bu ekranda **"I agree..."** kutusunu işaretledikten sonra **Next** düğmesine tıklayın.

Eğer Microsoft'un **Müşteri Deneyimi Geliştirme Programı**'na katılmak istiyorsanız hemen alttaki **"Join ..."** kutusunu da işaretleyebilirsiniz. Bu programa katılmayı seçtiğiniz zaman Visual Studio ve kullanım alışkanlıklarınız ile ilgili bazı veriler/hatalar Microsoft ürün geliştirme merkezine gider. Alınan verilerle daha iyi bir Visual Studio üretilmeye çalışılır.

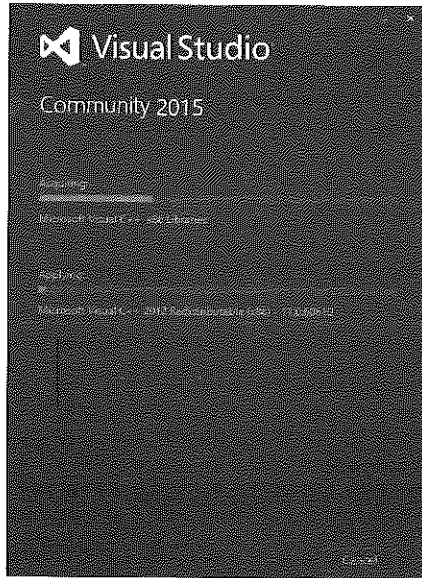


Şekil 4.17 VS kurulumunun başlatılması

Next düğmesine tıkladıktan sonra **“Optional features to install”** penceresi gelir. Burada size Blend, LightSwitch, Developer Tools gibi ekstra bazı bileşenleri kurma şansı verilir. Bu penceredeki tüm bileşenlerin kurulumu 2GB civarında yer kaplar. Eğer diskte alan sorunuz yoksa **“Select All”** kutusunu işaretleyerek tüm bileşenler kurmanızı tavsiye ederim. Bu bileşenler veritabanı veya diğer bazı konularda işinize yarayabilir. Eğer kısıtlı bir alana sahipseniz kurmayabilirsiniz. Eğer **Windows Store** uygulamaları geliştirecekseniz Windows 8.1 kurmalısınız.

Kurmak istediğiniz bileşenleri işaretledikten sonra **INSTALL** düğmesine tıklayıp kurulumu başlatabilirsiniz.

INSTALL düğmesine tıkladığınız anda aşağıda da görüldüğü gibi artık kurulum başlamıştır. Bundan sonra yapmanız gereken sabırla kurulumun bitmesini beklemektir. Bilgisayarınızın donanımına ve seçtiğiniz bileşenlere bağlı olarak kurulum 5 ile 30 dakika arası sürebilir. Örneğin .NET Framework daha önceden kurulu ise kurulum daha kısa sürebilir. .NET Framework kurulu değilse Visual Studio .NET Framework’ü de kuracağı için doğal olarak kurulum daha fazla zaman alır.



Şekil 4.18 VS kurulumu

Eğer kurulumu herhangi bir nedenle iptal etmek istiyorsanız **Cancel** düğmesine tıklayabilirsiniz. Bu durumda kurulan bileşenler kaldırılır. İsterseniz daha sonra kurulumu tekrar başlatabilirsiniz.

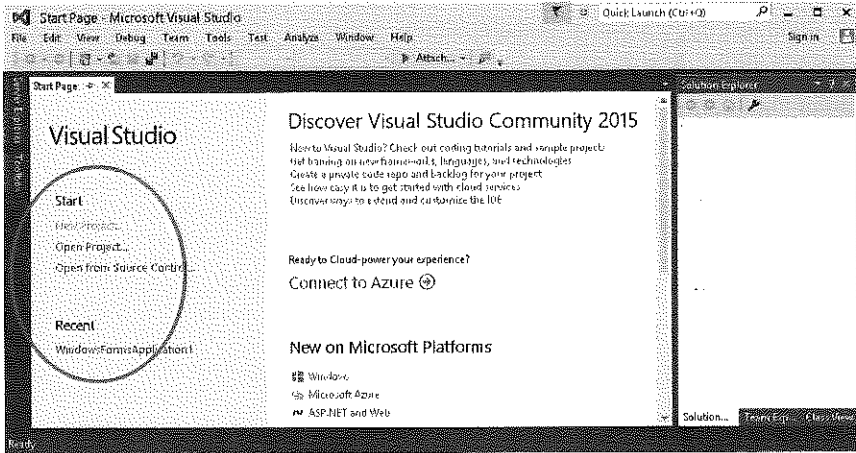
Kurulum başarıyla tamamlandıysa **“Setup Successful!”** yazısı çıkar. Artık alttaki kocaman **LAUNCH** düğmesine tıklayarak Visual Studio’yu çalıştırabilirsiniz.

Program ilk çalıştığında kullandığınız sürüme göre sizden **Ürün Anahtarı/Product Key** isteyebilir. Eğer **Express** sürümlerden birini kurduysanız **Register Online** linkine tıklayarak Microsoft hesabı açın ve ürün anahtarı alın. Eğer almazsanız 30 gün sonra kullanım süresi sona erer. İsterseniz **Cancel** düğmesine tıklayarak kayıt olmadan da devam edebilirsiniz.

Programın ilk çalışmasıyla beraber temelde ne tür uygulamaların geliştirileceğini **Development Setting** ile belirlemeniz istenir. Burada **General** veya **Visual C#** seçeneklerini seçebilirsiniz. Bunun hemen altında ise hangi temanın (renk ayarlarının) kullanılacağı sorulur. Hoşunuza giden bir temayı seçtikten sonra **Start Visual Studio** düğmesine tıklayarak devam edebilirsiniz.

Visual Studio'nun ilk açılışında karşımıza aşağıda görülen ve adına **Start Page** denilen ekran gelir. "**Discover Visual Studio Community 2013**" bölümünde değişik programlama dilleri ile ilgili yenilikleri görebilirsiniz. Daha aşağıdaki konularda Microsoft ile ilgili değişik haberlere ve eğitim dokümanlarına ulaşabilirsiniz.

Sol tarafta **Start** bölümünde **New Project** komutu ile yeni bir proje başlatabilir, hemen onun altındaki **Open Project** komutu ile daha önceki projelerinizi açabilirsiniz. **Recent** bölümünde ise son çalışılan projeleri görebilir ve kolayca açabilirsiniz.



Şekil 4.19 Start Page

Visual Studio İçinde Neler Var?

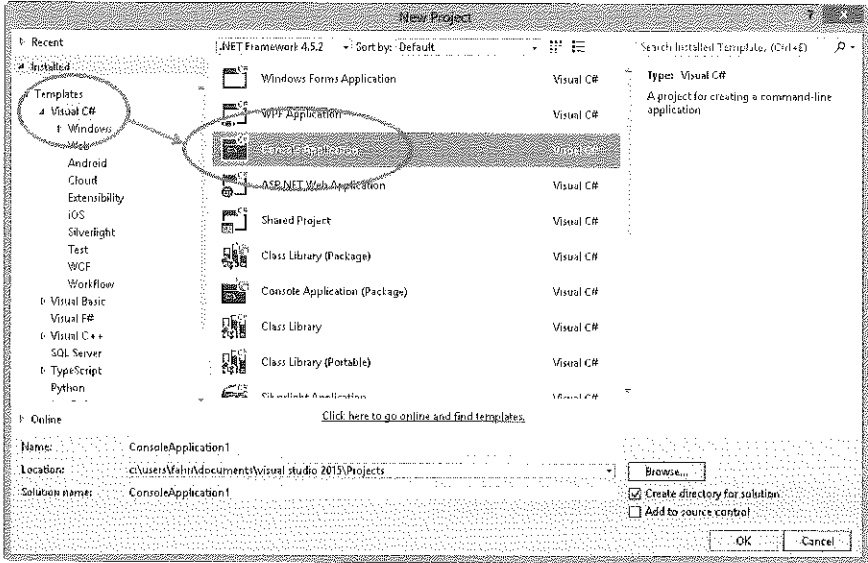
VS başarılı bir şekilde kurulduktan sonra artık program yazmaya hazırsınız demektir. Daha önceki konularda da bahsettiğim gibi VS oldukça gelişmiş bir IDE'dir. İçinde aşağıdaki bileşenleri barındırır:

- Kod Editörü
- Hata Ayıklayıcı (Debugger)
- Windows Forms
- .NET Framework
- .NET Class Library
- Microsoft SQL Server Compact
- IIS Express Web Sunucu (Web sürümünde)

Visual Studio ile Proje Oluşturma

Eğer Visual Studio kullanarak kod yazmak istiyorsanız yeni bir proje oluşturmak zorundasınız. VS ile yeni bir proje oluşturmak için **Start Page** sayfasında **New Project** veya **File** menüsünde **New Project** komutu çalıştırılır.

New Project penceresinde sol tarafta **Installed Templates** menüsünde hazır olarak gelen kurulu diller listelenir. Templates altındaki üçgenlere tıklayarak desteklenen dilleri veya teknolojileri görebilirsiniz. Gördüğünüz gibi, Visual Studio 2015 Visual Basic, C# ve C++ dillerini herhangi bir kurulum yapmadan destekler. Bu menüde önce **Visual C#** daha sonra da uygulama türü seçilir. Biz algoritma dersleri boyunca sadece kod ekranında çalışmamızı sağlayan **Console Application** uygulama türünü kullanacağız. Eğer Windows Forms Application veya WPF Application seçilirse görsel Windows uygulamaları geliştirebilirsiniz.

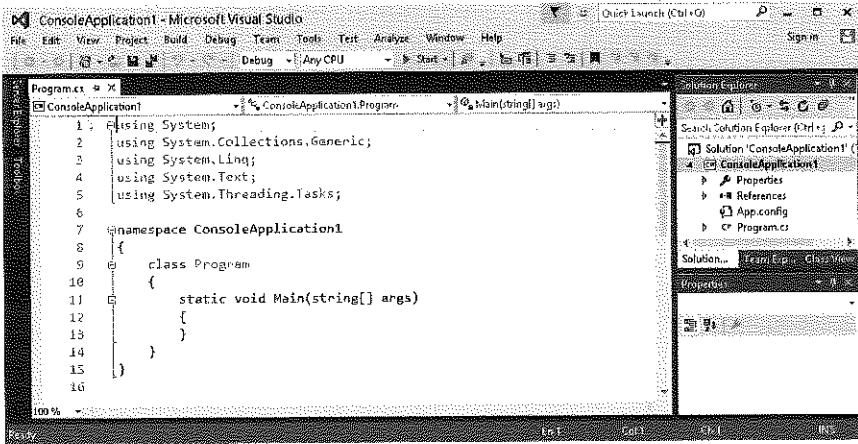


Şekil 4.20 New Project penceresi

Name kutusuna yazacağımız programın adını yazdıktan sonra **Browse** butonuna tıklıyoruz ve projenin nereye kaydedileceğini belirliyoruz.

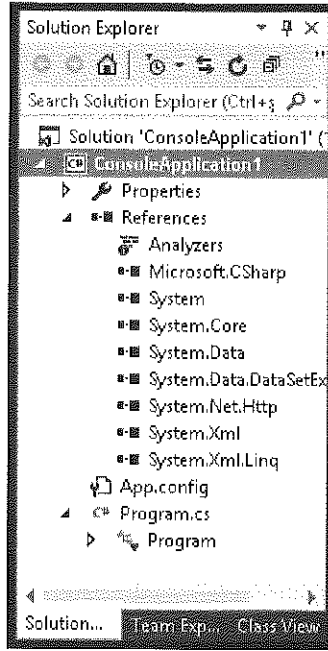
Eğer sağ tarafta bulunan **Create directory for solution** kutusu işaretli ise oluşturacağınız proje için otomatik olarak belirlediğiniz proje adına sahip bir klasör oluşturulur ve projeye ilgili tüm dosyalar bu proje klasörüne kaydedilir.

OK butonu tıklandıktan sonra aşağıda görülen ekran gelir ve artık kod yazmaya hazırsınız demektir.



Şekil 4.21 VS kod penceresi

Peki! Neden Visual Studio dosyası açmıyoruz da Visual Studio projesi açıyoruz? Çünkü Visual Studio'da yazdığımız kodlar tek bir dosyadan oluşmaz; birbiri ile ilgili birkaç dosyadan oluşur. Yeni bir proje oluşturduğunuz zaman sağ taraftaki **Solution Explorer** panelini dikkatlice inceleyiniz.



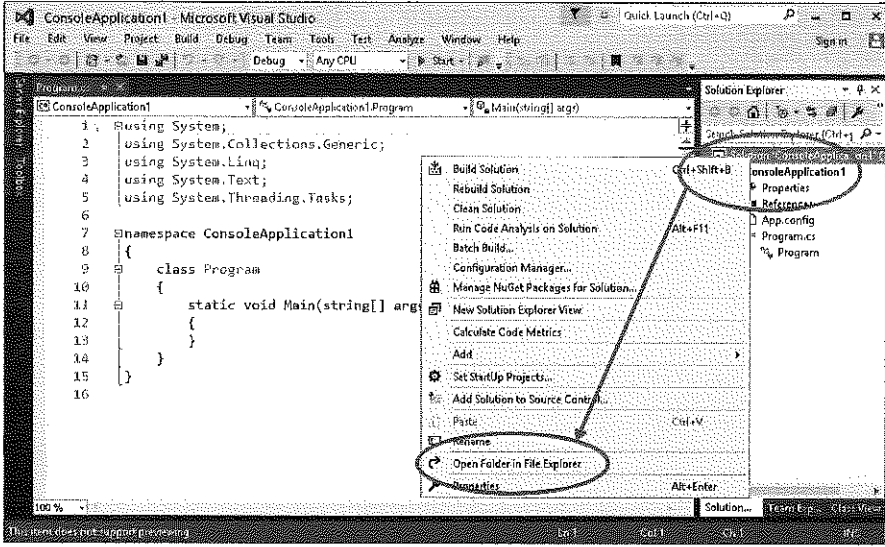
Şekil 4.22 Solution Explorer

Solution Explorer paneli projenize yeni dosya ekleme, gereksiz dosya silme, dosya açma veya dosyaların adını değiştirme gibi işlemlerin yapılmasını sağlar. Eğer **Solution Explorer** açık değilse **View\ Solution Explorer** komutu ile açabilirsiniz.

Solution Explorer dikkatlice incelendiğinde burada kodlarımızla ilgili birçok bileşenin bulunduğunu göreceksiniz. Örneğin **References** altında kullanılan namespace adları görülmektedir. **Program.cs** altında ise programınızdaki sınıfları ve metotları görebilirsiniz. Ayrıca gerekirse yeni yeni bileşenler de ekleyebilirsiniz.

Uzun lafın kısıası, bir C# programı aslında zamanla öğreneceğiniz bir sürü bileşenden oluşmaktadır. Projemiz tüm bu bileşenleri kapsamaktadır.

Eğer projenizin oluşturduğu dosyaları görmek isterseniz **Solution Explorer** penceresinde **Solution** ögesine sağ tıklayın ve **Open Folder in File Explorer** komutuyla proje dosyalarını **Windows Gezgini** içine açın.



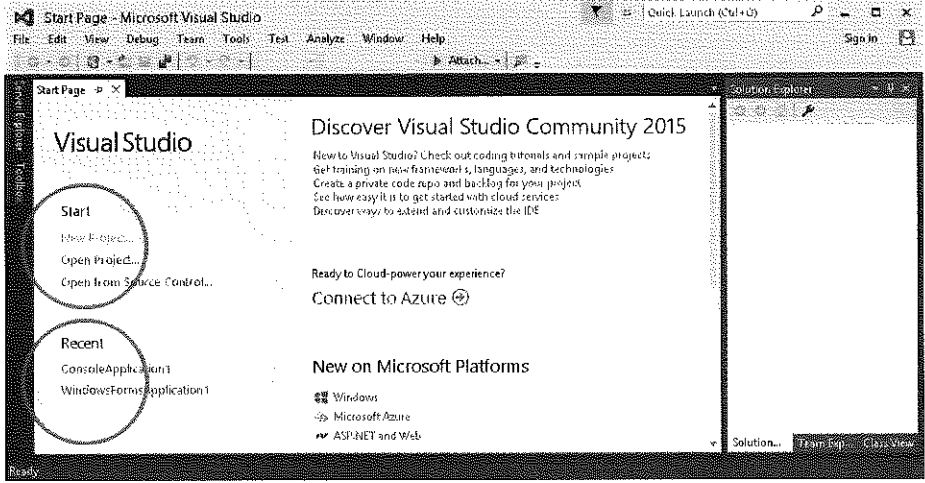
Şekil 4.23 Proje dosyaları

Windows Gezgini açıldığında **.sln** uzantılı proje dosyanızı ve proje klasörünüzü göreceksiniz. Eğer proje klasörünü açarsanız **bin**, **obj** ve **properties** klasörlerinin yanında içinde yazdığımız kodların da bulunduğu **Program.cs** dosyasını görürsünüz.

Şimdilik buradaki klasörler ve dosyalar ne işe yarıyor? diye düşünmenize gerek yok. Hatta bilmeseniz de olur. Anlatmaya çalıştığım şey şu: Visual Studio ile oluşturduğumuz yazılımlar tek bir dosyadan değil, onlarca dosyayı kapsayan **proje** adlı yapılardan oluşur.

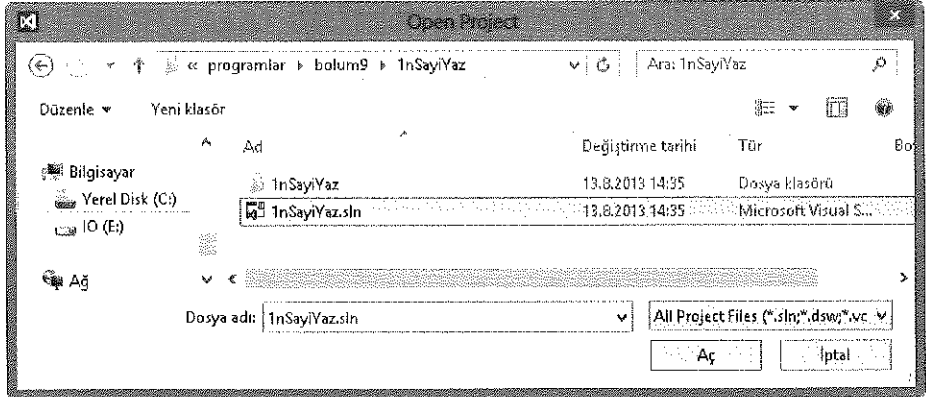
Projelerin Açılması

Daha önceden kaydedilmiş projelerin açılması için **Start Page** ekranında bulunan **Open Project** komutu veya **File\Open\Project** komutu kullanılır. Eğer açmaya çalıştığınız proje son projeler arasında ise **Recent** bölümünden de açabilirsiniz.



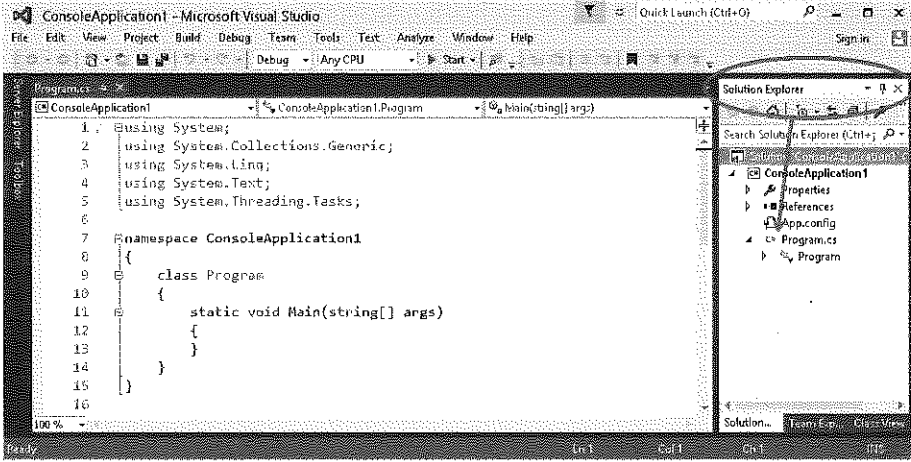
Şekil 4.24 Open Project

Visual Studio editöründe kodlarımıza ve diğer dosyalarımıza ulaşabilmemiz için daha önceden oluşturduğumuz proje dosyasının açılması gerekmektedir. Visual Studio proje dosyalarının uzantısı **.sln**'dir. Herhangi bir projeyi açmak için **Open Project** komutunu verdikten sonra ilgili klasöre gidip projenizin **.sln** uzantılı dosyasını seçmeniz ve **Aç/Open** komutunu vermeniz gerekiyor.



Şekil 4.25 Proje açma

Projeyi açtıktan sonra yazdığınız kodlar görünmüyorsa **View\ Solution Explorer** komutu ile **Solution Explorer** panelini açıp kodların bulunduğu **Program.cs** dosyasına çift tıklamamız gerekiyor.



Şekil 4.26 Solution Explorer ve kodların açılması

Projelerin Kaydedilmesi ve Kapatılması

Her programda olduğu gibi Visual Studio projelerinin kaydedilmesi için **File\ Save** komutu kullanılır. Eğer açık bir projeyi kapatmak istiyorsanız **File\ Close Solution** komutunu kullanabilirsiniz.

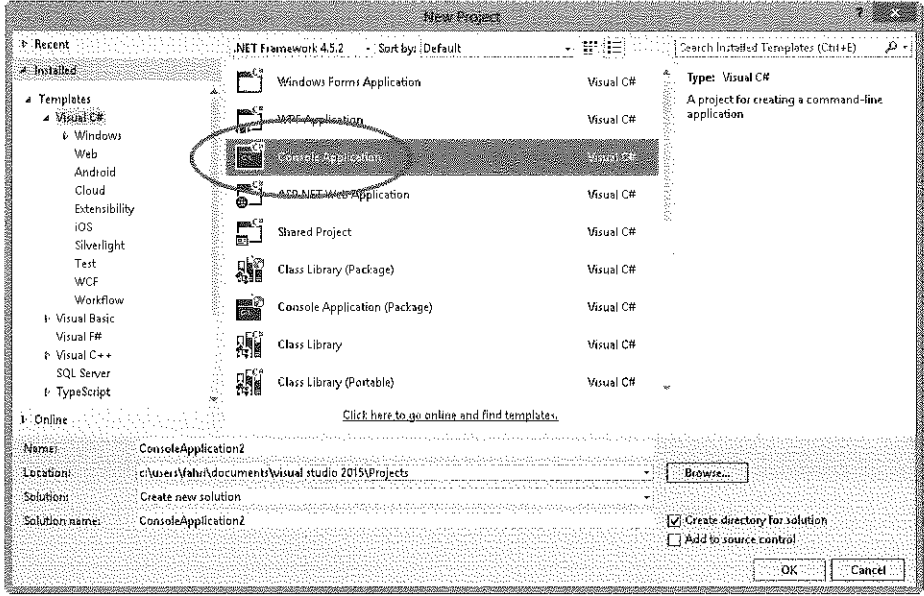
Visual Studio Express ile diğer Visual Studio sürümleri arasında neredeyse hiç fark yoktur. Dolayısıyla yukarıda anlatılan konular aynen Visual Studio Express sürümleri için de geçerlidir. Proje oluşturma, proje kaydı ve daha önceden oluşturulan projelerin açılması aynı şekilde yapılır.

Özellikle programcılığa yeni başlayan öğrenciler veya amatör programcılar için Visual Studio Express sürümleri ücretsiz olmasından dolayı en iyi çözümdür. Visual Studio Express sürümleri ile hem masaüstü, hem web, hem de mobil uygulamalar geliştirebilirsiniz. Sadece farklı uygulama türleri için farklı sürümler kullanmanız gerekiyor.

Console Application

Metin tabanlı program yazmak için kullanılan uygulama türüdür. **Konsol** uygulamalarında form, buton, metin kutusu gibi görsel nesneler olmaz; veri alışverişi MS DOS Komut İstemi olarak da bilinen komut satırından yapılır.

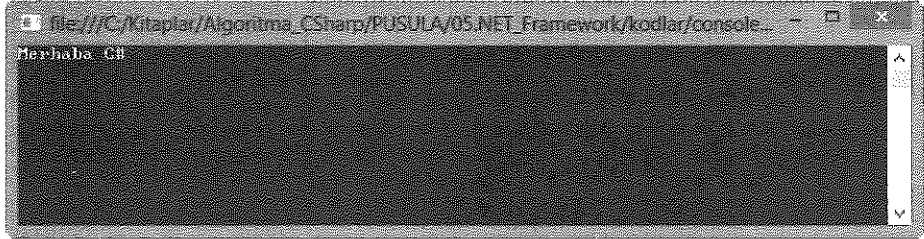
Aşağıdaki şekilden de görüldüğü gibi, **File\ New\ Project** komutunu verirseniz **Installed\ Templates\ Visual C#\ Windows** menüsünde **Console Application** seçeneği karşınıza çıkar.



Şekil 4.27 Console Uygulaması

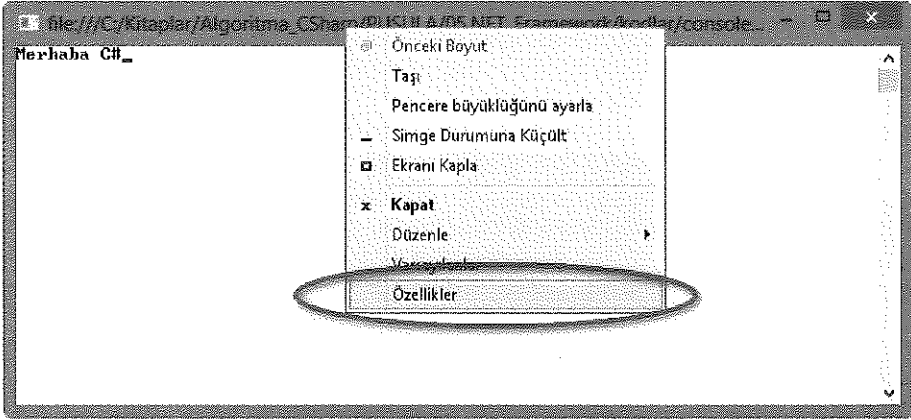
Algoritmada bizim amacımız programlama kavramlarını ve programlama mantığını öğrenmek olduğu için kodlama üzerine odaklanacağız. Kodlamaya odaklanmak için en iyi ortam konsol ortamıdır; dikkatimizi dağıtacak fazla görsel öğe ve arayüz elemanları yoktur.

Console ya da komut ekranı normalde siyah ardalana ve beyaz metinlere sahiptir.



Şekil 4.28 Normal konsol ekranı ya da komut penceresi

Başlık çubuğuna sağ tıklayıp **Özellikler\Renkler** menüsünden ardalana rengini, metin rengini ya da metin boyutunu değiştirebilirsiniz. Ben daha iyi okunabilir diye ardalana rengini beyaz, metin rengini de siyah yaptım. Ayrıca **Yazı Tipi** sekmesinden yazıtipi boyutunu da artırdım.



Şekil 4.29 Console penceresi

Daha önce de belirttiğim gibi konsol uygulamalarında form, buton, radyo, onay kutusu gibi görsel öğeler bulunmaz. Ama diğer her türlü sınıf ve nesne bulunur. Örneğin veritabanı uygulamaları geliştirilebilir.

Konsol ortamında form elemanları yerine veri giriş-çıkışı için **System** aduzayı içinde bulunan **Console** sınıfının özellik ve metotları kullanılır. Yani konsoldan okuma yapmak istersek veya konsola bir değer yazdırmak istersek **Console.Metot()** gibi bir komut kullanacağız.

MSDN

Bir programcının en iyi yardımcısı ve öğretmeni MSDN (Microsoft Solution Developer Network) yardım dokümanlarıdır. Çünkü bazen öyle sorunlarla karşılaşırız ki hiçbir yerde çözümünü bulamayız. İşte bu durumlarda MSDN yardımımıza koşar. MSDN, Microsoft firmasının kendi ürünleri ile ilgili deneme sürümü, yardım dokümanı, örnek kod ve problem çözümlerinin bulunduğu devasa bir kütüphanedir. Bu devasa kütüphaneye <http://msdn.microsoft.com> adresinden ulaşabilirsiniz. Bu adrese gittikten sonra menülerden veya arama kutusunu kullanarak istediğimiz bilgilere ulaşabilir. Tabi, çok büyük bir kütüphane olduğundan dolayı istediğiniz bilgiye ulaşmak biraz arama yapmanız gerekebilir.

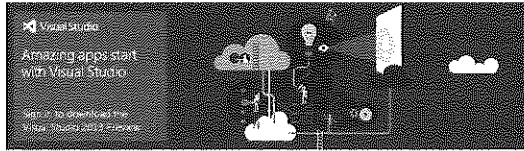
msdn

Ana Sayfa Kitapçılık Etkinlikler Ürün Yayıncılar Destek Topluluk Forumlar

Türkçe (Türkçe) | Oturum aç

Geliştiriciye Hoş Geldiniz

Microsoft Developer Network



MSDN Üyelik

BT sektörü projeler için ihtiyaç duyulan her şeyi MSDN Üyelik ile.

MSDN Üyelik alın.

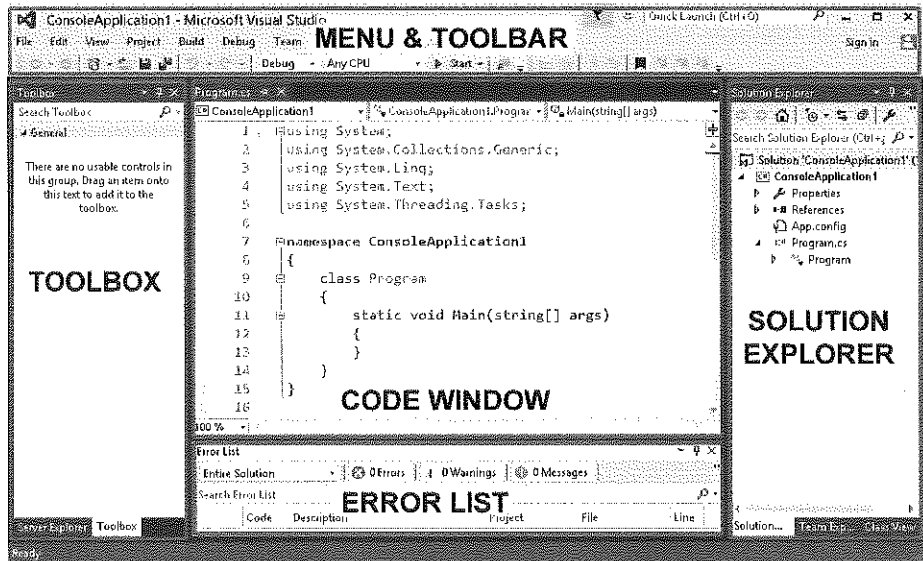
Yeni üye misiniz? Tıklayın

MSDN Üyesi misiniz? Tıklayın

Şekil 4.30 MSDN

Visual Studio Arayüzü

Bu başlık altında size Visual Studio'nun arayüzünü ve bazı özelliklerini anlatacağım. Şekil 4.27'de görülen **Console Application** seçildikten sonra **OK** komutunu çalıştırırsanız aşağıdaki gibi bir ekran karşınıza çıkar. Bu ekran açıldıktan sonra bir sürü öğe ile karşı karşıya kalırsınız. Bunlar gerektiğinde açıp kapatabileceğiniz **Toolbox**, **Solution Explorer**, **Properties** gibi öğelerdir.



Şekil 4.31 VS Arayüzü

Menu & Toolbar

Menü, VS ile ilgili komutlara ulaşmamızı ve proje ile ilgili ayarları değiştirmemizi sağlar. **Toolbar** ise menülerde çok kullanılan komutlara daha hızlı ulaşmamızı sağlar. VS ilk açıldığında varsayılan olarak **Standart** ve **Text Editor** toolbar açıktır. Algorit-

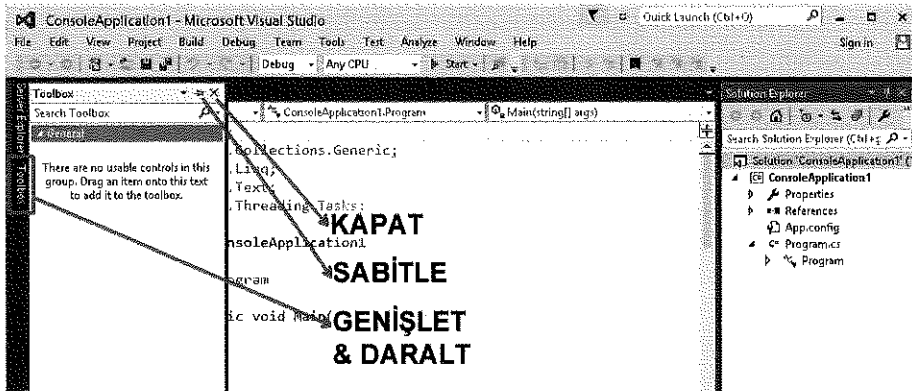
ma ile ilgili yapacağımız çalışmalarda bu iki araç çubuğu bize yeterli olacaktır. **View\ Toolbars** menüsünden diğer araç çubuklarını açıp kapatabilirsiniz.

Toolbox

Sol tarafta **Toolbox** denilen pencere vardır. **Toolbox**, görsel programlamada buton, metin kutusu, onay kutusu, liste vb. elemanları sürük-le-bırak yöntemiyle form üzerinde oluşturmak için kullanılır. **Console** uygulamalarında bu tür nesneler kullanılamayacağı için **Toolbox** üzerinde herhangi bir **class** ya da **kontrol** göremezsiniz. Eğer **Console Application** yerine **Windows Form Application** projesi oluşturursanız **Toolbox**'ta görsel uygulamalar geliştirmek için kullanılan kontrolleri görebilirsiniz.

En soldaki dikey **Toolbox** yazısına tıklayarak **Toolbox**'ı daraltıp genişletebilirsiniz. Eğer sabitlemek istiyorsanız raptiye şeklinde görünen butona tıklayabilirsiniz. Tamamen kapatmak için (x) butonuna tıklayınız. Eğer **Toolbox**'ı açmak istiyorsanız **View\ Toolbox** komutunu çalıştırabilirsiniz.

Console uygulamalarında **Toolbox** hiçbir işe yaramadığı için (x) butonuna tıklayarak kapatmak en güzeli. Böylece kod yazmak için daha fazla alan kalır.

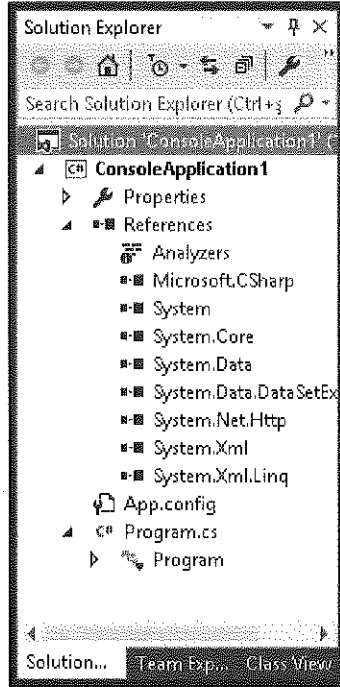


Şekil 4.32 Toolbox

Solution Explorer

Proje ilgili bütün dosyaları görmenizi, dosya açmanızı, dosya adı değiştirmenizi veya yeni dosyalar oluşturmanızı sağlar. Ayrıca **Properties** ile proje özelliklerine ulaşmanızı ve bu özellikleri değiştirmenizi sağlar.

Pencerenin genişletilip daraltılması, sabitlenmesi ve kapatılması **Toolbox**'ta anlatıldığı gibidir. Eğer tamamen kapalı ise **View\ Solution Explorer** menüsünden açılabilir.

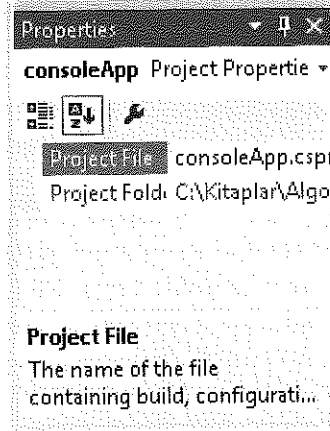


Şekil 4.33 Solution Explorer

Properties

Properties penceresi seçilen nesnelerin özelliklerini göstermeye veya değiştirmeye yarar. Eğer **Windows Form Application** projesi oluşturursanız seçilen nesnelerin (form, buton, metin kutusu vb.) özelliklerini görüp değiştirebilirsiniz. **Console Application** projelerinde görsel nesneler olmadığı ve kullanılmadığı için **Properties** penceresi de kullanılmaz. Dikkat ederseniz **Properties** penceresi bomboştur.

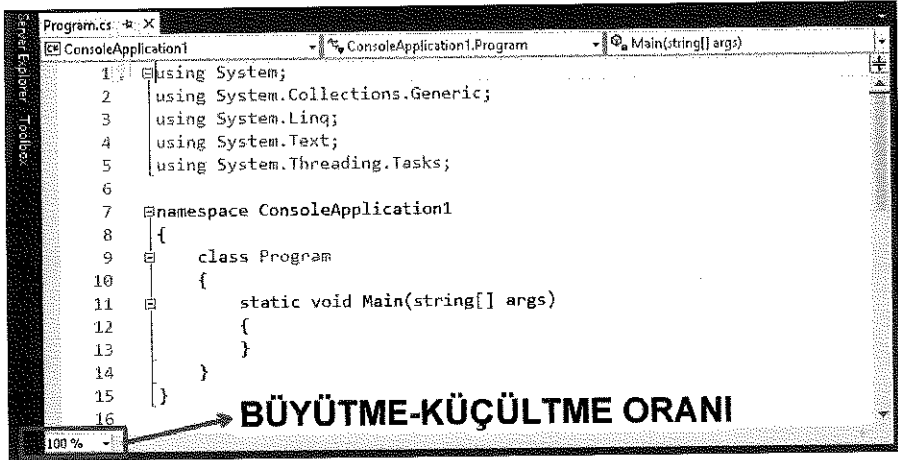
Pencerenin genişletilmesi, daraltılması ve kapatılması daha önceki pencereler gibidir. **Console** uygulamalarında bu pencere kullanılmadığı için en iyisi (x) düğmesine tıklayıp kapatmaktır.



Şekil 4.34 Properties penceresi

Kod Penceresi

Proje açıldığında tam ortada görünen ve kod yazmamızı sağlayan penceredir. Pencerenin üst tarafında o an çalıştığınız dosyanın sekmesini **Program.cs** adıyla görebilirsiniz. Eğer C# kullanıyorsanız dosya uzantısı **.cs**, C++ kullanıyorsanız **.cpp**, Visual Basic kullanıyorsanız **.vb** olur.



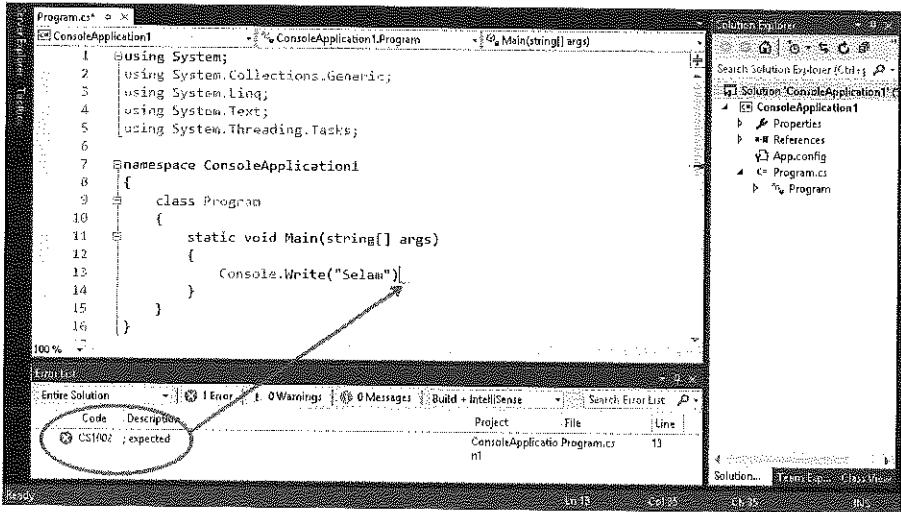
Şekil 4.35 Kod penceresi

Kod penceresindeki metinleri büyütüp küçültmek istiyorsanız **Ctrl+Tekerlek** kullanılabilir. Kontrol tuşu ile beraber tekerleği ileri-geri döndürürseniz metinlerin büyüüp küçüldüğünü görebilirsiniz. Hangi oranda büyüüp küçüldüğünü görmek için alt köşede 100 % şeklindeki metne bakabilirsiniz.

Error List

Adından da belli olduğu gibi, programda gerçekleşen yazım (syntax) hatalarını listeler. Eğer bir komutu yanlış ya da eksik yazarsanız o komutun altı kırmızı renkle çizilir ve yapılan hata ya da hatalar **Error List** penceresinde listelenir.

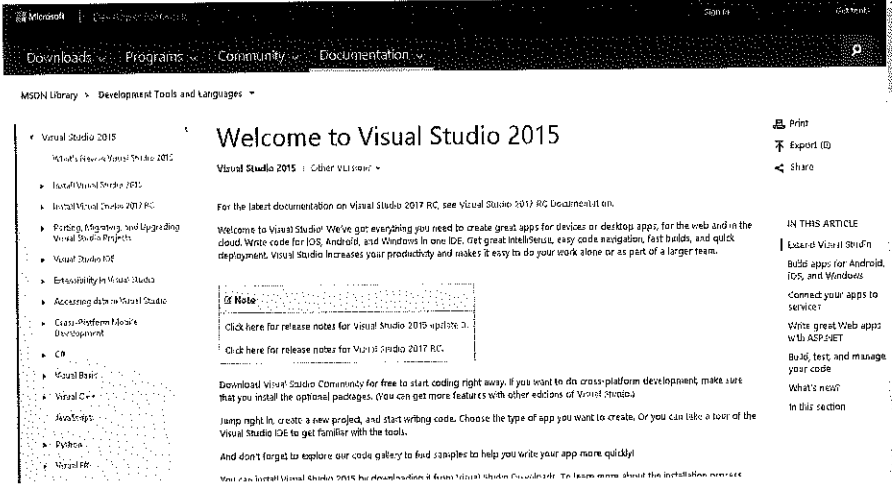
Error List penceresi sadece hataları listelemekle kalmaz aynı zamanda hatanın kaynağını ve hatanın düzeltilmesi için ne yapılması gerektiğini de söyleyebilir. Örneğin aşağıdaki örnekte komut sonuna noktalı virgül (;) koyulmadığı için hata oluşmuştur. **Error List** penceresi “; expected” açıklaması ile noktalı virgül koyulması gerektiğini, hatanın hangi satır ve sütunda olduğunu çok açık bir şekilde gösterir. Eğer **Error List** penceresindeki hata uyarısına fareyle çift tıklarsanız farenin imleci hatanın olduğu konuma gider. O nedenle bir hata aldığınızda açıklamayı iyice okuyup ondan sonra hatayı düzeltmeye çalışın.



Şekil 4.36 Error List

Yardım

Eğer biraz İngilizceniz varsa Visual Studio ve desteklenen programlama dilleri hakkında çığırın bir dokümantasyona sahipsiniz demektir. **Help** menüsünde **View Help** komutuna tıklarsanız msdn.microsoft.com adresine gidersiniz. Bu adreste Visual Studio ve desteklenen dillerle ilgili yenilikler, deneme programları, yardım dokümanları, örnek kodlar, videolar ve forumlar bulabilirsiniz.

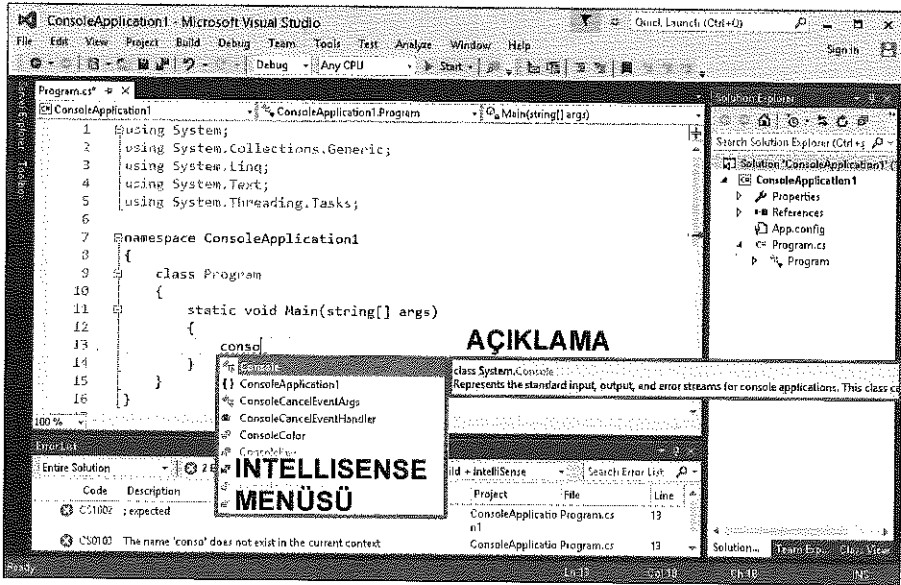


Şekil 4.37 Visual Studio Yardım sitesi

Intellisense

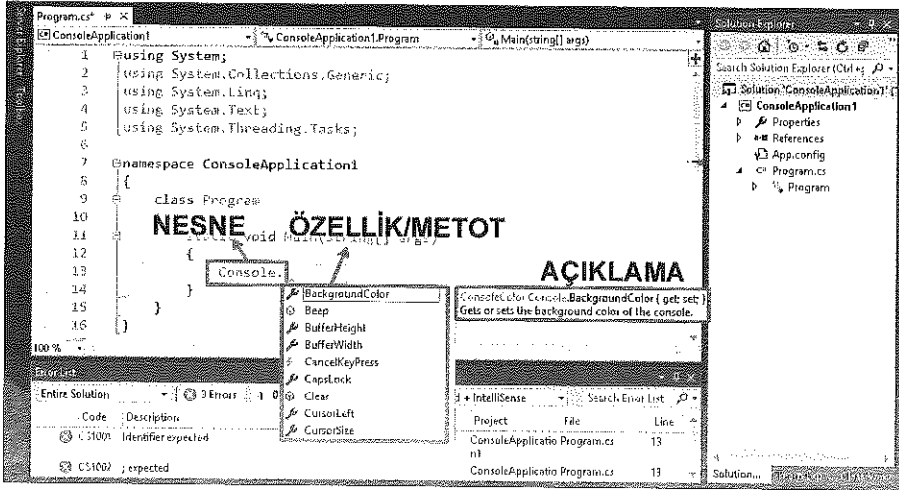
Visual Studio'nun kod önerme ve tamamlama özelliğine **Intellisense** denir. Bir komutun ilk birkaç harfini yazdığınızda VS size önerilerde bulunarak komutun tamamlanmasını sağlar.

Örneğin aşağıdaki şekilde görüldüğü gibi **conso** yazdığınızda Visual Studio **Intellisense** özelliği size muhtemel komutları listeler. Fare ile çift tıklayarak veya yön tuşu ile yukarı-aşağı hareket edip ENTER'layarak istediğiniz komutu seçebilirsiniz. Herhangi bir komutu seçtiğiniz zaman yanında komut ile ilgili açıklama da gösterilir.



Şekil 4.38 IntelliSense

Sınıf/Nesne adı yazıldıktan sonra nokta koyularak o nesne ile ilgili özellik ve metotlar listelenir. Listedeki üyeleri çift tıklayarak veya yön tuşlarıyla üzerine gelip ENTER'layarak seçebilirsiniz.



Şekil 4.39 Nesne, özellik ve metotlar

Visual Studio'nun **IntelliSense** özelliğini kullanmanızı şiddetle tavsiye ediyorum çünkü bu özellik sadece komutları tamamlayıp hızlı kod yazmanızı sağlamakla kalmıyor aynı zamanda komutları doğru yazmanızı da sağlıyor.

Örneğin “**conso**” yazdığınızda komutu Intellisense özelliği ile tamamlarsanız komut “**Console**” olur. Dikkat ederseniz komutun ilk harfi büyük yapıldı. Eğer **Console**’yi **console** yazarsanız hata alırsınız çünkü C# dili büyük-küçük harf duyarlı bir dildir. Yani Intellisense özelliği hem işinizi hızlandırdı hem de hatalı kod yazmanızı engelledi.

Komut/nesne yazıldıktan sonra o nesneye ait tüm özellik ve metotlar açıklamasıyla beraber listelenir. Böylece eğer bir nesnenin özellik veya metodunu unuttuysanız **Intellisense** özelliği sayesinde hatırlamış olursunuz.

Dikkat edilirse nesnenin özellikleri anahtar ile metotları ise küp şeklinde bir ikon ile gösterilir. Lütfen daha önce anlattığım özellik ve metotların kullanımını unutmayın.

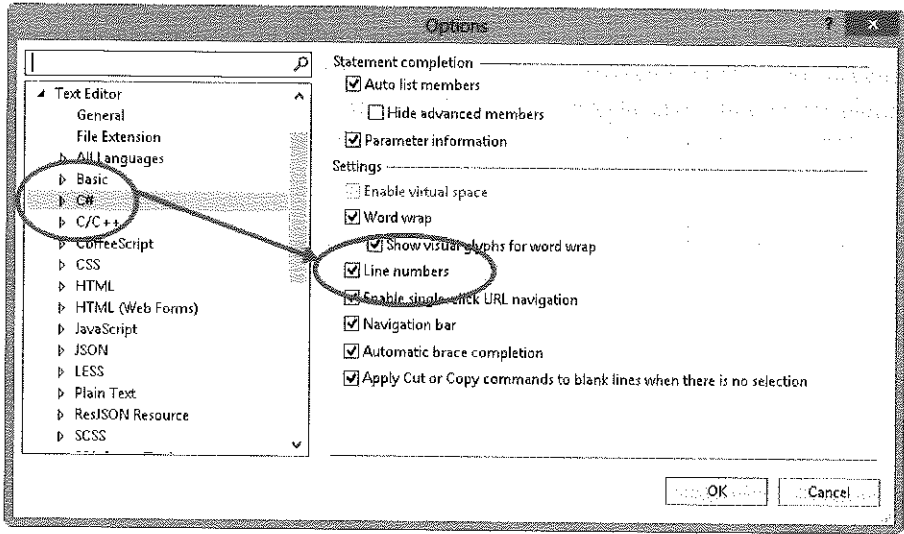
```
Nesne.Özellik=Değer;
Nesne.Metot();
```

Bir nesnenin özelliğine yukarıdaki gibi bir değer atanmalı, metodu ise parantezle kullanılmalıdır. Aksi takdirde hata alırsınız.

Satır Numaraları

Yazdığınız satırların başında numaralar görmek istiyorsanız satır numaralarını aktif hale getirebilirsiniz. Bunlar programın bir parçası değildir sadece programcıya yardımcı olmak amacıyla koyulmuşlardır.

Satır numaralarını görebilmek için **TollS\ Options\ Text Editor\ C#** menüsüne gidilmelidir. Her satırın başında görülen satır numaralarını açmak için **Display** bölümünde **Line numbers** kutusu işaretlenir.



Şekil 4.40 Satır numaraları

5

Değişkenler & Veri Giriş-Çıkışı

- Açıklama Satırları
- C# Yazım ve Noktalama Kuralları
- Veri Giriş-Çıkış Komutları
- Veri Tipleri
- Değişken Tanımlama
- Sabitler
- Değişken Ömrü ve Kapsamı
- Veri Tipi Dönüştürme
- Boxing-Unboxing
- Sık Yapılan Hatalar
- Uygulamalar

Hatırlatma

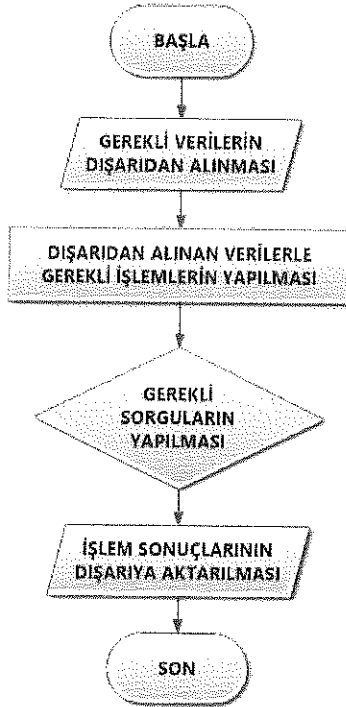
Daha önce algoritma konusunda da anlatıldığı gibi, genelde basit bir algoritma 3 temel aşamadan oluşur:

- **Veri girişi:** Dışarıdan gerekli verilerin alınması.
- **İşlem ve sorgu komutları ile alınan verilerin işlenmesi.**
- **Veri çıkışı:** Elde edilen işlem sonuçlarının ekrana veya kâğıda dökülmesi.

Bundan sonra yapacağımız örneklerde hep aşağıdaki programlama mantığını kullanacağız. Lütfen aşağıdaki akış diyagramını iyice inceleyin ve açıklamaları okuyun.

Adımların Açıklanması:

1. Her algoritma köşeleri yuvarlatılmış bir dikdörtgen içine alınan BAŞLA komutu ile başlar.
2. İkinci adımda dışarıdan alınması gereken veriler alınır; dışarıdan alınan veriler klavye veya fare gibi bir cihazla elde edileceği gibi barkod okuyan bir taramayıcıyla da elde edilebilir.
3. Üçüncü ve dördüncü adımda elde edilen veriler belli formüller kullanılarak bir işlemden veya sorgudan geçirilir. Örneğin dışarıdan alınan verilerin toplanması veya bu verilerin belli değerler arasında olup olmadığının sorgulanması bu adımda gerçekleştirilir.
4. Beşinci adımda elde edilen işlem sonuçları dışarıya aktarılır yani kullanıcıya gösterilir. Dışarıya aktarılan verilerin kullanıcıya gösterilmesini sağlayan cihazlar genelde ekran veya yazıcıdır.
5. Son adımda ise algoritma SON komutu ile bitirilir; unutmayın, önceki konularda bahsedildiği gibi her algoritmanın bir başı ve sonu olmalıdır.

**Şekil 5.1** Genel yöntem algoritması

Açıklama Satırları

Yazdığınız kodlara açıklama satırı ekleme alışkanlığı edinmek çok önemlidir. Eğer programı birkaç ay veya birkaç yıl sonra açıp değişiklik yapma ihtiyacı duyarsanız açıklamalar büyük kolaylık sağlar. Eğer açıklama yoksa birkaç yıl önceki bilgilerinizi veya algoritmayı unutmuş olabileceğinizden dolayı büyük sıkıntılar çekebilirsiniz. Ayrıca açıklamalar sayesinde proje üzerinde çalışan diğer programcılar da ne yaptığınızı daha kolay anlayabilir ve gerekirse proje üzerinde değişiklikler yapabilir.

Açıklama satırları sadece programcıya yöneliktir; derleyici tarafından dikkate alınmaz. Yani işlemci yazdığınız açıklamaları (derlenmediği için) görmez. Bu nedenle açıklama satırları programı yavaşlatmadığı gibi boyutunu da arttırmaz.

Özellikle yazdığınız programın en başına programın ne ile ilgili olduğu, hangi amaca hizmet ettiği, kimler tarafından yazıldığı, kimlere yazıldığı, ne zaman yazıldığı, hangi teknolojiler ve algoritmalar kullanıldığı, programın sürümünün ne olduğu vb. açıklama ve yorumlar yazmakta fayda vardır.

Yazdığınız kodların üstüne veya yanına kodların ne işe yaradığı, nasıl çalıştığı, amacının ne olduğu, hangi algoritmanın kullanıldığı gibi açıklamalar yıllar sonra sizin veya başkalarının bu kodları daha iyi anlamalarını ve düzeltme yapmalarını sağlayacaktır.

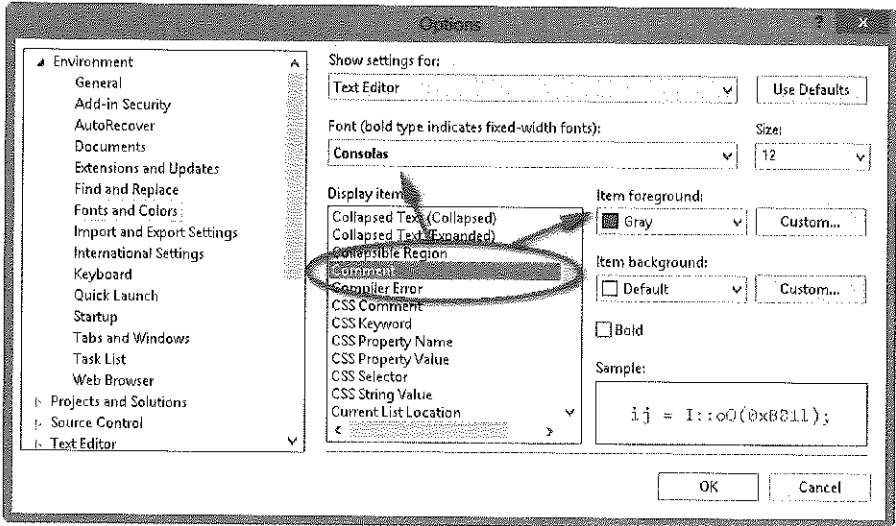
Bu kitapta gerekli görülen her satıra açıklama eklenecektir. Açıklamalar konuyu daha hızlı kavramanızı, kodları daha kolay anlamanızı ve daha iyi bir programcı olmanızı sağlayacaktır.

C# dilinde 2 farklı şekilde açıklama oluşturabilirsiniz. Bunlardan biri tek satırda açıklama veya yorum yazılmasını sağlayan (//) iki adet bölü karakteridir. Diğeri ise çoklu satır açıklama yapılmasını sağlayan (/ * ... */) bölü ve yıldız karakterleridir.

```
// Tek satır açıklama. Bu satır derlenmez
// bu satır sadece programcıya yöneliktir
// kodlarla ilgili açıklama ve yorum yapılmasını sağlar
```

```
/* Çok satır açıklama.
iki işaret (bölü-yıldız, yıldız-bölü) arasında kalan metinler
işlemci tarafından derlenmez
Buradaki kodlar programcıya yöneliktir. Kodlarla ilgili açıklama ve
yorumları barındırır
*/
```

VS'da varsayılan olarak açıklamalar yeşil renkle gösterilir. **Tools\ Options\ Environment\ Fonts and Colors\ Text Editor\ Comment** menüsünden açıklamaların yazıtipi ve rengini değiştirilebilirsiniz.



Şekil 5.2 Açıklama renginin ve fontunun ayarlanması

Aşağıdaki kodlarda örnek açıklama ifadelerini görmekteyiz. İlk açıklama çoklu satırdan oluşur ve daha çok kodların en başında programın işlevi ile ilgili uzun bilgiler vermek için kullanılır. Satır açıklamaları ise genelde fazla uzun değildir ve ilgili satırda yazılan kodun ne anlama geldiği veya işlevi ile ilgili ifadeleri barındırır.

```
class Program
{
    /*
    örnek çoklu açıklama satırları
    ekrana merhaba yazan program
    */
    static void Main(string[] args)
    {
        //konsola merhaba yaz
        Console.WriteLine("merhaba");

        Console.Read(); //pencereyi kapatma
    }
}
```


C# Yazım ve Noktalama Kuralları

Her dilde olduğu gibi C# dilinde de bazı yazım ve noktalama kuralları vardır. Doğal olarak tüm kodlarınızı bu kurallara göre yazmak zorundasınız. Eğer bu kurallara uymazsanız hata mesajıyla karşılaşabilirsiniz. Kodlardaki hata veya hataları düzeltmeden programı derleyemez ve dolayısıyla da çalıştıramazsınız.

Bu başlık altında size C# dilinin yazım ve noktalama kurallarını öğreteceğim. Bu kuralların çoğu sadece C# dilinde değil, aynı zamanda C tabanlı diğer dillerde de geçerlidir. Yani bu kuralların çoğu C++, Java gibi dillerde de var.

Noktalı Virgöl (;)

C# dilinde ve diğer C tabanlı dillerde her komut sonuna noktalı virgöl (;) koyulur.

```
int x=0, y, top=0;
x=x+1;
top=top+x;
```

Noktalı virgöl bir komutun bittiğini ve başka bir komutun başlayabileceğini belirtir. Örneğin yukarıdaki komutları aşağıdaki gibi de yazabiliriz.

```
int x=0, y, top=0; x=x+1; top=top+x;
```

Örneğin **Console.Write()** ile **Console.ReadLine()** komutlarını yan yana yazabileceğiniz gibi alt alta da yazabilirsiniz.

```
Console.Write("Adınız:"); ad=Console.ReadLine();
```

veya;

```
Console.Write("Adınız:");
ad=Console.ReadLine();
```

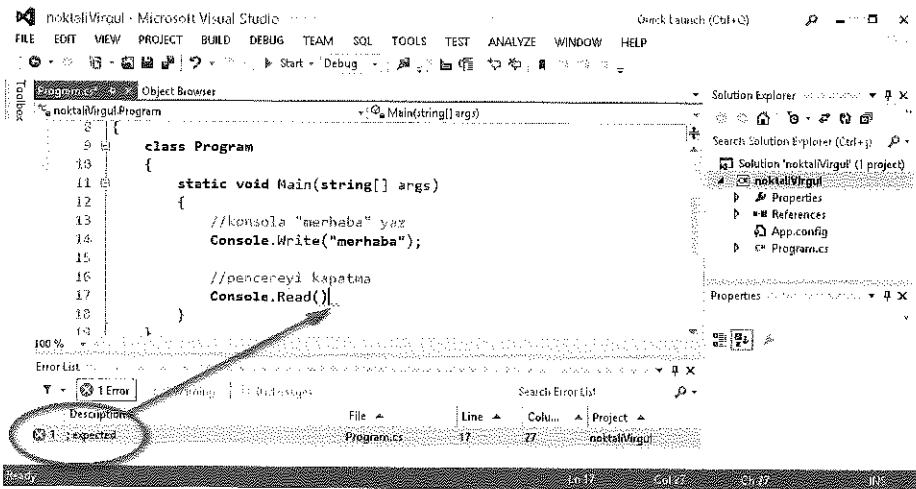
Komutları noktalı virgülden sonra aynı satıra yazmak programımızın daha küçük bir alana sığmasını sağlar ama aynı zamanda da daha karışık görünmesini sağlar. Bu da bizim kodlarımızı okuyan diğer programcıların programı anlamasını zorlaştırabilir.

Programın daha iyi okunabilmesi için genelde bir satıra birden fazla komut yazmak yerine tek komut yazılır. Tek satıra yazılan kodlar daha sade görünür ve nispeten daha kolay anlaşılır. Bu da bizim programımızın kalitesini arttıran etkenlerden biridir.

Uzun lafın kısıası, programcılıkta marifet aynı satıra çok fazla komut yazmak değil, komutların ve dolayısıyla programın daha kolay anlaşılmasını sağlamaktır. Bu konu eğitimde daha da önemli hale gelmektedir. Sizin yazdığınız kodları bir öğrenci veya öğretmen okuyup değerlendirme yapacağına göre kodları açık ve anlaşılır şekilde yazmak çok daha önemlidir.

Eğer bir program başka biri tarafından yazıldıysa ve siz bir programcı olarak bu kodlar üzerinde değişiklik ya da güncelleme yapmak zorunda kaldıysanız ne demek istediğimi çok daha iyi anlayacaksınız. Birçok programcı böyle bir durumla karşılaştığında sıkıntı yaşar ve saatler yerine günler harcamak zorunda kalır.

Noktalı virgülün unutulması C# dilinde en çok karşılaşılan hatalardan biridir. Eğer noktalı virgül koymayı unutursanız aşağıdaki şekilde de görüldüğü gibi **Error List** penceresinde “; expected” hatasını görürsünüz.



Şekil 5.3 Noktalı virgül hatası

Error List penceresi hangi karakterin eksik olduğunu, eksik karakterin hangi satırda (Line) ve sütunda (Column) olması gerektiğini çok açık bir şekilde belirtir. Eğer hatanın üzerine çift tıklarsanız faremin imlecini ilgili yere götürür. Noktalı virgülü koyduktan sonra **Error List** penceresindeki hata iletisi kaybolur.

Güzel (Süslü) Parantezler {...} ve Blok Kavramı

C# dili bloklardan oluşur. Blokların başlangıç ve bitiş yerini de adına güzel veya süslü parantez denilen işaretler belirler. Örneğin **Main()** isimli ana programımız **{}** işareti ile başlar ve **{}** işareti ile biter.

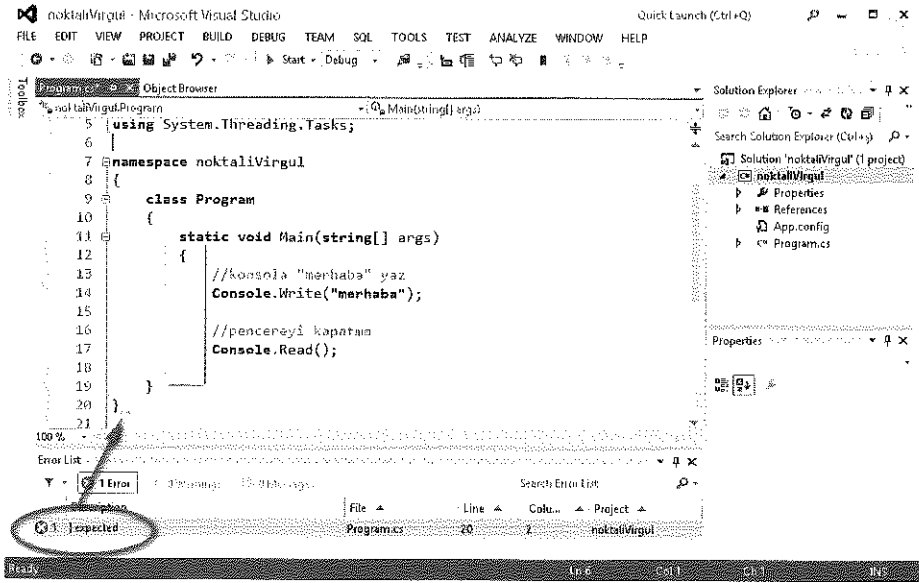
```
int void Main()
{ //blok başlangıcı
    Komut1
    Komut2
    ...
} //blok bitişi
```

{ } işaretleri içinde bulunan komutlara blok denir. Bloklar belli bir amaç için bir araya getirilmiş komutlardan oluşur ve ilgili kodların ayırt edilmesini sağlar. Ayrıca bloklar programın daha rahat okunabilmesine de yardımcı olur.

Açılan her parantez ya da blok mutlaka kapatılmalıdır; eğer kapatmazsanız hata alırsınız ki bu hata da çok karşılaşılan bir diğer hatadır. Bu nedenle bir güzel parantez açarsanız hemen içine kod yazmadan kapatın ki daha sonra kapatmayı unutmayasınız.

Aşağıdaki programda **Error List** penceresinde parantez kapatma karakteri beklenmediğine dair bir hata uyarısı görmektesiniz. Buradaki kodlarda parantez karakteri kasıtlı olarak silinmiştir ve hatanın meydana gelmesi sağlanmıştır. Kodlara dikkat edilirse programda üç parantez açılmış ama iki parantez kapatılmıştır. Eğer bir parantez daha koyarsanız hata hallolacaktır.

Aynı noktalı virgül karakterinde olduğu gibi parantezlerde de eksik karakterin satır ve sütun numarası belirtilir. Eğer **Error List** penceresindeki hata uyarısına çift tıklarsanız fare imleci eksik karakterin olduğu yere gider. Eksik karakteri yazdıktan sonra hata uyarısı kaybolur.



Şekil 5.4 Parantez hatası

Boşluk Karakteri

C# dili için boşluk karakterlerinin bir anlamı yoktur. Yani C# dilinde boşluk karakterleri derlenmez. O nedenle istediğiniz kadar boşluk karakteri kullanabilirsiniz.

Programın daha iyi okunabilmesi ve dolayısıyla diğer herkes tarafından daha iyi anlaşılabilmesi için gerekli olan yerlere boşluk karakteri veya boş satırlar koyulmalıdır.

Örneğin bir programı aşağıdaki gibi yazdığınızı düşünelim.

```
int void Main(){Console.WriteLine("merhaba");Console.Read();}
```

Yukarıdaki program ekrana "merhaba" yazar ve bekler. Tüm komutlar yan yana ve boşluk kullanılmadan yazıldığı için okunması biraz zordur. Yukarıdaki programı boşluk ve alt satır kullanarak yazarsak aşağıdaki gibi olur.

```
int void Main()
{
    Console.WriteLine("merhaba");
    Console.Read();
}
```

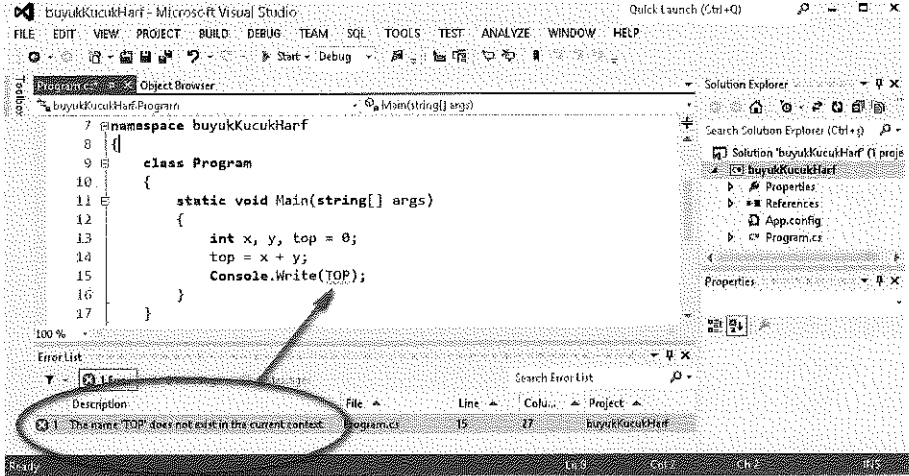
Görüldüğü gibi program artık çok daha rahat okunabilmektedir. Dikkat edilirse, yukarıdaki programda kodlar farklı satırlara dağıtılmış ve blok içindeki komutlar daha içeriye yazılarak okunabilirlik artırılmıştır. Eğer editör olarak Visual Studio kullanırsanız Visual Studio boşlukların düzenli kullanılmasını kendisi halletmeye çalışacaktır.

Büyük-Küçük Harf Kullanımı

C# dili büyük-küçük harf duyarlı bir dildir. Yani C# dili için “ali” ile “Ali” farklı şeylerdir.

```
int x=1, y=2, top=0;
top=x+y;
Console.Write(TOP);
```

Yukarıdaki programın ilk satırında **x**, **y** ve **top** isimli üç değişken tanımlanmıştır. Sonraki satırda **x** ile **y** değişkenleri toplanıp sonuç **top** değişkenine aktarılmıştır. Son satırda ise **top** değişkeninin değeri ekrana yazılmak istenmiştir. **Console.Write()**; komutunda “**top**” yerine yanlışlıkla “**TOP**” yazılmıştır. “**top**” ile “**TOP**” birbirinden farklı şeyler olduğu için aşağıda görülen hata oluşmuştur.



Şekil 5.5 Değişken hatası

Örneğin **Console.Write()** komutunu küçük harflerle **console.write()** şeklinde kullanmaya kalkarsanız hata alırsınız. Çünkü bizim için pek fark etmese de C# dilinde **Console** ile **console** birbirinden farklı şeylerdir. Bu tür hatalardan kaçınmak için mümkün olduğunca Visual Studio **Intellisense** özelliğini kullanmanızı tavsiye ederim. Bu özellik hem komutların tamamlanmasını hem de düzgün yazılmasını sağlar.

Veri Giriş-Çıkış Komutları

Bu başlık altında sizlere veri giriş-çıkış ve biçimlendirme komutlarını anlatacağım. Daha önceki algoritma konusunda da gördüğünüz gibi, bilgisayarın işlem yapabilmesi için önce dıranıdan gerekli olan veriler alınmalıdır. Bizim çalışma ekranımız **Console** penceresi olduğu için bu bölümde sizlere konsol ekranından yani komut satırından verileri nasıl okuyacağımızı ve yaptığımız bir takım işlemler sonucunda elde ettiğimiz değerleri konsola nasıl yazdıracağımızı anlatacağım.

Bundan sonra anlatılan örnekleri iyice okuyun ve yorumlamaya çalışın. Kodları yazıp deneyerek öğrendiklerinizi iyice pekiştirin. Böylece hem öğrendiğinizi hissederek özgüveniniz artacak hem de sonraki konuları daha rahat anlayacaksınız.

Lütfen Dikkat!

Eğer kiptaktaki örnek kodları Visual Studio'da yazıp çalıştırırsanız, yazdığınız kodlar çalıştıktan sonra konsol penceresi otomatik olarak kapanacak ve program sonucu görünmeyecektir. Konsol penceresinin kapanmaması için yazdığımız kodların en sonuna **Console.Read();** veya **Console.ReadLine();** komutunu koyacağız. Bu komutlar konsola veri yazılmasını ve ENTER'lanmasını beklediği için biz pencereyi kapatıncaya veya ENTER'layıncaya kadar pencere açık kalacaktır.

Veri Çıkışı

Program içinde üretilen bir değeri konsol ekranına yazdırmak için **Console** sınıfının **Write()** veya **WriteLine()** metodu kullanılır.

```
Console.Write();           //yaz
Console.WriteLine();      //yaz ve alt satıra git
```

İki komut arasında çok basit fakat çok karıştırılan bir fark vardır; **Write()** metodu istenilen değeri yazdırır ve imleci aynı satırda bırakır. **WriteLine()** metodu ise istenilen değeri yazdıktan sonra imleci alt satıra götürür ve işlemler o satırdan devam eder. İmleç neredeyse yazdırma işlemi de oraya yapılır.

Örnek Kod:

```
Console.Write("Merhaba C#");
```

Çıktı:

```
Merhaba C#_ (metin ve imleç aynı satırda)
```

Aşağıdaki örnekte ise aynı işlem **WriteLine()** metodu ile yapılmıştır. **WriteLine()** metodunda yazma işlemi bittikten sonra imleç alt satıra geçer.

Örnek Kod:

```
Console.WriteLine("Merhaba C#");
```

Çıktı:

```
Merhaba C#
_ (imleç alt satırda)
```

Aşağıdaki örnekte alt alta iki adet **Write()** metodu görüyorsunuz. Alt alta oldukları için metinleri de alt alta yazacaklarını düşünebilirsiniz ama maalesef öyle değil. **Write()** metodunda imleç aynı satırda kaldığı için hem "**Merhaba**" hem de "**Dünya**" aynı satıra yazılır.

Örnek Kod:

```
Console.Write("Merhaba"); //Merhaba yaz ve imleci aynı satırda bırak
Console.Write(" Dünya"); //imlecin olduğu yere Dünya yaz
```

Çıktı:

```
Merhaba Dünya
```

Yukarıdaki örneği **Write()** ve **WriteLine()** metotlarını kullanarak yaparsak alt alta yazdırma şansımız olabilir. **Merhaba** metni **WriteLine()** metodu ile yazdırılır. Böylece imleç alt satıra gider. Daha sonra da **Write()** metodu ile **Dünya** yazdırılır. Bu durumda kodlar ve çıktısı aşağıdaki gibi olur.

Örnek Kod:

```
Console.WriteLine("Merhaba"); //Merhaba yaz ve alt satıra git
Console.Write("Dünya"); //imlecin olduğu satıra (alt satıra) Dünya yaz
```

Çıktı:

```
Merhaba
Dünya _
```

Örnek Kod:

```
Console.WriteLine("Merhaba"); Console.WriteLine(" Dünya");
```

Çıktı:

```
"Merhaba
Dünya"
```

Özel Karakterler

Veri çıkışıında değişik işlem ve biçimlendirmeler yaparken özel karakterler de kullanılabilir. Özel karakterler alt satıra geçme, tırnak içinde yazdırma, tab, bip sesi çıkarma vb. işlevler sağlar. Çok kullanılan özel karakterler aşağıdaki tabloda görülebilir:

Karakter	Açıklama
\n	Alt satıra geç
\r	Satır başına git
\t	Tab (Sekme)
'	Tek tırnak
"	Çift tırnak
\\	Ters bölü
\?	Soru işareti
\a	Bip sesi

Aşağıda özel karakterler ile ilgili değişik örnekler görülmektedir. İlk örneğimizde (\n) karakterini kullanarak kendisinden sonra gelen metinleri alt satırda gösteriyoruz.

Örnek Kod:

```
Console.WriteLine("Adım Fahrettin\n Soyadım Erdinç");
//Adım Fahrettin yazdıktan sonra \n karakterinden dolayı alt satıra gider
```

Çıktı:

```
Adım Fahrettin // \n karakterinden dolayı imleç alt satıra gider
Soyadım Erdinç
```

Eğer metinler arasında bir sekme (tab) boşluk bırakmak isterseniz (\t) karakterini kullanabilirsiniz.

Örnek Kod:

```
Console.WriteLine("Adım Fahrettin\t Soyadım Erdinç");
//Adım Fahrettin yazdıktan sonra bir sekme boşluk bırak
```

Çıktı:

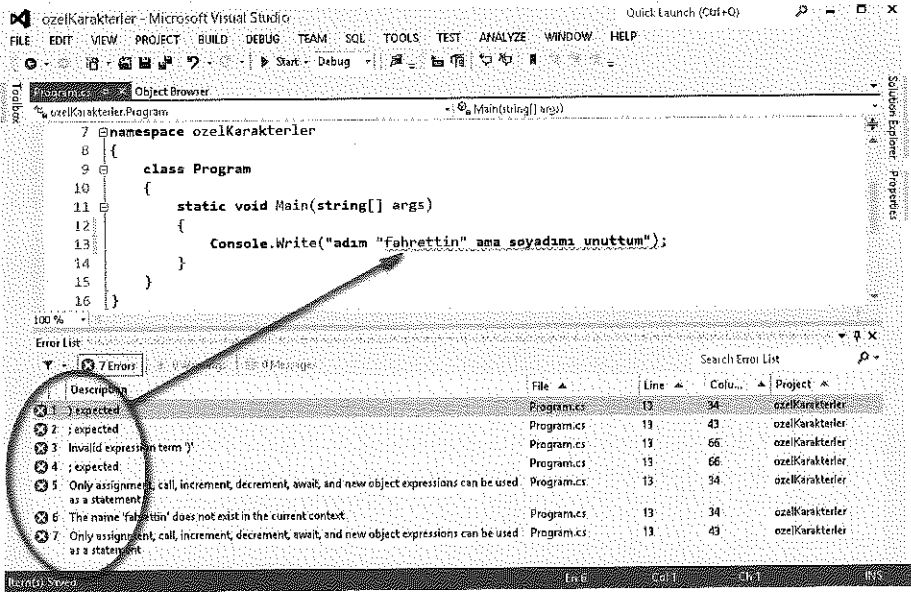
```
Fahrettin      Erdinç
```

En çok hata yapılan ve karıştırılan karakterlerden biri çift tırnak (") karakteri, diğeri ise tek tırnak (') karakteridir. Bu karakterler özellikle veritabanı işlemlerinde sıkça karşımıza çıkmaktadır. Eğer bu karakterlerin kullanımını doğru düzgün anlamazsanız veritabanı işlemlerinde saç baş yolacağınıza emin olabilirsiniz.

Aşağıda birkaç örnekle sizlere bu karakterleri anlatacağım ve eğer hata yaparsanız alacağınız hata uyarılarını göstereceğim.

Bu karakterlerin hatalı kullanımında o kadar çok hata mesajı alırsınız ki sanki bütün dijital dünya size karşı ve program yazdırmak istemiyor sanırsınız. Eğer bir metni "çift tırnak" içine yazmak istiyorsanız ve aşağıdaki gibi yazarsanız hata alırsınız. Hem de öyle bir hata ki neye uğradığınızı şaşırırsınız.

```
Console.Write("adım "fahrettin" ama soyadımı unuttum..");
```



Şekil 5.6 Error list penceresindeki hatalar

Error List penceresine bakacak olursanız yedi adet hata bulunmuştur ve sizden bir sürü düzeltme beklenmektedir. Eğer (") karakterini bilmezseniz çok uğraşırsınız. Eğer kodu aşağıdaki gibi yazarsanız tüm hatalardan kurtulursunuz.

```
Console.Write("adım \"Fahrettin\" ama soyadımı unuttum..");
```

Sanırım özel karakterlerin hayat kurtardığını artık anlamaya başladınız. Aşağıdaki örnekte (") karakteri ile iki adet metni çift tırnak içine alıyoruz.

Örnek Kod:

```
Console.Write("Adım \"Fahrettin\" Soyadım \"Erdinç\"");
```

Çıktı:

Adım "Fahrettin" Soyadım "Erdoğan"

Eğer ekrana ters bölü (\) karakteri yazdırmak istiyorsanız bunu (\\) özel karakteri ile yapıyorsunuz. Örneğin bir dosyanın ya da klasörün yolunu göstermek istiyorsanız aşağıdaki örneği kullanabilirsiniz.

Örnek Kod:

```
Console.Write("Klasör C:\\Deneme adresinde bulunur");
// "\\" ile çıktıda tek bir \ (ters bölü) karakteri elde ederiz
```

Çıktı:

Klasör C:\Deneme adresinde bulunur

Eğer kullanıcıların dikkatini çekerek ekrana bir şeyler yazmak istiyorsanız (\a) karakterini kullanabilirsiniz. Bu karakter "bip" sesi gibi sesli bir uyarı ile kullanıcının dikkatini çekmeye yarar.

Örnek Kod:

```
static void Main(string[] args)
{
    int x = 1, y = 2, top = 0;
    top = x + y;
    Console.WriteLine("\a toplam="+top); //uyarı ziliyle toplamı konsola yaz

    Console.Read(); //pencereyi kapatma
}
```

Sözel Toplama

Adından da belli olduğu gibi metinleri veya değişkenleri yan yana toplamak veya başka bir ifade ile yan yana yazdırmak için kullanılır. Sözel toplamada artı (+) karakteri aşağıdaki gibi kullanılır. Dikkat edilirse metinler çift tırnak içinde ve eğer metinden sonra değişken ya da değişkenden sonra metin gelecekse mutlaka artı (+) işareti koyulmuştur.

```
Console.Write("metin1" + "metin2" + değişken + "metin3");
```

Aşağıdaki örneklerde ekrana metin ve değişkenlerin nasıl yazılacağını görebilirsiniz.

Örnek Kod:

```
string ad="Fahrettin";
Console.WriteLine("Adım "+ad);
```

Çıktı:

Adım Fahrettin

Örnek Kod:

```
string ad= "Fahrettin", soyad= "Erdoğan";
Console.Write("Adım " + ad + "\t Soyadım " + soyad);
//Adım Fahrettin yazdıktan sonra bir sekme ilerle ve Soyadım Erdoğan yaz
```

Çıktı:

Adım Fahrettin Soyadım Erdoğan

Veri Girişi

Console penceresinde sadece tek bir aktif satır vardır. Bu aktif satıra yazılan değerleri okumak için **Console** nesnesinin **Read()** ve **ReadLine()** metotları kullanılır.



Şekil 5.7 Aktif satır

```
Console.Read(); //komut satırındaki ilk karakteri ASCII (int olarak) okur
Console.ReadLine(); //komut satırındaki veriyi metin (string olarak) okur
```

Dikkat ederseniz iki metot arasında çok önemli bir fark vardır. **Read()** metodu komut satırından tek bir karakter okurken **ReadLine()** metodu tüm satırı yani tüm karakterleri okur.

Read() metodu tek karakter okuduğu için kullanıcıyı kısıtlar ve pek kullanılmaz. O nedenle genelde konsoldan veri okumak için **ReadLine()** metodunu kullanacağız ve aktif satırdaki (ENTER tuşuna basılıncaya kadar) tüm değerleri alacağız.

ReadLine() metodunu kullanırken unutulmaması gereken bir durum vardır; bu metot komut satırında yazılan her şeyi **metin** yani **string** veri tipinde kabul eder. Eğer konsoldan "ali" veya "merhaba" gibi metinler okursanız sorun yok. Fakat **3**, **6.34**, **16 Haziran 2013** gibi sayısal veya tarihsel değerler okursanız onları da metin kabul

eder. Dolayısıyla girilen veriler üzerinde doğrudan sayısal veya tarihsel işlemler yapamayız. Okunan değerlerin önce uygun veri tipine dönüştürülmesi gerekir. Bu konuyu önümüzdeki konularda daha ayrıntılı inceleyeceğiz.

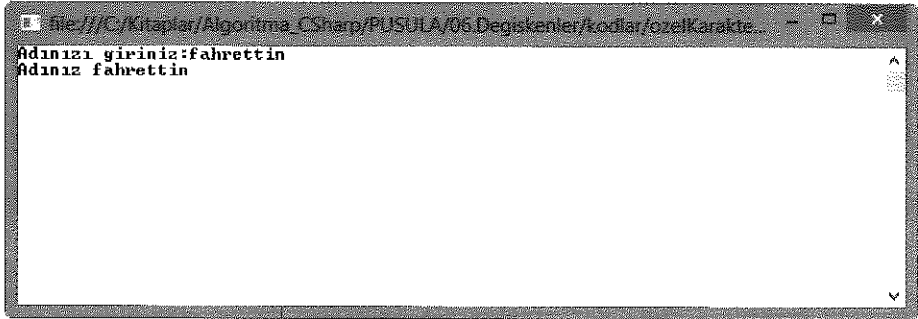
Aşağıdaki örnekte **Console.ReadLine()** komutu ile ekrana yazılan adı/metni alıyoruz ve **ad** değişkenine aktarıyoruz. Ekrandan okunan değer zaten **string** yani **metin** kabul edildiği için **ad** değişkenine aktarırken bir sorun yaşamıyoruz. Daha sonra da **Console.Write()** komutu ile aldığımız adı ekrana yazıyoruz.

Örnek Kod:

```
string ad; // string tipinde ad değişkeni oluştur
Console.Write("Adınızı giriniz:"); //ekrana "Adınızı giriniz:" yaz
ad=Console.ReadLine(); //ekrana yazılanları oku ve ad değişkenine ata
Console.Write("Adınız "+ad); // ad değişkenini yazdır

Console.ReadLine(); //pencereyi kapatma, bekle
```

Çıktı:



Şekil 5.8 Örnek kodun çalışması ve ekran çıktısı

Aşağıdaki örnekte sayısal (**int**) tipte **sayi** adlı bir değişken tanımlıyoruz. Daha sonra da konsoldan alınan değeri **sayi** değişkenine aktarıyoruz. Konsoldan okunan her değer **string** yani **metin** tipinde olduğu için **int** tipindeki **sayi** değişkenine doğrudan atayamayız. Her değişkene ancak kendi tipinde değer atayabilirsiniz.

Örnek Kod:

```
int sayi; //integer (tamsayı) tipinde sayi isimli bir değişken oluştur
Console.Write("Bir sayı giriniz:");
sayi=Console.ReadLine(); //yazılan değeri al ve sayi değişkenine ata
Console.Write(sayi+1); //sayi değerini 1 arttırıp ekrana yaz
Console.Read(); //pencereyi kapatma
```

Çıktı:

Hata verir!

Tekrar hatırlatayım! **ReadLine()** metodu konsola yazılan her şeyi metin olarak algılar. Bizim oluşturduğumuz **sayi** değişkeni sayısal (**int**) tipte olduğundan “ahmet” veya “mehmet” gibi **metinsel** değerleri kabul etmez. Bu nedenle bu program hata verir. Hata şudur: **Cannot implicitly convert type int to string**. Yani, sözel bir veriyi sayısal bir değişkene atayamazsınız der...

Değişkenler ve veri tiplerine önümüzdeki konularda daha ayrıntılı değineceğiz ve bu tür hatalardan nasıl kurtulacağımızı öğreneceğiz.

Değişkenler

Değişkenler, dışarıdan alınan veya program içerisinde üretilen değerleri geçici olarak saklamaya yarayan belleklerdir.

Saklanmak istenen verinin boyutuna ve tipine göre değişkende farklı boyutta ve tipte olmalıdır. Aşağıdaki tabloda dışarıdan okunabilecek bazı verileri ve tiplerini görebilirsiniz:

Okunan Değer	Değişken Veritipi
12	0-255 arası küçük tamsayı (byte)
19500	0-65535 arası tamsayı (short)
3255000	0-4 milyar arası (int)
3,1415	Ondalıklı sayı (float)
07/07/1972	Tarih (date)
Selam	Metin (string)
1 veya 0	Mantıksal (bool)

Dikkat etmeniz gereken en önemli konu dışarıdan alınan değer ile atanacak değişkenin uyumlu olmasıdır:

- Eğer dışarıdan 5 gibi bir tamsayı alıyorsanız bu tamsayının atanacağı değişken de tamsayı tipinde (byte, short, int) olmalıdır.
- Eğer dışarıdan 3,14 gibi ondalıklı bir sayı alıyorsanız kullanılacak değişkenin de ondalıklı (float, double) olması gerekiyor.
- Eğer dışarıdan “Ahmet”, “Merhaba” gibi bir metin alıyorsanız atanacak değişkenin sözel (string) tipinde olması gerekiyor.

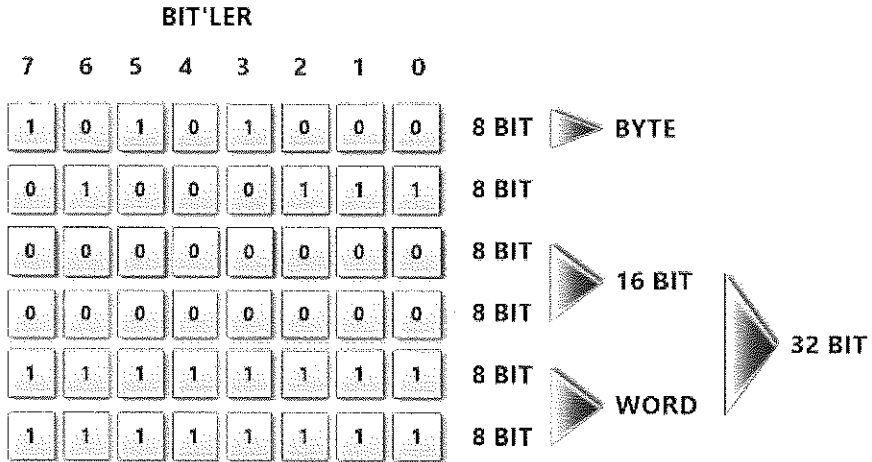
Eğer verileri uygun tipteki değişkenlere atamazsanız bir önceki örneğimizde de karşımıza çıkan **Cannot implicitly convert type...** şeklinde bir hata alırsınız. Bu hata size veri ile değişkenin uygun tipte olmadığını söyler.

Uzun lafın kisası, bir değişkene değer atarken atamanın her iki tarafındaki verinin aynı tipte olmasına dikkat edin. Eğer aynı değilse uygun veri tipine çevirmeniz gerekmektedir. Veri tipi değiştirme **Convert()** metodu ile yapılır fakat bu konuyu daha ilerde göreceğiz.

Değişken (aynı veri tipi) = Değer (aynı veri tipi);

Değişkenlerin Fiziksel Yapısı

Kısaca biraz da değişkenlerin fiziksel yapısından bahsetmek istiyorum. Değişkenler RAM bellekteki raf benzeri alanlarda saklanır. 8 tane 0 veya 1 bilgisini saklayan hafıza (bit) bir araya gelir ve 1 byte'lık hafızayı oluşturur. RAM bellekte 1 byte'lık hafızalar alt alta raf gibi dizilmişlerdir.



Şekil 5.9 RAM belleğin fiziksel yapısı

Bazı bilgiler:

- Bir bit'teki (kutucuktaki) değer ya 0 ya da 1 olabilir.
- Eğer bir satırdaki (byte) tüm bitler 0 olursa onluk değeri de 0 eder. Eğer bir satırdaki tüm bitler 1 olursa onluk değeri 255 eder.
- Eğer 16 bitin (alt alta 2 byte) hepsi 0 olursa onluk karşılığı 0 eder. Hepsi 1 olursa 65.535 olur.
- Eğer 32 bitin (alt alta 4 byte) hepsi 0 olursa onluk karşılığı 0 eder. Hepsi 1 olursa 4.294.967.295 (dört milyar) eder.

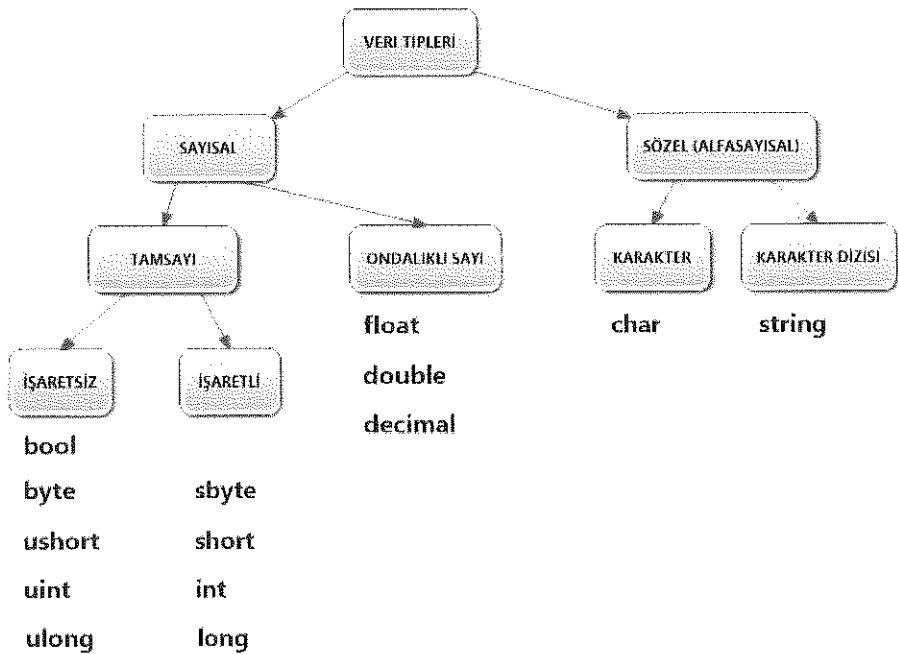
- A,b,@ gibi Unicode karakterlerin her biri RAM bellekte 2 byte yer kaplar. Yani "Ali" kelimesi RAM bellekte 3 harf x 2=6 byte yer kaplar.

RAM bellek yapısı özellikle değişkenler ve veri tipleri için önemlidir. Belleğe kaydedilen farklı türdeki veriler farklı miktarda alan kaplar. Aşağıda değişik örnekler bulabilirsiniz:

- Bay/bayan bilgisi 0/1 olarak kaydedilecekse her kayıt RAM bellekte 1 bit yer kaplar.
- Evli/bekar bilgisi 0/1 olarak kaydedilecekse her kayıt RAM bellekte 1 bit yer kaplar.
- 0-255 arasındaki değerler RAM bellekte 1 byte yani 8 bit yer kaplar; örneğin öğrencilerin 0-100 arasında notu girilecekse her not RAM bellekte 1 byte yer kaplar.
- 0-65.535 arası değerler RAM bellekte 2 byte yani 16 bit yer kaplar. Örneğin 0-5.000 kg arasında değişen yük konteynır ağırlığını kaydetmek için RAM bellekte 2 byte yer ayırmak gerekir.
- 0-4 Milyar arası değerler RAM bellekte 4 byte yani 32 bit yer kaplar. Örneğin cirosu 1 ile 4 milyar arasında değişen bir şirketin bilgisini kaydetmek için RAM bellekte 4 byte (32 bit) yer ayırmak gerekir.

Veri Tipleri

Veri tipleri bilgisayarların işlediği verilerin nasıl bir yapıya sahip olduğunu ifade eder. C# dilinde genel olarak üç farklı veri tipi vardır: tamsayı, ondalıklı sayı ve metin. Bu veri tipleri de kendi aralarında alt veri tiplerine ayrılırlar. Aşağıdaki diyagramda C# dilinde kullanılan veri tiplerini görebilirsiniz. Sözel verilerle ilgili sadece "char" ve "string" veri tipi varken sayısal verilerle ilgili çok daha fazla veri tipi olduğuna dikkatinizi çekerim.



Şekil 5.10 Veri tipleri

Bool Veri Tipi

En az yer kaplayan veri tipidir; RAM bellekte 1 bit yer kaplar ve bir değişkenin ya da ifadenin 1/0, evet/hayır, doğru/yanlış, true/false şeklinde temsil edilmesini sağlar. Örneğin bir şahsın medeni durumunu evet/hayır şeklinde ifade edebilirsiniz. Ya da bir siteye üyelik durumunu true/false şeklinde gösterebilirsiniz.

Bu veri tipinde eğer bir ifade doğru ise **1** veya **true** ile yanlış ise **0** veya **false** ile gösterilir. Örneğin **evli=true** ifadesi kişinin evli olduğunu belirtirken **uye=false** ifadesi kişinin üye olmadığını belirtir.

Byte Veri Tipi

RAM bellekte 8 bit yani 1 byte yer kaplar. Eğer 8 bitin hepsi sıfır ise onlu sayı sistemindeki karşılığı sıfır, hepsi 1 ise onlu sayı sistemindeki karşılığı 255'tir. Yani bu veri tipi 0-255 arasında değerler alır.

00000000 → 0

11111111 → 255

Eğer yazdığımız programda 0-255 arası küçük tamsayı değerler ile çalışacaksak en mantıklı şey bu veri tipini kullanmaktır. Örneğin 0-100 arası öğrenci notları ile çalışacaksak **short** veya **int** veri tipi yerine bu veri tipi kullanılmalıdır çünkü **int** RAM bellekte 4 byte yer kaplarken **byte** veritipi 1 byte yer kaplar. Sonuçta daha az bellek isteyen ve kaynakları daha iyi kullanan bir program yazmış oluruz.

Sbyte Veri Tipi

RAM bellekte 8 bit yani 1 byte yer kaplar. Byte kelimesinin başındaki "s" harfi "signed" yani "işaretli" kelimesinden gelmektedir. Bu da veri tipinin işaretli yani başına eksi (-) gelebileceğini gösterir.

İşaretli veri tiplerinde ikili sayının en solundaki rakam işaret biti olarak kullanılır. Eğer en soldaki bit 0 ise sayı pozitif, 1 ise sayı negatif olur. Bu durumda değerler 7 bit ile ifade edilir.

```
0-00000000 → 0
0-11111111 → 127
1-00000000 → -1
1-11111111 → -128
```

Sonuçta **sbyte** veri tipi -128 ile +127 arası tamsayı değerler alır.

Ushort Veri Tipi

RAM bellekte 16 bit yani 2 byte yer kaplar. Short kelimesinin en başındaki "u" harfi "unsigned" yani işaretsiz kelimesinden gelir ve sayının eksi (-) değer alamayacağını belirtir. Ushort veri tipinin alabileceği en yüksek ve en düşük değerler aşağıda ifade edilmiştir.

```
00000000 00000000 → 0
11111111 11111111 → 65535
```

Eğer çalışacağımız değer 0-65535 arası pozitif tamsayılar ise **ushort** veri tipi en iyi tercih olacaktır.

Short Veri Tipi

RAM bellekte 16 bit yer kaplar. Aynı **sbyte** veri tipinde olduğu gibi en soldaki bit sayının negatif veya pozitif olacağını belirler. Dolayısıyla bu veri tipinde değerler 15 bit ile ifade edilir.

```

0-00000000 00000000 →0
0-11111111 11111111 →32767
1-00000000 00000000 →-1
1-11111111 11111111 →-32768

```

Yukarıdaki hesaptan da anlaşılacağı üzere **short** veri tipi -32768 ile +32767 arası tamsayı değerler alabilir.

UInt Veri Tipi

32 bitlik bir veri tipidir. RAM bellekte 4 byte yer kaplar. UInt kelimesinin en başındaki “u” harfi “unsigned” yani işaretsiz kelimesinden gelir ve sayının eksi (-) değer alamayacağını belirtir. UInt veri tipinin alabileceği en yüksek ve en düşük değerler aşağıda ifade edilmiştir.

```

00000000 00000000 00000000 00000000 →0
11111111 11111111 11111111 11111111 →4294967296

```

Eğer çalışacağımız değer 0-4294967296 (0-4 milyar) arası pozitif tamsayılar ise **uint** veri tipi en iyi tercih olacaktır.

Int Veri Tipi

32 bitlik bir veri tipidir ve aynı **uint** gibi RAM bellekte 4 byte yer kaplar. En soldaki bit sayının negatif veya pozitif olacağını belirler. Dolayısıyla bu veri tipinde değerler 31 bit ile ifade edilir.

```

0-00000000 00000000 00000000 00000000 →0
0-11111111 11111111 11111111 11111111 →2147483647
1-00000000 00000000 00000000 00000000 →-1
1-11111111 11111111 11111111 11111111 →-2147483648

```

Eğer -2 milyar ile +2 milyar arası tamsayılar ile çalışıyorsanız tercih etmeniz gereken veri tipi **int** olmalıdır.

Ulong Veri Tipi

RAM bellekte 64 bit yer kaplar ve çalışabileceğiniz en büyük pozitif tamsayı değerlerini barındırır. Eğer 4 milyardan daha büyük tamsayılar ile çalışmanız gerekiyorsa 0-18446744073709551615 arası değerler alabilen **ulong** veritipini kullanmak zorundasınız.

Long Veri Tipi

Aynı **ulong** veri tipi gibi RAM bellekte 64 bit yer kaplar. En soldaki bit sayının negatif veya pozitif olacağını belirler. Dolayısıyla bu veri tipinde değerler 61 bit ile ifade edilir.

Eğer 4 milyardan daha büyük pozitif veya negatif sayılarla çalışmanız gerekiyorsa -9223372036854775808 ile +9223372036854775807 arası değerler alabilen **long** veri tipi en iyi tercihtir.

Float Veri Tipi

Eğer tamsayılarla değil de adına ondalıklı ya da kesirli denilen sayılarla çalışacaksanız **float** veri tipini kullanabilirsiniz. RAM bellekte 32 bit yer kaplar ve -3.402823e38 ile 3.402823e38 arası sayılarla çalışmamızı sağlar. Buradaki "e" harfi üslü sayıları temsil eder ve e3 üslü sayılardaki 10^3 anlamına gelir.

Double Veri Tipi

RAM bellekte 64 bit yer kaplar ve **float** veri tipinden daha küçük veya daha büyük sayılarla çalışmamızı sağlar. Tam olarak ifade etmek gerekirse -1.79769313486232e308 ile 1.79769313486232e308 arası sayılarla çalışmamızı sağlar.

Decimal Veri Tipi

Decimal veri tipi hassasiyetin çok yüksek olması gereken bilimsel hesaplamalarda kullanılır. Virgülden sonra 28 karakteri tam olarak saklayabilir. Oysa **float** veri tipi 6, **double** veri tipi ise 15 karakter saklar.

Eğer float veri tipinde virgülden sonra 10 karakterli bir kesirli sayı girerseniz hassasiyeti 6 basamak olduğu için virgülden sonraki son 4 basamağı yuvarlar. Örneğin float veri tipi 3,534534545123 şeklindeki bir değeri yuvarlatarak 3,534535 şeklinde virgülden sonra 6 basamak saklayan bir değere çevirir. Eğer son 4 basamak da sizin için önemliyse virgülden sonra 28 basamak hassasiyete sahip **decimal** veri tipi kullanmanız gerekir.

Char Veri Tipi

Eğer bellekte 'a', '@', 'e', '\$' gibi tek bir karakteri saklamak istiyorsanız char veri tipini kullanmanız gerekir. Bu veri tipi **unicode** karakterleri yani tüm dillerdeki sayısal ve alfasayısal karakterleri saklayabilir ve RAM bellekte 16 bit yer kaplar.

String Veri Tipi

Eğer kelime, cümle veya paragraf gibi değerleri bellekte saklamak istiyorsanız **string** veri tipini kullanmak zorundasınız. Uzun lafın kısıası tüm sözel ifadeleri bu veri tipi ile saklıyoruz. Bu veri tipinde saklanan her karakter RAM bellekte 2 byte yer kaplar. Eğer daha önce C ile uğraştıysanız C dilinin sadece İngilizce harfleri kabul ettiğini biliyorsunuzdur. C# dili Unicode yani çok dilli karakterleri de destekler. Bu nedenle C#’ta string tipindeki değişkenlere ü, ş, ç, ı, ğ gibi Türkçe karakterleri de atayabilirsiniz.

Object Veri Tipi

En son anlattığım bu veri tipi aslında C# dilinde tüm veri tiplerinin türediği veri tipidir. C# tamamen nesne tabanlı bir dil olduğu için, şu ana kadar öğrendiğimiz veri tipleri ve değişkenler de aslında birer nesnedir. C# dilinde tüm veri tipleri **object** tipinden türetilir. Konuyla ilgili daha ayrıntılı bilgileri ileriki sayfalarda öğrenebilirsiniz.

Aşağıdaki tabloda farklı tipteki değişkenlerin alabileceği değerleri ve RAM bellekte kapladığı alanı özet olarak görebilirsiniz:

Değişken Tipi	Alan (bit)	Alabileceği Değer
bool	1	0/1, true/false
byte	8	0:255
sbyte	8	-128:127
short	16	-32.768:32.767
ushort	16	0:65.535
int	32	-2.147.483.648:2.147.484.647
uint	32	0:4.294.967.295
long	64	-9,223,372,036,854,775,808:9,223,372,036,854,775,807
ulong	64	0 : 18,446,744,073,709,551,615
float	32	-3.402823e38:3.402823e38
double	64	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$
decimal	128	$\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$
char	16	Tek bir karakter
string	--	Her karakter 16 bit
object		

Bu kitapta yapacağımız örneklerde kolaylık olsun diye genellikle tamsayı değerler için **int**, ondalıklı değerler için **double** ve metinler için de **string** veri tipini kullanacağız.

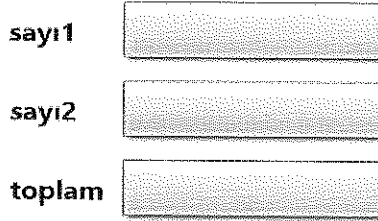
Değişken Oluşturma ve Değer Atama

Artık değişkenlerin RAM'de belli bir isme sahip bir bellek olduğunu biliyoruz. Örneğin klavyeden girilen 2 tamsayının toplamını bulmak istersek bu verileri saklamak için 3 adet değişkene ihtiyaç duyarız; birinci sayı, ikinci sayı ve toplam değişkenleri. Bu örneği size daha görsel bir şekilde anlatmaya çalışayım.

1. Öncelikle pseudo kod kullanarak 3 adet değişken tanımlayalım:

```
tanımla sayı1, sayı2, toplam
```

Yukarıdaki komut sonucunda RAM bellekte **sayı1**, **sayı2** ve **toplam** isimli 3 adet içi boş bellek oluşur. Bu bellekler Şekil 5.11'de olduğu gibi görünür. Herhangi bir atama olmadığı için değişkenlerin içleri boştur yani herhangi bir değere sahip değildir.

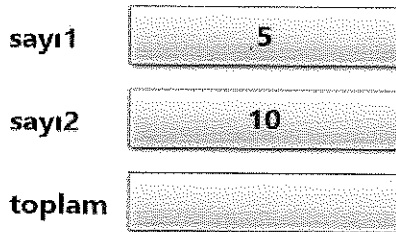


Şekil 5.11 Değişken tanımlama

2. Değişkenler tanımlandıktan sonra bu değişkenlere değer atanır. Değer atamak için aşağıdaki pseudo kodu kullanalım:

```
sayı1=5  
sayı2=10
```

Yukarıdaki kodlar sonucunda değişkenler ve içerikleri aşağıdaki şekilde görüldüğü gibi olur. Dikkat edilirse **sayı1** değişkeninin içinde **5** değeri, **sayı2** değişkeninin içinde ise **10** değeri vardır. **toplam** değişkenine henüz bir atama yapılmadığı için içi boştur.

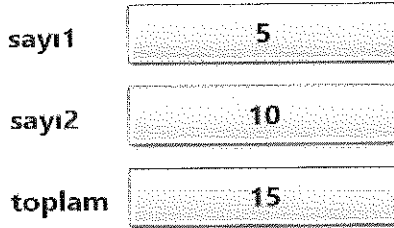


Şekil 5.12 Değişkenlere değer atama

3. **sayı1** ve **sayı2** değişkenlerine değer atandıktan sonra bu değişkenleri kullanarak farklı işlemler yapabiliriz. Örnek olarak **sayı1** ve **sayı2** değişkenlerini toplayalım ve toplama sonucunu **toplam** değişkenine atayalım. Toplama işlemi için aşağıdaki pseudo kod kullanılabilir:

```
toplam=sayı1+sayı2
```

Yukarıdaki kod satırında **sayı1** ile **sayı2** toplanır ve işlem sonucu eşitliğin solunda bulunan **toplam** değişkenine atanır. Bu işlem sonucunda RAM'deki değişkenler aşağıda görüldüğü gibi olur.



Şekil 5.13 Değişken içerikleri

Yukarıdaki işlemler sonunda şunlar iyice anlaşılmalıdır: Değişkenler içinde değişik veriler barındıran belleklerdir. Dışarıdan alınan veya program içinde üretilen veriler değişkenlerde saklanır. Değişkenleri yukarıdaki örnekte yaptığımız gibi, değişik matematiksel veya mantıksal ifadelerde kullanabiliriz. Elde edilen işlem sonuçlarını değişkenlere atayabiliriz. Ayrıca değişkenlerin değerlerini kullanıcılara bildirmek üzere ekrana yazabiliriz.

Tüm bu önbilgilerden sonra artık C#'ta değişken işlemlerinin nasıl yapıldığını öğrenmeye başlayabiliriz. C#'ta değişken oluşturma ve değer atama aşağıdaki gibidir:

```
değişken tipi Değişken_Adı=ilk Değer;
```

İstenirse değişken oluşturulduğu anda değer de atanabilir:

```
int a=5;
string ad="ali";
double pi=3.14;
```

İstenirse önce değişken oluşturulup sonra değer atanabilir:

```
int b;
b=20;

string soyad;
soyad= "erdinç";

double pi;
pi=3.14;
```

Bir satırda birden fazla değişken tanımlanabilir ve değer de atanabilir:

```
int a,b,c=30;
a=10;
b=20;

string ad="fahrettin", soyad="erdinç";
int x=y+5;
string ad =Console.ReadLine();
```

Boolean Değişkenler

Bilgisayarda her türlü veri **sıfır** veya **bir**, **doğru** veya **yanlış**, **evet** veya **hayır** şeklindeki değerlerden oluşur. Bazı durumlarda bu yapıyı biz de kullanıyoruz.

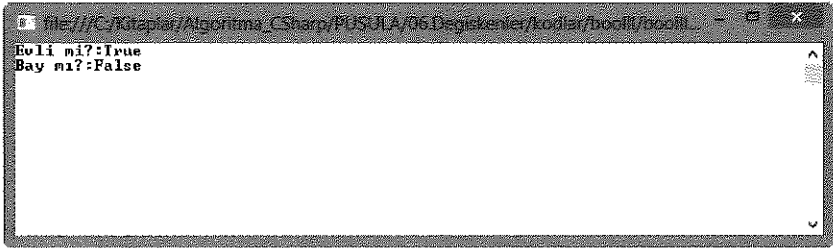
Aşağıdaki örneklerde emeklilik ve üyelik durumunu gösteren iki değişken tanımlanmıştır. Eğer kişi emekli ise bunun doğru olduğunu belirten **"true"** değeri kullanılmıştır. Eğer emekli değilse **"false"** değeri kullanılır. Diğer örnekte ise üyelik durumu ayarlanmaktadır. Eğer kişi siteye üye ise **uye=true**; değilse **uye=false**; ile üyelik durumu belirtilir.

```
bool emekli=false;
bool uye=true;
```

Örnek Kod:

```
static void Main(string[] args)
{
    bool evli = true, bay = false;
    Console.WriteLine("Evli mi?: " + evli);
    Console.WriteLine("Bay mı?: "+bay);
    Console.Read();
}
```

Çıktı:



Şekil 5.14 bool veri tipi

Eğer mantıksal işlemlerle uğraşıyorsanız **bool** veri tipini kullanmak zorundasınız. **bool** veri tipi **true** veya **false** değerlerini alabilir. Eğer bir işlem doğruysa **true** yanlışsa **false** değerini alır.

Aşağıdaki örnekte eğer **x** değişkeni 1'den büyükse konsola "**x 1den büyük**" yazdırılmaktadır.

```
int x=5;
if(x>1==true)
    Console.Write("x 1den büyük");
```

C tabanlı dillerde çok kullanılan bir özellikten de hemen bahsedeyim. **if(x>1==true)** ifadesi ile **if(x>1)** ifadesi aynı anlama gelmektedir. Yani **==true** ifadesini yazmaya gerek kalmadan ilgili ifade **true** kabul edilir. Bu durumda yukarıdaki örneği aşağıdaki gibi yazabiliriz.

```
int x=5;
if(x>1)
    Console.Write("x 1den büyük");
```

Bazı durumlarda VE ya da VEYA operatörlerini kullanmak zorunda kalabiliriz. Eğer ifademiz birden fazla şart içeriyorsa mantıksal yapıyı oluştururken çok daha dikkatli olunmalıdır.

Örnek Kod:

Aşağıdaki örnekte **a**, **b** ve **c** değişkenlerinin durumu sorgulanmaktadır. **if()** sorgusu iki durumu kontrol etmektedir. Eğer **a>b==true** ve **b>c==true** ise **if()** komutu içindeki sorgu sonucu da **true** olmakta ve ona göre işlem yapılmaktadır.

```
int a=10, b=20, c=30;
if(a>b && b>c) //a b'den büyük VE b de c'den büyükse
```



```
Console.Write("her iki şart sağlandı"); //bu satırı çalıştır
Console.Read();
```

Örnek Kod:

```
int x=1, y=2, z=3;
if(x<y || y<z) //x y'den küçük VEYA y z'den küçükse
Console.Write("iki şarttan biri sağlandı");
Console.Read();
```

Tamsayı Değişkenler

Eğer 2, 547, 9150, -33 gibi tamsayılarla işlem yapacaksanız boyutuna ve işaretine göre **byte**, **sbyte**, **ushort**, **short**, **uint**, **int**, **ulong** veya **long** gibi tamsayı değişkenleri kullanmanız gerekir.

Örneğin 0-100 arası sayılarla çalışacaksanız **byte** veri tipini kullanabilirsiniz. Bu veri tipi 0-255 arası değerleri saklayabilir.

```
byte not1=25, not2=50, ort=0;
```

Eğer -100:+100 gibi tamsayılarla çalışacaksanız **sbyte** veri tipi en mantıklısıdır. Bu veri tipi -128:+127 arası değerleri saklayabilir.

```
sbyte sicaklikOcak=-12;
sbyte sicaklikSubat=-5;
sbyte sicaklikMart=2;
sbyte sicaklikOrtalama;
```

Eğer 0-65.000 arası tamsayılarla çalışacaksanız **ushort** veri tipini tercih edebilirsiniz. Bu veri tipi 0:65.535 arası değerleri saklayabilir.

```
ushort maas=12000;
ushort uyeSayisi=25000;
```

-32000 ile +32000 arası tam sayılarla çalışmak için en mantıklı veri tipi **short** veri tipidir.

```
short saturnSicaklik=-1600;
short aySicaklik=-300;
short demirErime=1538;
```

Eğer çok daha büyük tam sayılarla çalışmanız gerekiyorsa **uint**, **int**, **ulong** veya **long** veri tipini tercih edebilirsiniz. **uint** veri tipi -2 milyar:+2 milyar arası tam sayılarla çalışmanıza izin verirken **int** veri tipi 0:4 milyar arası pozitif tam sayılarla çalışmanıza izin verir.

```
int ikramiye=-2000000;  
uint zarar=1500000;
```

C# dilinde **short**, **int** ve **long** veri tiplerinin yerine istenirse **Int16**, **Int32** veya **Int64** veri tipleri de kullanılabilir.

```
short x;  
int y;  
long z;
```

Yukarıdaki değişken tanımlamalarının yerine aşağıdaki tanımlamalar da kullanılabilir.

```
Int16 x;  
Int32 y;  
Int64 z;
```

Tam sayılarla çalışırken dikkat etmeniz gereken bir konu vardır; Eğer bir işlem sonucunda ondalıklı (kesirli) bir değer ortaya çıkarsa virgülden sonraki değeri atarlar.

Aşağıdaki kodları inceleyelim:

```
int x=1;  
int y=2;  
int ort=0;  
ort=(x+y)/2; //normalde (1+2)/2=1.5 olur  
Console.Write(ort); // ekranda 1 görünür. Virgülden sonrakiler atılır
```

Normalde $(x+y)/2$ yani $(1+2)/2=1.5$ yani ondalıklı sayı olur. Fakat **ort** değişkeninin veri tipi tamsayı (**int**) olduğu için virgülden sonraki rakamlar atılır ve **ort=1** olur.

İşin özeti, eğer bir tam sayı değişkene ondalıklı değer atanırsa virgülden sonraki rakamlar silinir.

Ondalıklı Değişkenler

Tamsayı olmayan, 1.25, 13.5454 gibi sayılara ondalıklı, kayar noktalı ya da kesir-

li sayı denir. Bu tür sayıları saklayabilmek için **float**, **double** ya da **decimal** veri tipi kullanılır.

```
float pi=3.14;
double r=24.434365;
decimal borc=23.4343434;
```

float veri tipi virgülden sonra 6 karakter saklayabiliyorken **double** veri tipi 15, **decimal** veritipi ise 28 karakter saklayabilir. Yani eğer virgülden sonraki rakamlar sizin için çok önemliyse, bilimsel vb. hesaplarla uğraşıyorsanız **decimal** veri tipini kullanmanız tavsiye edilir.

Ondalıkli sayıları kullanarak yukarıdaki örneği ortalama örneğini tekrar yapalım.

```
int x=1;
int y=2;
double ort=0;
ort=(x+y)/2;           //normalde (1+2)/2=1.5 olur
Console.Write(ort);    // ekranda 1 görünür.
```

Yukarıdaki kodlarda **ort** değişkeni ondalıklı sayıları saklayabilen **double** veri tipine sahip olmasına rağmen sonuç gene 1 olur. Çünkü **x** ile **y** tamsayıdır ve eşitliğin sağında yapılan $(x+y)/2$ yani $(\text{tamsayı } 1 + \text{tamsayı } 2)/2$ işleminin sonucu da tamsayı olur. Elde edilen tamsayı 1 değeri eşitliğin solundaki **double** veri tipine aktarılır. Bu hesabın düzgün yapılabilmesi için tüm değişkenlerin **double** veya **float** veri tipinde tanımlanması gerekir.

```
double x, y, ort=0;
ort=(x+y)/2;
Console.Write(ort); // ekranda 1.5 görünür.
Console.Read();
```

Sözel Değişkenler

C# dilinde sözel verileri saklayabilmek için **char** veya **string** veri tipi kullanılır. **char** veri tipi tek bir sayısal veya alfasayısal karakter saklayabiliyorken **string** veri tipi birden fazla karakter saklayabilir.

char veri tipinde atama işlemi tek tırnak ('...') kullanılarak yapılır.

```
char harf= 'a';
```

```
char cevap;
```

```
cevap= 'e';
```

string veri tipinde bir kelime, cümle, paragraf veya koca bir hikaye saklayabiliriz. **String** tipindeki değişkenlere atama işlemi çift tırnak ("...") kullanılarak yapılır.

```
String ad;
```

```
ad= "fahrettin";
```

```
string firma= "bilimsel akademi";
```

```
string slogan= "hep destek tam destek!!!";
```

Toplama & Atama

Karşımıza çok çıkacak işlemlerden bir tanesi toplamalı atamadır. Bu atama işleminde doğrudan atama yapılmaz; önce bir toplama işlemi yapılır, daha sonra da elde edilen sonuç soldaki değişkene atanır.

Aşağıdaki kodları incelerseniz toplamalı atama işlemini daha iyi anlayacaksınız. Buradaki atama işleminde önce **sayı** değişkeni ile **toplam** değişkeni toplanır ve daha sonra da elde edilen toplama sonucu soldaki **toplam** değişkenine aktarılır. Sonuçta **toplam** değişkenine **sayı** değeri eklenmiş olur. Yani **toplam** değişkeninin değeri 5 **sayı** değişkenin de değeri 1 ise bu işlem sonucunda toplam değişkeninin değeri 6 olur.

```
toplam=toplam+sayi;
```

Örneğin 1'den N'e kadar olan sayıların toplamını bulmak istiyorsanız toplamalı atama yapabilirsiniz. Bu işlem için bir for() döngüsü kullanılır ve aşağıdaki kodlar yazılır.

```
for(int sayi=1; sayi<=n; sayi++)
```

```
    toplam=toplam+sayi;
```

Yukarıdaki örnekte **sayı** değişkeni 1'den başlayıp **n** değerine kadar birer birer artarak değişmektedir ve her artışta/döngüde **toplam** değişkenine eklenmektedir. Sonuçta 1'den **n**'e kadar olan tüm sayılar **toplam** değişkeninde toplanmaktadır.

Yukarıdaki kodun değişken tablosunu yaparsak aşağıdaki gibi olur:

n	sayi	eski toplam	yeni toplam
5	1	0	1
5	2	1	3
5	3	3	6
5	4	6	10
5	5	10	15

Her satırda eski/önceki **toplam** ile **sayi** değişkeni toplanır ve **toplam** değişkeninde saklanır.

Çarpma ve Atama

Çarpmalı atama işlemi toplamalı atama işlemine çok benzemektedir. Tek farkı toplama yerine çarpma işlemi yapılmasıdır. Yani işlem aşağıdaki gibidir:

```
islem=islem * sayi;
```

Örneğin 1'den **N** değerine kadar olan sayıların çarpımını yani **N** faktöriyeli düşünün. Burada 1'den **N**'e kadar olan tüm sayılar birbiriyle çarpılır ve bir sonuç elde edilir. Bu işlem için aşağıdaki kodlar kullanılabilir:

```
int f=1;
for(int sayi=1; sayi<=n; sayi++)
f=f*sayi;
```

Yukarıdaki kodlarda **f** değişkeni **sayi** ile çarpılır ve işlem sonucu gene **f** değişkeninde saklanır. Sonuçta 1'den **n**'e kadar değer alan **sayi** değişkeni ile **f** değişkeni çarpılır ve **n** faktöriyel bulunur.

Yukarıdaki kodun değişken tablosunu yaparsak aşağıdaki gibi olur:

n	sayi	eski f	yeni f
5	1	1	1
5	2	1	2
5	3	2	6
5	4	6	24
5	5	24	120

Her satırda eski/önceki **f** ile **sayi** değişkeni çarpılır ve işlem sonucu **f** değişkeninde saklanır.

Değişken Tanımlarken Nelere Dikkat Etmeliyiz?

- Değişkenin ilk karakteri harf (alfa numerik) olmalıdır. Rakam (0-9) ile başlayan değişkenler hataya neden olur.
- Değişken tek kelime olmalıdır. Eğer iki kelimeden oluşan değişkenler tanımlayacaksanız araya alt çizgi (_) koyabilirsiniz. Örneğin **string Ad_Soyad** gibi bir değişken tanımlayabilirsiniz ama **string Ad Soyad** şeklinde bir değişken tanımlayamazsınız.
- Değişkenlere sözel değer atanacaksa çift tırnak ("...") içinde atanmalıdır. **string ad="fahrettin"** gibi...
- C# büyük-küçük harf duyarlı bir dildir; sayı ile Sayı değişkenleri birbirinden farklıdır.
- C# dilinde Türkçe karakterlere sahip "çap", "sayısal", "sözel" gibi değişken adları kullanılabilir fakat pek tercih edilmemelidir çünkü umulmadık yerlerde uyumsuzluktan dolayı sıkıntı çıkabilir.
- Değişkenlerin veri tipini hatırlamak için değişken adının önüne veri tipi kısaltması yazabilirsiniz. Örneğin **int** tipinde **sayı1** değişkeni tanımlayacaksanız değişkenin adını **iSayı1** yapabilirsiniz. Ya da **double** tipinde **yaricap** değişkeni tanımlayacaksanız değişken adını **dYaricap** olarak ayarlayabilirsiniz. Bu işlem sizi biraz yorabilir ama uzun değişken veri tipi ile ilgili hatalar yapmanızı engeller.
- C# komutları (deyimleri) değişken adı olarak kullanılamaz. Aşağıdaki tabloda değişken olarak kullanamayacağınız C# komutlarını (reserved keywords) görebilirsiniz.

C# Deyimleri					
abstract	event	new	struct	as	explicit
null	switch	base	extern	object	this
bool	false	operator	throw	break	finally
out	true	byte	fixed	override	try
case	float	params	typeof	catch	for
private	uint	char	foreach	protected	ulong
checked	goto	public	unchecked	class	if
readonly	unsafe	const	implicit	ref	ushort
continue	in	return	using	decimal	int
sbyte	virtual	default	interface	sealed	volatile
delegate	internal	short	void	do	is
sizeof	while	double	lock	stackalloc	else
long	static	enum	namespace	string	

Aşağıdaki kodlarda uygun ve uygun olmayan bazı değişken adları görüyorsunuz.

```
string ad soyad;           //hata: değişken adından boşluk olamaz
string ad_soyad;          //doğru
string sAdSoyad;          //doğru: değişken önüne veri tipi koyulmuş

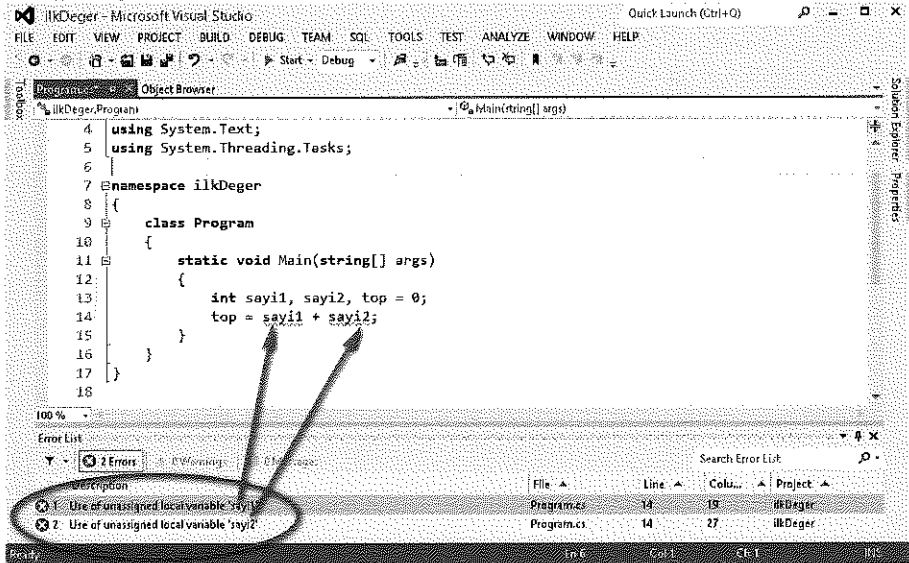
int 35izmir;              //hata: değişken rakamla başlayamaz
int i35izmir;             //doğru: değişken önüne veri tipi koyulmuş

double yarıçap;           //hata: Türkçe karakter tavsiye edilmez
double dYaricap;          //doğru: değişken önüne veri tipi koyulmuş

bool medeni durum=true;   //hata: iki kelimedenden değişken olmaz
bool bMedeniDurum=true;   //doğru
```

Değişkenlerin Başlangıç Değeri

Eğer bir değişkeni tanımlar ve o değişkene bir değer atamadan program içinde kullanmaya kalkarsanız "use of unassigned variable" hatası alırsınız. Yani bir değişkene değer atamadan onu kod içinde kullanamazsınız; derleyici aşağıda görüldüğü gibi hata verir.



Şekil 5.15 Atama yapılmamış değişken hatası

Eğer program içinde işlem sonuçlarının atanacağı "top" gibi bazı değişkenler varsa bu değişkenlere bir başlangıç değeri atamakta fayda var. Bildiğiniz gibi bir değiş-

ken oluşturmak aslında RAM bellekte ilgili veri için bir alan ayırmaktır. Bu alanda istenmeyen bazı veriler olabilir ve bu veriler bazı yanlış sonuçlara neden olabilir. Her ihtimale karşı değişken değerlerini sıfırlamakta fayda vardır. Bu nedenle eğer bazı işlem sonuçlarını kaydedeceğiniz ve ekrana yazdıracağınız değişkenler varsa **int top=0;** veya **string ad= "";** gibi ifadelerle değişken içerikleri sıfırlanmalıdır. Sayısal değişkenlere değer olarak sıfır atayarak, sözel değişkenlere ise boş karakter atayarak sıfırlama işlemi yapılabilir.

Aşağıda değişik başlangıç değerleri görebilirsiniz:

```
int x, y, toplam=0;
string sAd= "";
bool bMedeniDurum=true;
double r;
r=0;
double dOrtalama=0;
```

Sabitler

Program içinde kullanılan fakat değeri hiç değişmeyen değişkenlere sabit denir. C#'ta sabit tanımlamak için aşağıdaki yapı kullanılır:

```
const veritipi Değişken_Adı=Değer;
```

Aşağıdaki örnekleri inceleyiniz:

Örnek Kod:

```
const double pi=3.14;
const int Katsayi=5;
const double oran;
oran=2.741;
```

Aşağıdaki programda dairenin alanı hesaplanmaktadır. Daire alanı hesaplanırken **pi** değeri kullanılmaktadır. **Pi** değeri hiçbir zaman değişmediği için sabit olarak tanımlanır ve program içinde o şekilde kullanılır.

```
//değişkenleri tanımlayalım
const double pi = 3.1415;
double r=1, alan;
```

```
//alan hesabı
```



```
alan = pi * r * r;

//sonucu yazalım
Console.Write("alan="+alan);

Console.Read(); //bekle
```

Değişken Ömrü ve Kapsamı

Çok basit bir giriş yapayım ve daha sonra örneklerle açıklamaya çalışayım: Bir blok içinde tanımlanan bir değişken sadece o blok içinde kullanılabilir, o bloktaki kodların çalışması bittikten sonra silinir. Eğer bir değişkenin diğer bloklar tarafından da kullanılması istiyorsanız ilgili değişken blok dışında tanımlanmalıdır.

Eğer bir blok, yani, küme parantezi ({...}) içinde değişken tanımlarsanız o değişkeni sadece o blok içinde kullanabilirsiniz. Örneğin **Main()** metodu içinde tanımlanan **s1**, **s2** ve **top** değişkenlerini sadece **Main()** bloğunun içinde kullanabilirsiniz. Eğer bu değişkenleri **hesapla()** metodunun içinde, yani başka bir blokta kullanmaya kalkarsanız hata alırsınız. Dikkat ederseniz Visual Studio bu değişkenlerin altını kırmızı ile çizmiştir.

Eğer aşağıdaki kodları yazarsanız **top**, **s1** ve **s2** değişkenleri için “**does not exist in the current context**” hatası alırsınız. Bu hata yerel değişkenlerin başka bir blokta kullanılmasından kaynaklanmaktadır.

```
class Program
{
    static void Main(string[] args)
    {
        //local (yerel) değişkenler
        int s1 = 5, s2 = 10, top = 0;
        top = s1 + s2;

    }

    static int hesapla()
    {
        //Main()'deki yerel değişkenler
        top = s1 + s2;
        return top;
    }
}
```

Şekil 5.16 Lokal değişkenler başka blokta kullanılamaz

Main() bloğunun içinde tanımlanan **s1**, **s2** ve **top** değişkenleri bu bloktaki komutlar çalıştırıldıktan sonra silinir ve bir daha bu değişkenlere ulaşılamaz. Blok içinde tanımlanan bu değişkenlere yerel (local) değişken denir.

Eğer bir değişkenin diğer bloklar tarafından da kullanılmasını istiyorsanız bu değişkeni **küresel** (global) olarak tanımlamalısınız. Küresel/global olarak tanımlamak istediğiniz değişkeni Şekil 5.17'de görüldüğü gibi **class** içine yazmalısınız. Blok dışında ve **class** içinde tanımlanan değişkenler küreseldir ve tüm bloklar tarafından ulaşılır. Yani küresel değişkenler blok dışında ama sınıf içinde bir yerde tanımlanmalıdır. Genelde küresel değişkenleri tanımlamak için Şekil 5.17'de görüldüğü gibi en üst satır kullanılır.

Global olarak tanımlanıp bloklar içinde kullanılan değişkenlerin değerlerini muhafaza etmesi için **static** olarak tanımlanması gerekir. Bu nedenle bizim örneğimizde de blok dışında tanımlanan değişkenler **static int** şeklinde tanımlanmıştır.

Yerel değişkenler içinde bulundukları bloğun kodları çalıştıktan sonra bellekten silinirken küresel değişkenler bellekten ancak program sonlandığında silinir. Bu nedenle küresel değişkenler program çalıştığı sürece tüm bloklar tarafından kullanılabilirler.

Aşağıdaki kodlarda **top**, **s1** ve **s2** değişkenleri blok dışında tanımlanmıştır. Böylece bu değişkenler küresel olarak tanımlanmış olur ve **hesapla()** gibi diğer bloklar tarafından kullanılabilir.

```

class Program
{
    //global (küresel) değişkenler
    static int s1 = 5, s2 = 10, top = 0;
    static void Main(string[] args)
    {
        //küresel değişkenler
        top = s1 + s2;
        Console.WriteLine("toplam=" + top);

        //hesapla metodunu çağıralım
        Console.Write("hesapla toplam=" + hesapla());

        Console.Read();//bekle
    }

    static int hesapla()
    {
        //küresel değişkenler
        top = s1 + s2;
        return top;
    }
}

```

Şekil 5.17 Global değişkenler

Veri Tipi Dönüştürme

İyi bilinmesi gereken konulardan biri de veri tipi dönüştürme işlemidir. Daha önceden de bahsettiğim gibi konsol satırından **ReadLine()** metodu ile okunan tüm veriler **string** tipinde gelir. Eğer konsoldan **ad**, **soyad** gibi sözel veriler okuyorsanız ve bunları **string** tipindeki değişkenlere atıyorsanız sorun olmaz. Fakat **boy**, **kilo**, **uzunluk** gibi sayısal değerler okuyorsanız ve bunları **int** gibi sayısal değişkenlere atamaya çalışıyorsanız konsoldan okunan her veri string tipinde geldiği için hata alırsınız.

Örneğin dışarıdan iki değer alıp **x** ve **y** değişkenlerine atadınız. Eğer **x** ve **y** değişkenlerini **int** gibi sayısal birer değişken olarak tanımladıysanız dışarıdan alınan verilerin atanması sırasında hata alırsınız. Çünkü konsoldan okunan tüm veriler **string** tipindedir; **string** tipindeki verileri **int** tipindeki bir değişkene atarsanız hata alırsınız.

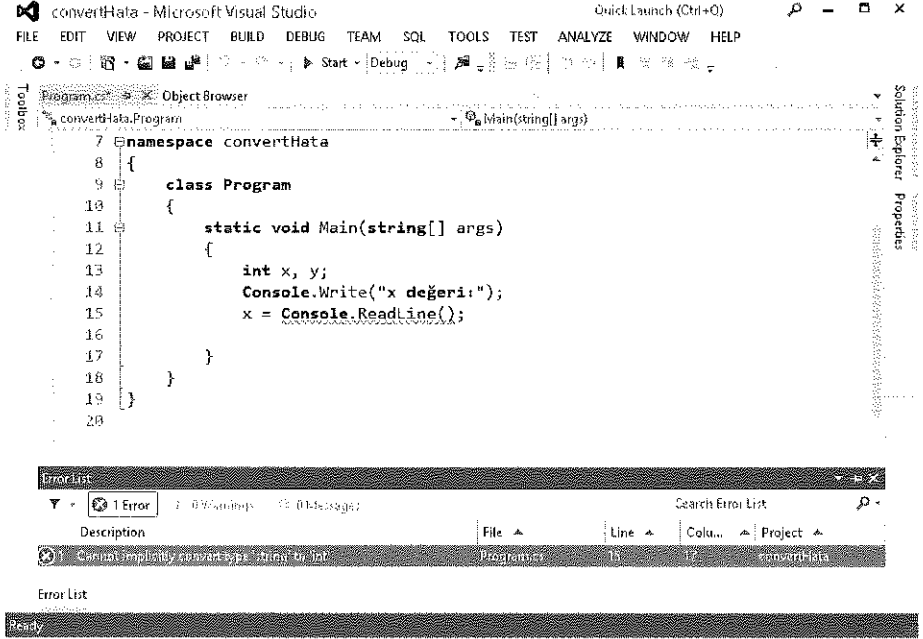
Aşağıdaki programda **int** tipinde tanımlanmış **x** ve **y** değişkenleri vardır.

```

int x,y;
Console.WriteLine("x değeri:");
x=Console.ReadLine();

```

Konsoldan okunan değer doğrudan **int** tipindeki **x** değişkenine atanmaktadır. Dolayısıyla, bu kodlar yazıldığı anda aşağıda görülen **Cannot implicitly convert type...** hatası alınacaktır.



Şekil 5.18 Veri tipi dönüştürme hatası

Hatanın nedenini şöyle açıklayayım: **Console.ReadLine()** komutu konsoldaki her şeyi metin (**string**) olarak okur. Siz **20** gibi sayısal bir değer bile girseniz bu **string** olarak kabul edilir. Yani **ReadLine()** metodu okuduğu her veriyi **string** olarak döndürür. Siz **string** olarak okunan bir değeri **int** tipindeki bir değişkene atarsanız hata alırsınız. Ne demiştik? Eşitliğin her iki tarafı da aynı veri tipinde olmalıdır. Aşağıdaki atamada eşitliğin sağ tarafı **string** veri tipinde iken sol taraf **int** veri tipinde olduğu için hataya neden olmaktadır.

```
x(int)=(string)Console.ReadLine(); //aynı veri tipi olmalı
```

Peki! Bu sorundan nasıl kurtulacağız? Bu sorundan konsoldan okunan verinin tipini uygun veri tipine dönüştürüp ondan sonra atama yaparak kurtulacağız. Yani, farklı türlerdeki değişkenleri aynı işlem veya formül içinde kullanmak istiyorsanız hepsinin aynı veri tipinde olmasını sağlayacağız. Veri tipini dönüştürmek için aşağıdaki yöntemleri kullanabilirsiniz:

1. **Değer Sabiti:** Değişkenlere uygun sandığınız tipte değer atamak hataya neden olabilir. Örneğin **float** tipindeki **f1** değişkenine **3.14** değerini atayalım. Eğer bu kodu yazarsanız hata alırsınız çünkü ondalıklı değişkene atanan bir değer varsayılan olarak **double** veri tipinde ve tamsayı değişkene atanan bir değer ise **int** veri tipinde kabul edilir. Bundan dolayı **float** tipindeki **f1** değişkenine atanan **double** tipindeki 3.14 değeri hataya neden olur.

```
float f1=3.14; //float f1 değişkenine double tipinde 3.14 atandı
```

- Yukarıdaki sorundan kurtulmak için değer sabitlerini kullanabiliriz. Eğer değer atama işlemini **float f1=3.14f;** şeklinde yaparsak sorunu çözmüş oluruz. Aynı şekilde başka atamalarda da değerın veritipini değiştirmek için aşağıdaki tabloda görülen değer sabitlerini kullanabiliriz.

Veritipi	Değer Sabiti
UInt	U/u
long	L/l
Ulong	UL/ul
float	F/f
Double	D/d
decimal	M/m

```
float f1=3.14F;
decimal d=3.141534234423424M;
uint u=15000000U;
```

- **Convert Sınıfı:** Özellikle veri tipi dönüştürmek için kullanılan bir sınıftır. Her veri tipini uygun diğer tüm veri tiplerine dönüştürebilir. Aşağıda ayrıntılı olarak anlatılmaktadır.
2. **Parse() Metodu:** Genelde konsoldan ya da metin kutularından okunan ve varsayılan olarak **string** tipinde olan değerleri sayısal veri tiplerine çevirmek için kullanılır.

```
b=byte.Parse(Console.ReadLine());
i=int.Parse(Console.ReadLine());
d=double.Parse(Console.ReadLine());
```
 3. **İstemsiz (Otomatik) Dönüşüm:** Küçük veri tiplerinin büyük veri tiplerine otomatik olarak dönüştürülmesi işlemidir. Aşağıda ayrıntılı olarak anlatılmıştır.
 4. **İstemli (Elle) Dönüşüm:** Büyük veri tiplerinin küçük veri tiplerine dönüştürülmesi işlemidir. Aşağıda ayrıntılı olarak anlatılmıştır.

Convert Sınıfı

Eğer herhangi tipte bir veriyi uygun başka bir veri tipine dönüştürmek istiyorsanız **Convert** sınıfını kullanabilirsiniz. **Convert** sınıfı herhangi tipteki bir veriye çok kolay ve sorunsuz şekilde uygun başka bir veri tipine dönüştürebilir. Ayrıca veri tipi dönüştürme için **Parse()** metodu da kullanılabilir ama biz burada **Convert()** sınıfını kullanacağız.

Yukarıdaki örneğimizde **x** değişkenine atanan değeri aşağıdaki gibi yaparsak, yani 32 bitlik Integer (Int32) veri tipine dönüştürsek sorundan kurtuluruz.

```
int x,y;
string okunan;
Console.WriteLine("x değeri:");

//konsoldan alınan değeri string tipindeki okunan değişkenine ata
okunan=Console.ReadLine();

//okunan değişkenini Int32 veritipine dönüştür ve x değişkenine ata
x=Convert.ToInt32(okunan);
```

İsterseniz dönüştürme işlemini aşağıdaki gibi daha kısa bir şekilde de yapabilirsiniz:

```
int x,y;
Console.WriteLine("x değeri:");

//okunan değeri Int32 veritipine dönüştür ve x değişkenine ata
x=Convert.ToInt32(Console.ReadLine());
```

Convert sınıfının metotları parantez içine yazılan her değeri belirtilen veri tipine çevirir:

```
Convert.ToBoolean(değer/değişken);
Convert.ToByte(değer/değişken);
Convert.ToChar(değer/değişken);
Convert.ToDateTime(değer/değişken);
Convert.ToInt32(değer/değişken);
Convert.ToDouble(değer/değişken);
Convert.ToString(değer/değişken);
```

Convert sınıfı ile uygun verileri başka bir veri tipine kolay bir şekilde dönüştürebiliyoruz. Uygun derken şunu kastediyorum; string tipindeki "Ali" değerini sayısal bir

veri tipine dönüştüremezsiniz fakat sayısal tipteki 35 değerini string tipinde "35" değerine dönüştürebilirsiniz.

Örnek Kod:

```
int yas=40, x;
string ad= "fahrettin", y;
y=Convert.ToString(yas); //hata vermez: y= "40" şeklinde string oldu
x=Convert.ToInt32(ad); //hata verir: x= "fahrettin" şeklinde int olamaz
```

İstemsiz ve İstemli Veri Tipi Dönüşümü

Yukarıda bahsedilen **Convert()** ve **Parse()** tip dönüşüm komutlarının yanı sıra C# dilinde **İstemsiz** (Implicit) ve **İstemli** (Explicit) diye bilinen dönüşüm yöntemleri de vardır.

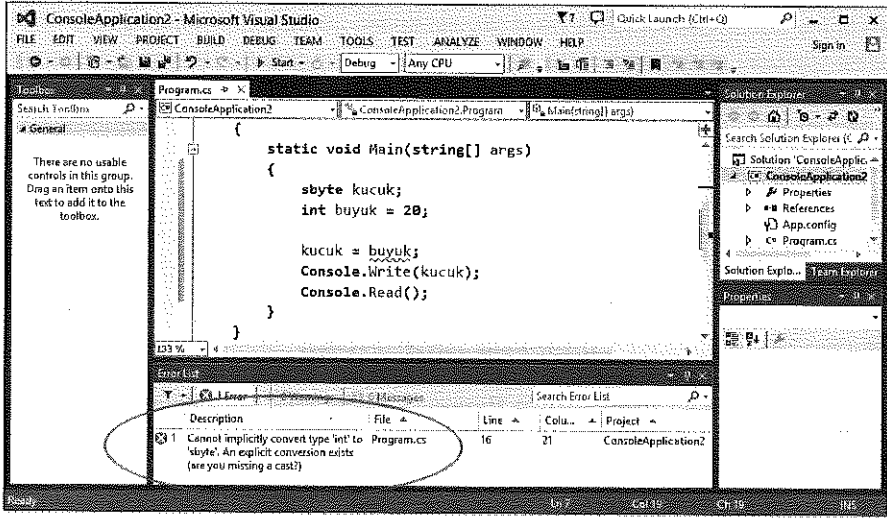
İstemsiz dönüşüm yönteminde daha küçük bir değer aralığına sahip değişken onu kapsayan daha büyük bir değer aralığına sahip başka bir değişkene atanabilir. Örneğin, **sbyte** tipindeki bir değişken **int** tipindeki başka bir değişkene atanabilir. Çünkü **sbyte** tipinde bir değişkeninin alabileceği bütün değerleri **int** tipindeki değişken de kapsar.

```
sbyte kucuk=20;
int buyuk;

buyuk=kucuk;
Console.Write(buyuk);
Console.Read();
```

sbyte tipindeki **kucuk** değişkeni -128:+127 arası değerler alabilirken **int** tipindeki **buyuk** değişkeni -2 milyar:+2 milyar arası değer alabilir. Dolayısıyla, herhangi bir komuta gerek kalmadan daha az değer aralığına sahip **kucuk** değişkeni daha geniş değer aralığına sahip **buyuk** değişkenine atanabilir.

Eğer yukarıdaki işlemin tam tersini yaparsanız, yani, daha geniş değer aralığına sahip **buyuk** değişkenini **kucuk** değişkenine atamaya kalkarsanız aşağıda görülen hata mesajını alırsınız.



Şekil 5.19 Veri tipi dönüştürme hatası

Yukarıdaki şekilde görülen hata mesajının Türkçesi **int** veri tipini doğrudan **sbyte** veri tipine atayamayacağınızdır. Bu durumda **İstemli** (Explicit) veri tipi dönüşümü kullanılır.

İstemli veri tipi dönüşümü daha büyük değer aralığına sahip veri tiplerinin daha küçük değer aralığına sahip veri tiplerine dönüştürülmesini sağlar. Yalnız, büyük değer aralığına sahip veri tipi değeri küçük değer aralığına sahip veri tipi değerinden büyük olmamalıdır. Örneğin 0:200 arası değerler alabilen **int** tipinde bir değişkeniniz varsa bu değişkeni **byte** tipinde başka bir değişkene aşağıdaki gibi atayabilirsiniz.

```
int buyuk=90;
```

```
byte kucuk;
```

```
kucuk=(byte)buyuk; //daha küçük değer aralıklı veri tipine ata
```

```
Console.Write(kucuk);
```

```
Console.Read();
```

Yukarıdaki kodlarda **int** tipindeki **buyuk** değişkeni, önüne dönüştürülmek istenen veri tipi (**byte**) yazılarak **byte** tipindeki **kucuk** değişkenine aktarılır. Bu tür bir dönüşüm için **buyuk** değişkeninin değerinin en fazla 255 olabileceği düşünülmüş ve atama gerçekleştirilmiştir. Eğer **buyuk** değişkeninin değeri **byte** veri tipi sınırlarını aşarsa işlem hatalı olur.

Aşağıdaki örnekte **int** tipindeki **x** ve **y** değişkenlerinin ortalaması alınmakta ve **double** tipine dönüştürülerek **ort** değişkenine atanmaktadır. Böylece **ort=1.5** olmakta-

dır. Eğer eşitliğin sağındaki işlem **double** tipine dönüştürülmezse sonuç 1 olacaktır ve bu da hata demektir. **x** ve **y** değişkenlerinin ortalaması aslında 1.5 eder fakat her iki değişken de tamsayı olduğu için virgülden sonraki ondalıklı değer atılır ve sonuç 1 olur. Bu sorundan istemli dönüşüm ile kurtuluyoruz.

```
int x=1, y=2;
double ort;
ort=(double)(x+y)/2;
Console.Write(ort);
Console.Read();
```

Boxing & Unboxing

Daha önce de öğrendiğimiz gibi, C# dilinde tüm veri tipleri **object** veri tipinden türetilir. C# tamamen nesne tabanlı bir dil olduğu için, şu ana kadar öğrendiğimiz veri tipleri ve değişkenler de aslında birer nesnedir. Dolayısıyla, tüm değişkenler ve değerler **object** tipine atanabilir. Bu durumu aşağıdaki kodlarla pekiştirebiliriz.

```
static void Main(string[] args)
{
    //nesne türünde değişken
    object x;

    x = 1;
    Console.WriteLine(x);

    x = 3.14;
    Console.WriteLine(x);

    x = 'a';
    Console.WriteLine(x);

    x = "ali";
    Console.WriteLine(x);

    x = true;
    Console.WriteLine(x);
}
```

Yukarıdaki kodlarda **object** tipinde bir **x** değişkeni oluşturulmuş ve bu değişkene farklı veri tipinde değerler atanarak ekrana yazdırılmıştır. Anlaşılabacağı üzere, **ob-**

ject veri tipi her tür değeri barındırabilmektedir. C# değişkenlerinin veya değerlerinin **object** tipine dönüştürülmesi ya da atanması işlemine **boxing** (kutulama) denilmektedir.

Aşağıdaki kodlarda değişik kutulama işlemlerini görebilirsiniz. Kutulama işlemi sonucunda ilkel türdeki **int**, **double** veya **string** benzeri değişkenler nesne/object tipine dönüşür.

```
object a;
a=20;

int x=10;
object y=x;

double pi=3.14;
object z=(object)pi;
```

Boxing işleminin tersine ise, yani **object** tipindeki değişkenlerin diğer veri tiplerine dönüştürülmesi işlemine ise **unboxing** (kutudan çıkarma) denir.

Unboxing işlemi yaparken nesnenin doğru veri tipine dönüştürülmesi gerekmektedir. Aksi takdirde hata alırsınız. Örneğin, değeri **true** olan bir **object** değişkenini veri tipi **int** olan bir değişkene atarsanız hata alırsınız. Yani, atanacak değişkenin uygun veri tipi ve değer aralığında olması gerekmektedir.

Unboxing işlemi yaparken dikkat etmeniz gereken diğer bir konu ise değişkenin daha önce **boxing** işlemi yapılmış olmasıdır. Yani, sadece nesneye çevrilen değişkenleri ilkel veri tiplerine dönüştürebilirsiniz. Aşağıdaki kodlarda boxing-unboxing ile ilgili örnek kodlar görebilirsiniz.

```
int x=10;           //ilkel değişken
object j=x;         //ilkel değişken nesneye atanıyor (boxing)
int z=(int)x;       //nesne ilkel değişkene atanıyor (unboxing)
Console.WriteLine(z);
```

Aşağıdaki kodları yazarak boxing-unboxing işlemlerini kendiniz de deneyebilirsiniz.

```
using System;
namespace uunboxing
{
    class Program
    {
```

```

static void Main(string[] args)
{
    int x = 10;
    object y = x;
    int z=(int)y;
    Console.WriteLine("int:"+z);
    Console.WriteLine("object:" + y);
}
}
}

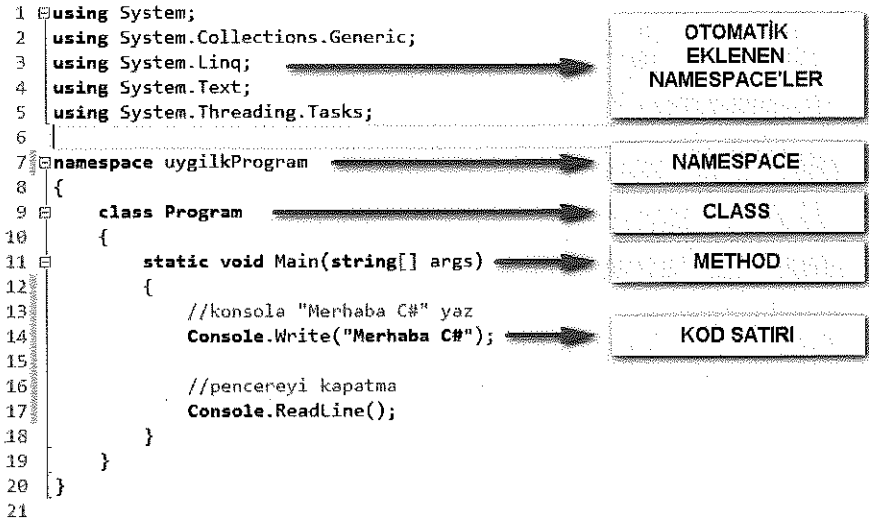
```

Uygulamalar

Bu başlık altında bu bölümde gördüğümüz konularla ilgili basit uygulamalar yapacağız. Özellikle veri giriş çıkışları, okunan verilerin uygun veri tiplerine dönüştürülmesi ve uygun değişken tanımlama işlemlerini iyice pekiştirmeye çalışacağız.

İlk Program

İlk programımızda kolay bir uygulama yapacağız ve kod penceresinde olan bütün şeyleri anlamaya çalışacağız. **File\ New Project** komutunu verelim ve yeni bir Console uygulaması başlatalım. Aşağıdaki şekilde gördüğümüz gibi **Console.WriteLine("Merhaba C#");** komutunu yazalım.



Şekil 5.20 İlk uygulama

Önce kod penceresini güzelce inceleyelim: Dikkat ederseniz en üst tarafta Visual Studio tarafından hazır eklenen namespace/aduzayı dediğimiz kodlar var. Ad uzaylarının içinde bizim kullanımımıza sunulan ve adına class/sınıf dediğimiz hazır kodlar bulunur. Örneğin **Console** sınıfı **System** aduzayının içindedir.

Visual Studio tarafından otomatik eklenen aduzaylarının altında **namespace uygilkProgram** adında bir komut görülmektedir. Bu komut otomatik olarak bizim projemizi bir aduzayına dönüştürmektedir. Yani biz yeni bir proje oluştururken proje adıyla aynı adı taşıyan bir aduzayı da otomatik olarak oluşturulmaktadır.

Programın 9. satırında, **uygilkProgram** aduzayının içinde **class Program** komutu görülmektedir. Bu komut otomatik olarak bizim yazdığımız programı bir class ya da sınıfa çevirmektedir.

class Program komutunun altında/içinde, 11. satırda **static void Main()** adlı bir metod bulunmaktadır. Bu bizim ana programımızı oluşturan ve belli bir iş yapmamızı sağlayan ana metottur. Dikkat ederseniz kodlarımızı bu metodun içine yazmamız gerekmektedir. İstersek başka metotlar oluşturup o metotların da içine kod yazabiliriz fakat **Main()** metodu bizim ana metodumuzdur ve programımızın da başlangıç noktasıdır. Herhangi bir C# programını çalıştırdığımızda önce **Main()** metodu çalışır daha sonra eğer varsa diğer metotlar çalıştırılabilir.

Yukarıdaki kodları iyice incelersek, .NET Framework'ün bizi kendi yapısına uydurduğunu görürüz. Hatırlayacak olursak, Microsoft .NET Framework'te namespace'ler vardı; namespace içinde de class dediğimiz hazır kodlar vardı. Class dediğimiz yapıların da metod ve özellikleri vardı. Bizim programımız da aynı hiyerarşiye sahiptir.

Daha projenin ilk aşamasında, henüz herhangi bir kod bile yazmamışken basit ama çok düzenli bir yapı vardır; kodlar hiyerarşik bir sırayla güzel parantez {...} denilen parantezlerin/blokların içine yazılmaktadır.

Yazılan her komutun sonuna noktalı virgül (;) koymak gerekmektedir. Komut sonuna noktalı virgül (;) karakterini koyduktan sonra aynı satıra veya alt satıra başka bir komut daha yazabilirsiniz.

İsterseniz programın okunabilirliğini arttırmak amacıyla komutlar ya da satırlar arasında boşluk bırakabilirsiniz. Boşluklar C# için sorun teşkil etmez. Yani derleyici boşluk karakterini ya da boş satırları derlememektedir.

Biz komutlarımızı programın başlangıç metodu olan **Main()** metodunun içine yazacağız. Amacımız ekrana "**Merhaba C#**" yazdırmak olduğu için komut olarak **Console** sınıfının **Write()** metodunu kullanacağız.

Programın 13. satırında açıklama satırı bulunmaktadır. Açıklama satırı koymakta fayda var çünkü hem siz kodları daha iyi anlıyorsunuz hem de ben ne yaptığımı ve amacımı hatırlayabiliyorum.

14. satırda ise asıl amacımızı gerçekleştirmemizi sağlayan **Console.Write()** komutu bulunmaktadır. Bu komut parantez içine yazılan verileri ekranda göstermeye yarar. Parantez içine **"Merhaba C#"** yazdığımıza göre program çalıştığında ekranda bu metin görünecek.

16. satırda bir başka açıklama satırı daha bulunmaktadır. 17. satırda ise tüm komutlar çalıştıktan sonra programın sona ermesini ve pencerenin kapanmasını engellemeye yarayan **Console.Read()** komutu bulunmaktadır. Bu komut aslında konsoldan veri okumak için kullanılır. Fakat biz bu komutu pencereyi bekletmek için kullanıyoruz; bu komut sayesinde bilgisayar bir veri girmemizi istiyor ve pencereyi kapatmıyor. Programın sonunda bu komut olmazsa tüm komutlar çalıştıktan sonra pencere kapanır ve bir şey göremeyiz. O nedenle her programın sonuna **Console.Read()** veya **Console.ReadLine()** komutu koyulur.

Kodlarda bir hata varsa altı kırmızıyla çizilmekte, **Error List** penceresinde gerekli uyarı ve öneriler yapılmaktadır. Hata penceresinde hatanın neden kaynaklandığı ve hangi satırda olduğu belirtilmektedir. Eğer hata iletisine çift tıklarsanız hatanın olduğu yere gidersiniz. Böylece hatanın yerini daha kolay bulup düzeltebilirsiniz. Hatalı (syntax/yazım hatası) programlar kesinlikle çalışmaz, programın çalışabilmesi için hatayı mutlaka düzeltmeniz gerekmektedir.

Genelde yeni başlayanlar oldukça fazla yazım (syntax) hatası yapmaktadır; çoğunlukla komut sonuna (;) karakterinin koyulması veya açılan güzel parantezlerin kapatılması (}) unutulmaktadır. Visual Studio bu hataları tespit etmekte ve gerekli düzeltmeleri yapmanızı istemektedir.

Örneğin aşağıdaki satırda (;) karakteri unutulmuştur.

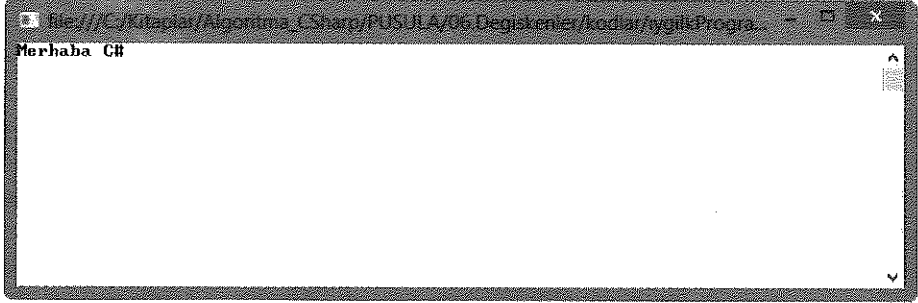
```
Console.Write()
```

Aşağıdaki satırda ise parantezin kapatılması unutulmuştur.

```
Console.ReadLine(;
```

Eğer yazdığınız kodları çalıştırmak isterseniz **F5** tuşuna basabilir veya Standart Toolbar'daki **>Start** butonuna tıklayabilirsiniz.

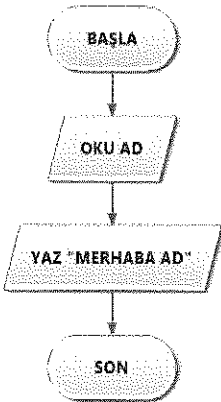
Program çalışır, ekrana **Merhaba C#** yazar ve hemen kapanır. Konsol penceresinin hemen kapanmasını istemiyorsanız her programın en sonuna **Console.Read()** veya **Console.ReadLine()** komutunu yazabilirsiniz. Programın çalışmasını gördükten sonra ENTER'layarak ya da penceredeki (x) işaretine tıklayarak konsol penceresini kapatabilirsiniz.



Şekil 5.21 Ekran çıktısı

Ad Yazdırma

Bu uygulamamızda konsoldan kullanıcının adını alacağız ve "Merhaba Ad" şeklinde konsola yazdıracağız. Yani burada önce dışarıdan bir veri alınacak ve daha sonra da dışarıya "Merhaba Veri" yazdırılacak.



```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    string ad;

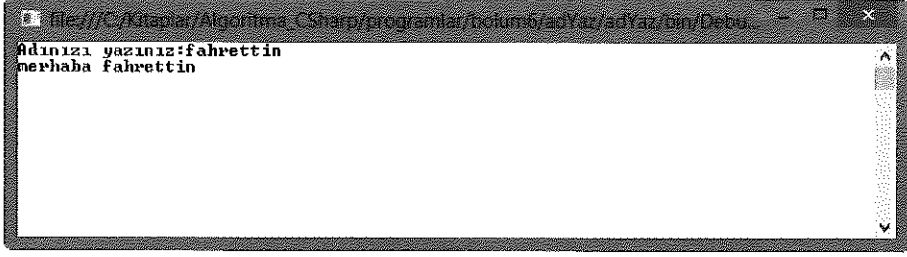
    //konsola "Adınızı yazınız" yazdıralım
    Console.Write("Adınızı yazınız:");

    //konsola yazılan veriyi okuyalım ve
    //ad değişkenine atayalım
    ad=Console.ReadLine();

    //"merhaba ad" yazdıralım
    Console.Write("merhaba " + ad);

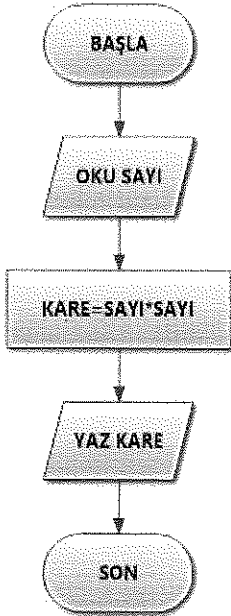
    //pencerenin kapanmasını engellemek için
    Console.ReadLine();
}
```

Ekran Çıktısı:



Girilen Sayının Karesi

Bu uygulamamızda girilen sayının karesini bulan bir program yapacağız. Burada dikkat etmemiz gereken konu veri tipi dönüştürme meselesidir. Dışarıdan sayıyı alıp bir değişkene atayacağız. Unutmayın konsoldan okunan her şey **string** tipindedir. Eğer atamayı doğrudan yaparsak hata alırız. Atama yapılmadan önce dışarıdan alınan verinin uygun veri tipine dönüştürülmesi gerekir.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    int sayi, kare;

    //konsola "sayı gir" yazdıralım
    Console.Write("sayı gir:");

    //konsola girilen değeri/sayıyı
    //32 bitlik Int veritipine çevirip
    //sayi değişkenine aktaralım
    sayi = Convert.ToInt32(Console.ReadLine());

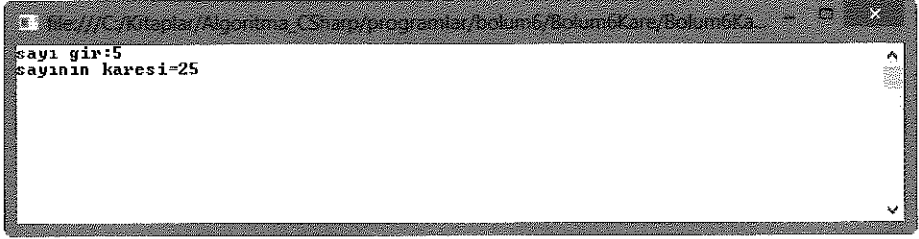
    //sayının karesini hesaplayıp kare
    //değişkenine atayalım
    kare = sayi * sayi;

    //bulunan işlem sonucunu konsola yazdıralım
    Console.Write("sayının karesi=" + kare);

    //konsolun kapanmaması için
    //Console.Read() metodunu kullanalım
    Console.Read();
}

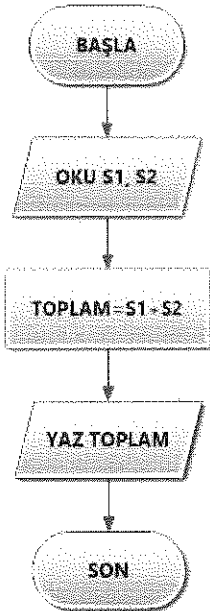
```

Ekran Çıktısı:



Girilen İki Sayının Toplamı

Bu uygulamada konsoldan iki değer okuyacağız ve her iki değeri de uygun veri tipine dönüştüreceğiz.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    int s1, s2, toplam = 0;

    //s1 değerini konsoldan alalım
    Console.Write("s1 değeri:");
    s1 = Convert.ToInt32(Console.ReadLine());

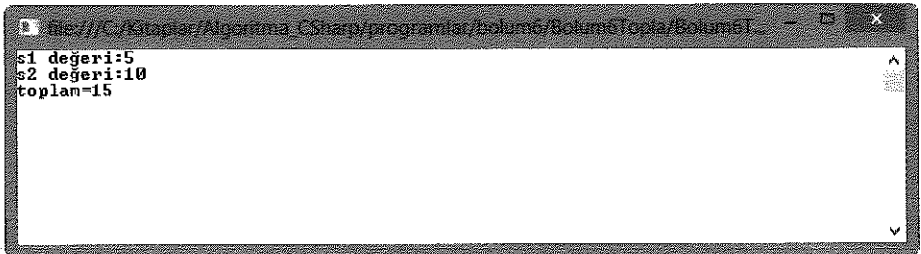
    //s2 değerini konsoldan alalım
    Console.Write("s2 değeri:");
    s2 = Convert.ToInt32(Console.ReadLine());

    //toplama işlemini yapalım
    toplam = s1 + s2;

    //toplamı konsola yazdıralım
    Console.Write("toplam=" + toplam);

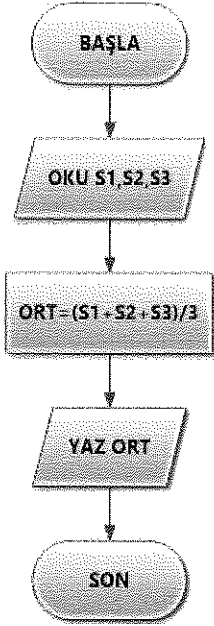
    //konsol kapanmasın
    Console.ReadLine();
}
  
```

Ekran Çıktısı:



Üç Sayının Ortalaması

Bu örnekte dikkat etmeniz gereken komutlar Write() ve ReadLine() metotlarıdır. Write() metodu ekrana bir metin yazdırdıktan sonra imleci aynı satırda bırakır. ReadLine() metodu ise satırı okuduktan sonra imleci alt satıra gönderir.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    int s1, s2, s3, ort;

    //konsoldan her değişkene değer alalım
    Console.Write("s1 değeri:");
    s1 = Convert.ToInt32(Console.ReadLine());

    Console.Write("s2 değeri:");
    s2 = Convert.ToInt32(Console.ReadLine());

    Console.Write("s3 değeri:");
    s3 = Convert.ToInt32(Console.ReadLine());

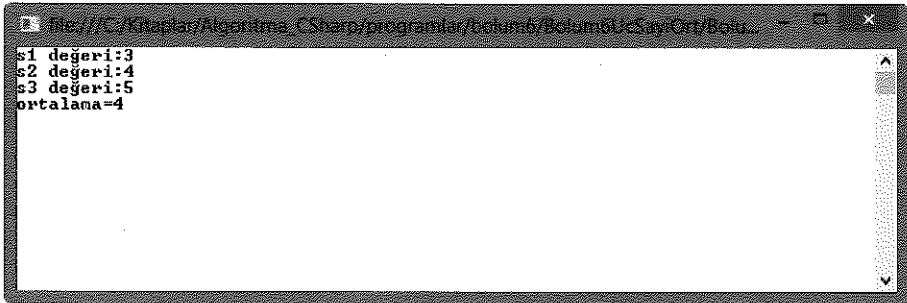
    //ortalama hesabını yapalım
    ort = (s1 + s2 + s3) / 3;

    //ortalamayı konsola yazdıralım
    Console.Write("ortalama=" + ort);

    //konsolu kapatmayalım
    Console.ReadLine();
}

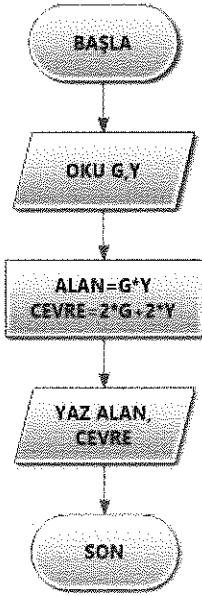
```

Ekran Çıktısı:



Dikdörtgenin Alanı ve Çevresi

Bu uygulamada dikkat etmeniz gereken konu değişkenlerdir. Girilen ölçülerin ve işlem sonuçlarının ondalıklı olduğunu düşünerek değişkenleri **double** tipinde tanımlayacağız. Ayrıca ekrana işlem sonuçlarını yazdırırken sözel toplama (+) operatörünü kullanacağız.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    //girilen değerler ve işlem sonuçları
    //ondalıklı olabileceği için değişkenleri
    //double veri tipinde tanımlayacağız
    double g, y, a, c;

    //dikdörtgenin genişliğini ve yüksekliğini
    //konsoldan okuyalım ve okunan değerleri
    //double veritipine dönüştürüp
    //ilgili değişkenlere atayalım
    Console.Write("genişlik:");
    g = Convert.ToDouble(Console.ReadLine());

    Console.Write("yükseklik:");
    y = Convert.ToDouble(Console.ReadLine());

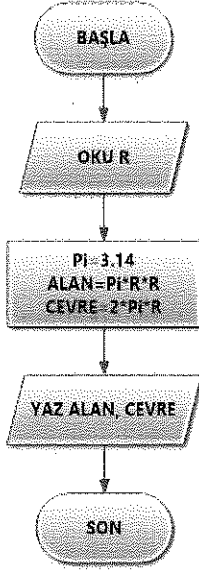
    //alan ve çevreyi hesaplayalım
    a = g * y;
    c = 2 * g + 2 * y;

    //sonuçları konsola yazdıralım
    Console.Write("alan=" + a + " çevre=" + c);

    //pencereyi kapatmayalım
    Console.ReadLine();
}
  
```

Dairenin Alanı ve Çevresi

Girilen değerler ve işlem sonuçları ondalıklı olabileceği için değişkenleri double veri tipinde tanımlayacağız. Bu örnekte sabit bir değişken kullanacağız; Pi değeri hiç değişmediği için onu sabit olarak tanımlayacağız.



```

static void Main(string[] args)
{
    //değişkenleri double tipinde tanımlayalım
    double r, a, c;

    //Pi değeri sabit olduğu için
    const double pi=3.14;

    //konsoldan yarıçapı okuyalım
    Console.Write("yarıçap:");
    r = Convert.ToDouble(Console.ReadLine());

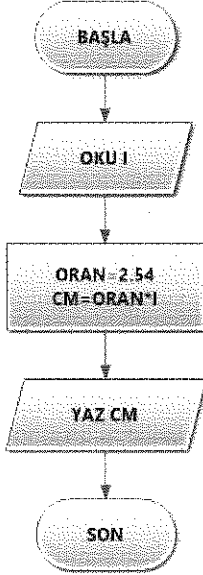
    //alan ve çevreyi hesaplayalım
    a = pi * r * r;
    c = 2 * pi * r;

    //işlem sonuçlarını yazdıralım
    Console.Write("alan=" + a + " çevre=" + c);

    //konsolu açık tutalım
    Console.ReadLine();
}
  
```

Inch-Cm Dönüşümü

Bu örneğimizde Inch olarak verilen bir değeri santimetreye çevireceğiz. 1 inch 2,54 santimetre olduğuna göre girilen değeri 2,54 ile çarpıp ekrana yazdıracağız. Inch ile cm arasındaki oran hep 2,54 olduğuna göre bu örnekte sabit bir değişken tanımlayabiliriz.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    double i, cm;
    const double oran = 2.54;

    //konsoldan inch değerini okuyalım
    //ve double veritipine çevirip
    //i değişkenine atayalım
    Console.Write("Inch değeri:");
    i = Convert.ToDouble(Console.ReadLine());

    //okunan Inch değerini santimetreye çevirelim
    cm = oran * i;

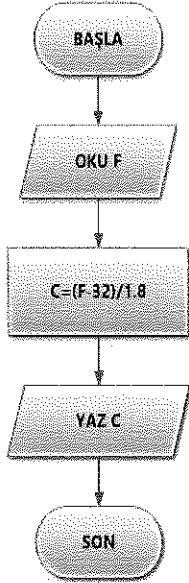
    //işlem sonucunu konsola yazalım
    Console.Write(i + " inch " + cm + " cm.");

    //konsol açık kalsın
    Console.ReadLine();
}
  
```

Fahrenheit-Santigrat Dönüşümü

Bu örneğimizde bir önceki örneğimize benzer bir uygulama yapacağız. Fahrenheit olarak alınan değeri bir formül kullanarak santigrat dereceye çevireceğiz. Fahrenheit dereceyi santigrat dereceye çevirmek için aşağıdaki formül kullanılır.

$$C = (F - 32) / 1.8$$



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    double c, f;

    //konsoldan fahrenheit değerini okuyalım
    Console.Write("F değeri:");
    f = Convert.ToDouble(Console.ReadLine());

    //alınan değeri santigrata çevirelim
    c = (f - 32) / 1.8;

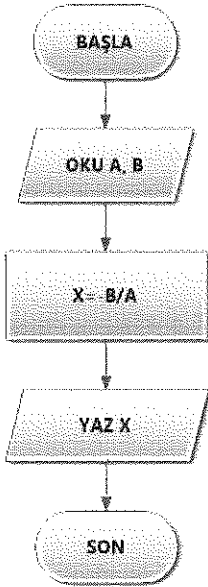
    //işlem sonucunu yazdıralım
    Console.Write("f=" + f + " c=" + c);

    //konsolu kapatma
    Console.ReadLine();
}
  
```

Tek Bilinmeyenli Denklem Çözümü

Bu örneğimizde $ax+b=0$ şeklinde olan tek bilinmeyenli bir denklemin çözümünü yapacağız. Burada bilinmeyen x değeridir. a ile b ise denklemin katsayılarıdır ve klavyeden girilmesi gerekir. x 'i bulmak için aşağıdaki işlemleri yapmamız gerekir.

- $ax+b=0$
- $ax=-b$
- $x=-b/a$



```

static void Main(string[] args)
{
    double a, b, x;

    //a değerini alalım
    Console.Write("a:");
    a = Convert.ToDouble(Console.ReadLine());

    //b değerini alalım
    Console.Write("b:");
    b = Convert.ToDouble(Console.ReadLine());

    //x değerini hesaplayalım
    x = -b / a;

    //x değerini ekrana yazalım
    Console.Write("x=" + x);

    Console.ReadLine();//bekle
}
  
```

Not: Dikkat ederseniz örnekler artık kendini tekrar etmeye başladı; değişen tek şey kullanılan formül. Bu nedenle değişken ve temel algoritmalarla ilgili örneklerimizi burada bitiriyoruz. Önümüzdeki konularda yeni şeyler öğrendikten sonra daha değişik programlar yazmaya başlayacağız.

6

Operatörler

- Operatör Nedir?
- Aritmetik Operatörler
- Arttırma ve Azaltma Operatörleri
- Eşitlik ve İlişki Operatörleri
- Mantıksal Operatörler
- Atama Operatörleri
- Operatörlerde Öncelik Sırası

Operatör Nedir?

Matematiksel, mantıksal vb. işlemleri yapmaya yarayan sembollerdir. Toplama, çıkarma gibi çok kullanılan operatörlerin yanı sıra çarpma, bölme, mod, ve, veya gibi bir sürü operatör vardır. Bu bölümde çok kullanılan operatörleri öğreneceğiz ve bunlarla ilgili örnekler yapacağız.

Aritmetik Operatörler

Adından da belli olduğu gibi aritmetiksel operatörler toplama, çıkarma, çarpma, bölme gibi aritmetiksel işlemleri yapmaya yarayan sembollerdir. Aşağıdaki tabloda çok kullanılan aritmetiksel operatörleri görebilirsiniz.

Operatör	İşlevi	Örnek
+	Toplama	$x=y+2;$
-	Çıkarma	$z=z-1;$
*	Çarpma	$c=a*b;$
/	Bölme	$x=a/b;$
%	Mod (Kalan)	$m=k\%3;$

Aritmetiksel işlemlerde önce parantez içindeki işlemler, sonra çarpma veya bölme veya mod, ondan sonra da toplama veya çıkarma işlemi yapılır.

Örnek Kod:

Aşağıdaki örnekte basit bir hesap makinesi yapılmıştır. Konsoldan okunan iki sayı üzerinde matematiksel işlemler yapılmaktadır.

```
//değişkenleri tanımlayalım
int a, b, topla, cikar, carp, bol;
//konsoldan değerleri okuyalım
Console.Write("a:");
a=Convert.ToInt32(Console.ReadLine());
Console.Write("b:");
b=Convert.ToInt32(Console.ReadLine());

//işlemleri yapalım
topla=a+b;
cikar=a-b;
carp=a*b;
bol=a/b;

//sonuçları yazalım
Console.Write("top="+topla+" carp="+carp+" fark="+cikar+" bol="+bol);
Console.ReadLine(); //bekle
```

Örnek Kod:

Aşağıdaki işlemde önce n1 ile n2 toplanır ve 2'ye bölünür, elde edilen sonuç 0.4 ile çarpılır.

```
int n1=50, n2=60, Ort=0;
Ort=(n1+n2)/2*0.4;
Console.Write("%40 Ağırlıklı Ortalama="+Ort);
```


Örnek Kod:

Aşağıdaki işlemde önce parantez içindeki **v1** ile **v2** toplanır, toplam sonucu ikiye bölünür ve işlem sonucu 0.4 ile çarpılır. Daha sonra **f** ile 0.6 çarpılır ve elde edilen sonuç bir önceki sonuç ile toplanır.

```
int v1=60, v2=70, f=80, Yso=0;
Yso=(v1+v2)/2*0.4+f*0.6;
Console.Write("Yıl Sonu Ortalaması:"+Yso);
```

Örnek Kod:

Mod işlemi bir sayının başka bir sayıya bölünmesinden sonra kalan değeri verir. Aşağıdaki örnekte **x** değişkeninin **y** değişkenine göre modu alınmakta ve işlem sonucu konsola yazılmaktadır.

```
int x=5, y=3, sonuc;
sonuc=x%y; // x mod y yani 5/3=1, kalan=2
Console.Write("Sonuç="+sonuc); //sonuc=2 olur
Console.Read(); //bekle, pencereyi kapatma
```

Arttırma ve Azaltma Operatörleri

C tabanlı dillerde ++ ve -- operatörleri oldukça fazla kullanılmaktadır ve iyi bilinmesinde fayda vardır. Bu operatörler değişkenin soluna veya sağına konulabilir. Temelde yaptıkları iş aynıdır ama değişkenin solunda veya sağına olmasına göre ayrınımda bir fark vardır: Operatör solda ise önce arttırma yapılıp sonra işlem yapılır, operatör sağda ise önce işlem yapılıp sonra arttırma yapılır. Lütfen aşağıdaki tabloyu inceleyiniz.

İfade	Açıklama
++x	Önce x'ı 1 arttır, daha sonra x değişkenini yeni değeri ile kullan
x++	Önce x değerini kullan, işlemten sonra x değerini 1 arttır
--x	Önce x'ı 1 azalt, daha sonra x değişkenini yeni değeri ile kullan
x--	Önce x değerini kullan, işlemten sonra x değerini 1 azalt

Örnek Kod:

```
//değişkenleri tanımlayalım
int x=5, y=10, a=20, b;

//işlemleri yapalım
x++; //x değerini 1 arttır
y--; //y değerini 1 azalt
b++; //önce a değişkenini 1 arttır sonra b değişkenine ata
//sonuçları yazalım
Console.WriteLine("x="+x);
Console.WriteLine("y="+y);
Console.WriteLine("b="+b);
```

Örnek Kod:

Aşağıdaki örnekle arttırma ve eksiltme operatörlerinin görevi çok daha iyi anlaşılabilir.

```
int x;
//değişkenleri tanımlayalım
int x=5, y=10, a=20, b;

//işlemleri yapalım
x++; //x değerini 1 arttır
y--; //y değerini 1 azalt
b++; //önce a değişkenini 1 arttır sonra b değişkenine ata
//sonuçları yazalım
Console.WriteLine("x="+x);
Console.WriteLine("y="+y);
Console.WriteLine("b="+b);
x=10;
Console.WriteLine(x); //10
Console.WriteLine(x++); //10 önce x değerini yaz sonra 1 arttır
Console.WriteLine(x); //11
Console.WriteLine(); //boş satır

x=10;
Console.WriteLine(x); //10
Console.WriteLine(++x); //11 önce x değerini 1 artır sonra yaz
Console.WriteLine(x); //11
```

Eğer operatörler değişkenden önce geliyor ise önce arttırma veya eksiltme yapılır ondan sonra komut çalıştırılır.

Örnek Kod:

Aşağıdaki işlemde önce parantez içindeki **v1** ile **v2** toplanır, toplam sonucu ikiye bölünür ve işlem sonucu 0.4 ile çarpılır. Daha sonra **f** ile 0.6 çarpılır ve elde edilen sonuç bir önceki sonuç ile toplanır.

```
int v1=60, v2=70, f=80, Yso=0;
Yso=(v1+v2)/2*0.4+f*0.6;
Console.Write("Yıl Sonu Ortalaması:"+Yso);
```

Örnek Kod:

Mod işlemi bir sayının başka bir sayıya bölünmesinden sonra kalan değeri verir. Aşağıdaki örnekte **x** değişkeninin **y** değişkenine göre modu alınmakta ve işlem sonucu konsola yazılmaktadır.

```
int x=5, y=3, sonuc;
sonuc=x%y; // x mod y yani 5/3=1, kalan=2
Console.Write("Sonuç="+sonuc); //sonuc=2 olur
Console.Read(); //bekle, pencereyi kapatma
```

Arttırma ve Azaltma Operatörleri

C tabanlı dillerde ++ ve -- operatörleri oldukça fazla kullanılmaktadır ve iyi bilinmesinde fayda vardır. Bu operatörler değişkenin soluna veya sağına konulabilir. Temelde yaptıkları iş aynıdır ama değişkenin solunda veya sağına olmasına göre ayrışır bir fark vardır: Operatör solda ise önce arttırma yapılıp sonra işlem yapılır, operatör sağda ise önce işlem yapılıp sonra arttırma yapılır. Lütfen aşağıdaki tabloyu inceleyiniz.

İfade	Açıklama
++x	Önce x'ı 1 arttır, daha sonra x değişkenini yeni değeri ile kullan
x++	Önce x değerini kullan, işlemden sonra x değerini 1 arttır
--x	Önce x'ı 1 azalt, daha sonra x değişkenini yeni değeri ile kullan
x--	Önce x değerini kullan, işlemden sonra x değerini 1 azalt

Örnek Kod:

```
//değişkenleri tanımlayalım
int x=5, y=10, a=20, b;

//işlemleri yapalım
x++; //x değerini 1 arttır
y--; //y değerini 1 azalt
b++; //önce a değişkenini 1 arttır sonra b değişkenine ata
//sonuçları yazalım
Console.WriteLine("x="+x);
Console.WriteLine("y="+y);
Console.WriteLine("b="+b);
```

Örnek Kod:

Aşağıdaki örnek ile arttırma ve eksiltme operatörlerinin görevi çok daha iyi anlaşılabilir.

```
int x;
//değişkenleri tanımlayalım
int x=5, y=10, a=20, b;

//işlemleri yapalım
x++; //x değerini 1 arttır
y--; //y değerini 1 azalt
b++; //önce a değişkenini 1 arttır sonra b değişkenine ata
//sonuçları yazalım
Console.WriteLine("x="+x);
Console.WriteLine("y="+y);
Console.WriteLine("b="+b);
x=10;
Console.WriteLine(x); //10
Console.WriteLine(x++); //10 önce x değerini yaz sonra 1 arttır
Console.WriteLine(x); //11
Console.WriteLine(); //boş satır

x=10;
Console.WriteLine(x); //10
Console.WriteLine(++x); //11 önce x değerini 1 artır sonra yaz
Console.WriteLine(x); //11
```

Eğer operatörler değişkenden önce geliyor ise önce arttırma veya eksiltme yapılır ondan sonra komut çalıştırılır.

Eşitlik ve İlişki Operatörleri

İki değerin birbiriyle eşit olup olmadığını veya büyük olup olmadığını ölçmek için kullanılan operatörlerdir. Sorgu ifadelerinde veya döngülerde sıklıkla kullanılırlar. Aritmetik operatörlerde olduğu gibi değişkenlerin değerini değiştirmezler; sadece durumunu kontrol ederler.

Aşağıdaki tabloda eşitlik ve ilişki operatörlerini görebilirsiniz.

Operatör	İşlevi	Örnek
==	Eşit	x==y
!=	Eşit Değil	a!=b
>	Büyük	a>b
<	Küçük	b<c
>=	Büyük Eşit	d>=k
<=	Küçük Eşit	k<=z

Örnek Kodlar:

Aşağıda sıklıkla karşılaşacağınız bazı eşitlik operatörlerini ve anlamlarını bulabilirsiniz.

```
if(x==3)        // x değişkeni 3'e eşitse
if(x<5)         //x değişkeni 5'ten küçükse
if(y>=10)       //y değişkeni 10 veya daha büyükse
if(x!=y)        //x y'den farklı ise yani eşit değilse
if(x<=20)       //x 20 veya daha küçükse
```

Mantıksal Operatörler

Ve, veya gibi mantıksal işlemlerin yapılmasını sağlayan operatörlerdir. İki veya daha fazla şartın mantıksal olarak sınanmasında kullanılır.

Operatör	İşlev	Örnek
&&	Ve	z=a && b
	Veya	x=y z
^	XOR	a=a ^ b
~	Değil	z=~z

Aşağıdaki tabloda x ve y değişkenlerinin **ve**, **veya**, **XOR** ve **değil** işlemi sonucunda elde edilen değerleri görebilirsiniz.

x	y	x && y	x y	x ^ y	~x
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

&& ve **||** operatörleri sorgularda iki veya daha fazla ifadenin mantıksal durumunu kontrol etmek için kullanılır.

```
(a==5 ve b==7) yerine (a==5 && b==7)
(x<10 veya y>5) yerine (x<10 || y>5)
(z==1 veya x>5 veya y>5) yerine (z==1 || x>5 || y>5)
```

Örnek Kod:

```
int x=5, y=10;
if(x<5 && y<x) //x 5'ten küçük ve y x'ten küçükse
    Console.Write ("Şart sağlandı...");
else
    Console.Write("Şart sağlanmadı...");
```

Örnek Kod:

```
int x=20, y=30;
if(x>5 || y>10) //x 5'ten büyük veya y 10'dan büyükse
    Console.Write("Şart sağlandı...");
else
    Console.Write("Şart sağlanmadı...")
```

Atama Operatörleri

Atama operatörleri, atama işlemini yapmak ve atama işlemini kolaylaştırmak için kullanılan operatörlerdir. Örneğin $x=x+1$ yerine $x+=1$ gibi bir operatör kullanılabilir.

Operatör	Kullanımı	Açıklama
=	$x=y$	Atama
+=	$x+=y$	$x=x+y$
-=	$x-=y$	$x=x-y$
=	$x=y$	$x=x*y$
/=	$x/=y$	$x=x/y$
%=	$x\%=y$	$x=x\%y$
<<=	$x<<=y$	$x=x<<y$
>>=	$x>>=y$	$x=x>>y$
=	$x =y$	$x=x y$
&=	$x\&=y$	$x=x\&y$
^=	$x\^=y$	$x=x\^y$

Örnek Kod:

```
//değişkenleri tanımlayalım
int a, b, c;
string konsol;

//b değerini konsoldan okuyalım
Console.Write("b:");
konsol=Console.ReadLine(); //konsolu oku ve konsol değişkenine ata
b=Convert.ToInt32(konsol); //konsol değişkenini dönüştür ve b'ye ata

//işlemler
a=5; //atama işlemi
a+=1; //a=a+1
b-=1; //b=b-1
c*=a; //c=c*a

//sonuçları yazalım
Console.WriteLine("a="+a+ "b="+b);
Console.WriteLine("c="+c);
```

Operatörlerde Öncelik Sırası

Eğer aynı ifade içinde birden fazla operatör kullanılırsa operatörlerin öncelik sırasına dikkat etmek gerekiyor; önceliği olan operatör işlemini daha önce yapmalıyız. Aşağıdaki tabloda operatörlerin öncelik sırası yüksek olandan düşük olana göre sıralanmıştır. Aynı satırdaki operatörlerin işlem önceliği eşittir.

İşlem Önceliği	Operatör
1	()
2	x++, x--
3	++y, --y
4	*, /, %
5	+, -
6	<<, >>
7	<, >, <=, >=
8	==, !=
9	&
10	^
11	
12	&&
13	
14	=, +=, -=, /=, %=, <<=, >>=, =, &=, ^=

7

Karar Yapıları

- Karar Yapısı Nedir?
- If Komutu
- If-Else Komutu
- If-Else If-Else Komutu
- Short If (?: Opratörü)
- Switch Komutu
- Dallanma Komutu (goto)
- Karar Yapısı Uygulamaları

Karar Yapısı Nedir?

Sorgular günlük hayatımızın ayrılmaz bir parçasıdır. Sürekli şunu mu yapalım, bunu mu yapalım şeklinde kararlar vermeye çalışırız. Örneğin karşıdan karşıya geçerken araba var mı yok mu? diye sorgularız. Duruma göre geçer ya da bekleriz. Ya da satın almak istediğimiz bir ürün varsa fiyatına bakarız. Fiyatı belli bir değerin altındaysa alırız, değilse almazız. Yani belli bir şartı kontrol ederiz. Eğer şart sağlanırsa belli bir eylemi, sağlanmazsa başka bir eylemi gerçekleştiririz.

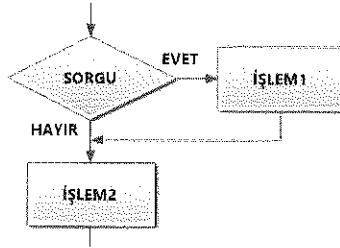
Bilgisayar da bize benzer; saniyede milyonlarca defa matematiksel ve mantıksal işlem yapmakla kalmaz aynı zamanda verdiğimiz bir veya daha fazla şarta göre bazı işlemlerin yapılmasını veya yapılmamasını sağlar.

Örneğin *x* isimli bir değişken olduğunu düşünün. Eğer bu değişkenin değeri 10'dan küçükse bazı kodların çalışmasını aksi takdirde çalışmamasını veya başka bazı

kodların çalışmasını sağlayabiliriz. İşte bu tür yapılara ya da kodlara **karar yapıları** denir. Bazı yerlerde sorgu ya da kontrol yapısı adıyla da karşınıza çıkabilir.

Normalde komutlar yukarıdan aşağıya doğru sırayla çalıştırılır. Fakat aşağıdaki akış diyagramında da görüldüğü gibi, karar yapıları bir sorgu sonucunda dallanmaya sebep olur ve programın yönünü değiştirebilir. Böylece değişkenlerin durumuna göre farklı kodların çalışması sağlanabilir.

Gelin karar yapılarını daha ayrıntılı inceleyelim. Aşağıdaki akış diyagramında bir sorgu yapılır. Bu sorgu ile örneğin bir değişkenin pozitif veya negatif olması sorgulanabilir. Ya da $(x > 5)$ gibi bir sorgu gerçekleştirilebilir. Eğer sorgu sonucu **EVET** ise, yani istenilen şart sağlandıysa algoritma sağa dallanır ve **İŞLEM1** gerçekleştirilir. Eğer sorgu sonucu **HAYIR** ise, yani istenilen şart sağlanmadıysa algoritma aşağıya dallanır ve sıradaki komut olan **İŞLEM2** gerçekleştirilir. Böylece değişkenlerin aldıkları farklı değerlere göre farklı kodlar çalıştırılabilir.



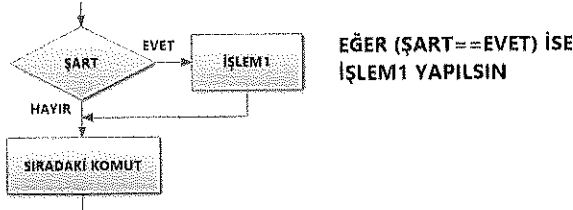
Şekil 7.1 Karar yapısı ve dallanma oluşumu

Aşağıdaki konularda karar yapılarını daha ayrıntılı olarak öğreneceğiz ve C# komutlarını kullanarak uygulamalar yapacağız.

If Komutu

Bilgisayarların en çok yaptığı işlerden biri farklı durumlarda farklı kodlar çalıştırmaktır. Örneğin belli bir süre geçtikten sonra bilgisayarınız ekranını karartır. Çünkü bilgisayar kullanılmayan zamanı sürekli sorgular ve belli bir zaman aşılsa ekranını karartmaya yarayan kodları çalıştırır. Gece olunca farları yakan arabalar artık herkeşe tanıdık gelmektedir. Ya da bazı arabalardaki bilgisayarlar öndeki arabaya belli bir mesafeden fazla yaklaşırsanız sizi uyarır. Bunlar hep sorgu işlemlerinin gerçekleşmesi sonucu gerçekleştirilen eylemlerdir. Eğer sorgudaki şart sağlanırsa dallanma olur ve belli bazı komut ya da komutlar çalıştırılır. Şart sağlanmazsa sıradaki komuttan devam edilir ya da bir şey yapılmaz.

Yukarıda verilen örneklerden de anlaşılacağı gibi, sorgu ya da dallanma komutları programcılıkta çok önemli ve en çok kullanılan komutlardır. Lütfen aşağıdaki akış diyagramını inceleyin ve **if** komutunun mantığını anlamaya çalışın. Baklava şekli içindeki **şart** sağlanırsa (evet/true) program yön değiştirir ve **İŞLEM1** gerçekleştirilir. **Şart** sağlanmazsa (hayır/false) sıradaki komut çalıştırılır.



Şekil 7.2 If komutu

if komutu iki farklı şekilde kullanılabilir. Eğer **şart** gerçekleştiğinde tek bir komut çalıştırılacaksa **if** komutunun kalıbı aşağıdaki gibi yazılır:

```
if (şart)
Komut;
```

Eğer **şart** gerçekleştiğinde birden fazla komut çalıştırılacaksa **if** komutu aşağıdaki gibi yazılmalıdır:

```
if (şart)
{
    Komut 1;
    Komut 2;
    ...
    Komut X;
}
```

Dikkat edilirse birinci yöntemde güzel parantez kullanılmazken ikinci yöntemde komutlar güzel parantez/blok {...} içine yazılmaktadır.

Örnek Kod:

Aşağıdaki örnekte eğer **not1** değişkeni 50'den küçükse yani **şart** sağlanırsa konsola "**Kaldı**" yazdırılır. Eğer 50 ya da daha büyükse **if** komutunun altındaki satır çalıştırılmaz yani program **if** komutuna girmez ve sıradaki satırdan (Altındaki komut) devam eder.

```
if (not1<50) //eğer not1 50'den küçükse
Console.Write("Kaldı"); //Kaldı yaz. Değilse alttaki komuttan devam et
(alttaki komut)
```

Örnek Kod:

Bu örnekte şart sağlandığı takdirde birden fazla komut çalıştırıldığı için komutlar parantez içine alınmıştır. Eğer şart sağlanmazsa alttaki komuttan devam edilecek.

```
if (not1>=50) //eğer şart sağlanırsa yani true olursa
{
    Console.WriteLine("Tebrikler!");
    Console.WriteLine("Sınıfı geçtiniz...");
}
(alttaki komut)
```

Örnek Kod:

Karar yapılarında birden fazla sorgu da kullanılabilir. Aşağıdaki kodlarda okunan notun 0-100 arası olup olmadığı kontrol edilmekte ve ona göre işlem yapılmaktadır.

```
if(not1>=0 && not1<=100) //her iki şart da sağlanırsa
Console.Write("Geçerli değer...");
```

Örnek Kod:

Aşağıdaki kodlarda iki şarttan birinin sağlanması durumunda ekrana "geçti" yazılmaktadır.

```
if(vizeOrt>90 || final>90) //şartlardan biri sağlanırsa
Console.Write("geçti");
```

Örnek Kod:

```
int a=50, b, c;
if(a>=20)
{
    b++;
    c=0;
}
```

Örnek Kod:

if komutu iç-içe de kullanılabilir. Aşağıdaki örnekte $x==y$ olması durumunda blok içindeki işlemler yapılmaktadır. if bloğunun içinde ise $y<z$ olması durumunda bir başka işlem daha yapılmaktadır.

```
int x=10, y=10, z=20;
x++;
if(x==y)
{
```

```

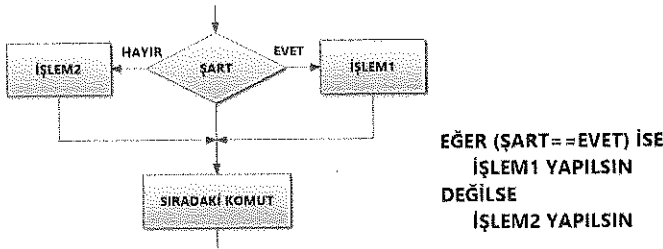
y++;
if(y<z)
    Console.Write("y z den küçük");
}

```

If-Else Komutu

if-else sorgusunda herhangi bir şart sağlanırsa belli bir komut ya da komut grubunun çalışması sağlanır. Eğer şart sağlanmazsa başka bir komut ya da komut grubu çalıştırılır. Yani her iki durumda da belli bir komut ya da komut grubu çalıştırılır.

Aşağıdaki akış diyagramını incelenirse **if-else** komutunun nasıl çalıştığı çok daha iyi anlaşılabilir. Program baklava şeklindeki sorgu (if-else) komutuna geldiğinde şart kontrol edilir. Eğer şart sağlanırsa (evet/true) **İŞLEM1** dalındaki komutlar, sağlanmazsa **İŞLEM2** dalındaki komutlar çalıştırılır. Ondan sonraki aşamada sıradaki komut çalıştırılır.



Şekil 7.3 If-Else komutu

Peki, **if** ile **if-else** arasındaki fark nedir? Hatırlayacak olursanız **if** komutunda sadece şart sağlanırsa bazı komutlar çalıştırılıyordu. Eğer şart sağlanmazsa program alttaki satırdan devam ediyordu. **if-else** komutunda ise şart sağlansa da sağlanmasa da bazı komutların çalıştırılması sağlanabiliyor. Eğer şart sağlanırsa belli bir komut ya da komut grubu, sağlanmazsa bir başka komut ya da komut grubu çalıştırılır.

Örneğin hava durumunu kontrol ediyorsunuz. Eğer hava sıcaksa t-shirt, soğuksa kazak giymeye karar veriyorsunuz. Yani hava durumu nasıl olursa olsun her durumda bir işlem yapıyorsunuz.

if-else komutu iki farklı şekilde kullanılabilir. Eğer tek bir komut çalıştırılacaksa aşağıdaki gibi yazılır:

```

if (şart)
komut1; //eğer şart sağlanırsa bu komut
else
komut2; //şart sağlanmazsa bu komut çalışsın
(sıradaki komut);

```

Eğer birden fazla komut çalıştırılacaksa **if-else** komutu aşağıdaki gibi yazılmalıdır:

```

if (şart) //eğer şart sağlanırsa
{ // bu parantez içindeki komutlar çalışsın
    komut 1;
    komut 2;
    ...
}
else //şart sağlanmazsa
{ //bu parantezdeki komutlar çalışsın
    komut 3;
    komut 4;
    ...
}

```

Dikkat edilirse birinci yöntemde güzel parantez kullanılmazken ikinci yöntemde komutlar güzel parantez {...} içine yani blok içine yazılmaktadır.

Örnek Kod:

```

if (not1<50) //eğer not1 50'den küçükse
Console.Write("kaldı"); //kaldı yaz
else //aksi takdirde
Console.Write("geçti"); //geçti yaz
(Sıradaki komut;)

```

Örnek Kod:

```

if (not1<100 || not1>100)
Console.Write("geçersiz not1 değeri");
else
Console.Write("geçerli not1 değeri");
(Sıradaki komut;)

```

Örnek Kod:

```

if (not1>=50) //eğer not1 50 veya daha büyükse
{ //bu parantez içindeki komutlar çalışsın
    Console.WriteLine("Tebrikler!");
}

```

```

    Console.WriteLine("Sınıfı geçtiniz...");
}
else //aksi takdirde
{ //bu parantez içindeki komutlar çalışsın
    Console.Write("Çok yazık!");
    Console.Write("Sınıfta kaldınız...");
}

```

Örnek Kod:

```

if(a>b)
    Console.Write("a büyük");
else
{
    b++;
    c++;
}

```

If-Else If-Else Komutu

Bu komut aslında iç içe geçmiş **if-else** yapılarından oluşur. Doğal olarak da en karmaşık görünen kontrol komutudur; eğer birden fazla şart kontrol edilip uygun şarta göre komut çalıştırılacaksa **if-else if-else** komutu kullanılır.

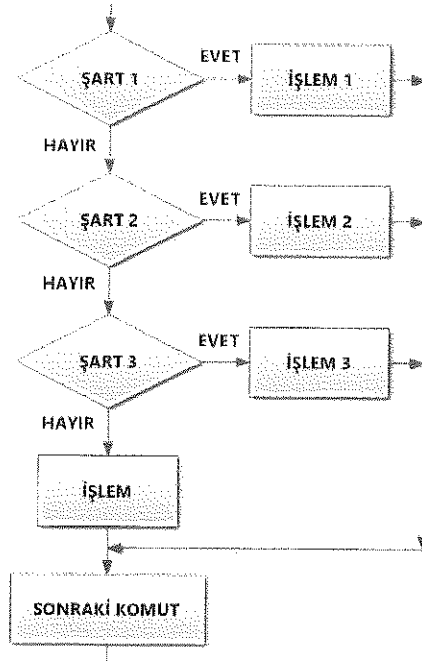
Örneğin hava durumunu kontrol ettiniz. Eğer hava çok soğuksa mont giymeye karar veriyorsunuz. Fazla çok soğuk değilse kazak giyiyorsunuz. Eğer ılık ise gömlek, sıcak ise t-shirt giymeye karar veriyorsunuz. Aksi durumda (yani çok sıcaksa) hiçbir şey giymiyorsunuz. Kaç şartı kontrol etmiş olduk? Beş şartı kontrol ediyoruz ve ona göre davranıyoruz.

- hava çok soğuksa
- hava çok soğuk değilse
- hava ılıksa
- hava sıcaksa
- yukarıdaki şartlar sağlanmazsa

İşte bir sürü şartın kontrol edildiği durumlarda **if-else if-else** komutu kullanılır. **if** ve **else-if** komutlarında tek şart kontrol edilirken **if-else if-else** komutunda birden fazla şart kontrol edilebilir. Sağlanan şarta göre de farklı komutlar çalıştırılır.

Aşağıdaki akış diyagramını dikkatlice inceleyelim; program **ŞART 1** sorgusuna geldiğinde eğer şart sağlanırsa **İŞLEM1**'i çalıştırır ve sorgudan çıkılır. Eğer **ŞART1**

sağlanmazsa doğrudan **ŞART2**'ye gider. Bu durumda da **İŞLEM2** çalıştırılır. Şartların sayısı istenildiği kadar artırılabilir. Eğer hiçbir şart sağlanmazsa en sondaki **İŞLEM** gerçekleştirilir.



Şekil 7.4 if-else if-else komutu

if-else if-else komutunun sözdizimi aşağıdaki gibidir:

```

if (şart 1)
    {işlem 1;}
else if (şart 2)
    {işlem 2;}
else if (şart 3)
    {işlem 3;}
else
    {işlem 4;}
(sonraki komut;)
  
```

if komutundan başlayarak **else if** komutları ile farklı şartlar kontrol edilir. Şartlardan biri sağlanırsa ilgili işlem yapılır ve sorgudan yani **if-else if-else** yapısından çıkılır, sonraki komuta geçilir. Eğer hiçbir şart sağlanmazsa **else** bloğundaki işlem yapılır ve sonraki komuta geçilir. Dikkat edilirse hiçbir şart gerçekleşmese bile mutlaka bir işlem gerçekleştiriliyor.

Örnek Kod:

İki değeri karşılaştırarak birbirlerine göre durumlarını değerlendireceğiz. Ortaya büyük, küçük veya eşit şeklinde bir sonuç çıkacak.

```
int x=10, y=20;
if(x<y)
    Console.Write("x küçük");
else if(x==y)
    Console.Write("x ile y eşit");
else
    Console.Write("x büyük");

Console.ReadLine();
```

Örnek Kod:

Öğrencinin aldığı 0-100 arası nota göre başarı durumunu tespit edelim. Eğer 50 veya daha az aldı ise kaldı, 50 ile 65 arası orta, 66 ile 80 arası iyi ve diğer durumlarda (yani 81 ile 100 arası) pekiyi olsun.

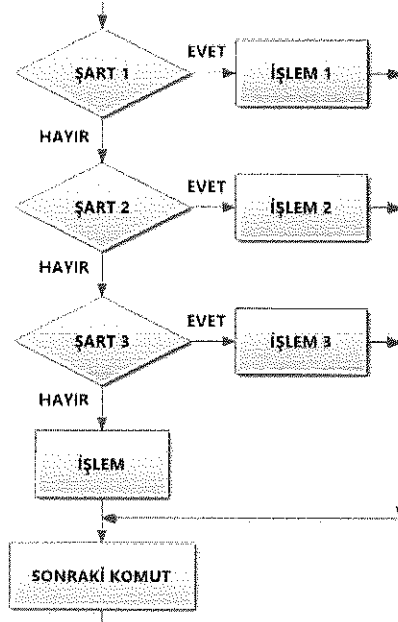
```
if (ort<50)
    {Console.Write("Kaldı");}
else if (ort<=65)
    {Console.Write("Orta");}
else if (ort<=80)
    {Console.Write("İyi");}
else
    {Console.Write("Pekiyi");}
```

Örnek Kod:

```
int yas;
yas=Convert.ToInt32(Console.ReadLine());
if(yas<2) //yaş 2'den küçükse
    Console.Write("Bebek");
else if(yas<12) //yaş 12'den küçükse
    Console.Write("Çocuk");
else if(yas<42) //yas 12 ile 42 arası ise (42 hariç)
    Console.Write("Genç");
else if(yas<62) //yas 42 ile 62 arası ise (62 hariç)
    Console.Write("Yetişkin");
else //diğer durumlarda
    Console.Write("İhtiyar");
```

Switch Komutu

if-else if-else komutuyla aynı mantıkta çalışır. Bir değişkenin farklı değerleri için farklı işlemlerin yapılmasını sağlar. Eğer şartlardan biri gerçekleşmezse en sonda belirtilen işlemin yapılmasını sağlar.



Şekil 7.5 Switch komutu

Switch komutunun sözdizimi aşağıdaki gibidir:

```

switch (değişken)
{
    case deger1:           //değişken bu değere sahipse
        işlem 1;           //buradaki işlem yapılır
        break;             //switch komutundan çıkılır
    case deger2:
        işlem 2;
        break;
    default:               //eğer yukarıdaki şartlardan herhangi biri çalışmazsa
        işlem 4;           //buradaki işlem yapılır
}
  
```

Örnek Kod:

Bu örnekte **x** adlı bir değişkenimiz var. **x** değişkenine bir değer atanıyor ve **switch** komutuyla değişkenin farklı değerleri için farklı işlemler yapılıyor. Eğer **x** değişkeni 0, 1 veya 2 değilse **default** adımındaki işlem yapılıyor.

```
int x=2;
switch(x) //x değişkenini kontrol edeceğiz
{
    case 0: //eğer x değeri 0 ise
        Console.Write("Değişken değeri 0"); //ekrana yaz
        break; //komuttan çık
    case 1:
        Console.Write("Değişken değeri 1");
        break;
    case 2:
        Console.Write("Değişken değeri 2");
        break;
    default: //x'in değeri yukarıda belirtilen değerlerden biri değilse
        Console.Write("Değişken farklı bir değere sahip"); //bunu yaz
}
```

?: Operatörü

Çok pratik olmasından ve az yer kaplamasından dolayı ?: operatörü yeri geldiğinde çok pratik kodlar yazmanızı sağlar. **if-else** komutu gibi çalışır. Bir şart kontrol edilir; eğer şart sağlanırsa bir işlem, sağlanmazsa başka bir işlem gerçekleşir.

?: Operatörünün sözdizimi aşağıdaki gibidir:

```
(şart)? işlem 1: işlem 2
```

Örnek Kod:

Bu örneğimizde **x** ve **y** adlı iki değişkenimiz var. **y** değişkeninin değeri **x** değişkeninin durumuna göre belirleniyor. Kodlara dikkat edilirse bir atama işlemi var. Eğer **x** 10'dan küçük olursa **y** değişkenine 0, aksi takdirde **y** değişkenine 10 değeri atanır.

```
int x, y;
Console.Write("x:");
x=Convert.ToInt32(Console.ReadLine()); //x değerini oku
y=(x<10) ? 0 : 10; //x<10 ise y=0 aksi takdirde y=10
Console.Write(y);
```

Örnek Kod:

Bu örnekte **a** ve **b** adlı iki değişken var. **b** değişkeninin değeri **a** değişkenine göre belirleniyor. Eğer **a** değişkeninin değeri 10 ise **b** 1 artırılıyor, aksi takdirde 1 azaltılıyor.

```
int a=5, b=0;
b=(a==10)?++b:--b;
Console.Write(b);
```

Dallanma Komutu (goto) & Etiket Kullanımı

Kodların takibini ve okunmasını zorlaştırdığı için, ayrıca yapısal programlamaya aykırı olduğu için "**goto**" komutu pek tavsiye edilen bir komut değildir. Her ne olursa olsun C# dilinde "**goto**" komutu vardır ve bilinmesinde de fayda vardır.

goto komutu veya **goto** komutuna benzeyen dallanma komutları özellikle assembly'de çok kullanılır; gerek 8086 işlemci ile çalışanlar veya gerekse PIC mikrokontrolör ile kod yazarlar bu ve buna benzer komutları kullanmak zorunda kalırlar.

goto komutunun çalışabilmesi için öncelikle bir etiket oluşturulmalıdır. Etiket belli bir satıra verilen isimdir. Etiket oluşturulduktan sonra belli bir şart sağlandığında program etiketin olduğu satıra gönderilir. **goto** komutu ve etiket aşağıdaki gibi kullanılabilir:

```
komut 1;
komut 2;
etiket:
komut 3;
goto etiket; //etiketin olduğu satıra git
```

Örnek Kod:

```
int x, kare;
gir: //gir isimli etiket

Console.Write("Sayı:");
x = Convert.ToInt32(Console.Read());

if (x < 0) //eğer x sıfırdan küçükse
goto gir; //gir etiketine git

kare = x * x; //kare hesaplanır

Console.Write("Kare=" + kare); //hesaplanan kare yazdırılır
```

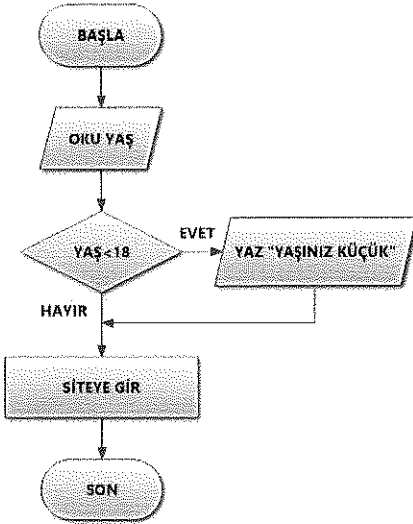
Karar/Sorgu Uygulamaları

Önceki konularda anlatılan **if**, **if-else** ve **switch** komutları ile ilgili örnekler yaparak kontrol komutlarını pekiştirmeye çalışacağız. Kolay örneklerden başlayarak daha zor ve karmaşık örneklere doğru ilerleyeceğiz.

Verilen örneklerdeki akış diyagramlarını iyice inceleyip programın algoritmasını yani nasıl çalıştığını anlamaya çalışın. Daha sonra da kodları gözden geçirin ve yazıp test edin.

Yaş Kontrolü

Bu örnekte kullanıcıdan yaş bilgisi alınacak. Eğer yaşı 18'den küçükse uyarılıp siteye öyle giriş yapacak. Yaşı 18'den büyükse herhangi bir uyarı yapılmadan siteye girecek.



```

static void Main(string[] args)
{
    //önce değişken tanımlayalım
    int yas;

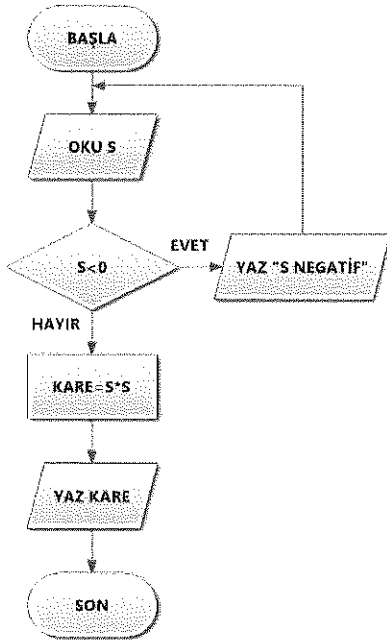
    //kullanıcının yaşını alalım
    Console.Write("Yaşınız:");
    yas = Convert.ToInt32(Console.ReadLine());

    //yaş bilgisini sorgulayalım
    //18'den küçükse uyaralım
    if (yas < 18)
        Console.WriteLine("Yaşınız küçük");

    //siteye giriş yaptıralım
    Console.Write("Siteye Gir...");

    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Sadece Pozitif Sayıların Karesini Alan Program



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, kare;

    sayi_al: //etiket

    //sayıyı alıp x'e atayalım
    Console.Write("Sayı:");
    x = Convert.ToInt32(Console.ReadLine());

    //okunan sayıyı sorgulayalım
    //sıfırdan küçükse yeniden alalım
    if (x < 0)
        goto sayi_al;

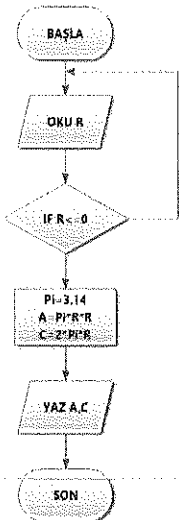
    //kareyi hesaplayalım
    kare = x * x;

    //sayıyı ve karesini yazalım
    Console.Write("sayı=" + x + " karesi=" + kare);

    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Dairenin Alanı ve Çevresi

Bu örneğimizde dairenin yarıçapı alınıyor ve kontrol ediliyor. Eğer yarıçap sıfır veya daha küçükse yeniden alınıyor.



```

static void Main(string[] args)
{
    //değişken ve sabitleri tanımlayalım
    double r, a, c;
    const double pi = 3.1415;

    r_al:

    //yarıçapı alalım
    Console.Write("yarıçapı");
    r = Convert.ToDouble(Console.ReadLine());

    //yarıçapı kontrol edelim
    //eğer sıfır veya sıfırdan küçükse
    //yeniden okuyalım
    if (r <= 0)
        goto r_al;

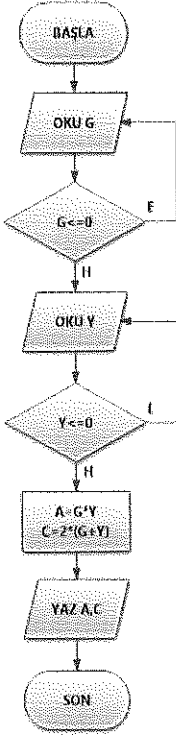
    //alan ve çevre hesabı
    a = pi * r * r;
    c = 2 * pi * r;

    //alan ve çevreyi yazalım
    Console.Write("alan=" + a + " çevre=" + c);

    Console.ReadLine(); //bekle
}
  
```

Dikdörtgenin Alanı ve Çevresi

Bu örneğimizde dikdörtgenin genişlik ve yükseklik değerleri kontrol ediliyor. Eğer girilen değerler sıfır veya daha küçükse yeniden alınıyor.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double a, c, g, y;

    //genişliği okuyalım
    g_oku:
    Console.Write("genişlik:");
    g = Convert.ToDouble(Console.ReadLine());
    if (g <= 0) goto g_oku;

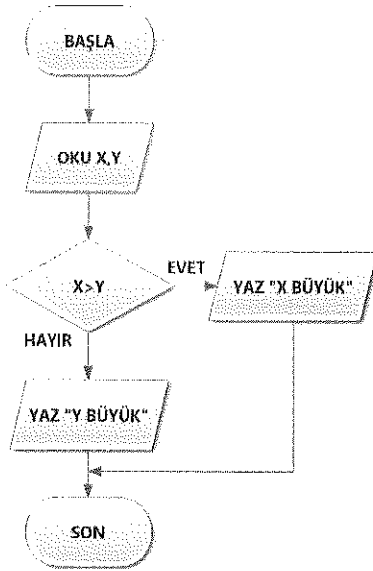
    //yüksekliği okuyalım
    y_oku:
    Console.Write("yükseklik:");
    y = Convert.ToDouble(Console.ReadLine());
    if (y <= 0) goto y_oku;

    //alan ve çevreyi hesaplayalım
    a = g * y;
    c = 2 * (g + y);

    //alan ve çevreyi yazalım
    Console.Write("alan=" + a + " çevre=" + c);

    Console.ReadLine(); //pencereyi kapatma
}
  
```

Girilen İki Sayıdan Büyük Olanı Bulan Program



```

static void Main(string[] args)
{
    //değişkenleri oluşturalım
    int x, y;

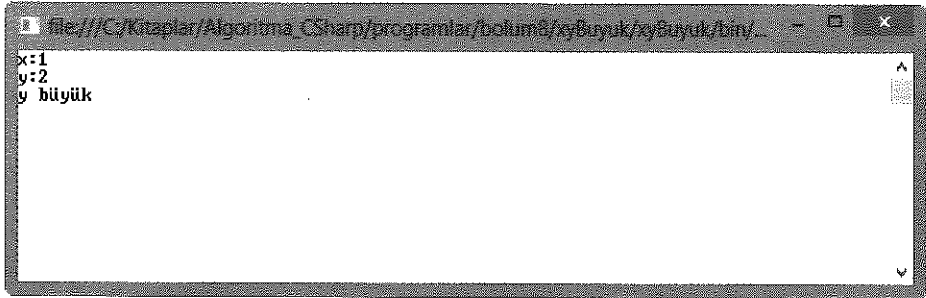
    //x değerini okuyalım
    Console.Write("x:");
    x = Convert.ToInt32(Console.ReadLine());

    //y değerini alalım
    Console.Write("y:");
    y = Convert.ToInt32(Console.ReadLine());

    //x ile y durumunu sorgulayalım
    if (x > y)
        Console.WriteLine("x büyük");
    else
        Console.WriteLine("y büyük");

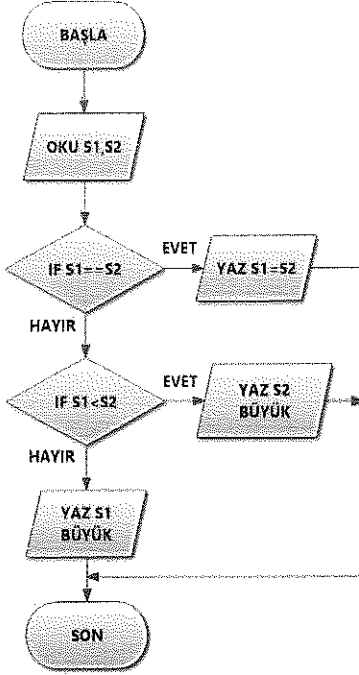
    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Ekran Çıktısı:



Girilen İki Sayının Durumu

Bu örneğimizde konsoldan iki sayı alacağız ve bu sayıların büyük, küçük veya eşit olma durumunu kontrol edeceğiz.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s1, s2;

    //s1 ile s2 değerlerini okuyalım
    Console.Write("s1:");
    s1 = Convert.ToInt32(Console.ReadLine());

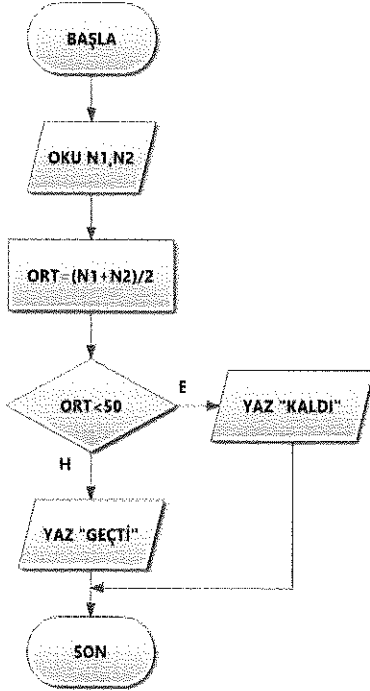
    Console.Write("s2:");
    s2 = Convert.ToInt32(Console.ReadLine());

    //s1 ile s2 değerlerinin sınanması
    if (s1 == s2)
        Console.Write("s1=s2");
    else if
        (s1 < s2) Console.Write("s2 büyük");
    else
        Console.Write("s1 büyük");

    Console.ReadLine();//bekle
}
  
```

Geçti-Kaldı Programı

Bu örneğimizde bir derste alınan 0-100 arası notun ortalamasını alacağız. Alınan notların 0-100 arası olması gerekiyor. Eğer alınan değer 0-100 arası değilse tekrar isteniyor.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n1, n2, ort;

    //n1 değerini alalım
n1:
    Console.Write("n1:");
    n1 = Convert.ToInt32(Console.ReadLine());
    if (n1 < 0 || n1 > 100) goto n1;

    //n2 değerini alalım
n2:
    Console.Write("n2:");
    n2 = Convert.ToInt32(Console.ReadLine());
    if (n2 < 0 || n2 > 100) goto n2;

    //ortalama hesabı
    ort=(n1+n2)/2;

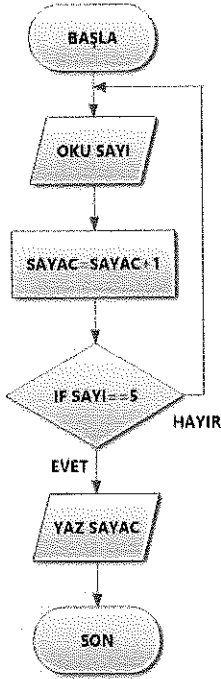
    //sonuç
    if (ort >= 50) Console.Write("geçti");
    else Console.Write("kaldı");

    Console.ReadLine();//bekle
}
  
```

Sayaç Kullanımı

Bu uygulamamızda girilen sayıların kaç adet olduğunu öğrenmek için sayaç kullanacağız. Örneğin 5 gibi belli bir sayı girildiğinde sayaç saymayı bırakacak ve sayaç değeri ekrana yazdırılacak.

Algoritmaya dikkatli bir şekilde bakacak olursanız önce sayı okunur. Sayı okunduktan hemen sonra sayaç değeri 1 artar. Eğer girilen sayı 5 ise sayaç değeri yazdırılır. Aksi takdirde yeni bir sayı okunur.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int sayi, sayac = 0;

    oku_sayi:
    //konsoldan bir sayı okuyalım
    Console.Write("sayı giriniz:");
    sayi = Convert.ToInt32(Console.ReadLine());

    //sayacı 1 arttıralım
    sayac = sayac + 1;

    //sayı kontrolü
    if (sayi == 5) Console.Write("sayaç=" + sayac);
    else goto oku_sayi;

    Console.ReadLine(); //bekle
}
  
```

Ekran Çıktısı:

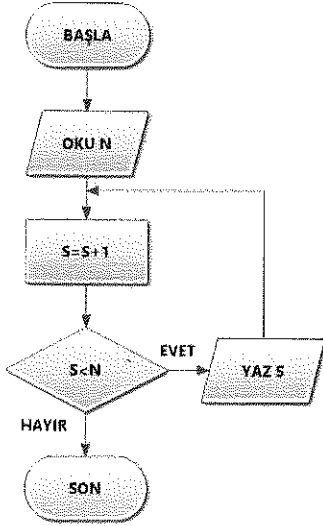
```

file:///C:/Kitaplar/Algoritma_CSharp/programlar/bolum2/sayac/sayac/bin/Debug...
sayı giriniz:1
sayı giriniz:2
sayı giriniz:3
sayı giriniz:1
sayı giriniz:2
sayı giriniz:3
sayı giriniz:4
sayı giriniz:5
sayaç=8
  
```

1'den N'e Kadar Olan Sayıların Yazdırılması

Bu örneğimizde konsoldan bir sayı (n) alacağız ve 1'den n'e kadar olan tüm sayıları ekrana yazdıracağız.

Sayıları sayaç (s) kullanarak oluşturacağız ve sayacı n değerine kadar arttıracacağız. Elde edilen her değeri konsola yazdıracağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, s = 0;

    //konsoldan n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

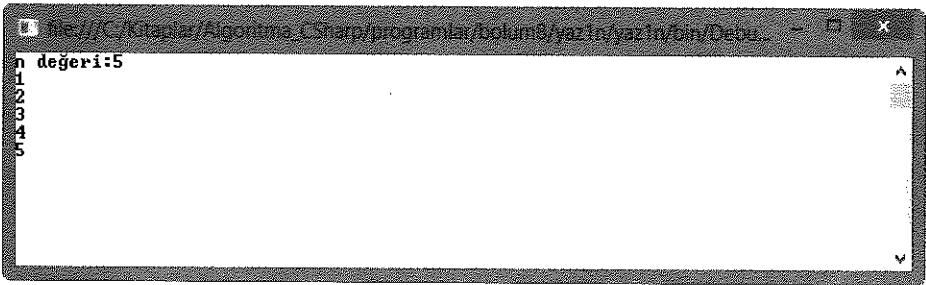
    art:
    //sayacı 1 arttıralım
    s = s + 1;

    //sayacın değerini konsola yazalım
    Console.WriteLine(s);

    //sayacın değerini kontrol edelim
    //eğer n'den küçükse "art" etiketine gitsin
    if (s < n) goto art;

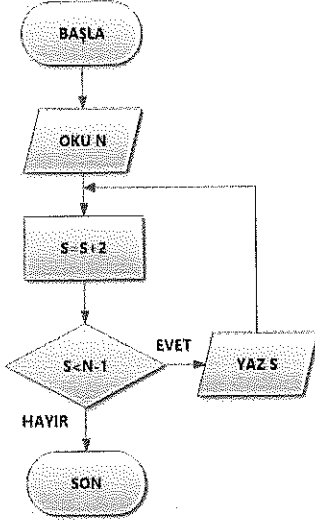
    Console.ReadLine(); //bekle
}
  
```

Ekran Çıktısı:



1'den N'e Kadar Olan Çift Sayıların Yazdırılması

Bir önceki örneğe çok benzeyen bu örnekte gene sayaç kullanarak çift sayıları oluşturacağız. Sayıları teker teker arttırmak için $s=s+1$ kullanılırken, çiftler çiftler arttırmak için $s=s+2$ kullanacağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, s = 0;

    //konsoldan n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

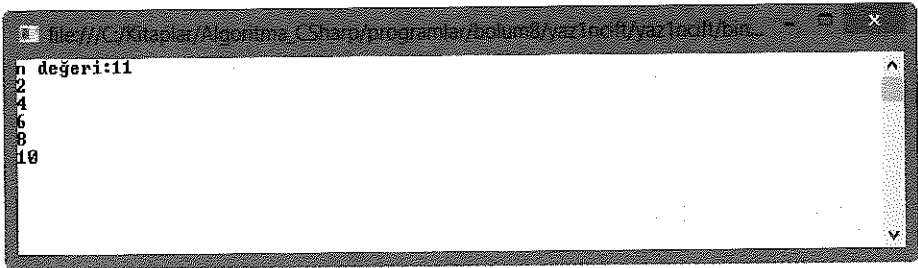
    art:
    //sayacı 2 arttıralım
    s = s + 2;

    //sayacın değerini konsola yazalım
    Console.WriteLine(s);

    //sayacın değerini kontrol edelim
    //eğer n-1'den küçükse "art" etiketine gitsin
    if (s < n-1) goto art;

    Console.ReadLine(); //bekle
}
  
```

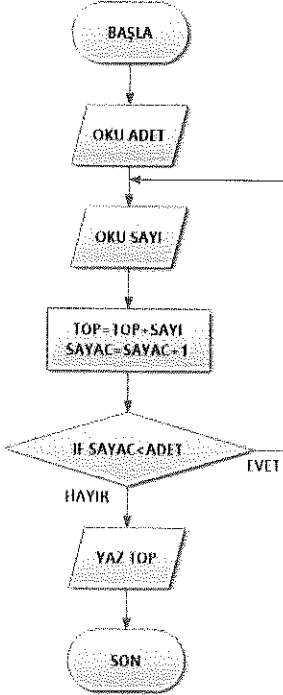
Ekran Çıktısı:



Ekrandan Belli Bir Adet Sayı Alınması

Ekrandan belli bir adet sayı alınması için önce girilecek sayı adeti alınmalıdır. Daha sonra sayaç kullanılarak belirtilen sayıya gelinceye kadar dışarıdan sayı alınmalıdır.

Aşağıdaki örnekte girilen belli adet sayının toplamı bulunur.



```

static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    int adet, sayi, top=0, sayac=0;

    //okunacak sayı adetini alalım
    Console.Write("adet:");
    adet = Convert.ToInt32(Console.ReadLine());

    oku:
    //konsoldan sayıyı okuyalım
    Console.Write("sayı gir:");
    sayi = Convert.ToInt32(Console.ReadLine());

    //sayacın değerini 1 artıralım
    sayac = sayac + 1;

    //okunan sayıyı top değişkenine ekleyelim
    top = top + sayi;

    //sayacın durumunu kontrol edelim
    if (sayac < adet)
        goto oku;
    else
        Console.WriteLine("toplam="+top);

    Console.ReadLine();//bekle
}
  
```

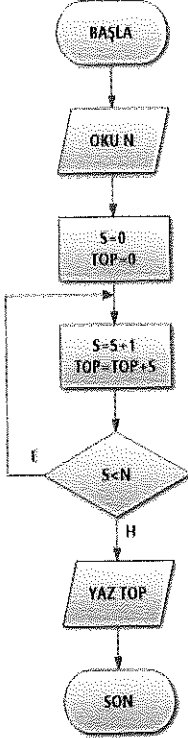
Ekran Çıktısı:

```

file:///C:/Kutuplu/Algoritma/CSharp/programlar/bolum8/toplam1n/toplam1n/bi...
adet:5
sayı gir:1
sayı gir:2
sayı gir:3
sayı gir:4
sayı gir:5
toplam=15
  
```

1'den N'e Kadar Olan Sayıların Toplamı

Bu algoritmada S adlı bir sayaç değişkeni kullanacağız. Sayaç 1'den başlayacak ve girilen N değerine kadar artacak. Böylece 1, 2, 3, ..., N değerine kadar olan sayıları elde etmiş olacağız. Sayaç N değerine geldiğinde son defa işlem yapılacaktır; sayacın her yeni değerinde S değeri TOP değişkenine eklenecek.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int top = 0, s = 0, n;

    //n değerini alalım
    Console.WriteLine("n:");
    n = Convert.ToInt32(Console.ReadLine());

    art:
    //sayacı 1 arttıralım ve top'a ekleyelim
    s = s + 1;
    top = top + s;

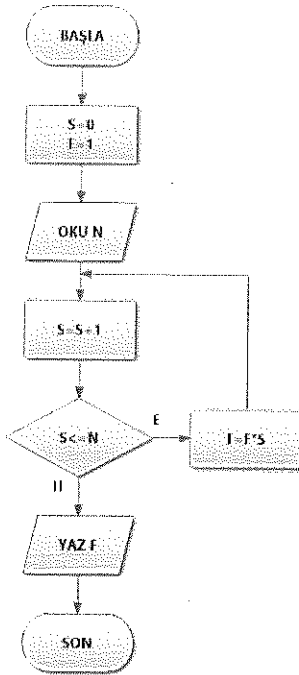
    //sayacı kontrol edelim
    if (s < n)
        goto art;
    else
        Console.WriteLine("toplam=" + top);

    Console.ReadLine(); //bekle
}
  
```

Faktöriyel Hesabı

Karşımıza çok çıkan işlemlerden bir tanesi de faktöriyel hesabıdır. Başlıktan da anlayabileceğiniz gibi faktöriyel 1'den N'e kadar olan sayıların çarpımı ile elde edilir.

Faktöriyel hesabında dikkat edilmesi gereken en önemli konu kullanılacak olan ve faktöriyel sonucunu gösteren F değişkenidir. Hatırlarsanız daha önceki toplama işlemlerinde TOP=0 şeklinde bir atama yaparak toplamın başlangıç değerinin sıfır olmasını sağlamıştık. Faktöriyel hesabında ise F değişkeninin başlangıç değeri 1 olmalıdır. Eğer sıfır yaparsanız faktöriyel işleminde sonuç hep sıfır olur.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s = 0, f = 1, n;

    //n değerini okuyalım
    Console.Write("n:");
    n = Convert.ToInt32(Console.ReadLine());

    art:
    //sayacı 1 arttıralım
    s = s + 1;

    //sayacın durumunu sorgulayalım
    if (s <= n)
    {
        f = f * s;
        goto art;
    }

    //n faktoriyeli yazalım
    Console.Write(n + " faktoriyel=" + f);

    Console.ReadLine(); //bekle
}

```

Haftanın Gününü Yazmak

Bu örneğimizde haftanın kaçınıcı günü olduğunu gireceğiz ve girilen sayıya göre günün adını yazdıracağız.

Bir haftada en fazla 7 gün olduğu için değişken veritipi olarak **byte** (0:255) kullanılabilir. **Integer** veritipi yerini **byte** kullanmak bellekte daha az yer kaplar ve programımızın kaynak ihtiyacını azaltır.

Girilen sayının 1 ile 7 arasında olup olmadığını kontrol edeceğiz. Eğer farklı aralıkta bir sayı girildiyse yeniden sayı girilmesini isteyeceğiz.


```

static void Main(string[] args)
{
    //değişkeni tanımlayalım
    byte gun_no;

    //haftanın kaçınıcı günde olduğunu alalım
gunoku:
    Console.Write("Gün No:");
    gun_no = Convert.ToByte(Console.ReadLine());

    //alınan gün numarasını sorgulayalım ve işlem yapalım
    switch (gun_no)
    {
        case 1: Console.WriteLine("Pazartesi"); break;
        case 2: Console.WriteLine("Salı"); break;
        case 3: Console.WriteLine("Çarşamba"); break;
        case 4: Console.WriteLine("Perşembe"); break;
        case 5: Console.WriteLine("Cuma"); break;
        case 6: Console.WriteLine("Cumartesi"); break;
        case 7: Console.WriteLine("Pazar"); break;
        default: goto gunoku;
    }

    Console.ReadLine(); //bekle
}

```

Alıştırmalar

- Konsoldan okunan bir sayının (x) 10 değerine göre durumunu (büyük, küçük veya eşit) karşılaştırın.
- Konsoldan okunan bir sayının (x) ikinci sayıya (y) göre durumunu karşılaştırın.
- İki vize notunun %40'ı ve final notunun %60'ını kullanarak ortalama notunu hesaplayın. Ortalama 50'den küçükse "KALDI" aksi takdirde "GEÇTİ" yazınız.
- Konsoldan okunan bir değer 0 ile 100 arasında olup olmadığını kontrol edin. 0-100 arası ise "EVET" değilse "HAYIR" yazdırın.
- Konsoldan okunan bir değer 0'dan küçük veya 100'den büyük olup olmadığını kontrol edin. Şart sağlanırsa "EVET" sağlanmazsa "HAYIR" yazdırın.
- Konsoldan bir sayı alın. Eğer sayı 0'dan küçükse soğuk, 0-15 arası ise ılık aksi takdirde sıcak olarak yazın.
- Girilen iki değer (x ile y) arasındaki sayıları ekrana yazdırın.
- Girilen iki değer (x ile y) arasındaki çift sayıları ekrana yazdırın.
- Girilen iki değer (a ile b) arasındaki sayıların toplamını bulun.
- Girilen n adet sayının ortalamasını bulunuz.

8

Döngüler

- Döngü Nedir?
- Döngü Tipleri
- For Döngüsü
- Foreach Döngüsü
- While Döngüsü
- Do-While Döngüsü
- Break ve Continue Komutları
- Döngü Uygulamaları

Döngü Nedir?

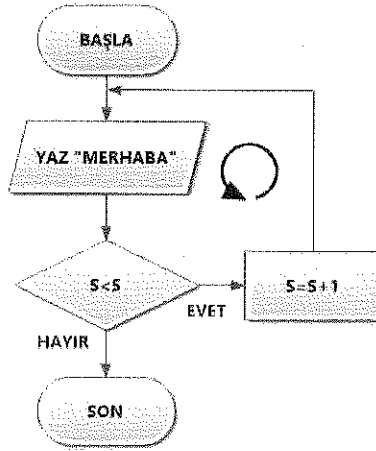
Bir işlemi belli bir sayıda veya belli bir şart gerçekleştiği sürece veya belli bir şart gerçekleşene kadar tekrar eden kod yapısına döngü denir. Kontrol yapıları ile beraber döngü komutları programcılıkta en çok kullanılan komutlardır.

Hepimizin bildiği gibi, bilgisayarların en önemli özelliklerinden biri saniyede milyonlarca işlem yapmasıdır. Ayrıca bilgisayarlar insanlar gibi sürekli aynı işi yapmaktan şikayet etmezler. Bu nedenle bilgisayarların veya bilgisayar kontrollü makinelerin çok sık kullanıldığı yerlerden biri de sürekli aynı ürünün üretilmesini sağlayan işlemlerdir. Örneğin su fabrikasındaki bilgisayar kontrollü bir makine hiç üşenmeden ve şikayet etmeden günde binlerce şişenin kapağını kapatabilir.

İsterseniz çok fazla ilerlemeden döngülere günlük hayatımızdan örnekler vermeye çalışayım. Örneğin evden çıktınız. Varmak istediğiniz yere kadar adım atarak ilerliyorsunuz. Aslında hedefe varıncaya kadar adım atma eylemi bir tür döngüdür; belli bir şart gerçekleşene kadar aynı eylemi tekrar ediyorsunuz. Ya da köfte yaptığınızı düşünün. Kıyma bitinceye kadar tekrar tekrar eti/kıymayı küçük parçalara bölüyorsunuz. Bir restoranda çalışığınızı ve 10 adet tost siparişi aldığınızı düşünün. Bu durumda aynı işi belli bir sayıda yapacaksınız demektir ki bu da bir döngüdür.

Şimdi de programcılıktan bir örnek verelim. Ekrana 5 defa "Merhaba" yazdırmak istediğinizi düşünün. Burada da tekrarlı bir yapı söz konusudur ve bu sorunu halletmenin en kolay yöntemi bir döngü oluşturmaktır.

Aslında kontrol yapılarında döngülerle uğraştık ama gelin döngülerin nasıl oluştuğuna tekrar bir göz atalım: Eğer bir işlem belli bir defa tekrar edilecekse bir sayaç olmalıdır; sayaç her işlemde 1 artırılır ve istenilen değere ulaştığında işlemler sonlandırılır. Bu işlemin algoritması aşağıdaki akış diyagramında görüldüğü gibidir:



Şekil 8.1 Döngü

Yukarıdaki akış diyagramını iyice inceleyecek olursak aşağıdaki adımları görürüz:

1. Program çalışmaya başlar.
2. İlk işlem olarak "MERHABA" yazar.
3. Sayaç kontrol edilir. Eğer 5'ten küçükse sayaç 1 arttırılır ve yeniden "MERHABA" yazdırılır. Böylece bir döngü gerçekleşmiş olur.
4. Eğer sayaç 5 ise program sona erer.

Döngü Tipleri

Daha önce de bahsedildiği gibi döngüler bir işlemi belli bir sayıda yapan veya belli bir şart sağlandığı sürece yapan kodlardır. Eğer döngü sayısı belli ise bu tür döngülere **sabit döngü** denir. Eğer döngü sayısı belli değilse ve belli bir şart gerçekleştiği sürece döngü çalışırsa bu tür döngülere **şartlı döngü** denir.

Döngü Tipleri:

- Sabit Döngü Komutları
 - for
 - for-each
- Şartlı Döngü Komutları
 - while
 - do-while

Yukarıda bahsi geçen döngü komutlarını ayrıntılı olarak inceleyeceğiz ve bol bol uygulama yaparak pekiştirmeye çalışacağız.

For Döngüsü

Basit ve az yer kaplayan yapısından dolayı **for()** döngüsü en çok kullanılan döngü komutlarından biridir; eğer bir işlemin veya döngünün tekrar sayısı belli ise **for()** döngüsü kullanılır.

for() döngüsünde döngünün kaç defa çalıştığını belirleyen bir **sayaç**, döngünün çalışıp çalışmayacağını belirleyen bir **şart** ve sayacın artış durumunu belirleyen **sayaç artış** ifadesi bulunur. **for()** döngüsü ile bir veya daha fazla komut çalıştırabiliriz. Eğer tek bir komut çalıştırılacaksa **for()** döngüsünün genel yapısı aşağıdaki gibidir:

```
for(sayaç; şart; sayaç artışı)
komut;
```

Birden fazla komut çalıştırılacaksa komutlar blok içine, yani güzel parantez içine alınmalıdır. Bu durumda **for()** döngüsü aşağıdaki gibi olur:

```
for(sayaç; şart; sayaç artışı)
{
    komut1;
    komut2;
```

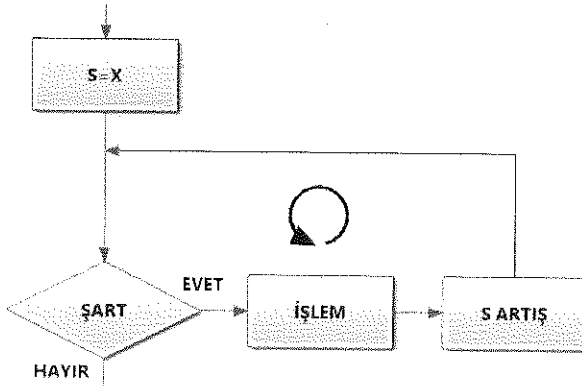
```

    komut3;
    ...
}

```

Sayaç, adından da belli olduğu gibi, döngünün kaç defa çalıştığını sayar. Sayacın daha önceden veya **for()** döngüsü içinde tanımlanması ve başlangıç değerinin belirtilmesi gerekmektedir. Şart, döngünün hangi şart sağlanırsa devam edeceğini belirler. Sayaç artışı ise sayacın nasıl artacağını belirler. Örneğin sayaç 1 artırılarak veya 2 azaltılarak işlem yapılabilir.

For komutunun akış diyagramı aşağıda görüldüğü gibidir. Akış diyagramı iyice incelenirse önce sayaç değişkeni (**S**) tanımlanarak bir başlangıç değeri atanmaktadır. Daha sonra şart kontrol edilmektedir; eğer şart sağlanırsa ilgili komut veya komutlar çalıştırılır ve sayaç artırılarak döngüye devam edilir. Eğer şart sağlanmazsa döngü sona erer.



Şekil 8.2 For döngüsü

Örnek Kod:

```

//konsola 5 defa "Merhaba C#" yazalım
//sayaç 1'den başlar ve 5 oluncaya kadar artar
for(int s=1; s<=5; s++)
Console.WriteLine("Merhaba C#"); //işlemi yap
Console.Read(); //bekle

```

Yukarıdaki örnek kodlar iyice incelenecek olursa **for** döngüsünde sayacın **int s** olarak tanımlandığı ve başlangıç değeri olarak **s=1** değerinin atandığı görülecektir. Şart olarak **s** değişkeninin 5 veya daha küçük olması (**s<=5**) belirtilmiştir. Sayacın artış değeri ise **s++** komutu kullanılarak her döngüde 1 fazla olacak şekilde ayarlanmıştır.

Örnek Kod:

```
//sıfırdan n'e kadar olan çift sayıların toplamı
int top=0;
for (int s=0; s<=10; s=s+2)
    top = top+s; //döngü çalıştığı sürece bu işlem yapılır
Console.Write("Toplam="+top); //top değişkeninin değerini yaz
Console.Read(); //bekle
```

Örnek Kod:

Aşağıdaki örnekte **for()** döngüsü içinde iki komut bulunmaktadır. İki komut olduğu için komutlar blok {...} içinde yazılmaktadır. Dolayısıyla döngünün her adımında **x** değeri konsola yazılmakta ve aynı zamanda **y** değeri 1 arttırılmaktadır.

```
int y=0;
for(int x=0; x<=10; x++)
{
    Console.WriteLine(x);
    y++;
}
Console.Write("döngü sayısı="+y);
Console.Read(); //bekle
```

Bazı durumlarda **for()** döngüsünün sayaç, şart veya artış değeri döngü dışında veya döngü içinde tanımlanabilir. Bu durumlarda ilgili parametre boş bırakılır. Örneğin döngü sayacı döngüden önce tanımlandıysa **for()** döngüsü aşağıdaki gibi olur:

```
int sayac=0; //sayaç başlangıcı
for(; sayac<10; sayac++)
    Console.WriteLine(sayac);
Console.Read();
```

Örnek Kod:

1'den 10'a kadar olan sayıların toplamının konsola yazılması için aşağıdaki kodlar kullanılabilir:

```
int s=1; //sayaç başlangıcı
int top=0;
for(; s<=10; s++)
    top=top+s;
Console.Write("toplam="+top);
Console.Read();
```

Örnek Kod:

Yukarıdaki program aşağıdaki gibi de yazılabilir. Bu programda sayaç artışı **for()** içinde değil de döngü bloğu içinde belirtilmiştir.

```
int top=0;
for(int s=1; s<=10; )
{
    top=top+s;
    s++; //sayaç artışı
}
Console.Read();
```

Bazen **for()** döngüsünde sadece şart ifadesini görebiliriz. Aşağıdaki programda döngünün başlangıç değeri ve sayaç artışı **for()** parantezinin dışında tanımlanmıştır.

```
int s=1; //sayaç başlangıç değeri
for(; s<=10; )
{
    top=top+s;
    s++; //sayaç artışı
}
Console.Read();
```

Bazı durumlarda **for()** döngüsü içinde birden fazla sayaç veya artış ifadesi olabilir. Şart ifadesi tek olmalıdır. Aşağıdaki örnekte bu durumu görebilirsiniz.

```
for(int i=0, int y=5; i<=10; i++, y++)
Console.Write("i="+i+ " y="+y);
Console.Read();
```

Görüldüğü gibi **for()** döngüsü hem kolay hem de çok esnek bir yapıya sahiptir. Bu nedenle en çok tercih edilen döngü komutudur. Şahsen ben eğer kullanabileceksem **for()** döngüsünü kullanırım. Eğer **for()** döngüsü uygun değilse diğer döngülerini kullanırım.

Foreach Döngüsü

foreach döngüsü daha çok dizilerde veya veritabanında kullanılır; dizi elemanları veya veritabanı kayıtları üzerinde listeleme veya diğer bazı komutları çalıştırmamızı sağlar.

For döngüsü daha çok sıralı değerler üzerinde (örneğin 1'den 5'e kadar olan sayıların toplamı) işlem yapmamızı sağlarken **foreach** komutunda bir dizinin daha karmaşık değerlere sahip elemanları kullanılabilir.

foreach komutunun genel yapısı aşağıdaki gibidir:

```
foreach(döngü_değişkeni in dizi)
işlem;
```

veya

```
foreach(döngü_değişkeni in dizi)
{
    işlem1;
    işlem2;
    işlem3;
    ...
}
```

Aynı **for()** döngüsünde olduğu gibi eğer tek bir işlem yapılacaksa güzel paranteze gerek yok fakat birden fazla işlem yapılacaksa komutlar güzel parantez içine yazılmaktadır.

foreach döngüsünde döngü değişkeni dizi içindeki değerleri sırayla alarak dizinin eleman sayısı kadar işlem yapılmasını sağlar. Yani dizinin 3, 1, 7, 12 gibi dört elemanı varsa her eleman için işlem yapılır.

Henüz daha dizileri öğrenmiş değiliz. O nedenle aşağıdaki örnek size biraz zor veya anlaşılmasa gelebilir. Şimdilik sadece **foreach** yapısını anlamaya çalışın. Dizi konusu işlendikten sonra **foreach** komutu ile uygulamalar yapılacak ve daha iyi anlaşılması sağlanacaktır.

Örnek Kod:

```
int[] dizim={3,1,7,12}; //dizim isimli dizi değişkeni ve elemanları
int top=0; //toplamın tutulacağı değişken
foreach(int s in dizim) //dizinin s ile temsil edilen her elemanı için
top=top+s; //dizinin elemanlarını top değişkeninde topla
Console.WriteLine("toplam="+top);
Console.Read(); //bekle
```

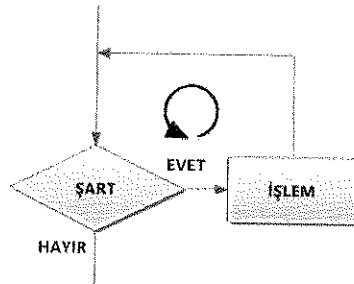
Dikkat edilirse ilk satırda **dizim** isimli dizi değişkeni ve dizinin elemanları tanımlanmaktadır. **foreach** komutu ile dizinin her elemanını temsil eden ve işlem içinde kul-

lanılmasını sağlayan bir **s** değişkeni tanımlanmaktadır. **s** değişkeni her döngüde sırayla dizinin başka bir elemanını alarak **top** değişkeninde toplamaktadır. Yani döngü dizinin her elemanı için tekrar edilir ve işlem yapılır.

While Döngüsü

Döngü sayısının belli olmadığı durumlarda **while** döngüsü kullanılır; belirtilen şart sağlandığı sürece döngü çalışmaya devam eder.

While döngüsünün akış diyagramını aşağıda görebilirsiniz. Dikkat edilirse önce şart kontrol edilir; eğer şart sağlanırsa işlem yapılır ve döngü devam eder. Şart sağlanmazsa döngüden çıkılır. Yani **while** döngüsünün çalışabilmesi için önce şartın sağlanması gerekir.



Şekil 8.3 While döngüsü

While döngüsünün yapısı aşağıdaki gibidir:

```
while(şart)
işlem;
```

Eğer **while** döngüsünde birden fazla komut olacaksa komutlar aşağıdaki gibi parantez içinde olmalı.

```
while(şart)
{
    işlem1;
    işlem2;
    işlem3
    ...
}
```

Örnek Kod:

```
//ekrana 5 defa merhaba yazdıralım
int s=1 //sayaç değişkeni
while(s<=5) //s=5 veya daha küçükse
{
    Console.Write ("merhaba");
    s++; //s değerini 1 arttır
}
Console.Read(); //bekle
```

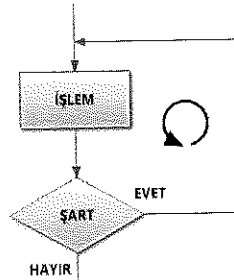
Örnek Kod:

```
//1'den 5'e kadar olan sayıların toplamı
int s=1, top=0 //sayaç ve toplam değişkenleri
while(s<=5) //s=5 veya daha küçükse
{
    top=top+s; //sayaç değişkenini top değişkeninde topla
    s++; //s değerini 1 arttır
}
Console.Write ("toplam="+top);
Console.Read(); //bekle
```

Do-While Döngüsü

Bir önceki konuda **while** döngüsünü anlatırken önce şartın kontrol edildiğini ve şart sağlanırsa işlem yapıldığını öğrenmiştik. **do-while** komutunda ise önce işlem yapılır daha sonra şart kontrol edilir. Şart sağlandığı sürece de döngü devam eder. Yani **do-while** komutunda işlem ya da döngü en az bir defa çalışır.

do-while komutunun akış diyagramını aşağıdaki şekilde görebilirsiniz. Dikkat edilirse önce işlem yapılmaktadır. İşlem yapıldıktan sonra şart kontrol edilmekte ve eğer şart sağlanırsa döngü devam etmektedir. Eğer şart sağlanmazsa döngü sona ermektedir.



Şekil 8.4 do-while döngüsü

Eğer döngünün en az bir defa çalışmasını istiyorsanız **do-while** komutunu kullanmalısınız. **do-while** döngüsünün yapısı aşağıda görüldüğü gibidir:

```
do
{
    işlemler;
}
while(şart);
```

Örnek Kod:

```
//konsola 5 defa merhaba yazdıralım
int s=1;
do
{
    Console.WriteLine("merhaba");
    s++;
}while(s<=5);
Console.Read(); //bekle
```

Örnek Kod:

```
//1'den 5'e kadar olan sayıların toplamı
int s=1, top=0;
do
{
    top=top+s;
    s++;
}while(s<=5);
Console.Write("toplam="+top);
Console.Read();
```

Örnek Kod:

Aşağıdaki kodlarda konsoldan bir pozitif sayı girilinceye kadar yeni değer okunmaktadır. Pozitif bir değer girildiği anda döngü sona ermekte ve okunan değer konsola yazılmaktadır.

```
int s;
do
{
    Console.Write("sayı giriniz:");
    s=Convert.ToInt32(Console.ReadLine());
}while(s<=0);
Console.Write(s);
Console.Read();
```

Break ve Continue Komutları

break ve **continue** komutları kontrol ve döngü komutlarının akışını kontrol etmek için kullanılır. Kısaca açıklamak gerekirse **break** komutu yapıları sonlandırırken **continue** komutu ise döngüde belli şarta uyan adımın atlanmasını sağlar.

Break Komutu

break komutu **while**, **for**, **do-while** ve **switch** komutlarında ilgili kontrol ya da döngü yapısının sonlandırılmasını ve sıradaki komuta geçilmesini sağlar. Örneğin bir okul programında 500 kişi arasından istenilen numara bulununca işlem sona erdirilebilir ve geri kalan adımlar çalıştırılmayabilir.

Örnek Kod:

Bu örneğimizde 1-5 arası sayıların toplamı hesaplanır. Ancak s=3 ise **break** komutundan dolayı döngü sona erer. Bu durumda toplama 3 dahil edilmez. Sonuçta 1 ile 2 değerlerinin toplamı bulunur.

```
int top;
for(int s=1; s<=5; s++)
{
    if(s==3) //eğer s=3 ise
        break; // döngüden çık
    top=top+s;
}
Console.Write("toplam="+top);
Console.Read();
```

Örnek Kod:

Aşağıdaki örnekte **switch** komutunda her işlem sonunda **break** komutu çalıştırılır. Böylece işlem bittikten sonra yapı dışına çıkılır ve sıradaki komut çalıştırılır.

```
int d;
Console.Write("1-5 arası değer giriniz:");
d=Convert.ToInt32(Console.ReadLine());
switch(d)
{
    case 1: Console.Write("1 girildi"); break;
    case 2: Console.Write("2 girildi"); break;
    case 3: Console.Write("3 girildi"); break;
    case 4: Console.Write("4 girildi"); break;
    case 5: Console.Write("5 girildi"); break;
    default: Console.Write("yanlış giriş"); break;
}
```

Continue Komutu

continue komutu, **while**, **for** ve **do-while** yapılarında bir şart sağlandığında o şarta uyan değerın çalışmamasını ve döngünün bir sonraki değerden devam etmesini sağlar.

Örnek Kod:

Bu örneğimizde 1-5 arası sayıların toplamı hesaplanır. Ancak s=3 ise **continue** komutundan dolayı döngü sonraki değere geçer. Bu durumda toplama 3 dahil edilmez. Sonuçta 1, 2, 4, 5 değerlerinin toplamı bulunur.

```
int top;
for(int s=1; s<=5; s++)
{
    if(s==3) //eğer s=3 ise
        continue; // sonraki değere geç
    top=top+s;
}
Console.Write("toplam="+top);
Console.Read(); //bekle
```

Sonsuz & Ölü Döngü

Eğer döngüdeki şart sürekli sağlanırsa sonsuz yani hiç bitmeyen bir döngü oluşur. Eğer döngüdeki şart hiç sağlanmazsa ölü yani hiç çalışmayan bir döngü oluşur.

Örnek Kod:

```
int x=5, y=10;
while(x<y)
Console.Write("x küçük");
Console.Read();
```

Yukarıdaki kodlarda **x** her zaman **y** değişkeninden küçük olacağı için sonsuz bir döngü oluşur. Sonsuz döngüler muhtemelen yazım hatası veya mantık hatalarından meydana gelmektedir. Sonsuz döngü oluştuğunda tüm işlemci gücünü sömürür ve bilgisayarın çok yavaş çalışmasını sağlar.

Aşağıdaki örnekte ise şart hiçbir zaman sağlanamayacağı için döngü de hiç çalışmayacaktır.

Örnek Kod:

```
int x=5, y=10;
while(x>y)
Console.Write("x küçük");
Console.Read();
```

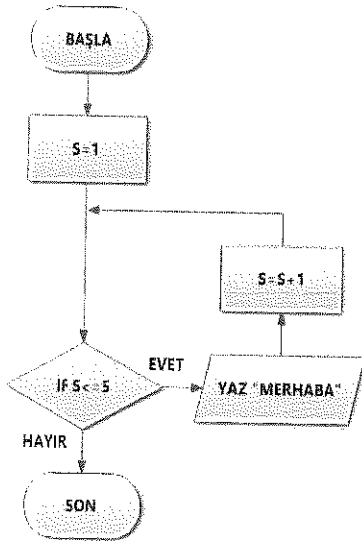
Döngü Uygulamaları

Şu ana kadar **for**, **foreach**, **while** ve **do-while** yapılarını gördük. Eğer örnek kodları ve akış diyagramlarını incelediyseniz döngülerin çalışma mantığını anlamış olmanız lazım. Aslında bir önceki bölümde **if** ve **goto** komutlarını kullanarak döngüler oluşturmuştuk fakat bu döngüler döngü komutları kullanılarak yapılmamıştır. Bu başlık altında programları uygun döngü veya döngüler kullanarak geliştireceğiz.

Bazı programlar hem **for**, hem **while** hem de **do-while** komutları kullanılarak yapılabilir; aynı programı farklı döngü komutları veya kontrol yapıları kullanarak nasıl yazabileceğimizi örneklerde anlatmaya çalışacağım. İlgili programın akış diyagramını yaptıktan sonra eğer mümkünse **if**, **for** ve **while** komutlarını kullanarak kodları yazacağız.

Beş Defa Merhaba Yaz

Bu örneğimizde **if**, **for** ve **while** komutlarını kullanarak konsola 5 defa merhaba yazdıracağız. Yani, aynı işi farklı komutlarla yapacağız. Sorgulanan şarta göre sayacı değişkeni 0 veya 1'den başlar. Önce programın akış diyagramını daha sonra da kodları inceleyin.



```

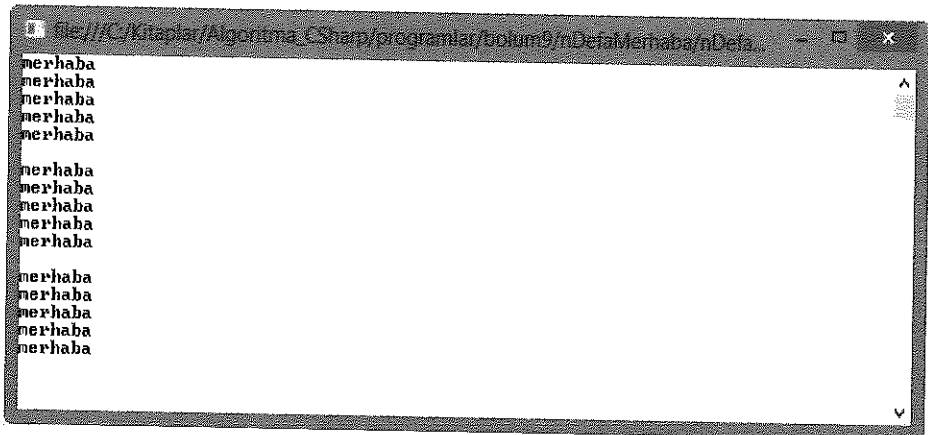
static void Main(string[] args)
{
    //programı if ile yapalım
    int s = 1; //sayacı
yaz:
    Console.WriteLine("merhaba");
    s = s + 1;
    if (s <= 5) goto yaz;

    Console.WriteLine();//boş satır
    //-----
    //programı for ile yapalım
    for (s = 1; s <= 5; s++)
        Console.WriteLine("merhaba");

    Console.WriteLine();//boş satır
    //-----
    //programı while ile yapalım
    s = 1;
    while (s <= 5)
    {
        Console.WriteLine("merhaba");
        s++;
    }

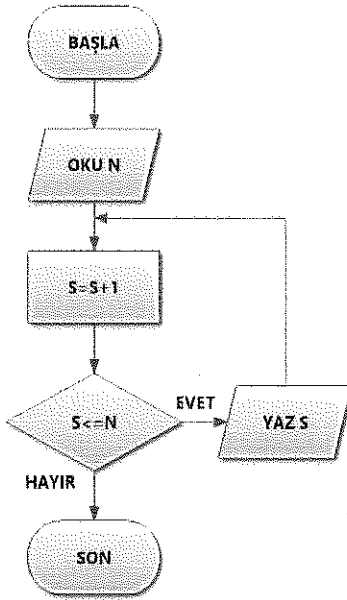
    Console.ReadLine(); //bekle
}
  
```

Ekran Çıktısı:



1'den N'e Kadar Olan Sayıların Yazdırılması

Bu örneğimizde önce konsoldan n değerini alacağız. Daha sonra da oluşturacağımız döngülerle 1'den n 'e kadar olan sayıları konsola yazdıracağız. Kullanılan algoritma ve şarta göre sayacı 0 veya 1'den başlar. Karşılaştırma ve uygulama amacıyla kodları hem **for()** hem de **while()** ile yazacağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s = 0, n;
    //n değerini alalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

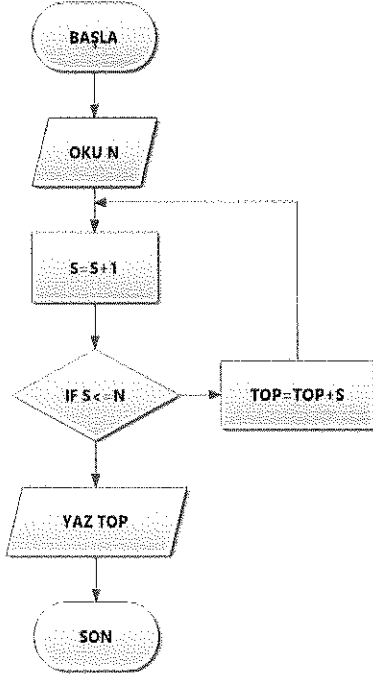
    //-----
    //programı for ile yapalım
    for (s = 1; s <= n; s++)
        Console.WriteLine(s);

    //-----
    //programı while ile yapalım
    s = 1;
    while(s<=n)
    {
        Console.WriteLine(s);
        s++;
    }

    Console.ReadLine();//bekle
}
  
```

1'den N'e Kadar Sayıların Toplamı

Önceki örneğimizde N'e kadar olan sayıları ekrana yazmıştık. Burada ise N'e kadar olan sayıları toplayacağız. Karşılaştırma ve uygulama amacıyla kodları hem **for()** hem de **while()** ile yazacağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s, n, top=0;
    //n değerini alalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

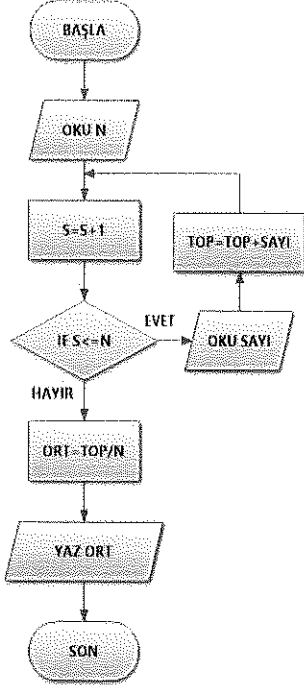
    //-----
    //programı for ile yapalım
    for (s = 1; s <= n; s++)
        top = top + s;
    Console.WriteLine("for ile toplam="+top);

    //-----
    //programı while ile yapalım
    top = 0;
    s = 1;
    while (s <= n)
    {
        top = top + s;
        s++;
    }
    Console.WriteLine("while ile toplam="+top);

    Console.ReadLine();//bekle
}
  
```

N Adet Sayının Ortalaması

Bu örneğimizde önce konsoldan n değerini alacağız. Daha sonra konsoldan n adet sayıyı okuyacağız ve okuduğumuz sayıları toplayıp n değerine böleceğiz. Karşılaştırma ve uygulama amacıyla kodları hem **for()** hem de **while()** ile yazacağız.

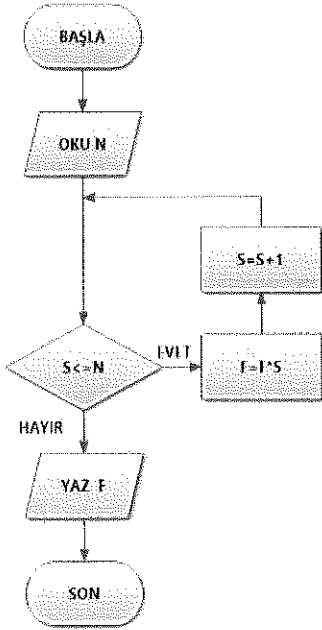


```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s, n, sayı, top = 0;
    //n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());
    //-----
    //programı for ile yapalım
    for (s = 1; s <= n; s++)
    {
        Console.Write(s + ". sayı:");
        sayı = Convert.ToInt32(Console.ReadLine());
        top = top + sayı;
    }
    Console.WriteLine("for_ortalama=" + top / n);
    //-----
    //programı while ile yapalım
    s=1;
    top = 0;
    while (s <= n)
    {
        Console.Write(s + ". sayı:");
        sayı = Convert.ToInt32(Console.ReadLine());
        top = top + sayı;
        s++;
    }
    Console.WriteLine("while_ortalama=" + top / n);
    Console.ReadLine();
}
  
```

Faktöriyel Hesabı

Eğer 1'den n'e kadar olan sayıları birbiri ile çarparsanız $n!$ değerini bulursunuz. Aynı bir önceki alıştırmada olduğu gibi n değerini alacağız. n değerini aldıktan sonra $1*2*3*...*n$ şeklinde çarpma işlemi yapacağız. Karşılaştırma ve uygulama amacıyla kodları hem **for()** hem de **while()** ile yazacağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, f = 1;

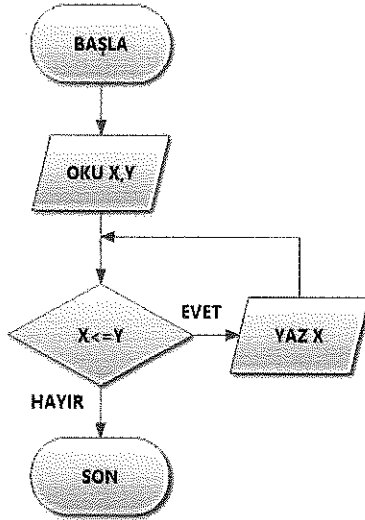
    //n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());
    //-----
    //kodları for ile yazalım
    for (int s = 1; s <= n; s++)
        f = f * s;
    Console.WriteLine("for ile " + n + "!=" + f);
    //-----
    //kodları while ile yazalım
    int say = 1;
    f = 1;
    while (say <= n)
    {
        f = f * say;
        say++;
    }
    Console.WriteLine("while ile " + n + "!=" + f);

    Console.ReadLine();
}
  
```

Girilen İki Değer Arasındaki Sayıları Ekrana Yaz

Bu örneğimizde **x** ve **y** gibi iki değer alıp bu iki değer arasındaki sayıları konsola yazdıracağız. **x** değerinin küçük **y** değerinin daha büyük olduğunu varsayıyoruz.

Burada özellikle **for** döngüsüne dikkat ediniz; sayaç başlangıç değeri boş bırakılmıştır çünkü sayaç **x**'ten başlamaktadır.



```

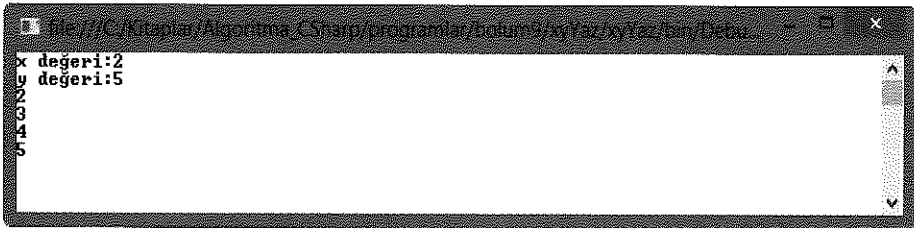
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y;

    //değerleri okuyalım
    Console.Write("x değeri:");
    x = Convert.ToInt32(Console.ReadLine());
    Console.Write("y değeri:");
    y = Convert.ToInt32(Console.ReadLine());

    //kodları for ile yazalım
    for (; x <= y; x++)
        Console.WriteLine(x);

    Console.ReadLine();//bekle
}
  
```

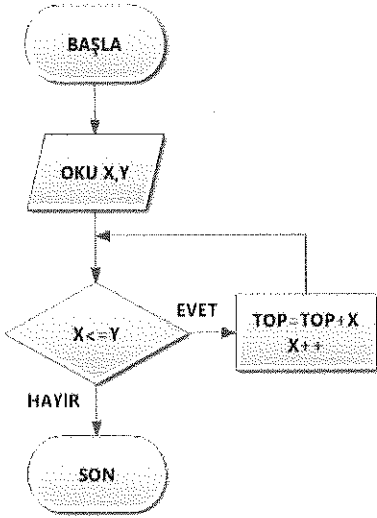
Ekran Çıktısı:



Girilen İki Değer Arasındaki Sayıların Toplamı

Bu örneğimizde **x** ve **y** gibi iki değer alıp bu iki değer arasındaki sayıların toplamını konsola yazdıracağız. **x** değerinin küçük **y** değerinin daha büyük olduğunu varsayıyoruz.

Burada özellikle **for** döngüsüne dikkat ediniz; sayaç başlangıç değeri boş bırakılmıştır çünkü sayaç **x**'ten başlamaktadır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y, top = 0;

    //değerleri okuyalım
    Console.Write("x değeri:");
    x = Convert.ToInt32(Console.ReadLine());
    Console.Write("y değeri:");
    y = Convert.ToInt32(Console.ReadLine());

    //işlemi yapalım
    for (; x <= y; x++)
        top = top + x;

    //sonucu yazalım
    Console.WriteLine("toplam=" + top);

    Console.ReadLine(); //bekle
}
  
```

Tek Bilinmeyenli Denklem

Daha önceki konularda tek bilinmeyenli $ax+b=0$ şeklinde bir denklemin çözümünü yapmıştık. Burada bilinmeyen **x** değeridir. **a** ile **b** ise denklemin katsayılarıdır ve klavyeden girilmesi gerekir. **x**'i bulmak için aşağıdaki işlemleri yapmamız gerekir.

- $ax+b=0$
- $ax=-b$
- $x=-b/a$

Yukarıdaki formülden de anlaşılacağı üzere **x** değeri iki parametreye, **a** ve **b**'nin değerlerine bağlıdır. Aşağıdaki örneğimizde **b=1** iken **a** değerini 1'den 10'a kadar değiştireceğiz ve **x** değerinin nasıl değiştiğini göreceğiz.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double x, a, b=1;

    //a değişkenini 1-10 arası değiştirerek
    //x değerini hesaplayalım
    for (a = 1; a <= 10; a++)
    {
        x = -b / a;
        Console.WriteLine("a=" + a + "\t x=" + x);
    }

    //pencere kapanmasın
    Console.ReadLine();
}
```

Ekran çıktısı:

```
file:///C:/Kutuplar/Algoritma CSharp/programlar/bolum9/tekBilinmeyen/tekBilin...
a=1      x=-1
a=2      x=-0,5
a=3      x=-0,3333333333333333
a=4      x=-0,25
a=5      x=-0,2
a=6      x=-0,16666666666666667
a=7      x=-0,142857142857143
a=8      x=-0,125
a=9      x=-0,11111111111111111
a=10     x=-0,1
```

Tek Bilinmeyenli Denklem Sürüm 2

Bir önceki denklem örneğimizde sadece **a** parametresini 1-10 arası değiştirmiştik. Bu örneğimizde hem **a** hem de **b** parametrelerini 1-5 arası değiştireceğiz.

Bu örnekte bir önceki örnekten farklı olarak iç içe iki **for** döngüsü kullanılmaktadır. Kodlar ve çıktı dikkatle incelenecek olursa **b** parametresi 1'den 5'e kadar değişirken **b**'nin her değeri için **a** parametresi 1 ile 5 arası değerler almaktadır. Eğer çıktı incelenirse bu durum çok daha iyi görülebilir.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double x, a, b;

    //b değişkenini 1-5 arası değiştirelim
    for (b = 1; b <= 5; b++)
    {
        //her b değeri için
        //a değerini 1-5 arası değiştirelim
        for (a = 1; a <= 5; a++)
        {
            x = -b / a;
            Console.WriteLine("b=" + b + "\t a=" + a + "\t x=" + x);
        }
    }

    Console.ReadLine();
}
```

Program Çıktısı:

```

file:///C:/Kitaplar/Algoritma/Sharp/programlar/bolum9/tekBilinmeyenSurum2/
b=1      a=1      x=-1
b=1      a=2      x=-0,5
b=1      a=3      x=-0,3333333333333333
b=1      a=4      x=-0,25
b=1      a=5      x=-0,2
b=2      a=1      x=-2
b=2      a=2      x=-1
b=2      a=3      x=-0,6666666666666667
b=2      a=4      x=-0,5
b=2      a=5      x=-0,4
b=3      a=1      x=-3
b=3      a=2      x=-1,5
b=3      a=3      x=-1
b=3      a=4      x=-0,75
b=3      a=5      x=-0,6
b=4      a=1      x=-4
b=4      a=2      x=-2
b=4      a=3      x=-1,3333333333333333
b=4      a=4      x=-1
b=4      a=5      x=-0,8
b=5      a=1      x=-5
b=5      a=2      x=-2,5
b=5      a=3      x=-1,6666666666666667
b=5      a=4      x=-1,25
b=5      a=5      x=-1

```

Alıştırmalar

- 0-10 arası çift sayıları konsolda alt alta yazdırınız.
- 0-n arası çift sayıları konsolda yan yana 1 sekme aralık (t) vererek yazdırınız.
- 0-n arası çift sayıların toplamını bulunuz.
- 5 hariç 1-10 arası sayıların toplamını bulunuz.
- Konsoldan okunan 5 adet sayının ortalamasını bulunuz.
- $x=2a-b$ formülünde **a** ve **b** parametrelerinin değerlerini 1-10 arası değiştirerek **x** değişkeninin alacağı değerleri konsola **x** (sekme) **a** (sekme) **b** şeklinde yazdırınız.
- Bir dairenin alanını ve çevresini yarıçapı 1 ile 10 arası değerler için hesaplayınız ve konsola **yarıçap** (sekme) **alan** (sekme) **çevre** şeklinde yazdırınız.
- 0-100 santigrat derecenin fahrenheit karşılıklarını konsola **santigrat** (sekme) **fahrenheit** şeklinde alt alta yazdırınız.
- Kullanıcıdan başlangıç değeri, bitiş değeri ve artış değerini alarak başlangıç-bitiş arası sayıları konsola yazdırınız.
- Çarpım tablosunu konsola yazdırınız.

9

Metotlar

- Metot Nedir?
- Metot Nasıl Oluşturulur
- Main() Metodu
- Metot Nasıl Çağırılır?
- Lambda Fonksiyonları
- Metot Uygulamaları

Metot Nedir?

Ana program tarafından gerektiğinde çağırılıp çalıştırılan program parçacıklarına metot denir. Metotlar bazı dillerde prosedür, altprogram veya fonksiyon adıyla da anılır.

Metotlar C tabanlı dillerde en önemli konulardan biridir çünkü çok sık kullanılır. O nedenle çok iyi bilinmesi gereken konulardan biridir. Hatta diyebiliriz ki metot bilmeden programcı olunmaz. Metotları öğrenmesi biraz zor olabilir ama öğrendikten sonra çok faydasını göreceksiniz.

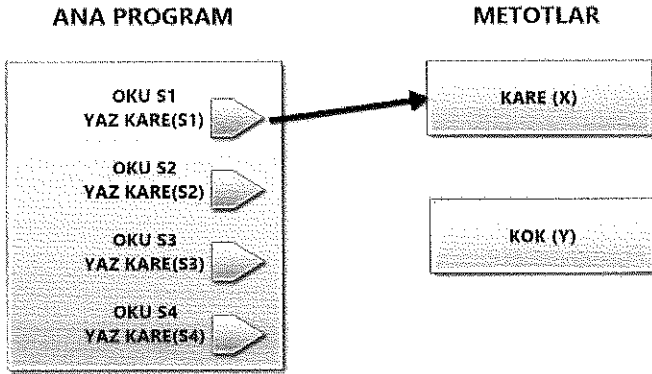
Peki! Metotlara neden ihtiyaç duyarız? Eğer program içinde bazı işlemleri sık sık yapıyorsanız her işlem için kod yazmak yerine bu kodu bir defa yazar ve metot olarak kaydedersiniz. Daha sonra da gerektiğinde bu kodları/metotları kullanırsınız.

Örneğin sık sık konsoldan bir sayı okuduktan sonra bu sayının karesini hesaplamamız ve konsola bu sayının karesini yazmamız gerekiyorsa bu iş için bir metot yaz-

makta fayda var. Böylece programımız kılalacak ve okunabilirliğı artmış olacaktır. Ayrıca zamandan kazanırken aynı kodları tekrar tekrar yazmaktan da kurtulmuş oluruz; hamallığı azaltırız.

Metotlar büyük bir programı küçük parçalara bölmemizi ve daha basit küçük parçalar halinde kod yazmamızı sağlar. Böylece algoritma kolaylaşır ve kod yazmak basitleşir. Böl ve yönet mantığı...

Aşağıdaki şekli inceleyelim. Burada bir ana program ve bazı metotlar vardır. Ana program (defalarca) bir sayıyı almakta ve karesini konsola yazmaktadır. Her defasında alınan sayının karesini tekrar tekrar hesaplamak yerine x değerinin karesini hesaplayan bir metot yazılmış ve gerektiğinde bu metot çağırılmış yani kullanılmıştır. Sonuçta, program kolaylaşmış, daha anlaşılır hale gelmiş, hız ve zamandan tasarruf edilmiştir.



Şekil 9.1 Metot çağırma

Bir metot sadece ana program tarafından kullanılmaz. Metotlar ihtiyaç halinde başka metotları çağırıp kullanabilir. Örneğin eğer gerekliyse Şekil9.1'de görülen KOK(Y) metodu da KARE(X) metodunu çalıştırabilir. Ya da eğer gerekliyse KARE(X) metodu da KOK(Y) metodunu çalıştırabilir.

Metotlar programcı tarafından oluşturulabilir veya **.NET Framework** içinde hazır olarak gelebilir. **.NET Framework** içinde kullanılmaya hazır binlerce metot vardır ve bunları kullanmak bize hem zaman hem de hız kazandırır. Örneğin **Console.WriteLine()** komutu aslında **Console** nesnesinin **Write()** metodunun çalıştırılmasıdır. **Write()** metodu parantez içine yazılan metin ve değişkenlerin konsola yazılmasını sağlar. Aynı şekilde **Console.ReadLine()** komutunda **Console** nesnesinin **ReadLine()** metodunu kullandık ve konsol satırına yazılan verileri okuyarak bir değişkene aktardık. Yani aslında farkında olmadan şu ana kadar bir sürü hazır metot kullandık.

Gene farkında olmadan **Main()** metodunu yani programımızın çatısını oluşturan ana programı ya da ana metodu kullandık. **Main()** metodu program çalıştığında kodların ilk çalışmaya başladığı yerdir; program **Main()** metoduna girer ve kodları sırayla çalıştırmaya başlar.

Aşağıdaki kodlar incelendiğinde aynı **.NET Framework Class Library** yapısında olduğu gibi önce bir **namespace** görülür. Onun içinde/altında bir **class** ve onun içinde de **Main()** metodu vardır. Eğer gerekliyse **class** içinde farklı metotlar yazılıp ana program veya diğer metotlar tarafından çalıştırılabilir.

```
namespace mainMetot
{
    class Program
    {
        static void Main(string[] args)
        {
            //Metot() isimli metodu çağırılım
        }
    }
}
```

Metot Nasıl Oluşturulur?

Eğer belli bazı kodları bir metot olarak kaydetmek ve tekrar tekrar kullanmak istiyorsak metodumuzu aşağıdaki kalıba uygun bir şekilde hazırlamalıyız:

```
erişim_dönüş_tipi metot_adı(parametreler)
{
    kodlar;
    return değişken;
}
```

Metot oluştururken aşağıdaki konulara dikkat edilmelidir:

- Metotlar belli bir işi yapmak için oluşturulmalı ve mümkün olduğunca basit tutulmalıdır. Örneğin hem bir sayının karesini alan hem de bir sayıyı başka bir sayıyla toplayan (çift işlevli) bir metot yapılmamalıdır.
- Metot adı genel olarak değişken adlarıyla aynı kurallara sahiptir; değişken tanımlarken kullandığımız kurallar metot tanımlarken de geçerlidir.
- Metodun yapacağı işler yani kodlar güzel parantez **{}** içine yazılmalıdır.
- Metot tipi ana programa göndereceği değere göre **int**, **char**, **double**, **string**, **boolean** veya **void** gibi bir veri tipine sahip olabilir.

- Eğer metot ana programa bir değişken döndürecekse **return** (döndür) komutu kullanılır.

Yukarıda muhtemelen ilk defa duyduğunuz ve yabancı olabileceğiniz bazı terimlerden bahsettim. Aşağıdaki konularda bu terimleri ayrıntılı ve açık biçimde anlatmaya çalışacağım.

Erişim Belirteci

Erişim belirteci bir metoda bir kodun nereden erişebileceğini belirler. Eğer bir metoda sadece aynı **class** içindeki kodlar erişecekse yani hem erişmek isteyen kodlar hem de metot aynı **class** içinde ise erişim belirteci **private** yapılır. Eğer bir metoda herhangi bir erişim belirteci yazılmazsa varsayılan olarak **private** olur.

Aşağıdaki kodlarda hem ana program hem de çağırılmak ya da çalıştırılmak istenen metot aynı **class** içinde bulunmaktadır. **Metot()** isimli metodun erişim belirteci **private** olduğu için bu metoda sadece aynı **class** içinde bulunan metotlar erişebilir.

```
class Program
{
    static void Main(string[] args)
    {
        //ana programın kodları
        Metot(); //Metot adlı metodu çalıştır
    }
    static private void Metot()
    {
        //metodun kodları
    }
}
```

Eğer çok büyük bir program yazıyorsanız ve farklı **class**'lar içinde bulunan kodların diğer **class**'lar içinde bulunan metotlara erişmesini istiyorsanız bu metotların erişim belirtecini **public** yapmak zorundasınız. Bu durumda yukarıdaki **Metot()** isimli metot aşağıdaki gibi olmalıdır:

```
static public void Metot()
{
    //metodun kodları
}
```

Bir metoda erişmek için önce ilgili metodun içinde bulunduğu **class** adı daha sonra da **metot adı** kullanılır. Yani başka class içinde bulunan bir metoda **class_adi.me-**

tot_adi() şeklinde erişim sağlanır. Aşağıdaki kodlarda da görüldüğü gibi, **Main()** metodu içinden çağırılmak istenen **Metot()** isimli metot için önce **class** adı daha sonra da **metot adı** yazılmıştır.

```
class Program
{
    static void Main(string[] args)
    {
        //Metot() isimli metodu çağırılım
        Fonksiyon.Metot();
    }
}
```

```
class Fonksiyon
{
    static public void Metot()
    {
        //metodun kodları
    }
}
```

Eğer tanımlanan metot aynı sınıf içinde bulunan ana program tarafından çağırılacaksa **static** olarak tanımlanabilir ve ana program tarafından **class** adı yazmaya gerek kalmadan, aşağıdaki kodlarda görüldüğü gibi doğrudan metot adıyla çağırılabilir.

```
class Program
{
    static void Main(string[] args)
    {
        //Metot() isimli metodu çağırılım
        Metot();
    }

    static private void Metot()
    {
        //metodun kodları
    }
}
```

Sınıf ve metot konusu daha çok nesne tabanlı programlama ile ilgilidir ve bu bölümün kapsamı dışında olduğu için çok fazla ayrıntıya girilmeyecektir. Metot erişim parametreleri ile ilgili daha ayrıntılı bilgi edinmek için nesne tabanlı programlama konula-

rina bakınız. Biz metotları hep ana programdan ve aynı **class** içinden çağıracağız. Bu nedenle metotlarımızı **static metot_adi()** şeklinde oluşturacağız. Herhangi bir belirteç olmadığı için tüm metotlarımız varsayılan olarak **private** olacaktır.

Dönüş Tipi

Bir metot çalıştıktan sonra ana programa veya çağırıldığı metoda bir değer döndürebilir. Dönüş tipi metodun hangi tipte bir değer döndürdüğünü belirtir. Dönüş tipi ana programa göndereceği değere göre **int**, **char**, **double**, **string**, veya **boolean** gibi bir veri tipi olabilir. Eğer metot geriye bir değer döndürmüyorsa dönüş tipi **void** olmalıdır.

Değer Döndürmeyen Metotlar

Bu tür metotlar ana program tarafından çalıştırılır fakat ana programa bir değer göndermezler. O nedenle bu tür metotlarda veri tipi olarak boş anlamına gelen "**void**" kelimesi kullanılır.

Örnek Kod:

Aşağıda çalışan kodlarda konsola "merhaba" yazdırılır ve ana programa bir değer döndürülmez. Bu nedenle metodun veri tipi **void** olarak yazılmıştır.

```
static void merhaba()
{
    Console.Write("merhaba");
    Console.Read(); //bekle
}
```

Örnek Kod:

Aşağıdaki metot konsoldan iki tamsayı okur ve toplamını konsola yazdırır. Ana programa herhangi bir değer döndürmediği için gene "**void**" veri tipi kullanılır.

```
static void topla()
{
    int s1,s2,topla=0;

    Console.Write("birinci sayı:");
    s1=Convert.ToInt32(Console.ReadLine());

    Console.Write("ikinci sayı:");
    s2=Convert.ToInt32(Console.ReadLine());

    topla=s1+s2;
```

```

Console.Write ("Toplam="+topla)

Console.Read(); //bekle
}

```

Tek Parametrelili Metotlar

Parametreler metoda ana program tarafından gönderilen uygun veri tipinde değerlerdir. Metot ana programdan gelen değeri alarak bir takım işlemler yapar.

Örnek Kod:

Aşağıdaki kodlarda **kare(int a)** metoduna ana programdan **int** veri tipinde bir parametre gönderilmelidir. Metot ana programdan gelen bu değeri alır ve ilgili işlemi yapar. İşlem bittikten sonra **return** komutu kullanılarak ana programa **k** değeri ya da değişkeni gönderilmektedir. Bu nedenle metoda ana programa gönderdiği değere uygun bir veri tipi yani **int** verilmiştir.

```

static int kare(int a)
{
    int k=a*a;
    return k;
}

```

Örnek Kod:

Aşağıdaki örnekte **kupal** metodunun veri tipi **int** olarak belirtilmiştir. Yani metot ana programa **int** tipinde bir değer ya da değişken gönderecektir. Parantez içinde **int x** ifadesi bulunmaktadır. Yani ana programdan metoda **int** tipinde bir parametre ya da değer gönderilecektir. Metot ana programdan gelen **x** değerini kullanarak küp hesabı yapmaktadır. Gelen değer küpünü bulur ve **kup** değişkeninde saklar. Daha sonra da **return** komutu ile **kup** değişkeni/değeri ana programa gönderilir.

```

static int kupal(int x)
{
    kup=x*x*x;
    return kup;
}

```

Çok Parametrelili Metotlar

Ana programdan metoda bir veya daha fazla değer yani parametre gönderilebilir. Eğer tek bir parametre gönderilecekse parantez içine yazılır. Birden fazla değer gönderilecekse parametreler gene parantez içine fakat araya virgül konularak yazılır.

Örnek Kod:

Aşağıdaki metotta **double** tipinde **a** ve **int** tipinde **b** olmak üzere iki parametre bulunmaktadır. Metot ana programdan gönderilen bu iki değeri toplayarak **return** komutuyla ana programa göndermektedir. Ana programa gönderilen değer **double** tipinde olduğu için fonksiyonun da dönüş tipi **double**'dir.

```
static double topla(double a, int b)
{
    double topla=a+b;
    return topla;
}
```

Örnek Kod:

Bu örnekte **bol()** isimli ana programa değer döndürmeyen **void** tipinde bir metot görüyorsunuz. Metodun **double** tipinde iki parametresi vardır. Ana programdan gelen ilk parametre (**x**) ikinci parametreye (**y**) bölünmekte ve işlem sonucu konsola yazılmaktadır. Ana programa herhangi bir değer gönderilmemektedir. Bu nedenle fonksiyon dönüş tipi **void** yapılmıştır.

```
static void bol(double x, double y)
{
    double sonuc;
    sonuc=x/y;
    Console.Write("Sonuç="+sonuc);
}
```

Main Metodu

Ne zaman yeni bir proje oluştursak **Main()** metodu otomatik olarak karşımıza gelir. **Main()** metodu bizim oluşturduğumuz metotlardan çok da farklı olmayan bir metottur. Tek farkı yazdığımız ana programın başlangıç noktası olmasıdır. Yani isterseniz milyonlarca satır kod yazın programınız **Main()** metodunun ilk satırından başlar.

Main() metodu her C# programında olmak zorundadır ve adı değiştirilemez. Genelde yeni açılan projelerde **Main()** metodu **void** tipine sahiptir çünkü genelde ana program geriye (işletim sistemine) değer döndürmez.

Eğer **Main()** metodunun veri tipini **int** yaparsanız program kapandığında işletim sistemine **int** tipinde bir değer döndürür. Eğer bu değer sıfır ise program normal kapanmıştır sıfırdan farklı bir değer ise program hatalı kapanmış demektir.

Main() metodunda parametre olarak (**string[] args**) ifadesi vardır. Bu da eğer istenirse **Main()** metoduna **string** tipinde dizi olarak veri gönderilebileceğini gösterir.

Metot Nasıl Çağırılır?

Eğer bir metodunuz varsa ve bu metoda iş yaptırmak istiyorsanız o metodu çağırmanız yani çalıştırmanız gerekmektedir. Metot çağırmak için aşağıdaki yöntemler kullanılabilir:

1. Eğer parametresiz bir metot çağırıcaksanız doğrudan çağırabilirsiniz. Örneğin kullanacağınız metodun adı **topla** ise doğrudan **topla();** şeklinde bir kod yazabilirsiniz.
2. Eğer parametrelili bir metot kullanıcaksanız ana programdan metoda parametre göndermeniz gerekir. Örneğin **topla(int a, int b)** şeklinde bir metot kullanıcaksanız ilgili değerleri veya değişkenleri metoda göndermeniz gerekir. Örneğin ana programda **x** ve **y** gibi **int** tipinde iki değişkeni toplamak istediğinizi düşünün. Bu durumda metodu **topla(x, y);** şeklinde çağırmanız gerekir.
3. Metotları değişkenlere atayarak da kullanabilirsiniz. Örneğin **x** ve **y** değişkenlerinin toplamını **top** değişkeninde saklamak istiyorsanız **top=topla(x, y);** şeklinde bir kod yazabilirsiniz.
4. Metotları başka metotlar içinde de kullanabilirsiniz. Örneğin **x** ve **y** değerlerinin toplamını konsola yazdırmak istiyorsanız **Console.WriteLine("toplam="+topla(x, y));** şeklinde bir kod yazabilirsiniz. Burada **Console** nesnesinin **Write** metodu kullanılarak konsola **x** ve **y** değişkenlerinin toplamı metot kullanılarak yazdırılır.

Lambda (İfadeleri) Fonksiyonları

Lambda ifadeleri ya da lambda fonksiyonları, değişkenlere değer atamak için kullanılan sadeleştirilmiş anonim (isimsiz) fonksiyonlardır. Bu fonksiyonlar matematikteki ve bilgisayar bilimlerindeki Lambda Calculus'un C# ve Visual Basic uygulamasıdır.

Lambda ifadeleri kullanarak parametre geçilebilen ve değer döndüren isimsiz yerel fonksiyonlar oluşturabilirsiniz. Bu ifadeler genelde basit işlemleri bildirmekte kullanılır. Ayrıca, LINQ sorgularının yazımında da sıklıkla kullanılırlar.

Örnek Kod:

```
n => n+1
```

Yukarıdaki örnekte de görüldüğü gibi, bir lambda ifadesi oluşturmak için, eğer varsa sol tarafa parametre bildirimi, lambda operatöründen ($\Rightarrow$) sonra sağ tarafa da ifade veya kod bloğu yazılmalıdır. Üstteki ifadenin anlamı şudur; Bir parametre al ve bu parametreyi n kabul et (soldaki n), n 'yi 1 artırıp sonucu döndür (sağdaki $n + 1$).

Yukarıdaki örnekte anlatılan lambda fonksiyonu aşağıdaki metotla aynı işi yapmaktadır. Tahmin edebileceğiniz gibi, metodun adı art olarak belirlenmiştir. Lambda fonksiyonlarında metotlara ad verilmemektedir.

```
int art(int n)
{
    return n + 1;
}
```

Lambda fonksiyonları tek satır halinde yazılır. Tek satır halindeki lambda ifadelerine **Expression Lambdas** (İfade Lambda'ları) denir. İfadenin sağ tarafında süslü parantezler ve sonunda da noktalı virgül kullanılarak çok satırlı bir lambda ifadesi de yazılabilir. Böyle yazılan lambda ifadelerine **Statement Lambdas** (Komut Lambda'ları) denir.

```
n =>
{
    string s = "Merhaba " + n;
    Console.WriteLine(s);
};
```

Lambda ifadelerinde parametre zorunlu değildir. Aşağıdaki de geçerli bir ifadedir.

```
() => birFonksiyon()
```

Parametreler bir parametreden fazla olabilir. Bu durumda parametreleri parantez içinde bildiririz.

```
(a, b) => a > b
```

Yukarıdaki fonksiyonun anlamı şudur: İki parametre al, birincisi a , ikincisi b olsun. a 'nın değerinin b 'den büyük olup olmadığını döndür.

Yukarıda anlatılan lambda ifadelerini doğrudan program koduna eklerseniz **"Only assignment, call, increment, decrement, await, and new object expressions can be used as a statement."** (Sadece atama, yordam çağırısı, artırma ($++$), eksiltme ($--$), bekleme (await) ve yeni nesne ifadeleri (new) komut olarak kullanılabilir.)

şeklinde bir uyarı görürsünüz. Yani bu ifadeleri tek başına kullanamazsınız.

Örnek Kod:

Lambda ifadelerinde mutlaka Lambda Operatörü ($\Rightarrow$) bulunmalıdır. Bu operatörün sol tarafındaki değişkenler fonksiyonun giriş değerlerini belirtir. Operatörün sağında ise fonksiyonun döndüreceği değeri üreten ifade ya da ifade bloğu yer alır.

```
//x değerini al, 2*x + 1 işleminin sonucunu geriye döndür.
x => 2*x + 1
```

Örnek Kod:

Aşağıdaki kodlarda sayılar dizisindeki çift sayıların toplamını bulan bir **LINQ** ifadesi görülmektedir. Burada kullanılan **Count()** metodu dizideki eleman sayısını döndürmeye yarayan bir LINQ metodudur. **Count()** metodunun içine $x \Rightarrow x \% 2 == 0$ şeklinde bir lambda ifadesi yazılarak sadece çift sayıların dönmesi sağlanmıştır. Yani, x değeri öyle olmalı ki, 2'ye bölündüğünde sıfır değeri kalmalıdır.

```
using System;
using System.Linq;

namespace lambdaIfadesi
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] sayilar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
            int ciftsayi = sayilar.Count(x => x % 2 == 0);
            Console.WriteLine("Çift Sayı Adedi:" + ciftsayi);
        }
    }
}
```

Dikkat edilirse lambda ifadeleri doğrudan herhangi bir kod satırına yazılmamakta, bir komutun satırının içinde yardımcı olarak kullanılmaktadır.

Metot Uygulamaları

Bu başlık altında metotlarla ilgili uygulamalar yapacağız. En basit metotlardan başlayarak daha karmaşık olanlara doğru ilerleyeceğiz.

Lütfen örnekleri sırayla ve atlamadan inceleyiniz. Her örnekte bir öncekinden farklı yapı ve uygulamalar bulunmaktadır.

Merhaba Metodu

Bu örneğimizde konsola "Merhaba" yazan basit bir metod oluşturacağız ve metodu ayrıntılı olarak inceleyeceğiz. Bir metodun nasıl oluşturulduğunu ve ana program tarafından nasıl çağırıldığını uygulamalı olarak göreceğiz.

```
class Program
{
    static void Main(string[] args)
    {
        //ana program buradan başlıyor

        //yaz isimli metodu çağır
        yaz();

        Console.Read(); //bekle
    }

    static void yaz() //yaz isimli parametresiz metod
    {
        Console.WriteLine("Merhaba");
    }
}
```

Dikkat edilirse hem **Main()** metodu hem de **yaz()** metodu **Program** isimli **class** içinde bulunuyor. Bu durumda **yaz()** metodunu **static** olarak tanımlarsak class adını yazmadan ana programdan çağırabiliriz.

Main() metodunda açıklama satırından hemen sonra **yaz();** komutu ile **yaz()** metodu çağırılır. Hemen altındaki satırda konsol penceresinin kapanmaması için **Console.Read();** komutu kullanıldı.

Main() metodunun hemen altında **yaz()** metodu bulunmaktadır. Metotlar **Main()** metodunun üst veya alt kısmına yazılabilir.

yaz() metodunun herhangi bir parametresi yok. Yani ana programdan metoda herhangi bir değer gönderilmiyor.

yaz() metodu ana programa herhangi bir değer döndürmediği için **void** olarak oluşturuldu. **yaz()** metodunun yaptığı tek şey ana program tarafından çağırıldığında "merhaba" yazmak.

N Defa Merhaba Yaz

Bir önceki örnekte bir defa **merhaba** yazdırdık. Eğer **N** defa **merhaba** yazdırmak istersek **for** komutunu kullanabiliriz. Konsoldan okunan **n** değeri ile **for** döngüsü **n** defa çalıştırılır. **for** döngüsü ile de **yaz()**; metodu çalıştırılır.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n;

    //konsoldan n değerini okuyalım
    Console.Write("n:");
    n = Convert.ToInt32(Console.ReadLine());

    //yaz isimli metodu n defa çalıştıralım
    for(int i=1; i<=n; i++)
        yaz();

    Console.Read(); //bekle
}

//yaz isimli parametresiz metot
static void yaz()
{
    Console.WriteLine("Merhaba");
}
```

Parametrelili Merhaba Metodu

Bu program bir önceki örneğimize benziyor. Farklı olarak kaç defa merhaba yazdırılacağı metoda parametre olarak gönderiliyor. Aynı iş yapılıyor fakat farklı yöntemler; Döngü artık metot içinde.

```
static void Main(string[] args)
{
    //şimdi program buradan başlıyor
    int n;

    //n değerini alalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

    //yaz metodunu okunan n değeri ile çalıştıralım
    yaz(n);
    Console.Read(); //bekle
}

static void yaz(int x) //yaz isimli parametrelili metot
{
    for(int i=1; i<=x; i++)
        Console.WriteLine("Merhaba");
}
```

Kare Metodu

Bu örneğimizde konsoldan okunan bir sayının karesini metot kullanarak hesaplayacağız.

Konsoldan okunan **s** değişkeninin karesi **kare(s)** metoduyla hesaplanıyor ve **karesi** değişkenine atanıyor; **karesi=kare(s)**; komutunun çalışmasıyla **s** değişkeni **kare(int x)** metoduna gönderiliyor, orada kendisiyle çarpılıyor ve işlem sonucu yani **s**'nin karesi dönüyor ve **karesi** değişkenine atanıyor.

kare(int x) metodu incelenirse ana programdan gelen **int** tipindeki parametre ya da değişken alınıyor ve kendisiyle çarpılıp **k** değişkenine atanıyor. Elde edilen **k** değeri **return** komutu ile ana programa gönderiliyor.

```
class Program
{
    static void Main(string[] args)
    {
        //değişkenleri tanımlayalım
        int s, karesi;

        //konsoldan sayıyı okuyalım
        Console.Write("sayı:");
        s = Convert.ToInt32(Console.ReadLine());

        //sayının karesini metotla hesapla ve karesi değişkenine ata
        karesi = kare(s);

        //sayıyı ve karesini yaz
        Console.WriteLine("sayı=" + s + "\t" + "karesi=" + karesi);

        //pencereyi kapatma
        Console.ReadLine();
    }

    static int kare(int x) //kare isimli int tipinde metot
    {
        //ana programdan gönderilen x parametresi ile kare hesabı
        int k = x * x;

        //işlem sonucunu (x'in karesini) ana programa gönder
        return k;
    }
}
```

Kare Metodu Sürüm 2

Bir önceki örneğimizden farklı olarak bu örnekte **kare()** metodunu doğrudan **Console.Write()** metodunun içinde kullanıyoruz ve gereksiz bazı kodlardan kurtuluyoruz. **Console.Write()** metodu ile önce **s** değişkenini sonra da **kare(s)** metoduyla **s** değişkeninin karesini yazdırıyoruz.

```
class Program
{
    static void Main(string[] args)
    {
        //değişkenleri tanımlayalım
        int s;

        //konsoldan sayıyı okuyalım
        Console.Write("sayı:");
        s = Convert.ToInt32(Console.ReadLine());

        //sayıyı ve karesini yaz
        Console.Write("sayı=" + s + "\t" + "karesi=" + kare(s));

        //pencereyi kapatma
        Console.ReadLine();
    }

    static int kare(int x) //kare isimli int tipinde metot
    {
        //ana programdan gönderilen x parametresi ile kare hesabı
        int k = x * x;

        //işlem sonucunu (x'in karesini) ana programa gönder
        return k;
    }
}
```

Konsolu Oku Metodu

Şu ana kadar konsolu okumak için hep **Console.Read()** metodunu kullandık. Değerleri okuduktan sonra **Convert** komutu ile uygun veri tipine dönüştürdük. Bu örnekte yazacağımız metod ile konsolu daha az kod yazarak daha kolay bir şekilde okuyacağız.

Oku() metodu ile konsola **sayı giriniz:** yazdırıyoruz. Daha sonra konsola yazılan sayıyı okuyup **int** veri tipine döndürüyoruz. Okunan değeri **s** değişkenine atıyoruz ve son satırda **s** değişkenini **return** komutu ile ana programa gönderiyoruz.

```
class Program
{
    static void Main(string[] args)
    {
        //oku() metodunu çalıştırarak
        //s1 ve s2 değişkenlerine değer ata
        int s1 = oku();
        int s2 = oku();

        //s1 ile s2 değişkenlerini topla
        int top = s1 + s2;

        //sonucu yaz
        Console.Write("toplam= " + top);
        Console.ReadLine();//pencereyi kapatma
    }

    static int oku()//geriye int tipinde değer döndüren oku metodu
    {
        Console.Write("sayı giriniz:");//konsola yaz

        //konsoldan değeri oku ve int tipine
        //dönüştürdükten sonra s değişkenine ata
        int s = Convert.ToInt32(Console.ReadLine());

        //konsoldan okunan değeri ana programa gönder
        return s;//
    }
}
```


Topla Metodu

Konsoldan okunan iki sayıyı ya da herhangi iki değişkeni bir metot kullanarak toplamını hesaplayacağız. Konsoldan okunan sayılar ondalıklı olabileceği için veri tipini **double** yapacağız.

Bu örneğimizde konsoldan sayı okuma yerine doğrudan **a** ve **b** şeklinde iki **double** değişken tanımlayıp ondalıklı değer atıyoruz. Siz isterseniz bu iki değişkenin değerini konsoldan da alabilirsiniz.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double a=1.1234, b=2.4321, t=0 ;

    //metot ile a ve b değişkenlerini toplayalım
    //ve sonucu t değişkenine atayalım
    t = topla(a, b);

    //sonucu yazalım
    Console.WriteLine("t değeri: " + t);

    //işlem kısaca şöyle de olabilirdi
    Console.Write("toplama sonucu=" + topla(a, b));
    Console.ReadLine();//bekle
}

//ana programa double tipinde veri gönderen metot
//metodun double tipinde iki parametresi var
static double topla(double x, double y)
{
    //ana programdan gelen x ve y parametrelerini topla
    double top = x + y;

    //toplama sonucunu ana programa gönder
    return top;
}
```

1'den N'e kadar Olan Çift Sayıların Yazdırılması

Daha önceki konularda yaptığımız bu programı metot haline getireceğiz ve o şekilde kullanacağız. Bu programda önce konsoldan bir değer okunur ve **n** değişkenine atanır. Daha sonra konsola 0'dan **n**'e kadar olan çift sayılar yazdırılır.

```
class Program
{
    static void Main(string[] args)
    {
        //n değerini okuyalım
        int n;
        Console.Write("n değeri:");
        n = Convert.ToInt32(Console.ReadLine());

        //1'den n'e kadar olan sayıları yazan metodu çağıralım
        yazCiftSayi(n);

        Console.ReadLine();//bekle
    }

    //ana programa değer döndürmeyen void metot tanımlanır
    static void yazCiftSayi(int n)
    {
        for (int i = 2; i <= n; i = i + 2)
            Console.WriteLine(i);
    }
}
```

X ile Y Arasındaki Sayıların Yazdırılması

Bu örneğimizde konsoldan iki sayı alınacak. X değerinin daha küçük olduğu varsayılacak. X ile Y arasındaki sayılar metotla konsola yazılacak.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y;

    //x ve y değerlerini okuyalım
    Console.Write("x değeri:");
    x = Convert.ToInt32(Console.ReadLine());
    Console.Write("y değeri:");
    y = Convert.ToInt32(Console.ReadLine());

    //metodu çağıralım ve sayıları yazdıralım
    xyyaz(x, y);
    Console.ReadLine();//bekle
}

//geri değer döndürmeyen iki parametrelili metot
static void xyyaz(int x, int y)
{
    for (; x <= y; x++)
        Console.WriteLine(x);
}
```

1'den N'e Kadar Sayıların Toplamı

Daha önceden yaptığımız bu programı metot haline getireceğiz ve o şekilde kullanacağız. Programda önce konsoldan bir değer okunur ve **n** değişkenine atanır. Daha sonra konsola **n** değeri ve **ntopla(n)** metodu yazdırılır.

```
class Program
{
    static void Main(string[] args)
    {
        //değişkeni tanımlayalım
        int n;

        //n değerini alalım
        Console.Write("n değeri:");
        n = Convert.ToInt32(Console.ReadLine());

        //1'den n'e kadar olan sayıların toplamını metotla yazdıralım
        Console.Write("n=" + n + " toplam=" + ntopla(n));
        Console.ReadLine();//bekle
    }

    //ana programa int tipinde değer döndüren metot
    static int ntopla(int x)
    {
        int toplam=0;
        for (int i = 1; i <= x; i++)
            toplam = toplam + i;
        return toplam;//toplam değerini ana programa gönder
    }
}
```

N Faktöriyel

N faktöriyel, 1'den N'e kadar olan sayıların çarpımı ile elde edilir. Bu uygulama için konsoldan n değerini alacağız ve metot kullanarak faktöriyel hesaplayacağız.

```
class Program
{
    static void Main(string[] args)
    {
        //n değerini okuyalım
        int n;
        Console.Write("n değeri:");
        n = Convert.ToInt32(Console.ReadLine());

        //n faktöriyel değerini yazdıralım
        Console.Write("n=" + n + "  n!=" + fk(n));
        Console.ReadLine();
    }

    //fk isimli faktöriyel hesaplayan metot
    static int fk(int x)
    {
        int f=1;
        for(int i=1; i<=x; i++)
            f=f*i;
        return f;
    }
}
```

1'den N'e Kadar Olan Sayıların Faktöriyeli

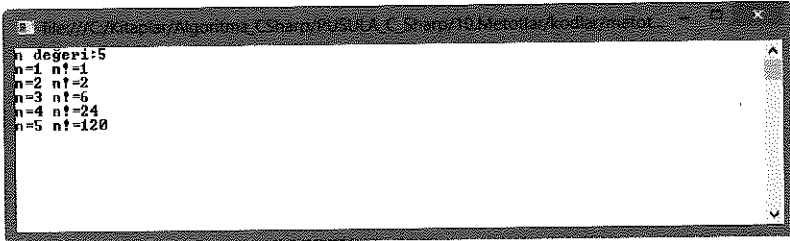
Bu örneğimizde konsoldan bir n değeri alacağız ve 1'den n'e kadar olan sayıları ve faktöriyel değerlerini yazdıracağız.

```
class Program
{
    static void Main(string[] args)
    {
        //n değerini alalım
        int n;
        Console.Write("n değeri:");
        n = Convert.ToInt32(Console.ReadLine());

        //1'den n'e kadar olan sayılar ve faktöriyeli
        for (int i = 1; i <= n; i++)
            Console.WriteLine("n=" + i + " n!=" + fk(i));
        Console.ReadLine();//bekle
    }

    //faktöriyel hesaplayan metot
    static int fk(int x)
    {
        int f = 1;
        for (int i = 1; i <= x; i++)
            f = f * i;
        return f;
    }
}
```

1'den 5'e kadar olan sayıların faktöriyelinin çıktısı:



```
File:///C:/Kutuplar/Algoritma/CSharp/PU50UA/CSharp10/Metotlar/kodlar/metot...
n değeri:5
n=1 n!=1
n=2 n!=2
n=3 n!=6
n=4 n!=24
n=5 n!=120
```

Alıştırmalar

- İki sayının çarpımını yapan metodu yazınız.
- Girilen iki sayının toplamını ve çarpımını döndüren metodu yazınız.
- Konsoldan okunan iki sayıdan büyük olanı konsola yazan metodu yazınız.
- Bir sayının (x) y. kuvvetini alan metodu yazınız.
- X'in karekökünü hesaplayan metodu oluşturunuz.
- Konsoldan girilen bir metni konsola alt alta 10 defa yazdıran metodu yazınız.
- Çalıştığında çarpım tablosunu konsola yazan metodu oluşturunuz.
- Kilo ile boy arasındaki fark 10 ya da küçükse "normal" daha büyükse "fazla" değerlerini döndüren metodu yazınız. Örneğin kişinin boyu 1.85 cm ve kilosu 90 ise (85-90) hesabıyla işlem yapınız.
- Taban uzunluğu ve yüksekliği verilen bir üçgenin alanını hesaplayan bir metot yazınız.
- Yarıçapı verilen bir çemberin alanını hesaplayan bir metot yazınız.
- Girilen bir kelimenin uzunluğunu bulan metodu yazınız.
- Çarpma işlemini toplama kullanarak yapan metodu geliştiriniz.

10

Sayısal ve Sözel Metotlar

- .NET Framework Metotları
- Math Sınıfı ve Matematiksel Metotlar
- Math Sınıfı Uygulamaları
- Sözel Metotlar

.NET Framework Metotları

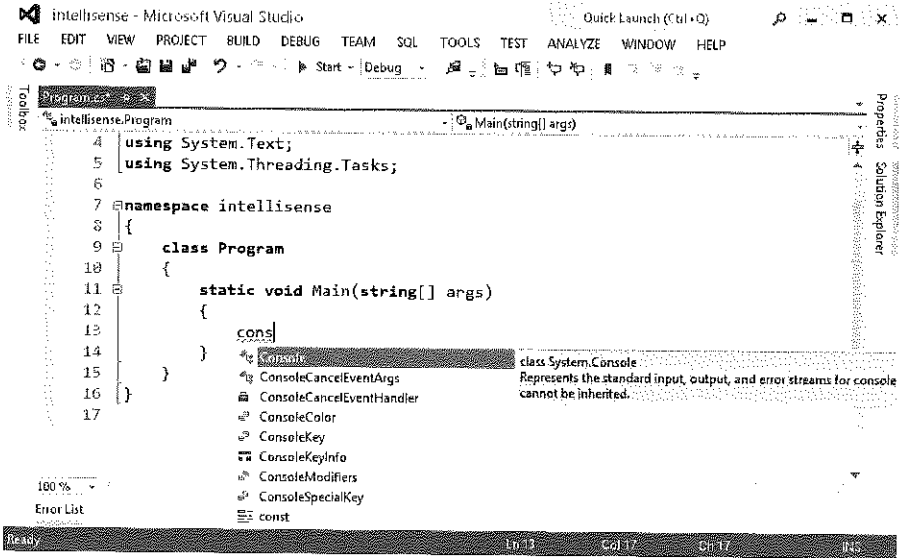
.NET Framework'te kullanılmaya hazır binlerce **class** yani hazır kod olduğunu daha önceki konularımızda öğrenmiştik. Her class'ın kendine ait özellik ve metotları olduğunu da önceki konulardan biliyoruz. Özellikler nesnenin görselliğini değiştirirken metotlar nesnenin bir iş yapmasını sağlıyordu. Bir nesnenin özellik veya metodunu aşağıdaki gibi kullanabiliriz:

```
Nesne.Özellik=değer; //Özellik nesnenin görselliğini değiştirir  
Nesne.Method(); //Metot nesnenin bir iş yapmasını sağlar
```

Herhangi bir **class** adı yazmaya başladığınız zaman Visual Studio **Intellisense** özelliği sayesinde Şekil 10.1'de görüldüğü gibi muhtemel sınıfları listeler. Yön tuşlarıyla veya fare ile yazılmak istenen sınıf adı seçilebilir. Örneğin editöre **conso...** yazdığınızda **conso** ile başlayan nesneler listelenir ve aralarından istediğinizi seçebilirsiniz. İlgili sınıfı seçmek için yön tuşlarını kullanın ve sınıf üzerine geldikten sonra **ENTER** tuşuna basın ya da ilgili sınıfa fare ile çift tıklayın.

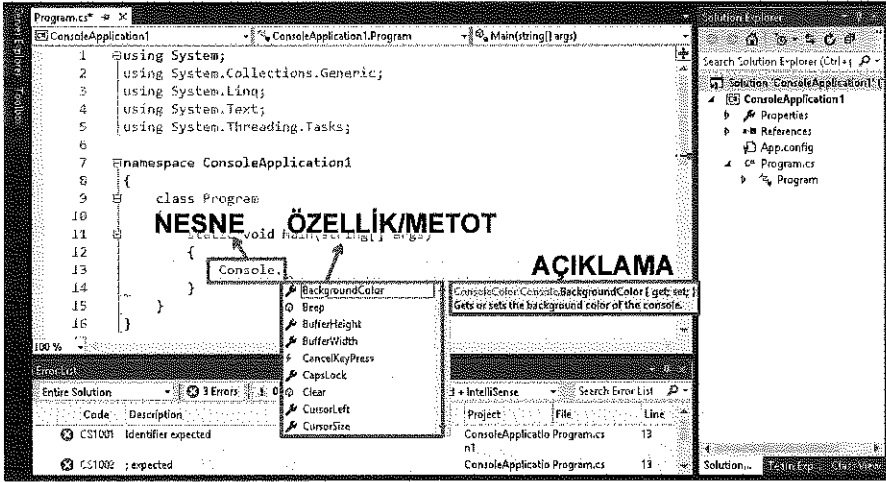
Visual Studio **Intellisense** özelliği sadece ilgili sınıfları listelemekle kalmaz aynı zamanda onları doğru şekilde yazmamızı sağlar. Örneğin **console** yazarsanız gelen seçenekten **Console** ile düzeltme yapabilirsiniz. **Console** nesnesinin ilk harfi büyüktür; C# harf duyarlı olduğu için **c** ile **C** birbirinden farklıdır ve düzeltilmesi gerekir.

Intellisense özelliği ilgili komutları listelemek ve düzeltmekle kalmaz aynı zamanda seçilen metodun ne işe yaradığını da gri kutu içinde açıklar.



Şekil 10.1 VS Intellisense özelliği

Sınıf adı tamamlandıktan sonra bir **nokta (.)** koyularak o sınıfa ait özellik ve metotlar listelenir. Aşağıdaki şekilden de görüldüğü gibi ilgili sınıfa ait özellikler **anahtar** simgesiyle gösterilirken metotlar **küp** simgesiyle listelenir. Böylece hem uygun özellik veya metodu seçmiş oluruz hem de onu doğru şekilde yazarız. Gri kutu içinde seçilen metot veya özelliğin açıklamasını okuyabilirsiniz.



Şekil 10.2 Sınıfın özellik ve metotları

Şu ana kadar **Console** sınıfını bolca kullandık; **Console** sınıfı metin penceresini temsil ediyor ve onu kullanmamızı sağlıyor. Aynı sınıfı kullanarak metot ve özellikler ile ilgili bir uygulama yapalım.

```
static void Main(string[] args)
{
    //konsol açıldığında uyarı sesi çıksın
    Console.Beep();

    //konsoldaki metinlerin ardalan ve rengini değiştirelim
    Console.BackgroundColor = ConsoleColor.Yellow;
    Console.ForegroundColor = ConsoleColor.Blue;

    //konsola "merhaba" yaz
    Console.Write("merhaba");

    //pencereyi kapatma
    Console.ReadLine();
}
```

Şekil 10.3 Console sınıfının bazı özellik ve metotları

Dikkat edilirse **Console** sınıfının özelliklerini kullanmak için **eşittir (=)** yazıyoruz ve özelliğe değer giriyoruz. Metot kullanmak için ise ilgili metodu seçip sonuna **parantez ()** koyuyoruz. Tabii ki her komut satırının sonuna **noktalı virgül (;)** koymayı unutmayoruz.

Math Sınıfı ve Matematiksel Metotlar

.NET Framework içinde matematiksel işlemlerde kullanılmak üzere hazır olarak gelen sınıfın adı **Math**'tir. **Math** sınıfının içinde gelen hazır metotları kullanarak kare, kök, mutlak değer, yuvarlama gibi birçok işlemi kolayca yapabilirsiniz.

Daha önce de gördüğümüz gibi metotlar kendisine verilen değeri veya değerleri alır, bir işlem yapar ve işlem sonucunu ana programa döndürür. **Math** sınıfının metotları- nı da aynı mantıkla kullanacağız. Tek fark burada metotların hazır olmasıdır.

Aşağıdaki tabloda **Math** sınıfında sıkça kullanılan bazı metotları ve açıklamasını görebilirsiniz.

Metot & Özellik	Açıklama	Giriş	Çıkış
Math.Sqrt(x)	x'in karekökünü alır	4	2
Math.Pow(x,y)	x'in y'ninci kuvvetini alır	x=2, y=3	8
Math.Abs(x)	x'in mutlak değeri	-5	5
Math.Min(x,y)	x ile y'den küçük olanı döndürür	x=2, y=3	2
Math.Max(x,y)	x ile y'den büyük olanı döndürür	x=2, y=3	3
Math.Floor(x)	x'i en yakın küçük tamsayıya yuvarlar	3.7	3
Math.Ceiling(x)	x'i en yakın büyük tamsayıya yuvarlar	3.7	4
Math.Round(x)	x'i yuvarlar	3.7	4
Math.Sin(x)	x radyan açısının sinüsünü verir		
Math.Cos(x)	x radyan açısının kosinüsünü verir		
Math.Log(x)	e tabanına göre x'in logaritmasını verir		
Math.Log10(x)	10 tabanına göre x'in logaritmasını verir		
Math.PI	pi (3.14...) sayısı		
Math.E	e (2.718...) sayısı		

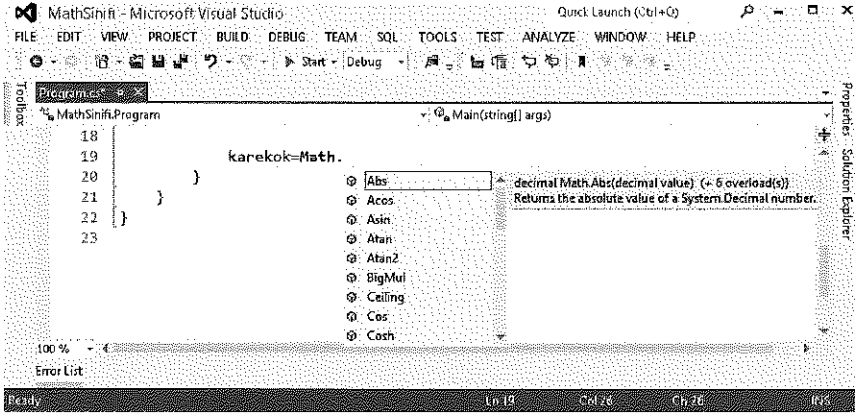
Math Sınıfı Uygulamaları

Math sınıfını kullanarak bazı uygulamalar yapacağız ve Math sınıfının yeri geldiğinde işimizi ne kadar kolaylaştırdığını göreceğiz. Lütfen yazılan kodları iyice inceleyin ve metotların nasıl kullanıldığını iyice anlamaya çalışın.

Konsoldan Okunan Sayının Karekökü

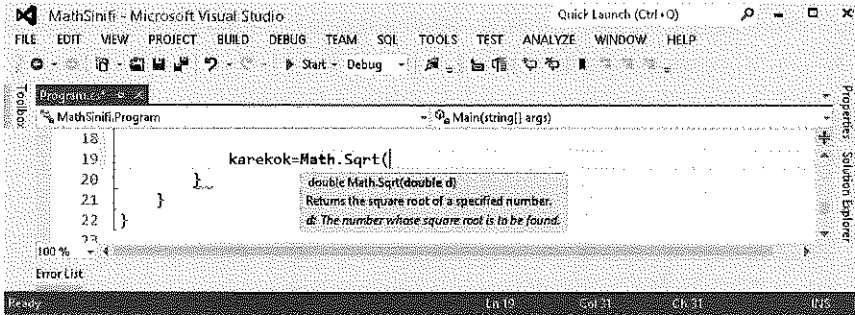
Math sınıfını kullanarak konsoldan okunan bir sayının karekökünü bulacağız. Karekök için **Math** sınıfının **Sqrt()** metodunu kullanmamız gerekiyor. Yani **karekök=Math.Sqrt(sayı)**; şeklinde bir komut yazmamız gerekiyor. Buradaki komut ile **sayı** değişkeninin karekökü alınıyor ve işlem sonucu **karekok** değişkenine atanıyor.

Editörde **Math** sınıfını yazıp nokta (.) koyduğunuz zaman aşağıdaki şekilde görüldüğü gibi **Intellisense** özelliği size kullanabileceğiniz metotları listeliyor. Uygun metodu seçtikten sonra parametre olarak parantez içine işlemek istediğiniz değeri koymanız gerekiyor.



Şekil 10.4 Math sınıfı

Metot seçildikten sonra parantez içine değişken yazarken değişkenin olması gereken veritipi de gösteriliyor. Lütfen çıkan gri kutu içindeki ipuçlarına dikkat edin...



Şekil 10.5 Metot ve açıklamaları

Ekrandaki açıklamaları da dikkate alarak metodun parametrelerini yazıyoruz.

Kodlarımız aşağıdaki gibi olmalıdır:

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double sayi, karekok;

    //konsoldan sayıyı okuyalım
    Console.Write("sayı:");
    sayi = Convert.ToDouble(Console.ReadLine());

    //sayının karekökünü hesaplayalım
    karekok = Math.Sqrt(sayi);

    //işlem sonucunu konsola yazalım
    Console.Write("sayı=" + sayi + " karekökü=" + karekok);

    //pencere kapanmasın
    Console.ReadLine();
}
```

X ile Y Arası Sayıların Karekökleri

Bu örneğimizde konsoldan iki sayı okuyacağız ve bu sayıların ve aradaki sayıların kareköklerini konsola yazdıracağız. Bu işlem için **Math.Sqrt()** metodunu kullanacağız.

Sayıları ve kareköklerini konsola yazdırırken aralarında bir tab (sekme) mesafe bırakmak için "\t" karakterini kullanacağız.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y;

    //konsoldan sayıları okuyalım
    Console.Write("x değeri:");
    x = Convert.ToInt32(Console.ReadLine());
    Console.Write("y değeri:");
    y = Convert.ToInt32(Console.ReadLine());

    //x ile y arası sayıların kendilerini ve köklerini yazdıralım
    for (; x <= y; x++)
        Console.WriteLine("sayı=" + x + "\t karekökü=" + Math.Sqrt(x));

    Console.ReadLine();//bekle
}
```

Ekran Çıktısı:

```

x değeri:5
y değeri:11
sayı=5 karekökü=2.23606797749979
sayı=6 karekökü=2.44948974278318
sayı=7 karekökü=2.64575131106459
sayı=8 karekökü=2.82842712474619
sayı=9 karekökü=3
sayı=10 karekökü=3.16227766016838
sayı=11 karekökü=3.3166247903554

```

Bir Sayının N. Kuvveti

Bu örneğimizde önce konsoldan bir sayı alacağız. Daha sonra da konsoldan alacağımız diğer sayıyı ilk sayının kuvveti olarak kullanacağız. Örneğin 2'nin 3. kuvvetini bulmak için önce 2 daha sonra da 3 alacağız. Bu işlem için **Math.Pow()** metodunu kullanacağız.

```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double s, k;

    //konsoldan sayıyı okuyalım
    Console.Write("sayı:");
    s = Convert.ToDouble(Console.ReadLine());

    //konsoldan sayının kuvvetini okuyalım
    Console.Write("kuvvet:");
    k = Convert.ToDouble(Console.ReadLine());

    //sayı ve kuvvetini konsola yazdıralım
    Console.WriteLine("sayı="+s+"\t"+k+".kuvveti="+Math.Pow(s,k));

    //bekle
    Console.ReadLine();
}

```

Program Çıktısı:

```

sayı:2
kuvvet:3
sayı=2 3.kuvveti=8

```

1'den N'e Kadar Olan Sayıların Kareleri Toplamı

Daha önceki örneklerimizde 1'den N'e kadar olan sayıların toplamını bulmuştuk. Bu örneğimizde sayıların kendilerini değil de karelerini toplayacağız. Bir sayının karesi-ni yani 2. kuvvetini bulmak için **Math.Pow(sayı,2)** metodunu kullanabiliriz.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n;
    double i, ntop=0;

    //konsoldan n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

    //1'den n'e kadar sayıların kareleri toplamı
    for (i = 1; i <= n; i++)
        ntop = ntop + Math.Pow(i, 2);

    //işlem sonucunu yazdıralım
    Console.Write("sayı:" + n + "\t toplam=" + ntop);
    Console.ReadLine();//bekle
}
```

Dairenin Alanı ve Çevresi

Yarıçapı verilen bir dairenin alanını ve çevresini **Math** sınıfının metodlarını kullanarak hesaplayacağız. **Pi** değeri için **Math.PI** kullanılacak.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double r, alan, cevre;

    //konsoldan yarıçapı alalım
    Console.Write("yarıçap:");
    r = Convert.ToDouble(Console.ReadLine());

    //alan ve çevreyi hesaplayalım
    alan = Math.PI * Math.Pow(r, 2); //pi*r*r
    cevre = 2 * Math.PI * r; //2*pi*r

    //işlem sonuçlarını konsola yazdıralım
    Console.WriteLine("yarıçap="+r+"\t çevre="+cevre+"\t alan="+alan);

    Console.ReadLine();//bekle
}
```

Program Çıktısı:

```

file:///C:/Kutupleri/Algoritma/CSharp/PUSULA_C_Sharp/11 Sayisal_Sozel_Metotlar...
varıcap:1
varıcap:1           çevre=6,28318530717959   alan=3,14159265358979

```

Çeşitli Metotlar

Konsoldan girilen bazı değerleri alarak bu değerleri değişik fonksiyonlarda kullanacağız ve çıktılarını göreceğiz.

```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double x, y;

    //konsoldan değerleri okuyalım
    Console.Write("x:");
    x = Convert.ToDouble(Console.ReadLine());
    Console.Write("y:");
    y = Convert.ToDouble(Console.ReadLine());

    //metotları kullanalım
    Console.WriteLine("x'in karesi:" + Math.Pow(x, 2));
    Console.WriteLine("x'in küpü:" + Math.Pow(x, 3));
    Console.WriteLine("x'in karekökü:" + Math.Sqrt(x));
    Console.WriteLine("x'in mutlak değeri:" + Math.Abs(x));
    Console.WriteLine("küçük olan:" + Math.Min(x, y));
    Console.WriteLine("büyük olan:" + Math.Max(x, y));
    Console.WriteLine("x'i küçük tamsayıya yuvarla:" + Math.Floor(x));
    Console.WriteLine("x'i büyük tamsayıya yuvarla:" + Math.Ceiling(x));
    Console.WriteLine("x'i tamsayıya yuvarla:" + Math.Round(x));
    Console.WriteLine("Log10(x):" + Math.Log10(x));
    Console.WriteLine("Sin(x):" + Math.Sin(x));
    Console.WriteLine("Cos(x):" + Math.Cos(x));

    Console.ReadLine();
}

```

Ekran Çıktısı:

```

file:///C:/Kutupleri/Algoritma/CSharp/PUSULA_C_Sharp/11 Sayisal_Sozel_Metotlar...
x:2
y:3
x'in karesi:4
x'in küpü:8
x'in karekökü:1.4142135623731
x'in mutlak değeri:2
küçük olan:2
büyük olan:3
x'i küçük tamsayıya yuvarla:2
x'i büyük tamsayıya yuvarla:2
x'i tamsayıya yuvarla:2
Log10(x):0.301029995663981
Sin(x):0.909297426825682
Cos(x):-0.416146836547142

```

Sözel Metotlar

Metinleri büyük veya küçük harfe çevirmek, metinlerdeki boşluk karakterlerini silmek, bazı karakterleri silmek vb. işlemler için sözel metotları kullanırız.

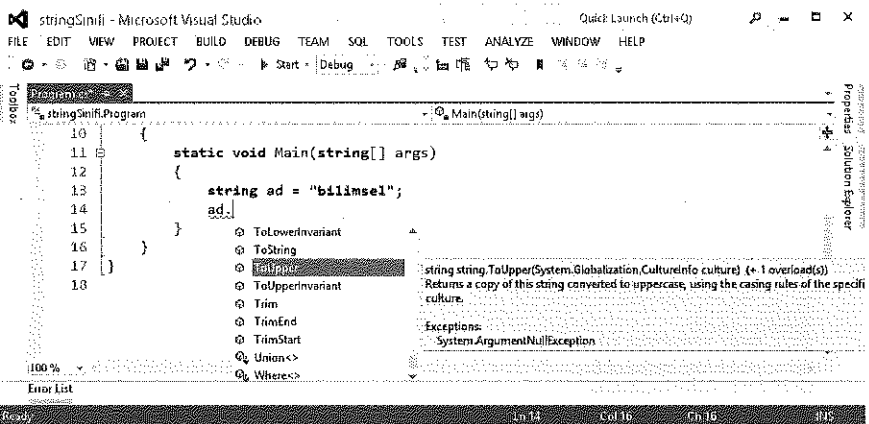
C# dilinde tek karakter saklayan değişken veri tipine **char**, birden fazla karakter saklayan veri tipine ise **string** denir. **char** tipindeki verilerin atanması tek tırnakla yapılırken **string** tipindeki veriler ise çift tırnakla atanır. Aşağıdaki örnekleri dikkatlice inceleyiniz:

```
char sonuc= 'e';
string ad= "bilimsel";
ad= ""; //ad değişkeninin değerini sil
```

C# dilinde **string** tipindeki değişkenler aslında birer karakter dizisidir; yani yan yana gelen harflerden oluşur. Örneğin yukarıda tanımlanan dizinin elemanlarına **ad[3]** gibi ifadelerle ulaşılabilir. Bu ifade **ad** değişkenindeki 3. karakteri işaret etmektedir. Dizi konusu daha sonraki konularda anlatılacağı için burada fazla ayrıntıya girilmeyecektir.

Sözel metotları kullanmak için **Math** gibi herhangi bir sınıfa ihtiyacımız yok; **string** veri tipinde tanımlanan her nesneye doğrudan sözel metotları uygulayabiliriz. Eğer **string** tipindeki bir nesnenin sağına nokta (.) koyarsanız uygulayabileceğiniz metotların bir listesini görebilirsiniz.

Aşağıda görünen kodlara dikkatlice bakalım. **Main()** metodunun ilk satırında **string** tipinde **ad** isimli bir değişken tanımlanır ve **"bilimsel"** metni atanır. Sonraki satırda **ad** değişkeninin sağına **nokta** konur ve kullanılabilecek metot ve özelliklerin listesi gelir. Bu listeden istenen metot veya özellik kullanılabilir.



Aşağıdaki tabloda sıkça kullanılan bazı metotları ve açıklamalarını görebilirsiniz.

Metot	Açıklama	Giriş	Çıkış
ToUpper()	büyük harfe çevir	abc	ABC
ToLower()	küçük harfe çevir	Abc	abc
Length	metin uzunluğu	abc	3
Trim()	sağdaki ve soldaki boşlukları sil	_abc_	abc
Substring(x)	x. karakterden son karaktere kadar kopyala		
Substring(x,y)	x. karakterden sonra y karakter kopyala		
Remove(x)	x. karakterden sonrasını sil		
Remove(x,y)	x. karakterden sonra y karakter sil		
Replace(a,b)	a stringini b stringi ile değiştir		

Aşağıdaki örnekte yukarıda bahsi geçen metotların uygulamasını görebilirsiniz.

```
static void Main(string[] args)
{
    //string değişkeni oluşturalım
    //son karakter "boşluk" karakteridir
    string metin="DeNeMe ";

    //metotları sırayla yazalım
    Console.WriteLine("boşluk karakterleri sil:" + metin.Trim());
    Console.WriteLine("tüm karakterleri küçült:" + metin.ToLower());
    Console.WriteLine("tüm karakterleri büyüt:" + metin.ToUpper());
    Console.WriteLine("metin karakter sayısı:" + metin.Length);
    Console.WriteLine("3. karakterden sonrasını sil:" + metin.Remove(3));
    Console.WriteLine("2. karakterden sonra 2 karakter sil:" + metin.Remove(2,2));
    Console.WriteLine("-Ne- karakterlerini -ZZ- ile değiştir:" + metin.Replace("Ne", "ZZ"));

    //pencereyi kapatma
    Console.ReadLine();
}
```

Ekran Çıktısı:

```
boşluk karakterleri sil:DeNeMe
tüm karakterleri küçült:deneme
tüm karakterleri büyüt:DENEME
metin karakter sayısı:7
3. karakterden sonrasını sil:DeN
2. karakterden sonra 2 karakter sil:DeMe
-Ne- karakterlerini -ZZ- ile değiştir:DeZZMe
```


11

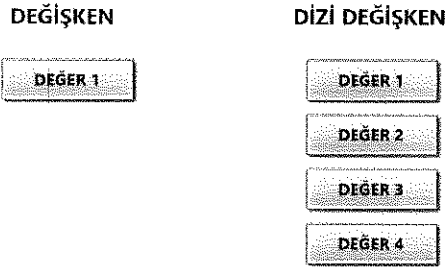
Diziler

- Dizi Nedir?
- Dizi Tanımlama
- İki Boyutlu Diziler
- Array Sınıfı
- Metinler ve Diziler Arasındaki İlişki
- Dizi Uygulamaları

Dizi Nedir?

Bellekte art arda yer alan aynı veri tipinden değerler kümesine dizi denir. Şu ana kadar oluşturduğumuz bir değişken içine sadece tek bir veri koyduk. Örneğin **int x;** komutuyla bir tamsayı **x** değişkeni tanımladık ve **x=5** gibi atama komutlarıyla bu değişkenlere tek bir değer atadık. Pekii! Eğer bir değişkene birden fazla değer atamak istersek ne yapacağız? Örneğin **string hafta;** şeklinde tanımlanan bir sözel değişkene **Pazartesi, Salı, Çarşamba, Perşembe, Cuma, Cumartesi** değerlerini atamak istersek ne yapacağız? Ya da bir futbol takımının oyuncularını bir değişkende saklamak istersek ne yapacağız? İşte bu durumlarda dizi değişkenlerini kullanmamız gerekiyor.

Normal değişkenler tek bir belleğe sahipken dizi değişkenler çok sayıda belleğe sahiptir. Aşağıdaki şekilde görüldüğü gibi, normal değişken tek bir değere sahipken dizi değişken birçok değere sahip olabilir.



Şekil 11.1 Dizi değişken ve bellek ilişkisi

Dizinin her bir değerine eleman denir. Örneğin 5 elemanlı bir dizi tanımlamak demek bu dizinin 5 adet belleği var demektir ve her belleğe aynı türde farklı değerler koyulabilir.

Tek satır veya sütundan yani tek sıradan oluşan dizilere **vektör** de denilmektedir ve özellikle matematiksel işlemlerde karşımıza çıkmaktadır.

Dizi Tanımlama

C# dilinde dizi değişkenler iki farklı şekilde tanımlanabilir:

```
veritipi[] değişken=new veritipi[eleman sayısı];
veritipi[] değişken={değer1, değer2, değer3,...,değerN};
```

Eğer birinci yöntemi kullanarak dizi tanımlayacaksanız aşağıdaki dizi tanımlamaları iyi birer örnek olabilir. Bu yöntemde veri tipi ile beraber eleman sayısı da belirleniyor.

```
int[] x = new int[5]; //5 elemanlı tamsayı x dizisi
double[] alan = new double[10]; //10 elemanlı ondalıklı alan dizisi
string[] fb = new string[11]; //11 elemanlı sözel fb dizisi
```

İlk yöntemle dizi tanımlandıktan sonra elemanlar atanmalıdır. Örneğin x dizisine eleman atanacaksa aşağıdaki gibi yapılmalıdır:

```
x[0]=5; //dizinin ilk indeksi/indisi 0'dan başlar
x[1]=22; //1 indeksli/indisli dizi elemanı
x[2]=33;
x[3]=3;
x[4]=10;
```

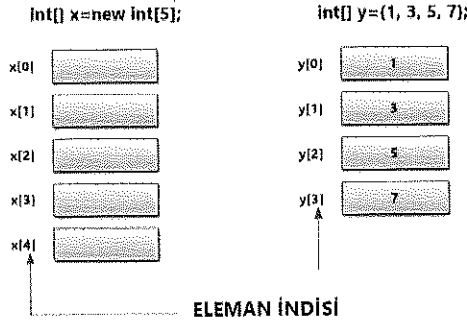
Eğer ikinci yöntemi kullanarak dizi değişkeni oluşturacaksak aşağıdaki örneklere benzer diziler tanımlanabilir. Bu yöntemde hem dizi değişkeni tanımlanıyor hem de değerler (elemanlar) atanıyor.

```
int[] y={1, 3, 5, 7, 11};
double[] cevre={3.22, 5.232, 6.1212 };
string fener={"rüşü", "ismail", "şenol", "müjdat"};
```

Eğer dizinin bir elemanını bir değişkene atayacaksanız aşağıdaki yapıyı kullanmalısınız:

```
int x=dizi[y]; //dizinin y indisli elemanını x değişkenine ata
```

Diziyi nasıl tanımlarsanız tanımlayın sonuçta aşağıdaki gibi bir yapı ortaya çıkar; dizinin eleman sayısı kadar bellek oluşturulur ve yöntemine göre bellekler boş veya dolu olur. Aşağıdaki şekilde görüldüğü gibi birinci yöntemde dizi bellekleri boş iken ikinci yöntemde dizi bellekleri atama yapıldığından dolayı dolu olur.



Şekil 11.2 Dizi değişken ve indisleri

Dizi değişken oluştururken dikkat edilmesi gereken konular:

- Dizi değişkeninin adı normal değişken kurallarıyla aynı olmalıdır.
- Dizide aynı türde veriler olmalıdır; örneğin bir diziyi **int** tipinde tanımladıysanız tüm elemanlar da **int** tipinde olmalıdır.
- Dizi indisi sıfırdan başlar. Dizi elemanlarının numarasına indis denir. Örneğin **x[2]** ifadesi dizideki 2 numaralı indise sahip elemanı belirtir.
- Veri tipini belirttikten hemen sonra köşeli parantez (**[]**) oluşturulmalıdır. Örneğin **string[] ad;** komutuyla metin tipinde bir dizi tanımlanabilir. Eğer iki boyutlu dizi tanımlanacaksa köşeli parantez içine virgül (**[,]**) koyulur.
- Eğer değerler atanacaksa **={d1, d2, d3}** şeklinde atanmalıdır.
- Değer atanmayacaksa **=new veritipi[eleman_sayısı]** şeklinde tanımlama ve boyut belirtmelidir.

Örnek Kod:

```
int[] asal=new int[5]; //asal isimli 5 elemanlı dizi
asal[0]=1; //dizinin 0. indisine 1 atandı
asal[1]=3;
asal[2]=5;
asal[3]=7;
asal[4]=11;
foreach(int eleman in asal) //asal dizisindeki her değeri
Console.WriteLine(eleman); //eleman olarak tanımla ve alt alta yaz
Console.Read();
```

Yukarıda tanımlanan dizi bellekte aşağıdaki gibi görünür:



Şekil 11.3 asal dizisi, indisleri ve elemanları

Örnek Kod:

Dizi elemanlarının for() döngüsü ile ekrana yazılması.

```
string[] takim={"ali","veli","ahmet","mehmet"}; //takim isimli dizi
for(int i=0;i<=3;i++) //indisi 0-3 olan dizi elemanlarını
Console.WriteLine(takim[i]); //konsolda alt alta yaz
Console.Read();
```

Örnek Kod:

Dizi elemanlarını for() döngüsü yerine foreach() döngüsü ile de yazabilirsiniz.

```
double[] alanlar= new double[4]; //alanlar isimli double tipinde dizi
alan[0]=1.234; //0 indisli eleman değeri=1.234
alan[1]=2.184;
alan[2]=4.254;
alan[3]=1.4;
foreach(int eleman in alanlar) //alanlar dizisindeki her değeri
Console.WriteLine(eleman); //eleman olarak tanımla ve alt alta yaz
Console.Read();
```

Örnek Kod:

Dizi elemanlarının konsoldan okunması için genelde for() döngüsü kullanılır. 5 elemanlı tamsayı bir dizi olduğunu düşünürsek dizi elemanlarının konsoldan okunması aşağıdaki gibi olur:

```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int[] dizi = new int[5];
    int i; //dizi indeksi

    //konsoldan dizi elemanlarını okuyalım
    //dizi indeksi 0,1,2,3,4 olacak
    for (i = 0; i < 5; i++)
    {
        Console.Write("gir dizi indeksi " + i + ":");

        //okunan her değeri dizinin i. indeksine atayalım
        dizi[i] = Convert.ToInt32(Console.ReadLine());
    }

    //bekle
    Console.ReadLine();
}

```

Eğer yukarıdaki programda okunan dizi elemanlarını ekrana yazmak isterseniz for() veya foreach() döngülerinden birini kullanabilirsiniz.

Örnek Kod:

Dizi elemanlarını kullanarak matematiksel işlemler de yapabilirsiniz. Örneğin bir dizinin 2. indeksi ile 4. indeksini toplayıp **ikitop** değişkenine atamak isterseniz veya dizi elemanlarının ortalamasını bulmak isterseniz aşağıdaki kodları yazabilirsiniz:

```

int[] dizi={1, 2, 3, 4, 5};
int ikitop=0, dizitoplam=0, ort=0;
ikitop=dizi[2]+dizi[4];
dizitoplam=dizi[0]+dizi[1]+dizi[2]+dizi[3]+dizi[4];
ort=toplam/5;

```

Örnek Kod:

Yukarıdaki toplama işlemini for() döngüsü ile yapmak çok daha mantıklı. Özellikle dizinin eleman sayısının çok fazla olduğu durumlarda yukarıdaki toplama işlemi hantalıktan başka bir şey değil. O nedenle aşağıdaki for() döngüsünün ne yaptığını çok iyi anlamanız gerekir.

```
int[] dizi={1, 2, 3, 4, 5};
int toplam=0, ort=0;
for(int i=0; i<5; i++)
    toplam=toplam+dizi[i];
ort=toplam/5;
Console.Write("toplam="+toplam+ " ortalama="+ort);
```

Örnek Kod:

Yukarıdaki kodlarda dizi uzunluğu için **Length** özelliği kullanılabilir. Bu durumda kodlar aşağıdaki gibi olur:

```
int[] dizi={1, 2, 3, 4, 5};
int toplam=0, ort=0;
uzunluk=dizi.Length; //dizi uzunluğu ya da dizinin eleman sayısı
for(int i=0; i<uzunluk; i++) //dizi indeksleri 1-4 arası
    toplam=toplam+dizi[i];
ort=toplam/uzunluk;
Console.Write("toplam="+toplam+ " ortalama="+ort);
```

İki Boyutlu Diziler

Tek boyutlu diziler belirli isimde listeler oluşturup bu listelerle ilgili işlem yapmak için kullanılırken iki boyutlu diziler daha çok satır ve sütunlardan oluşan yapılar oluşturmak için kullanılır.

Örneğin bir otel olduğunu düşünün. Bu otelin 4 katı olduğunu ve her katta 5 oda olduğunu varsayın. Bu durumda kat:0 oda:0, kat:0 oda:1, kat:0 oda:2 şeklinde iki boyutlu bir dizi elde etmiş oluruz. Bir resim bin kelimeye bedel diyerek iki boyutlu dizileri size aşağıdaki Şekil 11.4 üzerinden anlatayım; her kattaki odaları yan yana koyarsanız satır ve sütunlardan oluşan bir yapı elde edersiniz. Satırlar katları temsil ederken sütunlar da odaları temsil etmektedir. Böylece {0,2} ifadesi bize 0. kattaki 2. odayı gösterirken {2,4} ifadesi ise 2. kattaki 4. odayı gösterir.

Aşağıdaki şekilde iki boyutlu bir dizideki elemanları tanımlamaya yarayan indis numaralarını görebilirsiniz. Örneğin {0,0} ifadesi 0. satır ile 0. sütunu temsil etmektedir. {1,3} ifadesi ise 1. satır ile 3. sütunu temsil etmektedir.

	oda:0	oda:1	oda:2	oda:3	oda:4
kat:0	0,0	0,1	0,2	0,3	0,4
kat:1	1,0	1,1	1,2	1,3	1,4
kat:2	2,0	2,1	2,2	2,3	2,4
kat:3	3,0	3,1	3,2	3,3	3,4

kat:3 oda:0	kat:2 oda:1	kat:1 oda:3
----------------	----------------	----------------

Şekil 11.4 İki boyutlu dizi

İki boyutlu dizileri aşağıdaki gibi tanımlayabiliriz:

```
veritipi[,] değişken=new veritipi[satır_sayısı, sütun_sayısı];
veritipi[,] değişken={{sa0,sü0},{sa0,sü1},{sa1,sü0},{sa1,sü1}};
```

Eğer birinci yöntemi kullanarak dizi tanımlayacaksa aşağıdakileri örnek alabilirsiniz. Bu yöntemde veri tipi ile beraber eleman sayısı da belirleniyor.

```
int[,] x = new int[5][3]; //5 satır 3 sütunlu iki boyutlu x dizisi
double[,] alan = new double[2][3];
string[,] odalar = new string[2][3];
```

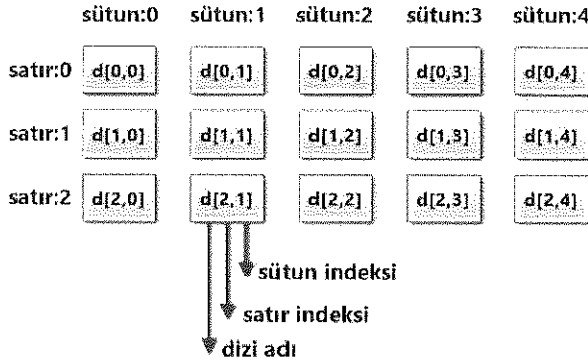
Bu yöntemle iki boyutlu dizi tanımlandıktan sonra elemanlar atanmalıdır. Örneğin **odalar** dizisine eleman atanacaksa aşağıdaki gibi yapılmalıdır:

```
odalar[0,0]="ali";           //0. kat 0. odada ali kalıyor
odalar[0,1]="veli";         //0. kat 1. odada veli kalıyor
odalar[0,2]="ahmet";        //0. kat 2. odada ahmet kalıyor
odalar[1,0]="ayşe";         //1. kat 0. odada ayşe kalıyor
odalar[1,1]="fatma";
odalar[1,2]="zeynep";
```

Eğer ikinci yöntemi kullanarak dizi değişkeni oluşturacaksa aşağıdaki örneklere benzer diziler tanımlanabilir. Bu yöntemde hem dizi değişkeni tanımlanıyor hem de değerler (elemanlar) atanıyor.

```
int[,] y={{1, 3, 5},{ 7, 11,13}}; //2 satırlı 3 sütunlu dizi ve değerleri
string odalar={"ali","veli","ahmet"},{"ayşe","fatma","zeynep"};
```

Eğer elimizde `int[,] d=new int[3][5];` şeklinde tanımlanmış üç satırdan ve beş sütundan oluşan bir dizi olduğunu düşünersek dizi indeksleri aşağıdaki gibi olur:



Şekil 11.5 Dizi indeksleri

Belirli satır ve sütun sayısından oluşan iki boyutlu dizilere matris de denmektedir ve matematikte önemli bir yer tutmaktadır.

Örnek Kod:

İki boyutlu dizi elemanlarının okunması ve yazılması en çok karıştırılan konulardan biridir. Sorunun kaynağı satır, sütun ve indekslerin unutulması veya karıştırılmasıdır. Bu örneğimizde iki boyutlu bir dizi tanımlayıp dizi elemanlarını konsoldan okuyacağız. Dizinin 2 satır 3 sütun olması konunun anlaşılması için yeterlidir.

Dizi elemanlarının okunması 0.satır-0.sütun, 0.satır-1.sütun, 0.satır-2.sütun, 1.satır-0sütun, 1.satır-1.sütun, 1.satır-2.sütun sırasını takip edecektir. Yani önce 0. satır, sonra da 1. satır okunacaktır. Hazırlanacak olan for() döngüsü de bu mantıkla çalışacaktır.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int[,] dizim=new int[2,3];
    int i; //satırlar
    int j; //sütunlar

    //dizinin elemanlarını okumayı e[satır][sütun]şeklinde yapacağız
    //dizi 2 satır 3 sütun olduğuna göre aşağıdaki gibi okuyacağız
    //e00, e01, e02
    //e10, e11, e12
    //okumayı for() döngüsü ile yapacağız
```

```

for(i=0; i<2;i++) //satırları oku
{
    for(j=0; j<3;j++) //her satır için sütunları oku
    {
        Console.Write("e"+i+j+":");
        dizim[i, j] = Convert.ToInt32(Console.ReadLine());
    }
}
Console.ReadLine(); //bekle
}

```

Array Sınıfı, Dizi Özellikleri ve Metotları

Diziler **Array** sınıfından oluşturulmaktadır. Yani bir dizi tanımlamak için **int[] dizi = new int[5];** komutunu kullandığımızda aslında **Array** sınıfından **int** tipinde 5 elemanlı **dizi** isimli bir dizi ya da nesne tanımlamış oluruz.

Array sınıfının bazı özellik ve metotlarını kullanarak dizilerle ilgili işlemlerde hız ve kolaylık kazanabiliriz. **Array** sınıfının özellik ve metotlarını tıpkı **Math** sınıfında olduğu gibi kullanacağız. Örneğin **int[] dizi = new int[5];** şeklinde tanımlanmış bir dizi olduğunu düşünelim. Bu dizinin özellik ve metotlarını aşağıdaki gibi kullanabiliriz:

```

dizi.Method();
dizi.Özellik;

```

Aşağıdaki tabloda sıkça kullanılan bazı metotları, özellikleri ve açıklamalarını görebilirsiniz.

Metot & Özellik	Açıklama
dizi.Length	dizinin eleman sayısı
dizi.Rank	dizinin boyutu
Array.Sort(dizi)	diziyi sırala
Array.Reverse(dizi)	dizinin elemanlarını ters çevir/sırala
Array.Clear(dizi,a,b)	dizi'nin elemanlarını a indeksinden b indeksine kadar sil
dizi.SetValue(x,y)	y. elemana x değeri atar
Array.BinarySearch(dizi,anahtar)	dizide anahtar kelime aranır. eğer anahtar kelime bulunursa indeks, bulunamazsa -1 döner

Metinler ve Diziler

C# dilinde **string** veri tipi birden fazla karakteri saklayabiliyorken **char** veri tipi tek bir karakter saklar. Eğer bir **string** değişkene **ad= "bilimsel"**; şeklinde bir atama yapılsa aşağıdaki gibi bir yapı veya dizi oluşur:



Şekil 11.6 String diziler

string tipindeki değişkene atanmış bir verinin her karakteri dizinin elemanı kabul edildiği için değişik değişik kelime işlem programları yazılabilir. Örneğin okunan kelimelerin tersten yazılması, karakterlerin alt alta yazılması sıkça karşınıza çıkabilir.

Eğer **string** tipindeki **ad** değişkeninin tüm karakterlerini alt alta yazdırmak isterseniz aşağıdaki kodları kullanabilirsiniz:

```
string ad = "bilimsel";
for (int i = 0; i < ad.Length; i++) //dizinin her i değeri için
Console.WriteLine(ad[i]); //elemanı konsola yaz
Console.ReadLine();
```

Yukarıdaki örnekte **for** döngüsü kullanılarak **ad** değişkeninin her karakteri (**ad[i]**) konsolda alt alta yazdırılmaktadır.

String verileri karakterlerden oluştuğu için **char** tipinde tanımlanmış bir dizi olarak da düşünülebilir. Aşağıdaki kodları incerseniz ne demek istediğimi anlayacaksınız.

```
char[] dizi={'b', 'i', 'l', 'i', 'm', 's', 'e', 'l'};
string ad=new string(dizi);
Console.Write(ad);
```

Dizi Uygulamaları

Her zaman olduğu gibi gene kolay ve anlaşılır örneklerden başlayarak zor örneklerle doğru dizi uygulamaları yapacağız.

Dizi elemanlarının konsola nasıl yazıldığını, konsoldan okunan sayıların bir diziye nasıl aktarıldığını, dizilerde sıralama ve aramanın nasıl yapıldığını örneklerle pekiştirmeye çalışacağız.

Diziler programcılık için çok önemli olduğundan her örneği iyice inceleyin ve anlamaya çalışın.

Dizi Elemanlarının Konsola Yazdırılması

Bu örneğimizde bir dizi oluşturacağız ve dizi elemanlarına değer atayacağız. Daha sonra da iki farklı yöntem kullanarak dizi elemanlarını konsola yazacağız.

Bir dizinin elemanlarına ulaşmak için o dizinin eleman sayısı kadar dönen **for** veya **foreach** döngüleri oluşturulur. Gerekli açıklamalar kodların arasına yazılmıştır. Lütfen açıklamaları okuyarak deneyiniz.

```
static void Main(string[] args)
{
    //takim isimli 5 elemanlı diziyi tanımlayalım
    //dizi bir basket takımının isimlerini barındırıyor
    string[] takim = new string[5];

    //dizinin elemanlarını atayalım
    takim[0] = "ali"; //0. indeks elemanı ali
    takim[1] = "veli"; //1. indeks elemanı veli
    takim[2] = "ahmet"; //2. indeks elemanı ahmet
    takim[3] = "mehmet"; //3. indeks elemanı mehmet
    takim[4] = "osman"; //4. indeks elemanı osman

    //dizi elemanlarını alt alta konsola yazdıralım
    foreach (string eleman in takim)
        Console.WriteLine(eleman);

    //dizi elemanlarını yan yana konsola yazdıralım
    foreach (string eleman in takim)
        Console.Write(eleman + "-");

    Console.WriteLine("\n"); //boş satır
    //dizi elemanlarını for döngüsü ile yazalım
    for (int i = 0; i < takim.Length; i++)
        Console.WriteLine(takim[i]);

    //bekle, pencereyi kapatma
    Console.ReadLine();
}
```

Konsoldan Okunan Adların Dizide Saklanması

Konsoldan **N** değeri alacağız ve döngü kullanarak **N** adet ismi bir dizide saklayacağız.

Dizilerde en çok karıştırılan konu **diziadi[x]** şeklindeki ifadelerdir. Buradaki **x** dizinin indeksidir. Dizi indeksi dizinin **x+1**. elemanını ifade eder. Örneğin **diziadi[2]** dizinin indeksi 2 olan elemanı gösterir fakat aslında bu dizinin 3. elemanıdır çünkü dizi indeksi 0'dan başlar.

```

diziadi[0].->dizinin 1. elemanı
diziadi[1] ->dizinin 2. elemanı
diziadi[2] ->dizinin 3. elemanı //indeks 2 eleman 3
diziadi[3] ->dizinin 4. elemanı

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n;

    //n değerini (yani dizinin eleman sayısını) alalım
    Console.Write("dizinin eleman sayısı:");
    n = Convert.ToInt32(Console.ReadLine());

    //adlar isimli n elemanlı string dizi tanımlayalım
    string[] adlar = new string[n];

    //isimleri okuyup dizi değişkenine atayalım
    for (int i = 0; i <= adlar.Length-1; i++)
    {
        Console.Write(i + ". eleman:");
        adlar[i] = Console.ReadLine();
    }

    //okunan isimleri yazdıralım
    for (int j = 0; j <= adlar.Length - 1; j++)
        Console.WriteLine("dizinin " + j + ". elemanı:" + adlar[j]);

    //bekle
    Console.ReadLine();
}

```

N Adet Sayının Ortalaması

Bu örnekte dışarıdan okunan **n** adet tamsayının ortalamasını bulacağız. Bir önceki örnekte olduğu gibi önce **n** sayısını yani dizinin eleman sayısını alacağız. Daha sonra da girilen **n** değerine göre konsoldan **n** adet sayı okuyup dizi değişkenine atayacağız. Ondan sonra da dizi elemanlarını toplayıp **n** değerine böleceğiz ve ortalama-yı bulmuş olacağız.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, top=0, ort;

    //konsoldan n değerini okuyalım
    Console.Write("n değeri:"); n = Convert.ToInt32(Console.ReadLine());

    //n elemanlı bir tamsayı dizi tanımlayalım
    int[] sayilar = new int[n];

    //sayıları teker teker okuyalım ve diziye atayalım
    for (int i = 0; i <= sayilar.Length-1; i++)//dizinin her i indeksi için
    {
        //konsola "elemanı gir" yaz
        Console.Write(i + " indeksli elemanı gir:");
        //konsoldan okunan değeri tamsayıya çevir ve sayilar[i] değişkenine ata
        sayilar[i] = Convert.ToInt32(Console.ReadLine());
    }

    //girilen sayıların toplamını bul
    for (int i = 0; i <= sayilar.Length - 1; i++)//dizinin her i indeksi için
        top = top + sayilar[i];

    //ortalamayı bulalım ve konsola yazdıralım
    ort = top / n;
    Console.WriteLine("ortalama=" + ort);

    Console.ReadLine();//bekle
}
```

Kelime Uzunluğu

Konsoldan girilen kelimenin ya da string değerin uzunluğunu bulup konsola yazdıracağız. Hatırlayacağınız gibi string veri tipi aslında bir karakter dizisi idi. String tipindeki değerler bir dizi olduğuna göre kelime uzunluğu **dizi.Length** özelliği ile bulunabilir.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    string kelime="";
    int uzunluk = 0;

    //konsoldan kelimeyi okuyalım
    Console.Write("kelime:");
    kelime = Console.ReadLine();

    //kelime uzunluğu
    uzunluk = kelime.Length;

    //kelime uzunluğunu yazalım
    Console.WriteLine("kelime uzunluğu=" + uzunluk);

    //veya
    Console.WriteLine("kelime uzunluğu=" + kelime.Length);

    Console.ReadLine();
}
```


N Adet Sayının Ortalaması

Bu örnekte dışarıdan okunan **n** adet tamsayının ortalamasını bulacağız. Bir önceki örnekte olduğu gibi önce **n** sayısını yani dizinin eleman sayısını alacağız. Daha sonra da girilen **n** değerine göre konsoldan **n** adet sayı okuyup dizi değişkenine atayacağız. Ondan sonra da dizi elemanlarını toplayıp **n** değerine böleceğiz ve ortalamayı bulmuş olacağız.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, top=0, ort;

    //konsoldan n değerini okuyalım
    Console.Write("n değeri:"); n = Convert.ToInt32(Console.ReadLine());

    //n elemanlı bir tamsayı dizi tanımlayalım
    int[] sayilar = new int[n];

    //sayıları teker teker okuyalım ve diziye atayalım
    for (int i = 0; i <= sayilar.Length-1; i++)//dizinin her i indeksi için
    {
        //konsola "elemanı gir" yaz
        Console.Write(i + " indeksli elemanı gir:");
        //konsoldan okunan değeri tamsayıya çevir ve sayilar[i] değişkenine ata
        sayilar[i] = Convert.ToInt32(Console.ReadLine());
    }

    //girilen sayıların toplamını bul
    for (int i = 0; i <= sayilar.Length - 1; i++)//dizinin her i indeksi için
        top = top + sayilar[i];

    //ortalamayı bulalım ve konsola yazdıralım
    ort = top / n;
    Console.WriteLine("ortalama=" + ort);

    Console.ReadLine();//bekle
}
```

Kelime Uzunluğu

Konsoldan girilen kelimenin ya da string değerin uzunluğunu bulup konsola yazdıracağız. Hatırlayacağınız gibi string veri tipi aslında bir karakter dizisi idi. String tipindeki değerler bir dizi olduğuna göre kelime uzunluğu **dizi.Length** özelliği ile bulunabilir.

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    string kelime="";
    int uzunluk = 0;

    //konsoldan kelimeyi okuyalım
    Console.Write("kelime:");
    kelime = Console.ReadLine();

    //kelime uzunluğu
    uzunluk = kelime.Length;

    //kelime uzunluğunu yazalım
    Console.WriteLine("kelime uzunluğu=" + uzunluk);

    //veya
    Console.WriteLine("kelime uzunluğu=" + kelime.Length);

    Console.ReadLine();
}
```

Sayıların Sıralanması

Klavyeden N adet tamsayı alacağız ve dizi özellik ve metotlarını kullanarak sayıları küçükten büyüğe veya büyükten küçüğe yazdıracağız.

```
static void Main(string[] args)
{
    //n değerini yani dizinin eleman sayısını alalım
    Console.Write("n değeri:");
    int n = Convert.ToInt32(Console.ReadLine());

    //sayıların atanacağı diziye tanımlayalım
    int[] sayilar=new int[n];

    //n adet sayıyı teker teker okuyup sayilar[] dizisine atayalım
    for (int i = 0; i < sayilar.Length; i++)//dizinin her i değeri için
    {
        Console.Write(i + " indeksli elemanı giriniz:");
        sayilar[i] = Convert.ToInt32(Console.ReadLine());
    }

    //dizinin elemanlarını sıralayalım ve yazdıralım
    Array.Sort(sayilar);//dizi elemanlarını küçükten büyüğe sırala
    foreach (int eleman in sayilar) //dizinin her elemanı için
        Console.WriteLine(eleman); //elemanı konsola yaz ve alt satıra geç

    //dizinin elemanlarını ters sıralayalım ve yazdıralım
    Array.Reverse(sayilar);//sayılar dizisinin elemanlarını ters çevir
    foreach (int terseleman in sayilar)
        Console.WriteLine(terseleman);

    Console.ReadLine();//bekle
}
```

Dizide Arama Yapmak

Dizilerde arama yapmak çok önemli ve çok kullanılan konulardan biridir. Bu örneğimizde sözel tipte 5 elemanlı bir dizimizin olduğunu varsayacağız ve elemanlar arasında arama yapacağız. Eğer aranan değer bulunduysa konsola yazacağız bulunmadıysa "bulunamadı" yazacağız.

Bu örnekte arama için **.NET Framework** içinde gelen **BinarySearch()** metodunu kullanacağız. Bu metodun kullanılabilmesi için önce dizide sıralama (Sort) yapılmalıdır.

```
static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    string anahtar; int indeks = 0;
    string[] isimler = new string[5];
    isimler[0] = "ali";
    isimler[1] = "veli";
    isimler[2] = "ahmet";
    isimler.SetValue("mehmet", 3);
    isimler.SetValue("osman", 4);

    //dizi elemanlarını A'dan Z'ye sıralayalım (ahmet,ali,mehmet, osman,veli)
    Array.Sort(isimler);

    //konsoldan aranacak anahtar kelimeyi alalım
    Console.Write("anahtar kelime:");
    anahtar = Console.ReadLine();
    anahtar = anahtar.Trim();//varsa boşluk karakterlerini silelim

    //anahtar kelimeyi arayalım,kelime bulunursa indeksini değişkene atalım
    indeks = Array.BinarySearch(isimler, anahtar);

    //anahtar kelime bulunduysa BinarySearch() elemanın indeksini döndürür
    //bulunmadıysa -1 döner
    if (indeks <= -1) Console.Write("aranan metin bulunamadı");
    else Console.Write("aranan metin dizinin "+(indeks+1)+ ". elemanı");

    Console.ReadLine();
}
```

Dizide Arama Yapmak (Metotsuz Yöntem)

Bir önceki örnekte dizide arama işlemini "BinarySearch()" metoduyla yaptık. Bu örneğimizde ise aramayı herhangi bir metod kullanmadan daha klasik bir yöntemle yapacağız.

Do-while döngüsü kullanarak dizinin her elemanını anahtar kelime ile karşılaştıracğız. Eğer aranan kelime bulunursa indeksi sonuç değişkenine aktaracağız.

```
static void Main(string[] args)
{
    //önce değişkenleri tanımlayalım
    string anahtar, sonuc="bulunamadı"; int indeks=0;
    string[] isimler = new string[5];
    isimler[0]= "ali";
    isimler[1]= "veli";
    isimler[2]= "ahmet";
    isimler.SetValue("mehmet", 3);
    isimler.SetValue("osman", 4);

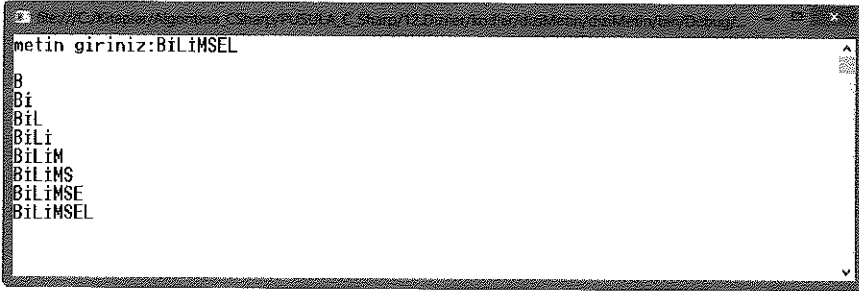
    //konsoldan aranacak anahtar kelimeyi alalım
    Console.Write("anahtar kelime:");
    anahtar = Console.ReadLine();
    anahtar = anahtar.Trim();//varsa boşluk karakterlerini silelim

    //anahtar kelimeyi arayalım, bulunursa indeksini değişkene atalım
    do
    {
        if (isimler[indeks] == anahtar)
        {
            sonuc = indeks + ". indekste!";
            break;
        }
        indeks++;
    } while (indeks < isimler.Length);
    Console.Write(sonuc);
    Console.ReadLine();//bekle
}
```

Bir String'in Karakterleri ile Çalışmak

Bu örneğimizde konsoldan bir **string** değer okuyacağız ve okuduğumuz değeri büyük harflere çevirdikten sonra konsola aşağıdaki gibi yazdıracağız.

Ekran Çıktısı:



Kodlar:

```

static void Main(string[] args)
{
    //konsoldan bir değer/metin okuyalım
    Console.Write("metin giriniz:");
    string metin = Console.ReadLine();

    //alınan metni büyük harfe çevirelim
    metin = metin.ToUpper();

    //metni ekrana yazdıralım
    for(int i=0; i<=metin.Length; i++)
        Console.WriteLine(metin.Substring(0,i));

    Console.ReadLine();//bekle
}
  
```

Substring(x,y) metodu **x.** karakterden **y.** karaktere kadar olan metni seçer. Bizim örneğimizde bu metot **metin.Substring(0,i)** şeklinde kullanılmaktadır. Dolayısıyla her döngüde metot parametreleri (0,0), (0,1), (0,2),... şeklinde değişmekte ve her döngüde artarak bir karakter fazla seçmektedir.

İki Boyutlu Dizinin elemanları

İki satır üç sütunluk bir dizi oluşturup konsoldan bu dizinin elemanlarını okuyacağız. Daha sonra da okunan değerleri konsola yazdıracağız.

```
static void Main(string[] args)
{
    //2 satır 3 sütun bir dizi tanımlayalım
    int[,] dizi = new int[2, 3];

    //konsoldan dizi değerlerini okuyalım
    for (int i = 0; i < 2; i++) //satırlar
    {
        for (int j = 0; j < 3; j++) //sütunlar
        {
            Console.Write("eleman i=" + i + " j=" + j + " :");
            dizi[i, j] = Convert.ToInt32(Console.ReadLine());
        }
    }

    //dizi elemanlarını yazalım
    for (int i = 0; i < 2; i++)
    {
        if (i % 1 == 0) Console.WriteLine("\n");
        for (int j = 0; j < 3; j++)
        {
            Console.Write("eleman" + i + j + " : " + dizi[i, j] + "\t");
        }
    }
    Console.ReadLine();
}
```

Ekran Çıktısı:

```
File:///C:/Kutuplar/Algoritma_CSharp/PUSSULA_C_Sharp/12 Diziler/kodlar/diz2Boy...
eleman i=0 j=0:1
eleman i=0 j=1:2
eleman i=0 j=2:3
eleman i=1 j=0:4
eleman i=1 j=1:5
eleman i=1 j=2:6

eleman00:1      eleman01:2      eleman02:3
eleman10:4      eleman11:5      eleman12:6
```

Alıştırmalar

- FB ve GS isimli 11 elemanlı iki string dizi oluşturun. Daha sonra her iki dizinin elemanları konsoldan alın. Fenerbahçe ve Galatasaray takımlarının ilk 11'ini öğrenin ve gerçek isimleri girin. Son olarak her iki dizinin elemanlarını yan yana yazdırın.
- 10 kişilik bir sınıfta okuyan öğrencilerin yılsonu notlarını konsoldan okuyun. Notların ortalamasını bulun ve konsola yazdırın.
- Yukarıdaki öğrencilerin en küçük ve en büyük notlarını bulup konsola yazdırın.
- Milli Piyango çekilişi için 0-9 arası rastgele 6 rakam gerekmektedir. Rastgele 6 adet rakam elde edin ve 6 elemanlı diziye aktarın. Son olarak da dizi elemanlarını konsola yazdırın.
- Bir zar 100 defa atılmaktadır. Hangi değerlerin kaç defa geldiğini dizi kullanarak hesaplayan bir program yazınız.
- 10 kişilik bir sınıfta her öğrencinin adını ve ders notunu okuyan iki boyutlu bir dizi oluşturun. Daha sonra da isim ve ders notlarını konsola yazdırın.
- Yukarıdaki öğrencilerin aldığı en büyük notu bulun ve öğrencinin adını konsola yazın.
- Üç satırlı ve iki sütunlu a ve b isimli iki matris oluşturun. Matrislerin değerlerini konsoldan okutun ve bu matrislerin çarpımını konsola yazdırın.

12

Sıkça Kullanılan Algoritmalar

- Sayılarla İlgili Algoritmalar
- Dizilerle İlgili Algoritmalar
- Arama ve Sıralama Algoritmaları

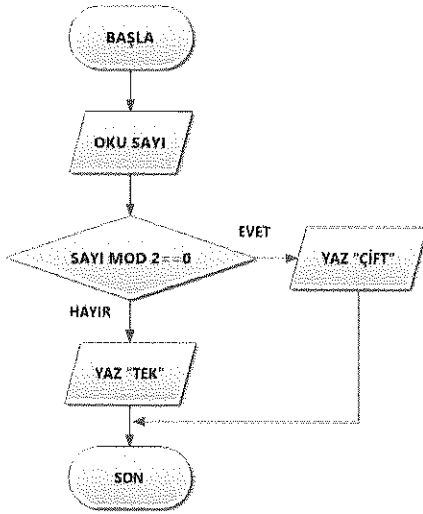
Bu bölümde karşımıza çok sık çıkan bazı algoritmaları öğreneceğiz ve C# ile kodlayacağız.

Çok sık kullanılan algoritmalar arasında arama, sıralama, rastgele sayı üretme ve sayısal problem algoritmaları bulunmaktadır. Bu algoritmaları öğrenirken hem yeni yeni teknikler öğreneceğiz hem de var olan bilgilerimizi pekiştireceğiz.

Bu bölümdeki algoritmalar daha önce anlatılan algoritmalardan biraz daha zordur. O nedenle daha fazla sabır ve daha fazla dikkat gerekmektedir. Bu nedenle mümkün olduğunca açık ve açıklamalı yazmaya çalıştım. Lütfen pür dikkat okuyunuz ve kodları deneyerek iyice anlamaya çalışınız.

Tek/Çift Sayı

Bir tamsayının tek veya çift olduğunu anlamanın en kolay yolu tamsayının 2'ye göre mod'unun alınmasıdır. Eğer bir tamsayı ikiye bölünüyorsa ve kalan sıfırsa yani **sayı mod 2 = 0** ise bu bir çift sayıdır. Aksi durumda tek sayıdır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int sayi;

    //konsoldan sayıyı okuyalım
    Console.Write("sayı giriniz:");
    sayi = Convert.ToInt32(Console.ReadLine());

    //sayının modunu alıp kontrol edelim
    if (sayi % 2 == 0)
        Console.Write("sayı çift");
    else
        Console.Write("sayı tek");

    Console.ReadLine();//pencereyi kapatma
}

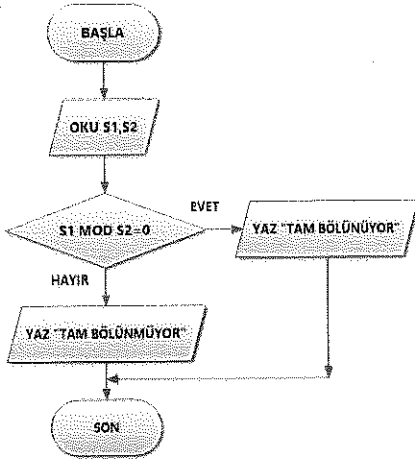
```

Tam Bölünebilme

Konsoldan iki sayı alacağız ve ilk sayının ikinci sayıya tam olarak bölünüp bölünmediğini kontrol edeceğiz. Bu örneğin anlaşılabilmesi için **mod** işleminin bilinmesi gerekmektedir. Hatırlanacağı gibi **mod** işlemi bize bölme sonucunda kalan değeri verir. Örneğin **5mod2=1** ederken **4mod2=0** eder. Yani eğer bir sayı başka bir sayıya tam bölünebiliyorsa **mod=0** olur. İşte biz de algoritmamızda bu durumu kontrol edeceğiz; eğer bir sayının başka bir sayıya bölümü **mod=0** ise tam bölünüyor demektir.

Örnek Mod İşlemleri:

- 3 mod 2=1
- 5 mod 3=2
- 8 mod 4=0
- 12 mod 8=4
- 21 mod 7 =0



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s1, s2;

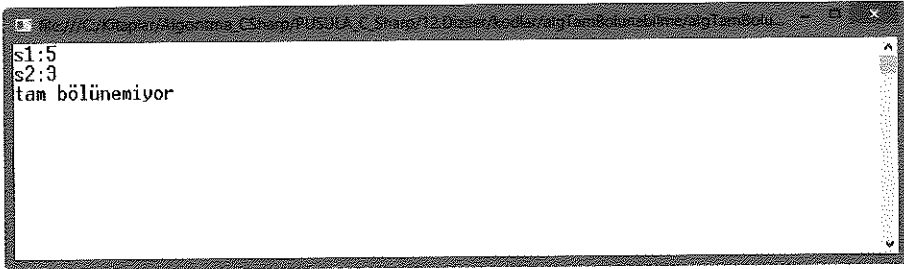
    //sayıları konsoldan okuyalım
    Console.Write("s1:");
    s1 = Convert.ToInt32(Console.ReadLine());
    Console.Write("s2:");
    s2 = Convert.ToInt32(Console.ReadLine());

    //bölünebilme durumunu kontrol edelim
    if (s1 % s2 == 0)
        Console.Write("tam bölünüyor");
    else
        Console.Write("tam bölünemiyor");

    //pencere kapanmasın
    Console.ReadLine();
}

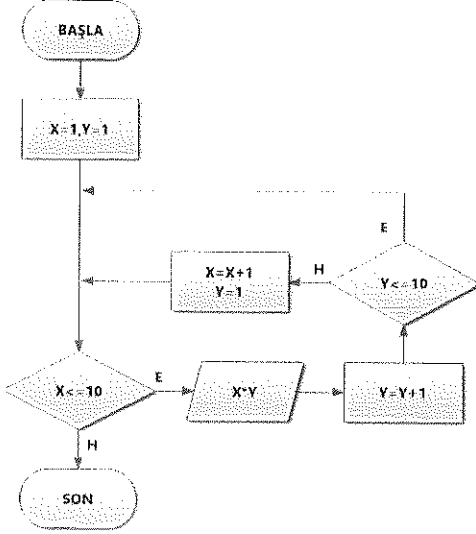
```

Ekran Çıktısı:



Çarpım Tablosu

Çarpım tablosunda önce $x=1$ yapılır ve 1'den 10'a kadar olan y değerleriyle çarpılır. Daha sonra $x=2$ yapılır ve 1'den 10'a kadar olan y değerleriyle çarpılır. $x=10$ olana kadar böyle devam eder. Bu işlemi gerçekleştirebilmek için iç içe iki for döngüsü kullanılır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y;

    //x ile y değerlerini çarpalım
    for (x = 1; x <= 10; x++) //x=1'den 10'a kadar
    {
        for (y = 1; y <= 10; y++) //her x için y=1'den 10'a kadar
            Console.WriteLine(x + "*" + y + "=" + x * y);
    }

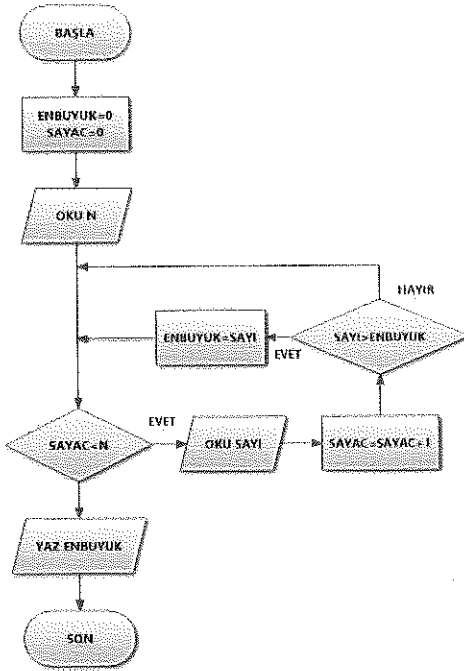
    //pencereyi kapatma
    Console.ReadLine();
}
  
```

En Büyük Sayı

Bu örneğimizde konsoldan girilen N adet sayının en büyüğünü bulup konsola yazdıracağız. Normalde bu tür problemler tipik olarak dizilerle çözülür fakat biz bu örneğimizde dizi kullanmayacağız.

Bu örnekte en büyük sayıyı saklayacak **enbuyuk** isimli bir değişken ve girilen sayıları sayacak bir **sayac** tanımlayacağız. Daha sonra kullanıcıdan N değerini yani okunacak sayı adetini alacağız. Ondan sonra da girilen her sayıyı daha önceki en büyük sayı ile karşılaştıracız. Eğer yeni girilen sayı önceki en büyük sayıdan büyükse yer değiştireceğiz. Yani burada aşağıdaki gibi bir kod yazmamız gerekiyor:

```
if(sayı>enbuyuk) //eğer girilen sayı büyükse
enbuyuk=sayı; //girilen sayıyı enbuyuk değişkenine ata
```



Akış diyagramını dikkatli bir şekilde incelenirse SAYAC<N sorgusundaki duruma göre akış diyagramı sağa dallanmaktadır. Ayrıca buradaki dalda SAYI>ENBUYUK sorgusu da bulunmaktadır. Sorgudan sonra da tekrar SAYAC<N sorgusuna gelinmektedir. Yani burada döngü içinde bir sorgu bulunmaktadır.

Program Kodları:

```
static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int sayi, enbuyuk = 0, sayac = 0, n;

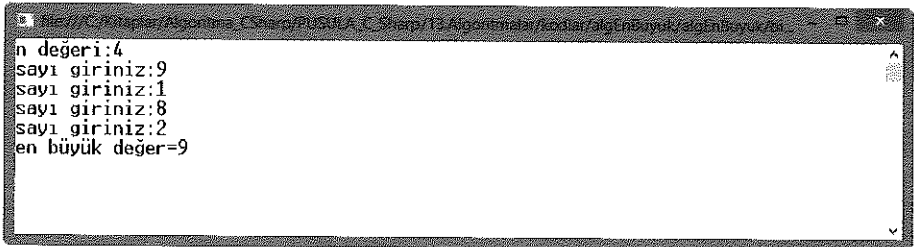
    //konsoldan n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

    //döngüyü oluşturalım
    while (sayac < n)
    {
        Console.Write("sayı giriniz:");
        sayi = Convert.ToInt32(Console.ReadLine());
        if (sayi > enbuyuk)
            enbuyuk = sayi;
        sayac++;
    }

    //en büyük değeri yazdıralım
    Console.Write("en büyük değer=" + enbuyuk);

    //pencereyi kapatma
    Console.ReadLine();
}
```

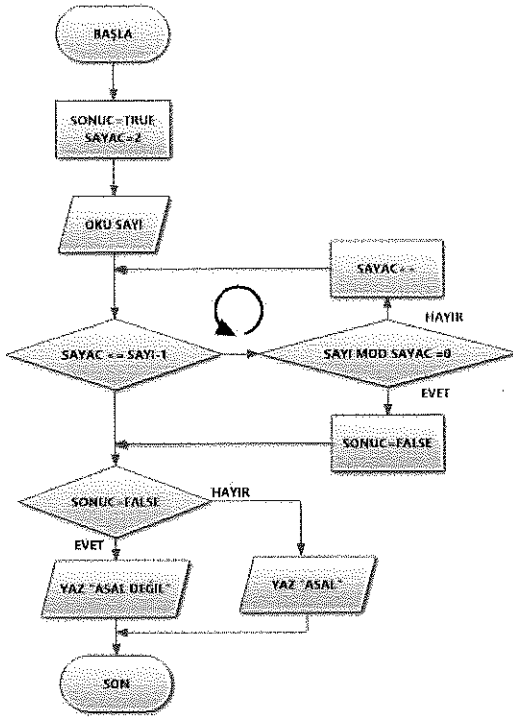
Ekran Çıktısı:



Asal Sayı

Asal sayılar 1'den ve kendisinden başka sayıya tam olarak bölünmeyen sayılardır. Örneğin 2, 3, 5, 7, 11, 13, 17 asal sayılardır.

Bir sayının asal olup olmadığını bulmak için o sayıyı **1** ve **kendisi** hariç, kendinden önce gelen sayılara bölümünü kontrol ederiz. Yani **2**'den **sayı-1** değerine kadar bir **sayac** kullanacağız. Eğer sayı kendinden önceki sayılardan birine tam olarak bölünüyorsa asal değildir.



Aşağıdaki kodları iyi anlarsanız programı da daha iyi anlarsınız. Lütfen açıklamalarla beraber inceleyiniz:

```

bool sonuc=true; //kontrol değişkeni. Sayı en başta asal sayılıyor
int sayi=5; //sayı şu an 5 fakat konsoldan da okunabilir
//konsoldan okunan sayi 2'den başlanarak kendisinden 1 küçük sayıya
//kadar mod işlemine tabi tutulur. Eğer mod=0 olursa bu sayı
//başka bir sayıya bölünüyor demektir ve sonuç=false olur. Yani sayı
//asal değildir. Aksi takdirde asaldır.
for(i=2; i<=sayi-1; i++)
{
    if(sayi % i==0)
        sonuc=false
}

```

Program Kodları:

```

static void Main(string[] args)
{
    //Önce değişkenleri tanımlayalım
    int sayi, sayac;
    bool sonuc = true; //sayı en başta asal kabul ediliyor

    //sayıyı okuyalım
    Console.Write("sayı:");
    sayi = Convert.ToInt32(Console.ReadLine());

    //2'den sayı-1'e kadar döngü ile sayıyı kontrol edelim
    for (sayac = 2; sayac <= sayi - 1; sayac++)
    {
        if (sayi % sayac == 0) //eğer kalan 0 ise
        {
            sonuc = false; //sayı asal değil
            break; //döngüden çık
        }
    }

    //sonuc değişkeninin durumunu kontrol edelim
    if(sonuc==true)
        Console.Write(sayi+" asaldır...");
    else
        Console.Write(sayi+ " asal değildir...");

    Console.ReadLine();//bekle
}

```

Rastgele Sayı & Sayısal Loto

Çok değişik amaçlar için rastgele sayılar gerekmektedir. Örneğin sayısal loto, tavla, çekiliş vb. oyunlarda rastgele veya x-y arası sayılara ihtiyaç duyulmaktadır. Örneğin sayısal lotoda 1-49 arası altı adet farklı sayı gerekirken tavlada 1-6 arası aynı olabilen iki sayı gerekmektedir.

Bu örneğimizde sayısal lotoyu ele alacağız. Sayısal lotoda 1-49 arası birbirinden farklı altı adet sayıya ihtiyacınız var.

C# dilinde rastgele sayı üretmek için **Random** sınıfı kullanılır. Aşağıda görüldüğü gibi **Random** sınıfından **ras** isimli bir nesne türetilir ve bu nesnenin metotları kullanılarak değişik aralıkta sayılar elde edilir.

```

Random ras=new Random(); //Random() sınıfından ras isimli nesne oluştur
int s1=ras.Next(); //pozitif her hangi bir tamsayı
int s2=ras.Next(10); //1-10 arası bir tamsayı
int s3=ras.Next(10,49); //10-49 arası tamsayı
Double s4=ras.NextDouble(); //0-1 arası ondalıklı sayı

```



```

static void Main(string[] args)
{
    //değişkenleri oluşturalım
    //elde edilen sayılar 6 elemanlı dizide saklanacak
    int[] sayisal = new int[6];

    //Random sınıfından ras isimli bir nesne oluşturalım
    Random ras = new Random();

    //1-49 arası 6 adet sayı oluşturalım
    for (int i = 0; i <= sayisal.Length-1; i++)
        sayisal[i] = ras.Next(1, 49);

    //istersek dizi elemanlarını sıralayabiliriz
    Array.Sort(sayisal);

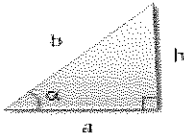
    //oluşturulan sayıları yani dizi elemanlarını yazdıralım
    for (int j = 0; j<=sayisal.Length-1; j++)
        Console.WriteLine(j + 1+" sayı: " + sayisal[j]);

    //bekle
    Console.ReadLine();
}

```

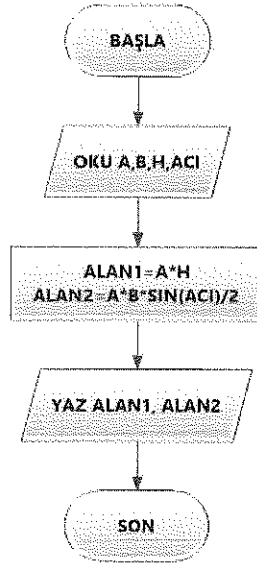
Not: Eğer üretilen 6 rastgele sayı içinde aynı sayılar varsa bunların nasıl farklı yapı-
lacağını araştırınız. Nereden mi? Google ☺

Üçgen Alanı



Bazı bilgiler:

- C#'ta derece cinsinden verilen açılar radyana çevrilmelidir. Bunun için de $\text{rad} = \text{açı} \cdot \pi / 180$ formülü yapılmalıdır.
- Eğer üçgenin tabanı ve yüksekliği verildiyse alan $a \cdot h$ formülüyle bulunur.
- Eğer üçgenin kenar uzunlukları ve açısı verildiyse alan hesabı için $a \cdot b \cdot \sin(\alpha) / 2$ formülü kullanılır.



Verilen parametrelere göre üçgen alanının hesaplanmasında aşağıdaki kodlar kullanılabilir:

```

static void Main(string[] args)
{
    //gereken değişkenleri tanımlayalım
    double a, b, h, alan1, alan2, aci;

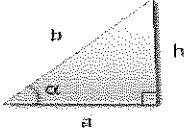
    //konsoldan değerleri okuyalım
    Console.Write("a açısı:");
    aci = Convert.ToInt32(Console.ReadLine());
    Console.Write("a uzunluğu:");
    a = Convert.ToDouble(Console.ReadLine());
    Console.Write("b uzunluğu:");
    b = Convert.ToDouble(Console.ReadLine());
    Console.Write("h uzunluğu:");
    h = Convert.ToDouble(Console.ReadLine());

    //hesaplamaları yapalım
    alan1 = a * h / 2;
    alan2 = a * b * Math.Sin(acı * Math.PI / 180) / 2;

    //Sonuçları yazdıralım
    Console.WriteLine("Alan1=" + alan1 + " Alan2=" + alan2);

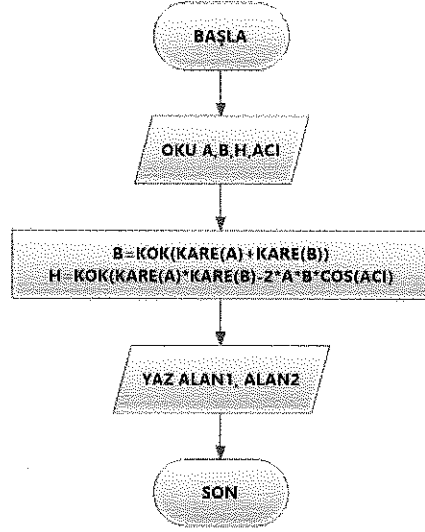
    Console.ReadLine();
}
  
```

Üçgen Kenar Uzunlukları



Bazı bilgiler:

- C#'ta derece cinsinden verilen açılar radyana çevrilmelidir. Bunun için de $\text{açı} \cdot \pi / 180$ işlemi yapılmalıdır.
- Hipotenüs uzunluğu $b = \sqrt{a^2 + h^2}$ formülüyle bulunur.
- Eğer a ve b uzunlukları ile α açısı verildiyse ve h uzunluğu isteniyorsa $h^2 = a^2 + b^2 - 2 \cdot a \cdot b \cdot \cos(\alpha)$ formülü kullanılır.



```

static void Main(string[] args)
{
    //gerekken değişkenleri tanımlayalım
    double a, b, hipo, h, aci;

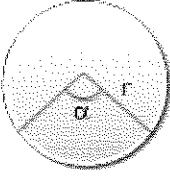
    //konsoldan değerleri okuyalım
    Console.WriteLine("a açısı:");
    aci = Convert.ToInt32(Console.ReadLine());
    Console.WriteLine("a uzunluğu:");
    a = Convert.ToDouble(Console.ReadLine());
    Console.WriteLine("b uzunluğu:");
    b = Convert.ToDouble(Console.ReadLine());

    //hesaplamaları yapalım
    hipo = Math.Sqrt(a * a + b * b);
    h = Math.Sqrt(a * a + b * b - 2 * a * b * Math.Cos(aci * Math.PI / 180));

    //Sonuçları yazdıralım
    Console.WriteLine("Hipotenüs=" + hipo + " h=" + h);

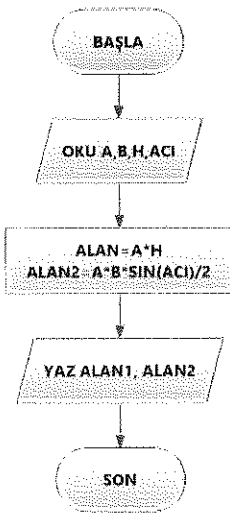
    Console.ReadLine();
}
  
```

Daire İşlemleri



Bazı bilgiler:

- Daire alanı $\pi * r * r$ formülüyle hesaplanır.
- Belli bir α açısı verilen daire diliminin alanı ise $\pi * r * r * \alpha / 360$ formülü ile hesaplanır



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double r, a, dilimalan, alan;
    const double pi = Math.PI;

    //konsoldan verileri alalım
    Console.Write("r değeri:");
    r = Convert.ToDouble(Console.ReadLine());

    Console.Write("açı değeri:");
    a = Convert.ToDouble(Console.ReadLine());

    //hesapları yapalım
    alan = pi * r * r;
    dilimalan = pi * r * r * a / 360;

    //sonuçları yazalım
    Console.Write("alan=" + alan + " dilim alan=" + dilimalan);

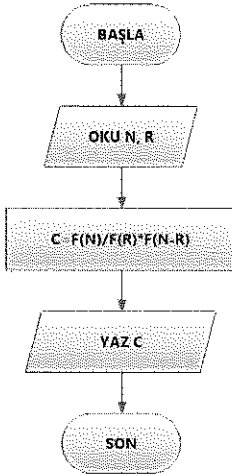
    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Kombinasyon

N elemanlı bir kümenin r'li kombinasyonunu hesaplamak için aşağıdaki formül kullanılır.

$$C(n,r)=n!/r!(n-r)!$$

Dikkat edilirse burada bir kaç tane faktöriyel işlemi vardır. Burada yapılacak en güzel iş bir faktöriyel metodu yazmak ve gerektiğinde onu kullanmaktır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n, r; double c;

    //konsoldan değerleri okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());
    Console.Write("r değeri:");
    r = Convert.ToInt32(Console.ReadLine());

    //işlemi yapalım
    c = f(n) / (f(r) * f(n-r));

    //sonucu yazdıralım
    Console.WriteLine("Kombinasyon=" + c);
    Console.ReadLine();//pencereyi kapatma
}

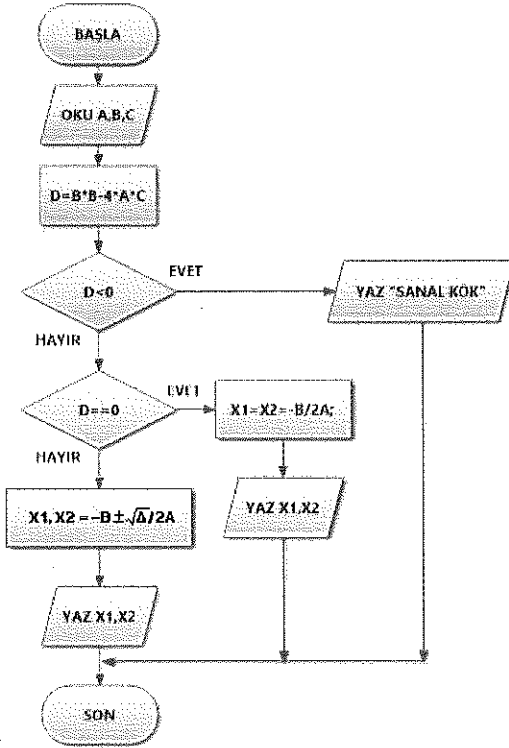
static int f(int x)
{
    int fk = 1;
    for (int i = 1; i <= x; i++)
        fk = fk * i;

    //hesaplanan faktöriyeli ana programa gönder/döndür
    return fk;
}
  
```

İkinci Dereceden Denklem Kökleri

Bu örneğimizde $ax^2+bx+c=0$ şeklinde olan ikinci derece bir denklemin köklerini bulacağız. Hatırlanacağı gibi, ikinci derece denklemlerin kök durumu diskriminanta (Δ) göre değişmektedir. Eğer $\Delta < 0$ ise sanal kök, $\Delta = 0$ ise tek kök ve $\Delta > 0$ ise çift kök vardır. Yani:

- $\Delta = b^2 - 4ac$
- $\Delta = 0$ ise $x_1 = x_2 = -b/2a$
- $\Delta > 0$ ise $x_{1,2} = -b \pm \sqrt{\Delta}/2a$



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    double a, b, c, x1, x2, d;

    //konsoldan değerleri alalım
    Console.Write("a değeri:");
    a = Convert.ToDouble(Console.ReadLine());
    Console.Write("b değeri:");
    b = Convert.ToDouble(Console.ReadLine());
    Console.Write("c değeri:");
    c = Convert.ToDouble(Console.ReadLine());

    //diskriminantı hesaplayalım
    d = b * b - 4 * a * c;

    //diskriminant durumuna göre işlem yapalım
    if (d < 0)
        Console.WriteLine("Sanal Kök Var");
    else if (d == 0)
    {
        x1 = -b / 2 * a;
        Console.WriteLine("Tek Kök Var:" + x1);
    }
    else
    {
        x1 = -b - Math.Sqrt(d) / 2 * a;
        x2 = -b + Math.Sqrt(d) / 2 * a;
        Console.WriteLine("Çift Kök Var. X1=" + x1 + " X2=" + x2);
    }
    Console.ReadLine();//bekle
}

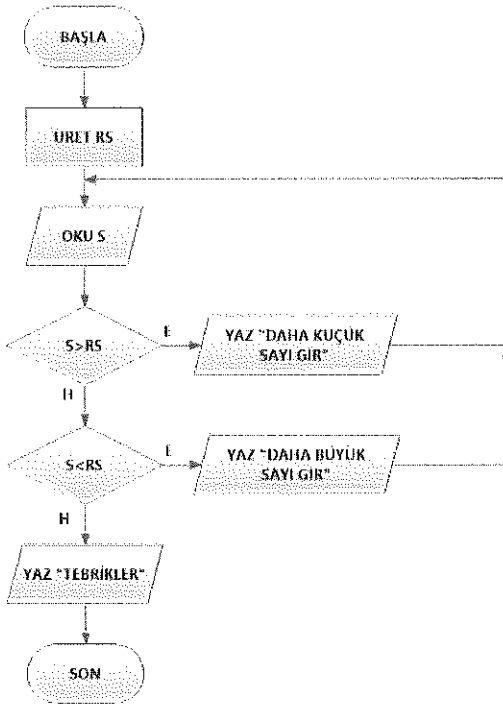
```

Sayı Tahmin Programı

Rastgele üretilen 1-10 arası bir sayıyı bulmaya çalışacağız. Eğer tahmin edilen değer küçükse "daha büyük sayı giriniz" yazılacak. Eğer rastgele sayıdan daha büyük bir değer girildiyse "daha küçük sayı giriniz" yazılacak. Rastgele olarak oluşturulan sayı bulunduyorsa "tebrikler!" yazılacak.

Rastgele sayı üretmek için **Random()** sınıfı kullanılmaktadır. **Random()** sınıfından **ras** isimli nesne türetilmekte ve **ras.Next(1,10)** komutuyla elde edilen değer **rs** değişkenine aktarılmaktadır.

Kodlar dikkatli bir şekilde incelenirse **do-while()** döngüsü içinde iki adet sorgu olduğu fark edilecektir. **while(s != rs)** ifadesinden de anlaşıldığı gibi girilen sayı rastgele sayıya eşit değilse döngü çalışmaya ve yeni sayı almaya devam eder. **s=rs** olduğunda döngü sona erer ve **tebrikler!** yazdırılır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s, rs;

    //rasgele sayı üretelim
    Random ras = new Random();
    rs = ras.Next(1, 10);

    //rs sayısını kontrol edelim
    do
    {
        Console.Write("sayı giriniz:");
        s = Convert.ToInt32(Console.ReadLine());
        if (s > rs) Console.WriteLine("daha küçük sayı giriniz!");
        if (s < rs) Console.WriteLine("daha büyük sayı giriniz!");
    } while (s != rs);

    //doğru tahmin
    Console.Write("tebrikler! " + s + " doğru tahmin...");

    //bekle
    Console.ReadLine();
}

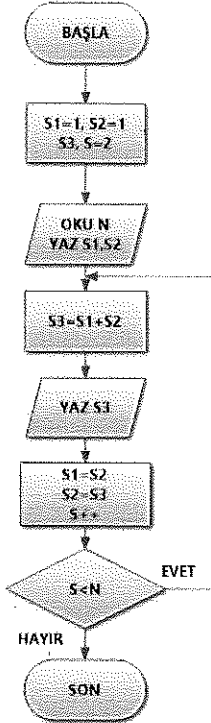
```


Fibonacci Sayıları

Fibonacci sayıları 0, 1, 1 şeklinde başlar ve her eleman önceki iki sayının toplamı olarak devam eder. Yani şu şekildedir:

0, 1, 1(0+1), 2(1+1), 3(1+2), 5(2+3), 8(3+5), 13(5+8), 21(8+13), ...

Burada ilk sayı yani $s1=1$, ikinci sayı da yani $s2=1$ değerini taşır. Diğer sayılar yani $s3$, $s1+s2$ şeklindedir. Bu durumda Fibonacci dizisinin ilk N terimini yazdıran algoritmamız aşağıdaki gibi olur:



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int s1 = 1, s2 = 1, s3, s = 2, n;

    //n değerini okuyalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

    //ilk iki sayıyı yazdıralım
    Console.WriteLine(s1);
    Console.WriteLine(s2);

    //s3 hesaplanıp yazdırılacak
    //sayacı n değerinden küçük eşit
    //olduğu sürece aşağıdaki işlem yapılacaktır
    while (s <= n)
    {
        s3 = s1 + s2; //sonraki değeri hesapla
        Console.WriteLine(s3); //değeri yaz
        s1 = s2; //yeni s1 değeri s2
        s2 = s3; //yeni s2 değeri s3
        s++; //sayacı arttır
    }
    //bekle
    Console.ReadLine();
}
  
```

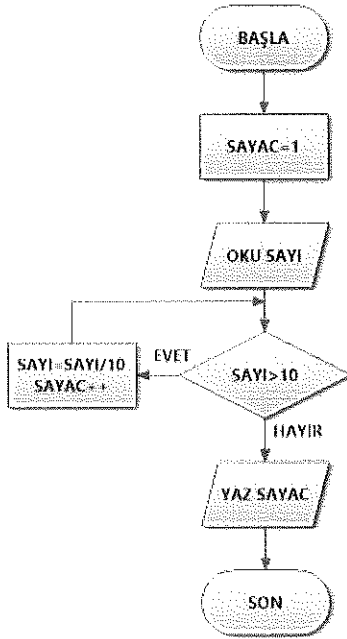
Sayının Basamakları

Girilen bir sayının kaç basamaklı olduğunu anlamak için sayı 10'dan büyük olduğu sürece sürekli 10'a bölünür. Her bölme işleminde de sayacı artırılır. Sayacı sayının basamak sayısını verir.

Örneğin:

333	Sayac=1
333/10=33	Sayac=2
33/10=3	Sayac=3 //kalan 3'den küçük olunca işlem biter

"Sayı 10'dan büyük olduğu sürece" ifadesinin C# dilindeki karşılığı **while(sayı>10)** komutudur. Sayı 10'dan büyük olduğu sürece 10'a bölünecek ve sayaç 1 arttırılacaktır.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int sayi, s = 1;

    //konsoldan sayıyı okuyalım
    Console.Write("sayı giriniz:");
    sayi = Convert.ToInt32(Console.ReadLine());

    //alınan sayıyı 10'dan büyük olduğu
    //sürece 10'a bölelim
    while (sayi > 10) //sayı 10'dan büyükse
    {
        sayi=sayi/10; //sayıyı 10'a böl
        s++; //sayacı arttır
    }

    //sayacı yani basamak sayısını yazalım
    Console.Write("sayının basamak sayısı=" + s);

    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Çarpmanın Toplama ile Yapılması

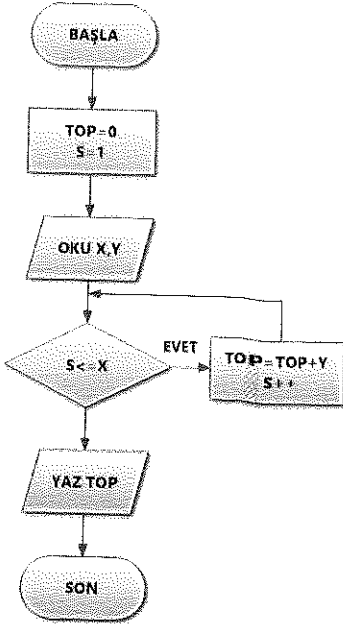
Klavyeden okunan iki sayının çarpımını toplama işlemi ile yapacağız. Bu işlem çok önemli çünkü bazı mikro kontrolörlerde çarpma komutu yoktur ve çarpma işlemi toplama ile yapmak zorunda kalırsınız.

Çarpma işlemi toplama ile yapmak için yani x sayısını y ile toplayarak çarpmak için x sayısı y defa toplanır veya y sayısı x defa toplanır.

$$3 \cdot 4 = 12$$

$$3 + 3 + 3 + 3 = 12$$

$$4 + 4 + 4 = 12$$



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int x, y, s, top = 0;

    //x ve y değerlerini okuyalım
    Console.Write("x değeri:");
    x = Convert.ToInt32(Console.ReadLine());
    Console.Write("y değeri:");
    y = Convert.ToInt32(Console.ReadLine());

    //çarpmayı hesaplayalım
    for (s = 1; s <= x; s++)
        top = top + y;

    //işlem sonucunu yaz
    Console.Write(x + "*" + y + "=" + top);

    //pencereyi kapatma
    Console.ReadLine();
}

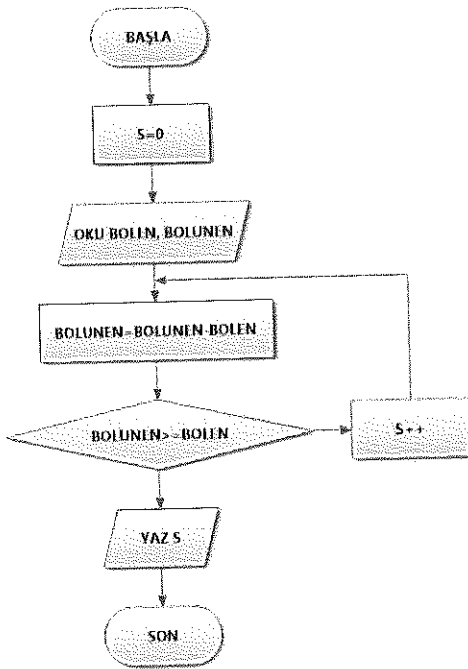
```

Bölmenin Çıkarma ile Yapılması

Çıkarma ile bölme yapmak için bölünen sayıdan bölen sayıyı (bölen sayı büyük olana kadar) çıkartırız. Her çıkarma işleminde sayacı 1 arttırırız. Elde edilen sayaç değeri bölme sonucunu verir. Yani:

$$15/3=5$$

Bölünen	bölen	sayaç
15	-3	0
12	-3	1
9	-3	2
6	-3	3
3	-3	4
0	-3	5 // bölünen (0) bölenden (3) küçük ise işlem biter



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int bolen, bolunen, s=0;

    //değerleri konsoldan okuyalım
    Console.Write("bölünen sayı:");
    bolunen = Convert.ToInt32(Console.ReadLine());
    Console.Write("bölen sayı:");
    bolen = Convert.ToInt32(Console.ReadLine());

    //ilk bölünen sayı(yazdırırken kullanılacak)
    int ilkbolunen = bolunen;

    //işlemi yapalım
    while (bolunen >= bolen)
    {
        bolunen = bolunen - bolen;
        s++;
    }

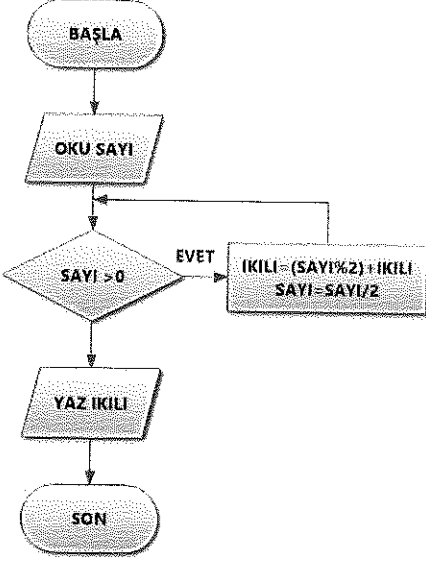
    //işlem sonucunu yazdıralım
    Console.WriteLine(ilkbolunen + "/" + bolen + "=" + s);

    //pencereyi kapatma
    Console.ReadLine();
}

```

Onluk Sayının İkili Sayıya Çevrilmesi

Onluk sayıları ikilik sayıya çevirmek için sayıyı (2'den küçük olana kadar) sürekli ikiye böler ve kalanları kullanırız. Bu örnekte kalanları **ikili** isimli bir **string** değişkeninde toplayacağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int sayi;
    string ikili = "";

    //konsoldan sayıyı okuyalım
    Console.Write("sayı:");
    sayi = Convert.ToInt32(Console.ReadLine());

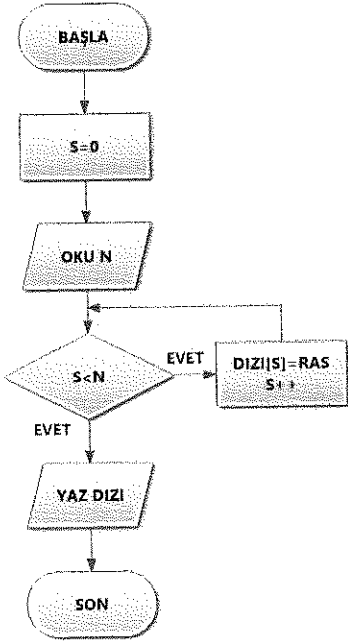
    //sayı sıfırdan büyük olduğu sürece
    //sayının ikiye bölümünden kalanını
    //ikili değişkenine ekleyelim
    //ve sayıyı da ikiye bölelim
    while (sayi > 0)
    {
        ikili = (sayi % 2) + ikili;
        sayi = sayi / 2;
    }

    //sonucu konsola yazdıralım
    Console.Write(ikili);

    Console.ReadLine();//bekle
}
  
```

Rastgele Sayı Dizisi

Random() sınıfını kullanarak **N** adet rastgele sayı oluşturacağız ve bu sayıları **rastgele** isimli **int** tipindeki diziye atayacağız. Son olarak da dizinin elemanlarını konsola yazdıracağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int n=0;

    //konsoldan n değerini alalım
    Console.Write("n değeri:");
    n = Convert.ToInt32(Console.ReadLine());

    //diziyi tanımlayalım
    int[] rasgele=new int[n];

    //Random() sınıfıyla rasgele sayı üretip
    //diziye atayalım
    Random ras = new Random();
    for(int i=0; i<n;i++)
    {
        //1-100 arası rasgele sayı
        rasgele[i]= ras.Next(1,100);
    }

    //dizinin elemanlarını yazdıralım
    foreach (int eleman in rasgele)
        Console.WriteLine(eleman);

    Console.ReadLine();//bekle
}
  
```

Dizi Elemanlarının Toplamı ve Ortalaması

Klavyeden girilen **n** elemanlı bir dizinin ortalamasını bulacağız. Öncelikle **n** değerini okumamız gerekiyor. Daha sonra bir döngü ile tüm dizi elemanlarını okuyacağız ve toplayıp ortalamasını bulacağız.

Programdaki döngüyü **while** veya **for** komutu kullanarak oluşturabiliriz. **for** komutunda şart ve artış parantez içinde ve tek satır olduğu için daha derli toplu görünüyor. Ama istenirse **while** komutu da aşağıdaki gibi kullanılabilir.

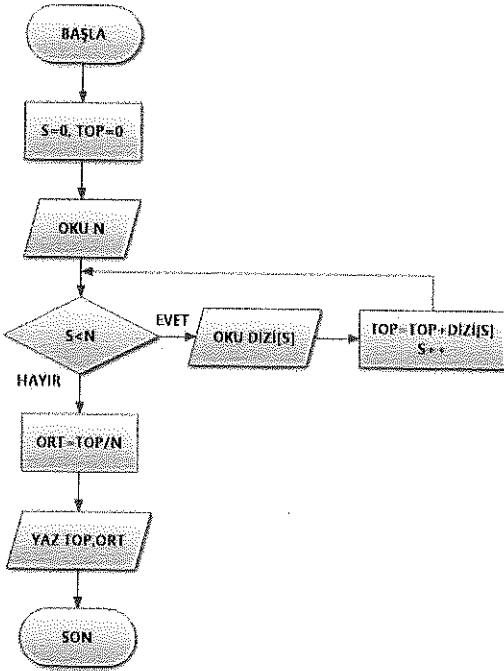
```

int s=0;
while(s<n)
{
    Console.Write("dizinin " + s + " indisli elemanı:");
    dizi[s] = Convert.ToInt32(Console.ReadLine());
}
  
```

```

top = top + dizi[s];
s++;
}

```



```

static void Main(string[] args)
{
    //değişkenlerini tanımlayalım
    int s, top = 0, ort=0, n;

    //n değerini okuyalım
    Console.Write("n değeri:");
    n=Convert.ToInt32(Console.ReadLine());

    //diziyi tanımlayalım
    int[] dizi = new int[n];

    //dizi elemanlarını alalım
    for (s = 0; s < n; s++)
    {
        Console.Write("dizinin " + s + " indisli elemanı:");
        dizi[s] = Convert.ToInt32(Console.ReadLine());
        top = top + dizi[s];
    }

    //ortalama
    ort = top / n;

    //toplam ve ortalamayı yazdıralım
    Console.Write("toplam=" + top + " ortalama=" + top / n);

    Console.ReadLine(); //bekle
}

```

Program Çıktısı:

```

n değeri:5
dizinin 0 indisli elemanı:1
dizinin 1 indisli elemanı:2
dizinin 2 indisli elemanı:3
dizinin 3 indisli elemanı:4
dizinin 4 indisli elemanı:5
toplam=15 ortalama=3

```

Dizilerde Matematiksel İşlemler

Programda 3 adet dizi tanımlayacağız. a[] ve b[] dizilerinde değerler olurken c dizisinde a ve b dizisinin elemanlarının toplamı olacak. Toplam yerine çarpma, çıkarma vb. matematiksel işlemler de yapılabilir.

Dizi elemanları konsoldan da alınabilirdi fakat bu örneğimizde diziyi ve elemanlarını kodlarla belirleyeceğiz.

```

static void Main(string[] args)
{
    //dizileri tanımlayalım
    int[] a = { 1, 2, 3, 4, 5 };
    int[] b = { 6, 7, 8, 9, 10 };
    int[] c = new int[5];

    //toplama işlemini yapalım
    for (int i = 0; i < 5; i++)
        c[i] = a[i] + b[i];

    //c dizisinin elemanlarını yazdıralım
    foreach (int eleman in c)
        Console.WriteLine(eleman);

    //bekle
    Console.ReadLine();
}

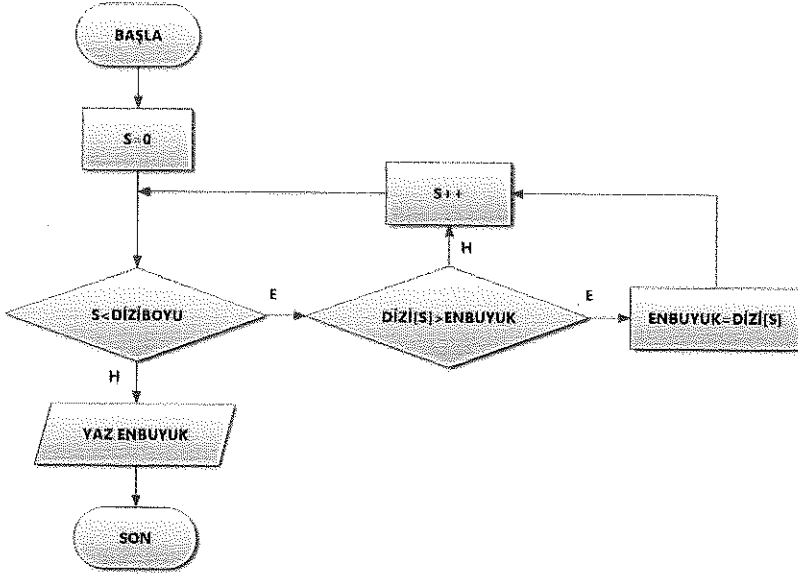
```

Dizinin En Büyük veya En Küçük Elemanı

Dizilerde en büyük veya en küçük eleman problemi sık sık karşımıza çıkmaktadır. İkisi de benzer mantıkla çalışır. Genel mantık **enbuyuk** veya **enkucuk** şeklinde bir değişken tanımlayıp dizinin tüm elemanları ile karşılaştırmak ve eğer dizi elemanı daha büyük veya daha küçükse değiştirmeye atmak.

Bu örneğimizde bir dizideki en büyük sayıyı bulacağız. Eğer isterseniz küçük bir değişiklik yaparak en küçük sayıyı da bulabilirsiniz. Eğer aşağıdaki algorithmada görülen "ENBÜYÜK" yerine "ENKÜÇÜK" koyarsanız ve $DİZİ[S] > ENBÜYÜK$ ifadesini $DİZİ[S] < ENKÜÇÜK$ ile değiştirirseniz dizideki en küçük sayıyı bulabilirsiniz.

En küçük sayı algoritmasını kendiniz çizin ve gerekli değişiklikleri yaparak en küçük sayı programını da siz yazınız.



Eğer aşağıdaki programdaki kodları incellerseniz hem en büyük hem de en küçük sayının bulunmasını sağlayan kodları görebilirsiniz. En büyük sayının bulunması için **enbuyuk** değişkenine başlangıç değeri olarak sıfır, **enkucuk** değişkenine ise 1000000 değeri atanmıştır.

```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int enbuyuk=0;
    int[] dizi = { 1, 9, 2, 7, 3, 6, 4, 5 };

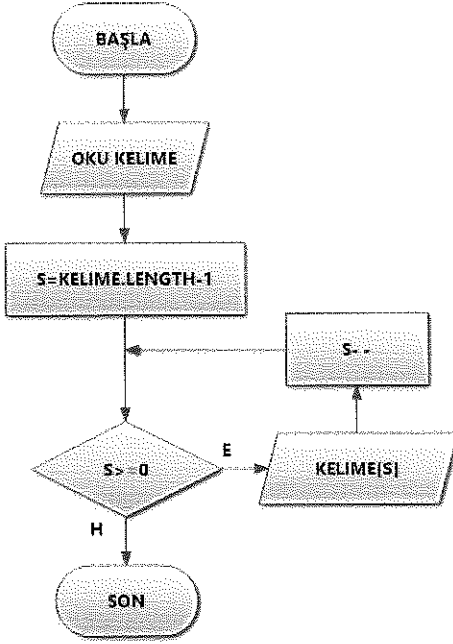
    //for döngüsüyle dizinin tüm elemanlarını
    //enbuyuk değişkeni ile karşılaştıralım.
    //eğer dizi elemanı daha büyükse enbuyuk değişkenine atayalım
    for (int s = 0; s < dizi.Length - 1; s++)
    {
        if (dizi[s] > enbuyuk)
            enbuyuk = dizi[s];
    }
    //enbuyuk değişkenini/sayıyı yazdıralım
    Console.WriteLine("dizideki en büyük sayı=" + enbuyuk);

    //dizinin en küçük elemanını bulmak için
    int enkucuk=1000000;
    for (int s = 0; s < dizi.Length - 1; s++)
    {
        if (dizi[s] < enkucuk)
            enkucuk = dizi[s];
    }
    //enkucuk değişkenini/sayıyı yazdıralım
    Console.WriteLine("dizideki en küçük sayı=" + enkucuk);
    Console.Read(); //bekle
}

```

Girilen Kelimenin Tersten Yazılması

Bu algoritmada **string** veri tipinin dizi özelliğini kullanacağız ve dizinin elemanlarını sondan başa doğru yazdıracağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    string kelime = "";
    int s;

    //kelimeyi konsoldan okuyalım
    Console.Write("kelime:");
    kelime = Console.ReadLine();

    //kelime uzunluğu-1 (dizi indeksi)
    s = kelime.Length-1;

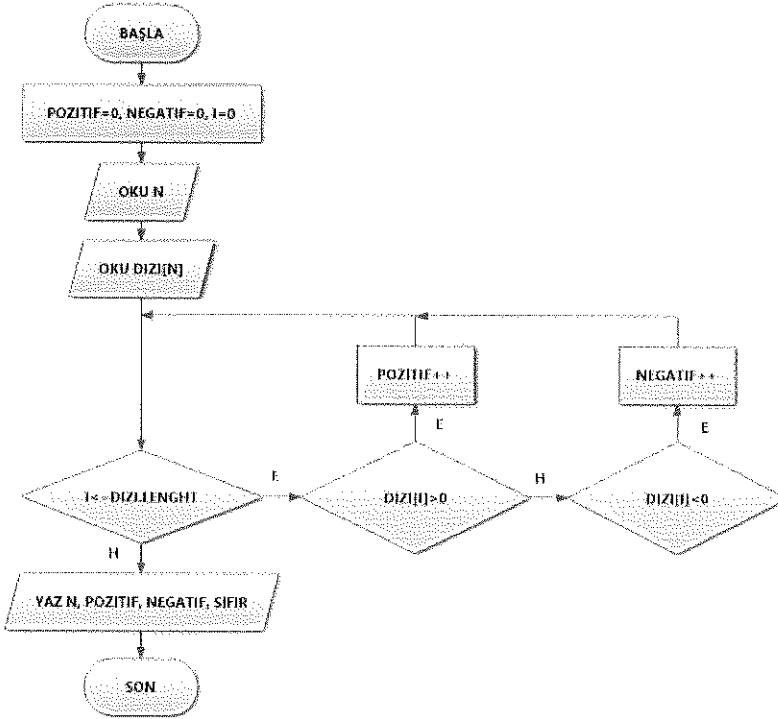
    while (s >= 0)
    {
        Console.Write(kelime[s]);
        s--;
    }

    //pencereyi kapatma
    Console.ReadLine();
}
  
```

Dizideki Pozitif ve Negatif Elemanlar

Bu örneğimizde **N** elemanlı bir dizi oluşturacağız. Daha sonra dizinin her bir elemanını sıfır (0) ile karşılaştıracız. Eğer dizinin elemanı sıfırdan büyükse **pozitif** sayacını, sıfırdan küçükse **negatif** sayacını arttıracız. Sıfırlar pozitif veya negatif sayı olmadığı için sıfırları saymayacağız.

Algoritmanın en sonunda girilen eleman sayısını, pozitif eleman sayısını, negatif eleman sayısını ve sıfır sayısını yazdıracağız.



```

static void Main(string[] args)
{
    //değişkenleri tanımlayalım
    int pozitif = 0, negatif = 0, n;

    //dizinin eleman sayısını okuyalım
    Console.Write("eleman sayısı:");
    n = Convert.ToInt32(Console.ReadLine());

    //diziyi oluşturacağız ve elemanlarını okuyalım
    int[] dizi=new int[n];
    for(int s=0; s<n; s++)
    {
        Console.Write("dizinin " + (s + 1) + ". elemanı:");
        dizi[s]=Convert.ToInt32(Console.ReadLine());
    }

    //dizinin elemanlarını kontrol edelim
    for (int i = 0; i <= dizi.Length - 1; i++)
    {
        if (dizi[i] > 0) pozitif++;
        if(dizi[i]<0) negatif++;
    }
    //eleman sayılarını yazdıralım
    Console.Write("eleman sayısı:" + n+ "\n pozitif:"+pozitif+"\n negatif:"+negatif);
    Console.WriteLine("sıfır sayısı:" + (n - pozitif - negatif));

    Console.ReadLine(); //bekle
}

```

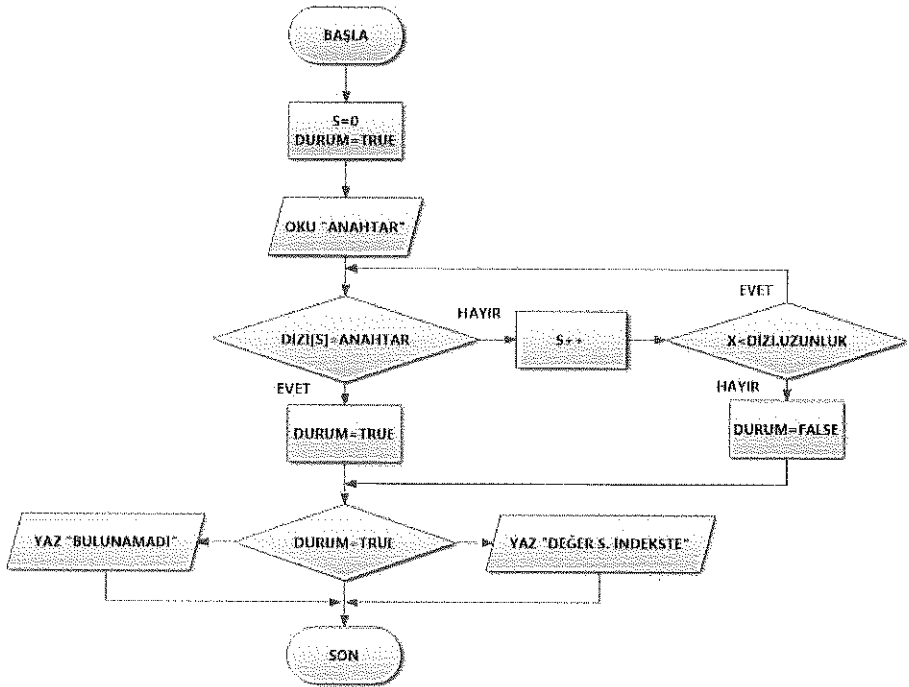
Dizide Doğrusal Arama Yapmak

Arama yapmak bilgi işlemde en çok kullanılan işlemlerden biridir. Örneğin internette arama yaparız, bilgisayarda dosya araması yaparız veya bir okul programında öğrenci araması yaparız.

Arama algoritmaları hakkında daha geniş bilgi için internette http://tr.wikipedia.org/wiki/Arama_algoritmaları adresini ziyaret edebilirsiniz.

Ben burada sizlere özellikle küçük dizilerde arama yapmak için kullanılan, diğer arama algoritmalarına göre nispeten daha kolay olan **Doğrusal Arama** algoritmasından bahsedeceğim. Bu yöntemde aranan değer anahtar kelime dizinin ilk elemanından başlanarak tüm elemanlarıyla karşılaştırılır. Eğer eşleşme bulunursa arama işlemi sonlandırılır.

Akış diyagramı incelenirse **durum** isimli bir değişken kullanıldığı görülür. Bu değişkenin başlangıç değeri **true**'dur; her döngüde dizinin her **s.** elemanı kontrol edilir; eğer aranan değer bulunursa **durum true**, bulunamazsa **false** olur. En son aşamada **durum** değişkeni kontrol edilir ve değerine göre işlem yapılır.



```

static void Main(string[] args)
{
    //değişkenlerimizi tanımlayalım
    bool durum = true; //elemanın bulundu/bulunmadı durumu
    int s = 0, anahtar;
    int[] dizi = { 2, 3, 5, 7, 11, 13, 17 };

    //anahtarı yani aranacak değeri alalım
    Console.Write("anahtar:");
    anahtar = Convert.ToInt32(Console.ReadLine());

    //dizi elemanlarını arayalım
    for (; s < dizi.Length; s++)
    {
        if (dizi[s] == anahtar) //eğer anahtar bulunduysa
        {
            durum = true; break; //döngüden çık
        }
        else
            durum = false;
    }
    //sonucu konsola yazalım
    if (durum == true)
        Console.Write("aranan değer " + s + ". indekste bulundu");
    else
        Console.Write("aranan değer bulunamadı");

    Console.Read(); //bekle
}

```

Sayı Sıralama

Sıralama yapmak algoritmadaki en önemli ve en çok kullanılan konulardan biridir çünkü iş veya gündelik hayatımızda sürekli sıralama yaparız; A'dan Z'ye göre, büyükten küçüğe göre, tarihe göre, yaşa göre vb.

Sıralamayla ilgili 17 farklı algoritma vardır. Bu algoritmaların kimisi zorken kimisi kolaydır, kimisi hızlı çalışırken kimisi yavaş çalışır. Doğrusunu söylemek gerekirse elinizdeki soruna göre farklı algoritmaları kullanmak zorunda kalabilirsiniz.

Ben size en çok kullanılan ve en basit sıralama algoritması olan **Seçme Sıralamasını** anlatacağım. Eğer tüm sıralama algoritmalarını daha yakından incelemek isterseniz http://tr.wikipedia.org/wiki/Sıralama_algoritması adresini ziyaret edebilirsiniz.

Seçme Sıralama Algoritmasının İşleyişi

Bu algorithmada dizinin ilk elemanı dizinin sonuna kadar olan diğer elemanlarla karşılaştırılır. Eğer x . eleman $x+1$. elemandan (sağıdakinden) büyükse yer değiştirilir. İçeriği "5 1 4 2 8" olan bir dizi en küçükten en büyüğe doğru aşağıdaki biçimde sıralanır. Her adımda dizinin **kalın** olarak işaretlenmiş elemanları karşılaştırılan elemanlardır.

Birinci Döngü:

- (5 1 4 2 8) $\rightarrow$ (1 5 4 2 8) ilk iki eleman karşılaştırılır ve yerleri değiştirilir.
- (1 5 4 2 8) $\rightarrow$ (1 4 5 2 8)
- (1 4 5 2 8) $\rightarrow$ (1 4 2 5 8)
- (1 4 2 5 8) $\rightarrow$ (1 4 2 5 8) elemanlar sıralı olduğu için yerleri değişmez.

İkinci Döngü:

- (1 4 2 5 8) $\rightarrow$ (1 2 4 5 8)
- (1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)
- (1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

Sonuç olarak dizi sıralanmıştır ve algoritma sonlanır. Burada dizimiz 5 elemanlı olduğu için 1. eleman 2., 3., 4. ve 5. elemanlarla karşılaştırıldı. Sonraki döngüde ikinci eleman 3., 4. ve 5. elemanlarla karşılaştırılacak. Yani, n elemanlı bir dizi için birinci döngü $x=1$ ile $n-1$ arası, ikinci döngü de $x=2$ ile n arası çalışır.

Sıralama algoritmasında daha önce anlatılan yer değiştirme mantığı kullanılır. Yani dizide yan yana iki sayı yer değiştirirken **yardımcı** isimli bir değişken kullanılır. Sonuçta aşağıdaki gibi bir kod ortaya çıkar:

```
for(x=0; x<n-1; x++)
{
    for(y=x+1; j<n; y++)
    {
        if(dizi[x]>dizi[y]) //soldaki eleman büyükse
            yordimci=dizi[x]; //yer deęiřtirme uygula
        dizi[x]=dizi[y];
        dizi[y]=yordimci;
    }
}
```

Programın kodları ařaęıdaki gibi olmalıdır:

```
static void Main(string[] args)
{
    //deęiřkenleri tanımlayalım
    int yordimci;

    //diziyi oluřturalım
    int[] dizi = { 5, 1, 4, 2, 8 };

    //diziyi sıralayalım
    for (int x = 0; x < dizi.Length - 1; x++) //0 indisli elemandan son elemana kadar
    {
        for(int y=x+1; y<dizi.Length; y++)//x'in saęındaki eleman
            if(dizi[x]>dizi[y]) //eđer soldaki eleman büyükse
            {
                yordimci = dizi[y]; //yerdeęiřtirme geręekleřtir
                dizi[y] = dizi[x];
                dizi[x] = yordimci;
            }
    }

    //dizinin sıralanmıř halini yazalım
    foreach (int eleman in dizi)
        Console.WriteLine(eleman);

    Console.Read(); //bekle
}
```


İki Boyutlu Dizilerde Matematiksel İşlemler

Bu örneğimizde iki boyutlu iki dizinin elemanlarını konsoldan okuyacağız, okunan dizilerin elemanlarını konsola yazdıracağız ve daha sonra da iki diziyi toplayıp sonucu konsola yazdıracağız. Çıkarma, çarpma ve bölme için de benzer işlem yapılabilir.

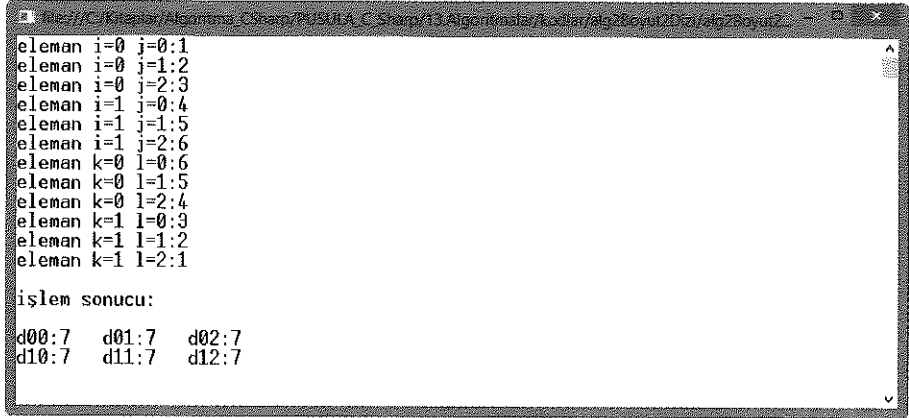
```
static void Main(string[] args)
{
    //iki boyutlu diziyi tanımlayalım
    //dizi 2 satır 3 sütun
    int i, j, k, l, m, n;
    int[,] dizi1 = new int[2, 3];
    int[,] dizi2 = new int[2, 3];
    int[,] dizi3 = new int[2, 3];

    //dizi1 değerlerini konsoldan okuyalım
    for ( i = 0; i < 2; i++)
    {
        for ( j = 0; j < 3; j++)
        {
            Console.Write("eleman i=" + i + " j=" + j + " :");
            dizi1[i, j] = Convert.ToInt32(Console.ReadLine());
        }
    }

    //dizi2 değerlerini konsoldan okuyalım
    for ( k = 0; k < 2; k++)
    {
        for ( l = 0; l < 3; l++)
        {
            Console.Write("eleman k=" + k + " l=" + l + " :");
            dizi2[k, l] = Convert.ToInt32(Console.ReadLine());
        }
    }

    //dizi3 elemanlarını hesaplayalım
    for ( m = 0; m < 2; m++)
    {
        for ( n = 0; n < 3; n++)
        {
            dizi3[m, n] = dizi1[m, n] + dizi2[m, n];
        }
    }
    Console.WriteLine(""); //bos satır
    //dizi3 elemanlarına yazdıralım
    Console.WriteLine("işlem sonucu:");
    for (m = 0; m < 2; m++)
    {
        if (m % 1 == 0) Console.WriteLine("\n"); //satır sonunda alte geç
        for (n = 0; n < 3; n++)
        {
            Console.Write("d"+m+n+": "+dizi3[m, n]+" ");
        }
    }
    Console.ReadLine(); //bekle
}
```

Ekran Çıktısı:



```
File:///C:/Kutuplu/Algoritma/CSharp/13.Algoritma/ko2lar/alg2Gir2Dizi/alg2Gir2Dizi2...
eleman i=0 j=0:1
eleman i=0 j=1:2
eleman i=0 j=2:3
eleman i=1 j=0:4
eleman i=1 j=1:5
eleman i=1 j=2:6
eleman k=0 l=0:6
eleman k=0 l=1:5
eleman k=0 l=2:4
eleman k=1 l=0:9
eleman k=1 l=1:2
eleman k=1 l=2:1

işlem sonucu:
d00:7 d01:7 d02:7
d10:7 d11:7 d12:7
```

13

Nesne Tabanlı Programlama

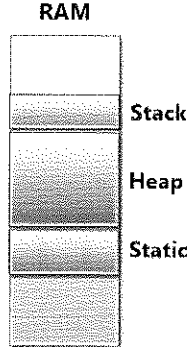
- Bellek Alanları
- Nesne Tabanlı Programlama Kavramları
- Sınıf ve Nesne Oluşturma
- Sınıf Üyeleri
- Erişim Düzenleyicileri
- Kapsülleme
- Yapıcı ve Yıkıcı Metotlar
- Statik Yapıcı Metotlar
- Statik Sınıflar
- This Göstericisi

Bellek Alanları

Nesne tabanlı programlamaya ayrıntılı olarak başlamadan önce kitabımızın **Bilgisayar Mimarisi** bölümünde anlatılan bellek kavramına daha ayrıntılı değinmemiz gerekmektedir. Eğer programcılıkta kullanılan bellek yapısını iyi anlamazsak, nesne tabanlı programlamada üretilen bazı veri tiplerini ve komutları da iyi anlayamayız. Bu nedenle bellek konusuna biraz daha ayrıntılı girmemiz gerekiyor. Temel olarak programcılıkta kullanılan 3 tür bellek alanı vardır. Bunlar aşağıdaki gibidir:

- Yığın (Stack)
- Küme (Heap)
- Sabit (Static)

Yukarıda sayılan bellek alanları Türkçe kitaplarda karşınıza farklı isimlerle çıkabilir ama İngilizceleri genelde yukarıda parantez içinde yazıldığı gibidir. Kabaca bahsetmek gerekirse, yukarıda bahsi geçen her bir bellek alanı RAM bellek içinde bulunur ve farklı türde verileri saklamak için kullanılır. Eğer bu alanları görsel olarak ifade etmek gerekirse aşağıdaki şekilde görüldüğü gibidir.



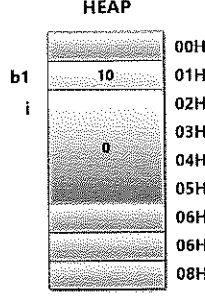
Şekil 13.1 RAM bellekteki hafıza alanları

Yukarıdaki şekli iyice incellerseniz **Stack** hafıza alanının **Heap** hafıza alanından daha az yer kapladığını görebilirsiniz. Bu durum gerçekte de böyledir; Heap hafıza alanı Stack hafıza alanına göre çok daha büyüktür.

Stack Bellek Alanı

Kısaca tarif etmek gerekirse, **Stack** hafıza alanında ilkel veri tipleri ile nesnelerin **Heap** alanındaki adresleri (referansları) tutulur. İlkel veri tipleri derken, **byte**, **int** veya **double** gibi, kitabın önceki bölümlerinde kullandığımız veri tiplerini kastediyoruz. Bu veri tiplerinin boyutları ve RAM bellekte kaplayacağı alanlar bellidir. Örneğin **byte** veri tipi hafızada 8 bit yer kaplar ve 0-255 arası değerleri alır. Eğer bu veri tipine 300 değerini atarsanız hata alırsınız. Aynı şekilde **int** veri tipi hafızada 32 bit yer kaplar ve -2147483648 ile 2147483647 arası değerlere sahip olabilir. Eğer **byte** ve **int** tipinde birer değişken oluşturursak, bu değişkenlerin hafızadaki görünümü aşağıdaki gibi olur.

```
byte b1=10;
int i=0;
```



Şekil 13.2 Stack hafızada değişkenlerin durumu

Yukarıdaki şekilde de görüldüğü gibi, **byte** veri tipi hafızada 8 bit, **int** veri tipi ise 32 bit yer kaplar. Hangi verinin hangi adrese yerleştirileceğini konusunda programcının bir yetkisi yoktur; değişkenlerin adresleri tamamen derleyici tarafından belirlenir. Buradaki örneğimizde **b1** değişkeni RAM bellekteki **01H** hafızasına, **i** değişkeni ise **02H** hafızasına yerleştirilmiştir.

C# derleyicisi kodlarınızı derlemeden önce **Stack** hafıza bölgesinde oluşturulacak olan ilkel değişkenlerin (verilerin) boyutlarını bilmek zorundadır. Anlaşılacağı üzere, **Stack** bellek alanı programımız çalışmaya başladığı anda belirlenir ve daha sonradan bu alanın boyutu değiştirilemez. Bu hafıza alanı işletim sistemleri tarafından çok kısıtlı bir şekilde kullanılır ve bu nedenle eğer değişkenlerin sınırı dışında bir atama olursa taşma hatası (stack overflow) alınır.

Stack hafıza, ilkel değişkenleri (**değer tipli** değişkenleri) saklamanın yanı sıra, metodların geri dönüş değerlerini saklamak ve metotlara parametre göndermek için de kullanılır.

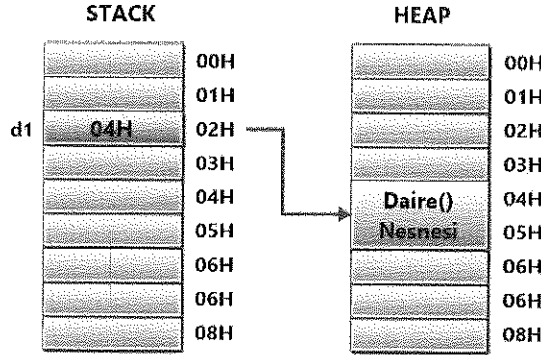
Heap Bellek Alanı

Daha önce de bahsedildiği gibi, **Heap** bellek alanı daha büyük bir alandır. Bu nedenle bu hafıza alanında bulunan değişkenler üzerinde daha fazla düzenleme ve değişkenlere (nesnelere) daha sonradan ekstra veri ekleme şansımız vardır. Heap hafıza alanında ne kadar yer kaplayacağı belli olmayan nesneler saklanır. Geliştirdiğimiz uygulamada **new** komutu ile oluşturulan bütün nesneler bu hafıza bölgesinde saklanır.

Daha sonraki konularda da öğreneceğimiz üzere, bir sınıftan nesne oluşturmak için **new** operatörü kullanılır. Örneğin, **Daire** sınıfından **d1** adlı bir nesne oluşturmak için aşağıdaki komutu kullanırız.

```
Daire d1=new Daire();
```

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir durum oluşur.



Şekil 13.3 Nesnenin oluşturulması ve hafıza alanı kullanımı

Yukarıdaki şekli iyice incelerseniz, herhangi bir nesne oluşturulduğunda hem **Stack** hem de **Heap** hafızasının kullanıldığını görebilirsiniz. Bir nesne oluşturulduğunda, bu nesneye ait olan tüm veriler **Heap** hafıza alanında tutulurken, bu nesnenin **Heap** hafıza bölgesindeki adresi (referansı) **Stack** hafızada tutulur. Bu nedenle bu tür değişkenlere **referans tip** değişken ya da veri denir. Yukarıdaki şekilde **Daire** sınıfından oluşturulan nesne **Heap** hafızadaki **04H** adresinde bulunmaktadır. Nesnenin bulunduğu adresi saklayan **d1** değişkeni ise **Stack** hafızada tutulmaktadır.

Şu ana kadar anlatılanlardan da anlaşılacağı üzere, boyutu belli olmayan ve zamanla boyutu değişebilen nesneler **Heap** hafızasında tutulur. Nesneye yeni özellik ve metotlar eklendikçe nesnenin **Heap** hafızasındaki boyutu artar.

Static Bellek Alanı

Bu bellek alanında **static** veya **const** olarak oluşturulan değişkenler saklanır. Daha önce de öğrendiğimiz gibi, **const** olarak tanımlanan değişkenlere bir değer atanır ve bu değer daha sonradan değiştirilemez. Gene, ileride de öğreneceğimiz gibi, **static** olarak tanımlanan değişken ve metotlar tanımlandığı sınıfa ait olur ve bu sınıftan türetilen tüm nesneler için içeriği aynıdır. Nesnelerden birisi aracılığıyla **static** elemanın içeriği değiştirilirse tüm nesneler için değiştirilmiş olur.

Nesne Tabanlı Programlama Nedir?

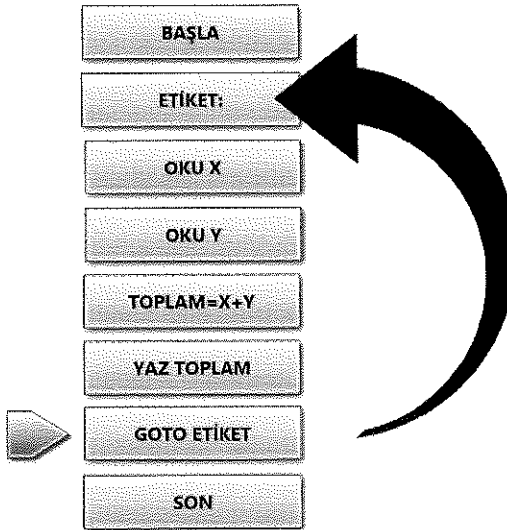
İnsanoğlu **Charles Babbage** adlı İngiliz mühendisin 1822 yılında yaptığı Fark Makinası (Difference Engine) adlı mekanik bilgisayarla beraber aşağı yukarı 200 yıldır programcılıkla uğraşmaktadır. Doğal olarak iki yüz yıl boyunca programlama kav-

ramları ve metodları değişik aşamalar kat etmiştir. Genel olarak programlama dillerini ve teknolojilerini sınıflandıracak olursak üç farklı teknoloji ile karşılaşırız.

- Doğrusal Programlama
- Yapısal Programlama
- Nesne Tabanlı Programlama

Doğrusal programlamada tüm komutlar alt alta yazılır ve sırayla çalıştırılır. Programda dallanma oluşturmak için **goto** benzeri komutlar ve etiketler kullanılır. Dallanma ve etiketler programı karmaşık hale getirdiği için özellikle büyük projelerde kodlar arap saçına döner; programın okuması ve anlaşılması iyice zorlaşır.

DOĞRUSAL PROGRAMLAMA



Şekil 13.4 Doğrusal programlama

Doğrusal programlama küçük programlar için uygunken özellikle büyük programlar için hiç uygun değildir. Bu nedenle **doğrusal programlama** terkedilmiş ve **yapısal programlama** kavramı ortaya atılmıştır. Yapısal programlamada kodlar, mantıksal bütünlük gösteren parçalara bölünebilirler. Bu yolla uzun ve karmaşık programların daha küçük parçalara ayrılarak daha kolay biçimde yazılabilmesi mümkün olmaktadır. **Yapısal programlama** özellikle “goto” komutuna karşı bağımlılığı azalttığı ve hatta kaldırdığı için popüler hale gelmiştir.

Yapısal programlama, yordamsal programlama dillerinin pek çoğu ile yapılabilmektedir. 1970’lerin başlarında popülerleşmeye başlayan yapısal programlama ile pek çok yeni yordamsal programlama dili yapısal programlamayı destekleyecek özellikleri barındırmaya başladılar. Bu dillere örnek olarak Pascal, Ada ve C verilebilir.

Yapısal programlamada kodların daha kolay okunabilmesi ve daha sade yapısından dolayı daha büyük yazılımların daha kolay oluşturulması imkânı elde edilmiştir. Fonksiyonlar kullanılarak karmaşık programlar daha küçük parçalara bölünmüş ve daha basit hale getirilmiştir. Küçük kod parçacıkları seviyesinde yapısal programlama hiyerarşik program akışı yapılarını tavsiye eder. Bu yapılar pek çok modern dilde kolayca elde edilebilen, “while”, “repeat”, “for” gibi yapılardır. Yapısal programlama bu yapılar için tek giriş ve tek çıkış noktalarını tavsiye eder. Bu teknik ile programcılar büyük kod parçalarını daha kısa alt programlar halinde yazarlar. Bu sayede parçacıklar anlaşılabilir kadar küçük olurlar. Genel olarak programlarda çok az küresel değişken kullanılır; küresel değişkenler yerine fonksiyonlar yerel değişkenler kullanılır ve değişkenlerini gösterici veya değer ile gönderirler.

Bu kitapta şu ana kadar öğrendiğiniz programlama teknikleri doğrusal ve yapısal programlama tekniklerini kapsamaktadır. Bundan sonraki kısımlarda ise nesne tabanlı programlama tekniklerini öğreneceğiz.

1960'lı yılların sonunda ortaya atılan nesne tabanlı programlama temel olarak programların bağımsız ve birbirleriyle veri alışverişi yapabilen küçük parçalardan oluşmasını öngörmüştür. Yani, modüler dediğimiz yöntem kullanılmakta ve farklı modüllerin birbiri ile iletişim kurabilmesi ve beraber çalışabilmesi benimsenmektedir. Bu sayede çok büyük ve çok karmaşık yazılımlar daha kolay üretilebilmektedir. Yani, nesne tabanlı programlamada böl ve yönet mantığı kullanılmaktadır.

Nesne tabanlı programlama mantığının ana fikri her programının etkileşim içerisinde olan birimler veya nesneler kümesinden oluştuğu varsayımdır. Bu nesnelerin her biri, kendi içerisinde veri işleyebilir ve diğer nesneler ile çift yönlü veri alışverişinde bulunabilir. Oysa nesne tabanlı programlamadan önce var olan yaklaşımda (yapısal programlama) programlar sadece bir komut dizisi veya birer fonksiyon olarak görülmektedirler.

Modülerliğin yanında nesne tabanlı programlama ayrıca, **bilgi gizleme** (encapsulation), **veri soyutlama** (data abstraction), **çok biçimlilik** (polymorphism) ve **kalıtım** (inheritance) gibi programların bakımını ve aynı program üzerinde birden fazla kişinin çalışmasını kolaylaştıran kavramları da programlama dünyasına kazandırmıştır. Tüm bu avantajlarından dolayı, nesne tabanlı programlama en çok tercih edilen programlama yöntemidir.

Günümüzde başta C++ olmak üzere C#, Java, Perl, Python ve PHP gibi birçok programlama dili nesne tabanlı programlamayı desteklemektedir.

Yukarıda sayılan kavramlar aynı zamanda nesne tabanlı programlamanın omurgasını oluştururlar. Bundan sonraki konularda bu kavramlar üzerinde daha ayrıntılı duracağız fakat şimdilik temel düzeyde bu kavramların ne olduğunu öğrenelim.

Bilgi Gizleme (Encapsulation) & Soyutlama

Nesne tabanlı programlamada yazılan kodlar adına **sınıf** denilen yapılar içine koyulur. **Sınıf** içinde bulunan **değişkenler** ve **metotlar** dışarıdaki kodların erişimine korumalı (gizli) olarak bir araya getirilir. Yani, sınıf dışındaki kodlar sınıf içindeki kodların bir kısmına erişebilirken bir kısmına kesinlikle erişemez. Ayrıca, yazılan kodlar dışarıdaki kodlara/kullanıcılara da görünmez. Zaten kullanıcıların sınıf içindeki kodları görmesi de gerekmez.

Örneğin bir cep telefonu satın aldığınızı düşünün. Bu cep telefonu yüzlerce yazılım ve donanım bileşeninden oluşur. Bu telefonu kullanabilmeniz için telefon içinde bulunan yüzlerce bileşen hakkında bilgi sahibi olmanız gerekmez. Eğer telefonu kapatmak isterseniz **kapat** düğmesine basarsınız ve telefon kapanır. Bu telefon kapanırken hangi elektronik bileşenlerin kullanıldığı ya da hangi kodların çalıştığı sizi ilgilendirmez. Siz sadece **kapat** düğmesine basarsınız. Böylece telefon içine gizlenen onlarca elektronik bileşen ve kodlar çalışarak telefonun kapanmasını sağlar.

Nesne tabanlı programlamada da yukarıdaki cep telefonu kullanımına benzer bir mantık vardır. Sınıf içinde yüzlerce satır kod bulunabilir fakat kullanıcının tüm bu kodları bilmesi gerekmez. Kullanıcılar sadece sınıfın belli üyelerini (kodlarını) çalıştırarak belirli işlemlerin yapılmasını sağlarlar.

Sınıf içinde **private** belirteci ile adlandırılan kodlara sadece sınıf içinde bulunan kodlar erişebilir. Sınıf dışındaki kodlar **private** belirteci ile adlandırılan **alan** ve **metotlara** erişemez. Sınıf dışındaki kodlar sadece **public** belirteci ile adlandırılan kodlara erişebilir. Yani, bazı kodlar dışarıdaki kodlar için gizlidir. Kodların **sınıf** denilen yapı içine alınıp kontrollü olarak kullanıma sunulmasına **encapsulation/gizleme** denir.

Sınıf içinde yüzlerce satırdan oluşan kodlar bulunabilir. Bu kodlar belirli işlevleri yerine getirirler. Bu işlevleri kullanmak isteyen programcılar çok basit kodlar yazarlar; tıpkı cep telefonunun **kapat** tuşuna basılıp telefonun kapatılması gibi. Yani sınıf içinde bulunan yüzlerce satır kod kullanıcıdan **soyutlanır**. Onlar sadece dışarıya açık olan bazı arayüzleri/komutları kullanarak sınıfın işlevlerinden faydalanırlar.

Çok Biçimlilik (Polymorphism)

Çok biçimlilik belirli kodların birbiriyle ilgili birden fazla iş için kullanılmasını sağlar. Örneğin iki tamsayının toplamını yapan bir fonksiyon yazdıysanız bu fonksiyon sadece iki tamsayının toplamını yapar. Eğer iki kesirli sayıyı toplamaya kalkarsanız hata alırsınız. Fakat çok biçimlilik sayesinde bu fonksiyonun farklı sürümlerini yazıp iki kesirli sayıyı toplamasını da sağlayabiliriz.

Çok biçimlilik özelliğini fonksiyonlar konusunda zaten öğrenmiştik: Aşırı yükleme. Aşırı yüklemede fonksiyonların farklı sürümlerini yazıp benzer işleri farklı sayıda veri veya veri tipi kullanarak yapıyoruz.

Fonksiyonlarda olduğu gibi operatörlerde de aşırı yükleme yapabiliriz. Böylece herhangi bir operatöre kullanıldığı yere göre farklı görevler yükleyebiliriz.

Kalıtım (Inheritance)

Nasıl doğduğumuzda anne ve babamızın bazı genetik özelliklerini/yeteneklerini alıyorsak, nesne tabanlı programlamada da yeni oluşturulan nesneler/kodlar daha önce oluşturulan bazı sınıfların/kodların özelliklerini/yeteneklerini alabilir. Bu sayede çok daha hızlı uygulama geliştirebiliriz.

Örneğin bir şeklin alanını hesaplayan bir kod yazdığınızı düşünün. Eğer bu kodları **Şekil** adlı bir sınıf içine koyar ve bu sınıftan yeni **Daire** gibi nesneler üretirseniz yeni türetilen nesneler sınıfın tüm kodlarını kullanabilir. Böylece türetilen nesneler ek kod yazmadan daire veya dikdörtgen gibi şekillerin alanını hesaplayabilir.

Yukarıda anlatılan kavramları bundan sonraki konularda çok daha ayrıntılı öğreneceğiz. Bu kavramlar soyut olduğu için anlaşılması zor olabilir. Lütfen kitaptaki örnek kodları siz de yazın ve yazarak anlamaya çalışın. Yazarak çalışmanız konuların daha iyi anlaşılmasını sağlayacaktır.

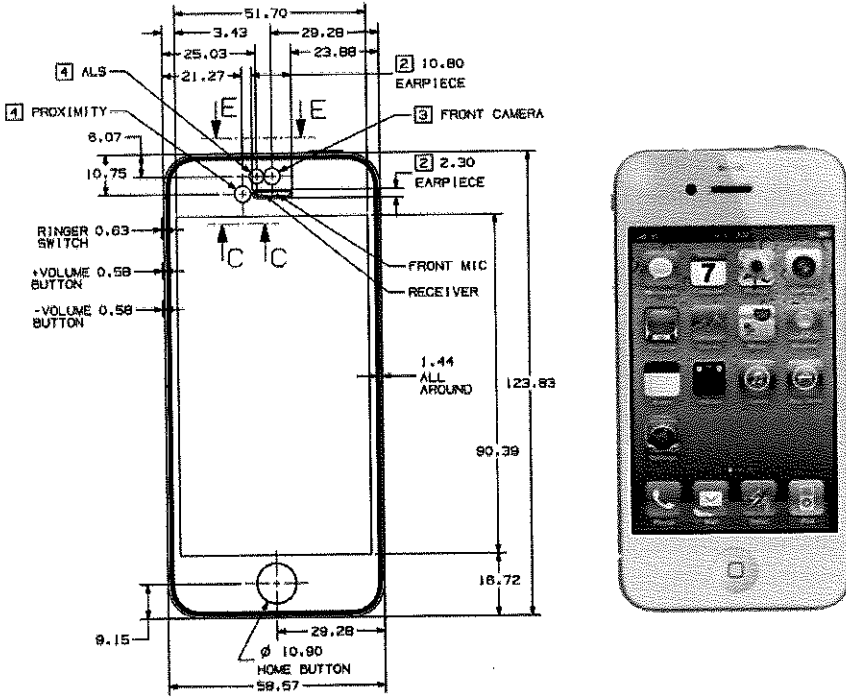
Sınıflar ve Nesneler

Nesne tabanlı programlama dilinde her şey bir **nesne** olarak görülmektedir. Her nesnenin de **alan** ve **metotları** bulunmaktadır. Şu ana kadar kullandığımız değişkenler sınıf içinde **alan** olarak adlandırılır. Alanlar/özellikler/değişkenler nesnenin **fiziksel** görünümünü belirlerken, metotlar da belirli **işlemlerin/eylemlerin** yapılmasını sağlamaktadır. Örneğin, kendimizi bir nesne olarak düşünelim. Boyumuz, kilomuz, ten rengimiz, göz rengimiz ve daha birçok özellik fiziksel görünüşümüzü belirler. İşte bu fiziksel görünüşümüzü belirleyen verilere (değişkenlere) **alan/özellik** diyoruz. Konuşma, yürüme, koşma gibi eylemler ise yapabildiğimiz işleri yani **metotlarımızı** belirler.

Sınıflar nesnelerin bir tür projesi ya da kalıbı gibidir. Nasıl bir makine üretilmeden önce onun bir projesi yapılıyorsa ve projeye göre seri üretim yapılıyorsa nesnelerin de üretilmeden önce projesi yani sınıfları yapılıyor. Yapılan sınıflardan da nesneler türetiliyor.

Aşağıdaki şekilde bir telefonun çizimi ya da tasarımı görülmektedir. Çizimde telefonun genişliği, yüksekliği, kalınlığı ve diğer tüm özellikleri bellidir. Tüm bu çizim ve tasarımlar telefon üretilmeden önce tasarımcı ve mühendisler tarafından oluşturulur. Daha sonra da oluşturulan çizime göre üretim yapılır; tek bir çizimden ya da kalıptan binlerce hatta milyonlarca telefon üretilir. Nesne tabanlı programlamada da buna benzer bir yaklaşım vardır.

Visual Studio Toolbox üzerinde görülen **TextBox** veya diğer sınıflar bir tür çizim ya da tasarımdır; mühendisler tarafından tüm özellik ve işlevleri tasarlanmış ve kodlanarak **Toolbox** üzerine koyulmuştur. Siz eğer isterseniz daha önceden tasarlanan **TextBox** gibi sınıfları kullanarak form üzerinde onlarca **TextBox** veya diğer nesneleri oluşturabilirsiniz.



SINIF -----> NESNE

Şekil 13.5 Sınıf ve nesne kavramları

Yukarıda yapılan çizim bir telefonun tasarımını ifade etmektedir. Nesne tabanlı programlama açısından baktığımızda ise bu çizim bir sınıftır. Üretilcek telefonun bütün alanlarını/özelliklerini ve metotlarını/işlevlerini barındırır. Sınıfın sahip olduğu özellik ve metotlara **üye** denir. Bir cep telefonundaki alan ve metotlar aşağıdaki gibi olabilir.

- Sınıf Adı: Telefon
- Değişkenler/Alanlar (Gerçek hayattaki özellikleri):
 - Renk
 - Ağırlık
 - Yükseklik
 - Genişlik
 - Kalınlık
- Metotları (gerçek hayattaki işlevleri):
 - Aç (Açma düğmesine basarak)
 - Kapat (Kapatma düğmesine basarak)
 - Sesi artır (Düğmeye basarak)
 - Sesi azalt (Düğmeye basarak)
 - Uygulama kapat (Düğmeye basarak)
 - Uygulama aç (simge üzerine basarak)

Yukarıda anlatılan **Telefon** sınıfının **özellikleri** telefonun görünümünü belirlerken, **Telefon** sınıfının **metotları** ise yapabildiği bazı eylemleri/işleri tanımlar. Örneğin, **Renk** özelliği ile telefonun siyah ya da beyaz olmasını sağlayabilirsiniz. **Aç** metodu telefonun açılma işlemini sağlarken **Kapat** metodu telefonun kapanma fonksiyonunu ya da işlemini sağlar.

Yukarıdaki şekilden de görüldüğü gibi, nesneler sınıfların türevleridir. Önce sınıflar tasarlanır, daha sonra da sınıftan istenildiği kadar nesne üretilir. Üretilen nesnelerin verileri ile nesnelerin görünümü değiştirilebilirken metotlar ile belirli işlemlerin yapılması sağlanır. Sınıfın sahip olduğu veri ve metotlara üye dendiğini hatırlayalım.

Büyük bir projede her sınıf ayrı bir programcı tarafından yazılabilir ve diğer programcılarının kullanımına sunulabilir. Diğer programcılar başka bir programcı tarafından yazılan sınıfın kodları ile ilgilenmezler; sadece sınıfın özelliklerini ve metotlarını kullanarak işlerini görürler. Büyük bir projede karşınıza onlarca belki de yüzlerce sınıf çıkabilir. Sizin bir programcı olarak göreviniz başkaları tarafından oluşturulan bu sınıfların özelliklerini ve metotlarını iyice anlayarak program içinde etkili bir şekilde kullanmanızdır.

Yazılan sınıfların başkaları tarafında daha rahat anlaşılabilmesi için **UML** (Unified Modeling Language) denilen bir standart ya da dil kullanılmaktadır. Bu standart sayesinde herhangi bir programcı tarafından yazılan sınıf ve üyeleri görsel olarak ifade edilmektedir. Bu standarda göre her sınıf üç satırdan oluşan bir kutu ile modellenmektedir. Kutunun ilk satırında **sınıf adı** bulunurken ikinci satırında **özellikler**, üçüncü satırında ise **metotlar** bulunmaktadır. Örneğin **Daire** adlı bir sınıfınız varsa ve bu sınıf **r** adlı bir özelliğe ve **alan()** adlı bir metoda sahipse **UML sınıf diyagramı** aşağıdaki gibi olur.

Daire
- r
+ alan()

UML sınıf diyagramları sayesinde projedeki programcılar diğer programcılarının yazdıkları sınıfları çok daha kolay anlayabilir ve projelerinde çok daha kolay kullanabilir.

Sınıf ve Nesne Oluşturma

İsterseniz artık biz de bir **sınıf** oluşturalım ve daha sonra da bu sınıftan **nesneler** türetelim. Yeni bir **sınıf** oluşturmak için aşağıdaki kalıp kullanılır.

```
class SinifAdi
{
    private int degisken1;    //alan
    public byte degisken2;    //alan
    private fonksiyon1()      //metot
    public fonksiyon2()      //metot
}
```

Yukarıda yazılan kodlarla **SinifAdi** adında bir sınıf tanımladık. Sınıf bloğu/gövdesi içinde de iki değişken/alan ile iki de metod tanımladık. Sınıf içinde tanımlanan değişken ve fonksiyonlara sınıfın **üyeleri** denir.

Sınıf içinde görülen ve **private** ile belirtilen kodlara sadece sınıf içinden ulaşılabilirken **public** ile belirtilen kodlara hem sınıf içinden hem de sınıf dışından ulaşılabilir. Yani, eğer sınıftaki bir özellik ya da metoda dışardan erişilmesini istiyorsanız **public** belirtecini kullanmalısınız. Bildiğiniz gibi, İngilizce **public** kelimesi halka açık ve herkesin erişebileceği anlamına gelmektedir. Eğer sınıf içinde **private** veya **public** ifadesi yazılmazsa tüm üyeler **private** kabul edilir. Bu durumda tüm kodlar sınıf dışından ulaşılamaz hale gelir. Yani, **public** dışındaki tüm veriler **private** olur.

Bir sınıftan nesne oluşturmak için aşağıdaki kalıbı kullanabilirsiniz:

```
Sinif nesne=new Sinif();
```

Yukarıdaki komut ile **Sinif** adlı sınıftan **nesne** adlı bir nesne türetmiş olduk. Yeni oluşturulan nesne sınıfın tüm değişken/alan ve metodlarını kullanabilir. Burada oluşturulan nesneyi içinde farklı veri tiplerinde bilgiler barındırabilen ve değişik işler yapabilen karmaşık bir değişken olarak da düşünebilirsiniz.

Değişken adları için geçerli olan tüm kurallar sınıf ve nesne adları için de geçerlidir. İsterseniz bunları kısaca hatırlayalım.

- Türkçe karakter kullanılmıyor.
- İki kelimededen oluşan adlar verilemiyor. İki kelime kullanmak gerekirse deve notasyonunu kullanabilirsiniz. Deve notasyonunda kelimeler birleşiktir fakat her kelimenin ilk harfi büyüktür. Örneğin **BilimselAkademi** adlı iki kelimededen oluşan bir sınıf oluşturabilirsiniz. Bir araya gelen büyük ve küçük harfler devenin sırtına benzediği için bu notasyona **deve notasyonu** denilmiştir.
- C# komutları sınıf ve nesne adı olarak kullanılmıyor.
- Sınıf ve nesne adının ilk karakteri rakam olamıyor.

Sınıfın Alanları

Sınıf içinde tanımlanan değişkenlere **alan** (field) denir. Yani, sınıf içinde tanımlanan tüm değişkenler sınıfın bir alanı olarak işlev görür. Örneğin, **Daire** sınıfında tanımlanan **r** değişkeni bir alan olarak işlev görür. Aşağıda **Daire** sınıfı içinde tanımlanmış değişkenini ya da alanını görüyorsunuz. Bu değişken dairenin fiziksel bir özelliğini, yani yarıçapını belirler. Bu değişken **public** belirteci ile tanımlandığı için sınıf dışındaki kodlar tarafından da kullanılabilir.

```
class Daire
{
    public double r;
}
```

Eğer **Daire** sınıfından **d1** adlı bir nesne oluşturursak, bu nesnenin **r** alanını kullanarak **d1** nesnesinin yarıçapını belirleyebiliriz.

```
Daire d1=new Daire();
d1.r=3.4;
```

Yukarıdaki örnekte de görüldüğü gibi, sınıf içinde tanımlanan değişkenin veri tipi nesnenin verisine atanacak değerin de hangi türde ve hangi aralıkta değer olabileceğini belirler. Verilere yanlış tipte ve aralıkta atanan değerler programda hataya sebep olur.

Sınıfın Metotları

Sınıf içinde oluşturulan fonksiyonlara **metot** denilmektedir. Herhangi bir sınıftan türetilen nesneler bu metotları kullanarak değişik işlemler gerçekleştirebilir. Örneğin, bir dairenin alanını bulan bir metodu yazmak istiyorsanız aşağıdaki kodları kullanabilirsiniz.

```

class Daire
{
    public double r;
    public double alan(double r)
    {
        return Math.PI*r*r;
    }
}

```

Yukarıda yazılan sınıftan **d2** adlı bir nesne oluşturup nesnenin üyelerini yani alan ve metodunu kullanarak dairenin alanını hesap edelim.

```

Daire d2=new Daire();
d2.r=1;
Console.WriteLine("alan="+alan(d2.r));

```

Bir **sınıftan** istediğiniz kadar **nesne** oluşturabilirsiniz. Örneğin, aşağıdaki kodlarda **Daire** sınıfından **d5**, **d6** ve **d7** adıyla üç tane nesne oluşturulmuştur.

```

Daire d5=new Daire();
Daire d5=new Daire();
Daire d7=new Daire();

```

İsterseniz artık yukarıda anlatılan konuları daha derli toplu bir örnekle uygulayalım. Bu örnekte önce **Dikdortgen** adlı bir sınıf oluşturulmaktadır. **Dikdortgen** sınıfının **yukseklik** ve **genislik** adlı iki adet **public** alanı bulunmaktadır. Yani bu değişkenlere/alanlara sınıf dışından erişilebilir. **Main()** fonksiyonunda **d1** adlı bir **Dikdortgen** nesnesi oluşturulmakta ve bu nesnenin alanlarına **d1.yuksekli=5;** ve **d1.genislik=10;** komutları ile değer atanmaktadır. Daha sonra da **d1.uzunluk*d1.genislik** ifadesi ile **d1** nesnesinin alanı hesaplanmaktadır.

```

using System;

class Dikdortgen
{
    public int uzunluk, genislik;
}

class Program
{
    static void Main (string[]args)
    {

```

```

        Dikdortgen d1=new Dikdortgen();
        d1.uzunluk=5;
        d1.genislik=10;

        Console.Write("d1 alanı="+ d1.uzunluk*d1.genislik);
        Console.Read();
    }
}

```

Eğer yukarıda tanımladığımız **Dikdortgen** sınıfını biraz daha geliştirmek istersek içine dikdörtgenin alanını hesaplayan bir metot da yazabiliriz. Bu durumda kodlar aşağıdaki gibi olur:

```

using System;

class Dikdortgen
{
    public int uzunluk, genişlik;
    public double alan()
    {
        return uzunluk*genişlik;
    }
}

class Program
{
    static void Main (string[]args)
    {
        Dikdortgen d1=new Dikdortgen();
        d1.uzunluk=5;
        d1.genislik=10;

        Console.Write("d1 alanı="+ d1.alan());
        Console.Read();
    }
}

```

Eğer programı daha da geliştirmek istiyorsanız **d1** nesnesinin **uzunluk** ve **genişlik** değerlerini konsoldan okuyabilirsiniz. Bunun için aşağıdaki kodları yazmanız yeterlidir:


```

Console.Write("Uzunluk:");
d1.uzunluk=Convert.ToInt32(Console.ReadLine());
Console.Write("Genişlik:");
d1.genislik=Convert.ToInt32(Console.ReadLine());

```

Örnek Kod:

Bu örneğimizde **Daire** adlı bir sınıf oluşturalım. Daha sonra dairenin yarıçapını belirlemek üzere **r** adlı bir alan/değişken ve dairenin alanını hesaplamak üzere **alan()** adlı bir metod oluşturalım. Ondan sonra da **d** adlı bir **Daire** nesnesi oluşturup yarıçapını okuyarak alanını hesaplayalım. Sınıfın **Main()** fonksiyonu dışında yazıldığına özellikle dikkat ediniz. **Daire** sınıfı **Main()** fonksiyonundan tamamen ayrı bir kod grubudur. Bu da nesne tabanlı programlamanın modüler (parçalı) olmasını sağlar. Eğer gerekirse **Main()** fonksiyonu içinde **Daire** sınıfından nesneler oluşturulup kullanılabilir.

```

1  using System;
2  namespace ConsoleApplication4
3  {
4      class Daire
5      {
6          public double r;           //r alanı
7          public double alan()       //alan metodu
8          {
9              return Math.PI * r * r;
10         }
11     }
12     class Program
13     {
14         static void Main(string[] args)
15         {
16             Daire d = new Daire();   //d nesnesi
17             Console.Write("Yarıçap:");
18             d.r = Convert.ToDouble(Console.ReadLine()); // d nesnesinin r yarıçapına ata
19             Console.WriteLine("Alan=" + d.alan());      // d nesnesinin alan() metodunu çalıştır
20             Console.Read();
21         }
22     }
23 }
24

```

Şekil 13.6 Sınıf ve nesne oluşturma

Konunun daha iyi anlaşılabilmesi için bazı satırları açıklayalım:

- Programın 4. satırında **class Daire** komutu ile **Daire** adında bir sınıf tanımlanıyor. Bu sınıf kullanılarak bir sürü daire nesnesi oluşturulabilir ve oluşturulan her bir nesnenin alan ve metotları kullanılarak daire işlemleri yapılabilir.
- Programın 6. ve 7. satırında **public** ifadesi bulunmaktadır. Bu satırlarda tanımlanan alan ve metotlara sınıf dışından erişilebilir.
- 12. satırda proje açıldığında otomatik olarak gelen **Program** sınıfı bulunmaktadır. Bu sınıfın adı isteğe göre değiştirilebilir. Şimdilik varsayılan adı olan **Program** şeklinde kalsın.
- 14. satırda **Main()** metodu başlar. Kodlarımızı bu metod içine yazabiliriz. Programı çalıştırdığınızda ilk önce bu metod içindeki kodlar çalıştırılır.

- 16. satırda **Daire** sınıfından **d** adlı bir nesne oluşturulmaktadır. Eğer **Daire** sınıfından daha fazla nesne oluşturmanız gerekirse **d1**, **d2**, **d3** gibi adlarla birden fazla nesne tanımlayabilirsiniz.
- 17. satırda konsola "**Yarıcap:**" yazdırılmakta ve kullanıcıdan dairenin yarıçapı istenilmektedir.
- 18. satırda ise okunan yarıçap **d** nesnesinin **r** özelliğine atanmaktadır.
- 19. satırda **d** nesnesinin alanını konsola yazmak için **alan()** metodu kullanılır.

Eğer **d** nesnesinin çevresini de hesap etmek istiyorsanız sınıf içine aşağıdaki metodu yazabilirsiniz:

```
double cevre()
{
    return 2*pi*r;
}
```

Çevreyi konsola yazdırmak için 19. Satırın altına aşağıdaki kodları yazabilirsiniz.

```
Console.WriteLine("Cevre="+cevre());
```

Böylece bir dairenin hem alanını hem de çevresini hesaplayan bir sınıf yazmış olursunuz.

Örnek Kod:

Dikdörtgenin alanını ve çevresini hesaplayan **Dikdortgen** adlı bir sınıf oluşturalım. Daha sonra oluşturduğumuz sınıftan bir **dik** nesnesi oluşturup genişliği ve yüksekliği alınan bir dikdörtgenin alan ve çevresini hesaplayalım.

Bu örneğimizde sınıf içinde tanımlanan fonksiyonlarda parametre de kullanacağız. İsterseniz fonksiyonları parametresiz de yapabilirsiniz çünkü parametresiz yapmak daha mantıklı. Fakat sadece değişiklik olsun diye, bu örneğimizde fonksiyonlarda parametre kullanacağız.

```

1  using System;
2  namespace dikdortgen
3  {
4      class Dikdortgen
5      {
6          public int gen, yuk; //alanlar
7          public int alan(int g, int y) { return g * y; } //alan() metodu
8          public int cevre(int g, int y) { return 2 * (g + y); } //cevre() metodu
9      }
10     class Program
11     {
12         static void Main(string[] args)
13         {
14             Dikdortgen dik = new Dikdortgen(); //dik nesnesi olustur
15             Console.Write("Genişlik:");
16             dik.gen = Convert.ToInt32(Console.ReadLine()); //genişliğe ata
17             Console.Write("Yükseklik:");
18             dik.yuk = Convert.ToInt32(Console.ReadLine()); //yüksekliğe ata
19             Console.WriteLine("Alan=" + dik.alan(dik.gen, dik.yuk));
20             Console.WriteLine("Çevre=" + dik.cevre(dik.gen, dik.yuk));
21             Console.ReadLine();
22         }
23     }
24 }
25

```

Şekil 13.7 Dikdörtgen sınıfı ve nesneleri

Alan ve Metotların (Üyelerin) Kullanımı

Herhangi bir nesnenin alanına değer atamak için **Nesne.Alan=Değer**; ifadesini kullanıyoruz. Örneğin, yukarıdaki örnek kodlarda oluşturulan **d1** nesnesinin **gen** ve **yuk** alanlarına değer atamak için aşağıdaki komutları kullanabiliriz.

```

d1.gen=5;
d1.yuk=3;

```

Eğer değerleri doğrudan atamak yerine konsoldan okuyarak atamak isterseniz aşağıdaki komutları kullanabilirsiniz.

```

d1.gen=Console.ReadLine();
d1.yuk=Console.ReadLine();

```

Herhangi bir nesnenin metodunu çalıştırmak istersek **Nesne.Metot()**; ifadesini kullanıyoruz. Örneğin **d1** nesnesinin **alan()** ve **cevre()** metotlarını kullanmak için aşağıdaki kodları yazıyoruz.

```

d1.alan();
d1.cevre();

```

Nesnenin sahip olduğu alan ve metotlar o nesnenin **üyelerini** (member) oluşturur. Örneğin, aşağıda gördüğümüz **d1** nesnesinin iki adet alanı ve iki adet metodu olmak üzere dört adet üyesi bulunmaktadır.

```
d1.gen=3;
d1.yuk=1;
d1.alan();
d1.cevre();
```

Yaptığımız uygulamalardan da anladığınız üzere bir nesnenin herhangi bir üyesine erişebilmek için nokta (.) operatörü kullanıyoruz.

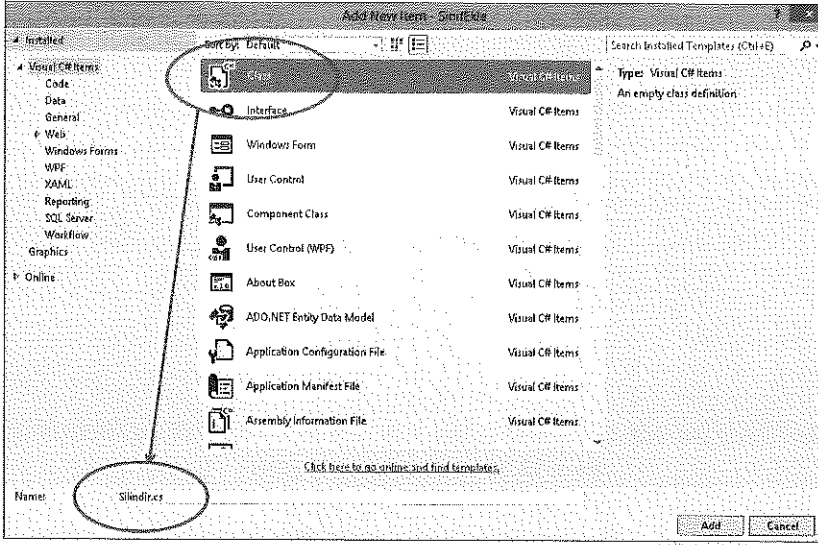
```
nesne.alan=değer;
nesne.metot();
```

Visual Studio Sınıf İşlemleri

Bu başlık altında **Visual Studio** ile sınıf oluşturma ve düzenleme işlemlerini öğreneceğiz. Bunun için önce bir **sınıf** oluşturacağız. Daha sonra oluşturulan sınıfa **alan** ve **metot** ekleyerek **üyeleri** oluşturacağız.

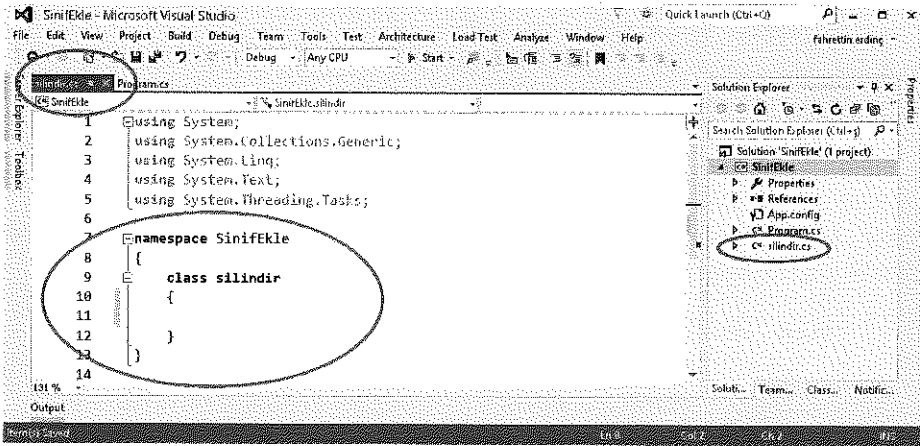
Öncelikle **File → New → Project → Console Application** komutu ile yeni bir konsol uygulaması başlatalım. Proje adı **SinifEkle** olsun. Bu işlem sonunda kod penceresinde **SinifEkle** adlı bir aduzayı/namespace ve **Program** adlı bir sınıf/class görürsünüz. Visual Studio varsayılan olarak proje adında bir aduzayı ve **Program** adında bir sınıf oluşturur. Eğer sağdaki **Solution Explorer** paneline bakarsanız **SinifEkle** projesinin altında **Program.cs** adlı sınıfı görebilirsiniz.

Biz projemize **Silindir** adlı bir sınıf daha eklemek istiyoruz. Bunun için **Solution Explorer** penceresindeki proje adına sağ tıklayıp **Add → New Item** komutunu çalıştırıp **Add New Item** penceresinde görülen **Class** seçeneğini işaretleyin. Daha sonra da aşağıda görülen **Name** kutusuna sınıfımızın adını belirleyecek olan **Silindir.cs** yazıp **Add** düğmesine tıklayın.



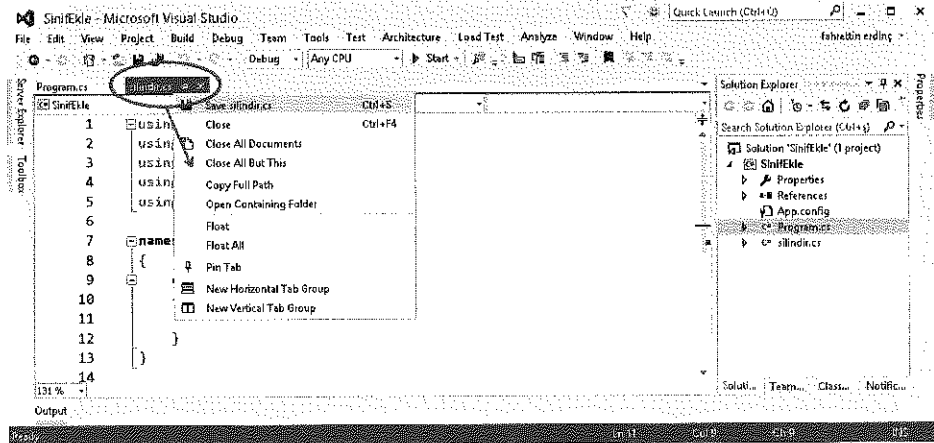
Şekil 13.8 Proje sınıf ekleme

Yukarıdaki işlem sonunda **Silindir.cs** adlı sınıf penceresi açılır ve **Solution Explorer** penceresindeki proje adı altında **Silindir.cs** adlı sınıf görülür.



Şekil 13.9 Sınıf penceresi

Solution Explorer penceresinde herhangi bir sınıfı açıp kod yazabilirsiniz. Açılan sınıfın sekmesi araç çubuklarının hemen altında görünür. Sınıf sekmesine sağ tıklayarak kod kaydı ve kapatma gibi işlemleri yapabilirsiniz.



Şekil 13.10 Sınıf penceresi işlemleri

Artık **class silindir{...}** içine sınıfın kodlarını yazabiliriz. Amacımız bir silindirin hacmini bulan bir sınıf oluşturmak.

```
class silindir
{
    private double r,h;
    public double hacim(double r, double h)
    {
        return Math.PI*r*r*h;
    }
}
```

Yukarıdaki kodları yazıp **silindir** sınıfını tamamladıktan sonra **Program.cs** sınıfına gidip **Main()** fonksiyonu içine aşağıdaki kodları yazabilirsiniz.

```
static void Main(string[] args)
{
    double yukseklik, yaricap;
    silindir s = new silindir();
    Console.Write("Yükseklik:");
    yukseklik = Convert.ToDouble(Console.ReadLine());
    Console.Write("Yarıçap:");
    yaricap = Convert.ToDouble(Console.ReadLine());
    //silindir hacmini yaz
    Console.Write("Hacim=" + s.shacim(yukseklik,yaricap));
    Console.ReadLine();
}
```

Şekil 13.11 Nesnelere özellik atama ve kopyalama

Nesne Kopyalama

Sınıflardan türetilen nesneler başka bir nesneye kopyalanabilir. Örneğin aşağıdaki gibi **Personel** adlı bir sınıfımızın olduğunu düşünelim.

```
class Personel
{
    public string ad;
    public string soyad;
    public string tckn;
}
```

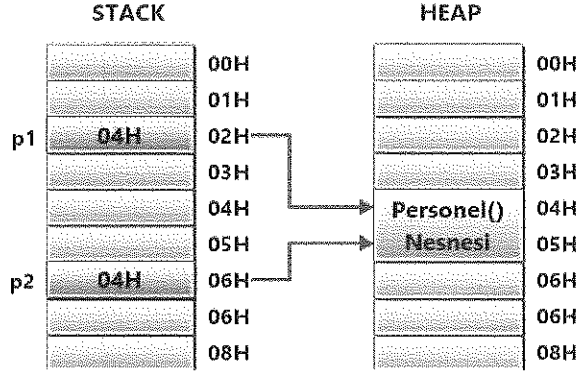
Yukarıda tanımlanan **Personel** sınıfından **p1** adlı bir nesne oluşturmak ve verilerini atamak için aşağıdaki kodlar kullanılabilir:

```
Personel p1=new Personel();
p1.ad= "fahrettin";
p1.soyad= "erdiñç";
p1.tckn= "64164112641";
```

Yukarıda oluşturulan nesneyi ve özelliklerini başka bir nesneye kopyalamak için aşağıdaki kodlar yazılabilir:

```
Personel p2;
p2=p1;
```

Yukarıda yazılan kodlarla **p2** adında bir **Personel** nesnesi oluşturuldu ve bu nesneye **p1** nesnesi kopyalandı. Bu durumda **p1** nesnesinin tüm özellikleri **p2** nesnesine de kopyalanmış oldu. Yani, **p2.ad= "fahrettin"**, **p2.soyad= "erdiñç"** ve **p2.tckn= "64164112641"** oldu. **p2** nesnesini oluşturulduğu satırda **Personel p2=new Personel();** ifadesi bulunmadığından dolayı, **p2** nesnesi için stack hafızada sadece **p2** nesnesinin adresini saklayan hafıza alanı bulunur; nesnenin üyelerini barındıran heap hafıza bölgesi oluşturulmamıştır. Bu durumu görsel olarak ifade etmek gerekirse yapılan işlem aşağıdaki şekilde görüldüğü gibidir. Dikkat edilirse hem **p1** nesnesi hem de **p2** nesnesi heap hafızada aynı nesneye referans vermektedir. Bu durumda, **p1** nesnesindeki üye değerleri değiştirildiğinde **p2** nesnesindeki üye değerleri de değişir. Aynı durum tam tersi için de geçerlidir; **p2** nesnesindeki üye değerleri değiştirildiğinde **p1** nesnesindeki üye değerleri de değişir.



Şekil 13.12 Nesnelerin kopyalanması

Yukarıdaki örneklerden de anlaşıldığı gibi, sınıflar aslında değişkenlerle fonksiyonların bir arada kullanıldığı oldukça esnek ve modüler yapılar ya da program parçalarıdır. Her sınıf sanki başka bir programmış gibi davranır ve diğer sınıf ve fonksiyonlarla veri alışverişinde bulunur. İşin en güzel tarafı da bir sınıfın kodlarını bilmek zorunda olmayışımızdır. Çünkü biz kullanmak istediğimiz sınıfın kodları ile ilgilenmeyiz; bizi ilgilendiren şey sınıfın yaptığı iş, bu sınıfın alanları ve metotlarıdır. Sınıfın alanlarını ve metotlarını kullanarak ilgili işlemleri kolayca yapabiliyoruz.

Erişim Belirteçleri

Sınıf içinde bulunan üyelere erişmek için değişik erişim belirteçleri kullanılır. Bunlar **private**, **public** ve **protected** adlı komutlardır. Bu komutlar sayesinde sınıfın bazı üyelerine sınıf dışından erişilebilirken bazı üyelerine ise sınıf dışından erişilemez. Daha önce de öğrendiğimiz gibi bu özelliğe **kapsülleme**, **soyutlama** veya **encapsulation** deniyor.

Erişim belirteçlerinin kullanımı aşağıdaki gibi olabilir:

```
class Denetle
{
    private int degisken1;
    public double degisken2;
    protected string degisken3;
}
```

Daha önceden de gördüğümüz gibi **public** belirteciye sahip üyelere programın her yerinden erişilebilir. Yani, **public** belirteciyle yazılan değişken ve metotlar sınıf dışındaki kodlarda kullanılabilir ve istediğiniz değerleri atayabilirsiniz.

Eğer sınıf içindeki bazı üyeleri sınıf dışındaki kodların erişimine kapatmak istiyorsanız **private** belirtecini kullanabilirsiniz. Yani, **private** belirteci ile tanımlanan üyelere sadece sınıf içinden ulaşılabilir. Varsayılan olarak sınıfın tüm üyeleri **private** kabul edilir. Eğer bir sınıf üyesini tanımlarken herhangi bir belirteç kullanmazsanız **private** olur ve bu üyeye sınıf dışından erişilemez.

protected olarak belirtilen üyeler daha çok **private** gibi davranır. Bu tip üyelere de tıpkı **private** üyelere olduğu gibi dışarıdan erişilemez. Protected üyelere sadece türetilmiş sınıflar erişebilir. Türetilmiş sınıf bir başka sınıftan oluşturulan sınıftır. Bu konuyu daha sonraki **Kalıtım** konusunda ayrıntılı olarak göreceğiz. Şimdilik bu kadar bilgi yeterli...

Erişim denetleyicileri ile ilgili erişim durumunu aşağıdaki tablodan da görebilirsiniz.

Erişim belirteci	Açıklama
public	Her yerden erişilir
private	Sadece sınıf içinden erişilir.
protected	Sınıf içinden veya sınıftan türetilen sınıflardan erişilir.

Yukarıdaki tablodan da görüldüğü gibi, aynı sınıf içinde olan tüm üyeler birbirlerine erişebilir. Türetilmiş sınıftaki bir üye temel sınıftaki **public** ve **protected** üyelere erişebilir fakat **private** üyelere erişemez. Sınıf dışındaki kodlar ise sadece **public** üyelere erişebilir. Bunu bir tablo ile gösterecek olursak durum aşağıdaki gibidir:

Erişim	public	protected	private
Aynı sınıf içindeki kod	evet	evet	evet
Türetilmiş sınıftaki kod	evet	evet	hayır
Sınıf dışındaki kod	evet	hayır	hayır

Genelde sınıfın değişkenleri **private**, sınıfın metotları ise **public** olarak tanımlanır. Böylece sınıfın metotları özelliklere ulaşp değerlerinde değişiklik yapabilir. Sınıf dışındaki kodlar ise ancak sınıfın metotlarını kullanarak özellik değerlerini değiştirebilir. Bu konuyu kapsülleme konusunda daha ayrıntılı öğreneceğimiz için şimdilik bu kadarını yazmakla yetiniyorum.

Yapıcı Metotlar (Constructors)

Yapıcı metotlar bir sınıftan bir nesne oluşturulduğunda otomatik olarak çalışan metotlardır. Eğer bir nesne oluşturulduğunda bazı kodların herhangi bir metot çağırma işlemine gerek kalmadan otomatik olarak çalışmasını istiyorsanız bu kodları yapıcı metot içine koyabilirsiniz. Örneğin, bir e-ticaret sitesinde bir üye oluşturduğunuzda

bu üyeye otomatik olarak bir email gönderilmesini istiyorsanız ilgili kodları sınıfın yapıcı fonksiyonuna koyabilirsiniz.

Yapıcı metotların kullanım amaçlarından biri ise sınıftan bir nesne oluşturulurken parametrelerin de belirlenmesidir. Örneğin, **Daire d=new Daire(2);** komutu ile **Daire** sınıfından bir nesne oluştururken dairenin yarıçapı da parametre olarak verilebilir. Bu konuyu parametrelili yapıcı metotlarda ayrıntılı olarak ele alacağız.

Yapıcı metotlar sınıfla aynı ada sahip olmalıdır. Örneğin, **Daire** sınıfı için oluşturulacak bir yapıcı metot aşağıdaki gibi olmalıdır. Eğer yapıcı metodun başka bir sınıf içinde kullanılabilir/çalışabilir olmasını istiyorsanız erişim belirtecinin **public** olmasına dikkat ediniz.

```
class Daire
{
    public Daire() //yapıcı metot
    {
        Console.WriteLine("Yapıcı metot çalıştı!!!");
    }
}
```

Eğer **Main()** metodu içinde **Daire** sınıfından **d** adlı bir nesne oluşturup kodları çalıştırsak yapıcı metot içinde bulunan kodlar otomatik olarak çalışırlar.

```
class Program
{
    static void Main (string[]args)
    {
        Daire d=new Daire();
    }
}
```

Yukarıdaki kodları yazacak olursanız aşağıdaki gibi bir program oluşur.

```

1  using System;
2
3  namespace _13._12oncesi
4  {
5      class Daire
6      {
7          public Daire()
8          {
9              Console.WriteLine("Yapıcı metot çalıştı!!!");
10         }
11     }
12
13     class Program
14     {
15         static void Main(string[] args)
16         {
17             Daire d = new Daire();
18             Console.Read();
19         }
20     }
21 }
22

```

Şekil 13.13 Yapıcı metodun çalışması

Kodların çalışması sonucu aşağıdaki gibi bir görüntü elde edersiniz. Bu da nesne oluşturulduğu anda yapıcı metodumuzun otomatik olarak çalıştırıldığını gösterir.



Şekil 13.14 Yapıcı metodun otomatik çalışması

Yapıcı metotların bir diğer kullanım amacı ise sınıf içinde tanımlanan değişkenlere/ alanlara değer atanmasıdır. Bir sınıf içindeki değişkenlere iki farklı şekilde değer atayabilirsiniz. Birinci yöntem, şu ana kadar yaptığımız üzere, nesne oluşturulduktan sonra ilgili nesnenin verisine uygun bir değerin atanmasıdır. İkincisi yöntem ise ilgili nesnenin alanına yapıcı fonksiyon ile uygun bir değerin atanmasıdır. Konunun daha iyi anlaşılabilmesi için aşağıdaki kodları inceleyelim:

```

class Daire
{
    public double r;

    void oku(double yarıcap)
    {

```

```

        r=yaricap;
    }

    double alan(double r)
    {
        return 3.14*r*r;
    }
}

```

Daire sınıfından **d** adlı bir nesne oluşturalım.

```
Daire d;
```

Yukarıda oluşturulan **d** nesnesinin **r** verisine iki farklı yöntemle değer atayabiliriz:

```

d.r=5; //birinci yöntem
d.oku(2); //ikinci yöntem

```

Konunun daha iyi anlaşılmasını sağlamak için lütfen aşağıdaki kodları yazıp çalıştırınız. Kodları yazdıktan sonra önce 21. satırdaki kodları açıklama haline getirin ve 22. satırdaki atamanın yapılmasını sağlayıp programı deneyin. Daha sonra da 22. satırdan açıklama işaretini kaldırın ve 21. satırı açıklama haline getirip programı deneyin. Her iki şekilde de sonucun aynı olduğunu göreceksiniz.

```

1  using System;
2  namespace ConsoleApplication1
3  {
4      class Daire
5      {
6          //dairenin yarıcapı
7          public double r;
8          //daire yarıcapının okumaya yarayan metot
9          public void oku(double yarıcap) { r = yarıcap; }
10         //dairenin alanını döndüren metot
11         public double alan(double r) { return 3.14 * r * r; }
12     }
13
14     class Program
15     {
16         static void Main(string[] args)
17         {
18             //Daire sınıfından d adlı nesnemizi oluşturalım
19             Daire d = new Daire();
20             //d nesnesinin r alanına değer atayalım
21             //d.r = 2; //birinci yöntem
22             d.oku(2); //ikinci yöntem
23             //Dairenin alanını yazdıralım
24             Console.WriteLine("Alan=" + d.alan(d.r));
25             Console.Read();
26         }
27     }
28 }
29

```

Şekil 13.15 Dairenin yarıcapına değer atanması

Yukarıdaki örnekte nesne oluşturulduktan sonra nesnenin alanlarına nasıl atama yapılabileceğini gördük. Pekî! Ya sınıf oluşturulduğu anda bir değer atamak istersek? İşte bu durumlarda yapıcı metotları kullanabiliriz.

Yukarıdaki örnekte tanımlanan **Daire** sınıfını ve sınıfın yapıcı metodunu aşağıdaki gibi yazabiliriz. Dikkat edilirse yapıcı metot içinde yarıçap değişkenine bir değer atanmakta ve bu değişkene hiçbir değer atanmasa bile alan hesabında bu değer kullanılabilir.

```
class Daire
{
    public double r;

    Daire()//yapıcı fonksiyon
    {
        r=1.0;
    }

    void oku(double yarıcap)
    {
        r=yarıcap;
    }

    double alan(double r)
    {
        return 3.14*r*r;
    }
}
```

Yapıcı metotlar yerine normal metotlar da kullanılabilir. Fakat normal metotların çalışabilmesi için çağrılmaları gerekir. Yukarıdaki örneklerden de görüldüğü gibi, yapıcı metotlar herhangi bir çağırma işlemine gerek kalmadan nesneler oluşturulduğu anda çalışırlar ve bazı işlemlerin otomatik yapılmasını sağlarlar.

Parametrelî Yapıcı Metotlar

Normal metotlar nasıl parametreye sahip olabiliyorsa yapıcı metotlar da parametreye sahip olabilirler. Bu durumda, nesne tanımlanırken yapıcı fonksiyonlara değer gönderip nesne alanlarına değer atayabilirsiniz. Bu işlemin nasıl yapılacağını öğrenmek için aşağıdaki kodları inceleyelim. Yapıcı fonksiyondaki parametrelere varsayılan olarak 1 değerleri atanmıştır. Yani yapıcı fonksiyondaki parametrelere herhangi bir değer atanmazsa bu değerler kullanılacaktır.

```

Class Ucgen
{
    int yükseklik, genislik;

    Ucgen(int y=1,int g=1)
    {
        yukseklik=y;
        genislik=g;
    }
}

```

Ucgen sınıfından **u** nesnesi oluşturulduğunda **Ucgen u(4,3)** komutu ile **Ucgen(int y, int g)** yapıcı metoduna sırayla **y=4** ve **g=3** atamaları yapılmaktadır.

```

using System;

class Ucgen
{
    public int yükseklik, genislik;

    Ucgen(int y=1,int g=1)
    {
        yukseklik=y;
        genislik=g;
    }

    int alan()
    {
        return yukseklik*genislik/2;
    }
}

class Program
{
    static void Main (string[]args)
    {
        Ucgen u=new Ucgen(4,3); //yükseklik ve genislik değerleri atandı
        Console.WriteLine("Yükseklik="+u.yukseklik);
        Console.WriteLine("Genişlik="+u.genislik);
        Console.WriteLine("Alan="+u.alan());
    }
}

```

Bir başka parametrelili yapıcı örneğini **Personel** sınıfı için yapalım. Bu örneğimizde **Personel** sınıfından bir nesne oluştururken TCKN değerinin de atanmasını istiyoruz. Bu durumda **Personel** sınıfı aşağıdaki gibi olabilir.

```
using System;

class Personel
{
    public string ad, tckn;
    public int bolumNo;

    public Personel(string TC)
    {
        tckn=TC;
    }
}
```

Personel sınıfından bir nesne oluştururken yapıcı fonksiyona parametre olarak kimlik numarası bilgisini de girmek istiyorsanız yazacağınız kodlar aşağıdaki gibi olabilir.

```
class Program
{
    static void Main (string[]args)
    {
        Personel p1=new Personel("64108412000");
        Console.WriteLine("TCKN="+p1.tckn);
        Console.Read();
    }
}
```

Programın tamamı aşağıdaki gibi olabilir. Kodları yazıp çalıştırarak program üzerine yorum yapınız.

```

1  using System;
2  namespace yapiciParametrelipersonel
3  {
4      class Personel
5      {
6          public string ad, tckn;
7          public int bolumNo;
8
9          public Personel(string TC)
10         {
11             tckn = TC;
12         }
13     }
14
15     class Program
16     {
17         static void Main(string[] args)
18         {
19             Personel p1 = new Personel("64108412000");
20             p1.ad = "fahrettin";
21             p1.bolumNo = 12;
22             Console.WriteLine("TCKN:" + p1.tckn);
23             Console.WriteLine("Ad:" + p1.ad);
24             Console.WriteLine("Bölüm No:" + p1.bolumNo);
25             Console.Read();
26         }
27     }
28 }
29

```

Şekil 13.16 Parametrelipersonel yapıcı metod örneği

Yukarıda yazılan kodların ekran çıktısı aşağıdaki gibi olmalıdır.



Şekil 13.17 Ekran çıktısı

Varsayılan Yapıcı Metotlar (Default Constructor)

Biz programcı olarak herhangi bir yapıcı metod oluşturmasak bile C# derleyicisi oluşturulan her nesne için bir yapıcı metod oluşturmaktadır. Derleyicinin otomatik olarak oluşturduğu yapıcı metotlara **Varsayılan Yapıcı Metot** denir. Varsayılan yapıcı metotlar tıpkı yapıcı metotlar gibi nesne oluşturulduğu anda otomatik olarak çalıştırılırlar. Varsayılan yapıcı metotların yapısı aşağıdaki gibidir:


```

class Personel
{
    Personel()
    {
        //varsayılan yapıcı metot
    }

    public string ad;
    public int yas;
    public bool evlimi;
}

```

Varsayılan yapıcı metotların görevi sınıfın üye elemanlarına varsayılan başlangıç değerleri atamaktır. Örneğin, aşağıda örnek programda **Personel** adlı bir sınıf oluşturulmuştur. Sınıfın **ad**, **yas**, ve **evli** adlarıyla alanları bulunmaktadır. **Main()** metoduunda **Personel** sınıfından bir nesne oluşturup bu nesnenin üyelerinin değerlerini yazdıralım ve aldıkları değerleri görelim.

```

1  using System;|
2  namespace varsayılanYapiciMetot
3  {
4      class Personel
5      {
6          public string ad;
7          public int yas;
8          public bool evlimi;
9      }
10
11     class Program
12     {
13         static void Main(string[] args)
14         {
15             Personel p1 = new Personel();
16             Console.WriteLine("Ad:" + p1.ad);
17             Console.WriteLine("Yaş:" + p1.yas);
18             Console.WriteLine("Evli mi?" + p1.evlimi);
19             Console.Read();
20         }
21     }
22 }
23

```

Şekil 13.18

Yukarıdaki kodların çalışması sonucu aşağıdaki çıktı elde edilmektedir. Dikkat ederseniz, değişkenlere hiçbir değer atanmamasına rağmen **ad=null**, **yas=0** ve **evlimi=False** değerleri görülmektedir.



Şekil 13.19 Varsayılan yapıcı metot değerleri

Yukarıdaki kodlardan da anlaşılıyor ki varsayılan yapıcı metotlar sınıf içindeki üyeler varsayılan değerler atamaktadırlar. Varsayılan olarak atanan bu değerleri ve değişkenlerin tiplerini aşağıdaki tabloda görebilirsiniz.

Alan Veri Tipi	Varsayılan Değeri
bool	False
int, long vb. tamsayı tipleri	0
double vb. ondalıklı sayı tipleri	0.0
string	null
char	""
Referans tipleri	null

Bir sınıfta yapıcı metot tanımlamazsanız C# derleyicisi otomatik olarak bir varsayılan yapıcı metot oluşturur. Eğer sınıf içinde kendi yapıcı metodunuzu tanımlarsanız varsayılan yapıcı metot oluşturulmaz. Kendi yapıcı metodunuzu oluşturduğunuz zaman sınıf üyelerine varsayılan değerler yerine istediğiniz değerleri atayabilirsiniz.

Yıkıcı Metot (Destructor)

Bir önceki konuda öğrendiğimiz yapıcı metotlar nesne oluşturulduğunda otomatik olarak çalışırken, yıkıcı metotlar ise nesne silindiğinde otomatik olarak çalışır. Tıpkı yapıcı metotlar gibi, yıkıcı metotlar da çağırılarak çalıştırılmaz; kendileri otomatik olarak çalışır. Yıkıcı metotlar tıpkı yapıcı metotlarda olduğu gibi sınıf adını taşır ama ek olarak metodun başına tilde (~) işareti koyulur. Yıkıcı metotları aşağıdaki gibi tanımlayabilirsiniz.

```
class Sınıf
{
    //kodlar
    ~Sınıf()
    {
        //kodlar
    }
}
```

Yıkıcı metotlar genellikle sınıf çalışırken kullanılan kaynakların serbest bırakılmasında kullanılır. Örneğin, sınıf çalışmaya başladığı anda veritabanına bir bağlantı yapılmış olabilir. Veritabanı işlemleri yapıldığı sürece bağlantı devam eder ve sunucu kaynaklarının (CPU, RAM) kullanımına devam edilir. Veritabanı işlemleri bittiği anda bağlantının kesilmesi ve kullanılan sunucu kaynaklarının serbest bırakılması gerekmektedir. Bu durumda, bağlantının kesilmesini sağlayan kodlar yıkıcı metot içine yazılarak, herhangi bir metodun çağrılmasına gerek kalmadan sunucu kaynakları serbest bırakılmış olur.

Bir diğer örnek ise dosyalar için verilebilir. Herhangi bir veriyi dosyaya kaydettikten ve ilgili işlemler bittikten sonra dosyanın kapatılması gerekmektedir. Dosyanın kapanmasını sağlayan kodlar yıkıcı metot içine yazılarak dosya kapatma işleminin otomatik yapılması sağlanabilir.

İsterseniz aşağıdaki örneğimizi yaparak yıkıcı metotlar hakkındaki bilgilerimizi biraz pekiştirelim. Bu örneğimizde **Personel** adlı bir sınıf bulunmaktadır. Sınıf içinde hem yapıcı hem de yıkıcı metotlar tanımlanmıştır.

```

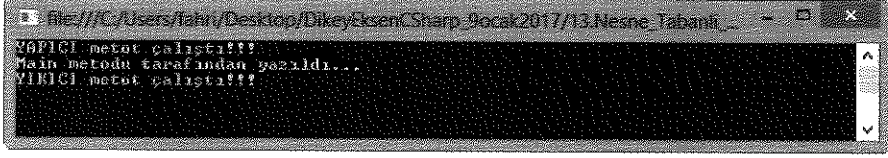
1  using System;
2  namespace yikiciMetot
3  {
4      class Personel
5      {
6          public Personel() //yapıcı metot
7          {
8              Console.WriteLine("YAPICI metot çalıştı!!!");
9          }
10
11         ~Personel() //yıkıcı metot
12         {
13             Console.WriteLine("YIKICI metot çalıştı!!!");
14             Console.Read(); //yıkıcı metot çalıştıktan sonra kapanma
15         }
16     }
17
18     class Program
19     {
20         static void Main(string[] args)
21         {
22             Personel p = new Personel(); //nesne oluşturuldu
23             Console.WriteLine("Main metodu tarafından yazıldı...");
24         }
25     }
26 }
27

```

Şekil 13.20 Yapıcı ve yıkıcı metot

Daha önce de öğrendiğimiz gibi, yapıcı metotlar nesne oluşturulduğu anda, yıkıcı metotlar ise nesne silindiği anda ya da nesnenin görevi/çalışması bittiği anda çalışırlar. Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir ekran elde ederiz. Bura-

daki sonuca göre önce yapıcı metodun çalıştığı anlaşılmaktadır. Daha sonra **Main()** metodu içindeki **Console.WriteLine()** metodu ve son olarak da yıkıcı metodun çalıştığı görülmektedir.



Şekil 13.21 Yapıcı ve yıkıcı metotların çalışma sırası

Yukarıdaki gibi bir çalışma sırasını bir hafıza veya dosya üzerinde çalışırken kullanabilirsiniz. Yapıcı metod dosyayı oluşturup açma işini otomatik olarak yaparken, nesnenin çalışması bittiğinde yapıcı metod ile otomatik olarak dosyayı kapatabilir.

Soyutlama (Kapsülleme / Encapsulation)

Şu ana kadar yaptığımız uygulamalarla da öğrendiğimiz gibi, nesne tabanlı programlamada yazılan kodlar adına **sınıf** denilen yapılar içine koyulur. **Sınıf** içinde bulunan **değişkenler** ve **metotlar** dışarıdaki kodların erişimine korumalı (gizli) ya da kısıtlı olarak açılır. Yani, sınıf dışındaki kodlar sınıf içindeki kodların bir kısmına erişebilirken bir kısmına kesinlikle erişemezler. Ayrıca, yazılan kodlar dışarıdaki kodlara/kullanıcılara da görünmez. Örneğin bir cep telefonu satın aldığınızı düşünün. Bu cep telefonu yüzlerce yazılım ve donanım bileşeninden oluşur. Bu telefonu kullanabilmeniz için telefon içinde bulunan yüzlerce bileşen hakkında bilgi sahibi olmanız gerekmez. Eğer telefonu kapatmak isterseniz **kapat** düğmesine basarsınız ve telefon kapanır. Bu telefon kapanırken hangi elektronik bileşenlerin kullanıldığı ya da hangi kodların çalıştığı sizi ilgilendirmez. Siz sadece **kapat** düğmesine basarsınız. Böylece telefon içine gizlenen onlarca elektronik bileşen ve kodlar çalışarak telefonun kapanmasını sağlar.

Nesne tabanlı programlamada da yukarıdaki cep telefonu kullanımına benzer bir mantık vardır. Sınıf içinde yüzlerce satır kod bulunabilir fakat kullanıcının tüm bu kodları bilmesi gerekmez. Kullanıcılar sadece sınıfın belli üyelerini (kodlarını) çalıştırarak belirli işlemlerin yapılmasını sağlarlar.

Sınıf içinde **private** belirteci ile adlandırılan kodlara sadece sınıf içinde bulunan kodlar erişebilir. Sınıf dışındaki kodlar **private** belirteci ile adlandırılan **alan** ve **metotlara** erişemez. Sınıf dışındaki kodlar sadece **public** belirteci ile adlandırılan kodlara erişebilir. Yani, bazı kodlar dışarıdaki kodlar için gizlidir. Kodların **sınıf** denilen yapı içine alınıp kontrollü olarak kullanıma sunulmasına **encapsulation/gizleme** denir.

Sınıf içinde yüzlerce satırdan oluşan kodlar bulunabilir. Bu kodlar belirli işlevleri yerine getirirler. Bu işlevleri kullanmak isteyen programcılar çok basit kodlar yazarlar; tıpkı cep telefonunun **kapat** tuşuna basılıp telefonun kapatılması gibi. Yani sınıf içinde bulunan yüzlerce satır kod kullanıcıdan **soyutlanır**. Onlar sadece dışarıya açık olan bazı arayüzleri/komutları kullanarak sınıfın işlevlerinden faydalanırlar.

Sonuç olarak, oluşturulan bir sınıf sadece belirli kodları içine alarak kalmaz, aynı zamanda içine aldığı kodların korunmasını, yani kısıtlı bir şekilde erişimini de sağlar. Bir sınıfın üyelerine erişimin kısıtlanması erişim belirteçleri (Access modifiers) ile sağlanır. Erişim belirteçleri ile ilgili erişim durumunu aşağıdaki tablodan da görebilirsiniz.

Erişim belirteci	Açıklama
public	Her yerden erişilir
private	Bu tipte oluşturulan üyelere sadece sınıf içinden erişilir. Sınıf dışındaki kodlar private tipindeki üyelere erişemez.
protected	Sınıf içinden veya sınıftan türetilen sınıflardan erişilir.

Yukarıdaki tablodan da görüldüğü gibi, aynı sınıf içinde olan tüm üyeler birbirlerine erişebilir. Türetilmiş sınıftaki bir üye temel sınıftaki **public** ve **protected** üyelere erişebilir fakat **private** üyelere erişemez. Sınıf dışındaki kodlar ise sadece **public** üyelere erişebilir. Bunu bir tablo ile gösterecek olursak durum aşağıdaki gibidir:

Erişim	public	protected	private
Aynı sınıf içindeki kod	evet	evet	evet
Türetilmiş sınıftaki kod	evet	evet	hayır
Sınıf dışındaki kod	evet	hayır	hayır

Soyutlama konusunu basit bir örnek ile pekiştirmek için aşağıdaki kodları yazabiliriz. Bu örnekte **toplam** adlı bir değişken oluşturulmuş ve başlangıç değeri olarak sıfır atanmıştır. Değişkenin erişim belirteci **private** olduğu için bu değişkenin değeri sınıf dışında yazılan kodlarla değiştirilemez. Zaten değişkenin erişim belirtecinin **private** yapılmasının nedeni koruma altına alınması, dışarıdaki kodlarla yapılabilecek müdahalelerin engellenmesidir. Değişkeni sınıf dışındaki kodlarla değiştiremeyeceğimiz için sınıf içindeki metotları kullanacağız. Sınıf içinde bulunan **Ekle(double d)** metodu ile toplam değişkenine **d** değeri eklenmektedir. Gene, sınıf içinde oluşturulan **Çıkar(double d)** metodu ile de toplam değişkeninden **d** değeri çıkarılmaktadır. Sınıf içinde kullanılan **toplamGoster()** metodu ile de toplam değeri öğrenilmektedir. Sınıf içinde oluşturulan metotlarla **toplam** değişkeninin/alanının doğrudan değiştirilmesi engellenmektedir.

Oluşturulan metotlara değişik kontrol mekanizmaları eklenerek **d** değerinin belli bir aralıkta veya özellikte olması sağlanabilir. Örneğin, **d** değerinin sadece pozitif onda-

lıkli sayı olması test edilerek işlem yapılabilir. Herhangi bir metod olmadan kullanıcılar değişkenlere istenmeyen değerler atayabilir; oluşturulan metotlar sayesinde kullanıcıların girdiği değerlerin doğrulanması ve programın düzgün çalışması çok daha rahat sağlanabilir.

```
class Hesap
{
    private double toplam = 0;

    public void Ekle(double b)
    {
        toplam = toplam + b;
    }

    public void Cikar(double b)
    {
        toplam = toplam - b;
    }

    public double toplamGoster()
    {
        return toplam;
    }
}
```

Yukarıda yazdığımız **Hesap** sınıfını aşağıdaki gibi kullanabiliriz. **Main()** metodunda oluşturulan **h** nesnesi ile **toplam** değişkenine önce 5 ekleyip sonra 2 çıkarıyoruz. Sonuçta, **h.toplamGoster()** metodunun 3 değerini döndürmesi gerekiyor.

```

1  using System;
2  namespace soyutlama
3  {
4      class Hesap
5      {
6          private double toplam = 0;
7          public void Ekle(double b) { toplam = toplam + b; }
8          public void Cikar(double b) { toplam = toplam - b; }
9          public double toplamGoster() { return toplam; }
10     }
11     class Program
12     {
13         static void Main(string[] args)
14         {
15             Hesap h = new Hesap();
16             h.Ekle(5);
17             h.Cikar(2);
18             Console.WriteLine("Toplam="+h.toplamGoster());
19             Console.Read();
20         }
21     }
22 }
23

```

Şekil 13.22 Soyutlama örneği

Hesap sınıfında bulunan metotlar daha da geliştirilebilir. Örneğin, kullanıcıdan alınan değerin pozitif olup olmadığı kontrol edilebilir. Bu işlemi basitçe aşağıdaki gibi yapabiliriz.

```

public void Ekle(double b)
{
    if(b>=0)
        toplam = toplam + b;
    else
        hata=true;
}

```

Yukarıdaki örnek ihtiyaca göre çok değişik şekillerde geliştirilebilir. Bütün bu işlemlerin yapılmasındaki temel amaç sınıf dışındaki kodların sınıf içinde bulunan bazı hassas değerleri istediği gibi değiştirme imkânına sahip olamamasıdır. Soyutlama sayesinde sınıf dışında bulunan kodlar sınıf içindeki **private** üyeleri bizim istediğimiz şekilde değiştirebilmektedir.

Özellikler (Properties), set ve get Erişimcileri

Daha önceki derslerimizde de öğrendiğimiz gibi, değişkenleri/alanları **public** yapıp sınıf dışındaki kodlarla doğrudan değer verme yerine, bu değişkenleri **private** yapıp

sınıf içinde kullanılan metotlarla değer vermek (kapsüllemek/yalıtmak) güvenlik ve veri bütünlüğünün sağlanması açısından çok daha iyidir.

Değişkenlere metotla değer atamak yerine, sırf bu işe özel geliştirilmiş olan **özellikler** (property) kullanmak çok daha güvenli ve kolaydır. **Özellikler** sayesinde sınıf içinde tanımlanan değişkenlere/alanlara kolayca değer atanabiliyor, değerleri okunabiliyor veya bazı kısıtlayıcı işlemler yapılabilir. Özelliklerden değer okumak veya özelliklere değer atamak için adına **erişimci/accessor** denilen **set** ve **get** adlı komutlar kullanılıyor. Adlarından da anlaşılacağı gibi, **set** adlı erişimci özelliğe değer atamak için kullanılabilirken, **get** adlı erişimci özelliklerden değer okumak için kullanılıyor.

Şu ana kadar okuduklarınızdan kafanız biraz karışmış olabilir ama aşağıdaki örneği yaptıktan sonra konu daha anlaşılır hale gelecek.

```
class Personel
{
    private int yas;    //alan
    public int Yas      //özellik
    {
        get {return yas;}    //erişimci
        set {yas=value;}    //erişimci
    }
}
```

Yukarıdaki kodlarda görülen **private** erişim tipindeki **yas** alanına sınıf dışından değer atanmasını istiyoruz. Yalnız, atanacak değerlerin bizim istediğimiz veri tipinde ve aralığında olmasını kontrol etmek için **Yas** adlı bir **özellik** oluşturuyoruz.

Zorunluluk olmamasına rağmen, genelde özelliklere alan adlarıyla aynı ad veriliyor. Fakat ikisinin arasındaki ilişki anlaşılşın diye özelliğin ilk harfi büyük yapılıyor. Buradaki örneğimizde de alan adı **yas** iken özellik adı **Yas** olarak tanımlanmıştır. Bu şekilde alan ile özellik arasında daha kolay görsel ilişki kurulabilir.

Yukarıda yazılan kodlara ve kullanılan **set** erişimcisine göre, **Yas** özelliğine atanan her değer **value** komutu sayesinde **yas** alanına atanacaktır. Gene, **get** erişimcisinde yazılan kodlara göre, **Yas** özelliği **yas** alanının değerini döndürecek.

Aşağıdaki kodları yazıp test ederek programın çalışmasını ve özelliklerdeki **set** ve **get** komutlarının kullanımını somut olarak görebilirsiniz.

```

1  using System;
2  namespace setGet
3  {
4      class Personel
5      {
6          private int yas;    //alan ya da değişken
7          public int Yas      //özellik
8          {
9              get { return yas; }
10             set { yas = value; }
11         }
12     }
13
14     class Program
15     {
16         static void Main(string[] args)
17         {
18             Personel p = new Personel();
19             //Yas" özelliğine değer atayacağız.
20             //Atanan değer otomatik olarak (yas=value) "yas" değişkenine atanacak.
21             p.Yas = 25;
22             //Yas özelliğinin değerini yazdıracğız.
23             //Bu aynı zamanda "yas" alanının (return yas) da değeridir.
24             Console.WriteLine("Personel Yaşı:" + p.Yas);
25         }
26     }
27 }
28

```

Şekil 13.23 Özelliklerin kullanımı

Yukarıdaki kodlardan da anlaşılacağı üzere, **public** olarak tanımlanan özellikler sayesinde, **private** olarak tanımlanan alanlara erişebiliyor ve alanlar üzerinde işlem yapabiliyoruz.

Her özellikte **get** ve **set** erişimcileri olmak zorunda değildir. Eğer amacınız herhangi bir alana değer vermek ise, sadece **set** erişimcisini kullanabilirsiniz. Örneğin aşağıdaki örnekte sadece **tckn** alanının değeri değiştirilmektedir. **set** erişimcisine yazılan kodlara göre, **Tckn** özelliğine atanan her değer **tckn** alanına da atanacaktır.

```

class Vatandas
{
    private string tckn;
    public string Tckn
    {
        set{tckn=value;}
    }
}

```

Eğer amacınız herhangi bir alanının değerini almak/okumak ise, sadece **get** erişimcisini kullanabilirsiniz. Örneğin aşağıdaki örnekte sadece **tckn** alanının değeri okunmaktadır. **get** erişimcisine yazılan kodlara göre, **Tckn** özelliği **tckn** alanının değerini döndürecektir.

```

class Vatandas
{
    private string tckn;
    public string Tckn
    {
        get{return tckn;}
    }
}

```

Yukarıdaki örnek kodlardan da anlaşılacağı üzere, **set** erişimcisi ile yazma, **get** erişimcisi ile ise okuma işlemi gerçekleştirilmektedir. Yani, **set** erişimcisi bir alanın değerini değiştirmek için, **get** erişimcisi ise bir alanın değerini almak/okumak için kullanılır.

Özelliklerin en önemli kullanım alanlarından birisi **private** değişkenlere atanacak değerlerin kontrol edilmesini sağlamasıdır. Örneğin, aşağıdaki kodlarda **yas** alanına/değişkenine atanacak değerin sıfırdan küçük olup olmadığı kontrol edilmektedir. Küçükse "Hatalı giriş!!!" mesajı çıkmakta, sıfır veya büyükse okunan değer yazdırılmaktadır.

```

1  using System;
2  namespace setGet
3  {
4      class Personel
5      {
6          private int yas;    //alan ya da değişken
7          public bool hata = false;
8          public int Yas     //özellik
9          {
10             get { return yas; }
11             set { if (value < 0) hata = true; else yas = value; }
12         }
13     }
14
15     class Program
16     {
17         static void Main(string[] args)
18         {
19             Personel p = new Personel();
20             Console.Write("Yaşınızı giriniz:");
21             p.Yas = Convert.ToInt16(Console.ReadLine());
22             if (!p.hata)
23                 Console.WriteLine("Personel Yaşı:" + p.Yas);
24             else
25                 Console.WriteLine("Hatalı giriş!!!");
26             Console.Read();
27         }
28     }
29 }

```

Şekil 13.24 Atanacak değerin uygun aralıktta olup olmadığının kontrolü

Eğer **private** alanlara atanacak değerleri kontrol etme ihtiyacı duymuyorsanız C# özelliklere atanacak değerler için hızlı ve kolay bir yöntem sunmaktadır. Örneğin, **private** özelliğine sahip bir üyeye **Yas** adlı özellik üzerinden erişilip okunmasını istiyorsanız aşağıdaki kodları yazabilirsiniz.

```
public string Yas{get; set;}
```

Yukarıdaki kodlardan da görüldüğü gibi, doğrudan **Yas** adlı bir özellik tanımlandı; ayrıca sınıf içinde **private string yas;** şeklinde bir değişken de tanımlamanıza gerek kalmadı. **Yas** adlı özellik tanımlandığı anda sınıf içinde aynı ada sahip **alan** da otomatik olarak tanımlanmaktadır.

Yukarıda öğrendiğimiz yeni bilgilerin ışığında **Personel** sınıfının kodlarını aşağıdaki gibi değiştirebiliriz.

```
class Personel
{
    public int Yas {get; set;}           //özellik
}

class Program
{
    static void Main(string[] args)
    {
        Personel p = new Personel();
        p.Yas=22;                       //özellikle değeri atandı
        Console.WriteLine(p.Yas);       //özellik değeri yazıldı
    }
}
```

Yukarıdaki kodlardan da görüldüğü gibi, **private** tipindeki alanlara değeri atamak için metotların terkedilmesiyle beraber hem kodlar sadeleşmiş hem de program yazımı daha kolay hale gelmiştir.

Static Öğeler

Şu ana kadar **static** anahtar kelimesi birçok yerde karşımıza çıktı. Örneğin **Main()** fonksiyonunun önünde **static void Main(string[])** şeklinde her zaman bir **static** komutu vardı. Bu başlık altında **static** komutunun ne işe yaradığını ve nerelerde kullanıldığını öğrenmeye çalışacağız.

Static Alanlar

Herhangi bir sınıfın üyeleri (alan, özellik, metod) **static** olarak tanımlanabilir. Herhangi bir sınıfın üyesini **static** olarak tanımladığınız zaman o üye sadece o sınıfa ait olabilir. Yani, **static** olarak tanımlanan üyeler sınıfın nesneleri tarafından kullanılamaz. Konuyu aşağıdaki örnekle daha somut hale getirelim. Buradaki kodlarda **Art** sınıfı içinde tanımlanan **top** alanı **static** olarak tanımlanmıştır. Bunun anlamı şudur: **top** alanı RAM bellekte belli bir adrese yerleştirilecek ve program içindeki her kod **Art.top** ifadesi ile bu bellek üzerinde işlem yapabilecektir. Alan **static** olarak tanımlandığı için sınıftan oluşturulan nesneler bu alanı kullanamayacaktır.

```

1  using System;
2  namespace statik
3  {
4      class Art
5      {
6          public static int top = 0;
7          public Art()
8          {
9              top++;
10         }
11     }
12
13     class Program
14     {
15         static void Main(string[] args)
16         {
17             //herhangi bir nesne tanımlamadan "top" değişkenini yazalım
18             Console.WriteLine("Toplam=" + Art.top);
19             Console.Read();
20         }
21     }
22 }
23

```

Şekil 13.25 Sınıfın static üyeleri ve kullanımları

Yukarıdaki kodlar dikkatlice incelenirse, **Main()** fonksiyonu içinde herhangi bir nesne oluşturmadan, doğrudan **Art.top (SınıfAdı.alan)** yazılarak **top** alanının değeri konsola yazdırılmıştır. Eğer **top** alanını **static** olarak tanımlamasaydık **Main()** metodu içinde önce bir nesne oluşturmamız, daha sonra da **nesne.top** şeklinde bir kod yazmamız gerekecekti. Alanın **static** olarak tanımlanmasından dolayı artık bir nesne oluşturmaya gerek yoktur; doğrudan **Sınıf.üye** şeklinde yazılarak kullanılabilir.

Üyeleri tanımlarken kullanılan **static** anahtar kelimesi erişim belirtecinden önce veya sonra gelebilir. Örneğin, **public static void Topla(){} ile static public void Topla(){} arasında bir fark yoktur.**

Aslında bir üyeyi **static** olarak tanımlayarak onu tüm program içinde kullanılabilecek (global) tek bir değişken/metot haline getirmiş oluyoruz. Örneğin, ticari bir yazılımda "**dolarkuru**" adlı değişken birçok sınıf tarafından kullanılacaksa, en iyi-

si “**dolarkuru**”nu **static** ve **public** olarak tanımlamaktır. Böylece, **Sınıf.dolarkuru** RAM bellekte tek bir adreste tutulacak ve programdaki herhangi bir kod tek bir adreste bulunan tek bir değişkeni kullanmış olacaktır. Değer tek bir global/küresel değişkende tutulduğu için farklı sınıflarda farklı **dolarkuru** değişkenleri olmayacaktır.

Şu ana kadar kullandığımız **Math** kütüphanesinin **PI** ve **E** gibi alanları veya **String** kütüphanesinin **Length** alanı aslında **static** alanlardır. Bu nedenle **Math** sınıfından bir nesne oluşturmaya gerek kalmadan, doğrudan **Math.PI** yazarak bu alanları kullanabildik.

Sonuç olarak, eğer herhangi bir üyeyi **static** olarak tanımlarsak ilgili sınıf üyelerine nesne oluşturmadan erişebiliriz. Herhangi bir nesneyle bağlantılı olmasını istemediğiniz sınıf üyelerini **static** yapıp onlara her yerden erişebilirsiniz.

Şu ana kadar oluşturduğumuz üyelerde **public** belirteci uygulayarak sınıf dışından da erişilmesini sağlamıştık. Bu sayede sınıf içindeki değişken ve metotlara **küresel/global** özellik kazandırıp programın her yerinden erişebilmiştik. Üyelere küresellik kazandırmanın bir diğer yolu da onları **static** yapmaktır. **Static** üyeler sınıfın bir nesnesi içinde olmadığı için her yerden erişilebilir. Static olarak tanımlanan üyeler bellekte belli bir değere sahiptir ve ilgili sınıftan oluşturulan nesnelerin üyelerinden etkilenmez; bu nedenle her static üyenin sadece tek bir değeri olabilir.

Statik olarak tanımlanmayan her üye dinamik olur. Bir sınıfın dinamik öğelerine o sınıftan oluşturulan her nesne için bellekte bir adres ayrılır. Yani, bir sınıftan 2 nesne oluşturursanız her iki nesnenin de ayrı ayrı değişken ve metotları oluşturulur.

İsterseniz biraz örnek yapalım ve öğrendiğimiz konuları pekiştirmeye çalışalım. Önce sadece **dinamik** üyelere sahip **Daire** adında bir sınıf oluşturalım. Daha sonra da sınıftan iki adet nesne oluşturup nesnelerdeki değişken ve metotları yorumlayalım.

Sınıfımız aşağıdaki gibidir:

```
class Daire
{
    private double r;
    public double alan(double r)
    {
        return Math.PI * r * r;
    }
}
```

Şimdi gidip **Program** sınıfı içinde bulunan **Main()** fonksiyonu içinde yukarıdaki sınıftan iki adet nesne tanımlayalım.

```
Daire d1=new Daire();
Daire d2=new Daire();
```

Yukarıdaki nesneler tanımlandığı anda her iki nesne için de **alan()** adında dinamik birer metot oluşturulur. Her nesnenin metodu, yani, **d1.alan()** ve **d2.alan()** için bellekte birer yer ayrılır. Her iki metodu da kullanarak işlem yapılabilir.

Aşağıdaki örnekte ise hem **r** değişkeni hem de **alan()** metodu **static** olarak tanımlanmaktadır. Bu durumda herhangi bir nesne oluşturmaya gerek kalmadan **r** değişkeni ve **alan()** metoduna programın her yerinden erişilebilir.

```
class Daire
{
    public static double r;
    public static double alan(double r)
    {
        return Math.PI * r * r;
    }
}
```

Sınıfın üyeleri **static** olarak tanımlandığı için herhangi bir nesne oluşturmadan sınıf üyeleri doğrudan kullanılabilir. **Main()** fonksiyonu içine aşağıdaki kodları yazabilirsiniz.

```
Daire.r = 1;
Console.Write("Alan="+Daire.alan(Daire.r));
Console.Read();
```

Yukarıdaki kodları iyice incelerseniz herhangi bir sınıfın tanımlanmadığını görebilirsiniz. Sınıf tanımlanmadığı halde **static** olarak tanımlanan sınıf üyeleri sanki küresel birer öğeymiş gibi kullanılmıştır. Sonuçta, uzun lafın kısası, **static** üyeler küresel birer öğe haline gelir ve programın her yerinden erişilebilir.

Yukarıda anlatılanları bir örnekle daha pekiştirerek devam edelim. Sınıfın **static** üyeleri herhangi bir nesne oluşturmadan, doğrudan **SınıfAdı.üye** sözdizimiyle kullanılırlar. Aşağıdaki örnekte oluşturulan **Personel** nesnesini (personel sayısını) veren kodları görebilirsiniz.

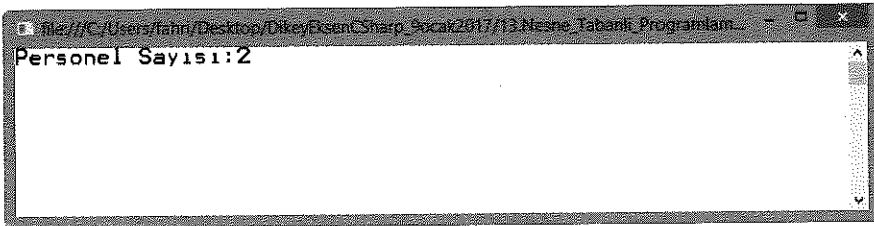
```

1  using System;
2  namespace statikPersonel
3  {
4      class Personel
5      {
6          public static int personelSayisi = 0;
7          public Personel()
8          {
9              //Personel nesnesi oluşturulduğunda Personel()
10             //yapıcı metodu çalışır ve değişkeni 1 arttırır
11             personelSayisi++;
12         }
13     }
14 }
15
16 class Program
17 {
18     static void Main(string[] args)
19     {
20         Personel p1 = new Personel();
21         Personel p2 = new Personel();
22         Console.WriteLine("Personel Sayısı:" + Personel.personelSayisi);
23         Console.Read();
24     }
25 }
26
27

```

Şekil 13.26 Personel sınıfı ve static değişken kullanımı

Yukarıdaki örnek kodlarda da görüldüğü gibi, sınıfın **static** üyelerine doğrudan **Sınıf.üye** şeklinde, yani, **Personel.personelSayisi** şeklinde ulaşıyoruz. **Personel** sınıfından bir nesne oluşturup **personelSayisi** değişkenini kullanamazsınız; kullanırsanız hata alırsınız. Yukarıdaki kodlarda her bir sınıfın oluşturulmasıyla **static** olarak tanımlanan **personelSayisi** 1 artırılmaktadır. Kodlar çalıştığında ekran aşağıdaki gibi görünmektedir.



Şekil 13.27 Personel sayısı

Static Metotlar

Tıpkı **static** alanlarda olduğu gibi, **static** metotlarda da RAM belleğin belli bir bölgesinde tek bir metot oluşturulur ve tüm nesneler/kodlar bu metodu kullanır. Dolayısıyla, global/küresel bir metot elde edilmiş olur.

Şu ana kadar kullandığımız **Math** sınıfındaki metotlar aslında birer **static** metottur. **Math** sınıfındaki **static** metotlara erişmek için herhangi bir nesne oluşturmadan, doğrudan **Sınıf.Metot** şeklinde bir kodla eriştik. Örneğin, **Math** sınıfının **Abs()** metodu doğrudan **Math.Abs(x)**; şeklinde kullanılır. Çok yaygın olarak kullanılan **Math** sınıfındaki metotların **static** olması bize nesne oluşturmadan sınıfın metotlarının kullanılması gibi bir kolaylık sağlar. Benzer şeyler **String** sınıfı için de geçerlidir.

Konuyu basit bir örnekle anlamaya ve pekiştirmeye çalışalım.

```

1  using System;
2  namespace statikMetot
3  {
4      class Kedi
5      {
6          //küresel statik metot
7          public static void Miyav()
8          {
9              Console.WriteLine("Miyaaaaavvvv");
10         }
11     }
12
13     class Program
14     {
15         static void Main(string[] args)
16         {
17             //nesne oluşturmaya gerek kalmadan statik metot kullanımı
18             Kedi.Miyav();
19             Console.Read();
20         }
21     }
22 }
23

```

Şekil 13.28 Statik metot kullanımı

Yukarıdaki kodlarda **Kedi** sınıfının içinde **Miyav()** adlı **static** bir metot oluşturulmuştur. Metot **static** olduğu için, herhangi bir nesne oluşturmaya gerek olmadan **Main()** metodunun içinde kullanılmıştır. Hemen şunu da belirtmekte fayda var; bir nesne oluşturarak **static** metotlara erişemeyiz. Örneğin, aşağıdaki kodları yazarak **Kedi** sınıfındaki **static** özelliğe sahip **Miyav()** metodunu kullanamayız. Böyle bir kod yazmaya kalkarsanız Visual Studio zaten **Kedi** sınıfına ait bu metodu listelemez. Yazsanız bile altını kırmızıyla çizerek sizi uyarır. Buna rağmen derlerseniz "**Member Kedi.Miyav() cannot be accessed ...**" şeklinde hata alırsınız.


```

1  using System;
2  namespace statikMetot
3  {
4      class Kedi
5      {
6          public static void Miyav()
7          {
8              Console.WriteLine("Miyaaaaavvvv");
9          }
10     }
11
12     class Program
13     {
14         static void Main(string[] args)
15         {
16             Kedi sarman = new Kedi();
17             sarman.Miyav();
18             Console.Read();
19         }
20     }
21 }
22

```

Şekil 13.29 Hatalı metot kullanımı

Sonuç olarak, **static** metotların doğrudan kendi sınıfları ile, yani, **Sınıf.Metot()** şeklinde kullanılması gerekmektedir. Nesne oluşturarak **static** sınıflara erişemezsiniz.

Statik metotlarla ilgili bilinmesi gereken en önemli konulardan birisi de aynı sınıf içinde bulunan **static metotların** sadece aynı sınıf içindeki diğer **static üyelere** erişebileceğidir. Örneğin, aşağıda görülen ve aynı sınıf içinde olan **Portakal()** adlı statik metottan **Elma()** adlı normal metodu çağırdığınızda hata alırsınız. Bunun nedeni normal metotların nesneler üzerinde işlem yapmasıdır. Normal metotlarda nesnelerin adresleri metoda **this** referansı ile gönderilir. Statik metotlar küresel olarak davrandıkları için **this** metotları yoktur. Bu nedenle static metot içinden normal metot çağıramazsınız. Zaten dikkat edilirse **Elma()** metodunun altı çizilidir.

```

class Metot
{
    public void Elma()           //normal metot
    {
        Console.Write("Elma");
    }

    public static void Portakal() //statik metot
    {
        Elma();
    }
}

```

Şekil 13.30 Statik metotlar normal metotlara erişemez

Yukarıda yazılan kodları aşağıdaki gibi değiştirirseniz, yani her iki metodu da statik yaparsanız, programınız çalışmaya başlar.

```

class Metot
{
    public static void Elma()           //statik metot
    {
        Console.Write("Elma");
    }

    public static void Portakal()      //statik metot
    {
        Elma();
    }
}

```

Şekil 13.31 Bir statik metot ancak sınıf içindeki diğer statik metotları çalıştırabilir

Yukarıda anlatılan konuları özetlemek gerekirse şunları yazabiliriz:

- Aynı sınıftaki statik metot sadece aynı sınıftaki statik metotları çalıştırabilir.
- Aynı sınıftaki normal metot aynı sınıftaki statik metotları çalıştırabilir.
- Statik metot diğer sınıftaki statik metotları **Sınıf.Metot()** sözdizimiyle çalıştırabilir.
- Statik metot diğer sınıftaki normal metotları ancak nesne oluşturduktan sonra çalıştırabilir.

Yukarıdaki maddelerle ilgili uygulamayı aşağıdaki kodlarda bulabilirsiniz.

```

1  using System;
2  namespace statikNormalMetot
3  {
4      class Metot
5      {
6          public static void Elma()           //statik metot
7          {
8              Console.WriteLine("Elma");
9              //Portakal();                    //hata! normal metot çalışmaz
10         }
11
12         public void Portakal()              //normal metot
13         {
14             Console.WriteLine("Portakal");
15             Elma();                          //statik metot çalışır
16         }
17     }
18
19     class Program
20     {
21         static void Main(string[] args)
22         {
23             Metot.Elma();                    //statik metot doğrudan çalıştırıldı
24             Metot m = new Metot();
25             m.Portakal();                     //normal metot nesne ile çalıştırıldı
26             Console.Read();
27         }
28     }
29 }

```

Şekil 13.32 Metotların çalıştırılması

Şu ana kadar yazdığımız her programda kullanılan **Main()** metodu statik olarak tanımlanmıştır. Bu metod bizim yazdığımız programın başlangıç noktasıdır. Dolayısıyla, **Main()** metodu içinden çağırılan tüm metotlar statik olmak zorundadır.

Sabit Değişkenler (const)

Yukarıdaki bilgilere ilave olarak şunu da hatırlatmakta fayda var; değişkenler konusunda öğrendiğimiz sabit değişkenler (const) de statik özelliklere sahiptir. Bir sınıftaki sabit değişkenlere tıpkı statik üyelere ulaştığımız gibi **Sınıf.sabit** sözdizimi ile ulaşabilirsiniz.

```

1  using System;
2  namespace sabitConst
3  {
4      class Sabit
5      {
6          public const double PI = 3.14;
7      }
8
9      class Program
10     {
11         static void Main(string[] args)
12         {
13             Console.WriteLine(Sabit.PI);
14             Console.Read();
15         }
16     }
17 }
18 
```

Şekil 13.33 Sabit değişkenlerin kullanımı

Yukarıdaki kodlarda başka bir sınıf içinde bir sabit değişken tanımlanmış ve tıpkı statik alanlarda olduğu gibi **Sınıf.sabit** şeklinde kullanılmıştır.

Statik Yapıcı Metotlar

Tıpkı normal metotlar gibi yapıcı metotlar da statik olabilir. Statik yapıcı metotlar herhangi bir sınıftan ilk nesne oluşturulduğunda yapıcı metottan önce otomatik olarak sadece bir defa çalışır. Sınıftan oluşturulan ikinci ve daha sonraki nesnelerde çalışmaz. Oysa, hatırlayacağınız üzere, normal yapıcı metotlar sınıftan her nesne oluşturulduğunda otomatik olarak çalıştırılıyordu. Statik yapıcı metotların kullanım amacı genellikle sınıftaki statik değişkenlere değer atamaktır. Statik olmayan değişkenlere statik yapıcı metotlarla erişmek mümkün değildir.

Statik yapıcı metotların oluşturulması tıpkı normal yapıcı metotlarda olduğu gibidir. Farklı olarak metodun başına **static** komutu gelir. Aşağıdaki kodlarda statik yapıcı metotların tanımlanmasına bir örnek görebilirsiniz.

```

1      using System;
2      namespace statikYapici
3      {
4          class Koordinat
5          {
6              public static int X { get; set; }
7              public static int Y { get; set; }
8
9              static Koordinat()
10             {
11                 X = 0;
12                 Y = 0;
13             }
14         }
15     }

```

Şekil 13.34 Statik yapıcı metot

Yukarıdaki kodlardan da anlaşılacağı üzere, statik olarak **X** ve **Y** adlı özellikler oluşturulmuş, statik olarak tanımlanan yapıcı metot ile bu özelliklere değerler atanmıştır.

Statik yapıcı metotlar sınıftan ilk nesne oluşturulduğunda yapıcı metottan önce otomatik olarak sadece bir defa olmak üzere çalışırlar. Aynı sınıftan oluşturulan diğer nesnelerde çalışmaz. Bu durumu aşağıdaki kodlarda görebilirsiniz.

```

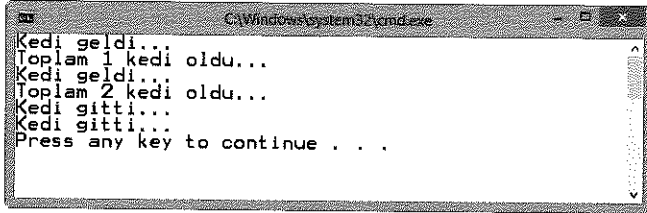
1      using System;
2      namespace statikYapiciDegisken
3      {
4          class Kedi
5          {
6              public static int kediSayisi;
7
8              //statik yapıcı metot ile statik değişken sıfırlanır
9              static Kedi() { kediSayisi = 0; }
10
11              //yapıcı metot ile her nesne oluşturulduğunda kedi sayısı artar
12              public Kedi() { Console.WriteLine("Kedi geldi..."); kediSayisi++; }
13
14              //yıkıcı metot ile her nesnenin işi bittiğinde kedi sayısı azalır
15              ~Kedi() { Console.WriteLine("Kedi gitti..."); kediSayisi--; }
16          }
17          class Program
18          {
19              static void Main(string[] args)
20              {
21                  //ilk nesne oluşturulduğunda önce statik sonra normal yapıcı çalışır
22                  Kedi k1 = new Kedi();
23                  Console.WriteLine("Toplam " + Kedi.kediSayisi + " kedi oldu...");
24                  //ikinci ve diğer nesnelerde sadece normal yapıcılar çalışır
25                  Kedi k2 = new Kedi();
26                  Console.WriteLine("Toplam " + Kedi.kediSayisi + " kedi oldu...");
27              }
28          }
29      }

```

Şekil 13.35 Statik yapıcı metotların kullanımı

Yukarıdaki kodlarda **Kedi** adında bir sınıf oluşturulmuştur. Bu sınıfın **kediSayisi** adlı bir statik değişkeni/alanı, **Kedi()** adlı statik yapıcı, yapıcı ve yıkıcı metotları vardır.

Main() metodu içinde **Kedi** sınıfından ilk nesne oluşturulduğunda hem statik yapıcı hem de normal yapıcı metod çalıştırılacaktır. Statik yapıcı metodun çalışması **kediSayisi** değişkeninin sıfırlanmasına neden olacaktır. **Kedi** sınıfından ikinci nesne oluşturulduğunda ise sadece normal yapıcı metod çalıştırılacaktır. Kod bloğunun sonunda, yani nesnelerin işi bittiğinde ise yıkıcı metodlar çalışacak ve kedi sayısının azalmasına neden olacaktır. Yukarıdaki kodları yazıp **Ctrl+F5** ile (**debug** yapmadan) çalıştırırsanız aşağıdaki görüntüyü elde edersiniz.



```

C:\Windows\System32\cmd.exe
Kedi geldi...
Toplam 1 kedi oldu...
Kedi geldi...
Toplam 2 kedi oldu...
Kedi gitti...
Kedi gitti...
Press any key to continue . . .

```

Şekil 13.36 Kodların çıktısı

Yukarıdakine benzer bir yapıyı bir sohbet odasında bulunan kullanıcı sayısını bulmak için de kullanabilirsiniz. İlk kullanıcı sohbet odasına girdiğinde (yani ilk nesne oluşturulduğunda) önce statik yapıcı metod çalışır ve sohbet odasındaki kişi sayısını sıfırlar. Daha sonra normal yapıcı metod çalışarak sohbet odasındaki kişi sayısını 1 artırır. Daha sonra oluşturulacak nesnelerde statik yapıcı metod çalışmaz, sadece yapıcı metod çalışarak kişi sayısını 1 artırır. Sohbet odasından her oyuncu çıkışında (yani nesne silindiğinde) ise yıkıcı metod çalışarak sohbet odasındaki kişi sayısını 1 azaltır.

Static Sınıflar

Şu ana kadar oluşturduğumuz sınıfların içinde hem normal üyeler hem de statik üyeler kullandık. Statik sınıfların içinde ise sadece statik üyeler kullanabiliriz. Eğer sınıfın içinde sadece statik üyeler tanımlansın şeklinde bir zorlama koymak isterseniz sınıfınızı statik olarak tanımlayabilirsiniz.

Statik sınıfların tanımlanması tıpkı diğer sınıfların tanımlanması gibidir; sadece sınıfın başına **static** anahtar kelimesi gelir.

```

static class Statik
{
    //statik değişkenler
    //statik metodlar
}

```

Statik olarak tanımlanan sınıfların kullanılabilmesi için herhangi bir nesne oluşturmaya gerek yoktur; doğrudan **Sınıf.alan** veya **Sınıf.Metot()** şeklinde kullanılabilirler. Buna örnek kodları aşağıda bulabilirsiniz. Buradaki örnekte **Hesap** adlı statik sınıf oluşturulmuş ve sınıf içinde statik **Kare** ve **Faktoriyel** metotları tanımlanmıştır. **Main()** metodunun içinde de **Sınıf.Metot()** şeklinde, herhangi bir nesne oluşturulmaya gerek kalmadan, doğrudan sınıfın statik metotları kullanılmıştır.

```

1  using System;
2  namespace statikSınıf
3  {
4      static class Hesap
5      {
6          static public int Kare(int sayi)
7          {
8              return sayi * sayi;
9          }
10
11         static public int Faktoriyel(int sayi)
12         {
13             int fakto = 1;
14             for (int i = 1; i <= sayi; i++)
15                 fakto = fakto * i;
16             return fakto;
17         }
18     }
19
20     class Program
21     {
22         static void Main(string[] args)
23         {
24             Console.WriteLine(Hesap.Kare(2));
25             Console.WriteLine(Hesap.Faktoriyel(3));
26         }
27     }
28 }
29

```

Şekil 13.37 Statik sınıfların kullanımı

Aslında yukarıdaki gibi statik sınıfları şu ana kadar bolca kullandık ama farkında değildik; örneğin, **Math**, **Array**, **String** ve **DateTime** sınıfları statik sınıflardır. Bu sınıfların metotlarını doğrudan kodlarımızın içine yazdık. Bu sınıfların bazılarını aşağıda bulabilirsiniz.

Aşağıda statik **Math** sınıfına ait bazı statik özellik ve metotlar listelenmektedir.

- Math.PI
- Math.E
- Math.Max()
- Math.Abs()

- `Math.Pow()`
- `Math.Sqrt()`

Aşağıda statik **Array** sınıfına ait bazı statik özellik ve metotlar listelenmektedir.

- `Array.Length`
- `Array.Rank`
- `Array.IsFixedSize`
- `Array.Sort()`
- `Array.Reverse()`

Aşağıda statik **String** sınıfına ait bazı statik özellik ve metotlar listelenmektedir.

- `String.Length`
- `String.Concat()`
- `String.Equals()`
- `String.Trim()`
- `String.ToUpper()`

Aşağıda statik **DateTime** sınıfına ait bazı statik özellik ve metotlar listelenmektedir.

- `DateTime.Date`
- `DateTime.Day`
- `DateTime.Hour`
- `DateTime.Add()`
- `DateTime.GetType()`

Aşağıda statik **Console** sınıfına ait bazı statik özellik ve metotlar listelenmektedir.

- `Console.BackgroundColor`
- `Console.ForegroundColor`
- `Console.Title`
- `Console.Write()`
- `Console.WriteLine()`
- `Console.ReadLine()`

Yukarıda bulunan statik sınıfların bazılarını kod içinde kullanmak gerekirse aşağıdaki gibi bir örnek olabilir. Dikkat edilirse buradaki sınıfların özellik ve metotları herhangi bir nesne tanımlamadan doğrudan program içinde kullanılmıştır. Bunun nedeni bu sınıfların statik olmalarıdır.

```

1  using System;
2  namespace statikSinifOrnek
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              string metin = "deneme";
9              int sayi = 2;
10
11              Console.WriteLine("Pi Değeri:" + Math.PI);
12              Console.WriteLine("Sayının Karesi:" + Math.Pow(sayi, 2));
13              Console.WriteLine("Metin Uzunluğu:" + metin.Length);
14              Console.WriteLine("Tarih & Saat:" + DateTime.Now);
15          }
16      }
17  }

```

Şekil 13.38 Statik sınıfların kullanımı

Statik üye ve sınıflar programcıya kolaylık olsun diye geliştirilmişlerdir; herhangi bir nesne oluşturmadan doğrudan sınıfın üyelerine ulaşmamızı ve zamandan kazanmamızı sağlar.

This Göstericisi

this göstericisi sınıf içindeki üyelere erişmek için kullanılır. Bazı durumlarda yerel değişken adları ile sınıf değişkenlerinin adları aynı olabilmektedir. Böyle bir durumla karşılaşıldığında değişkenleri doğru bir şekilde kullanabilmek için **this** göstericisi kullanılır. Normalde metotlar yerel değişkenlere öncelik tanır.

Aşağıdaki örneği incellerseniz ne demek istediğimi anlayabilirsiniz.

```

class Daire
{
    public double r;
    public double alan(double yaricap)
    {
        double r = yaricap;           //yerel değişken
        this.r = yaricap;              //sınıf değişkeni
        return Math.PI * r * r;
    }
}

```

Şekil 13.39 this göstericisinin kullanımı

Yukarıdaki kodlarda **r** adında iki değişken bulunmaktadır. Bu değişkenlerden ilki olan **r** değişkeni yerel değişkeni gösterirken ikinci **r** değişkeni **alan** metodunun hemen üzerinde tanımlanan sınıf üyesini göstermektedir. Bu şekilde, **alan** metodu içinde tanımlanan yerel **r** değişkeni ile sınıf içinde tanımlanan küresel **r** değişkeni karışmamaktadır.

Sınıf Uygulamaları

Bilişim eğitiminde nesne tabanlı programlama ve sınıf konusu en zor anlaşılan konulardan biridir. Bu nedenle konu teorik olarak iyi anlaşılmalı ve teorik bilgileri pekiştirmek için bol bol uygulama yapılmalıdır. Bu bölümde yapacağımız uygulamalarla öğrendiğimiz konuları pekiştirmeye çalışacağız. Lütfen, kolay olmasa da aşağıdaki uygulamaları yapın ve anlamadığınız yer olduğunda bölüm içindeki konuları tekrar edin.

Dikdörtgen Sınıfı ve Nesneleri

Bir dikdörtgenin yükseklik ve genişliğini kullanıcıdan alıp alanını hesaplayan sınıfı tanımlayalım. Sınıfı oluştururken girilen yükseklik ve genişlik değerlerinin sıfırdan büyük olmasını kontrol edelim.

Dikdörtgen sınıfını aşağıdaki gibi yazabiliriz.

```

1  using System;
2  namespace uygulamaSinifDikdortgen
3  {
4      class Dikdortgen
5      {
6          public int yukseklik;    //alan
7          public int genislik;    //alan
8
9          public int Yukseklik    //özellik
10         {
11             get { return yukseklik; }
12             set { if (value > 0) yukseklik = value; }
13         }
14
15         public int Genislik      //özellik
16         {
17             get { return genislik; }
18             set { if (value > 0) genislik = value; }
19         }
20
21         //dikdörtgenin alanını hesaplamaya yarayan metot
22         public int alan(int genislik, int yukseklik)
23         {
24             return genislik * yukseklik;
25         }
26     }
27

```

Şekil 13.40

Dikdörtgen sınıfından oluşturulan nesne ve nesnenin özellik ve metodunun kullanımı aşağıdaki gibi olabilir.

```

class Program
{
    static void Main(string[] args)
    {
        //Dikdörtgen sınıfından bir nesne oluşturalım
        Dikdortgen d = new Dikdortgen();

        //Yükseklik değerinin okuyalım
        Console.Write("Yükseklik:");
        d.Yukseklık = Convert.ToInt16(Console.ReadLine());

        //Genislik değerini okuyalım
        Console.Write("Genislik:");
        d.Genislik = Convert.ToInt16(Console.ReadLine());

        //Dikdörtgenin alanını hesaplayalım
        Console.WriteLine("Dikdörtgen Alanı=" + d.alan(d.Yukseklık, d.Genislik));
    }
}

```

Şekil 13.41

Statik Dikdörtgen Sınıfı

Yukarıda yazdığımız sınıfı statik olarak yazalım ve nesne oluşturmada özellik ve metodunun kullanımını görelim. Statik dikdörtgen sınıfı aşağıdaki gibi yazabiliriz.

```

static class Dikdortgen
{
    public static int yukseklik;        //alan
    public static int genislik;        //alan

    public static int Yukseklik        //özellik
    {
        get { return yukseklik; }
        set { if (value > 0) yukseklik = value; }
    }

    public static int Genislik        //özellik
    {
        get { return genislik; }
        set { if (value > 0) genislik = value; }
    }

    //dikdörtgenin alanını hesaplamaya yarayan metod
    public static void alan(int genislik, int yukseklik)
    {
        Console.WriteLine("Alan:" + genislik * yukseklik);
    }
}

```

Şekil 13.42

Statik sınıfın kullanımını aşağıdaki gibi olabilir.

```

class Program
{
    static void Main(string[] args)
    {
        //Yükseklik değerini okuyalım
        Console.Write("Yükseklik:");
        Dikdortgen.Yukseklik = Convert.ToInt16(Console.ReadLine());

        //Genişlik değerini okuyalım
        Console.Write("Genişlik:");
        Dikdortgen.Genislik = Convert.ToInt16(Console.ReadLine());

        //Dikdörtgenin alanını hesaplayalım
        Dikdortgen.alan(Dikdortgen.Genislik, Dikdortgen.Yukseklik);
    }
}

```

Şekil 13.43

Yapıcı Metot

Bu örneğimizde **Daire** sınıfından bir nesne oluşturulduğunda "**Daire oluşturuldu**", nesne silindiğinde ise "**Daire hafızadan silindi**" yazan yapıcı ve yıkıcı metotlar oluşturacağız. Bu işlemi yapan **Daire** sınıfı ve **Main()** metodundaki kodları aşağıdaki gibi olabilir. Kodları **F5** ile değil **Ctrl+F5** ile test ediniz.

```

1  using System;
2  namespace uygulamaYapiciYikici
3  {
4      class Daire
5      {
6          //yapıcı metot nesne oluşturulduğunda çalıştırılır
7          public Daire()
8          {
9              Console.WriteLine("Daire oluşturuldu...");
10         }
11
12         //yıkıcı metot nesnenin çalışması bittiğinde çalıştırılır
13         ~Daire()
14         {
15             Console.WriteLine("Daire hafızadan silindi...");
16         }
17     }
18
19     class Program
20     {
21         static void Main(string[] args)
22         {
23             Daire d = new Daire();
24         }
25     }
26 }

```

Şekil 13.44

Statik Yapıcı Metot Uygulaması

Bir sohbet odası yazdığınızı düşünün. Sohbet odasına giren her bir kullanıcı için bir nesne oluşturulmaktadır. Herhangi bir kullanıcı sohbet odasına girdiğinde sayıyı artıran, sohbet odasından çıktığında ise sayıyı azaltan statik yapıcı, yapıcı ve yıkıcı metotlar yazalım. Sınıfın kodları aşağıdaki gibi olabilir. Kodları test etmek için **F5** yerine **Ctrl+F5** tuşlarını kullanın.

```

1  using System;
2  namespace uygulamaStatikYapici
3  {
4      class User
5      {
6          //statik değişken
7          public static int count;
8
9          //statik yapıcı metot
10         static User()
11         {
12             count = 0;
13         }
14
15         //yapıcı metot
16         public User()
17         {
18             count++;
19         }
20
21         //yıkıcı metot
22         ~User()
23         {
24             count--;
25         }
26     }

```

Şekil 13.45

Main() metoduna yazılacak kodlar aşağıdaki gibi olabilir.

```

class Program
{
    static void Main(string[] args)
    {
        //ilk nesne (kullanıcı) oluşturulduğunda hem statik
        //hem de normal yapıcı metotlar çalışır. count=0 olur
        User u1 = new User();
        Console.WriteLine("Kullanıcı Sayısı:" + User.count);

        //ikinci kullanıcı oluşturulduğunda sadece yapıcı çalışır
        User u2 = new User();
        Console.WriteLine("Kullanıcı Sayısı:" + User.count);
    }
    //nesnelerin işi bittiğinde yıkıcılar çalışır
}

```

Şekil 13.46

Statik Matematik Metotları

Daha önce de öğrendiğimiz gibi, statik metotlar herhangi bir nesne oluşturmaya gerek kalmadan doğrudan **Sınıf.Metot()** şeklinde kullanılırlar. Bu işlem, kolaylığın yanında daha hızlı kod yazmamızı da sağlar. Bu örneğimizde **toplama** ve **çıkarma** işlemleri için iki statik metot oluşturacağız ve bu metotları **Main()** metodunun içinde kullanacağız.

```

1  using System;
2  namespace uygulama5StatikToplaCikar
3  {
4      class Hesap
5      {
6          public static int Topla(int a, int b) { return a + b; }
7          public static int Cikar(int a, int b) { return a - b; }
8      }
9
10     class Program
11     {
12         static void Main(string[] args)
13         {
14             int s1, s2;
15
16             //ilk sayıyı okuyalım
17             Console.Write("Sayı1:"); s1=Convert.ToInt16(Console.ReadLine());
18
19             //ikinci sayıyı okuyalım
20             Console.Write("Sayı2:"); s2 = Convert.ToInt16(Console.ReadLine());
21
22             //toplama işlemi
23             Console.WriteLine("Toplam="+ Hesap.Topla(s1, s2));
24
25             //çıkarma işlemi
26             Console.WriteLine("Toplam=" + Hesap.Cikar(s1, s2));
27         }
28     }
29 }

```

Şekil 13.47

14

Kalıtım & Polimorfizm

- Kalıtım Nedir?
- Temel & Türetilmiş Sınıflar
- Türetilmiş Sınıflarda Erişim Denetimi & Kalıtım
- Yapıcı Metotlar & Kalıtım
- Sealed Sınıflar
- Polimorfizm & Sanal Metotlar
- Özet Sınıflar
- Arayüzler
- IEnumerable
- Uygulamalar

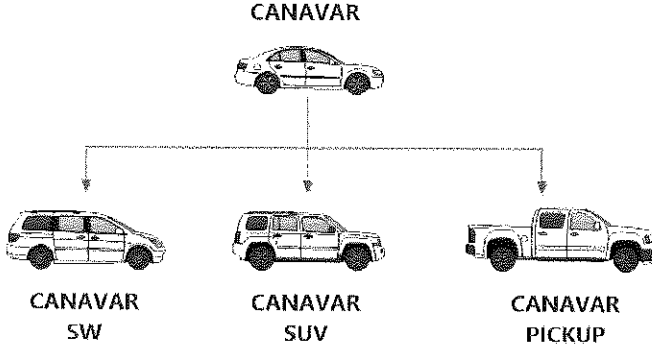
Kalıtım (Inheritance) Nedir?

Kalıtım konusunu ilkokuldan beri öğreniyoruz. Nedir kalıtım? Anne ve babamızdan bize geçen genetik/kalıtsal özelliklerdir. Örneğin anne-babamızın iki ayağı, iki kolu, bir kafası ve daha birçok kalıtsal özellikleri vardır. Bu kalıtsal özelliklerin çoğu veya bir kısmı istesek de istemesek de bize geçmektedir.

Eğer başka şeylerden de örnek verecek olursak arabaları örnek gösterebiliriz. **Bilimsel** firmasının **Canavar** adlı bir araba ürettiğini düşünün. Aynı firma daha sonra **Canavar** adlı arabanın alt yapısını ya da çoğu parçasını kullanarak **Canavar SW**, **Canavar SUV**, **Canavar Pickup** gibi modeller üretebilmektedir. Dolayısıyla, sonradan üretilen diğer tüm modeller ilk modelin kalıtsal özelliklerini taşırlar. Daha son-

radan türetilen arabalar örneğin ilk arabanın motorunu, şasisini veya direksiyonunu kullanabilirler.

Yukarıdaki örnekten de anlaşılacağı gibi, kalıtım sayesinde hem daha hızlı üretim yapılır hem de daha isteğe özel ürünler geliştirilmiş olur.

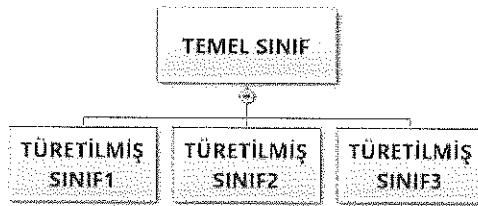


Şekil 14.1 Kalıtım

İnsanlarda, arabalarda ve diğer birçok canlı-cansız varlıkta görülen **özelliklerin yeni nesillere taşınması** olayı programcılıkta da karşımıza çıkmaktadır. Tıpkı arabalarda olduğu gibi, yazılımda da daha önce üretilmiş bir sınıftan yeni sınıflar türetilir. Böylece mevcut altyapı/kodlar kullanılarak çok daha hızlı üretim yapılabilir.

Kalıtım (inheritance) nesne tabanlı programlamanın en önemli konularından biridir. Kalıtım sayesinde elimizde bulunan bir sınıfı temel/ebeveyn sınıf kabul ederek o sınıftan türetilmiş sınıflar elde edebiliriz. Elde edilen türetilmiş sınıfları istediğimiz şekilde değiştirebiliriz; eğer istersek bazı kodları silebilir veya yeni kodlar ve özellikler ekleyebiliriz. Böylece az çaba sarf ederek çok iş yapmış oluruz.

Aşağıdaki şekilde bir ana sınıf ve bu ana sınıftan türetilmiş sınıflar görüyorsunuz. Türetilmiş sınıflar ana sınıfın tüm veri ve metotlarını kullanabilirler. Ayrıca, yazılacak ilave kodlarla yeni özelliklere de sahip olabilirler.



Şekil 14.2 Ana sınıf ve türetilmiş sınıflar

Büyük yazılım projelerinde genellikle önce bir temel sınıf ya da çerçeve (framework) oluşturulur. Daha sonra da program küçük parçalara yani modüllere bölünür. Her modül bir ya da daha fazla programcı tarafından yazılır. Modülleri yazmaya başlayan programcılar temel sınıfı kullanarak alt sınıflar ya da türetilmiş sınıflar denilen yapıları oluştururlar. Programcılar türetilmiş sınıfları kullanarak hem belli bir temel mantığı kullanarak kod yazarlar hem de daha hızlı ürün geliştirirler. Çünkü temel sınıfta ihtiyaç duyulan çoğu veri ve metot kendilerine hazır olarak verilir.

Sınıf Türetme

Bir temel sınıfı kullanarak alt sınıflar ya da türetilmiş sınıflar oluşturabiliriz. Türetilmiş sınıf oluşturmak için aşağıdaki sözdizimini kullanıyoruz:

```
class Temel
{
    //temel sınıf kodları
}

erişim_belirtecisi class Türetilmiş: Temel
{
    //türetilmiş sınıf kodları
}
```

Türetilmiş sınıflar kullanılan erişim belirtecine göre temel sınıfın tüm veri ve metotlarına erişebilir. Ayrıca, türetilmiş sınıfa değişik görevleri gerçekleştirmek için farklı kodlar da yazılabilir.

Örnek Kod:

En basit haliyle türetme aşağıdaki gibi olabilir. Buradaki örnekte **Hayvan** sınıfından bir **Kedi** sınıfı türetiyoruz.

```
class Hayvan
{
}

class Kedi:Hayvan
{
}
```

Örnek Kod:

Yukarıdaki kodları biraz daha geliştirecek olursak aşağıdaki gibi bir program elde edebiliriz. Bu programda **Hayvan** temel sınıf olurken, **Kedi**, **Hayvan** sınıfından türetilmektedir. Dolayısıyla, **Hayvan** sınıfında bulunan **public** tipindeki **ad** ve **yas** değiş-

kenleri kalıtımla **Kedi** sınıfına geçmektedir ve burada da **public** olarak davranmaktadır. Yani bu değişkenler sanki **Kedi** sınıfında tanımlanmış gibi kullanılabilir. Kalıtımla gelen kodların dışında **Kedi** sınıfında ayrıca **Kimlik()** adlı bir metod oluşturulmuş ve kedi özelliklerinin ekrana yazılması sağlanmıştır. Son olarak, **Main()** metodunda **Kedi** sınıfından bir **k** nesnesi oluşturularak kedinin adı ve yaşı alındıktan sonra **Kimlik()** metodu ile ekrana yazılmıştır.

```
using System;
namespace hayvanKediSinif
{
    class Hayvan
    {
        public string ad;
        public int yas;
    }

    class Kedi : Hayvan
    {
        //kalıtım yoluyla Hayvan sınıfının alanları bu sınıfa aktarıldı
        public void Kimlik()
        {
            Console.WriteLine("Ad:" + ad + " Yaş:" + yas);
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Kedi k = new Kedi();
            k.ad = "tırtıl"; //ad değişkeni Hayvan ana/temel sınıfından
            k.yas = 2;        //yas değişkeni Hayvan ana/temel sınıfından
            k.Kimlik();        //Kimlik() metodu türetilmiş Kedi sınıfından
        }
    }
}
```

Örnek Kod:

Tipik bir türetilmiş sınıf örneği aşağıdaki gibidir. Bu örnekte **Sekil** adında bir sınıf oluşturulmuştur. **Sekil** adlı sınıfın **yukseklık** ve **genislik** alanları ile bir adet **alan()** adlı metodu bulunmaktadır. Aynı parametreler üçgen ve dikdörtgen gibi şekiller için de geçerli olduğundan, **Sekil** sınıfından **Ucgen** adlı bir nesne türetilmiştir.

```

class Sekil
{
    public int yukseklik;
    public int genislik;
    public int alan()
    {
        return yukseklik * genislik;
    }
}

class Ucgen:Sekil
{
    //yukseklik ve genislik kalıtımla gelir
    //alan() metodu kalıtımla gelir
}

```

Yukarıdaki örnekte oluşturulan sınıfları kullanan bir program yazacak olursak aşağıdaki gibi bir program çıkarabiliriz. **Sekil** sınıfından türetilen **Ucgen** sınıfı **Main()** metodunda üçgenin alanını hesaplamak için kullanılmaktadır.

```

1  using System;
2  namespace turetme
3  {
4      class Sekil
5      {
6          public int yukseklik;
7          public int genislik;
8          public int alan()
9          {
10             return yukseklik * genislik;
11         }
12     }
13
14     class Ucgen:Sekil
15     {
16     }
17
18
19     class Program
20     {
21         static void Main(string[] args)
22         {
23             Ucgen u = new Ucgen();
24             u.yukseklik = 1;
25             u.genislik = 2;
26             Console.WriteLine("Alan=" + u.alan());
27         }
28     }
29 }

```

Şekil 14.3 Türetilmiş sınıfların kullanımı

Örnek Kod:

Bu örneğimizde çok basit bir kalıtım uygulaması yapacağız ve yukarıda anlatılan teorik konuları pekiştirmeye çalışacağız. Öncelikle **Temel** adlı bir sınıf oluşturacağız. Daha sonra da **Temel** sınıftan **Turetilmis** adlı bir türetilmiş sınıf oluşturacağız. Son olarak, **Program** sınıfı içinde **Turetilmis** sınıfından **t** adlı bir nesne oluşturacağız ve nesnenin özellik ve metodunu yazdırarak kalıtımın sonucunu göreceğiz.

```

1  using System;
2  namespace turetme
3  {
4      class Temel
5      {
6          public int deg_temel=10;
7          public void met_temel() { Console.Write("Temel sınıftan yazılır!"); }
8      }
9
10     class Turetilmis : Temel
11     {
12         public int deg_turetilmis = 5;
13         public void met_turetilmis() { Console.Write("Türetilmiş sınıftan yazılır!"); }
14     }
15
16     class Program
17     {
18         public static void Main(string[] args)
19         {
20             Turetilmis t = new Turetilmis();
21             Console.WriteLine(t.deg_temel);
22             t.met_temel();
23             Console.Read();
24         }
25     }
26 }

```

Şekil 14.4 Türetilmiş sınıfın oluşturulması

Yukarıdaki kodlarda **Program** sınıfı içinde **Turetilmis** sınıfından **t** adlı bir nesne oluşturulmuştur. Daha sonra **t.deg_temel** değişkeni ve **t.met_temel()** metodu yazdırılmış ve bu değerlerin **Temel** sınıfındaki üyelerden geldiği görülmüştür. Programın çıktısı aşağıdaki gibidir:

```

10
Temel sınıftan yazılmıştır!

```

Yukarıdaki çıktıda görülen 10 değeri **Temel** sınıfın **deg_temel** değişkeninden, alt satırda yazdırılan metin ise **Temel** sınıfın **met_temel()** metodundan kalıtım yoluyla gelmiştir.

Örnek Kod:

Aşağıdaki kodlarda **Sekil** adlı bir temel sınıf bulunmaktadır. **Sekil** adlı temel sınıftan **Dikdortgen** ve **Ucgen** adlı iki türetilmiş sınıf oluşturulmaktadır. **Dikdortgen** ve **Ucgen** adlı türetilmiş sınıflar **Sekil** adlı temel sınıftan türetildiği için **Sekil** temel sınıfının özellik ve metodlarına erişerek işlem yapabilmektedir.

```

1  using System;
2  namespace sekil
3  {
4      class Sekil
5      {
6          public double yukseklik = 0;
7          public double genislik = 0;
8          public double alan()
9          { return genislik * yukseklik; }
10     }
11     class Ucgen :Sekil { }
12     class Dikdortgen :Sekil { }
13     class Program
14     {
15         static void Main(string[] args)
16         {
17             Ucgen u = new Ucgen();
18             u.genislik = 5.5; u.yukseklik = 2.4;
19             Console.WriteLine("Üçgen Alanı=" + u.alan());
20
21             Dikdortgen d = new Dikdortgen();
22             d.genislik = 3.0; d.yukseklik = 2.2;
23             Console.WriteLine("Dikdörtgen Alanı=" + d.alan());
24             Console.Read();
25         }
26     }
27 }

```

Şekil 14.5 Türetilmiş sınıfların kullanımı

Yukarıdaki kodlarda türetilen **Ucgen** sınıfından **u** nesnesi, **Dikdortgen** sınıfından da **d** nesnesi oluşturulmuştur. Daha sonra **u** nesnesinin yükseklik ve genişlik değerleri, **d** nesnesinin de genişlik ve yükseklik değerleri atanmış ve ilgili metotlar kullanılarak nesnelerin alanları hesaplanmıştır.

Türetilmiş Sınıflarda Erişim Denetimi

Daha önce de öğrendiğimiz gibi, sınıf içinde bulunan üyelere erişmek için erişim belirteçleri kullanılır. Bunlar **private**, **public** ve **protected** adlı anahtar kelimelerdir. Bu anahtar kelimeler sayesinde sınıfın bazı üyelerine sınıf dışından erişilebilirken bazı üyelerine ise sınıf dışından erişilemez. Erişim belirteçlerinin kullanımı aşağıdaki gibidir:

```

class Temel
{
    private int degisken1;
    public double degisken2;
    protected string degisken3;
}

```

public belirtecine sahip üyelere programın her yerinden erişilebilir. Yani, **public** belirteciyle yazılan değişken ve metotlar sınıf dışındaki kodlarda kullanılabilir ve istediğiniz değerleri atayabilirsiniz.

Eğer sınıf içindeki bazı üyeleri sınıf dışındaki kodların erişimine kapatmak istiyorsanız **private** belirtecini kullanabilirsiniz. Yani, **private** belirteci ile tanımlanan üyelere sadece sınıf içinden ulaşılabilir. Varsayılan olarak sınıfın tüm üyeleri **private** kabul edilir. Eğer bir sınıf üyesini tanımlarken herhangi bir belirteç kullanmazsanız **private** olur ve bu üyeye sınıf dışından erişilemez.

protected olarak belirtilen üyeler daha çok **private** gibi davranır. Bu tip üyelere de tıpkı **private** üyelere olduğu gibi dışarıdan erişilemez. **Protected** üyelere sadece türetilmiş sınıflar erişebilir.

Yukarıda anlatılan üye erişim belirteçleriyle beraber türetilmiş sınıfta kullanılan erişim belirteci de türetilmiş sınıftaki üyelerin durumunu etkiler. Örneğin, aşağıdaki kodlarda türetilmiş sınıfın erişim belirteci **public** olarak uygulanmıştır. Duruma göre türetilmiş sınıfın erişim belirteci **private** veya **protected** yapılabilir.

```
class Temel
{
    private int degisken1;
    public double degisken2;
    protected string degisken3;
}

public Turetilmis:Temel
{
    private int test;
}
```

Sınıf türetirken kullanılan belirteçlerin işlevi aşağıdaki tablodan görülebilir. Dikkat edilirse türetilmiş sınıftaki kodlar temel sınıftaki **private** üyelere kesinlikle erişememektedir. Türetilmiş sınıfları **public** belirteci ile oluşturursanız temel sınıftaki **private** üyeler hariç diğer üyeler olduğu gibi gelmektedir. Genelde de bu erişim belirteci kullanılmaktadır.

Türetilen Sınıfın Erişim Belirtecisi	Açıklama
public	Temel sınıfın public üyeleri türetilmiş sınıfın da public üyeleri olurlar. Aynı şekilde temel sınıfın protected üyeleri türetilmiş sınıfın da protected üyeleri olurlar. Türetilmiş sınıftaki kodlar temel sınıftaki private üyelere erişemezler.
protected	Temel sınıftaki public ve protected kodlar türetilmiş sınıfın protected üyeleri olurlar.
private	Temel sınıftaki public ve protected kodlar türetilmiş sınıfın private üyeleri olurlar.

Eğer temel sınıftaki **public** ve **protected** üyelerin olduğu gibi (**public** ve **protected** olarak) türetilmiş sınıfa aktarılmasını istiyorsanız türetilmiş sınıfı **public** belirtecisiyle ve aşağıdaki gibi tanımlamanız gerekiyor. Bu durumda türetilmiş sınıftan oluşturulan nesneler aktarılan **public** ve **protected** üyelere erişebileceklerdir. Türetilmiş sınıftaki **public** üyelere sınıf dışındaki kodlar da erişebilecektir.

```
class TemelSınıf
{
    //temel sınıf kodları
}

public class TüretilmişSınıf: TemelSınıf
{
    //türetilmiş sınıf kodları
}
```

Eğer temel sınıftaki **public** ve **protected** üyelerin türetilmiş sınıfa **protected** olarak aktarılmasını istiyorsanız türetilmiş sınıfı **protected** belirtecisiyle ve aşağıdaki gibi tanımlamanız gerekiyor. Bu durumda türetilmiş sınıf içindeki kodlar kalıtımla gelen üyelere erişebilecek ama türetilmiş sınıf dışındaki kodlar bu üyelere erişemeyecektir.

```
class TemelSınıf
{
    //temel sınıf kodları
}

protected class TüretilmişSınıf: TemelSınıf
{
    //türetilmiş sınıf kodları
}
```

Eğer temel sınıftaki **public** ve **protected** üyelerin türetilmiş sınıfa **private** olarak aktarılmasını istiyorsanız sınıfı **private** belirteciyle ve aşağıdaki gibi tanımlamanız gerekiyor.

```
class TemelSınıf
{
    //temel sınıf kodları
}

private class TüretilmişSınıf: TemelSınıf
{
    //türetilmiş sınıf kodları
}
```

Daha önce de söylediğim gibi, kalıtımda genelde **public** denetleyicisi kullanılır. Böylece temel sınıftaki üyelerin olduğu gibi türetilmiş sınıfa aktarılması sağlanır.

Sealed (Kalıtımsız) Sınıflar

Adından da belli olduğu gibi, **sealed** (sızdırmaz/kalıtımsız) sınıflar türetme yoluyla sınıf yapılmasını engeller. Örneğin, aşağıdaki gibi bir kod yazarsanız hata alırsınız. Çünkü **sealed** anahtar kelimesi **Temel** adlı sınıftan nesne türetilmesini engeller.

```
sealed class Temel
{
    //temel sınıf kodları
}

class Türetilmis:Temel
{
    //türetilmiş sınıf kodları
}
```

Yukarıdaki temel sınıfta kullanılan **sealed** anahtar kelimesi sınıfımızın daha üst düzey güvenliğe sahip olmasını ve diğer sınıflar tarafından kesinlikle türetmeyle kullanılmamasını sağlar. Eğer sınıfınızın başka sınıflarda temel sınıf olarak kullanılmasını engellemek istiyorsanız **sealed** anahtar kelimesini kullanabilirsiniz.

Yapıcı/Yıkıcı Metotlar & Kalıtım

Hatırlanacağı üzere, yapıcı metotlar nesne oluşturulduğunda otomatik olarak çalışan, yıkıcı metotlar ise nesnenin işi bittiğinde otomatik olarak çalışan metotlardır. Bu başlık altında yapıcı metotların türetilmiş sınıflardaki durumunu irdelleyeceğiz.

Kalıtım konusundan öğrendiğimiz gibi, temel sınıftaki bütün özellik ve metotlar türetilmiş sınıfa geçebilmektedir. Temel sınıflardaki metotlar türetilmiş sınıfa geçerken yapıcı metotların durumu nedir? Örneğin, aşağıdaki gibi sınıfların olduğunu düşünelim. **Kedi** sınıfından türettiğimiz bir nesne hangi metotları otomatik olarak çalıştırır?

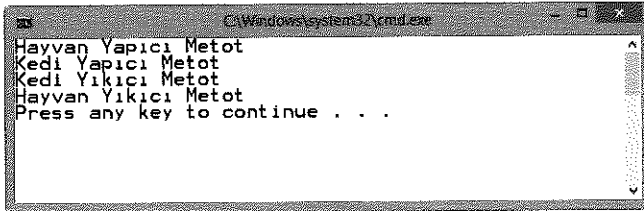
```

1  using System;
2  namespace yapiciTuretilmisSinif
3  {
4      class Hayvan    //Temel Sınıf
5      {
6          public Hayvan() { Console.WriteLine("Hayvan Yapıcı Metot"); }
7          ~Hayvan() { Console.WriteLine("Hayvan Yıkıcı Metot"); }
8      }
9
10     class Kedi:Hayvan    //Türetilmiş Sınıf
11     {
12         public Kedi() { Console.WriteLine("Kedi Yapıcı Metot"); }
13         ~Kedi() { Console.WriteLine("Kedi Yıkıcı Metot"); }
14     }
15
16     class Program
17     {
18         static void Main(string[] args)
19         {
20             //türetilmiş sınıftan oluşturulan nesne
21             Kedi k = new Kedi();
22         }
23     }
24 }

```

Şekil 14.6 Türetilmiş sınıflarda yapıcı ve yıkıcıların durumu

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir ekran çıktısı elde ediyoruz.



Şekil 14.7 Ekran çıktısı

Yukarıdaki ekran çıktısından da anlaşılıyor ki, türetilmiş sınıftan bir nesne oluşturduğumuzda öncelikle temel sınıfın, daha sonra da türetilmiş sınıfın yapıcı metotları çalışıyor. Nesnenin işi bittiğinde ise önce türetilmiş sınıfın, daha sonra da temel

sınıfın yıkıcı metotları çalışıyor. Buradaki sonuçtan da anlaşılıyor ki, türetilmiş sınıfa kalıtım yoluyla ana sınıfın metotları ve özelliklerinin yanında yapıcı/yıkıcı metotları da geçiyor.

Eğer temel sınıfın yapıcı/yıkıcı metodu yoksa doğal olarak bu metotlar türetilmiş sınıfa da geçmeyecek ve türetilmiş sınıftan bir nesne oluşturulduğunda sadece türetilmiş sınıfın yapıcı/yıkıcı metotları çalışacaktır.

Çok Biçimlilik (Polymorphism) ve Sanal Metotlar

Polymorphism Yunanca bir kelimedir ve poly (çok) ile morph (biçim) kelimelerinin birleşiminden oluşur. Adından da belli olduğu gibi, polimorfik nesneler birden fazla biçime/göreve sahip olurlar.

Eğer konuyu biraz daha somut hale getirecek olursak, çok biçimlilik belirli kodların birbiriyle ilgili birden fazla iş için kullanılmasını sağlar. Kabaca, daha önce gördüğümüz konulardan bir örnek vermek gerekirse, şöyle bir örnek olabilir: bir tamsayının (int) karesini alan bir metot yazdıysanız bu metot sadece bir tamsayının karesini alır. Eğer kesirli (double) bir sayının karesini hesaplamaya kalkarsanız hata alırsınız. Fakat çok biçimlilik sayesinde bu metodun farklı sürümlerini yazıp hem bir tamsayının hem de bir kesirli sayının karesini hesaplayabilirsiniz.

Polimorfizm tipik olarak sınıflar arası bir hiyerarşinin olduğu durumlarda kullanılır; kalıtım yoluyla ana sınıftan türetilmiş sınıfa gelen metotların işlevi değiştirilerek daha etkili ve kolay bir yapı sağlanır. Örneğin, kare, dikdörtgen, daire gibi şekiller çizmeye yarayan bir yazılım geliştirdiğinizi düşünün. Böyle bir yazılımda **Sekil** adlı bir temel sınıf ve bu sınıf içinde public belirteciyle **Ciz()** adlı bir metot oluşturup bu metodun türetilmiş sınıflarda kullanılmasını sağlayabilirsiniz. Yalnız, **Ciz()** metodunun türetilmiş sınıflarda farklı işlevlerle çalışması gerekecektir. Örneğin, türetilmiş **Kare** sınıfında kare çizmesi gerekirken, türetilmiş **Daire** sınıfında daire çizmesi gerekecektir. Bu olaya polimorfizm denilmektedir; türetilmiş sınıfa kalıtım yoluyla gelen bir metot farklı işlevler için kullanılır. Polimorfizm amacıyla kullanılan metotların **virtual** anahtar kelimesiyle belirtilmesi gerekmektedir. **virtual** anahtar kelimesine sahip metotlar türetilmiş sınıflarda değiştirilerek (override) farklı işlevler kazanabilir.

Aşağıdaki kodlarda polimorfizme tipik bir örnek görebilirsiniz.

```
class Sekil
{
    public virtual void Ciz() {Console.WriteLine("Temel sınıf metodu...");}
}

class Kare:Sekil
{
    public override void Ciz() {Console.WriteLine("Kare çiz...");}
}

class Daire:Sekil
{
    public override void Ciz() {Console.WriteLine("Daire çiz...");}
}
```

Yukarıdaki örnek kodlarda da görüldüğü gibi, temel sınıfta **virtual** anahtar kelimesiyle tanımlanan **Ciz()** metodu türetilmiş sınıflarda **override** anahtar kelimesiyle tanımlanarak kodları değiştirilmekte ve farklı işlevler için kullanılabilir.

Eğer yukarıdaki sınıfları **Main()** metodu içinde kullanıp nesne oluşturmak isterseniz aşağıdaki kodları yazabilirsiniz.

```
static void Main(string[] args)
{
    Sekil k = new Kare();
    k.Ciz();

    Sekil d = new Daire();
    d.Ciz();
}
```

Dikkat edilirse **Main()** metodu içinde **Kare** nesnesi oluşturulurken **Kare k=new Kare;** komutu kullanılmamıştır. Onun yerine **Sekil k=new Kare();** komutu kullanılmıştır. Aynı durum **Daire** nesnesi için de geçerlidir. Sanal metotlar sayesinde temel sınıflara türeyen sınıf referansları atanır ve temel sınıflar türeyen sınıfların metotlarını çağırabilir.

Sanal sınıfların en önemli özelliği **geç bağlama** (late binding) özelliğini destekliyor olmalarıdır. Sanal sınıflarda çağrılan metodun hangi sınıfa ait olacağı çalışma zamanında belirlenir. Yani, yapılacak işe göre uygun metod seçilir. Metodların çalışma zamanında belirlenmesine **geç bağlama** denilmektedir.

Yukarıdaki kodları bir araya getirirsek aşağıdaki program oluşur.

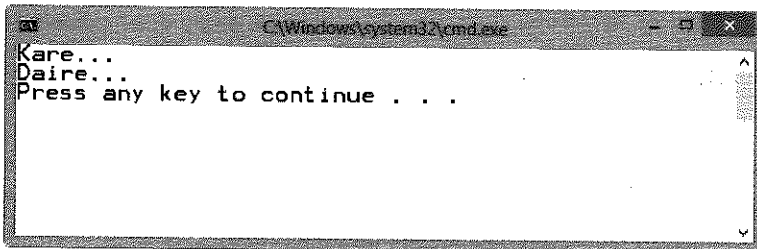
```

1  using System;
2  namespace virtualOverrideKalitim
3  {
4      class Sekil
5      { public virtual void Ciz() { Console.WriteLine("Temel Sınıf"); }
6      }
7
8      class Kare:Sekil
9      { public override void Ciz() { Console.WriteLine("Kare..."); }
10     }
11
12     class Daire:Sekil
13     { public override void Ciz() { Console.WriteLine("Daire..."); }
14     }
15
16     class Program
17     {
18         static void Main(string[] args)
19         {
20             Sekil k = new Kare();
21             k.Ciz();
22
23             Sekil d = new Daire();
24             d.Ciz();
25         }
26     }
27 )

```

Şekil 14.8 Polymorphism

Programın çalışması sonucu aşağıdaki gibi bir ekran elde edilir. Doğal olarak, **k.Ciz();** komutu ekrana **Kare...** yazarken, **d.Ciz();** komutu **Daire...** yazmaktadır.



Şekil 14.9 Ekran çıktısı

Özetlemek gerekirse, polimorfizm sayesinde farklı nesneler için aynı metodu çağırarak farklı işlemler gerçekleştirebiliyoruz. Bu özelliğin çalışabilmesi için ana sınıftaki

metot **virtual** anahtar kelimesiyle tanımlanırken, türetilmiş sınıflarda **override** anahtar kelimesiyle tanımlanır. Polimorfiz yaklaşımı sayesinde her nesneye aynı şekilde davranıyoruz; her nesne **Sekil** tipinde olduğundan bakım ve güncelleme daha kolay olabiliyor.

Polimorfizm kullanımına tipik bir örnek de dövüş oyunlarındaki karakterlerdir. Bu karakterlerin hepsi dövüş yeteneklerine sahiptir ama hepsi birbirinden farklı beceri ve dövüş stiline sahiptir. Bu durumda, **Oyuncu** adlı bir temel sınıf oluşturup sınıf içinde **Dovus()** adlı bir metod tanımlanabilir. **Dovus()** metodu farklı oyuncular için değiştirilerek her oyuncuya farklı yetenekler kazandırılabilir.

Özet (Abstract) Sınıflar ve Metotlar

Bir önceki konumuzda da öğrendiğimiz gibi, aynı temel dınıftan türetilmiş sınıflarda aynı metodu farklı işlevlerle kullanmak istersek polimorfizm yaklaşımını tercih edebiliyoruz. Polimorfizmi, metotları temel sınıfta **virtual** ve türetilmiş sınıfta **override** anahtar kelimeleriyle tanımlayarak oluşturuyoruz.

Bazı durumlarda temel sınıf içinde **virtual** (sanal) anahtar kelimesiyle metod tanımlamaya ve metod içine kod yazmaya ihtiyaç duyulmaz. Bu tür durumlarda **abstract** anahtar kelimesi kullanılarak temel sınıftaki metoda kod yazılmaz; bu metotlara türetilmiş sınıflarda kod yazılması istenir. Aşağıdaki örneği incellerseniz ne demek istediğimi daha iyi anlayabilirsiniz.

```
abstract class Sekil
{
    public abstract void Ciz();
}
```

Yukarıdaki kodlarda hem sınıfın kendisi hem de sınıf içindeki **Ciz()** metodu **abstract** olarak tanımlanmıştır. Bunun nedeni, türetilmiş metotlara kalıtımla geçirilmesi düşünülen **Ciz()** metoduna her sınıfa özel kodlar yazma zorunluluğunun getirilmesidir. Yani, her sınıf kendi **Ciz()** metodunu oluşturacak ve **Ciz()** metoduna kendine özel kodlar yazmak zorunda kalacaktır.

Abstract (**özet**) olarak tanımlanmış sınıflardan nesne oluşturamazsınız. Özet sınıfları sadece türetilmiş sınıf oluşturmak için kullanabilirsiniz. İçinde özet metod bulunan sınıflar özet (abstract) olarak tanımlanmak zorundadır. Yani, aşağıdaki gibi bir tanımlama yaparsanız hata alırsınız. Buradaki kodlarda **Ciz()** metodu özet olarak tanımlanmıştır ama içinde bulunduğu sınıf özet olarak tanımlanmadığı için kodlar hata vermektedir.

```
class Sekil
{
    public abstract void Ciz();
}
```

Sonuç olarak;

- Özet bir metod tanımlayacaksanız sınıfın da özet olması gerekmektedir.
- Özet bir sınıfta birden fazla özet üye olabilir.
- Özet sınıftan türetilen sınıfın içinde, özet sınıftaki tüm özet metodlar tanımlanmak zorundadır.
- Özet sınıflarda **sealed** anahtar kelimesi olamaz. Çünkü bunlar birbirinin zıttı işler yaptırmaya çalışmaktadır. Daha önce de öğrendiğimiz gibi, **sealed** anahtar kelimesi sınıftan türetme yapılmasını engellerken, **abstract** anahtar kelimesi bir sınıf türetmeyi ve türetilmiş sınıflarda metod oluşturmaya zorunlu kılmaktadır.

Aşağıdaki örnekte **Sekil** adlı özet bir sınıf oluşturulmuş ve özet sınıftan türetilen sınıflarda ayrı ayrı **Ciz()** metodu tanımlanmıştır.

```
abstract class Sekil
{
    public virtual void Ciz();
}

class Kare:Sekil
{
    public override void Ciz() {Console.WriteLine("Kare çiz...");}
}

class Daire:Sekil
{
    public override void Ciz() {Console.WriteLine("Daire çiz...");}
}
```

Yukarıdaki sınıfların **Main()** metodu içindeki kullanımı aşağıdaki gibidir.

```
static void Main(string[] args)
{
    Sekil k = new Kare();
    k.Ciz();
}
```

Arayüzler (Interfaces)

Interface ya da **arayüz** olarak isimlendirilen sınıflar tamamen özet (abstract) sınıflardır ve içindeki bütün üyeler de özet olmak zorundadır. Arayüz isimlerinin en başına, zorunluluk olmamasına rağmen, "I" harfi koyulması yaygın bir uygulamadır. Bir arayüz aşağıdaki gibi tanımlanabilir.

```
public interface ISekil
{
    void Ciz();
}
```

Arayüzlerin tüm üyeleri varsayılan olarak **abstract/özet** olduğu için ayrıca **abstract** anahtar kelimesinin kullanılmasına gerek yoktur. Ayrıca, arayüzlerin tüm üyeleri varsayılan olarak **public** özelliğine sahiptir. Bu nedenle herhangi bir belirteçle **public** olarak belirtilmesine gerek yoktur.

Tıpkı diğer sınıflar gibi, arayüzler de özellik ve metotlara sahip olabilir ama **alanlara/değişkenlere** sahip olamazlar. Yani aşağıdaki gibi bir arayüz yazarsanız hata alırsınız.

```
public interface ISekil
{
    int toplam=0;
    void Ciz();
}
```

Herhangi bir sınıf bir arayüzü temel sınıf olarak kullanmak (uygulamak/implement) isterse arayüzde tanımlanan bütün metotları kendi işlevine göre uyarlayıp kullanmak zorundadır. Örneğin, aşağıdaki gibi **ISekil** adlı bir arayüz olduğunu ve bu arayüzün **Ciz()** adlı bir metoda sahip olduğunu düşünün. Arayüzden türetilen tüm sınıflar **Ciz()** metodunu tanımlamak zorundadır.

```
public interface ISekil
{
    void Ciz();
}

class Daire:ISekil
{
    public void Ciz() { Console.WriteLine("Daire");}
}
```

```
public Kare:ISekil
{
    public void Ciz() { Console.WriteLine("Kare");}
}
```

Arayüzden nesne oluşturup kullanmak istiyorsanız **Main()** metoduna aşağıdaki kodları yazabilirsiniz.

```
static void Main(string[] args)
{
    ISekil k = new Kare();
    k.Ciz();
}
```

Peki! Neden özet sınıf kullanmıyoruz da arayüz kullanıyoruz? Bir sınıf sadece tek bir sınıftan kalıtım alabilir ama birçok arayüz kullanabilir. Bu özellik sayesinde birden fazla arayüzden kalıtım alabilen sınıflar türetebilir ve çok daha zengin özelliklere sahip türetilmiş sınıflar oluşturabilirsiniz.

Türetilmiş bir sınıfın birden fazla arayüzden kalıtım alabilmesi için aşağıdaki sözdizimi kullanılır.

```
class Daire:ISekil, IOyuncu
{
    public void Ciz() { Console.WriteLine("Daire");}
}
```

İsterseniz yukarıda anlatılan konuları basit bir örnekle pekiştirelim. Aşağıdaki örnekte **ISekil** adında bir arayüz tanımlanmıştır. Bu arayüz içinde de **Ciz()** adlı bir metod oluşturulmuştur. Arayüz içindeki tüm üyeler türetilen sınıfta olmak zorunda olduğuna göre **Daire** sınıfında **Ciz()** metodu tanımlanmalıdır.


```

1 using System;
2 namespace interfaceKalitimISekil
3 {
4     interface ISekil
5     {
6         void Ciz();
7     }
8
9     class Daire : ISekil
10    {
11        public void Ciz()
12        {
13            Console.WriteLine("Daire");
14        }
15    }
16
17    class Program
18    {
19        static void Main(string[] args)
20        {
21            ISekil d = new Daire();
22            d.Ciz();
23        }
24    }
25 }

```

Şekil 14.10 Arayüz kullanımı

İsterseniz sıradaki örneğimizde arayüz içindeki üye sayısını biraz artıralım ve **Ciz()** metodunun yanına iki adet özellik de ekleyelim. Bu durumda aşağıdaki gibi bir arayüz elde ederiz. Arayüzün tüm üyeleri türetilen sınıflarda kullanılmak zorundadır. Bu nedenle **ISekil** adlı arayüzden türetilen **Dikdortgen** sınıfında **Ciz()** metodu ve **Yukseklik** ile **Genislik** özellikleri tanımlanmak zorundadır.

```

1 using System;
2 namespace interfaceKalitimISekil
3 {
4     interface ISekil
5     {
6         int Genislik { set; get; }
7         int Yukseklik { set; get; }
8         int Ciz();
9     }
10
11    class Dikdortgen : ISekil
12    {
13        public int Genislik { set; get; }
14        public int Yukseklik { set; get; }
15        public int Ciz() { return Yukseklik * Genislik; }
16    }
17
18    class Program
19    {
20        static void Main(string[] args)
21        {
22            ISekil d = new Dikdortgen();
23            d.Genislik = 2;
24            d.Yukseklik = 3;
25            Console.WriteLine("Alan=" + d.Ciz());
26        }
27    }
28 }

```

Şekil 14.11 Arayüz kullanımı

IEnumerable Arayüzü

.NET Framework içinde hazır olarak gelen ve sıkça kullanılan birçok arayüz bulunmaktadır. Bunlardan bir tanesi de ve belki de en çok kullanılanı **IEnumerable** arayüzüdür. **IEnumerable**, çalışma mantığını pek merak etmediğimiz, fakat sıklıkla kullandığımız **"foreach"** döngüsünü kullanmamıza olanak sağlamaktadır.

Array, List, ArrayList, Hashtable gibi birçok koleksiyon **IEnumerable** sınıfını implement (kalıtlılamıştır) etmiştir. Bu sayede, **foreach** döngüsünü kullanarak bu koleksiyonlardaki elemanlara ulaşabilmekteyiz. Bu arayüzü biraz daha iyi tanımak için foreach yapısını hatırlayıp incelemek gerekiyor.

Foreach Döngüsü (Hatırlatma)

Foreach döngüsünün arka planında while döngüsü yatmakta. Yani çalışma anında foreach yapısı while döngüsüne dönmektedir. Döngünün ilerleyip dizideki diğer elemanlara erişebilmesi için **IEnumerable**'dan **"GetEnumerator"** metodu ile **"IEnumerator"** sınıfı talep edilmektedir.

Daha önce öğrendiğimiz **foreach** döngüsü ve kullanımı aşağıdaki gibidir. Aşağıdaki örnekte de görüldüğü gibi, **foreach** döngüsü ile bir dizinin elemanları üzerinde kolayca işlem yapılabilir.

```
using System;
namespace IEnumeratorOrnek
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] dizi = { 1, 3, 5, 7, 9 };
            foreach (int sayi in dizi)
                Console.WriteLine(sayi);
        }
    }
}
```

Foreach döngüsü ile bir koleksiyon ya da dizi üzerinde gezinerek elemanlarına ulaşabiliyoruz. Pekii! Acaba kendi oluşturduğumuz bir sınıf içindeki elemanları foreach döngüsü ile yazdırmak istersek ne olur? Çalışır mı? Örneğin, Sepet sınıfı içindeki Urun nesnelerini foreach döngüsü ile yazdırmak istersek ne olur? Bunu görmek için aşağıdaki örneği inceleyelim:

```

using System;

public class Urun
{
    public string Ad { get; set; }
    public double Fiyat { get; set; }
}

public class Sepet
{
    public Urun[] UrunListesi { get; set; }

    public Sepet()
    {
        UrunListesi = new Urun[]
        {
            new Urun(){ Ad="Bilgisayar", Fiyat=1022.35},
            new Urun(){ Ad="Tablet", Fiyat=699.99}
        };
    }
}

class Program
{
    static void Main(string[] args)
    {
        Sepet sepetim = new Sepet();

        foreach (Urun urun in sepetim)
        {
            Console.WriteLine("Urun:" + urun.Ad + ", Fiyat:" + urun.Fiyat);
        }
    }
}

```

Yukarıdaki kodlarda öncelikle **Urun** adlı bir sınıf oluşturulmaktadır. **Sepet** sınıfında ise **Urun** tipinde **UrunListesi** adlı bir dizi oluşturulmaktadır. Yapıcı metod içinde de listeye iki adet ürün eklenmektedir. **Program** sınıfı içinde **Sepet** sınıfından **sepetim** nesnesi oluşturulmakta ve **foreach** döngüsü kullanılarak **sepetim** nesnesinin elemanları yazdırılmak istenmektedir.

Maalesef bu kodlar “foreach statement cannot operate on variables of type ‘Sepet’ because ‘Sepet’ does not contain a public definition for ‘GetEnumerator’” şeklinde bir hata vermektedir. Bunun anlamı şudur: foreach döngüsü sadece dizi ve koleksiyonlarının elemanları üzerinde işlem yapmak için kullanılabilir. Eğer bir sınıftan türetilen nesnenin elemanları üzerinde işlem yapacaksanız **IEnumerable** kullanmak zorundasınız. Yani, sınıfımıza iteration (gezinme) özelliği kazandırmak için **IEnumerable** arayüzünü kullanmalıyız. Bu arayüz, uygulandığı sınıfa **GetEnumerator** adlı bir metod kazandırır ve bu metod geriye **IEnumerator** arayüzünü implement eden bir sınıf döndürür. Bir sınıf üzerinde foreach ile gezinebilmemiz/işlem yapabilmemiz için bu sınıfın **GetEnumerator** metodunun olması gerekmektedir.

Yield Operatörü

IEnumerator konusunun biraz daha kolaylaşabilmesi için **yield** operatörünü bilmemiz gerekiyor. Çok fazla bilinen bir operatör olmayan **yield** operatörü genelde **yield return ifade**; şeklinde kullanılır. **yield return** ifadesi ile iterator’a çağrı yapılan foreach döngüsüne bir eleman döndürülür. Konuyu daha iyi anlayabilmek için aşağıdaki paragrafı okuyalım ve hemen altındaki kodları inceleyelim.

IEnumerator arayüzünü neden öğrenmemiz ve kullanmamız gerektiğini anladıktan sonra gelelim bazı ayrıntılara. **IEnumerator** arayüzü 3 metod içermektedir. Bunlar: **MoveNext**, **Current** ve **Reset** metodlarıdır.

- **MoveNext**: Dizinin bir sonraki elemanına erişmemizi sağlar.
- **Current**: Dizinin o anki indeksinin değerini döndürür.
- **Reset**: Dizinin indeksini -1’e çeker.

Şimdi **Sepet** sınıfımızı **IEnumerable** arayüzünü uygulayacak (implemente edecek) şekilde değiştirelim:

```
class Sepet:IEnumerable
{
    public Urun[] UrunListesi { get; set; }

    public Sepet()
    {
        UrunListesi = new Urun[]
        {
            new Urun(){ Ad="Bilgisayar", Fiyat=1022.33},
            new Urun(){ Ad="Ipad", Fiyat=699.99}
        };
    }
}
```

```

public IEnumerator GetEnumerator()
{
    throw new NotImplementedException();
}
}

```

Uygulamalar

Kalıtım ve polimorfizm konusu özellikle büyük kurumsal yazılım projelerinde sıklıkla karşımıza çıkmaktadır. Bu nedenle, en azından temel düzey konuların çok iyi anlaşılması gerekmektedir. Şu ana kadar anlatılan konuları pekiştirmek için aşağıdaki uygulamaları yapabilirsiniz.

Satranç Sınıfı

Bir satranç yazılımı geliştireceksiniz. Bu satranç yazılımı için hangi sınıflar ve türetilmiş sınıflar oluşturulabilir? Öncelikle **Satranc** isimli temel bir sınıf oluşturulup bu sınıftan **Piyon**, **Kale**, **Fil**, **Vezir** gibi türetilmiş sınıflar oluşturulabilir. Aşağıdaki örneği temel alıp daha da geliştiriniz.

```

1  using System;
2  namespace uygulamaSatrancNesne
3  {
4      class Satranc //temel sınıf
5      {
6          //özellik ve metotlar
7      }
8
9      class Piyon: Satranc //türetilmiş sınıf
10     {
11         //özellik ve metotlar
12     }
13
14     class Kale : Satranc //türetilmiş sınıf
15     {
16         //özellik ve metotlar
17     }
18
19     class At : Satranc //türetilmiş sınıf
20     {
21         //özellik ve metotlar
22     }

```

Şekil 14.12

Dövüşçü Karakterler

Polimorfizm kullanımına tipik bir örnek dövüş oyunlarındaki karakterlerdir. Bu karakterlerin hepsi dövüş yeteneklerine sahiptir ama hepsi birbirinden farklı beceri ve dövüş stiline sahiptir. Bu durumda, **Oyuncu** adlı bir temel sınıf oluşturup sınıf içinde **Dovus()** adlı bir metod tanımlanabilir. **Dovus()** metodu farklı oyuncular için değiştirilerek her oyuncuya farklı yetenekler kazandırılabilir.

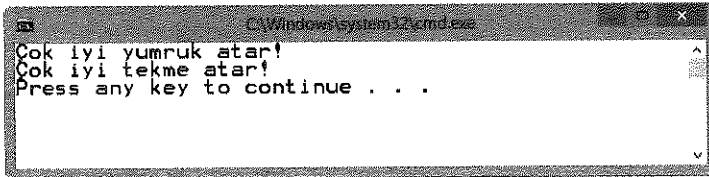
```

1  using System;
2  namespace uygulamaPolimorfizm{
3      class Oyuncu
4      {
5          public virtual void Dovus() { Console.WriteLine("Sanal Metod"); }
6      }
7
8      class DemirYumruk : Oyuncu
9      {
10         public override void Dovus() { Console.WriteLine("Çok iyi yumruk atar!"); }
11     }
12
13     class UcanTekme : Oyuncu
14     {
15         public override void Dovus(){ Console.WriteLine("Çok iyi tekme atar!"); }
16     }
17
18     class Program
19     {
20         static void Main(string[] args)
21         {
22             Oyuncu o = new DemirYumruk();
23             o.Dovus();
24
25             Oyuncu o2 = new UcanTekme();
26             o2.Dovus();
27         }
28     }
29 }

```

Şekil 14.13

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir çıktı elde edersiniz.



```

C:\Windows\system32\cmd.exe
Çok iyi yumruk atar!
Çok iyi tekme atar!
Press any key to continue . . .

```

Şekil 14.14

Polimorfizm ile Alan Hesabı

Bu örneğimizde **Sekil** sınıfında tanımlanan sanal **Alan()** metodu ile türetilmiş sınıflarda **Alan()** metodunun oluşturulmasını zorunlu hale getireceğiz. Türetilmiş sınıflarda oluşturulan **Alan()** metodu ile üçgen, kare ve dikdörtgen gibi şekillerin alanlarını hesaplayacağız.

```

1  using System;
2  namespace uygulamaPolimorfizmAlan {
3      class Sekil {
4          public double Genislik;
5          public double Yukseklik;
6          virtual public double Alan() { return Genislik * Yukseklik; }
7      }
8
9      class Ucgen : Sekil {
10         public Ucgen(double g, double y) { this.Genislik = g; this.Yukseklik = y; }
11         override public double Alan() { return Genislik * Yukseklik/2; }
12     }
13
14     class Kare : Sekil {
15         public Kare(double g) { this.Genislik = g; }
16         override public double Alan() { return Genislik * Genislik; }
17     }
18
19     class Program {
20         static void Main(string[] args) {
21             Sekil u = new Ucgen(2.0, 3.0);
22             Console.WriteLine("Alan=" + u.Alan());
23
24             Sekil k = new Kare(3.0);
25             Console.WriteLine("Alan=" + k.Alan());
26         }
27     }
28 }

```

Şekil 14.15

Özet Sınıf ile Şekillerin Alan Hesabı

Bu örneğimizde özet **Sekil** sınıfında tanımlanan özet **Alan()** metodu ile türetilmiş sınıflarda **Alan()** metodunun oluşturulmasını zorunlu hale getireceğiz. Türetilmiş sınıflarda oluşturulan **Alan()** metodu ile daire ve dikdörtgenlerin alanlarını hesaplayacağız.

```

1  using System;
2  namespace uygulamaAbstract
3  {
4      abstract class Sekil {
5          public abstract double Alan(); }
6
7      class Daire : Sekil {
8          public double Yarıcap;
9          public Daire(double r) { this.Yarıcap = r; } //Yapıcı metod
10         public override double Alan() { return Math.PI * Yarıcap * Yarıcap; }
11     }
12
13     class Dikdortgen : Sekil {
14         public double Genislik;
15         public double Yukseklik;
16         public Dikdortgen(double g, double y) { this.Genislik = g; this.Yukseklik = y; }
17         public override double Alan() { return Genislik * Yukseklik; }
18     }
19
20     class Program {
21         static void Main(string[] args) {
22             Sekil d = new Daire(2.0); //Yapıcı metod ile yarıcap al
23             Console.WriteLine("Daire Alanı=" + d.Alan());
24
25             Sekil dik = new Dikdortgen(2.0, 3.0); //Yapıcı metoto ile genişlik, yükseklik al
26             Console.WriteLine("Dikdörtgen Alanı=" + dik.Alan());
27         }
28     }
29

```

Şekil 14.16

Arayüz ile Şekillerin Alan Hesabı

Diğer yöntemlerle karşılaştırma amaçlı, bu örneğimizde **ISekil** arayüzünde tanımlanan **Alan()** metodu ile türetilmiş sınıflarda **Alan()** metodunun oluşturulmasını zorunlu hale getireceğiz. Türetilmiş sınıflarda oluşturulan **Alan()** metodu ile daire ve dikdörtgenlerin alanlarını hesaplayacağız.

```

1  using System;
2  namespace uygulamaArayuz
3  {
4      interface ISekil {
5          double Alan();
6      }
7
8      class Daire : ISekil {
9          public Daire(double y) { this.Yaricap = y; }
10         public double Yaricap { set; get; }
11         public double Alan() { return Math.PI * Yaricap * Yaricap; }
12     }
13
14     class Dikdortgen : ISekil {
15         public double Genislik { set; get; }
16         public double Yukseklik { set; get; }
17         public Dikdortgen(double g, double y) { this.Genislik = g; this.Yukseklik = y; }
18         public double Alan() { return Genislik * Yukseklik; }
19     }
20
21     class Program {
22         static void Main(string[] args) {
23             ISekil d = new Daire(1.0);
24             Console.WriteLine("Daire Alanı=" + d.Alan());
25
26             ISekil dik = new Dikdortgen(3.0, 5.0);
27             Console.WriteLine("Dikdörtgen Alanı=" + dik.Alan());
28         }
29     }
30 }

```

Şekil 14.17

15

Yapılar & Numaralandırma

- Yapılar (Struct)
- Yapılarda Parametrelili Yapıcı Metot Kullanımı
- Numaralandırma (Enum)
- Uygulamalar

Yapı (Struct) Nedir?

Şu ana kadar gördüğümüz **bool**, **char**, **int**, **float** veya **double** tipindeki değişkenlere temel ya da ilkel değişkenler denilmektedir. Bu tür değişkenler sadece belli bir tip-te ve sadece tek bir değer taşıyabilirler. Dizilerde birden çok değer barındırabiliyoruz ama hepsinin veri tipi aynı olmak zorunda.

Peki! Aynı değişkende birden fazla ve farklı veri tipinde değer saklamak istersek ne yapacağız? Örneğin bir öğrencinin numarasını, adını, soyadını ve cinsiyeti barındıran bir değişken tanımlamak istersek ne yapacağız? İşte bu durumda adına "**yapı**" ya da "**struct**" denilen değişkenleri kullanacağız.

Yapılar önceki bölümde öğrendiğimiz sınıflara çok benzer ama referans tipinde değil değer tipinde saklanan bir veri türüdür. Yani, yapıların içinde bulunan değişken ve metotlar hafızanın **stack** bölgesinde saklanır.

Daha önce de öğrendiğimiz gibi, referans tipindeki veriler için hem **stack** hem de **heap** bölgesinde hafıza ayrılır. Hafızanın heap bölgesinde nesneye ait verilerin kendisi bulunurken, stack bölgesinde ise nesnenin adresi tutulur. Hafızanın heap bölge-

sinde bulunan nesne verilerine her ulaşmak istediğimizde önce stack bölgesindeki adrese bakılır ondan sonra nesnenin verilerine erişilir. Bu da doğal olarak biraz zaman kaybına neden olur. Nispeten sınırlı sayıda değer içeren ve fazla karmaşık olmayan veriler için yapıları kullanmak performans açısından daha iyi sonuç verebilmektedir.

Sınıflardan farklı yapılar aşağıdaki özelliklere/kısıtlamalara sahiptirler:

- Kalıtım özelliği yoktur.
- Sanal metotları desteklemezler.
- Sadece parametrelili yapıcı metotlara sahip olabilirler.
- Parametrelili yapıcı metoda sahip yapılardan, tıpkı sınıflarda yaptığımız gibi, **new** anahtar kelimesi ile nesne oluşturabilirsiniz.

Yapılar büyük verileri barındırmaktan ziyade, daha çok birbiriyle ilgili değişkenleri ve metotları barındırmak için kullanılırlar. Örneğin, bir öğrencinin adı, soyadı, numarası ve bölümü gibi bilgiler yapılarda saklanabilir.

Yapının sözdizimi aşağıdaki gibidir:

```
struct Yapı_Adı
{
    //değişkenler & metotlar
}
```

Örnek Kod:

Bir öğrencinin temel bilgilerini saklamaya yarayan yapı aşağıdaki gibi tanımlanabilir.

```
struct Ogrenci
{
    public string adsoyad;
    public string tcno;
    public int bolumNo;
    public int okulNo;
}
```

Yukarıda tanımlanan yapıyı **Main()** metodu içinde kullanmak için aşağıdaki kodları yazabilirsiniz. Gördüğümüz üzere, tıpkı sınıflarda yaptığımız gibi, **Ogrenci** adlı yapıdan **o1** adlı bir nesne oluşturuyoruz. Nesne oluşturulduktan sonra yapının özellik ve metotlarını kullanabiliyoruz.

```

static void Main(string[] args)
{
    Ogrenci o1;
    o1.adsoyad = "Fahrettin Erdinç";
    o1.tckn = "9159292929";
    o1.bolumNo = 2;
    o1.okulNo = 31213;
}

```

Yapıdan nesne oluşturmak için **Yapı nesne;** veya **Yapı nesne=new Yapı();** ifadelerini kullanabilirsiniz. Aşağıdaki programı yazarak konuyu pekiştirebilirsiniz. Buradaki kodlarda değişkenlere ek olarak yapı içinde bir metot da tanımlanmıştır. Metot, öğrenci ile ilgili temel bilgilerin ekrana yazdırılması için kullanılmaktadır.

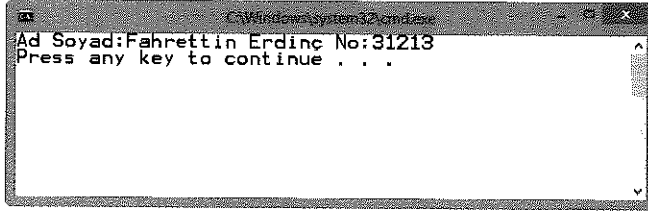
```

1  using System;
2  namespace yapi
3  {
4      struct Ogrenci
5      {
6          public string adsoyad;
7          public string tckn;
8          public int bolumNo;
9          public int okulNo;
10         public void Yaz()
11         {
12             Console.WriteLine("Ad Soyad:" + adsoyad + " No:" + okulNo);
13         }
14     }
15
16     class Program
17     {
18         static void Main(string[] args)
19         {
20             Ogrenci o1 = new Ogrenci();
21             o1.adsoyad = "Fahrettin Erdinç";
22             o1.tckn = "9159292929";
23             o1.bolumNo = 2;
24             o1.okulNo = 31213;
25             o1.Yaz();
26         }
27     }
28 }

```

Şekil 15.1

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir çıktı elde edersiniz.



Şekil 15.2 Ekran çıktısı

Yukarıdaki örnek kodlardan da görüldüğü gibi, yapıların kullanımı sınıfların kullanımı ile neredeyse tıpatıp aynıdır. Sadece, **class** yerine **struct** komutu kullanılmakta ve bir-iki özellik kısıtlanmaktadır.

Yapılarda Yapıcı Metot Kullanımı

Tıpkı sınıflarda olduğu gibi, yapılarda da yapıcı metotları kullanabilirsiniz. Yalnız, sınıflardan farklı olarak yapılarda sadece parametrelili yapıcı metotların kullanımına izin verilmektedir. Aşağıdaki kodlarda parametrelili bir yapıcı fonksiyona örnek görebilirsiniz.

```
struct Ogrenci
{
    public string adsoyad;
    public string tckn;
    public Ogrenci(string ad, string tc)
    {
        this.adsoyad=ad;
        this.tckn=tc;
    }
}
```

Yukarıda tanımlanan sınıfı **Main()** metodu içinde aşağıdaki gibi kullanabilirsiniz. Buradaki kodlarda **Main()** metodu içinde **Ogrenci** yapısından **o1** adlı bir nesne tanımlanmakta ve aynı zamanda da parantez içinde yapıcı metoda parametre olarak **adsoyad** ve **tckn** bilgileri girilmektedir.

```
static void Main(string[] args)
{
    Ogrenci ol=new Ogrenci(Fahrettin Erdiñç,91291929012);
    Console.WriteLine("Ad:"+ol.adsoyad+" TCKN:"+ol.tckn);
}
```

Konunun daha iyi anlaşılabilmesi için bir noktanın koordinatlarını belirlemeye yarayan yapıyı oluşturup **Main()** metodu içinde kullanalım.

```

1  using System;
2  namespace structNokta
3  {
4      struct Nokta
5      {
6          public double x;
7          public double y;
8
9          //parametrelili yapıcı metot
10         public Nokta(double a, double b)
11         {
12             this.x = a;
13             this.y = b;
14         }
15     }
16
17     class Program
18     {
19         static void Main(string[] args)
20         {
21             //Nokta adlı yapıdan n1 nesnesi oluşturalım
22             Nokta n1 = new Nokta(1.2, 3.56);
23             Console.WriteLine("x:" + n1.x + " y:" + n1.y);
24         }
25     }
26 }

```

Şekil 15.3 Parametrelili yapıcı metoda sahip yapının Main() metodu içinde kullanımı

Yukarıdaki örnekten de görüldüğü gibi, sınıflar ile yapılar kullanım açısından birbirine çok benzemektedir. Bazı durumlarda hangisinin kullanılması gerektiği bile karıştırılmaktadır. Genel olarak bir fikir vermek gerekirse, küçük ve basit tipte verilerin tutulması için yapıları, büyük ve karmaşık tipte verilerin tutulması için sınıfları tercih etmek gerekmektedir.

Numaralandırma (Enum)

Kendi veri tipimizi tanımlamanın yollarından biri de "**numaralandırma**" ya da "**enum**" denilen yöntemi kullanmaktır. Numaralandırma veri tipi değişkenlere sadece önceden belirlenen değerlerin atanmasını sağlar.

Adından da belli olduğu gibi, numaralandırmada her seçeneğe bir numara verilip bu numaralar üzerinde işlem yapılabilir. Örneğin, yön belirten **yön1** adlı bir değişkene sadece **kuzey**, **guney**, **dogu** ve **batı** değerlerinin atanmasını istiyorsunuz. Bu durumda bir **enum** oluşturarak seçenekleri ve bu seçeneklere karşı gelen numaraları belirliyoruz. Daha sonra da seçenek veya numaralar üzerinden işlem yapabi-

liyoruz. Özet olarak numaralandırma sayıları anlamlı şekilde isimlendirerek kullanabilmeye izin vermektedir.

Numaralandırma tipi tanımlama aşağıdaki gibi yapılır:

```
enum enum_adi
{
    deger1,
    deger2,
    deger3
}
```

Yukarıdaki tanımlama biçimi yerine aşağıdaki biçim de kullanılabilir. Böylece tanımlama tek satırda yapılabilir.

```
enum enum_adi {deger1, deger2, deger3}
```

Eğer **Yon** adlı bir **enum** tanımlamak ve değişkenlerin alabileceği değerleri **kuzey**, **guney**, **dogu** ve **batı** olarak belirlemek isterseniz aşağıdaki kodları yazabilirsiniz.

```
enum Yon
{
    kuzey,
    guney,
    dogu,
    batı
}
```

Eğer **Yon** veri tipine ait **yon1** adlı bir değişken tanımlamak isterseniz önce **enum** adını daha sonra da yeni oluşturulacak değişken adını yazmalısınız. Yani aşağıdaki kodları yazmanız gerekiyor:

```
Yon yon1;
```

Yukarıda tanımlanan **Yon** tipindeki **yon1** değişkenine sadece **kuzey**, **guney**, **dogu**, **batı** değerlerini atayabilirsiniz.

```
yon1=Yon.dogu;
```

Yukarıda tanımlanan değişkenin değerini konsola yazdırmak için aşağıdaki kodu kullanabilirsiniz:


```
Console.WriteLine(yon1); //Çıktı:doğu
```

Örnek Kod:

```
using System;
namespace enumOrnek
{
    class Program
    {
        enum Yon { dogu, bati, kuzey, guney}

        static void Main(string[] args)
        {
            Yon yon1;
            yon1 = Yon.bati;

            Console.WriteLine(yon1);
        }
    }
}
```

Enum ile oluşturulan veri tiplerinde aksi belirtilmediği sürece ilk elemanın değeri sıfırdan başlar. Yani **yon** adlı numaralamada **kuzey=0**, **guney=1**, **dogu=2** ve **batı=3** değerlerini alırlar. Eğer bu durumu somut olarak görmek isterseniz elemanların çıktısını alabilirsiniz.

```
Console.WriteLine(kuzey);    //0
Console.WriteLine(guney);    //1
Console.WriteLine(dogu);     //2
Console.WriteLine(bati);     //3
```

Numaralama elemanlarının farklı bir değerden başlamasını istiyorsanız ilk elemana bir değer atayabilirsiniz. Diğer elemanlar 1 artarak devam eder.

```
enum Yon
{
    kuzey=10,
    guney, //11
    dogu,  //12
    bati   //13
}
```

Eğer her elemana farklı bir değer atamak istiyorsanız aşağıdaki yöntemi kullanabilirsiniz:

```
enum Yon
{
    kuzey=10,
    guney=20,
    dogu=30,
    bati=40
}
```

Eğer değeri ortadaki bir elemana atarsanız ilk elemanın değeri sıfır, değer atanan elemandan sonraki değerler ise 1 artarak devam eder. Örneğin, aşağıdaki numaralandırmada Pazartesi'nin değeri 0 (sıfır) olurken, Salı'nın değeri 1 olmaktadır. Çarşamba'nın değeri 6 olarak atandığı için, ondan sonra gelen elemanların değeri 1 artarak devam eder. Bu durumda Perşembe'nin değeri 7, Cuma'nın değeri 8 olmaktadır.

```
enum Gunler{P.tesi, Salı, Çarşamba=6, Perşembe, Cuma, Cumartesi, Pazar}
```

Örnek Kod:

Bir okul programı yaptığınızı ve okuldaki herkesin bilgisayara kaydedilmesi gerektiğini düşünelim. Programa okuldaki herkesin adı, soyadı ve görevi kaydedilecek. Okuldaki kişilerin görevi için sadece belirli değerlerin atanmasını istiyorsanız aşağıdaki gibi bir **enum** oluşturabilirsiniz.

```
enum Gorev { mudur, ogretmen, memur, ogrenci }
```

Eğer **Gorev** veri tipinde bir değişken oluşturup sadece ilgili elemanların atanmasını istiyorsanız aşağıdaki kodları yazabilirsiniz.

```
Gorev gorevi;
```

Artık **gorevi** değişkenine sadece **mudur**, **ogretmen**, **memur** ve **ogrenci** değerleri atanabilir. Sonuçta aşağıdaki kodlar ortaya çıkar:

```
using System;
static void Main(string[] args)
{
    enum Gorev {mudur,ogretmen,memur,ogrenci}

    //değişkenleri tanımlayalım
```

```

string ad="ali";
string soyad="tok";
Gorev gorevi=Gorev.mudur;

//değişken değerlerini yazalım
Console.WriteLine(ad);
Console.WriteLine(soyad);
Console.WriteLine(gorevi);
}

```

Eğer istenirse **Enum.Eleman** söz dizimi kullanılarak numaralandırma değerleriyle de işlem yapılabilir. Bu durumda, atama işlemi aşağıdaki gibi olur.

```

enum Aylar{Ocak, Şubat, Mart, Nisan, Mayıs, Haziran}
static void Main(string[] args)
{
    int ay=(int)Aylar.Mart;
    Console.WriteLine(ay);    //Çıktı: 2
}

```

Yukarıdaki kodlardan da anlaşılacağı üzere, **int** veri tipindeki değişkenlere numaralandırma değerleri atamak için (**int**) veri tipi dönüştürme operatörü kullanılmıştır.

Numaralandırma işlemi özellikle **switch** komutu ile sıklıkla kullanılmaktadır. Aşağıdaki örnek kodlarda trafik ışığını rengine göre değişik işlemler yapılmaktadır.

Örnek Kod:

```

using System;
namespace enumLamba
{
    class Program
    {
        enum Lamba { SARI, YEŞİL, KIRMIZI}

        static void Main(string[] args)
        {
            Lamba l = Lamba.KIRMIZI;
            switch(l) {
                case Lamba.SARI:
                    Console.WriteLine("Geçmeye Hazırlanın");
                    break;
                case Lamba.YEŞİL:

```

```

        Console.WriteLine("Geçin");
        break;
    case Lamba.KIRMIZI:
        Console.WriteLine("Durun");
        break;
    }
}
}
}

```

Yukarıdaki kodlara göre `l` değişkeninin aldığı değer **Lamba.KIRMIZI** (2) olduğu için kodlar çalıştırıldığında ekranda **"Durun"** yazacaktır.

Uygulamalar

Yapılarla ilgili temel bilgileri öğrendikten sonra birkaç uygulama yaparak konuyu iyice pekiştirmeye çalışalım. Kolay bir uygulamayla başlayalım ve zorlaştırarak devam edelim.

Öğrenci Bilgilerini Tutan Yapı

Bu uygulamada **Oğrenci** adlı bir yapı oluşturup, öğrenci ile ilgili **Ad**, **Soyad** ve **Öğrenci No** bilgilerini yapı ile tanımlayıp **Main()** metodu içinde kullanacağız.

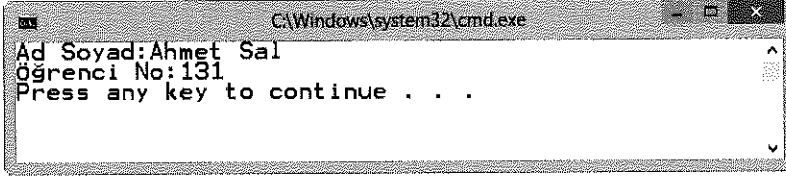
```

1  using System;
2  namespace uygulamaYapiOgrenci
3  {
4      class Program
5      {
6          struct Ogrenci {
7              public string Ad;
8              public string Soyad;
9              public int No;
10         }
11
12         static void Main(string[] args)
13         {
14             Ogrenci o1 = new Ogrenci();
15             o1.Ad = "Ahmet";
16             o1.Soyad = "Sal";
17             o1.No = 131;
18
19             Console.WriteLine("Ad Soyad:" + o1.Ad + " " + o1.Soyad);
20             Console.WriteLine("Öğrenci No:" + o1.No);
21         }
22     }
23 }

```

Şekil 15.4

Yukarıdaki kodların çıktısı aşağıdaki gibi olur.



Şekil 15.5

Eğer **Oğrenci** yapısından oluşturulan **o1** nesnesinin **Ad**, **Soyad** ve **No** özelliklerine değer atamazsanız **string** tipindeki değişken değerlerinin **boşluk** (""), **int** tipindeki değişken değerlerinin ise **sıfır** (0) olduğunu görebilirsiniz. Bunun nedeni, **o1** nesne oluşturulduğu anda yapıcı metodun nesne üyelerine varsayılan değer atamasıdır.

Yapılarda Nesne Kopyalama

Bir önceki örneğimizde **o1** nesnesinin üyelerine değer atayıp konsola yazdırdık. Bu sefer **o1** nesnesinden **o2** adlı bir nesne kopyalayacağız. Daha sonra da **o2** nesnesinin **Ad** üyesine "Osman" değerini atayıp **o2** nesnesinin aldığı değerleri inceleyeceğiz.

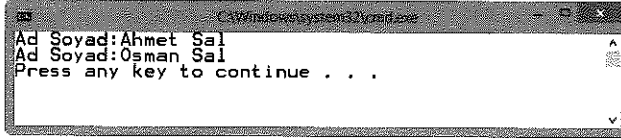
```

1  using System;
2  namespace uygulamaYapiOgrenci
3  {
4      class Program
5      {
6          struct Oğrenci {
7              public string Ad;
8              public string Soyad;
9              public int No;
10         }
11
12         static void Main(string[] args)
13         {
14             Oğrenci o1 = new Oğrenci();
15             o1.Ad = "Ahmet";
16             o1.Soyad = "Sal";
17             o1.No = 131;
18
19             Oğrenci o2 = new Oğrenci();
20             o2 = o1;
21             o2.Ad = "Osman";
22
23             Console.WriteLine("Ad Soyad:" + o1.Ad + " " + o1.Soyad);
24             Console.WriteLine("Ad Soyad:" + o2.Ad + " " + o2.Soyad);
25         }
26     }
27 }

```

Şekil 15.6

Yukarıdaki kodların çıktısı aşağıdaki gibi olur.



Şekil 15.7

Yukarıdaki örneği iyice incellerseniz, adlarının farklı, soyadlarının aynı olduğunu görebilirsiniz. Bunun nedeni yapıların değer türünde değişkenler olmasından kaynaklanmaktadır. Hatırlayacağınız üzere, sınıflardan bir nesne kopyalandığında hem kaynak hem de kopya nesne aynı adreslere referans veriyordu. Bu nedenle kopya sınıfın üye değerleri değiştiğinde kaynak sınıfın üye değerleri de değişiyordu. Yapılarda ise kopya nesne tamamen başka bir adreste tutulmaktadır. Bu nedenle, yapılardaki nesne kopyalandığında önce kaynak nesnelerin değerlerine sahiptir. Daha sonra kopya nesnelerin üyelerine kaynak nesneden bağımsız olarak değer atanabilir. Yukarıdaki örneğimizde **o2** nesnesinin soyadı **o1** nesnesinden kopyalanarak gelirken, adı atamaya belirlenmiştir.

Yapılarda Parametrelili Yapıcı Metot Kullanımı

Sınıflardan farklı olarak yapılarda tanımlanan yapıcı metotların parametre alma zorunluluğu vardır. Yani, yapılarda parametresiz bir yapıcı metot tanımlayamayız. Bu örneğimizde yukarıda tanımladığımız yapı için bir yapıcı metot tanımlayıp **Main()** metodu içinde kullanacağız. Kodlarımız aşağıdaki gibidir.

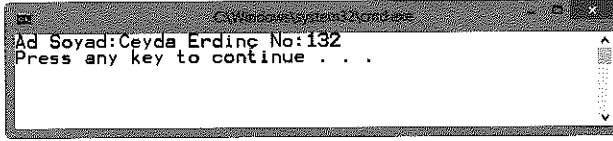
```

1  using System;
2  namespace uygulamaYapiOgrenci
3  {
4      class Program
5      {
6          struct Ogrenci {
7              public string Ad;
8              public string Soyad;
9              public int No;
10
11              public Ogrenci(string ad, string soyad, int no) {
12                  Ad = ad;
13                  Soyad = soyad;
14                  No = no;
15              }
16          }
17
18          static void Main(string[] args)
19          {
20              Ogrenci o3 = new Ogrenci("Ceyda","Erdinc",132);
21
22              Console.WriteLine("Ad Soyadı:" + o3.Ad + " " + o3.Soyad+" No:"+o3.No);
23          }
24      }
25  }

```

Şekil 15.8

Yukarıdaki kodlarda nesne tanımlanırken ad, soyad ve numara bilgisi de girildiği için ayrıca atama yapmaya gerek yoktur. Kodların ekran çıktısı aşağıdaki gibi olur.



Şekil 15.9

Şehir Kodlarının Enum ile Belirlenmesi

Şehirlerin plaka numaralarını hatırlamak zor olduğu için insanların plaka numarası yerine şehir isimlerini kullanmayı sağlayan bir numaralandırma yapacağız.

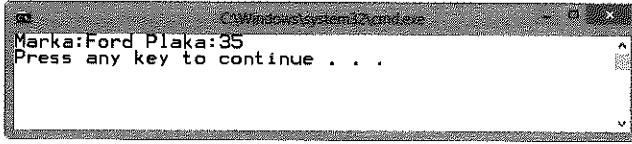
```

1  using System;
2  namespace uygulamaEnumPlaka
3  {
4      enum Plaka { Adana=01, Ankara=06, Bursa=16, İstanbul=34, İzmir=35 }
5
6      struct Araba {
7          public string Marka;
8          public int Sehir;
9      }
10
11     class Program
12     {
13         static void Main(string[] args)
14         {
15             Araba a1;
16             a1.Marka = "Ford";
17             a1.Sehir = (int)Plaka.İzmir;
18
19             Console.WriteLine("Marka:" + a1.Marka + " Plaka:" + a1.Sehir);
20         }
21     }
22 }

```

Şekil 15.10

Yukarıdaki kodların çıktısı aşağıdaki gibi olur.



Şekil 15.11

Yukarıdaki örnekten de anlaşılacağı gibi, plaka kodu girmek yerine hatırlanması daha kolay olan şehir adını girerek plaka numarasını tanımlamış olduk.

16

İstisna Yönetimi (Exception Handling)

- İstisna Nedir?
- İstisna Bildirimi
- Try-Catch-Finally
- İstisna Sınıfları
- Throw

İstisna (Exception Handling) Yönetimi Nedir?

Programın çalışması sırasında donanım ya da kullanıcıdan kaynaklanan beklenmedik durum ya da hatalara "**İstisna**" denir. Programcı kodları ne kadar dikkatli yazar sa yazsın mutlaka bazı konular gözden kaçmakta ve çalışma anında beklenmedik hatalara neden olmaktadır. Hata olmasını beklediğiniz kodlarda her zaman hata olmamakta ama bazı durumlar gerçekleştiğinde, ender de olsa hata oluşmaktadır. İstisna denmesinin nedeni de budur.

İstisna yönetimi ile ilgili komutlar, programda bir hata meydana geldiğinde bu hata ile ilgili bilgi alınmasını, hatanın kullanıcıya bildirilmesini ve sistemin aksamadan çalışmaya devam etmesini sağlar. Sonuç olarak, hata yönetimi komutları hem daha kararlı çalışan ve hem de daha kullanıcı dostu yazılımlar üretmemizi sağlar. İstisna yönetiminin mantığı aşağıdaki gibi işlemektedir:

Bir komut çalıştır

Eğer yukarıdaki komut düzgün çalışmadıysa/istisna oluştuysa
istisnanın nedenini/kaynağını bul ve kullanıcıya bilgi ver

Bir komut çalıştır

Eğer yukarıdaki komut düzgün çalışmadıysa/istisna oluştuysa
istisnanın nedenini/kaynağını bul ve kullanıcıya bilgi ver

...

İstisnalar aşağıdaki gibi durumlarda meydana gelebilir:

- Kullanıcının geçersiz veri girmesi
- Açılması gereken bir dosyanın silinmiş olması
- Programın çalışması sırasında ağ bağlantısının kopması
- Bilgisayarla ilgili fiziksel sorunlar
- ... ve diğer birçok neden

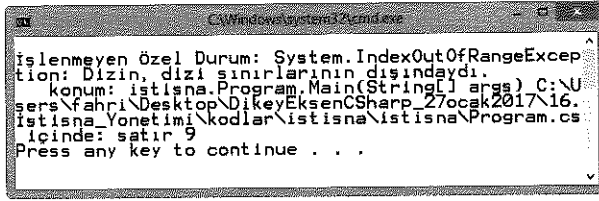
Örneğin, aşağıdaki kodlar çalıştığında bir istisna meydana gelecektir çünkü dizide olmayan bir indis yazdırılmaya çalışılıyor.

```

1  using System;
2  namespace istisna
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              int[] dizi = { 1, 2, 3 };
9              Console.WriteLine(dizi[5]);
10         }
11     }
12 }
```

Şekil 16.1 İstisna

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir istisna/hata ile karşılaşacaksınız. Buradaki pencerede istisnanın nerede ve hangi nedenden meydana geldiği anlatılmaktadır. Penceredeki açıklamaları dikkatlice okursanız hatanın dizi sınırlarının dışında olan bir indis (dizin) tarafından meydana geldiğini görebilirsiniz.



Şekil 16.2 İstisna açıklaması

Yukarıdaki örnekten de görebileceğiniz gibi, istisnanın nedeni kullanıcı hatası, programcı hatası veya donanım olabilir. Şunu da belirtmekte fayda var; istisna yönetimi dosyanın indirilmesinin çok uzaması, diskin yavaş çalışması, ağın yavaşlığı gibi nedenlerden kaynaklanan sorunlarla ilgilenmez. İstisna yönetimi programın normal olarak çalışması sırasında yanlış veri tipi girişi, taşma hataları, sıfıra bölme, eksik metod parametresi ve hafıza yetersizliği gibi durumlarla ilgilenir.

Burada, biraz bahsetmekte fayda var... Hatalarla ilgili ilginç olan şey, program hatalarının yazılım dünyasında son derece doğal karşılanmasıdır. Örneğin, bir işletim sistemini para verip aldığınızı ve bilgisayarınıza kurduğunuzu düşünün. Kabul ettiğiniz lisans anlaşmasında “as is” yani “olduğu gibi” satın aldığınız yazar. Yani satın aldığınız programın tüm hatalarını ve açıklarını kabul eder ve öyle satın alırsınız. Programdaki bir hata ya da güvenlik açığından dolayı yazılımı geri verip paranızı geri alma şansınız yok! En azından ben böyle bir olaya rastlamadım. Oysa pahalı bir araba ya da ucuz bir çakmak aldığınızda bile ürünü “kusurlu” deyip iade etme şansınız var. Durum böyle olunca yazılımcılar sanki hatalı kod yazmaktan çekinmiyor gibi görünüyorlar. Hâlbuki hatalı program hem kullanıcıya hem de üreticiye aşırı güncelleme ve bakım yükleri getirmektedir. Bu da ek maliyet demektir. Bu nedenle kodlarımızı mümkün mertebe en az hatayla yazmak gerekir.

Benim gözlemim programcılığa yeni başlayan arkadaşların daha çok hata yaptığı şeklindedir. O nedenle özellikle acemi programcılar çalıştıkları konuda ve kullandıkları teknolojiler konusunda kendilerini iyi yetiştirmeliler.

Programcılıkta yapılan hataların ya da program çalışırken meydana gelen istisnaların/aykırı-durumların tolere edilmesinin birinci nedeni yazılımın çok farklı donanımlar üzerinde çalıştırılmasıdır. Doğal olarak bir donanımda çalışan bir yazılım kaynak yetersizliği ya da başka nedenlerle başka bir donanımda çalışmamakta ya da hatalara neden olmaktadır. Hataların tolere edilmesinin ikinci nedeni ise programların birer sanat/fikir eseri sayılmasıdır. Aynen Picasso'nun yaptığı resimler gibi yazılımlar da birer sanar eseri sayılmaktadır.

Pekil Test ortamınızda yani kendi bilgisayarınızda kodlar çok güzel çalışırken başka bir bilgisayarda neden çalışmamakta ya da çalışma anında hatalar vermektedir.

Bu tür hataların en büyük nedeni donanım değişiklikleridir. Sizin işlemciniz Intel iken başka bir bilgisayarda AMD olabilir. Ya da sizin bilgisayarınızda 4GB RAM bellek varken diğer bilgisayarda 512KB RAM bellek olabilir ve bu da hatalara neden olabilir. Ayrıca, kullanıcılar veri olarak hiç beklenmedik değerler de girmiş olabilir. Örneğin meşhur sıfıra bölme hatası; eğer bir sayıyı sıfıra bölerseniz “sıfıra bölme” hatası alırsınız.

İstisna Yönetimi

Programın düzgün çalışması için her türlü çabayı sarfettik; kodları düzgün yazdık, testler yaptık, hatta programı farklı donanımlarda bile çalıştırdık. Peki, buna rağmen kodlarda bir hata ya da aykırı durum meydana geldiğinde ne olacak? Bir programcı olarak görevimiz mümkün olduğunca az hataya sahip kod yazmak ve oluşan hatalarla ilgili de kullanıcı ya da servis elemanına bilgi vermektr. Örneğin, kullanıcı bir sayıyı sıfıra böldüyse ve bu nedenle bir hata/istisna/aykırı-durum oluştuysa hatanın nedenini öğrenip kullanıcıya bildirmek gerekmektedir.

Hataları yönetmek için C# dilinde bazı komutlar bulunmaktadır. Bu komutlar kullanılarak hatalar tespit edilmekte ve kullanıcıya bildirilmektedir. Böylece kullanıcı ya da servis elemanı hatanın nedenini öğrenmekte ve ona göre işlem yapmaktadır.

Aykırı durumları yönetmek için **try**, **catch**, **throw** ve **finally** komutları kullanılır. Eğer kısaca açıklamak gerekirse, istisna meydana getirebilecek kodları **try** bloğu içine koyarız. Herhangi bir istisnai durum oluştuğunda **try** bloğu içindeki işlem sonlandırılır ve meydana gelen hata **throw** komutu ile **catch** bloğuna atılır/gönderilir. Meydana gelen hata **catch** bloğu tarafından yakalanır ve hatanın halledilmesi için gerekli işlemler ve uyarılar yapılır.

Aykırı durumların halledilmesinde kullanacağımız **try-catch-finally** komutlarının sözdizimi aşağıdaki gibidir. Görüldüğü gibi birden fazla **catch()** bloğu olabilir. Bu durumda her **catch()** bloğu farklı türde istisna için kullanılabilir.

```
try
{
    //hata/istisna oluşturabilecek kodlar
}

catch( hata h1 )
{
    //istisna/hata ile ilgili açıklama
}
```

```

catch( hata h2 )
{
    // istisna/hata ile ilgili açıklama
}

finally()
{
    //bu kodlar hata olsa da olmasa da çalışır
}

```

Yukarıda da görüldüğü gibi, C# programlama dili oluşan istisnaları yönetmek için kısaca **try-catch** adıyla bilinen oldukça esnek ve etkili bir yapı sunmaktadır. Bu yapı herhangi bir istisna oluştuğunda bilgisayarın kilitlenmesini veya yeniden başlamasını engeller ve bir uyarı vererek istisna hakkında bilgi sahibi olmamızı sağlar.

Basit bir örnek kod yazmak gerekirse **try-catch** yapısı aşağıdaki gibi kullanılabilir. Aşağıdaki kodlarda hata olur düşüncesiyle **Console.WriteLine(x/y);** komutu **try{}** bloğu içine yazılmıştır. Nitekim, $x=1$ ve $y=0$ olduğu için sıfıra bölme hatası olmakta ve program **catch{}** bloğuna girmektedir. Burada "**Hata oluştu!**" yazılarak kullanıcıya bilgi verilmektedir.

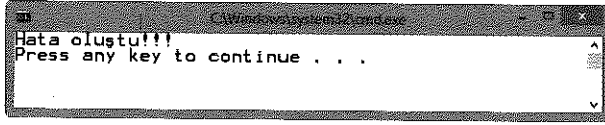
```

1  using System;
2  namespace istisnaTryCatch
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 int x = 1, y = 0;
11                 Console.WriteLine(x / y);
12             }
13
14             catch
15             {
16                 Console.WriteLine("Hata oluştu!!!");
17             }
18         }
19     }
20 }

```

Şekil 16.3 try-catch kullanımı

Yukarıdaki kodların çalıştırılması sonucu ekran çıktısı aşağıdaki gibi olur.



Şekil 16.4 catch bloğunun çalışması

Yukarıdaki örnekte kullanıcıya çok net olmasa da, en azından bir hata oluştuğu ile ilgili bilgi verilmekte ve programın göçmesi **catch** bloğu ile engellenmektedir. İleriki konularda daha ayrıntılı hata mesajları vermeyi de öğreneceğiz.

Bir başka örnek daha vermek gerekirse, **try-catch** yapısı aşağıdaki gibi kullanılabilir. Buradaki örnekte **Exception** sınıfı da kullanılmıştır.

Örnek Kod:

```

1  using System;
2  namespace istisna
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 int[] dizi = { 1, 2, 3 };
11                 Console.WriteLine(dizi[5]);
12             }
13             catch(Exception e)
14             {
15                 Console.WriteLine("Bir hata olustu. Açıklama:" + e.Message);
16             }
17         }
18     }
19 }
20

```

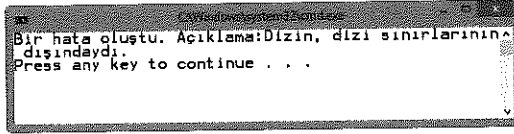
HATA VEREBİLECEK KODLAR

HATA YAKALAMA & AÇIKLAMA

Şekil 16.5 Hata yakalama

Yukarıdaki örnekte de görüldüğü gibi, hata meydana gelme ihtimali olan kodlar **try** bloğu içine koyulur. Herhangi bir hata meydana geldiği anda **try** bloğundaki işlem sonlandırılarak program **catch** bloğuna gider ve burada tespit edilen hata bir mesajla kullanıcıya bildirilebilir. **Catch** bloğunda parametre olarak yazılan **Exception** sınıfı ile yakalanan hata **e** gibi herhangi bir nesneye atanabilir. Hata ile ilgili açıklamalara hata değişkeni değeri (örneğinizde **e.Message**) de eklenerek kullanıcıya oluşan istisna ile ilgili bilgi verilir.

Yukarıdaki kodların çalışması sonucu oluşan hata mesajı aşağıdaki gibidir. Mesajı dikkatlice incellerseniz önce **Console.WriteLine()** metodu içine yazılan “Bir hata olustu. Açıklama:” metni görünür. Hemen onun yanında da **e** değişkeni kullanılarak hata mesajı ekrana yazdırılmaktadır.



Şekil 16.6 Hata mesajı

Doğal olarak, hata mesajları sadece bir istisna/hata oluştuğunda görüntülenir. Eğer **try** bloğu içinde herhangi bir hata olmazsa program **catch** bloğuna girmez ve dolaşısıyla buradaki kodlar çalışmaz.

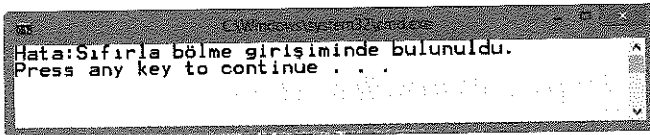
Yukarıdaki örnekte oluşan hatayı yakalamak için kullanılan **Exception** nesnesi genel hata yakalama nesnesidir. **Exception** nesnesi ile **try** bloğunun içinde oluşan her türlü hata yakalanabilir. Örneğin, aşağıdaki kodlarda sıfıra bölme hatası yakalanmakta ve yakalanan hata ile ilgili kullanıcıya bilgi verilmektedir.

```

1  using System;
2  namespace istisnaException
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              int x = 5;
9              int y = 0;
10
11              try
12              {
13                  int z = x / y;
14                  Console.WriteLine(z);
15              }
16
17              catch(Exception e)
18              {
19                  Console.WriteLine("Hata:" + e.Message);
20              }
21          }
22      }
23  }
```

Şekil 16.7 İstisna yönetimi

Yukarıdaki kodların çalışması sonucu aşağıdaki ekran görünür.



Şekil 16.8

Şu ana kadar yaptığımız örneklerde **finally** bloğunu hiç kullanmadık. Daha önce de bahsedildiği gibi, **finally** bloğuna hata olsa da olmasa da çalışmasını istediğimiz kodları koyarız. Örneğin, bir dosya açıp üzerinde işlem yapmayı düşünüyorsunuz. Dosya işlemi başarılı olsa da olmasa da açılan dosyanın kapatılması lazım. Aksi takdirde dosya açık kalır ve bilgisayarın kaynaklarını boşuna tüketmiş olursunuz. İşte bu tür durumlarda dosyayı kapatan kodları **finally** bloğunun içine koyarız. Dosya işlemi başarılı olsa da olmasa da **finally** bloğunun içindeki komut çalışır ve dosyayı kapatır. Eğer bir sonraki bölümde bulunan dosya işlemlerini gözden geçirirseniz dosya kapatma kodlarının **finally** bloğunun içinde yazıldığını görebilirsiniz.

Basit bir örnekle **finally** bloğunun uygulamasını yapalım. Aşağıdaki kodlarda ekrana **merhaba** yazdırıyoruz. Bir hata olması durumunda hata ile ilgili bilgi almak için **Console.WriteLine("merhaba");** metodunu **try** bloğunun içine koyuyoruz. Son olarak da **finally** bloğunun içinde "**finally çalıştı**" yazdırıyoruz. Son yazılan bloğun, yani **finally** bloğunun içindeki kodlar her halükarda çalışır.

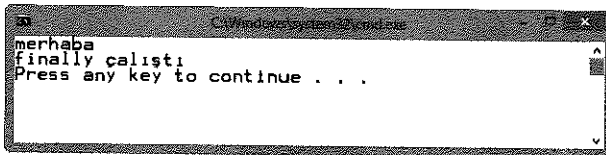
```

1  using System;
2  namespace istisnaFinally
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 Console.WriteLine("merhaba");
11             }
12
13             catch (Exception ex)
14             {
15                 Console.WriteLine("Hata:" + ex.Message);
16             }
17
18             finally
19             {
20                 Console.WriteLine("finally çalıştı");
21             }
22         }
23     }
24 }

```

Şekil 16.9 finally bloğunun kullanımı

Yukarıdaki kodların çalışması sonucu aşağıdaki ekran görüntüsü elde edilir.



Şekil 16.10

Yukarıdaki örneğimizde **try** bloğunun içinde bir hata olmadığı için **catch** bloğu içindeki kodlar çalışmadı. En altta bulunan **finally** bloğundaki kodlar her halükarda çalıştığı için ekrana "**finally çalıştı**" yazıldı.

Aşağıdaki örnekte ise sıfıra bölme işleminden dolayı **try** bloğunda bir hata oluşmakta ve **catch** bloğundaki hata yakalama kodları çalışmaktadır. Son olarak, **finally** bloğundaki kodların çalıştığını görebilirsiniz.

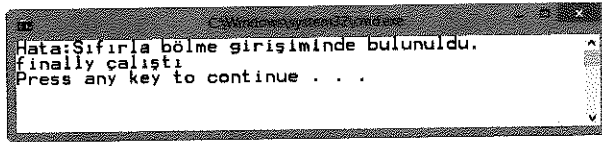
```

1  using System;
2  namespace istisnaFinally
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 int a = 1;
11                 int b = 0;
12                 Console.WriteLine(a/b);
13             }
14             catch (Exception ex)
15             {
16                 Console.WriteLine("Hata:" + ex.Message);
17             }
18             finally
19             {
20                 Console.WriteLine("finally çalıştı");
21             }
22         }
23     }
24 }
25
26

```

Şekil 16.11 sıfıra bölme hatası

Yukarıdaki kodların çalışması sonucu ekran çıktısı aşağıdaki gibi olur.



Şekil 16.12 catch{} ve finally{} bloklarının çalışması

Çoklu Hata Yakalama Blokları

Yukarıdaki kodlarda hataları yakalamak için **catch** bloğunu ve **Exception** sınıfını kullandık. Daha önce de bahsedildiği gibi, **Exception** sınıfı genel bir hata yakalama

sınıfıdır. Bunun anlamı, meydana gelen her türlü hatanın yakalanması ve kullanıcıya bildirilmesidir. **Exception** sınıfını kullanmak için **catch** bloğunda parantez içine **Exception** sınıfını koyduk ve **catch(Exception e)** yazarak **Exception** sınıfından **e** adlı bir nesne oluşturduk. Burada kullanılan **e** yerine herhangi bir isim de verilebilir. Biz örneklerimizde **e** ve **ex** isimlerini kullandık. Oluşturulan nesnenin üyelerini kullanarak kullanıcıya değişik bilgiler verebiliriz. Örneğin, yukarıdaki kodlarda **e.Message** yazarak kullanıcıya hata mesajı gösterdik.

Exception sınıfının sıkça kullanılan üyeleri aşağıdaki gibidir:

Üye	Açıklama
Message	Oluşan hata mesajı ile ilgili string türünden değer döndürür.
Source	İstisnanın gönderildiği sınıf/aduzayının adı.
HelpLink	Varsa, oluşan hata ile ilgili yardım dosyasının yol bilgisi.
InnerException	Catch{} bloğu içinde bir hata olursa catch{} bloğuna girilmesini sağlar.

Yukarıdaki tabloda anlatılan üyelerin kullanımına ilişkin kodlar aşağıdaki gibi olabilir.

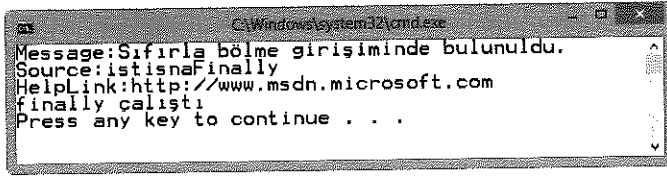
```

1  using System;
2  namespace istisnaFinally
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 int a = 1;
11                 int b = 0;
12                 Console.WriteLine(a/b);
13             }
14
15             catch (Exception ex)
16             {
17                 Console.WriteLine("Message:" + ex.Message);
18                 Console.WriteLine("Source:" + ex.Source);
19                 ex.HelpLink = "http://www.msdn.microsoft.com";
20                 Console.WriteLine("HelpLink:" + ex.HelpLink);
21             }
22
23             finally
24             {
25                 Console.WriteLine("finally çalıştı");
26             }
27         }
28     }
29 }

```

Şekil 16.13 Exception üyeleri

Yukarıda çalışan kodların çıktısı aşağıdaki gibi olur. Buradaki çıktıda hata mesajı, hatanın kaynağı (namespace istisnaFinally) ve hata ile ilgili yardım linki görülebilir.



Şekil 16.14 Exception üyeleri

Gelelim konumuzun da başlığı olan çoklu hata yakalama bloklarına... **Exception** sınıfı her ne kadar **try** bloğu içindeki hataları yakalıyor olsa da, bazen farklı nedenlerden kaynaklanan farklı türde hatalar olabilir ve bu hataları ayrı ayrı yakalamak gerekebilir. İşte bu tür durumlarda farklı **catch** blokları kullanılabilir. Örneğin, sıfıra bölme hatası için ayrı, yanlış veri türü için ayrı **catch** blokları kullanılabilir. Aşağıdaki kodlarda buna bir örnek görebilirsiniz.

```

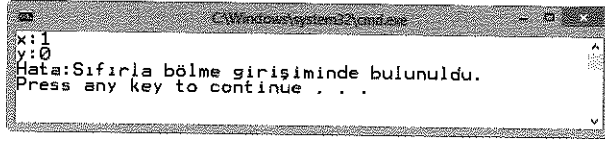
1  using System;
2  namespace istisnaCokluCatch
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              try
9              {
10                 Console.Write("x:");
11                 int x = int.Parse(Console.ReadLine());
12                 Console.Write("y:");
13                 int y = int.Parse(Console.ReadLine());
14                 Console.WriteLine(x / y);
15             }
16
17             catch(DivideByZeroException ez)
18             {
19                 Console.WriteLine("Hata:" + ez.Message);
20             }
21
22             catch(OverflowException eo)
23             {
24                 Console.WriteLine("Hata:" + eo.Message);
25             }
26         }
27     }
28 }

```

Şekil 16.15 Çoklu catch kullanımı

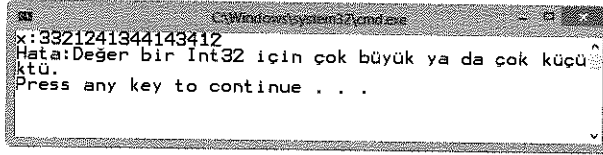
Yukarıdaki kodları dikkatlice incelerseniz iki tane **catch** bloğu olduğunu görebilirsiniz. Bunlardan ilki sıfıra bölme (**DivideByZeroException**) hatalarını yakalarken, ikincisi de taşma (**OverflowException**) hatalarını yakalamaktadır.

Yukarıdaki programı çalıştırıp x'e 1 ve y'ye sıfır değerlerini girerseniz aşağıdaki gibi sıfıra bölme hatası alırsınız.



Şekil 16.16 Sıfıra bölme hatası

Eğer x'e `int` veri tipi sınırlarını aşan bir değer atarsanız aşağıdaki gibi taşma hatası alırsınız.



Şekil 16.17 Taşma hatası

Yukarıdaki örnekte de görüldüğü gibi, bir `try` bloğunda birden fazla istisna oluşma ihtimali varsa, her istisnayı yakalamak için ayrı `catch` bloğu yazılabilir. Bu durumda yapmanız gereken şey her istisna için ilgili sınıfı kullanmaktır.

C# dilinde sıklıkla kullanılan istisna sınıflarının bazıları ve kısa açıklaması aşağıdaki gibidir:

İstisna Sınıfı	Ne Zaman Kullanılır?
<code>ArgumentException</code>	Metoda verilen parametrelerden biri geçerli değilse.
<code>ArithmeticException</code>	Yanlış aritmetik işlem ve yanlış veri tipi dönüşümlerinde.
<code>DivideByZeroException</code>	Sıfıra bölme hatalarında
<code>IndexOutOfRangeException</code>	Dizide olmayan bir indis kullanıldığında
<code>InsufficientMemoryException</code>	Yetersiz bellek olduğunda
<code>OutOfMemoryException</code>	Programın çalışması için gerekli olan bellek ükendiğinde
<code>OverflowException</code>	Veri tipi değer aralığının dışına taşıldığında
<code>IOException</code>	Giriş-çıkış hatası meydana geldiğinde

Eğer C# dilinde kullanılan istisna sınıfları ile ilgili daha ayrıntılı bilgi almak isterseniz şu linki ziyaret edebilirsiniz: <https://mikevallotton.wordpress.com/2009/07/08/net-exceptions-all-of-them/>

throw (fırlat) Komutu

Eğer `try` bloğu içinde bir istisna/hata meydana gelirse istisnaya ilgili sınıf otoma-

tik olarak **catch** bloğuna gönderilir/fırlatılır ve orada gerekli kodlar yazılarak kullanıcıya bilgi verilir. Örneğin, **try** bloğu içinde **taşma hatası** meydana gelirse hatayla ilgili olan **OverflowException** sınıfı **catch** bloğuna fırlatılır. **Catch** bloğuna fırlatılan **OverflowException** sınıfının **Message** gibi özellikleri kullanılarak kullanıcıya bilgi verilir.

Gerekirse **try** bloğu içinde belli durumlar gerçekleştiğinde kendimiz de bir istisna fırlatabiliriz. Bu işlem **throw** komutu ile yapılır. Örneğin, bir dairenin yarıçapı sıfırdan küçük girilirse **ArithmeticException** sınıfını **catch** bloğuna fırlatabiliriz. Örneğin, **throw** komutunu aşağıdaki gibi kullanarak **catch** bloğuna istisna fırlatabiliriz.

```
throw new ArithmeticException("Sıfırdan küçük değer girmeyiniz!");
```

Yukarıdaki örnekte **ArithmeticException** sınıfının yapıcı metodu kullanılarak hata mesajı belirleniyor.

Eğer isterseniz istisna nesnesi oluşturup özelliklerini de atayabilirsiniz. Bu durumda kodlar aşağıdaki gibi olabilir.

```
ArithmeticException ex=new ArithmeticException();
ex.Message="Sıfırdan küçük değer girmeyiniz!"
ex.HelpLink=http://www.bilimsel.com.tr/hatalar/exarithmetic;
```

Yukarıdaki kodlarda ise hata mesajı **ArithmeticException** sınıfının yapıcı metodu yerine **Message** özelliği kullanılarak belirleniyor. Ayrıca, **HelpLink** özelliği kullanılarak hata hakkında ayrıntılı bilgi alınabilecek link belirleniyor.

Yukarıdaki temel bilgileri kullanarak sıfırdan küçük bir yarıçap değeri girildiğinde istisna fırlatılmasını istediğimiz bir daire sınıfı oluşturmak istersek kodlar aşağıdaki gibi olabilir.

```

1  using System;
2  namespace throwDaire
3  {
4      class Daire
5      {
6          private double yaricap;    //alan
7
8          public double Yaricap      //Özellik
9          {
10             get { return yaricap; }
11             set
12             {
13                 if (value < 0)
14                     throw new ArithmeticException("Sıfırdan küçük değer girildi!");
15                 else
16                     this.yaricap = value;
17             }
18         }
19     }

```

Şekil 16.18

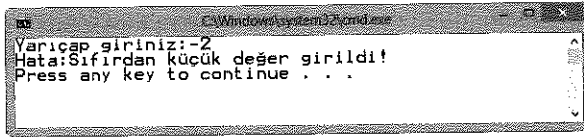
```

20
21
22     class Program
23     {
24         static void Main(string[] args)
25         {
26             try
27             {
28                 Daire d = new Daire();
29                 Console.Write("Yarıçap giriniz:");
30                 d.Yaricap = Convert.ToDouble(Console.ReadLine());
31                 Console.WriteLine("Daire Alanı=" + Math.PI * Math.Pow(d.Yaricap, 2));
32             }
33
34             //eğer yarıçap sıfırdan küçükse
35             catch(ArithmeticException ex)
36             {
37                 Console.WriteLine("Hata:" + ex.Message);
38             }
39         }
40     }

```

Şekil 16.19 throw komutu ile özel istisna fırlatma

Yukarıdaki kodları çalıştırdığınızda size dairenin yarıçapı sorulur. Eğer sıfır veya sıfırdan büyük bir değer girerseniz istisna fırlatılmaz ve ekranda dairenin alanını görürsünüz. Eğer sıfırdan küçük bir değer girerseniz **throw** komutu ile istisna fırlatılır ve **try** bloğundaki komutların çalışması sona erdirilerek hata mesajı gösterilir. Aşağıdaki ekranda -2 değeri girildiğinde istisna fırlatılmış ve ekrana hata mesajı yazılmıştır.



Şekil 16.20 Fırlatılan istisna mesajı

Aşağıdaki örnekte dikdörtgen alanını hesaplayan bir sınıf görünmektedir. Dikdörtgenin genişlik ve yükseklik değerleri parametrelili yapıcı metod kullanılarak alınmaktadır. Eğer parametrelerden biri sıfırdan küçükse istisna fırlatılmaktadır.

```

1  using System;
2  namespace throwDikdortgen
3  {
4      class Dikdortgen
5      {
6          private int genislik, yukseklik;           //alanlar
7
8          public Dikdortgen(int genislik, int yukseklik) //yapıcı metod
9          {
10             if (genislik < 0 || yukseklik < 0)
11                 throw new ArithmeticException("Girilen değer sıfırdan küçük!");
12             else
13             {
14                 this.yukseklik = yukseklik;
15                 this.genislik = genislik;
16             }
17         }
18
19         public void Alan()
20         {
21             Console.WriteLine("Alan=" + genislik * yukseklik);
22         }
23     }

```

Şekil 16.21

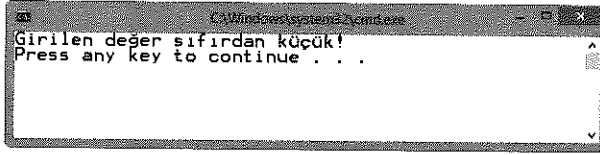
```

24
25 class Program
26 {
27     static void Main(string[] args)
28     {
29         try
30         {
31             Dikdortgen d = new Dikdortgen(-1, 3);
32             d.Alan();
33         }
34
35         catch(ArithmeticException e)
36         {
37             Console.WriteLine(e.Message);
38         }
39     }
40 }
41
42

```

Şekil 16.22 Yapıcı metod ile parametre fırlatma

Yukarıdaki kodları çalıştırınca aşağıdaki gibi bir ekran çıkmaktadır. Yukarıdaki kodlarda yapıcı metoda (-1,3) değerleri girildiği için istisna fırlatılmaktadır. Bu değerleri sıfırdan büyük yaparak programın normal bir şekilde çalıştığını görebilirsiniz.



Şekil 16.23 Ekran çıktısı ve hata mesajı

17

Dosya İşlemleri

- Klasör İşlemleri
- Directory Sınıfı
- Dosya İşlemleri
- File Sınıfı
- FileStream Sınıfı
- StreamWriter ve StreamReader Sınıfları
- Dosya İşlemlerinde İstisna Yönetimi
- Uygulamalar

Dosya İşlemleri

Günümüzde verileri saklamak ve veriler üzerinde işlem yapmak için genellikle veritabanları kullanılıyor olsa da temel düzeyde dosya işlemlerini bilmekte fayda vardır. Bu nedenle temel düzeyde dosya ve klasör işlemlerini gerçekleştirmeye çalışacağız.

C# programlama dilinde dosya ve klasör işlemleri **System.IO** aduzayında bulunan sınıflar kullanılarak yapılır. **System.IO** aduzayı, adından da belli olduğu gibi, bilgisayara giren-çıkan tüm verileri kontrol etmeye yarayan sınıfları barındırır. Bu nedenle eğer dosya veya klasörlerle ilgili işlem yapacaksak kodların en üstüne **using System.IO;** komutunu koymak zorundayız.

Dosya ve klasör işlemleri için **Directory**, **File**, **FileStream**, **StreamWriter**, **StreamReader**, **Path**, **FileInfo** ve **DirectoryInfo** sınıfları kullanılmaktadır. Bu sınıflarla ilgili temel açıklamaları aşağıdaki tabloda bulabilirsiniz.

Sınıf	Açıklama
Directory	Klasör işlemleri yapmak için kullanılır.
File	Dosya işlemleri yapmak için kullanılır.
FileStream	Dosya işlemleri için kullanılır.
StreamWriter	Dosyalara yazmak için kullanılır.
StreamReader	Dosyaları okumak için kullanılır.
Path	Dosya ve klasörlerin yolunu/adresini (path) tutmak için kullanılır.
FileInfo	Dosyalarla ilgili bilgi almak için kullanılır.
DirectoryInfo	Klasörlerle ilgili bilgi almak için kullanılır.

Yukarıdaki tabloda kısaca bahsedilen sınıflarla ilgili ayrıntılı bilgileri aşağıdaki başlıklar altında bulabilirsiniz.

Klasör İşlemleri

Yukarıdaki tabloda da bahsedildiği gibi, klasör işlemleri için **Directory** sınıfı kullanılır. Klasör oluşturma, silme ve diğer işlemler için kullanılan **Directory** sınıfının metodlarını ve açıklamalarını aşağıdaki tabloda bulabilirsiniz.

Metot	Açıklama
CreateDirectory("yol")	Belirtilen yola/adrese sahip bir klasör oluşturur.
Delete("yol", true/false)	Belirtilen yola/adrese sahip boş klasörü siler. Eğer metodun ikinci parametresi true ise klasörü içindeki dosya ve alt klasörlerle siler.
Move("kaynak", "hedef")	Kaynak adresindeki klasör hedef adrese taşınır.
Exist("yol")	Belirtilen adrese sahip bir klasör olup olmadığını true/false olarak döndürür. Klasör varsa true yoksa false döner.
GetCurrentDirectory()	Çalışan yazılımın hangi klasörde olduğunu döndürür.
GetDirectoryRoot("yol")	Belirtilen adrese sahip klasörün kök dizinini döndürür.
GetFiles("yol")	Belirtilen adrese sahip klasör içindeki dosya adlarını string tipinde bir dizi olarak döndürür.
GetLogicalDrives()	Bilgisardaki sürücü adlarını string tipinden bir dizi olarak döndürür.

Örnek Kod:

C:\ sürücüsünde "**deneme**" adlı bir klasör oluşturmak için aşağıdaki kodları yazabilirsiniz.

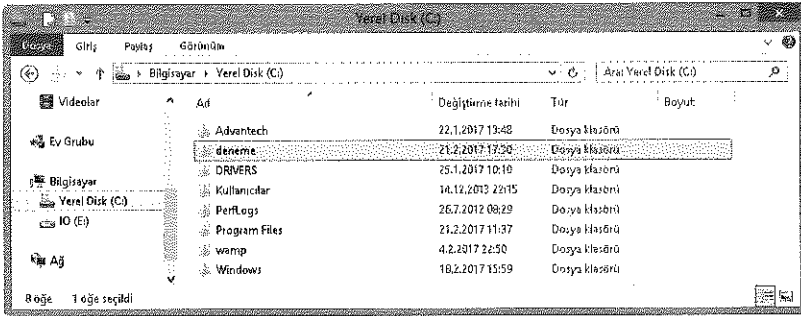
```

1  using System;
2  using System.IO;
3
4  namespace createDirectoryC
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             Directory.CreateDirectory(@"C:\deneme");
11          }
12      }
13  }

```

Şekil 17.1 Klasör oluşturma

Yukarıdaki kodları yazıp çalıştırdıktan sonra C sürücüsünde “deneme” adlı bir klasör oluşturulduğunu görebilirsiniz.



Şekil 17.2 deneme klasörü

Örnek Kod:

Bir önceki örnek kodda oluşturduğumuz klasörü silen kodları yazalım.

```

1  using System;
2  using System.IO;
3  namespace deleteDirectoryDeneme
4  {
5      class Program
6      {
7          static void Main(string[] args)
8          {
9              string klasor = @"C:\deneme";
10
11             //klasörün içi doluysa, içindeki dosya ve alt klasörleri de sil
12             Directory.Delete(klasor, true);
13         }
14     }
15 }

```

Şekil 17.3 Klasör silme

Yukarıdaki kodların çalışması sonucu daha önce C:\ sürücüsünde oluşturulan “deneme” klasörü içindeki dosya ve alt klasörlerle beraber silinir. Eğer **Delete()** meto-

dunda ikinci parametre olarak **true** değeri verilmezse sadece içi boş olan klasör silinir.

Örnek Kod:

Bu örneğimizde belirli bir klasörün olup olmadığını belirleyeceğiz. Eğer aranan klasör varsa **true**, yoksa **false** değeri döner. Eğer aranan klasör varsa klasör içindeki alt klasörleri bir diziye yerleştireceğiz. Bu örneğimizde C:\ sürücüsünde **Windows** adlı bir klasörün olup olmadığını kontrol edeceğiz. Eğer klasör varsa alt klasörlerini **string** tipindeki bir diziye atıp alt alta yazdıracacağız.

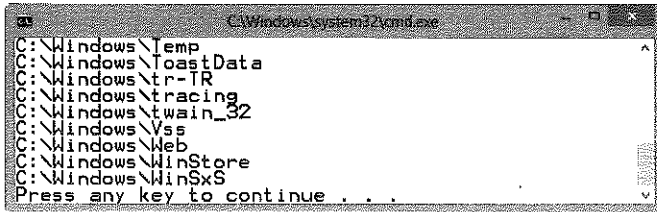
```

1  using System;
2  using System.IO;
3  namespace klasorExist
4  {
5      class Program
6      {
7          static void Main(string[] args)
8          {
9              //aranan klasor
10             string klasor = @"C:\Windows";
11
12             //eğer klasor varsa
13             if(Directory.Exists(klasor))
14             {
15                 Console.WriteLine("Aranan klasör bulundu.");
16                 Console.WriteLine("Alt klasör listesi aşağıdadır:");
17
18                 //alt klasorleri diziye ata ve yazdır
19                 string[] altklasor = Directory.GetDirectories(klasor);
20                 foreach (string k in altklasor)
21                     Console.WriteLine(k);
22             }
23         }
24     }
25 }

```

Şekil 17.4 Klasör kontrolü ve alt klasör listesi

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir çıktı elde edersiniz.



Şekil 17.5 Alt klasör listesi

Eğer bir değişiklik ya da farklı bir sürücüye kurulum yapılmadıysa, her **Windows** işletim sisteminin C:\ sürücüsünde **Windows** adlı bir klasör bulunmaktadır. Yukarıdaki örneğimizde de Windows adlı bir klasör bulunmakta ve klasörün alt klasörleri listelenmektedir.

Örnek Kod:

Bu örneğimizde **C:\Windows** adresli bir klasör olup olmadığını kontrol edeceğiz. Eğer bu adrese sahip bir klasör varsa klasör içindeki dosyaları bir diziye atayıp alt alta yazdıracağız.

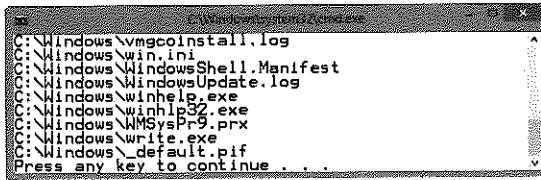
```

1  using System;
2  using System.IO;
3  namespace DirectoryGetFiles
4  {
5      class Program
6      {
7          static void Main(string[] args)
8          {
9              //aranan klasör
10             string klasor = @"C:\Windows";
11
12             //eğer klasör varsa
13             if(Directory.Exists(klasor))
14             {
15                 //klasör içindeki dosyaları diziye ata
16                 string[] dosyalar = Directory.GetFiles(klasor);
17
18                 //dizi elemanları yaz
19                 foreach (string dosya in dosyalar)
20                     Console.WriteLine(dosya);
21             }
22         }
23     }
24 }
25

```

Şekil 17.6 Klasör içindeki dosyaların diziye atanması

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir ekran elde ederiz. Bu ekranda **C:\Windows** adresindeki dosyalar listelenmektedir.



Şekil 17.7 Dosya listesi

Dosya İşlemleri

Yukarıdaki tabloda da bahsedildiği gibi, dosya işlemleri için **File** sınıfı kullanılabilir. **File** sınıfında bulunan metotların adları **Directory** sınıfındaki metotlarla aynıdır ve benzer şekilde çalışır. Bu metotların dışında dosyalama işlemlerinde kullanılan farklı metotlar da bulunmaktadır. Dosya oluşturma, silme ve diğer dosya işlemleri için kullanılan metotları ve açıklamalarını aşağıdaki tabloda bulabilirsiniz.

Metot	Açıklama
Create("yol")	Belirtilen yola/adrese sahip bir dosya oluşturur.
CreateText("yol")	UTF-8 olarak kodlanmış bir dosya oluşturur.
Delete("yol")	Belirtilen yola/adrese sahip dosyayı siler.
Move("kaynak", "hedef")	Kaynak adresindeki dosya hedef adrese taşınır.
Exist("yol")	Belirtilen adrese sahip bir dosya olup olmadığını true/false olarak döndürür. Klasör varsa true yoksa false döner.
Open("yol")	Belirtilen yol için bir FileStream nesnesi oluşturur.
OpenRead("yol")	Belirtilen yoldaki dosyayı okumak üzere açar.
OpenWrite("yol")	Belirtilen yoldaki dosyayı yazmak üzere açar.
Copy("kaynak", "hedef", "true/false")	Kaynak adresindeki dosya hedef adrese kopyalanır. Eğer üçüncü parametre olarak true verilirse ve aynı isimli bir dosya varsa üstüne yazılır.
ReadAllText("yol")	Metin dosyasını açarak bütün metni okur ve string tipindeki bir değişkene atayarak dosyayı kapatır.
ReadAllBytes("yol")	Resim gibi ikili (binary) dosyaları açarak bütün satırları okur ve string tipindeki bir değişkene atayarak dosyayı kapatır.
WriteAllText("yol", "metin")	Bir metin dosyası açar ve metinleri dosyaya yazıp işlem bittikten sonra dosyayı kapatır.
WriteAllBytes("yol", veri)	Bir binary dosyası açar ve bir byte dizisi içindeki verileri dosyaya yazıp işlem bittikten sonra dosyayı kapatır.
GetAttributes("yol")	Dosya özelliklerini döndürür.
AppendText("yol", "metin")	Bir metin dosyası açar ve var olan dosya içine UTF-8 olarak kodlanmış metin ekler.

Örnek Kod:

Aşağıdaki kodlarda **C:\test** klasöründe bulunan **yaz.txt** adlı dosyaya string tipindeki değişken değeri yazdırılıyor. Bu işlem için **File** sınıfının **WriteAllText()** metodu kullanılıyor.

```
using System;
using System.IO;

namespace fileWriteAllText
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        //dosya adresi
        string dosya = @"C:\test\yaz.txt";

        //yazdırılacak olan metin
        string metin = "Fahrettin Erding";

        //File sınıfı ile dosyaya yazdırma
        File.WriteAllText(dosya,metin);

        Console.WriteLine("İşlem tamamlandı...");
    }
}

```

Örnek Kod:

```

using System;
using System.IO;

namespace fileReadAllText
{
    class Program
    {
        static void Main(string[] args)
        {
            //okunacak dosya adresi
            string dosya = @"C:\test\yaz.txt";

            //dosya içeriğinin atanacağı değişken
            string okunan = "";

            //File sınıfı ile dosyadan okuma
            okunan = File.ReadAllText(dosya);

            //okunan metni konsola yaz
            Console.WriteLine(okunan);

            Console.WriteLine("İşlem tamamlandı...");
        }
    }
}

```

FileStream Sınıfı

Dosya işlemlerinde **File** sınıfına ek olarak **FileStream**, **StreamReader** ve **StreamWriter** sınıfları da kullanılmaktadır. **FileStream** sınıfı ile diskte bulunan bir dosya açılarak **StreamReader** ile okuma, **StreamWriter** ile ise yazma işlemi yapılır. Eğer binary dosyalarla çalışacaksanız okuma için **BinaryReader**, yazma için **BinaryWriter** metodlarını kullanmalısınız.

FileStream sınıfını kullanarak dosya açarken sözdizimi aşağıdaki gibi olmalıdır:

```
FileStream(String, FileMode, FileAccess, FileShare, Int32, FileOptions)
```

FileStream sınıfından bir nesne tanımlarken kullanılabileceğiniz parametreler ve açıklamaları aşağıdaki gibidir:

Parametre	Açıklama
FileMode	Dosya oluşturma, dosya okuma veya dosya açma işlemlerinin yapılmasını sağlar. Yapacağınız işlemi aşağıdaki parametrelerden birini kullanarak belirleyebilirsiniz. Create: Yeni bir dosya oluşturur. Eğer aynı ada sahip bir dosya varsa onu açar ve bilgileri onun üzerine yazar. CreateNew: Yeni dosya oluşturur. Eğer aynı ada sahip dosya varsa hata mesajı alırsınız. Open: Dosya açar. OpenOrCreate: Eğer belirtilen dosya varsa açar, yoksa oluşturur. Append: Dosyayı açar ve sonuna ekleme yapar. Eğer dosya yoksa oluşturulur. Truncate: Dosya açılarak içi silinir.
FAccess	Açılan dosya üzerinde yapılacak olan işlemi belirler. Aşağıdaki 3 parametreyi kullanarak dosya erişiminin amacını belirleyebilirsiniz. Read: Dosya sadece okuma amaçlı açılır. Yazma yapılamaz. Write: Dosya sadece yazma amaçlı açılır. Okuma yapılamaz. ReadWrite: Dosya hem okuma hem yazma amaçlı açılır.

File Share	Dosyanın nasıl paylaşılacağını belirler. Örneğin, siz dosyayı açmış ve üzerinde işlem yapıyorken başkalarının da işlem yapıp yapamayacağını bu parametre belirler. None: Dosyayı sadece bir kişi kullanabilir. Dosya açık olduğu sürece başka işlem tarafından kullanılamaz. Read: Dosya açık iken başka işlemler tarafından da açılıp okunabilir. Write: Dosya açık iken başka işlemler tarafından da açılıp yazılabilir. ReadWrite: Dosya açık iken başka işlemler tarafından da açılıp okunabilir ve yazılabilir.
Int32	Buffer boyutu.

Örnek Kod:

FileStream sınıfını kullanarak **C:\deneme** klasöründeki **uygulama.txt** adlı dosyayı açalım. Dosyayı **FileMode.Create** parametresi ile açacağız. Bu parametre yeni bir dosya oluşturur. Eğer aynı ada sahip başka bir dosya varsa onu açar.

```
using System;
using System.IO;
namespace dosyaFileStreamStreamWriter
{
    class Program
    {
        static void Main(string[] args)
        {
            //çalışacağımız dosya
            string dosya = @"C:\deneme\uygulama.txt";

            //FileStream ile dosyayı açalım
            FileStream f = new FileStream(dosya, FileMode.Create,
            FileAccess.Write, FileShare.Write);

            //İşlemin bittiğini bildirelim
            Console.WriteLine("Dosya açma işlemi tamamlandı...");
        }
    }
}
```

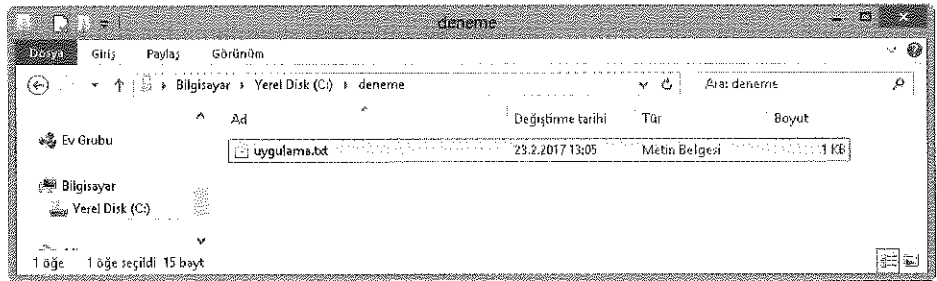
Dosyaya Yazma İşlemi & StreamWriter

FileStream sınıfı ile dosyaları açıktan sonra dosyaya veri yazmak için **StreamWriter** sınıfı kullanılır. **StreamWriter** sınıfının metotları ve açıklamaları aşağıdaki gibidir.

Metot	Açıklama
Write(veri)	Açılan dosyaya veri yazmak için kullanılır. Bu veri karakter, metin, sayı veya bir değişken olabilir.
WriteLine(veri)	Yazma işlemi yaptıktan sonra alt satıra geçer. Yani, veri sonuna "n" koyar.
Flush()	Tampondaki verilerin boşaltılmasını ve dosyanın güncellenmesini sağlar.

Örnek Kod:

String tipindeki bir değişken değerini **C:\deneme\uygulama.txt** dosyasına yazmak istiyoruz. Öncelikle C:\ sürücüsünde **deneme** klasörünü ve deneme klasörü içindeki **uygulama.txt** dosyasını oluşturuyoruz.



Şekil 17.8 deneme klasörü ve uygulama.txt dosyası

Oluşturduğumuz dosyaya değişken değerini yazmak için aşağıdaki kodları kullanabilirsiniz. Buradaki kodlarda **FileStream** ve **StreamWriter** sınıfları beraber kullanılarak dosyaya yazma işlemi gerçekleştirilmiştir.

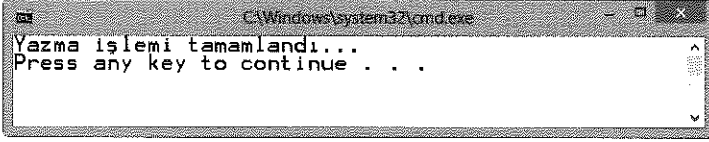
```

1  using System;
2  using System.IO;
3  namespace dosyaFileStreamStreamWriter
4  {
5      class Program
6      {
7          static void Main(string[] args)
8          {
9              //çalışacağımız dosya
10             string dosya = @"C:\deneme\uygulama.txt";
11
12             //dosyaya yazacağımız değişken ve değeri
13             string yaz = "Merhaba Mete!!!";
14
15             //FileStream ile dosyayı açalım
16             FileStream f = new FileStream(dosya, FileMode.Create, FileAccess.Write, FileShare.Write);
17
18             //StreamWriter nesnesi oluşturalım
19             StreamWriter sw = new StreamWriter(f);
20
21             //değişken değerini dosyaya yazıp dosyayı kapatalım
22             sw.Write(yaz);
23             sw.Close();
24
25             //İşlemin bittiğini bildirelim
26             Console.WriteLine("Yazma işlemi tamamlandı...");
27         }
28     }
29 }

```

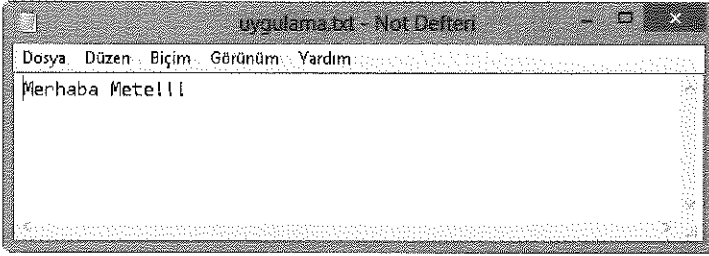
Şekil 17.9 Program kodları

Yukarıdaki kodların çalışması sonucu aşağıdaki ekranı görürsünüz.



Şekil 17.10 Ekran çıktısı

Yukarıdaki ekranı gördükten sonra pencereyi kapatın ve **C:\deneme\uygulama.txt** dosyasını açın. Dosya içinde değişken değerini göreceksiniz.



Şekil 17.11 Dosyaya eklenen değişken değeri

Yukarıdaki kodlarda görülen değişken değerini değiştirerek dosyaya farklı metinler yazabilirsiniz. Her defasında bir önceki değer silindiğini ve yeni değer yazıldığını göreceksiniz. Eğer önceki değerin silinmeden, yeni değişken değerinin alt satıra yazılmasını istiyorsanız **FileStream** sınıfında **FileMode.Append** parametresini kullanmalısınız.

Dosyadan Okuma İşlemi & StreamReader

FileStream sınıfı ile dosyaları açıktan sonra dosyadan veri okumak için **StreamReader** sınıfı kullanılır. **StreamReader** sınıfının metotları ve açıklamaları aşağıdaki gibidir.

Metot	Açıklama
Read()	Açılan dosyadan bir karakter okumak için kullanılır. Okunan karakter int tipine dönüştürülerek geri döndürülür.
Read(char[] dizi, indx, adet)	Açılan dosyadan belirtilen adette karakteri diziye indx indeksinden itibaren yerleştirir.
ReadLine()	Açılan dosyadan bir satır okumak için kullanılır.
ReadToEnd()	Dosyadaki verilerin tamamı okunur.

Örnek Kod:

C:\deneme\uygulama.txt dosyasında bulunan verilerin tamamını okuyup string tipindeki bir değişkene atamak istiyoruz. Bunun için aşağıdaki kodları yazabiliriz.

```

1  using System;
2  using System.IO;
3
4  namespace dosyaFileStreamStreamReader
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             //okuma yapılacak dosya
11             string dosya = @"C:\deneme\uygulama.txt";
12
13             //okunan verilerin atanacağı değişken
14             string okunan = "";
15
16             //dosyayı açalım
17             FileStream f = new FileStream(dosya, FileMode.Open);
18
19             //Açılan dosya için StreamReader nesnemizi oluşturalım
20             StreamReader sr = new StreamReader(f);
21
22             //dosyadaki tüm veriyi oku ve "okunan" değişkenine ata
23             okunan=sr.ReadToEnd();
24
25             //"okunan" değişkenini yaz
26             Console.WriteLine(okunan);
27         }
28     }
29 }

```

Şekil 17.12 StreamReader nesnesi ile dosya okuma

Dosya okuma ve yazma işlemlerinde bir önceki bölümde öğrendiğimiz “**İstisna Yönetimi**” konusu çok önemlidir. Yanlış klasör veya dosya adı veya arızalı sürücü gibi nedenlerden dolayı sıklıkla istisnalarla karşılaşılabilir. Bu nedenle dosya işlemlerinde oluşan istisnaları yakalamak için kullanılan **IOException** nesnesini kullanmakta büyük fayda var. Aşağıda yapacağımız uygulamalarda buna örnek görebilirsiniz.

Uygulamalar

Klasör Oluşturma & Try-Catch Kullanımı

İlk uygulamamızda C sürücüsünde "uygulama" adlı bir klasör oluşturacağız. Herhangi bir istisna gerçekleşme ihtimaline karşı **try-cath** yapısını ve **IOException** nesnesini kullanacağız.

```

1  using System;
2  using System.IO;
3
4  namespace uygulamaKlasorOlusturTryCatch
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             //klasör ve adresi
11             string klasor = @"C:\uygulama";
12
13             //istisna olma ihtimaline karşı try bloğu
14             try
15             {
16                 //klasörü oluşturalım
17                 Directory.CreateDirectory(klasor);
18
19                 //Klasör oluştuyorsa bilgi verelim
20                 Console.WriteLine("Klasör oluşturuldu...");
21             }
22
23             //istisnanın yakalanması ve mesaj
24             catch(IOException ex)
25             {
26                 Console.WriteLine("Hata: " + ex.Message);
27             }
28         }
29     }
30 }

```

Şekil 17.13 Klasör oluşturma

Dosyaya Yazma & Try-Catch Kullanımı

Yukarıdaki örnekte oluşturulan “uygulama” klasörünün içinde “test.txt” adlı bir dosya oluşturulalım.

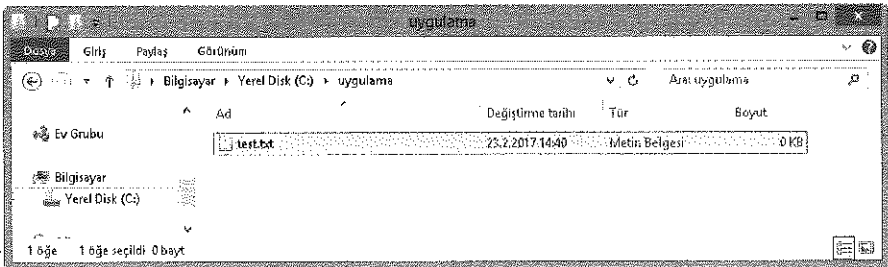
```

1  using System;
2  using System.IO;
3
4  namespace dosyaOlusturFileStream
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             //dosya adresi
11             string dosya = @"C:\uygulama\test.txt";
12
13             //istisna olma ihtimaline karşın try bloğu
14             try
15             {
16                 //dosyayı oluşturalım
17                 FileStream f = new FileStream(dosya, FileMode.Create);
18
19                 //dosyanın oluşturulduğunu bildirelim
20                 Console.WriteLine("Dosya oluşturuldu...");
21             }
22
23             //istisna oluşursa kullanıcıya bilgi verelim
24             catch(IOException e)
25             {
26                 Console.WriteLine("Hata oluştu:" + e.Message);
27             }
28         }
29     }
30 }

```

Şekil 17.14 Dosya oluşturma

Yukarıdaki kodların çalışmasından sonra **C:\uygulama** klasörünün içine bakarsanız “test.txt” adlı dosyanın oluşturulduğunu görebilirsiniz.



Şekil 17.15 test.txt dosyası

Dosyaya Veri Yazma & Try-Catch Kullanımı

Bu uygulamamızda önceki uygulamalarda oluşturduğumuz **C:\uygulama\test.txt** adresindeki dosyaya konsoldan okunan bir verinin yazılmasını kodlayacağız. Örnek olarak konsoldan kullanıcının adını isteyeceğiz ve **test.txt** dosyasına yazacağız. İstisna oluşması ihtimaline karşı **try-catch** bloğunu kullanacağız.

```

1  using System;
2  using System.IO;
3
4  namespace uygulamaFileStreamStreamWriter
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             //çalışacağımız dosya
11             string dosya = @"C:\uygulama\test.txt";
12
13             //okunacak veri
14             Console.Write("Adınızı giriniz:");
15             string ad = Console.ReadLine();

```

Şekil 17.16

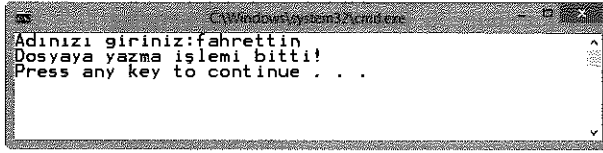
```

17             //istisna ihtimaline karşı try bloğu
18             try
19             {
20                 //dosyayı açalım
21                 FileStream f = new FileStream(dosya, FileMode.Open);
22
23                 //StreamWriter nesnesini oluşturalım
24                 StreamWriter sw = new StreamWriter(f);
25
26                 //konsoldan okunan veriyi yazalım
27                 sw.Write(ad);
28
29                 //dosyayı kapatalım
30                 sw.Close();
31
32                 //işlemin bittiğini bildirelim
33                 Console.WriteLine("Dosyaya yazma işlemi bitti!");
34             }
35
36             catch(IOException ex)
37             {
38                 Console.WriteLine("Hata:" + ex.Message);
39             }
40         }
41     }
42 }

```

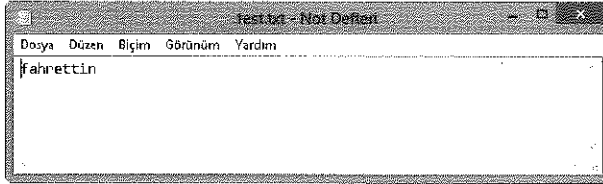
Şekil 17.17

Yukarıdaki kodların çalışmasıyla beraber **test.txt** dosyası açılır ve içine konsoldan girdiğiniz ad yazılır. Ben konsola **"fahrettin"** yazdığım için dosyada **"fahrettin"** metni görünüyor.



Şekil 17.18 Ekran çıktısı

Dosyaya yazılan metin aşağıdaki gibidir.



Şekil 17.19 Dosya görünümü

Dosya Okuma & Try-Catch Kullanımı

Bu bölümdeki son uygulama olarak **C:\uygulama** klasöründe bulunan **test.txt** dosyasındaki metni okuyup konsola yazdıracağız. İstisna oluşma ihtimaline karşın **try-catch** yapısını kullanacağız.

```

1  using System;
2  using System.IO;
3
4  namespace uygulamaFileStreamStreamReader
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             //dosya adresi
11             string dosya = @"C:\uygulama\test.txt";
12
13             //okunan verilerin atanacağı değişken
14             string okunan = "";
15
16             //istisna olmasına karşın try-catch bloğu
17             try
18             {
19                 //dosyayı açalım
20                 FileStream f = new FileStream(dosya, FileMode.Open);
21
22                 //açılan dosya için StreamReader nesnesi tanımlayalım
23                 StreamReader sr = new StreamReader(f);
24
25                 //açılan dosyadaki veriyi oku ve "okunan" değişkenine ata
26                 okunan = sr.ReadLine();
27
28                 //okunan veriyi konsola yaz
29                 Console.WriteLine(okunan);

```

Şekil 17.20


```
30  
31         //dosyayı kapat  
32         sr.Close();  
33     }  
34  
35     catch(IOException e)  
36     {  
37         Console.WriteLine("Hata:" + e.Message);  
38     }  
39 }  
40 }  
41 }  
42 }
```

Şekil 17.21

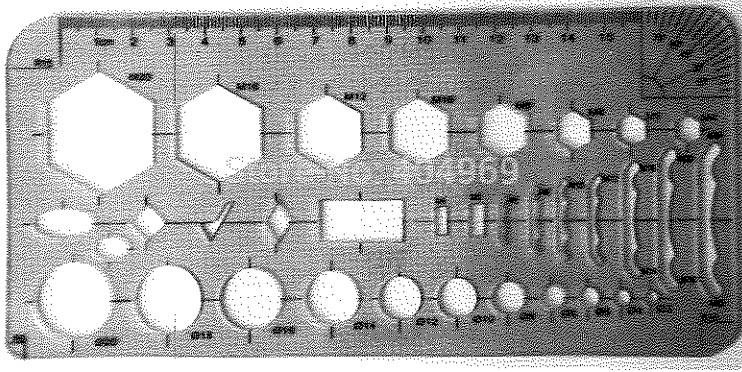
18

Şablonlar (Generics) ve Koleksiyonlar

- Şablon (Generic) Nedir?
- Şablon Metotlar
- Şablon Sınıflar
- Koleksiyonlar
- Stack
- Queue
- List
- Dictionary
- IEnumerable & IEnumerator

Şablon (Generic) Nedir?

İlkokul veya lisede geometrik şekiller çizerken kullandığımız şablonları hatırlarsınız; bu şablonlarda değişik geometrik şekiller bulunmakta ve düzgün bir şekil çizmemiz gerektiğinde kullanılmaktadır. Örneğin bir beşgen çizecekseniz şablonu kâğıdın üzerine koyar ve kalemi beşgen içinde dolaştırarak defalarca beşgen çizebilirsiniz. Bizim programcılıkta kullanılan şablonlar tam olarak aşağıda görülen şablona benzemese de kullanım mantığı aynıdır; bir kodu bir defa yazar ve defalarca kullanırsınız.



Şekil 18.1 Geomatrik şekil şablonu

Programcılıkta kullanılmak üzere oluşturacağımız şablonları farklı veri tipleriyle defalarca kullanabiliriz. Konuyu biraz daha pekiştirmek için basit bir örnek vereyim. Herhangi bir değeri ekrana yazdırmak için bir metod geliştirmeniz gerekirse aşağıda görüldüğü gibi metod aşırı yükleme yapmamız gerekmektedir. Yani, her olası veri tipi için ayrı metod yazmak zorundayız. Aşağıdaki kodları incellerseniz parametrelerin veri tiplerinin muhtemel tipte veriler için değiştiğini görebilirsiniz. Örneğin, ekrana tamsayı yazdırmak için **int**, ondalıklı değer yazdırmak için **double** tipinde parametreye sahip metod yazılması gerektiğini anlamışsındır.

```
static void yaz(int a) {Console.WriteLine(a);}
static void topla(double a) {Console.WriteLine(a);}
static void topla(string a) {Console.WriteLine(a);}
static void topla(bool a) {Console.WriteLine(a);}
```

Şablonlar bizi yukarıdaki kod hamallığından kurtarır. Eeee! Boşuna dememişler cahil kafanın cezasını ayaklar çeker diye... Şablonlar sayesinde yukarıdaki kodu bir defa yazar ve istediğimiz veri tipi için kullanabiliriz. Yukarıda yazılan dört metodun işini yapacak şablon metodumuz aşağıdaki gibi olabilir.

```
public static void Yaz<T>(T a)
{
    Console.WriteLine(a);
}
```

Şablon Metotlar

Yukarıda da bahsedildiği gibi, şablon metodlar kullanarak kendimizi kod hamallığından kurtarabilir ve yönetilmesi çok daha kolay programlar geliştirebiliriz.

Yukarıdaki anlatılan kodları kullanarak şablon bir metod yazıp program içinde denemek isterseniz aşağıdaki gibi bir program yazabilirsiniz. Aşağıdaki kodları iyice incelerseniz, yazılan tek bir metotla, yani şablon tipindeki **Yaz()** metoduyla her veri tipini ekrana yazdırabildiğimizi görebilirsiniz. Buradaki örnekte **Main()** metodu içinde **Yaz()** metodu çağrılmakta ve parametre olarak **int**, **double**, **bool** ve **string** tipinde parametreler kullanılmaktadır.

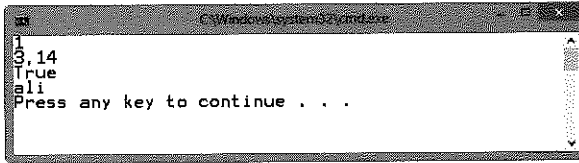
```

1  using System;
2
3  namespace sablonNedir
4  {
5      class Program
6      {
7          public static void Yaz<T>(T a)
8          {
9              Console.WriteLine(a);
10         }
11
12         static void Main(string[] args)
13         {
14             Yaz(1);
15             Yaz(3.14);
16             Yaz(true);
17             Yaz("ali");
18         }
19     }
20 }

```

Şekil 18.2 Sablon tipindeki Yaz metodu ve kullanımı

Yukarıdaki kodların çalışması sonucu aşağıdaki gibi bir çıktı elde edilir. Görüldüğü üzere, tüm veri tipleri sorunsuz olarak yazdırılmıştır.



Şekil 18.3 Ekran çıktısı

Peki! Ne yaptık da metodumuz şablon haline, yani, farklı veri tiplerini destekler hale geldi. Aslında yapılan iş çok basit; metod adının sağına kullanılacak **şablon veri tipini**/tiplerini (<T>) ve parantez içinde de **veri tipi** yerine parametrenin başına **şablon veri tipi** olan **T** harfini koyduk. Burada **T** harfi kullanmak zorunda değiliz; onun yerine başka bir harf de kullanabilirdik ama genel bir kabul olduğu için **T** harfi kullanılmaktadır.

Örnek Kod:

İsterseniz bir başka şablon metod örneği ile devam ederek konuyu pekiştirelim. Bu-

radaki örneğimizde **x** ile **y** değişkenlerinin değerleri karşılıklı olarak yer değiştirmektedir. Bir önceki örnekten farklı olarak burada iki parametre kullanılmaktadır.

```

1  using System;
2
3  namespace sablonMetot2Parametre
4  {
5      class Program
6      {
7          static void Degistir<T>(T x, T y)
8          {
9              T gecici = x;
10             x = y;
11             y = gecici;
12             Console.WriteLine("Yeni x=" + x + " Yeni y=" + y);
13         }
14
15         static void Main(string[] args)
16         {
17             Degistir(1, 2);
18             Degistir("3.14", "1.31");
19             Degistir("ali", "veli");
20         }
21     }
22 }

```

Şekil 18.4 Şablon metot ve kullanımı

Yukarıdaki şablon metotta her iki parametre aynı şablon veri tipine sahip olduğu için metot çağırıldığında her iki parametre de aynı veri tipinde olmalıdır. Örneğin, **Degistir(int, int);** şeklinde bir kullanım olmalıdır. Eğer **Degistir(int, string);** gibi iki farklı veri tipine sahip parametre kullanılırsa hataya neden olacaktır. Bu durumda iki farklı şablon veri tipi kullanan **Degistir<T,U>(T x, U y)** gibi bir metot tanımlanmalıdır.

Örnek Kod:

Aşağıdaki kodlarda iki şablon veri tipine sahip bir metot tanımlanmıştır. Dolayısıyla, iki farklı veri tipine sahip parametre ile işlem yapılabilir.

```

1  using System;
2
3  namespace sablonMetot2VeriTipi
4  {
5      class Program
6      {
7          static void Yaz<T,U>(T a, U b)
8          {
9              Console.WriteLine(a + " " + b);
10         }
11
12         static void Main(string[] args)
13         {
14             Yaz(1, "ali");
15             Yaz(3.14, 3);
16             Yaz(true, "veli");
17         }
18     }
19 }

```

Şekil 18.5 İki farklı şablon veri tipi

Yukarıdaki örneklerden de anlaşılacağı üzere, tanımlanan şablon metotlarda iki veya daha fazla parametre olabilir. Şablon metotlar konusunda dikkat edilmesi gereken en önemli konu şablon metotların içinde doğrudan matematiksel işlem yapılamayacağıdır. Örneğin, verilen sayının karesini döndüren aşağıdaki gibi bir şablon metot yazarsanız hata alırsınız.

```

1  using System;
2
3  namespace sablonMetotMatHata
4  {
5      class Program
6      {
7          static void Kare<T>(T x)
8          {
9              Console.WriteLine(x * x);
10         }
11
12         static void Main(string[] args)
13         {
14         }
15     }
16 }

```

Şekil 18.6 Şablonlar içinde matematiksel ifade olamaz

Dolaylı yöntemler kullanılarak şablon metot içinde matematiksel işlemler yapılabilir ama bu konu bu kitabın kapsamı dışında olduğu için üzerinde durmayacağız.

Şablon Sınıflar

Tıpkı şablon metotlar gibi şablon sınıflar da yazılabilir. Şablon sınıfların en çok kullanıldığı alan şablon türünde oluşturulan koleksiyonlardır. Adından da belli olduğu gibi, koleksiyonlar aynı veri tipine sahip bilgileri depolayan dizi benzeri veri yapılarıdır. Koleksiyonlarda kolayca veri ekleme, silme, temizleme, sayma ve sıralama gibi işlemler yapılabildiği için sıklıkla tercih edilebilmektedir.

Sıkça kullanılan şablon türündeki koleksiyonlar **Dictionary**, **List**, **Queue** ve **Stack** adlarıyla bilinirler. Bu yapılara bir sonraki konuda daha ayrıntılı olarak değineceğiz. Şimdilik **Stack** olarak bilinen yapıyı inceleyip **şablon sınıf** mantığını anlamaya çalışacağız.

Stack ya da **yığın** olarak bilinen yapı **Son Giren İlk Çıkar** (Last In First Out- LIFO) mantığıyla çalışır. Bu yapılar, aşağıdaki şekilde de görüldüğü gibi, sadece bir tarafında ağızı olan ve o ağızdan veri doldurulan bir kutuya benzer. Kutuya en son koyulan **Veri4** kapiya en yakın olduğu için diğer verilere erişmek için ilk önce Veri4'ün çıkması gerekmektedir. Ancak **Veri4** çıktıktan sonra **Veri3**'e ulaşılabilir. **Veri1**'e ulaşmak için **Veri2**'nin çıkması gerekmektedir.

SON GİREN → İLK ÇIKAR	Veri4	Veri3	Veri2	Veri1
--------------------------	-------	-------	-------	-------

Yukarıda anlatılan yığıt yapısı gibi çalışan **şablon bir sınıfı** dizi kullanarak tasarlayacak olursak kodlarımız aşağıdaki gibi olabilir.

```

1  using System;
2  namespace sablonSinifStack {
3      class Program {
4          class Stack<T> {
5              int i = 0;           //dizi indisi
6              T[] dizi = new T[100]; //100 elemanlı dizi
7
8              //diziye eleman ekle
9              public void Ekle(T eleman){ dizi[i++] = eleman; }
10             //diziden eleman çıkar/indisi azalt
11             public T Cik() { return dizi[--i]; }
12             //dizi elemanı oku
13             public T Oku(int k) { return dizi[k]; }
14         }
15
16         static void Main(string[] args)
17         {
18             //int tipinde değer barındıran yığıt oluşturalım
19             Stack<int> iStack = new Stack<int>();
20             iStack.Ekle(1);           //yığıta eleman ekle
21             iStack.Ekle(2);           //yığıta eleman ekle
22             int donen = iStack.Oku(1); //1 indisli elemanı oku
23             Console.WriteLine(donen); //okunan değeri yaz
24         }
25     }
26 }

```

Şekil 18.7 Şablon sınıf oluşturma

Kodların arasına yazılan açıklamalarda da görebileceğiniz gibi, yığıt yapısını oluşturmak için şablon veri tipinde (`T[]`) 100 elemanlı bir dizi oluşturuldu. **Ekle()** metoduyla diziyeye eleman eklenirken, **Cik()** metoduyla dizi indisi azaltılmakta ve bir sağdaki elemana ulaşılmaktadır. **Oku()** metodu kullanılarak **k** indisine sahip eleman okunmaktadır.

Yazılan şablon sınıftan bir nesne oluşturmak için **Main()** metodu içinde **Stack<int> iStack=new iStack<int>();** komutu kullanılmış ve tamsayı tipinde değerler saklayacak bir yığıt meydana getirilmiştir. **iStack.Ekle();** metodu kullanılarak yığıtta 2 eleman eklenmiş, **iStack.Ok();** metodu kullanılarak yığıttan değer okunmuştur.

Yukarıdaki örnekte **int** veri tipinde tamsayı değerleri barındıran bir yığıt oluşturduk. Yukarıdaki örneğe benzer şekilde **double**, **string** vb. veri tiplerini barındıran yığıtlar da yazabilirsiniz. Bunun için aşağıdaki örnek kodları kullanabilirsiniz.

```
Stack<int> iStack = new Stack<int>();
Stack<double> dStack = new Stack<double>();
Stack<string> sStack = new Stack<string>();
```

Şablon sınıflarda bulunan alanlara yapıcı metotları kullanarak değer atayabilirsiniz. Örneğin, aşağıdaki şablon sınıfta şablon veri tipine sahip **x** alanına yapıcı metot kullanılarak değer atanmıştır.

```
class Test<T>
{
    T x;
    public Test(T val)
    {
        x = val;
    }
}
```

Şablon Tipli Koleksiyonlar

.NET Framework içinde gelen ve verileri düzenli bir şekilde saklamaya yarayan şablon tipinde koleksiyonlar bulunmaktadır. Daha önceki konularda **Stack** adıyla bilinen veri yapısını incelemiş ve nasıl çalıştığını gösteren şablon sınıf yazmıştık. Bu bölümde **Stack** yapısının yanında, **Dictionary**, **List** ve **Queue** olarak bilinen veri yapılarını öğreneceğiz.

Şablon tipli koleksiyonlar C# diline 2.0 sürümüyle beraber eklendi. Eski tipteki koleksiyonlar yavaş çalıştığı için artık kullanımı pek tavsiye edilmemektedir. Bu neden-

İle bu kitapta eski tip klasik koleksiyonlar yerine daha hızlı çalışan şablon tipli koleksiyonlar anlatılacaktır.

Şablon tipli koleksiyonlar **System.Collections.Generic** aduzayının içindedir. Eğer koleksiyonları kullanacaksanız kodların en üstüne **using System.Collections.Generic;** komutunu yazmanız gerekmektedir.

Stack (Yığıt)

Daha önce de öğrendiğimiz gibi, **Stack** ya da **yığıt** olarak bilinen yapı **Son Giren İlk Çıkar** (Last In First Out- LIFO) mantığıyla çalışır. Bu yapılar, aşağıdaki şekilde de görüldüğü gibi, sadece bir tarafında ağız olan ve o ağızdan veri doldurulan bir kutuya benzer. Kutuya en son koyulan **Veri4** kapıya en yakın olduğu için diğer verilere erişmek için ilk önce **Veri4**'ün çıkması gerekmektedir. Ancak **Veri4** çıktıktan sonra **Veri3**'e ulaşılabilir. **Veri1**'e ulaşmak için **Veri2**'nin çıkması gerekmektedir.

İNDEKS DEĞERİ	3	2	1	0
SON GİREN →	Veri4	Veri3	Veri2	Veri1
← İLK ÇIKAR				

Yeni bir **Stack** nesnesi tanımlamak için aşağıdaki sözdizimini kullanabilirsiniz:

```
Stack<veri_tipi> nesne=new Stack<veri_tipi>();
```

Örneğin, tamsayı tipinde bir **Stack** nesnesi tanımlamak için aşağıdaki kod satırını yazabilirsiniz:

```
Stack<int> st=new Stack<int>();
```

Stack nesnesini tanımladıktan sonra **Stack** sınıfının özellik ve metotlarını kullanarak işlem yapabilirsiniz. Sıkça kullanılan **Stack** metotlarını/özelliklerini ve açıklamalarını aşağıdaki tabloda görebilirsiniz:

Metot	Açıklama
Count	Eleman sayısını döndürür
Push(eleman)	Yığıtın en başına eleman ekler
Pop()	Yığıtın en başındaki elemanı döndürür ve yığıttan siler
Peek()	Yığıtın en üstündeki elemanı silmeden döndürür
Clear()	Tüm elemanları siler
Contains(eleman)	Belirtilen elemanın olup olmadığını kontrol eder. Eğer varsa true, yoksa false döndürür.
CopyTo(int[] array, i)	Yığıt elemanlarını iki boyutlu bir diziye belirtilen indeks numarasından itibaren kopyalar.

Örnek Kod:

Yığıt tipinde bir nesne oluşturan ve bu nesneler üzerinde Stack metotlarını kullanarak işlem yapan kodları aşağıda görebilirsiniz.

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace koleksiyonStack
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10              Stack<int> st = new Stack<int>();
11
12              st.Push(1);           //yığıtta 1 ekle
13              st.Push(3);          //yığıtta 3 ekle
14              st.Push(5);          //yığıtta 5 ekle
15              if (st.Contains(1))  //eğer yığıtta 1 varsa "var" yaz
16                  Console.WriteLine("var");
17              Console.WriteLine("Eleman Sayısı: " + st.Count);
18              Console.WriteLine("En Baştaki Eleman:" + st.Peek());
19          }
20      }
21  }
```

Şekil 18.8 Yığıtın özellik ve metotları

Queue (Kuyruk)

Kuyruk veri yapısı yığıt yapısına çok benzer ama aralarında çok ciddi bir fark vardır: **Yığıt** olarak bilinen yapı **Son Giren İlk Çıkar** (Last In First Out- LIFO) mantığıyla çalışırken kuyruk olarak bilinen yapı **İlk Giren İlk Çıkar** (First In First Out-FIFO) mantığıyla çalışır. Adından da belli olduğu gibi, queue bir kuyruğu taklit etmektedir. Tıpkı banka ya da otobüs kuyruklarında olduğu gibi, kuyruğa daha önce giren içeriye de daha önce girer.

Kuyruk veri yapısı aşağıdaki şekilde de görüldüğü gibi, bir tarafında girişi bir tarafında da çıkışı olan bir kanala benzer. Kanala ilk koyulan **Veri1** çıkışa en yakın eleman olurken, kanala son koyulan **Veri4** çıkışa en uzak elemandır.

	3	2	1	0	İNDEKS DEĞERİ
SON GİREN → ELEMAN	Veri4	Veri3	Veri2	Veri1	→ İLK ÇIKACAK ELEMAN

Yeni bir **Queue** nesnesi tanımlamak için aşağıdaki sözdizimini kullanabilirsiniz:

```
Queue<veri_tipi> nesne=new Queue<veri_tipi>();
```

Örneğin, tamsayı tipinde bir **Queue** nesnesi tanımlamak için aşağıdaki kod satırını yazabilirsiniz:

```
Queue<int> qu=new Queue<int>();
```

Queue nesnesini tanımladıktan sonra **Queue** sınıfının özellik ve metotlarını kullanarak işlem yapabilirsiniz. Sıkça kullanılan **Queue** metotlarını/özelliklerini ve açıklamalarını aşağıdaki tabloda görebilirsiniz:

Metot	Açıklama
Count	Eleman sayısını döndürür
Enqueue(eleman)	Kuyruğun en sonuna eleman ekler
Dequeue()	Kuyruğun en başındaki elemanı döndürür ve yığıttan siler
Peek()	Yığın en üstündeki elemanı silmeden döndürür
Clear()	Tüm elemanları siler
Contains(eleman)	Belirtilen elemanın olup olmadığını kontrol eder. Eğer varsa true, yoksa false döndürür.
CopyTo(int[] array, i)	Yığın elemanlarını iki boyutlu bir diziye belirtilen indeks numarasından itibaren kopyalar.

Örnek Kod:

Kuyruk tipinde bir nesne oluşturan ve bu nesneler üzerinde Queue metodlarını kullanarak işlem yapan kodları aşağıda görebilirsiniz.

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace koleksiyonQueue
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             Queue<int> qu = new Queue<int>();
11
12             qu.Enqueue(1);
13             qu.Enqueue(3);
14             qu.Enqueue(5);
15             if (qu.Contains(3))
16                 Console.WriteLine("var");
17             Console.WriteLine("Eleman Sayısı:" + qu.Count);
18             Console.WriteLine("En Baştaki Eleman:" + qu.Dequeue());
19         }
20     }
21 }

```

Şekil 18.9 Kuyruğun özellik ve metotları

List (Liste)

List veri yapısı şu ana kadar öğrendiğimiz ve yeri geldiğinde sıkça kullandığımız dizilere çok benzer. Listeler dizilerle aynı işi yapar ama dizilere göre çok önemli bir avantaja sahiptir; Listeler belli bir boyuta sahip olmak zorunda değildir. Yani, listeleri tanımlarken belli bir boyut vermek zorunda değilsiniz; ihtiyaca göre eleman ekleyip çıkartabilirsiniz. Daha teknik bir dille ifade etmek gerekirse, dizi boyutunu derlemeden önce belirlememiz gerekiyordu. Eğer diziye boyutunda fazla eleman eklemeye kalkarsanız hata alırsınız. **List** ile oluşturulan veri yapısında böyle bir zorunluluk yoktur; derlenme yapıldıktan sonra program çalışırken bile liste boyutu değişebilir.

Aşağıdaki şekilde de görüldüğü gibi, **List** veri yapısı dizilere çok benzemektedir.

İNDEKS DEĞERİ	0	1	2	3
DİZİ ELEMANI	Veri1	Veri2	Veri3	Veri4

Yeni bir **List** nesnesi tanımlamak için aşağıdaki sözdizimini kullanabilirsiniz:

```
List<veri_tipi> nesne=new List<veri_tipi>();
```

Örneğin, tamsayı tipinde bir **List** nesnesi tanımlamak için aşağıdaki kod satırını yazabilirsiniz:

```
List<int> li=new List<int>();
```

List nesnesini tanımladıktan sonra **List** sınıfının özellik ve metotlarını kullanarak işlem yapabilirsiniz. Sıkça kullanılan **List** metotlarını/özelliklerini ve açıklamalarını aşağıdaki tabloda görebilirsiniz:

Metot	Açıklama
Count	Eleman sayısını döndürür
Add(eleman)	Kuyruğun en sonuna eleman ekler
RemoveAt(indeks)	Belirtilen Indeks değerine sahip eleman silinir.
Clear()	Tüm elemanları siler
Reverse()	Elemanları terst çevirir
Sort()	Dizi elemanlarını sıralar
Contains(eleman)	Belirtilen elemanın olup olmadığını kontrol eder. Eğer varsa true, yoksa false döndürür.
IndexOf(x)	Parametre olarak belirtilen nesnenin ilk bulunduğu elemanın indeks değerini döndürür
CopyTo(int[] array, i)	Yığıt elemanlarını iki boyutlu bir diziye belirtilen indeks numarasından itibaren kopyalar.

Örnek Kod:

Liste tipinde bir nesne oluşturan ve bu nesneler üzerinde Liste metotlarını kullanarak işlem yapan kodları aşağıda görebilirsiniz.

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace koleksiyonList
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             List<int> li = new List<int>();
11
12             li.Add(1);
13             li.Add(3);
14             li.Add(5);
15
16             if (li.Contains(3))
17                 Console.WriteLine("var");
18
19             Console.WriteLine("Eleman Sayısı: " + li.Count);
20             li.Reverse();
21             li.Sort();
22             Console.WriteLine("Sıfır İndeksli Eleman: " + li[0]);
23         }
24     }
25 }

```

Şekil 18.10 liste kullanımı

Dictionary (Sözlük)

Dictionary veri yapısı **List** veri yapısına çok benzer. Aralarındaki temel fark şudur: Listelerde saklanan elemanlara indeks numarası ile erişilebilirken sözlük olarak adlandırılan veri yapılarına **"anahtar"** olarak bilinen bir isimle erişilmesidir. Yani, elemanlara erişmek veya elemanlarla ilgili işlem yapmak için numara yerine isim kullanılmaktadır. Dolayısıyla, sözlüklerde elemanlar aşağıdaki şekilde de görebileceğiniz gibi **anahtar-değer ikilisi** olarak saklanmaktadır.

ANAHTAR	Anahtar1	Anahtar2	Anahtar3	Anahtar4
DEĞER	Değer1	Değer2	Değer3	Değer4

Yeni bir **Dictionary** nesnesi tanımlamak için aşağıdaki sözdizimini kullanabilirsiniz:

```
Dictionary<anahtar_veri_tipi, değer_veri_tipi> nesne=new Dictionary<
anahtar_veri_tipi, değer_veri_tipi >();
```

Örneğin, anahtar veri tipi **string**, değer veri tipi **int** olan bir **Dictionary** nesnesi tanımlamak için aşağıdaki kod satırını yazabilirsiniz:

```
Dictionary<string,int> di=new Dictionary<string,int>();
```

Dictionary nesnesini tanımladıktan sonra **Dictionary** sınıfının özellik ve metotlarını kullanarak işlem yapabilirsiniz. Sıkça kullanılan **Dictionary** metotlarını/özelliklerini ve açıklamalarını aşağıdaki tabloda görebilirsiniz:

Metot	Açıklama
Count	Eleman sayısını döndürür
Add(anahtar, değer)	Kuyruğun en sonuna eleman ekler
RemoveAt(anahtar)	Belirtilen indeks değerine sahip eleman silinir.
Clear()	Tüm elemanları siler
ContainsKey(anahtar)	Belirtilen anahtarın olup olmadığını kontrol eder. Eğer varsa true, yoksa false döndürür.
ContainsValue(değer)	Belirtilen değerın olup olmadığını kontrol eder. Eğer varsa true, yoksa false döndürür.
CopyTo(array, i)	Yığıt elemanlarını iki boyutlu bir diziye belirtilen indeks numarasından itibaren kopyalar.

Örnek Kod:

Liste tipinde bir nesne oluşturan ve bu nesneler üzerinde Liste metotlarını kullanarak işlem yapan kodları aşağıda görebilirsiniz.

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace koleksiyonDictionary
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10             Dictionary<string, int> di = new Dictionary<string, int>();
11             di.Add("adana", 01);
12             di.Add("istanbul", 34);
13             di["izmir"] = 35;
14
15             if (di.ContainsKey("izmir"))
16                 Console.WriteLine("var");
17
18             Console.WriteLine("Eleman Sayısı:" + di.Count);
19             di.Remove("adana");
20             di.Clear();
21         }
22     }
23 }
```

Şekil 18.11 Dictionary kullanımı

KAYNAKLAR

- C# Ders Notları, Fahrettin Erdiñ, Bilimsel Akademi Bilgisayar Kursu
- C# ile Nesne Tabanlı Programlama, Fahrettin Erdiñ, Abaküs Kitap
- C/C++, Fahrettin Erdiñ, Abaküs Kitap
- Her Yönüyle C#, Sefer Algan, Pusula Yayınları
- Her Yönüyle C# 5.0, Volkan Aktaş, KODLAB
- Stackoverflow.com, İnternet Web Sitesi
- Sololearn.com, İnternet Web Sitesi

